Department of Mathematics
and Computer Science

sdu.dk

#sdudk

Advanced Machine Learning

Background

Lukas Galke

Spring 2026

SDU

Slides based on Richard Röttger's material

Keywords

- **Linear algebra**
- Probabilities & Standard Distributions
- Eigendecomposition & PCA

Scalar and Vector

Scalar

- Single number
- Normally: $x \in \mathbb{R}$ or $x \in \mathbb{N}$

Vector

- An array of numbers
  - Arranged in order
  - Each no. identified by an index
- $\boldsymbol{x} = \begin{bmatrix} x_1 \\ \cdots \\ x_d \end{bmatrix}$ and $\boldsymbol{x}^T = [x_1, \ldots, x_d]$, $\boldsymbol{x} \in \mathbb{R}^d$
- We think of vectors as points in space
  - Each element gives coordinate along an axis

Matrix

- 2-D array of numbers
- Each element identified by two indices
- Denoted by bold typeface $\boldsymbol{A}$
- Elements indicated as $A_{m,n}$

$$\boldsymbol{A} = \begin{bmatrix} A_{1,1} & A_{1,2} \\ A_{2,1} & A_{2,2} \end{bmatrix}$$

- $A_{i:}$ is ith row of $\boldsymbol{A}$, $A_{:j}$ is jth column
- $\boldsymbol{A}$ has m rows and n columns, then $A \in \mathbb{R}^{m \times n}$

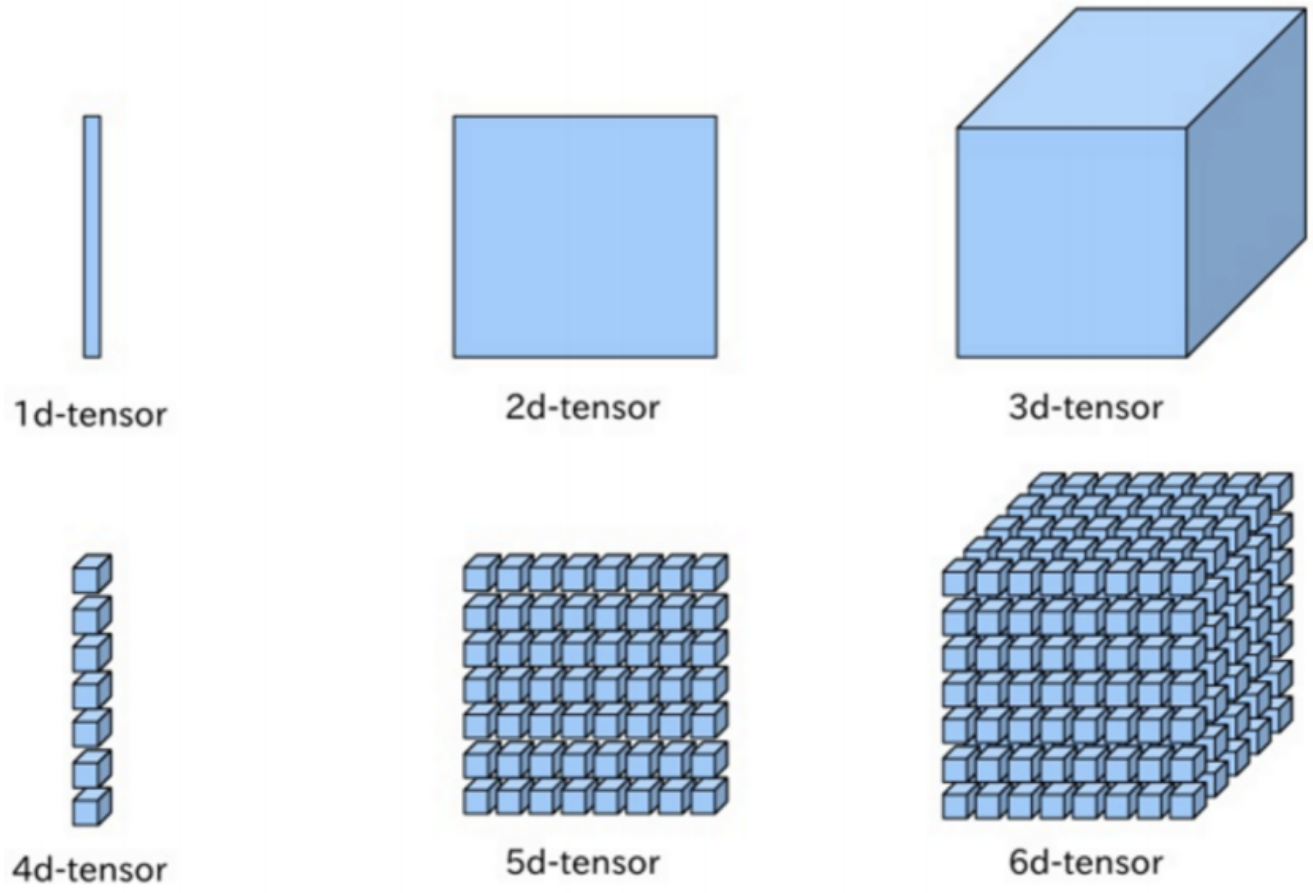
Tensor

- Sometimes need an array with more than two axes
- An array arranged on a regular grid with variable number of axes is referred to as a **Tensor**

- We denote a tensor with bold non-italic typeface: A in comparison to a normal Matrix $\boldsymbol{A}$
- I try to keep the notation consistent, but double-check ☺
- An Element (i, j, k) of tensor denoted by $\mathrm{A}_{i,j,k}$

- Example: RGB image is a 3D Tensor (height x width x color channel)

Shape of Tensors



Terminology:

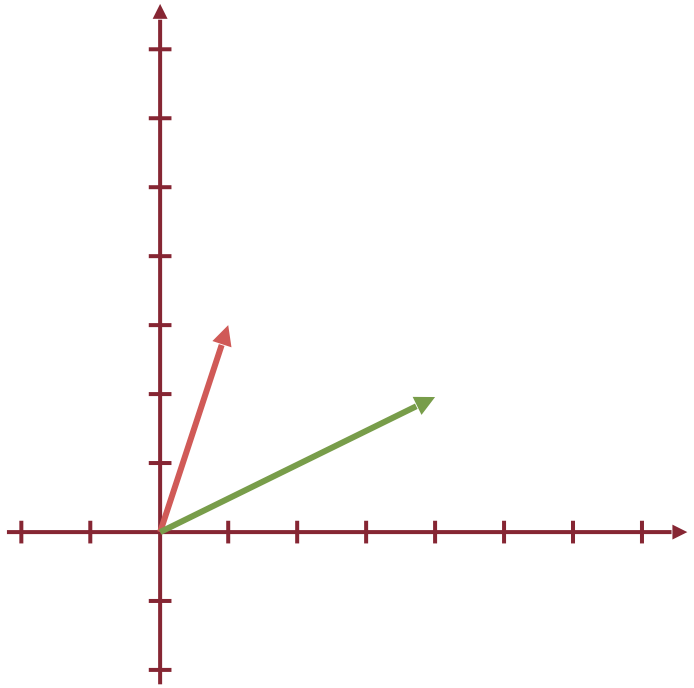
The number of axes is also denoted as **the rank** of a tensor.

Vector Interpretation

➢ Vectors can are often depicted as arrows in Euclidean space.

➢ Let assume two Vectors:

$$\boldsymbol{a} = \begin{pmatrix} 1 \\ 3 \end{pmatrix} \qquad \boldsymbol{b} = \begin{pmatrix} 4 \\ 2 \end{pmatrix}$$



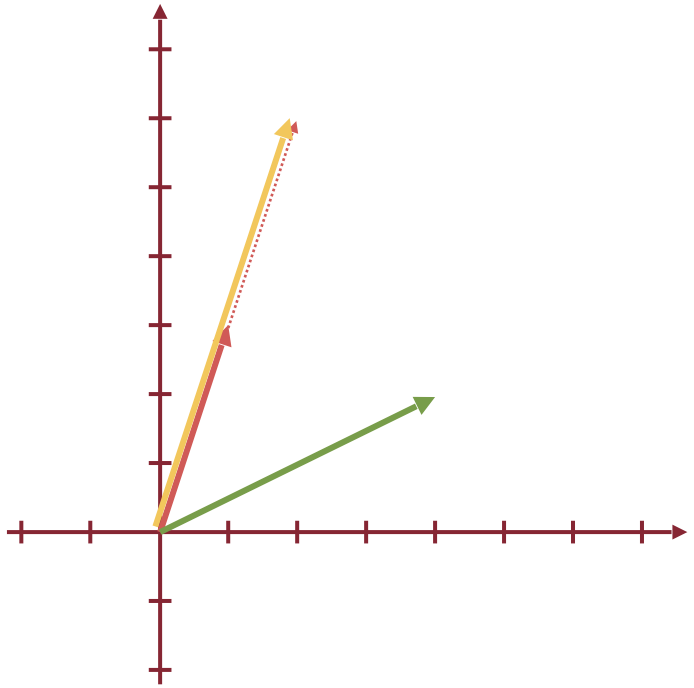
Vector Operations: Multiply with a Scalar

➢ Vectors can be multiplied with a scalar by elementwise multiplying the entries:

$$\lambda \cdot x = \lambda \cdot \begin{pmatrix} x_1 \\ \vdots \\ x_d \end{pmatrix} = \begin{pmatrix} \lambda \cdot x_1 \\ \vdots \\ \lambda \cdot x_d \end{pmatrix}$$

➢ Example:

$$2 \cdot \boldsymbol{a} = 2 \cdot \begin{pmatrix} 1 \\ 3 \end{pmatrix} = \begin{pmatrix} 2 \cdot 1 \\ 2 \cdot 3 \end{pmatrix} = \begin{pmatrix} 2 \\ 6 \end{pmatrix}$$



Vector Operations: Addition

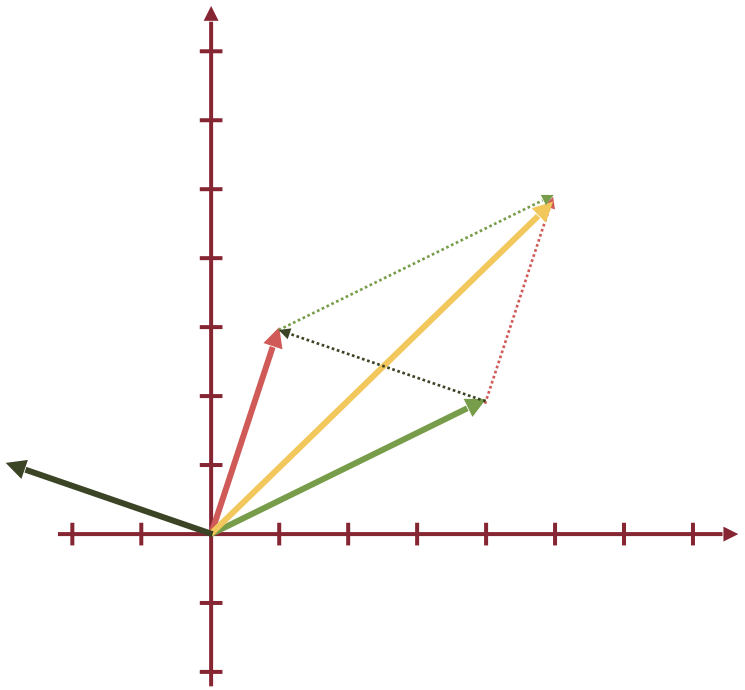
➢ Vectors can be added by element-wise adding the components:

$$x + y = \begin{pmatrix} x_1 \\ \vdots \\ x_d \end{pmatrix} + \begin{pmatrix} y_1 \\ \vdots \\ y_d \end{pmatrix} = \begin{pmatrix} x_1 + y_1 \\ \vdots \\ x_1 + y_1 \end{pmatrix}$$

➢ Examples:

$$\boldsymbol{a} + \boldsymbol{b} = \begin{pmatrix} 1 \\ 3 \end{pmatrix} + \begin{pmatrix} 4 \\ 2 \end{pmatrix} = \begin{pmatrix} 5 \\ 5 \end{pmatrix}$$

$$\boldsymbol{a} - \boldsymbol{b} = \begin{pmatrix} 1 \\ 3 \end{pmatrix} - \begin{pmatrix} 4 \\ 2 \end{pmatrix} = \begin{pmatrix} -3 \\ 1 \end{pmatrix}$$



Vector Operations: The Dot Product

➢ The scalar product of two vectors is the sum of the element-wise multiplication of the entries:

$$\boldsymbol{x} \cdot \boldsymbol{y} = \begin{pmatrix} x_1 \\ \vdots \\ x_d \end{pmatrix} \cdot \begin{pmatrix} y_1 \\ \vdots \\ y_d \end{pmatrix} = x_1 y_1 + \ldots + x_d y_d = \sum_{i=1}^{d} x_i y_i = \boldsymbol{x}^T \boldsymbol{y}$$

➢ Note: The result is a real number, not a vector

➢ For this reason, the dot product is sometimes called the scalar product (or inner product).

$$\boldsymbol{a} \cdot \boldsymbol{b} = \begin{pmatrix} 1 \\ 3 \end{pmatrix} \cdot \begin{pmatrix} 4 \\ 2 \end{pmatrix} = 1 \cdot 4 + 3 \cdot 2 = 10$$

Properties of the Scalarproduct

➢ The dot product obeys many of the laws that hold for ordinary products of real numbers.

➢ Let $\boldsymbol{a}, \boldsymbol{b}, \boldsymbol{c}$ be vectors, λ is a scalar.

➢ Then:

1. $\boldsymbol{a} \cdot \boldsymbol{b} = \boldsymbol{b} \cdot \boldsymbol{a}$
2. $\boldsymbol{a} \cdot \left(\boldsymbol{b} + \boldsymbol{c}\right) = \boldsymbol{a} \cdot \boldsymbol{b} + \boldsymbol{a} \cdot \boldsymbol{c}$
3. $(\lambda \boldsymbol{a}) \cdot \boldsymbol{b} = \lambda\left(\boldsymbol{a} \cdot \boldsymbol{b}\right) = \boldsymbol{a} \cdot \left(\lambda \boldsymbol{b}\right)$
4. $0 \cdot a = 0$

Vector Operations: Length of Vector

➢ The length of a vector is normally calculated as the square root of the summed squares of the entries:

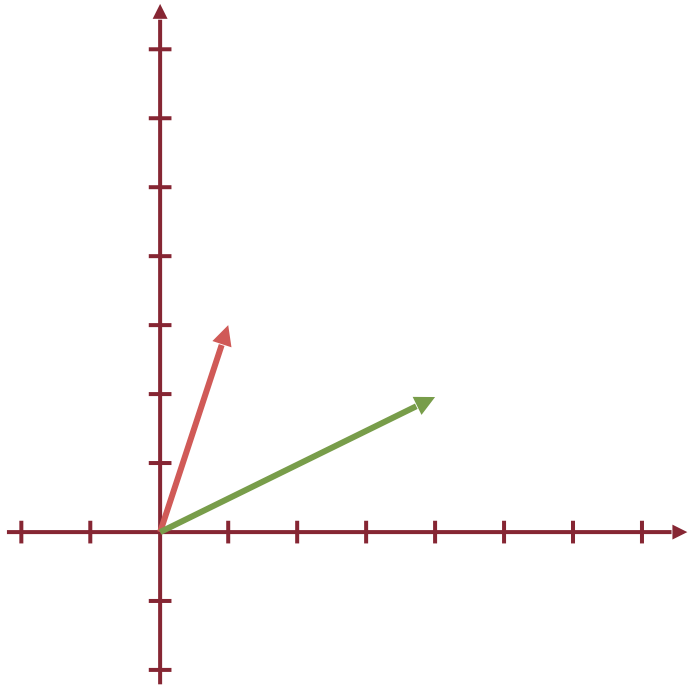
$$|x| = \left| \begin{pmatrix} x_1 \\ \vdots \\ x_d \end{pmatrix} \right| = \sqrt{x_1^2 + \ldots + x_d^2} = \left(\sum_{i=1}^{d} x_i^2 \right)^{\frac{1}{2}}$$

➢ Relation to the dot-product:

$$x \cdot x = |x|^2$$

➢ Example:

$$|a| = \left| \begin{pmatrix} 1 \\ 3 \end{pmatrix} \right| = \sqrt{1^2 + 3^2} = \sqrt{10} \approx 3.16$$



L^p Norms for Vectors

- What we have seen is just a member of a family of norms:

$$L^p = \left(\sum_i |x_i|^p \right)^{\frac{1}{p}}$$

- L^2 Norm
  - Most commonly used
  - If nothing is explicitly stated, the Euclidean norm is used
- L^1 Norm
  - Useful when 0 and non-zero have to be distinguished (since L^2 increases slowly near origin, e.g., $0.1^2 = 0.01$)
- L^∞ Norm
  - $\left|\left|\boldsymbol{x}\right|\right|_\infty = \max|x_i|$
  - Called max norm

Angle between Vectors

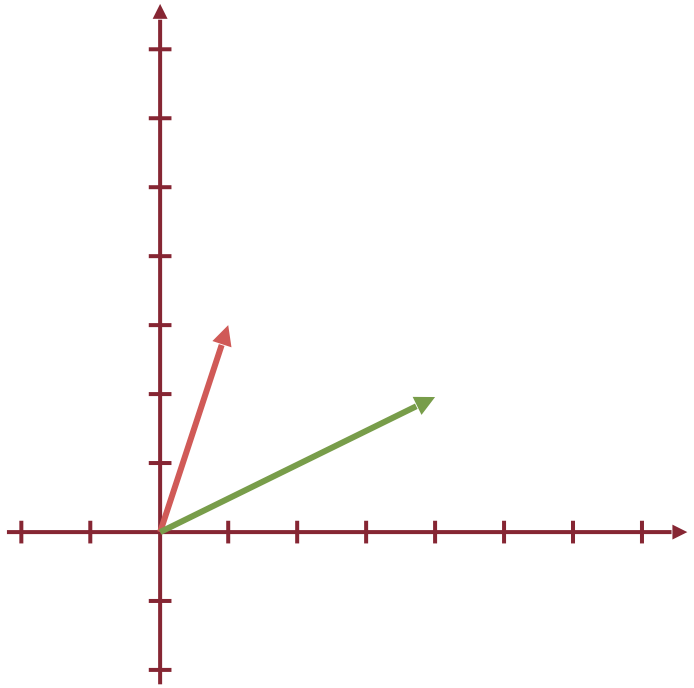
➢ Dot product of two vectors can be written in terms of their L^2 norms and angle θ between them

$$\boldsymbol{x} \cdot \boldsymbol{y} = |\boldsymbol{x}| \left|\boldsymbol{y}\right| \cdot \cos\theta$$

➢ Example:

$$\theta = \cos^{-1}\left(\frac{\boldsymbol{a} \cdot \boldsymbol{b}}{|a| \cdot |b|}\right) =$$

$$= \cos^{-1}\left(\frac{10}{\sqrt{10} \cdot \sqrt{20}}\right) = 45°$$



Names around Matrices

$$\begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix}$$

- **Order of matrix**: It represent the number of rows and number columns of a matrix. For above matrix, the order is 3×3
- **Square matrix**: If a matrix has same number of row and columns then it is called square matrix.
- Special Shapes:
  - Row Matrix: If a matrix only has one row: a transposed Vector
  - Column Matrix: Similarly, if a matrix only has one column: a Vector
  - 1×1 Matrix: A scalar

Matrix Operations: Addition

➢ Two Matrices can be added by simply adding the entries element-wise:

$$\begin{pmatrix} a & b \\ c & d \end{pmatrix} + \begin{pmatrix} e & f \\ g & h \end{pmatrix} = \begin{pmatrix} a+e & b+f \\ c+g & d+h \end{pmatrix}$$

➢ Example:

$$\begin{pmatrix} 1 & 2 \\ 5 & 6 \end{pmatrix} + \begin{pmatrix} 3 & 4 \\ 2 & 5 \end{pmatrix} = \begin{pmatrix} 1+3 & 2+4 \\ 5+2 & 6+5 \end{pmatrix} = \begin{pmatrix} 4 & 6 \\ 7 & 11 \end{pmatrix}$$

➢ The matrices necessarily need to be of the same order (or dimensionality)

Matrix Operations: Multiplication with a Scalar

➢ A Matrix can be multiplied by a scalar by multiplying each element of the matrix with the scalar:

$$\lambda \cdot \begin{pmatrix} a & b \\ c & d \end{pmatrix} = \begin{pmatrix} \lambda \cdot a & \lambda \cdot b \\ \lambda \cdot c & \lambda \cdot d \end{pmatrix}$$

➢ Example:

$$2 \cdot \begin{pmatrix} 3 & 4 \\ 2 & 5 \end{pmatrix} = \begin{pmatrix} 2 \cdot 3 & 2 \cdot 4 \\ 2 \cdot 2 & 2 \cdot 5 \end{pmatrix} = \begin{pmatrix} 6 & 8 \\ 4 & 10 \end{pmatrix}$$

Matrix Operations: Multiplication

- To multiply two matrices, we perform the “Dot Product” between rows of the first matrix and columns of the second matrix:

$$\boldsymbol{A} \cdot \boldsymbol{B} = \boldsymbol{C}$$

$$c_{ij} = \sum_{k} a_{ik} b_{kj}$$

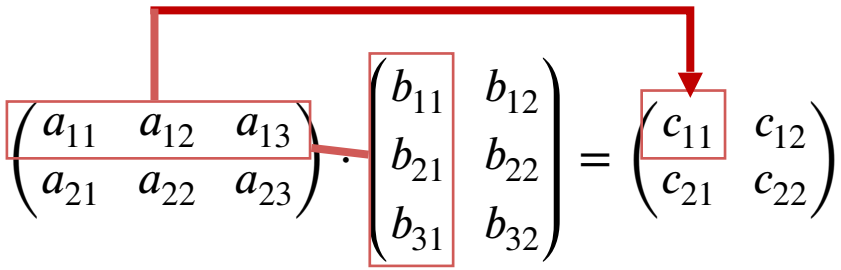
  - Two matrices can only be multiplied if number of columns in the first matrix is same as the number of rows of the second matrix.
  - If matrix $\boldsymbol{A}$ is of order $m \times n$ and matrix $\boldsymbol{B}$ is of order $r \times s$ the $\boldsymbol{A} \cdot \boldsymbol{B}$ will be possible if $n = r$.
  - The resulting matrix will be of order $m \times s$.

Matrix Operations: Multiplication

➢ Two multiply two matrices, we perform the “Dot Product” between rows of the first matrix and columns of the second matrix:

$$\boldsymbol{A} \cdot \boldsymbol{B} = \boldsymbol{C}$$

$$c_{ij} = \sum_{k} a_{ik} b_{kj}$$



$$\begin{pmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \end{pmatrix} \cdot \begin{pmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \\ b_{31} & b_{32} \end{pmatrix} = \begin{pmatrix} c_{11} & c_{12} \\ c_{21} & c_{22} \end{pmatrix}$$

Matrix Product Properties

➢ Distributivity over addition:

$$\boldsymbol{A}(\boldsymbol{B}+\boldsymbol{C}) = \boldsymbol{A}\boldsymbol{B}+\boldsymbol{A}\boldsymbol{C}$$

➢ Associativity:

$$\boldsymbol{A}(\boldsymbol{B}\boldsymbol{C}) = (\boldsymbol{A}\boldsymbol{B})\boldsymbol{C}$$

➢ Not commutative: $\boldsymbol{A}\boldsymbol{B} = \boldsymbol{B}\boldsymbol{A}$ is not always true

➢ Dot product between vectors is commutative:

$$\boldsymbol{x}^T\boldsymbol{y} = \boldsymbol{y}^T\boldsymbol{x}$$

➢ Transpose of a matrix product has a simple form:

$$\left(\boldsymbol{A}\boldsymbol{B}\right)^T = \boldsymbol{B}^T\boldsymbol{A}^T$$

Transpose of a Matrix

➢ Mirror image across principal diagonal

$$\boldsymbol{A} = \begin{bmatrix} A_{1,1} & A_{1,2} \\ A_{2,1} & A_{2,2} \\ A_{3,1} & A_{3,2} \end{bmatrix}, \quad A^T = \begin{bmatrix} A_{1,1} & A_{2,1} & A_{3,1} \\ A_{1,2} & A_{2,2} & A_{3,2} \end{bmatrix}$$

➢ Vectors are then either a Matrix with a single column or a single row; For convenience often written in a line: $\boldsymbol{x}^T = [x_1, x_2, x_3]$

➢ Scalar is a Matrix with just one element, thus:

$x^T = x$

Norm of a Matrix

➢Frobenius norm

$$\|\boldsymbol{A}\| = \left(\sum_{i,j} A_{i,j}^2 \right)^{\frac{1}{2}}$$

➢It is analogous to L^2 norm of a vector

Square Matrices

➢ Remember, square matrices have the same number of rows as columns

$$A = \begin{pmatrix} a_{11} & \cdots & a_{1d} \\ \vdots & \ddots & \vdots \\ a_{d1} & \cdots & a_{dd} \end{pmatrix}$$

➢ Some operations are special to square matrices and are not defined for arbitrary matrices

Identity Matrix

- An identity or unit matrix of size d is square matrix of order $d \times d$ where all the diagonal elements are ‘1' and all the other elements are ‘0'.
- It is denoted by $\boldsymbol{I}$.

$$\boldsymbol{I}_1 = (1) \quad \boldsymbol{I}_2 = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \quad \boldsymbol{I}_3 = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

- Important Property ($\boldsymbol{A}$ being a square matrix):

$$\boldsymbol{A} \cdot \boldsymbol{I} = \boldsymbol{I} \cdot \boldsymbol{A} = \boldsymbol{A}$$

Inverse of a Matrix

➢ For a square matrix, the inverse matrix will be such that the product of the original matrix and the inverse matrix will produce an identity matrix.

$$\boldsymbol{A}\boldsymbol{A}^{-1} = \boldsymbol{A}^{-1}\boldsymbol{A} = \boldsymbol{I}$$

➢ Note: Not all square matrices have inverses. A square matrix which has an inverse is called invertible or nonsingular matrix and a square matrix which does not has an inverse is called noninvertible or singular matrix.

➢ A square matrix is singular if and only if its **determinant** is 0

Square Matrices: Determinant of a Matrix

- Determinant of a square matrix det(A) is a mapping to a scalar
- It is equal to the product of all eigenvalues of the matrix
- Measures how much multiplication by the matrix expands or contracts space

Square Matrices: How To Calculate a Determinant

➢ For a 1x1 matrix:

$$\det(a_{11}) = a_{11}$$

➢ For a 2x2 matrix:

$$\begin{vmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{vmatrix} = a_{11}a_{22} - a_{12}a_{21}$$

➢ For a 3x3 matrix:

$$\begin{vmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{vmatrix} =$$

$$a_{11}a_{22}a_{33} + a_{12}a_{23}a_{31} + a_{13}a_{21}a_{32} - a_{11}a_{23}a_{32} - a_{12}a_{21}a_{33} - a_{13}a_{22}a_{31} =$$

$$a_{11}(a_{22}a_{33} - a_{23}a_{32}) - a_{12}(a_{21}a_{33} - a_{23}a_{31}) + a_{13}(a_{21}a_{32} - a_{22}a_{31})$$

➢ (There exists a rather simple way to do this with arbitrarily large matrices, but we omit this in this course).

Special Matrices & Vectors

- **Unit Vector**
  - A vector with unit norm

$$\left|\left|\boldsymbol{x}\right|\right|_2 = 1$$

- **Orthogonal Vectors**
  - A vector x and a vector y are orthogonal to each other if

$$\boldsymbol{x}^T\boldsymbol{y} = 0$$

  - Vectors are at 90 degrees to each other

- **Orthogonal Matrix**
  - A square matrix whose columns and rows are orthogonal unit vectors

$$\boldsymbol{A}^{-1} = \boldsymbol{A}^T$$

Special Matrices & Vectors

➢ Diagonal Matrix

- ➢ Mostly zeros, with non-zero entries in diagonal
- ➢ $diag(\boldsymbol{v})$ is a square diagonal matrix with diagonal elements given by entries of vector $\boldsymbol{v}$
- ➢ Multiplying $diag(\boldsymbol{v})$ by vector $\boldsymbol{x}$ only needs to scale each element x_i by v_i

➢ Symmetric Matrix

- ➢ Is equal to its transpose:

$$\boldsymbol{A} = \boldsymbol{A}^T$$

- ➢ E.g., a distance matrix is symmetric with $A_{ij} = A_{ji}$

Matrices as Linear Transformation

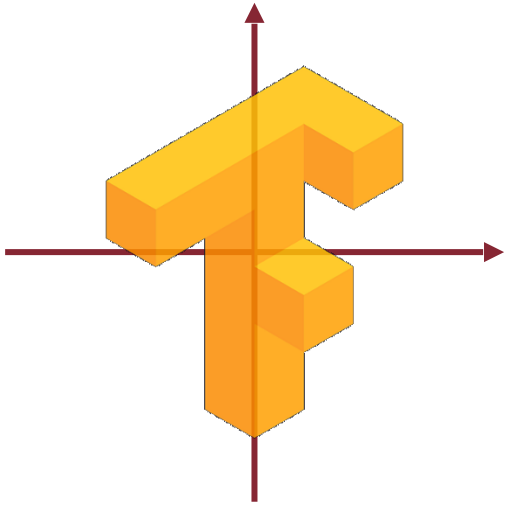
➢ At the end of the day, a Matrix defines a linear transformation

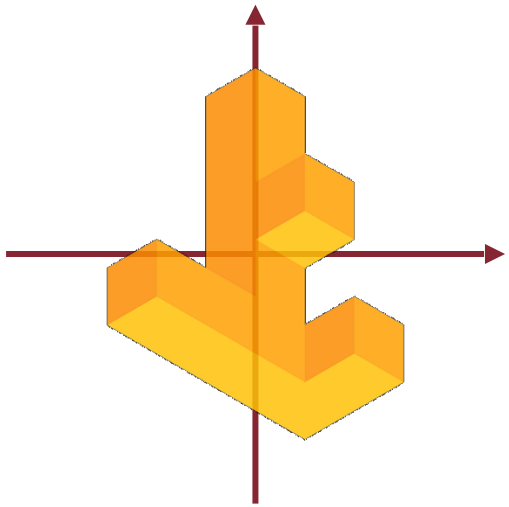
➢ A Matrix $A \in \mathbb{R}^{n \times m}$ projects points from $\mathbb{R}^n$ to their image in $\mathbb{R}^m$

➢ This is later exactly what we will do with the hidden layers

➢ Let's have a look at some examples for $\mathbb{R}^{2 \times 2}$ since we can visualize them

Examples: Mirror

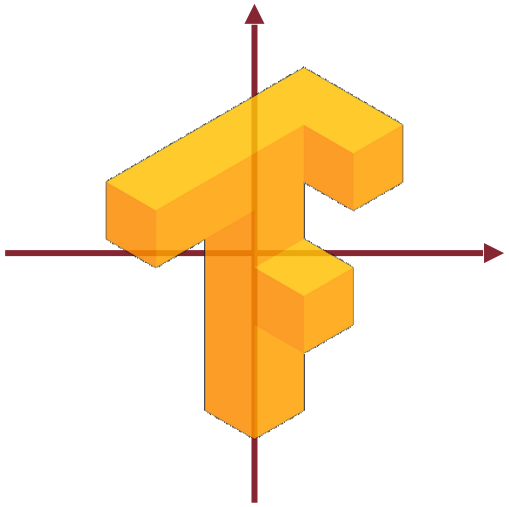


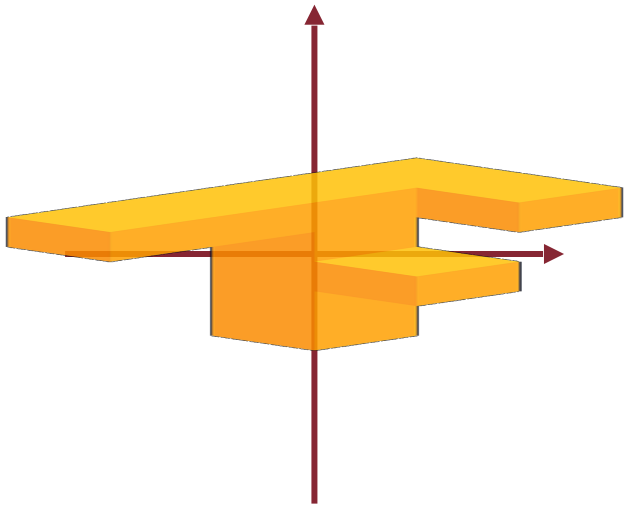


➢ Reflection on the x-Axis:

$$Mx = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} x \\ -y \end{bmatrix}$$

Examples: Stretching





➢ Stretching along the Axes:

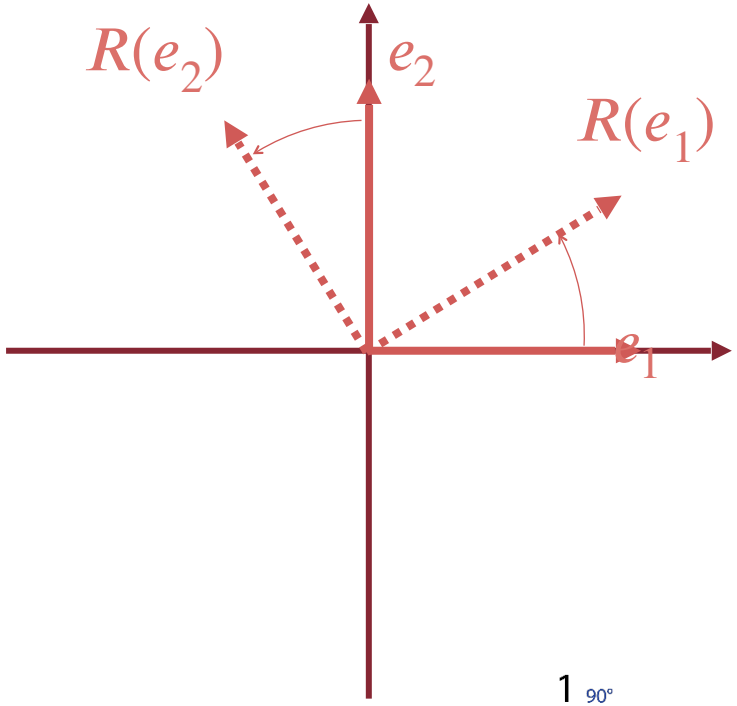
$$Sx = \begin{bmatrix} 2 & 0 \\ 0 & 0.5 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 2x \\ 0.5 \cdot y \end{bmatrix}$$

Example: Rotation

- We are looking for the image of the standard basis e_1, e_2
- We need the following images:

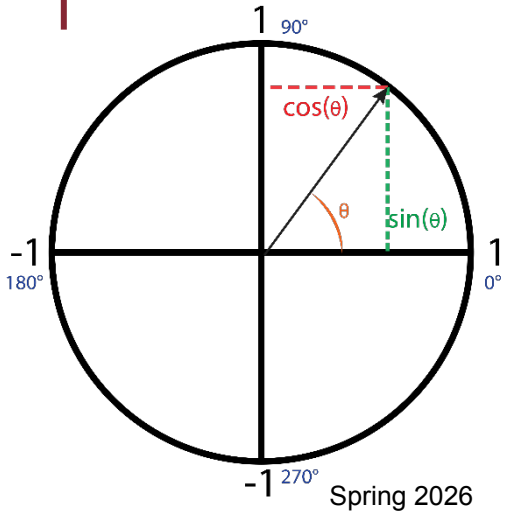
$$R(e_1) = R\begin{bmatrix}1\\0\end{bmatrix} = \begin{bmatrix}\cos\theta\\ \sin\theta\end{bmatrix}$$

$$R(e_2) = R\begin{bmatrix}0\\1\end{bmatrix} = \begin{bmatrix}-\sin\theta\\ \cos\theta\end{bmatrix}$$

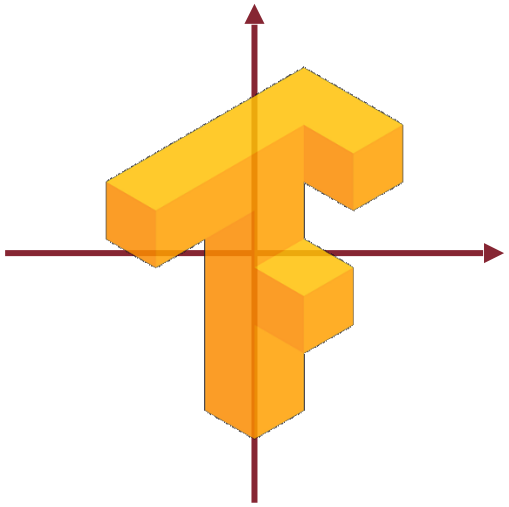


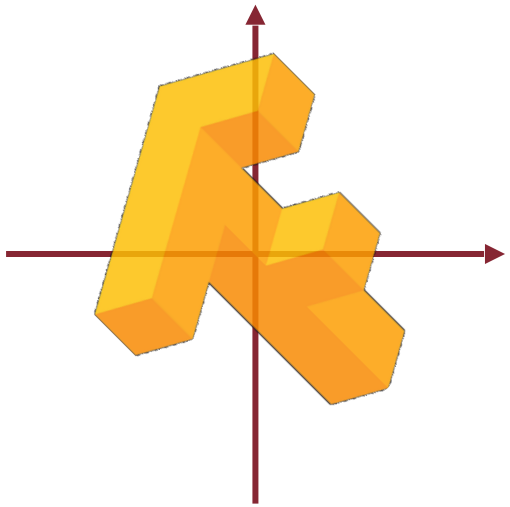
- Together for a rotation of 45 degree:

$$R = \begin{bmatrix}\cos\theta & -\sin\theta\\ \sin\theta & \cos\theta\end{bmatrix} = \begin{bmatrix}\frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}}\\ \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}}\end{bmatrix}$$



Examples: Rotation

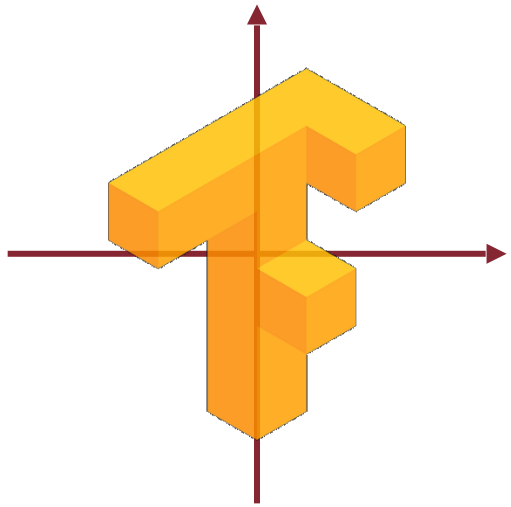


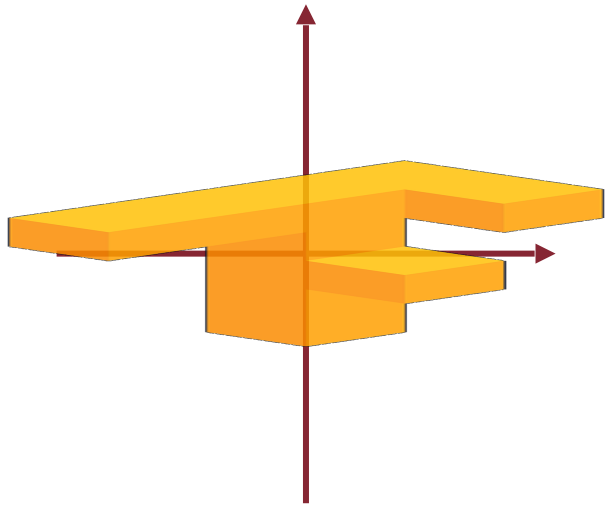


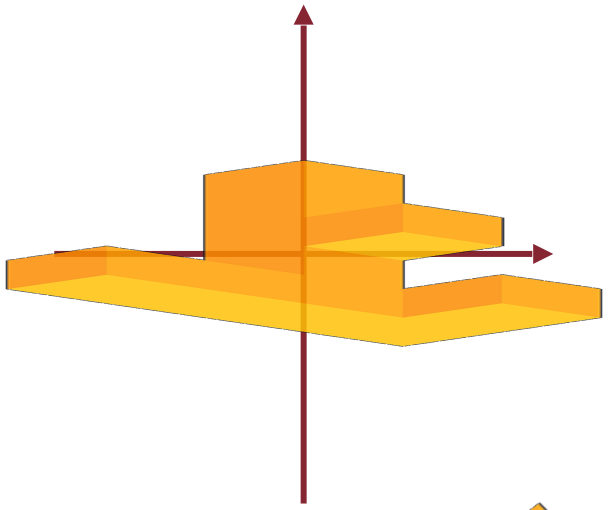
➢ Rotation on the x-Axis:

$$Rx = \begin{bmatrix} \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \end{bmatrix} \cdot \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} \frac{1}{\sqrt{2}}x - \frac{1}{\sqrt{2}}y \\ \frac{1}{\sqrt{2}}x + \frac{1}{\sqrt{2}}y \end{bmatrix}$$

Example: Combinations

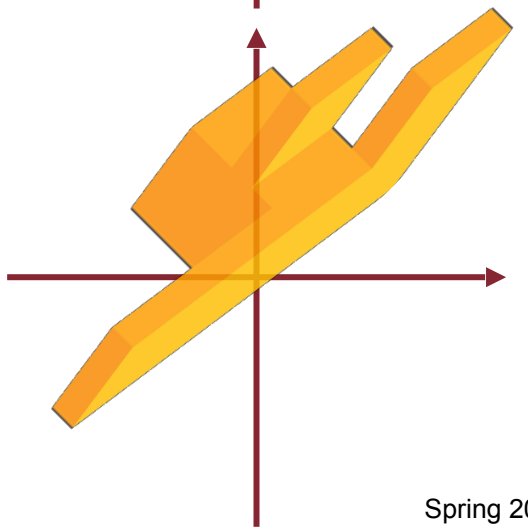




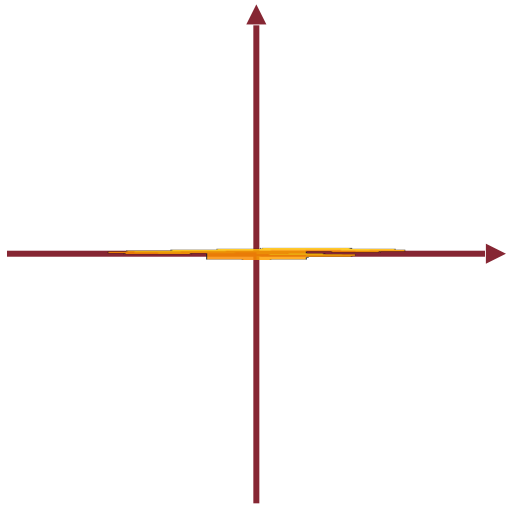


➢ Simply combine the transformations

$$M = RMS = \begin{bmatrix} 2\frac{1}{\sqrt{2}} & -2\frac{1}{\sqrt{2}} \\ -\frac{1}{2}\frac{1}{\sqrt{2}} & -\frac{1}{2}\frac{1}{\sqrt{2}} \end{bmatrix}$$



On the invertibility of the projections



- It becomes clear that not every transformation is invertible.
- For instance, $Mx = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = x$
  - Every point (x, y) gets projected onto the same value x
  - Impossible to "reconstruct" y
- Non square matrices project points into higher/lower dimensional spaces (as the example above does)

Keywords for this session

- Linear algebra
- **Probabilities & Standard Distributions**
- Eigendecomposition & PCA

Why Probability?

- Much of CS deals with entities that are certain
  - CPU executes flawlessly
    - At least almost … there are CPU bugs and CPUs can also be broken
  - CS and software engineers work in clean and certain environment
  - Surprising that ML heavily uses probability theory

- Reasons for ML use of probability theory
  - Must always deal with uncertain quantities
    - Also with non-deterministic (stochastic) quantities
  - Many sources for uncertainty and stochasticity

Sources of Uncertainty

1. Inherent stochasticity of system being modeled
   - ➢ Subatomic particles are probabilistic
   - ➢ Cards shuffled in random order

2. Incomplete observability
   - ➢ Deterministic systems appear stochastic when not all variables are observed

3. Incomplete modeling
   - ➢ Discarded information results in uncertain predictions

Practical to use uncertain rule

- Simple rule “Most birds fly” is cheap to develop and broadly useful
- Rules of the form “Birds fly, except for very young birds that have not learned to fly, sick or injured birds that have lost ability to fly, flightless species of birds...” are expensive to develop, maintain and communicate
  - Also still brittle and prone to failure

Tools of Probability

- Probability theory was originally developed to analyze frequencies of events
  - Such as drawing a hand of cards in poker
  - These events are repeatable
  - If we repeated experiment infinitely many times, proportion of p of outcomes would result in that outcome

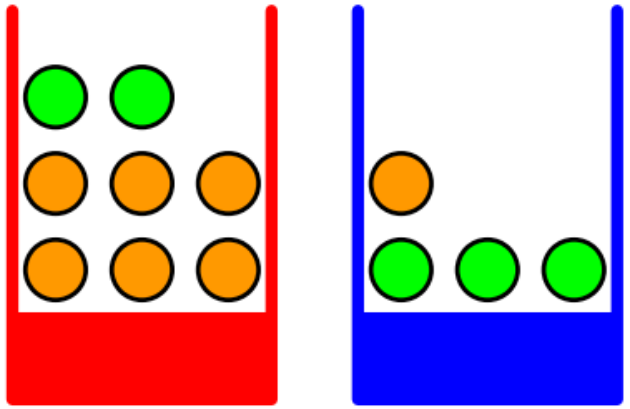
- Is it applicable to propositions not repeatable?
  - Patient has 40% chance of flu
    - Cannot make infinite replicas of the patient
  - We use probability to represent degree of belief

- Former is frequentist probability, latter Bayesian, just fyi.

Definition of Probability

➢ To begin with, we shall define the probability of an event to be the fraction of times that event occurs out of the total number of trials, in the limit that the total number of trials goes to infinity.

➢ Let's look at the following simple Experiment:
- ➢ Two boxes, red (r) and blue (b)
- ➢ Each boxes contain either apples (a) or oranges (o)
- ➢ The box that will be chosen is a random variable B. It takes the values r or b
- ➢ The fruit sampled out of the selected box is donated F and takes the values a and o



➢ Taken from Christopher M. Bishop, "Pattern Recognition and Machine Learning"

Random Variables

- Let's assume we select the blue box 60% of the time
- Repeating the box selection infinite number of times, we say that the probability of selecting the red box is 4/10 and the blue box 6/10.
- We formally write this as follows:

$$p(B = r) = \frac{4}{10} \qquad p(B = b) = \frac{6}{10}$$

  - Note that, by definition, probabilities must lie in the interval [0, 1]
  - Also, if the events are mutually exclusive and if they include all possible outcomes, then we see that the probabilities for those events must sum to one.

- Taken from Christopher M. Bishop, "Pattern Recognition and Machine Learning"

Joint Probability Distributions

- We can now ask questions such as:
  - "what is the overall probability that the selection procedure will pick an apple?"
  - "given that we have chosen an orange, what is the probability that the box we chose was the blue one?"

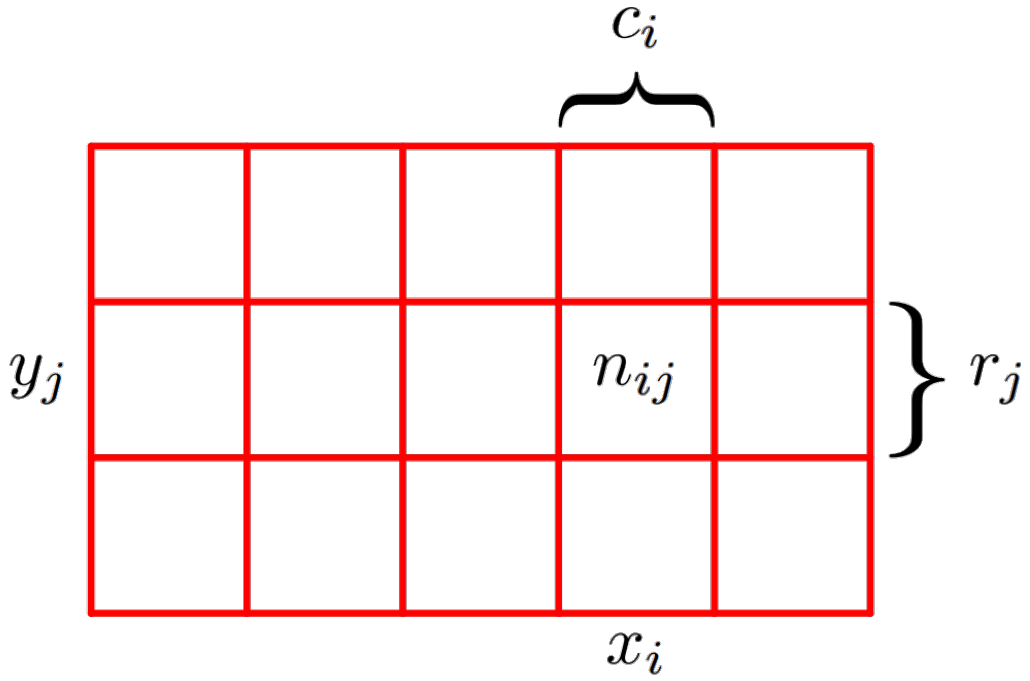
- To answer such questions, we need the elementary rules of probability:
  - the **sum rule**
  - the **product rule**

- Taken from Christopher M. Bishop, "Pattern Recognition and Machine Learning"

Joint Probability Distributions

- Consider we have two random variables X and Y
  - X can take any of the values x_i where $i = 1,\ldots, M$
  - Y can take the values y_j where $j = 1,\ldots, L$

- Consider a total of N samples
  - The number of such trials in which $X = x_i$ and $Y = y_j$ be n_{ij}
  - The number of trials in which X takes the value x_i is c_i
  - The number of trials in which Y takes the value y_j is r_j

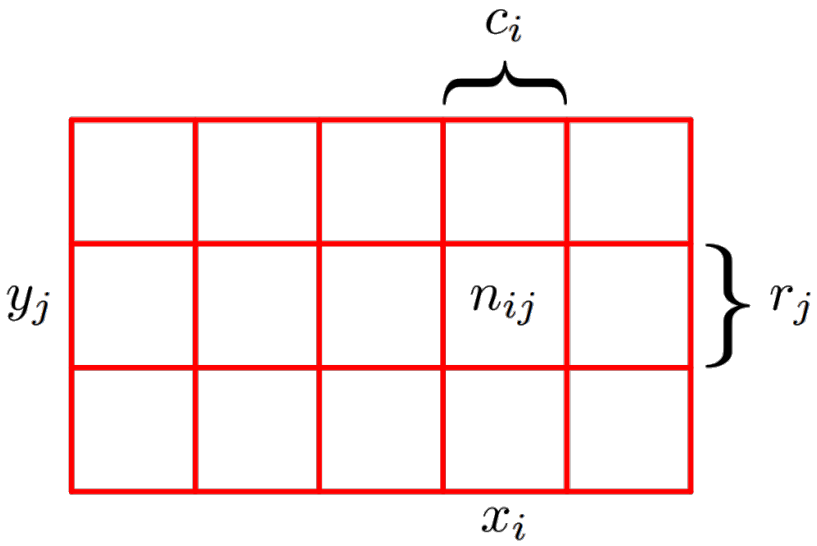


- Taken from Christopher M. Bishop, "Pattern Recognition and Machine Learning"

Joint Probability Distributions

➢ The probability that $X = x_i$ and $Y = y_i$ is written as:

$$p\Big(X = x_i, Y = y_j\Big)$$



➢ This is called the joint probability distribution

➢ It is given by the fractions of point in cell i, j

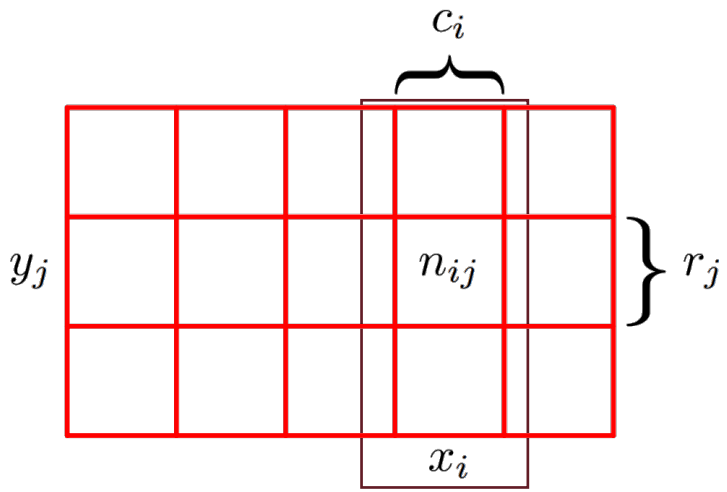
$$p\Big(X = x_i, Y = y_j\Big) = \frac{n_{ij}}{N}$$

➢ Taken from Christopher M. Bishop, "Pattern Recognition and Machine Learning"

Sum Rule

➢ The probability that X takes the value x_i irrespective of the value of Y is written as $p(X = x_i)$ and is given by the fraction of points in column i

$$p(X = x_i) = \frac{c_i}{N}$$



➢ Since c_i is just the sum of the cells of the column, we can also write:

$$p(X = x_i) = \sum_{j=1}^{L} p(X = x_i, Y = y_j)$$

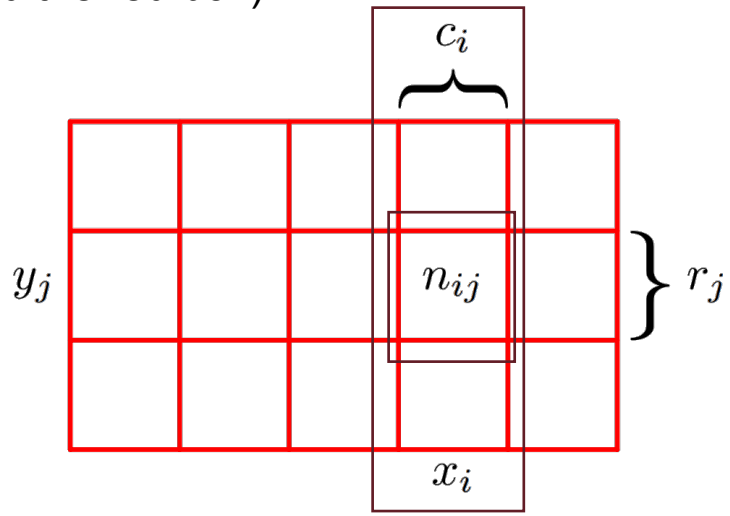
Note that $p(X = x_i)$ is sometimes called the marginal probability, because it is obtained by marginalizing, or summing out, the other variables (in this case Y).

➢ Taken from Christopher M. Bishop, "Pattern Recognition and Machine Learning"

Product Rule or Conditional Probabilities

- Now, let's only consider those cases, where $X = x_i$.
- We are now interested in the probability that $Y = y_j$ if X is already fixed to x_i
  (For example: What is the probability of sampling an apple, when we have selected the red box)
- We write this as:

$$p\left(Y = y_j \middle| X = x_i\right) = \frac{n_{ij}}{c_i}$$



- With the results from before, we get:

$$p\left(X = x_i, Y = y_j\right) = \frac{n_{ij}}{N} = \frac{n_{ij}}{c_i} \cdot \frac{c_i}{N} = p\left(Y = y_j \middle| X = x_i\right) p\left(X = x_i\right)$$

- Taken from Christopher M. Bishop, "Pattern Recognition and Machine Learning"

Rules of Probability

➢ **Sum Rule**

$$p(X) = \sum_{Y} p(X, Y)$$

➢ **Product Rule**

$$p(X, Y) = p(Y \mid X) p(X)$$

(Note the more compact notation: We simply write $p(B)$ to denote a distribution over the random variable B, or $p(r)$ to denote the distribution evaluated for the particular value r, provided that the interpretation is clear from the context.)

➢ Taken from Christopher M. Bishop, "Pattern Recognition and Machine Learning"

Bayes theorem

➢ From the rules, we can directly derive Bayes theorem

$$\overbrace{P(Y \mid X)}^{\text{posterior}} = \frac{\overbrace{P(X \mid Y)}^{\text{likelihood}} \overbrace{P(Y)}^{\text{prior}}}{\underbrace{P(X)}_{\text{evidence}}}$$

➢ **Prior**: Our assumptions about Y before observing the data.

➢ **Likelihood**: The effect of the observed data X.

➢ **Posterior**: The uncertainty in Y after we have observed X.

"A Bayesian is one who, vaguely expecting a horse, and catching a glimpse of a donkey, strongly believes he has seen a mule."

Bayes theorem in Action

- Consider a SPAM filter which classifies mail into:
  - Good: H (Ham)
  - Bad: S (Spam)
- We observe a message containing the word W "replica"
- We use Bayes theorem

$$p(S \mid W) = \frac{p(W \mid S)p(S)}{p(W \mid S)p(S) + p(W \mid H)p(H)} = \frac{p(W \mid S)p(S)}{p(W)}$$

  - $p(S \mid W)$ the probability that this message is SPAM
  - $p(S)$ the probability that any given message is SPAM (our **prior**)
  - $p(W \mid S)$ the probability that "replica" appears in SPAM (our **likelihood**)
  - $p(W)$ the probability that this word appears in any message (our **evidence**)

How does it work?

- We have trained the classifier, i.e., we have scanned through emails we know are SPAM or HAM and have calculated the following this:
  - 80% of the mail are SPAM, i.e. our prior $p(s) = 0.8$
    - That means, without any evidence, we are 80% certain, a message is SPAM
  - The probabilities of words occurring in SPAM and HAM

- Here, our filter again, in all its glory:

$$p(S \mid W) = \frac{p(W \mid S)p(S)}{p(W \mid S)p(S) + p(W \mid H)p(H)}$$

How does it work?

- We observe a completely irrelevant word: "the"
  - It appears in every singe message
  - $p(W \mid S) = 1$
  - $p(W \mid H) = 1$

- When we now apply the filter, we get:

$$p(S \mid W) = \frac{p(W \mid S)p(S)}{p(W \mid S)p(S) + p(W \mid H)p(H)}$$

$$= \frac{1 \cdot 0.8}{1 \cdot 0.8 + 1 \cdot 0.2} = 0.8$$

- That means, this useless evidence has neither strengthen nor weakened our prior believe

How does it work?

- Now we observe a typical SPAM word: "replica"
  - It appears in every fourth SPAM message, but only in every 100th HAM:
  - $p(W \mid S) = 0.25$
  - $p(W \mid H) = 0.01$

- When we now apply the filter, we get:

$$p(S \mid W) = \frac{p(W \mid S)p(S)}{p(W \mid S)p(S) + p(W \mid H)p(H)}$$

$$= \frac{0.25 \cdot 0.8}{0.25 \cdot 0.8 + 0.01 \cdot 0.2} = 0.99$$

- That means, the observed evidence, lead our posterior to be stronger than the prior!

How does it work?

- Now we observe a HAM, my name is spelled correct: "Galke"
  - It appears as follows:
  - $p(W \mid S) = 0.05$
  - $p(W \mid H) = 0.75$

- When we now apply the filter, we get:

$$p(S \mid W) = \frac{p(W \mid S)p(S)}{p(W \mid S)p(S) + p(W \mid H)p(H)}$$

$$= \frac{0.05 \cdot 0.8}{0.05 \cdot 0.8 + 0.75 \cdot 0.2} = 0.21$$

- That means, the observed evidence, lead our posterior to less leaning towards SPAM!

Random Variables

➢ A **random variable** X is a variable that can take on different values randomly

➢ On its own, a random variable is just a description of the states that are possible;

➢ It must be coupled with a probability distribution that specifies how likely each of these states are.

➢ Random variables may be **discrete** or **continuous**

Probability Distributions

- A probability distribution is a description of how likely a random variable or a set of random variables is to take each of its possible states

- The way to describe the distribution depends on whether it is discrete or continuous

Probability Mass Functions

- The probability distribution over discrete variables is given by a probability mass function
- PMFs of variables are denoted by P and inferred from their argument, e.g., $P(x)$, $P(y)$
- They can act on many variables and is known as a joint distribution, written as $P(x, y)$

- To be a PMF it must satisfy:
  - Domain of P is the set of all possible states of x
  - $\forall x \in X,\ 0 \leq P(x) \leq 1$
  - $\sum_{x \in X} P(x) = 1$ (normalization)

Continuous Variables and PDFs

- When working with continuous variables, we describe probability distributions using probability density functions

- To be a pdf p must satisfy:
  - The domain of p must be the set of all possible states of X
  - $\forall x \in X,\ p(x) \geq 0$. Note, there is no requirement for $p(x) \leq 1$.
  - $\int p(x)dx = 1$

Expectation

➢ Expectation or expected value of $f(x)$ w.r.t. $P(X)$ is the average or mean value that f takes on when x is drawn from P

➢ For discrete variables:

$$E[X] = \sum_{x \in X} P(x) \cdot x$$

➢ For continuous variables:

$$E[X] = \int_{x} p(x)x \; dx$$

Properties of Expectation

- **Linearity of expectation**
  - $E[X+Y] = E[X] + E[Y]$
  - $E[\alpha X] = \alpha E[X]$
- **Expectation is also defined for functions on random variables**
  - **Discrete:** $E[f] = \sum_x p(x) \cdot f(x)$
  - **Continuous**: $E[f] = \int_{-\infty}^{+\infty} p(x) \cdot f(x) dx$

Variance

$$Var(X) = E\left[(x - E[X])^2\right]$$

- Measures how much the value of $f(x)$ vary from the expectation
- Low variance means values cluster around its expectations
- Square root of the variance is the standard deviation

Bernoulli Distribution

- Distribution over a single binary random variable
- It is controlled by a single parameter $\phi \in [0,1]$
  - Which gives the probability a random variable being equal to 1
- It has the following properties (to the right)

$$P(\mathrm{x} = 1) = \phi$$

$$P(\mathrm{x} = 0) = 1 - \phi$$

$$P(\mathrm{x} = x) = \phi^x (1 - \phi)^{1-x}$$

$$\mathbb{E}_{\mathrm{x}}[\mathrm{x}] = \phi$$

$$\mathrm{Var}_{\mathrm{x}}(\mathrm{x}) = \phi(1 - \phi)$$

Multinoulli Distribution

- Distribution over a single discrete variable with k different states

- It is parameterized by a vector $\boldsymbol{p} \in [0,1]^{k-1}$
  - where p_i is the probability of the ith state
  - The final kth state's probability is implicitly given by $1 - \mathbf{1}^T \boldsymbol{p}$
  - We must constrain $\mathbf{1}^T \boldsymbol{p} \leq 1$

- Multinoullis refer to distributions over categories
  - So we don't assume state 1 has value 1, etc.
  - For this reason we do not usually need to compute the expectation or variance of multinoulli variables since the states are not necessarily ordered

Gaussian Distribution

- Probably one of the most commonly used distributions

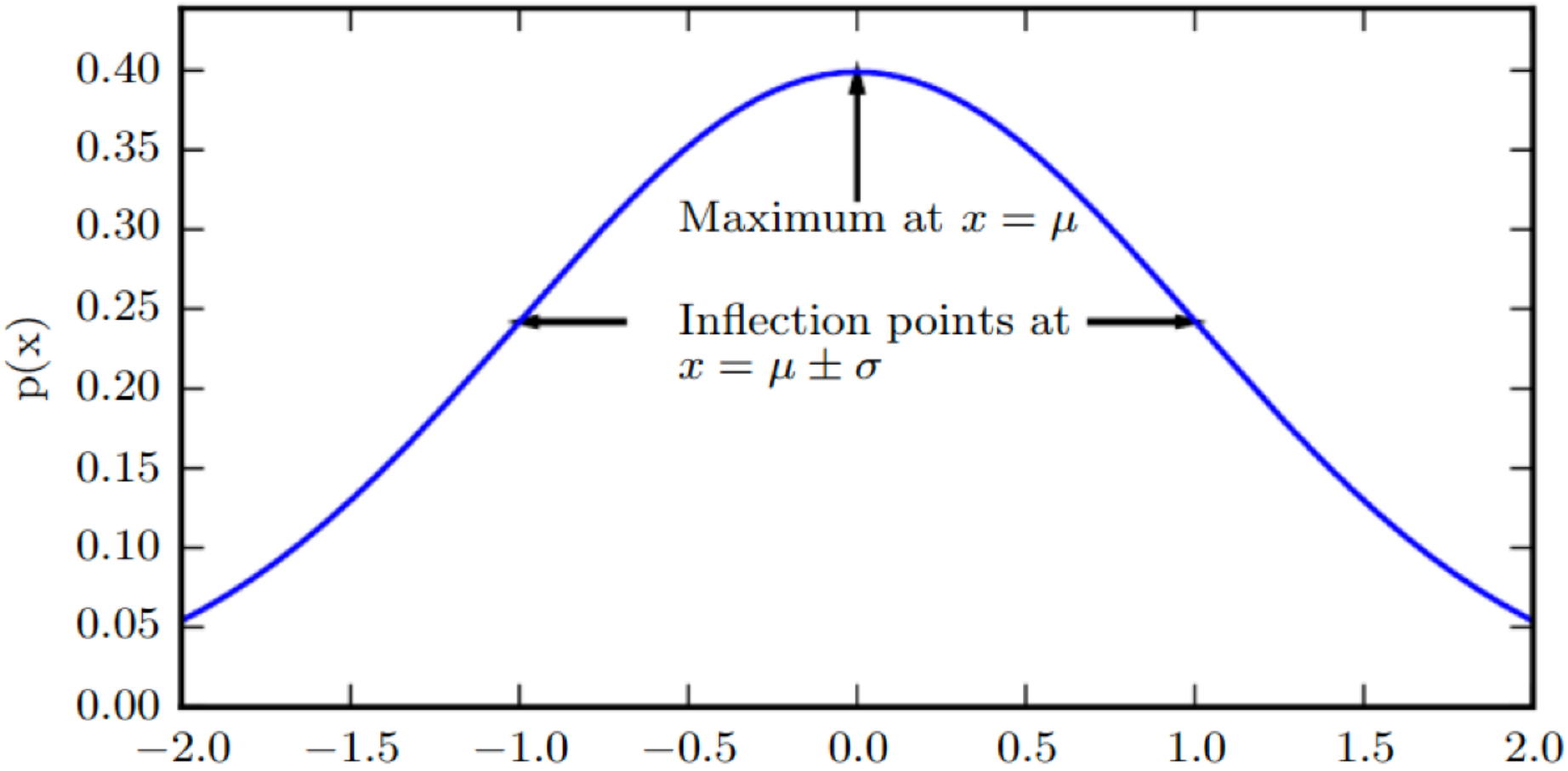
$$N(x;\mu,\sigma^2) = \sqrt{\frac{1}{2\pi\sigma^2}}\exp\left(-\frac{1}{2\sigma^2}(x-\mu)^2\right)$$

- Two parameters
  - μ gives the location of the central peak, which is also the mean of the distribution
  - The standard deviation is given by σ and variance by σ^2

- If this is evaluated frequently, sometimes parameterized with the inverse variance (or precision) β:

$$N(x;\mu,\beta^{-1}) = \sqrt{\frac{\beta}{2\pi}}\exp\left(-\frac{1}{2}\beta(x-\mu)^2\right)$$

Gaussian Distribution, $\mu = 0, \sigma = 1$



Justifications for Normal Assumption

- 1. **Central Limit Theorem**
  - Many distributions we wish to model are truly normal
  - Sum of many independent distributions is normal
  - Can model complicated systems as normal even if components have more structured behavior

- 2. **Maximum Entropy**
  - Of all possible probability distributions with the same variance, normal distribution encodes the maximum amount of uncertainty over real numbers
  - Thus, the normal distributions inserts the least amount of prior knowledge into a model

Multidimensional Gaussian Distributions

➢ The Gaussian Distribution can easily be extended to the multivariate case.

➢ Now, $\boldsymbol{x}$ and $\boldsymbol{\mu}$ are a vector, $\boldsymbol{\Sigma}$ a positive semidefinite symmetric matrix (the covariance matrix):

$$N(\boldsymbol{x};\boldsymbol{\mu},\boldsymbol{\Sigma}) = \sqrt{\frac{1}{(2\pi)^2|\boldsymbol{\Sigma}|}}\exp\left(-\frac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu})^T\boldsymbol{\Sigma}^{-1}(\boldsymbol{x}-\boldsymbol{\mu})\right)$$

➢ Analogously, with the precision Matrix

$$N(\boldsymbol{x};\boldsymbol{\mu},\boldsymbol{\beta}^{-1}) = \sqrt{\frac{|\boldsymbol{\beta}|}{(2\pi)^2}}\exp\left(-\frac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu})^T\boldsymbol{\beta}(\boldsymbol{x}-\boldsymbol{\mu})\right)$$

The *variance* of $f(x)$ is defined by

$$\text{var}[f] = \mathbb{E}\left[(f(x) - \mathbb{E}[f(x)])^2\right] \tag{2.44}$$

and provides a measure of how much $f(x)$ varies around its mean value $\mathbb{E}[f(x)]$. Expanding out the square, we see that the variance can also be written in terms of the expectations of $f(x)$ and $f(x)^2$:

Exercise 2.8

$$\text{var}[f] = \mathbb{E}[f(x)^2] - \mathbb{E}[f(x)]^2. \tag{2.45}$$

Keywords for this session

- Linear algebra
- Probabilities & Standard Distributions
- **Eigendecomposition & PCA**

Matrix Decomposition

- Matrices can be decomposed into factors to learn universal properties about them not discernible from their representation
  - E.g., from decomposition of integer into prime factors 12=2x2x3 we can discern that
    - 12 is not divisible by 5 or
    - any multiple of 12 is divisible by 3

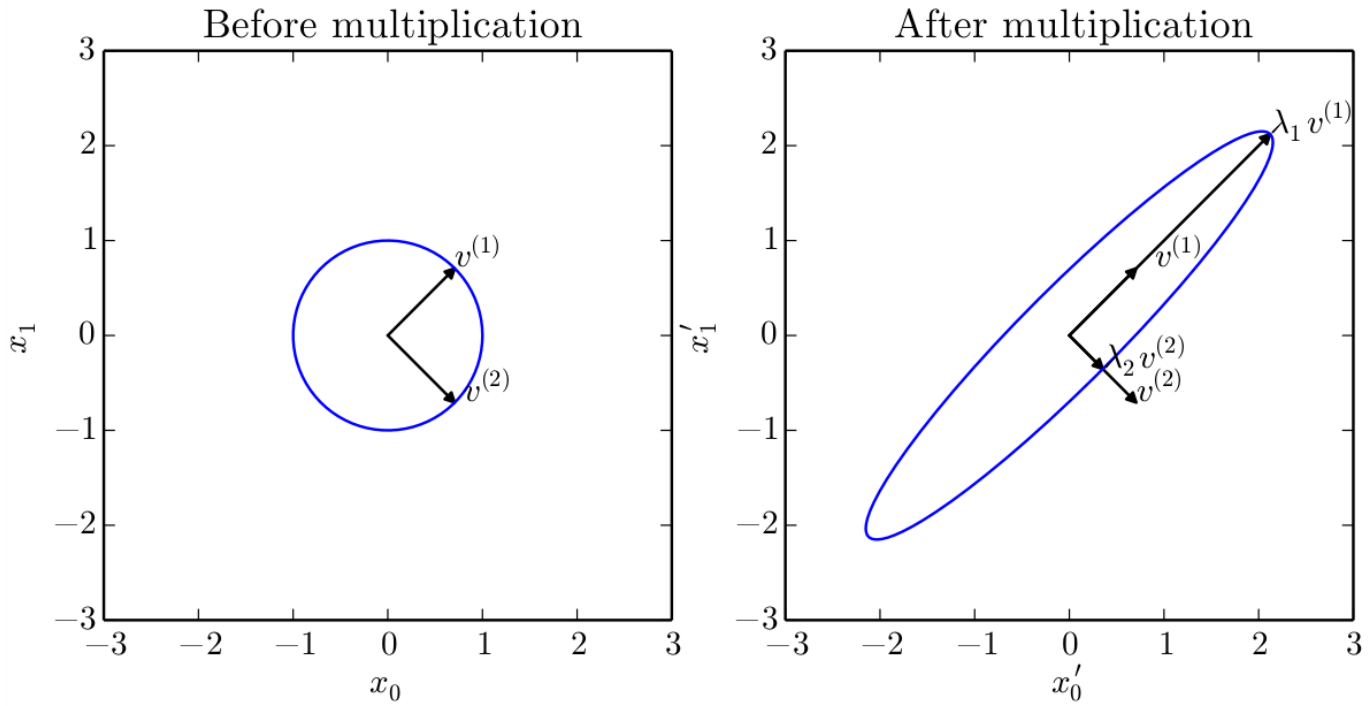
- Analogously, a matrix is decomposed into Eigenvalues and Eigenvectors to discern universal properties

Eigenvector

➢ An eigenvector of a square matrix $\boldsymbol{A}$ is a non-zero vector $\boldsymbol{v}$ such that multiplication by $\boldsymbol{A}$ only changes the scale of $\boldsymbol{v}$

$$\boldsymbol{A}\boldsymbol{v} = \lambda\boldsymbol{v}$$

➢ The scalar λ is known as eigenvalue
- ➢ If $\boldsymbol{v}$ is an eigenvector of $\boldsymbol{A}$, so is any rescaled vector $s\boldsymbol{v}$.
- ➢ $s\boldsymbol{v}$ still has the same eigen value.
- ➢ Thus, the unit Eigenvector is used



Eigenvalue and Characteristic Polynomial

➢ Consider $\boldsymbol{Av} = \boldsymbol{w}$

➢ If $\boldsymbol{v}$ and $\boldsymbol{w}$ are scalar multipliers, then

$$\boldsymbol{Av} = \lambda \boldsymbol{v}\left(\boldsymbol{A} - \lambda \boldsymbol{I}\right)\boldsymbol{v} = 0$$

➢ This has a non-zero solution if

$$\left|\boldsymbol{A} - \lambda \boldsymbol{I}\right| = 0$$

➢ The nulls of the polynomial of degree n are the eigenvalues of $\boldsymbol{A}$

Example

$$\boldsymbol{A} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$$

➢ Then, the characteristic polynomial is

$$\left| \boldsymbol{A} - \lambda \boldsymbol{I} \right| = \begin{bmatrix} 2-\lambda & 1 \\ 1 & 2-\lambda \end{bmatrix} = 3 - 4\lambda + \lambda^2$$

➢ The polynomial has the roots $\lambda = 1$ and $\lambda = 3$ which are the eigenvalues of $\boldsymbol{A}$.

➢ The eigenvectors can be found by solving the equation $Av = \lambda v$ using the different eigenvectors:

$$v_{\lambda=1} = \begin{bmatrix} 1 \\ -1 \end{bmatrix},\ v_{\lambda=3} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

Eigendecomposition

- Suppose that matrix $\boldsymbol{A}$ has n linearly independent eigenvectors $\left\{v^{(1)}, \ldots, v^{(n)}\right\}$ with the eigenvalues $\left\{\lambda_1, \ldots, \lambda_n\right\}$
- Concatenate eigenvectors to form matrix $\boldsymbol{V}$
- Concatenate eigenvalues to form vector $\lambda = \left[\lambda_1, \ldots, \lambda_n\right]$ (normally in descending order)
- The Eigendecomposition of $\boldsymbol{A}$ is given by

$$\boldsymbol{A} = \boldsymbol{V}\mathrm{diag}\left(\lambda\right)\boldsymbol{V}^{-1}$$

Decomposition of Symmetric Matrix

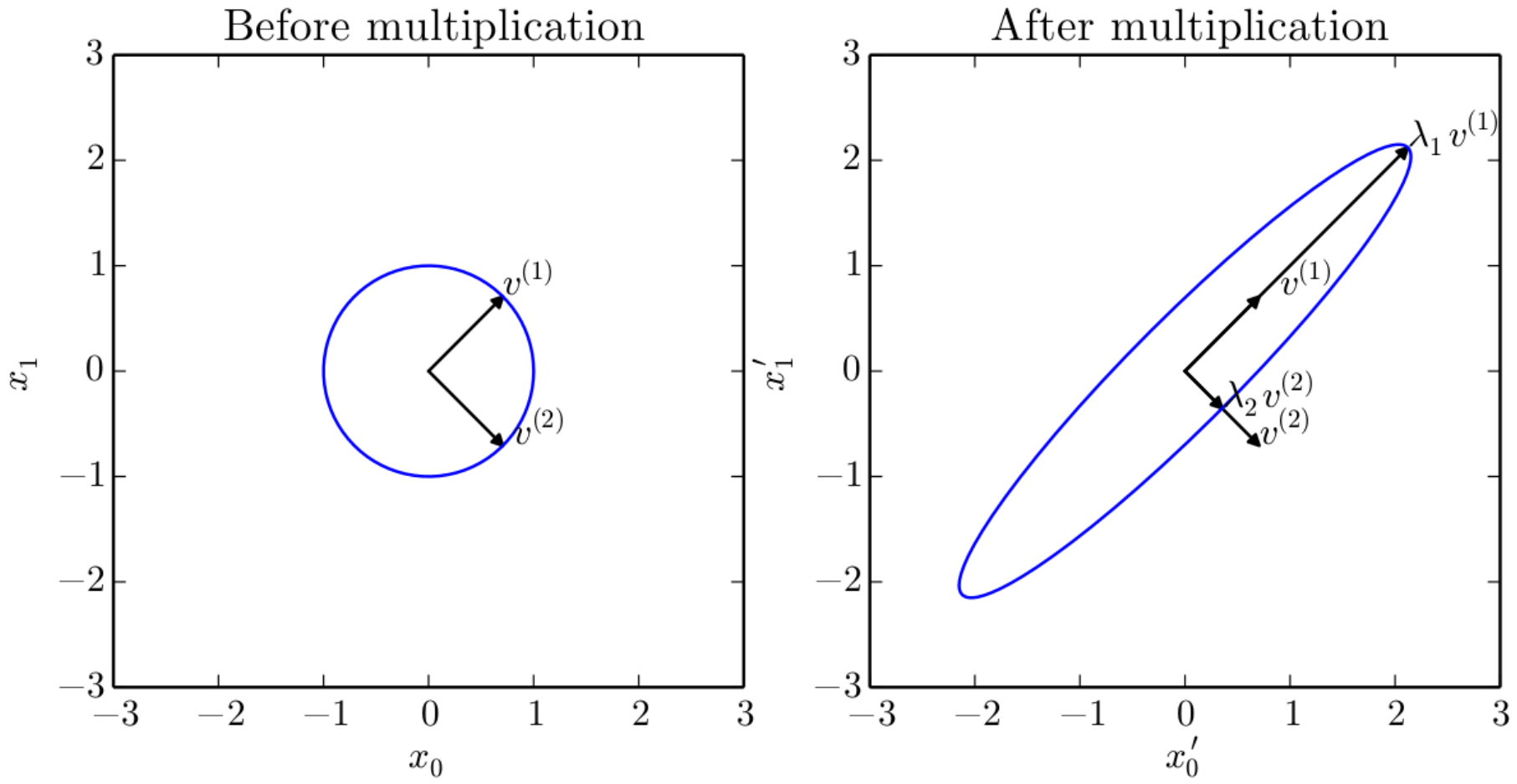
➢ Every real symmetric matrix $\boldsymbol{A}$ can be decomposed into real-valued eigenvectors and eigenvalues

$$\boldsymbol{A} = \boldsymbol{Q}\boldsymbol{\Lambda}\boldsymbol{Q}^T$$

➢ where $\boldsymbol{Q}$ is an orthogonal matrix composed of the eigenvectors and $\boldsymbol{\Lambda}$ the diagonal matrix of eigenvalues.

➢ We can think of $\boldsymbol{A}$ as scaling space by λ_i in direction $v^{(i)}$

Effect of Eigenvectors and Eigenvalues



Effect of Eigenvectors and Eigenvalues

- A matrix whose eigenvalues are
  - all positive is called **positive definite**
  - all positive or zero-valued is called **positive semidefinite**
  - all negative is called **negative definite**
  - all negative or zero-valued is called **negative semidefinite**

- Semidefinite matrices are interesting because they guarantee that

$$\forall \boldsymbol{x},\ \boldsymbol{x}^T \boldsymbol{A} \boldsymbol{x} \geq 0$$

- Positive definite matrices additionally guarantee that

$$\boldsymbol{x}^T \boldsymbol{A} \boldsymbol{x} = 0 \Rightarrow \boldsymbol{x} = 0$$

Application: Principal Component Analysis

- We have observed the usefulness of the decomposition of matrices to understand the effect of the mapping

- The PCA does follow a similar idea

- There are many interpretations to PCA
  - We look at it as a dimensionality reduction technique
  - Later, we will see the correspondence to layers in deep neural networks

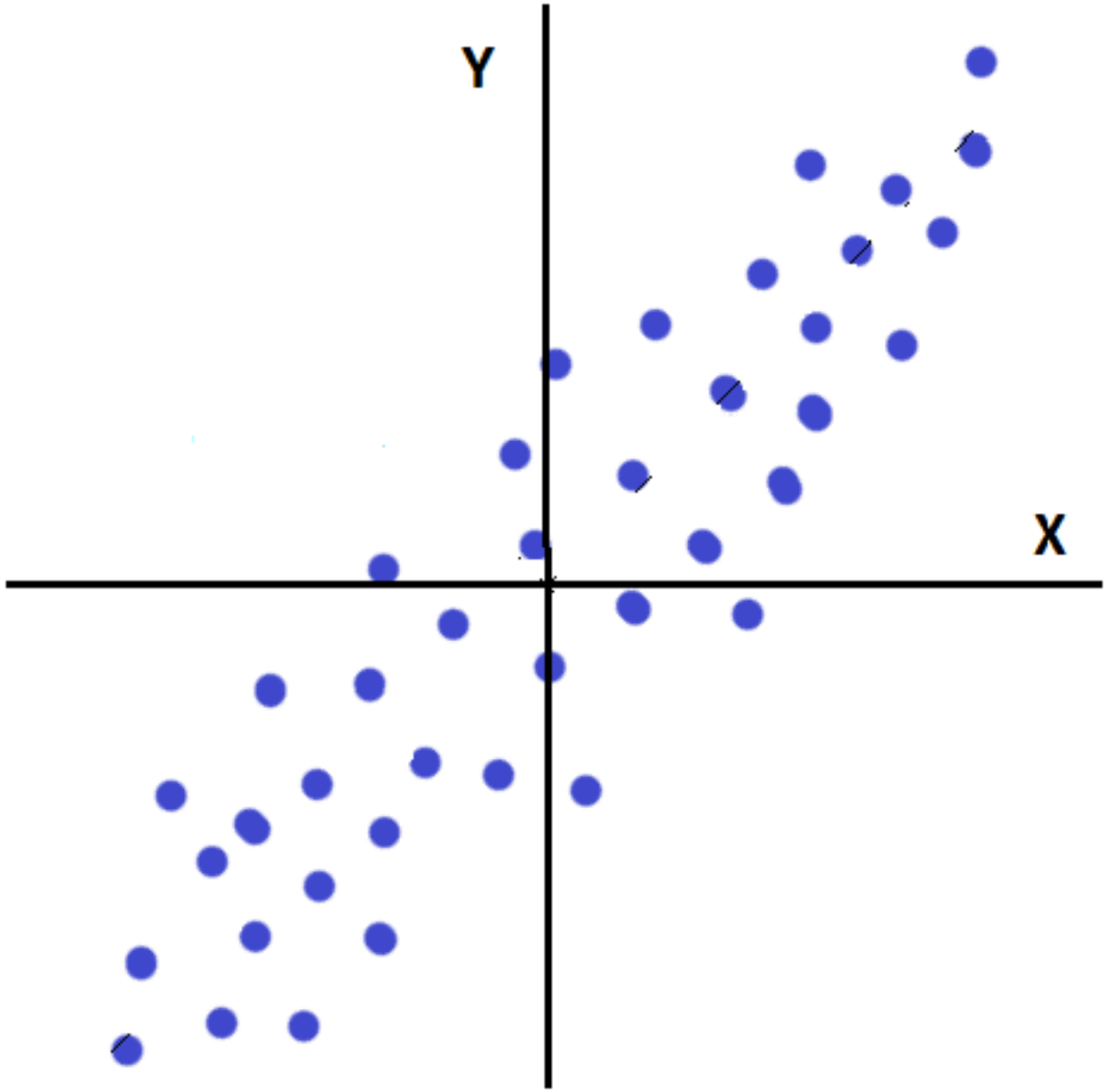
Dimensionality Reduction

- There are basically two ways to reduce dimensionality:

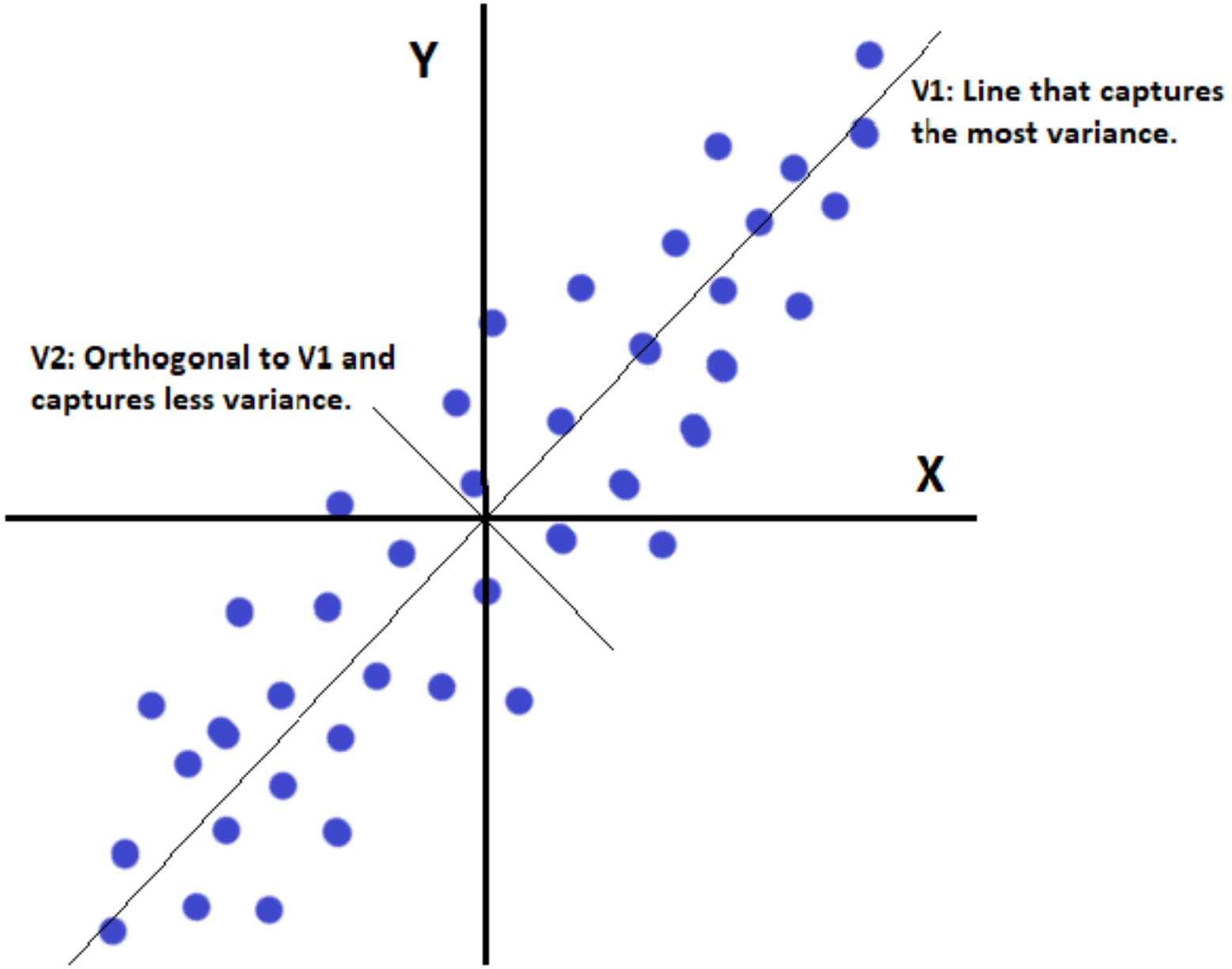
- **Feature extraction**: creating a subset of new features by combinations of the existing features
- **Feature selection**: choosing a subset of the available features

- What properties do we wish to retain by dimensionality reduction:
  - We aim to retain as much information and structure as possible in the reduced dimensions
  - In case of dimensionality, we want to capture most of the variance of the dataset
  - The dimension should be independent

Example



Example


Y
X
V1: Line that captures the most variance.
V2: Orthogonal to V1 and captures less variance.

Covariance Matrix

- The PCA tries to retain most of the variance, so we have to assess the variance within our dataset
  - Variance and Covariance are a measure of the “spread“ of a set of points around their center of mass (mean)
  - Variance – measure of the deviation from the mean for points in one dimension e.g. heights
  - Covariance as a measure of how much each of the dimensions vary from the mean with respect to each other
  - Covariance is measured between 2 dimensions to see if there is a relationship between the 2 dimensions
    - e.g. number of hours studied & marks obtained
  - The covariance between one dimension and itself is the variance

Covariance Matrix

➢ The covariance between to variables A and B is defined as

$$\mathrm{Cov}(A,B) = \frac{1}{n-1}\sum_{i=1}^{n}\left(a_i - \bar{A}\right)\left(b_i - \bar{B}\right)$$

➢ For all dimensions, e.g., A,B,C in Matrix-form:

$$Q = \begin{pmatrix} \mathrm{Cov}(A,A) & \mathrm{Cov}(A,B) & \mathrm{Cov}(A,C) \\ \mathrm{Cov}(B,A) & \mathrm{Cov}(B,B) & \mathrm{Cov}(B,C) \\ \mathrm{Cov}(C,A) & \mathrm{Cov}(C,B) & \mathrm{Cov}(C,C) \end{pmatrix}$$

- ➢ Diagonal is the variance of A, B, C
- ➢ The matrix is symmetric
- ➢ d-dimensional data will result in a $d \times d$ covariance matric

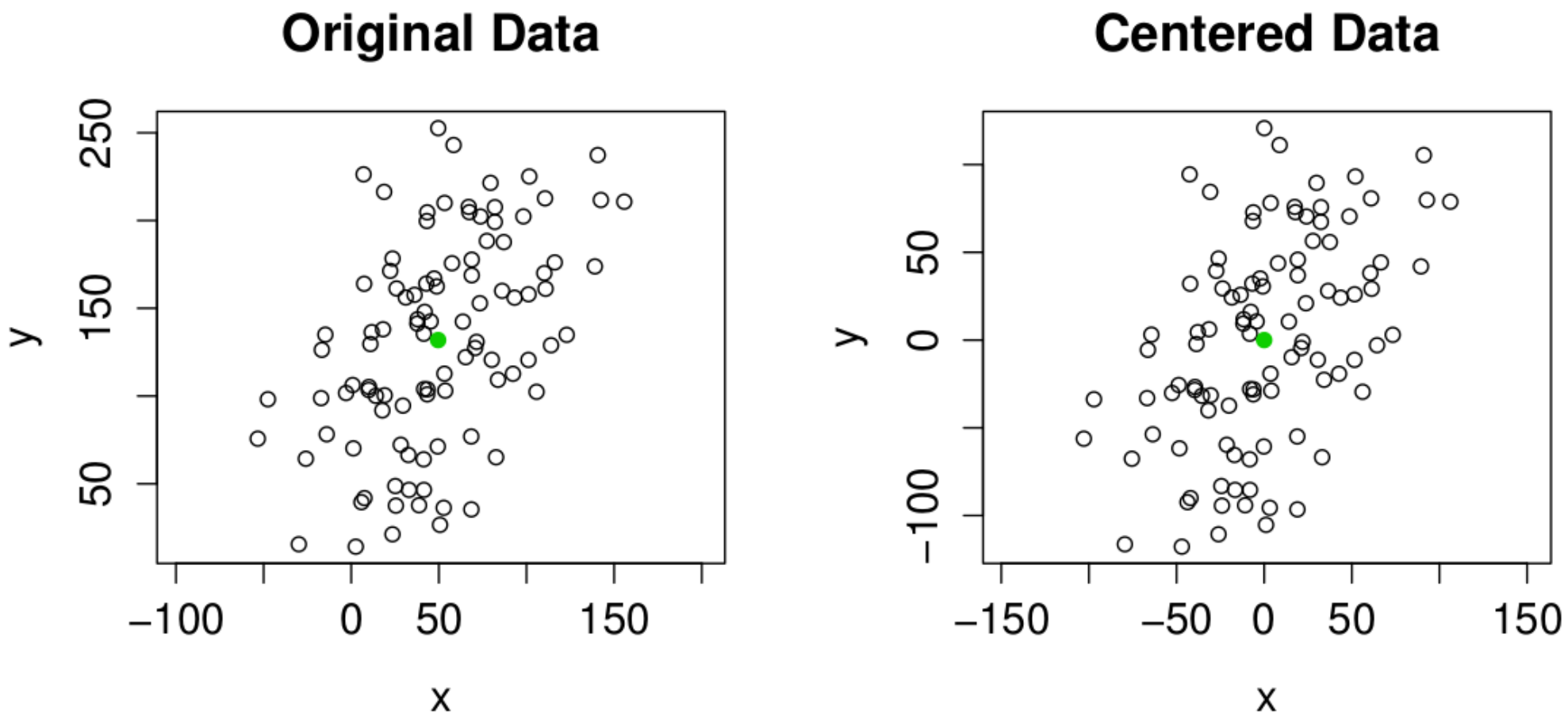
Decomposition

- Let X be our d-dimensional dataset which is centered
- Then we can calculate the co-variance simply by

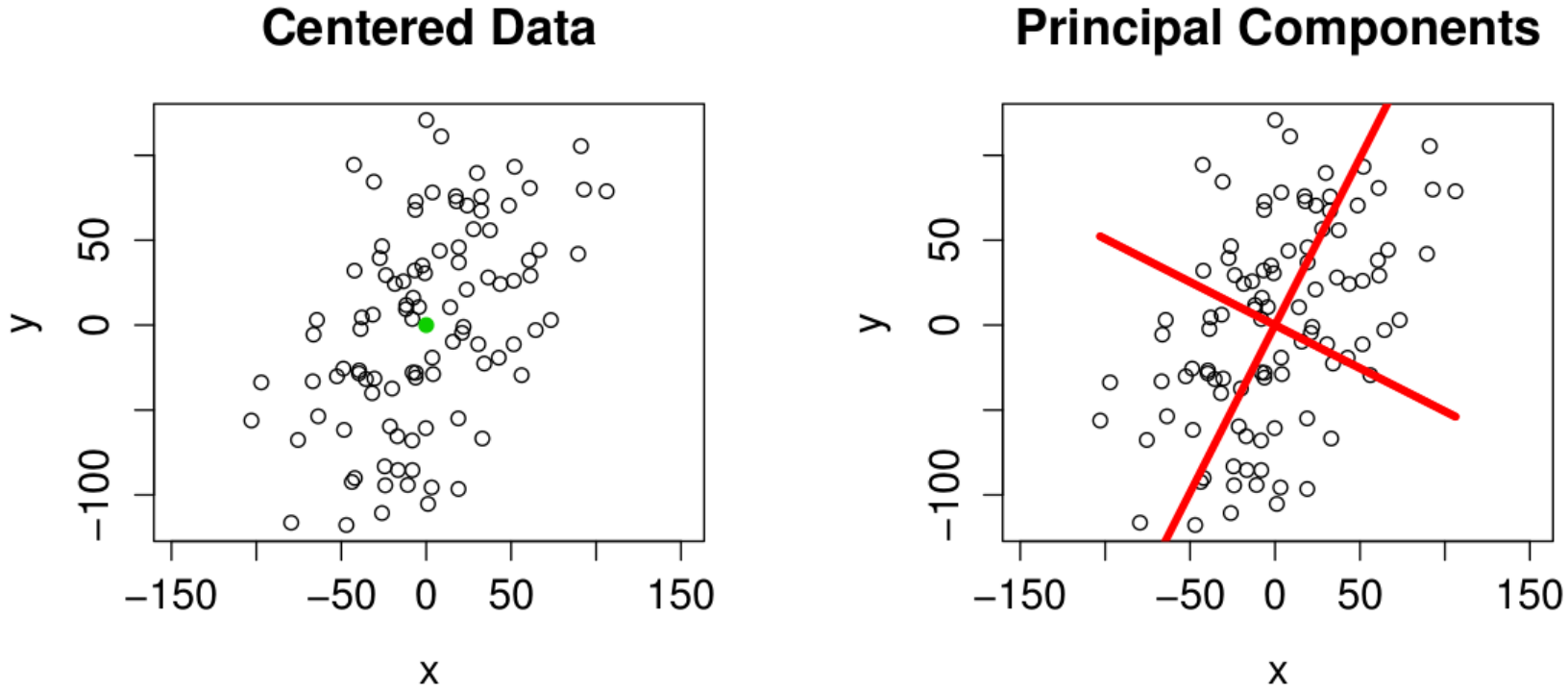
$$Q = \frac{1}{n-1} X X^T$$

- If we now perform an Eigen-Decomposition of Q, we will learn about the spread of the Variance like we did before
- If we order the Eigenvectors according to their Eigenvalue, we will see that along the first eigenvector, the most variance is captured, along the second, the second most, and so forth
- The eigenvectors are called the **principal components**

Step 1: Center Data



Step 2: Eigendecomposition of Q



➢ The Eigenvectors are:

$$e_1 = \begin{pmatrix} 0.45 \\ 0.89 \end{pmatrix} \qquad e_2 = \begin{pmatrix} -0.89 \\ 0.45 \end{pmatrix}$$

The Eigenbasis

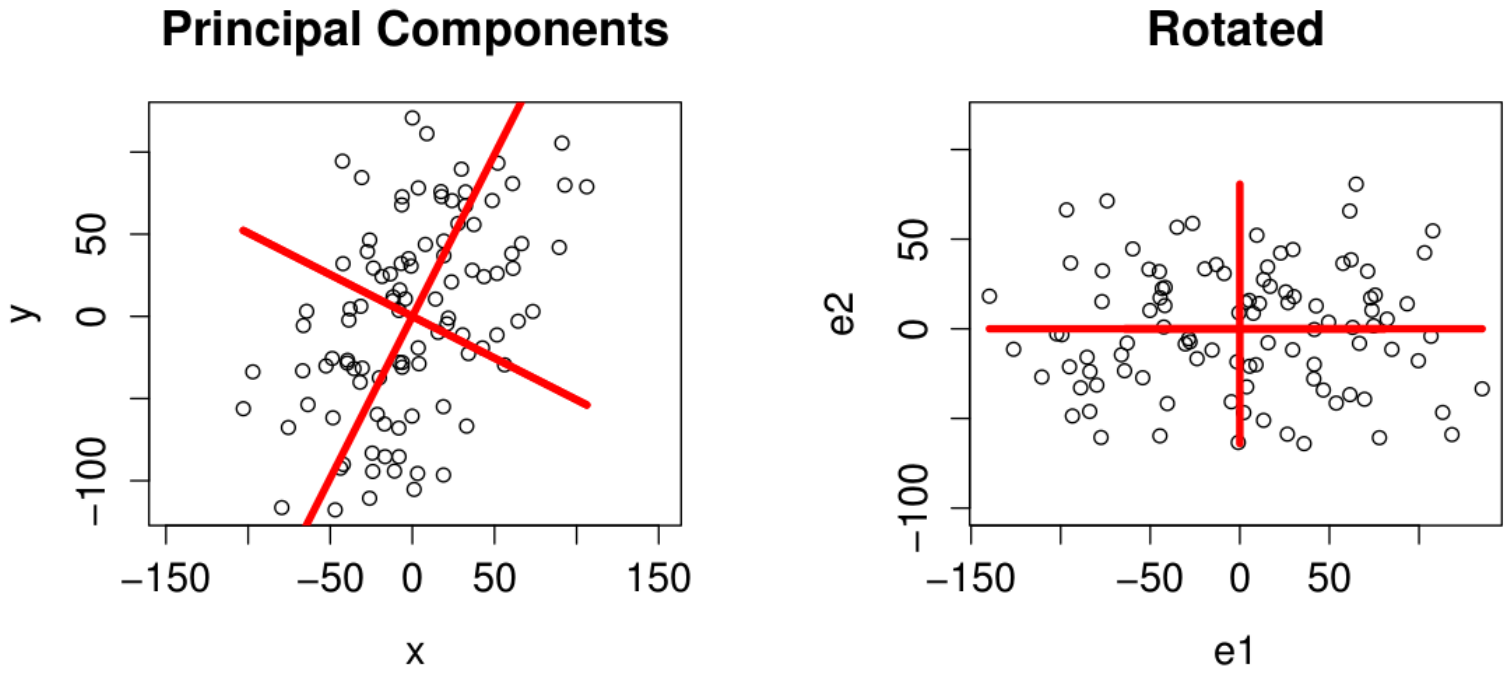
- The eigenvectors form a basis of the space
- **The eigenvectors are normalized**, i.e., $\|e_i\| = 1$

- That means, each vector x_j can be expressed by means of the eigenvectors:

$$x_j = \sum_{i=1}^{d} g_{ij} e_i$$

- The scalars are now the coordinates with respect to the eigenspace

Step 3: Rotate Data



➢ The rotation can be calculated by

$$R = E^T X^T$$

with E being the matrix of the eigenvectors

What is this good for?

- Fantastic, all that stuff in order to rotate some data?
- Expressing X in terms of $e_1, \ldots, e_d$ has not changed the size of the data at all, we just performed a basis transformation

- **BUT**: Hopefully, most of the new coordinates have values close to zero (as there is almost no variance " left" when calculating the PCAs)
- That means in turn, the data lie in a lower-dimensional linear subspace ...
- Thus, we don't use all of the eigenvectors to transform the data
- Which ones should we take?

Lower Dimensional Projection

➢ Let λ_i be the eigenvalue to the eigenvector e_i. Further, the eigenvalues are sorted, s.t.:

$\lambda_1 \geq \lambda_2 \geq \ldots \geq \lambda_d$

➢ Assume $\lambda_i \approx 0 \ \forall \ i > k$, then

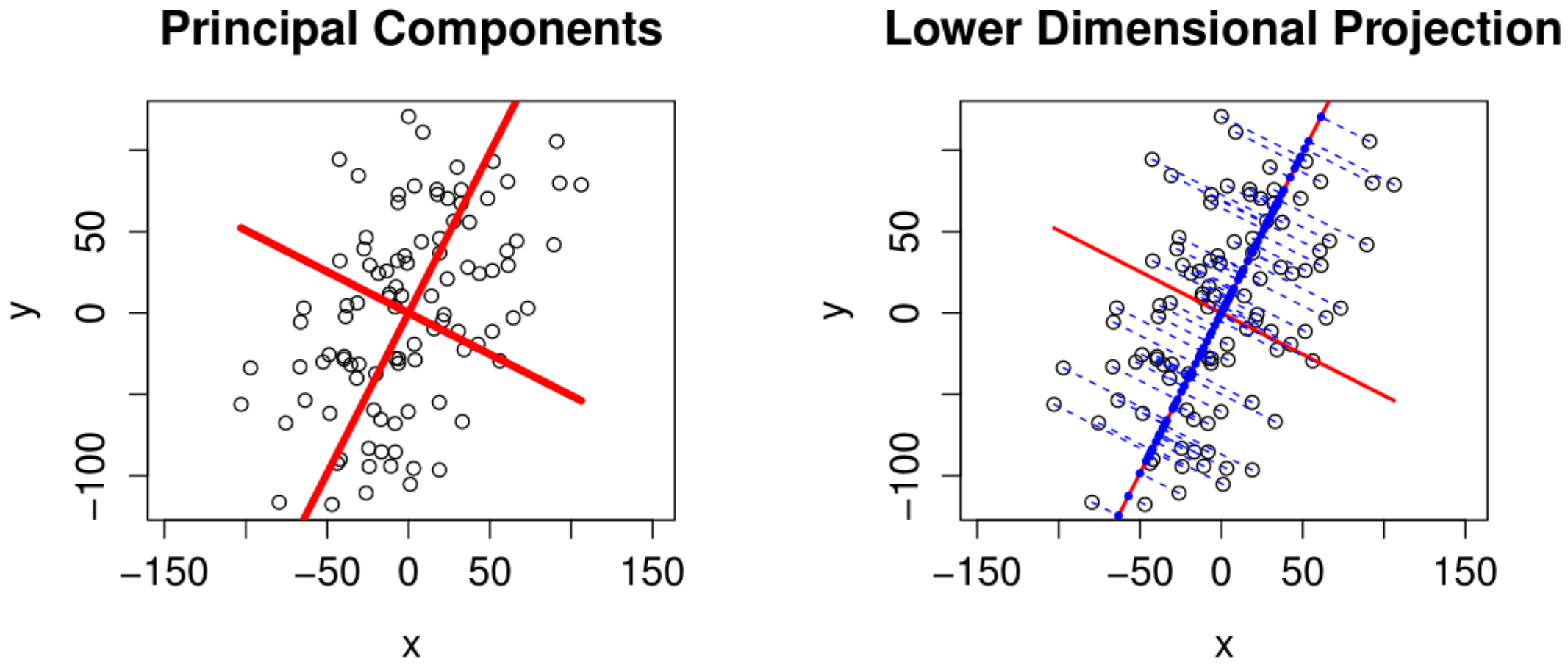
$$x_j \approx \sum_{i=1}^{k} g_{ij} e_i$$

➢ That means, we can represent x_j with $k < d$ components

➢ Variance captured in the lower dimensional eigenspace:

$$\frac{\sum_{i=1}^{k} \lambda_i}{\sum_{i=1}^{d} \lambda_i}$$

Lower Dimensional Projection



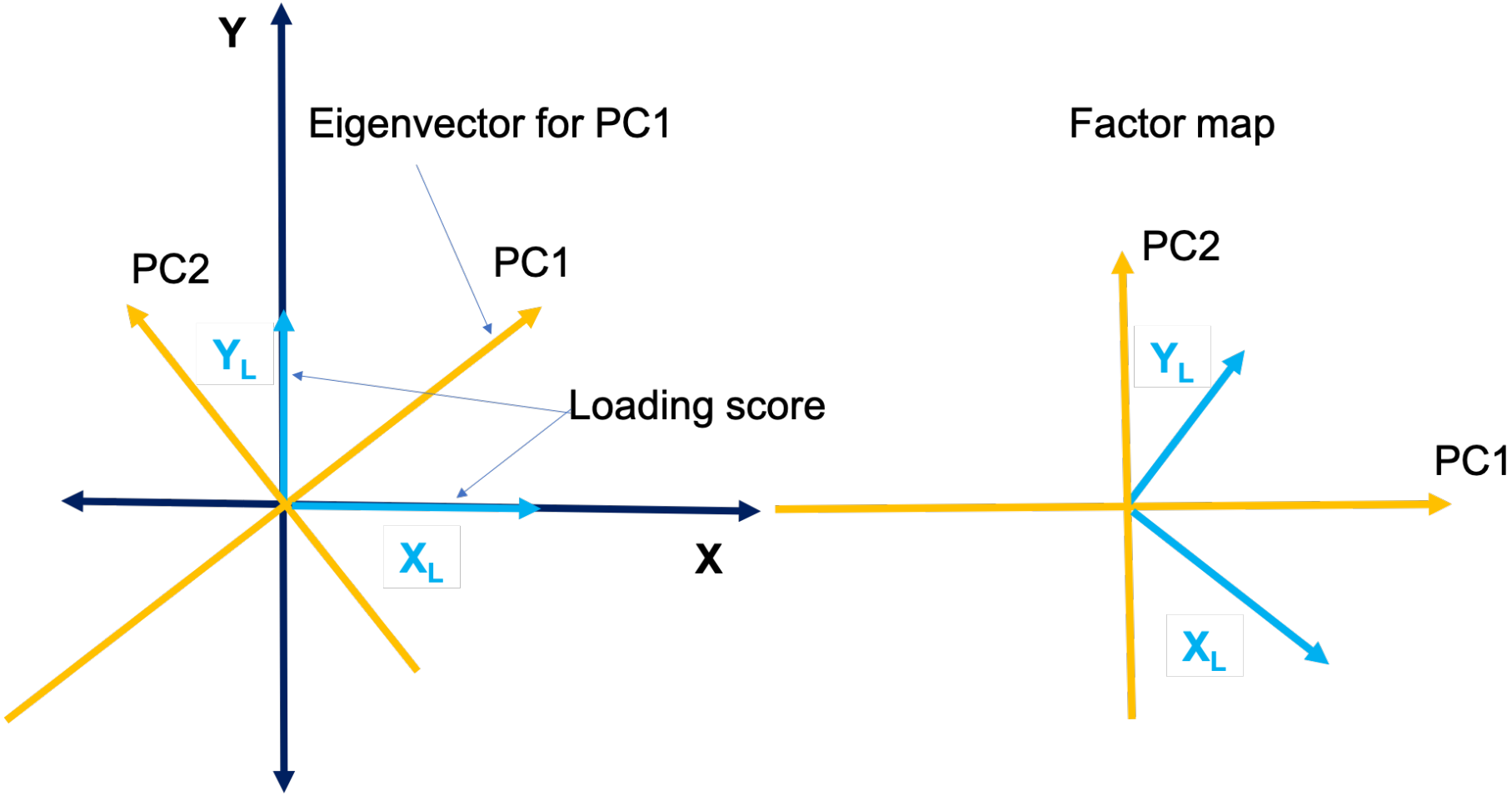
Interpretation of the Eigenvectors

➢ Each eigenvector is a linear combination of the original features $X_1, \ldots, X_d$:

$$e_i = \begin{pmatrix} \phi_{i1} \\ \phi_{i2} \\ \vdots \\ \phi_{id} \end{pmatrix} = \phi_{i1} X_1 + \phi_{i2} X_2 + \ldots + \phi_{id} X_d$$

➢ ϕ_{ij} Are called the loadings of eigenvector e_i

➢ Having a large loading means that the original feature has a great influence on the projection along this eigenvector


Y
Eigenvector for PC1
PC2
PC1
Y_L
Loading score
X_L
X
Factor map
PC2
Y_L
PC1
X_L

Keywords

- Linear algebra
- Probabilities & Standard Distributions
- Eigendecomposition & PCA

Department of Mathematics
and Computer Science

sdu.dk

#sdudk

Advanced Machine Learning

Neural Networks and Gradient-Based Learning

Lukas Galke

Spring 2026

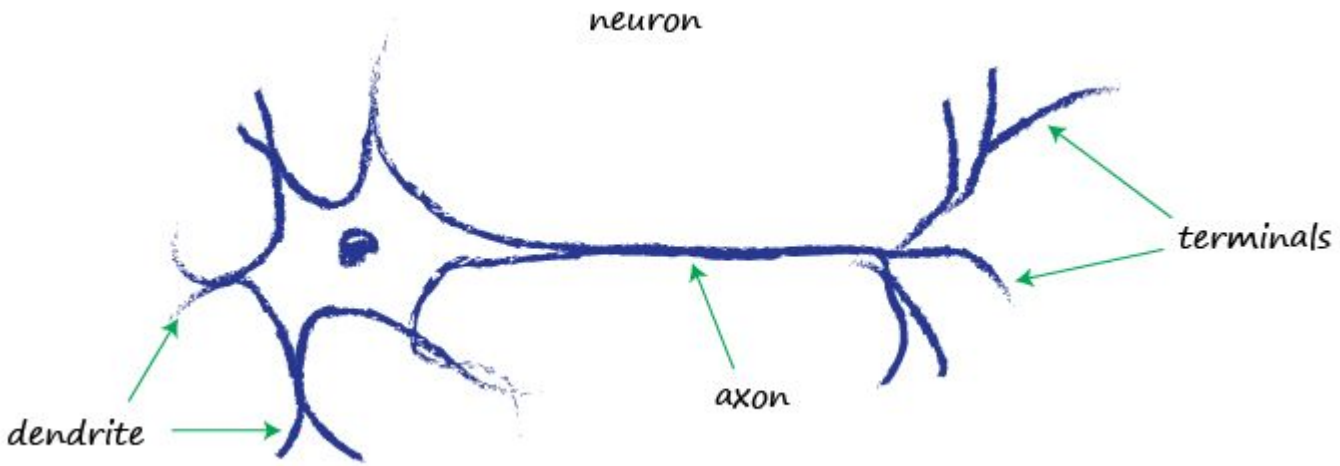
SDU

Slides based on Richard Röttger's material

Keywords for this session

- Neural Networks
- Gradient-based Learning

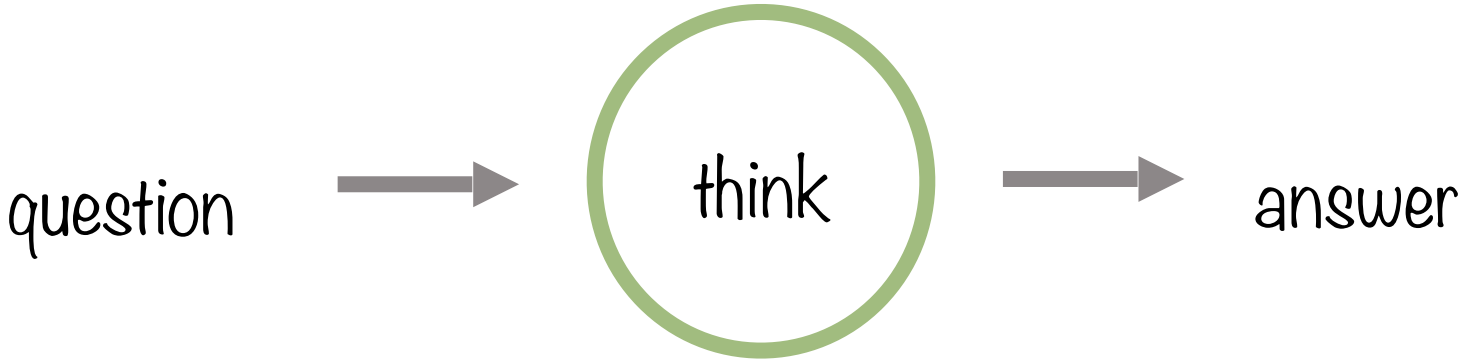
The Neuron … in Nature



That is nice … but

➢We will use Neurons as basic simple "prediction machines"

➢What does that mean?

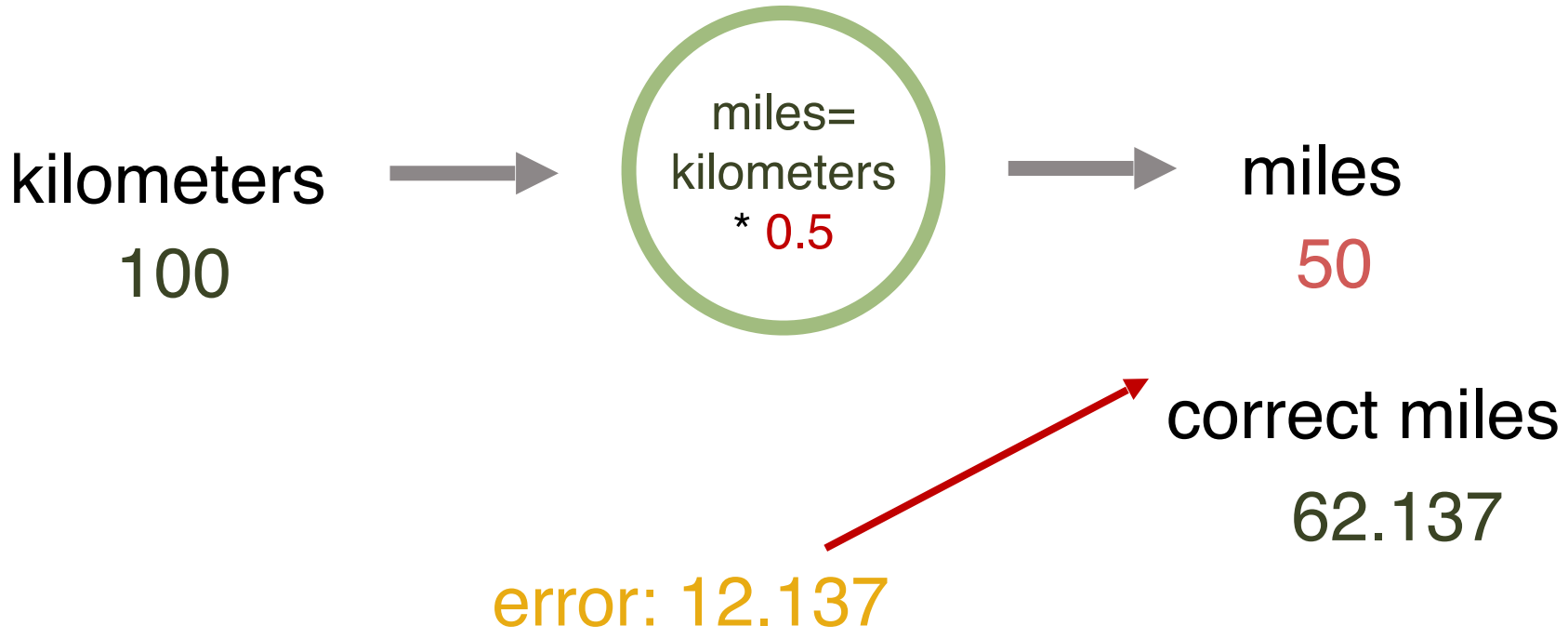


Example

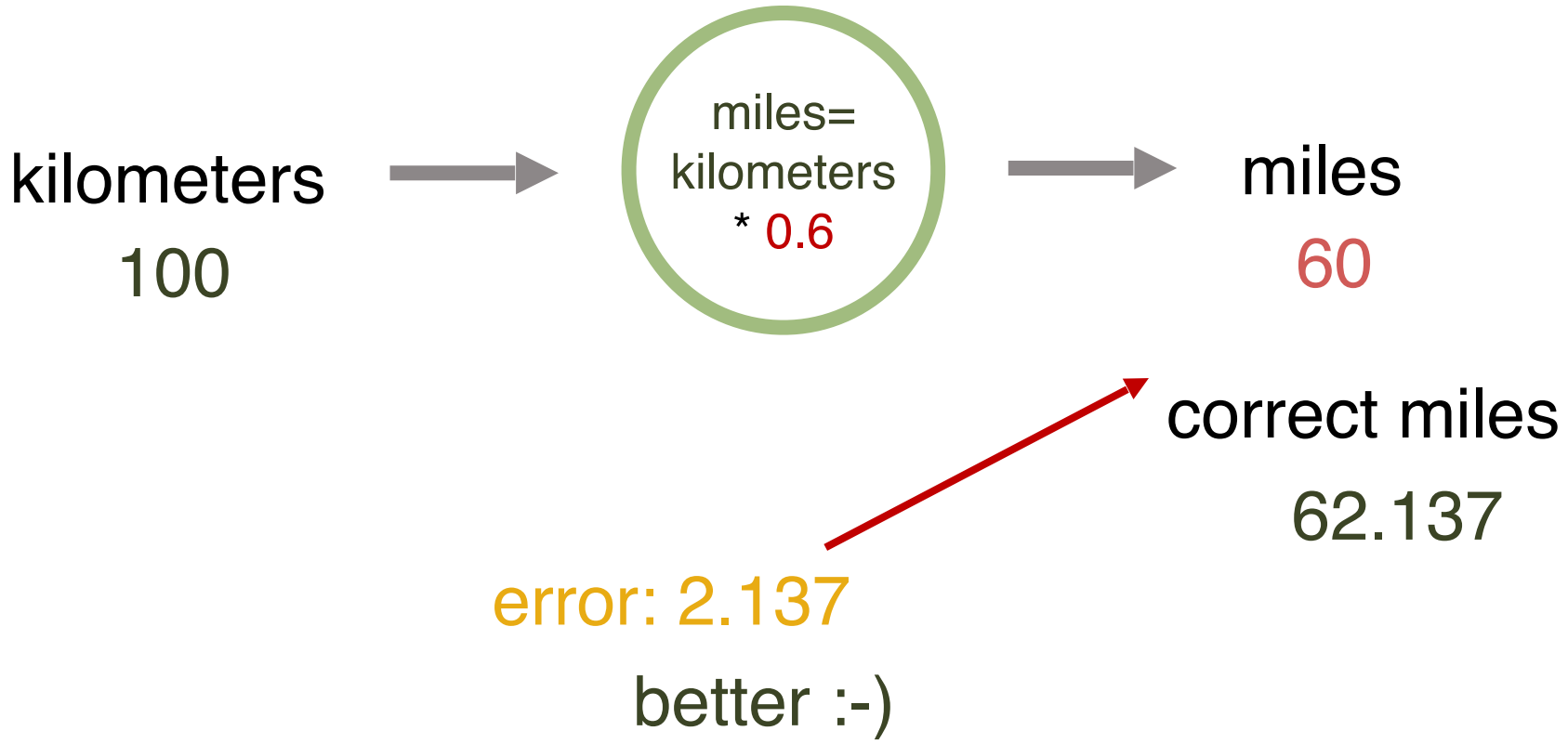
Example

kilometers
100
miles=
kilometers
* 0.5
miles
50
random starting point,
only model Parameter

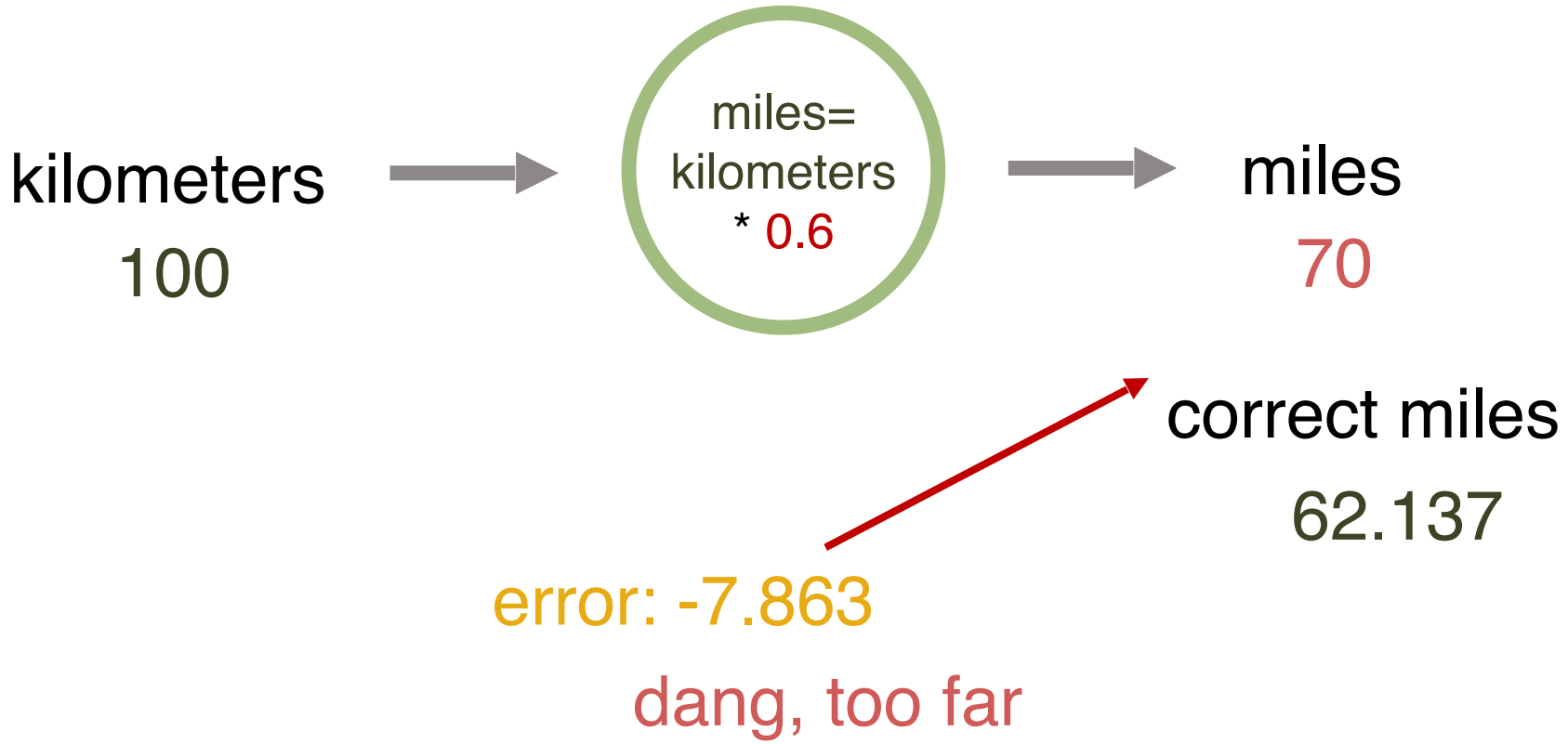
Example


kilometers
100
miles=
kilometers
* 0.5
miles
50
correct miles
62.137
error: 12.137

Example: Learning



Example: Learning



Now with more inputs

- This was extremely simplistic

- Now let's look at a more complicated thing:
  - You are thinking whether you should attend a sports event

- You base the decision on the following factors:
  1. Temperature?
  2. Ticket Price?
  3. Travel time?

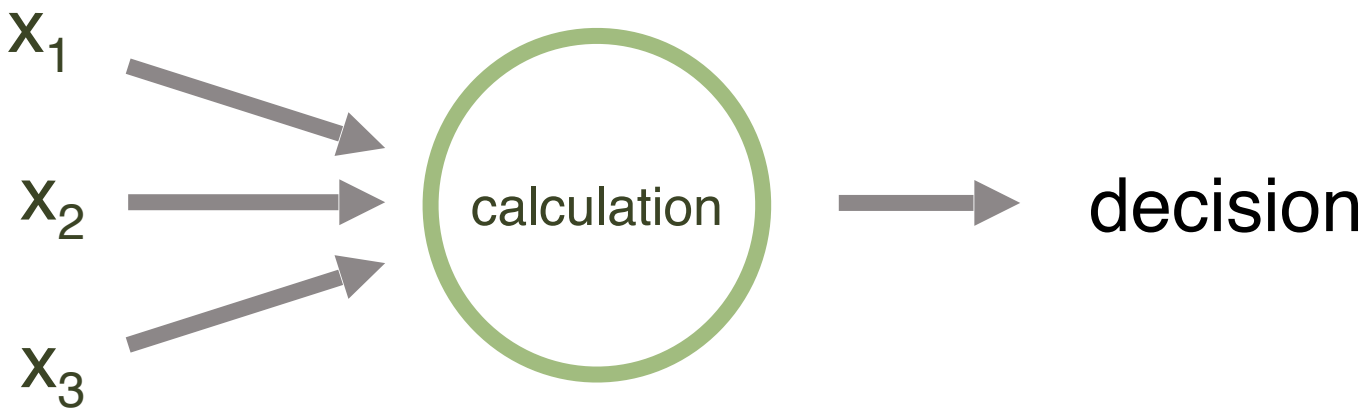
Now with more inputs

- This was extremely simplistic

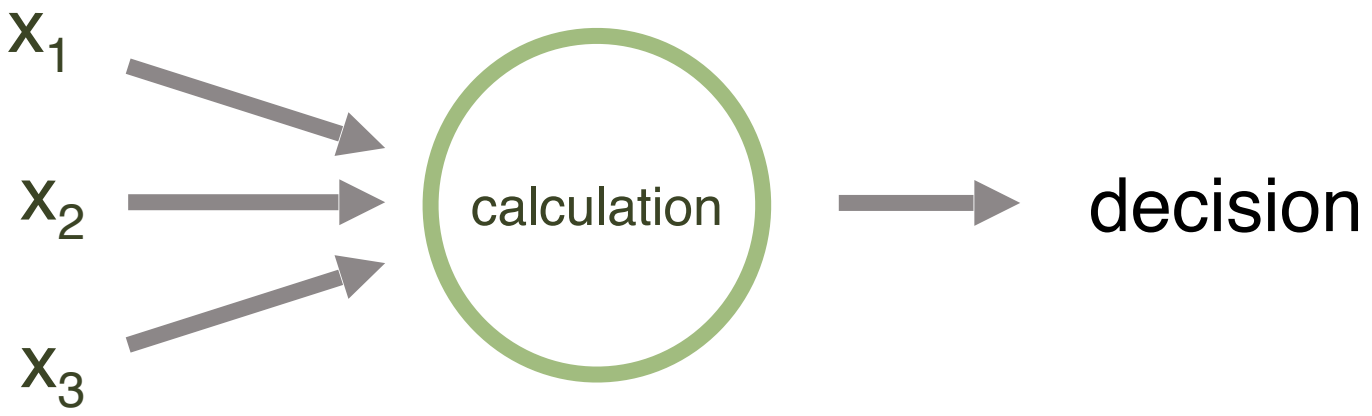
- Now let's look at a more complicated thing:
  - You are thinking whether you should attend a sports event

- You base the decision on the following factors:
  1. Temperature? x_1
  2. Ticket Price? x_2
  3. Travel time? x_3

Now with more inputs



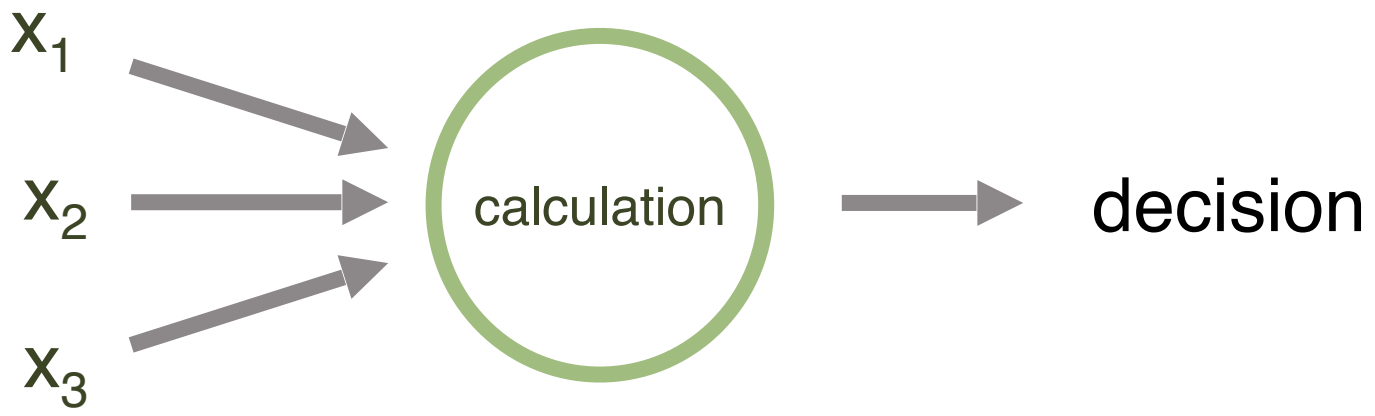
Now with more inputs



➢How to make the decision?

$$x_1 + x_2 + x_3 > \text{threshold?}$$

Now with more inputs

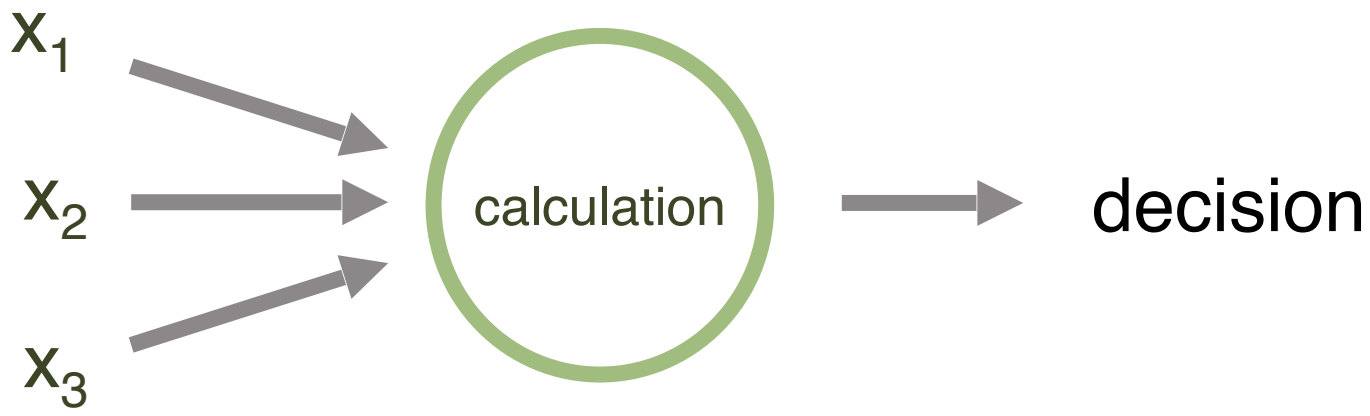


➢How to make the decision?

$$x_1 + x_2 + x_3 > \text{threshold?}$$

➢That would mean, the higher the price and the farther away, the more likely you are to go …

Now with more inputs



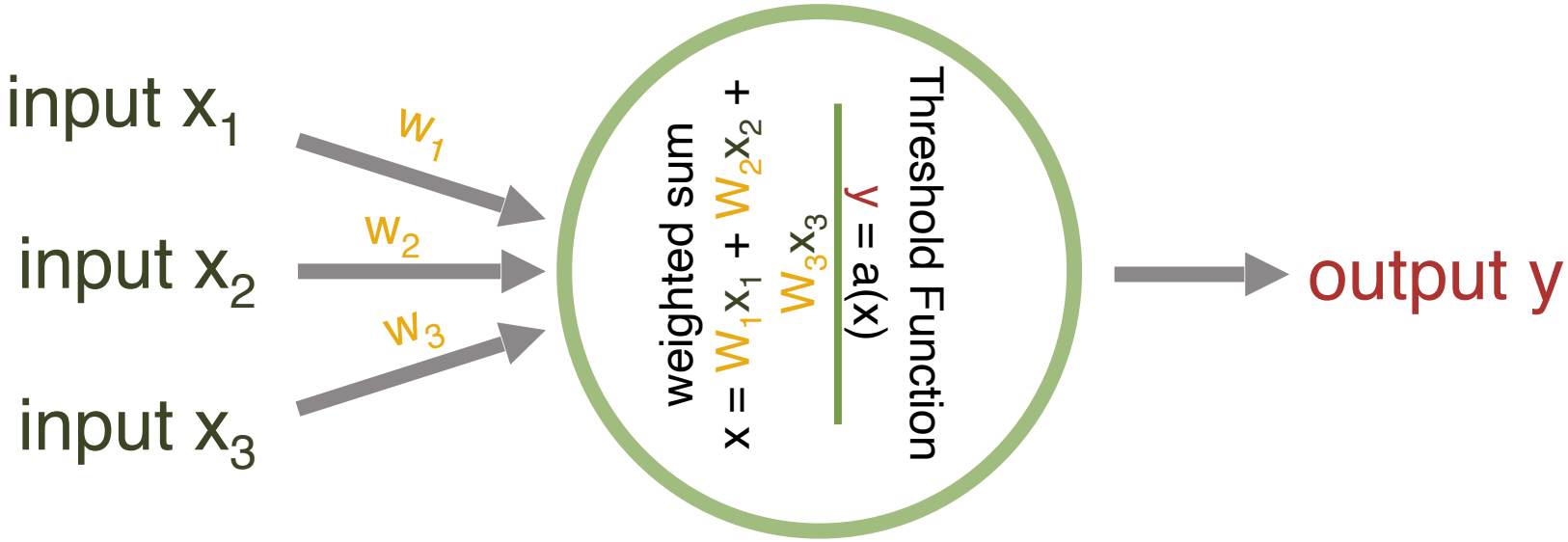
➢How to make the decision?

➢Therefore, introduce weights:

$$W_1x_1 + W_2x_2 + W_3x_3 > \text{threshold?}$$

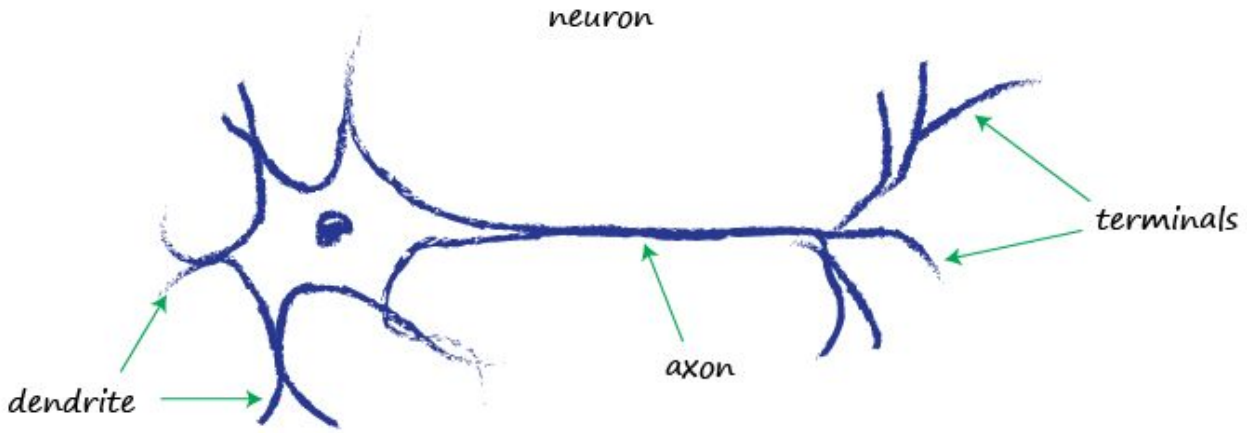
➢Now, you can set the weights and threshold according to your preference, e.g., 1, -4, -1

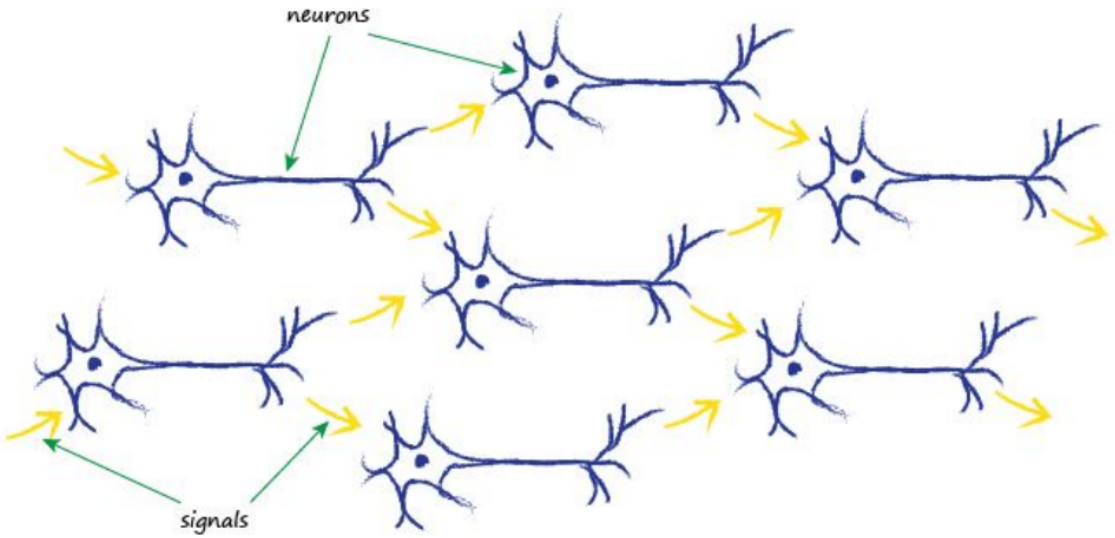
This is in fact how neurons work



- An artificial neuron (aka perceptron) consists of
  - A number of weighted inputs
  - An activation function
  - The generated output

From Neuron to "Brain"

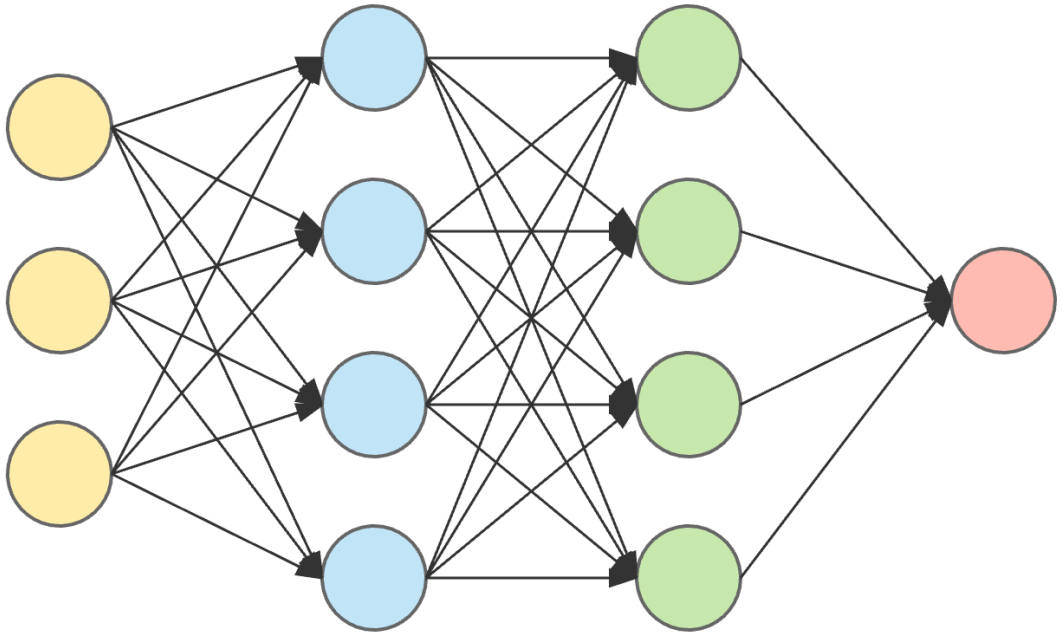




- We now can connect neurons
- The output of one neuron becomes the input of other neurons

In a more structured way

- We normally have more than one node
- Multiple Nodes are arranged in layers
- Each layer receives the generated output from the previous layer



How to build these networks?

- **Define the Architecture**
  - How many inputs?
  - How many hidden layers?
    - How many neurons per layer?
    - What activation function to use?
  - What is the desired output? Define output neurons
    - One-out-of-N classification: Softmax
    - Multiple-out-of-N classification: Sigmoid
    - Regression: linear output
- **Define the loss function**
  - Dependent on the output and task
- **Train the network**

How is a Network Trained?

- **General Procedure:**
  - We present the network with an example where we know the answer
  - We observe the answer of the network and adjust the weights accordingly

- **Ideal world:**
  - We simply look at the dataset and could exactly calculate the weights

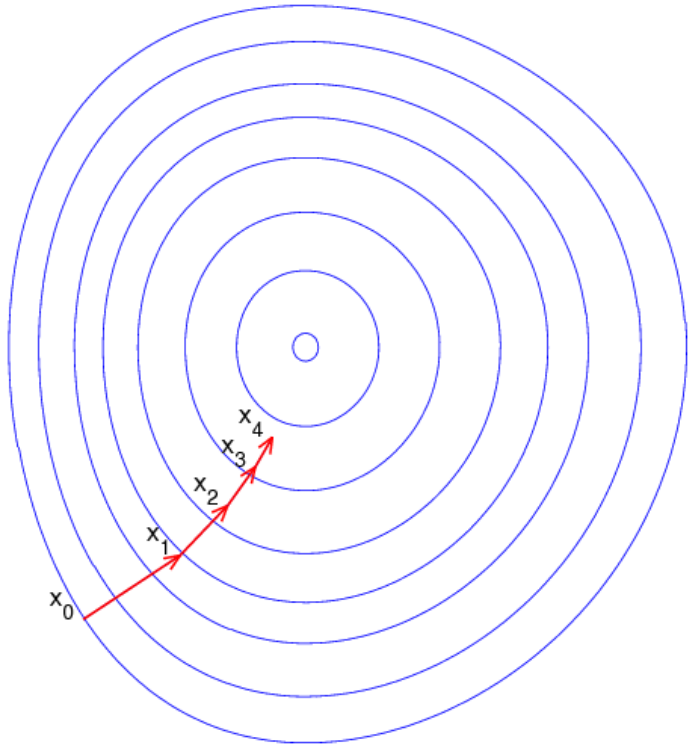
- **Reality:**
  - We define a **cost-function**, the so-called **loss function**
  - We iteratively approximate the best setting by trying to minimize the cost functions
  - We do this by a process called **gradient descent**

Keywords for this session

- Neural Networks
- **Gradient-based Learning**

The Central Idea

➢ Update the model parameters following the steepest slope of the loss function



More Mathematically

- Suppose function $y = f(x)$
- Derivative of function denoted: $f'(x)$ or as dy/dx
  - Derivative $f'(x)$ gives the slope of $f(x)$ at point x
  - It specifies how to scale a small change in input to obtain a corresponding change in the output:

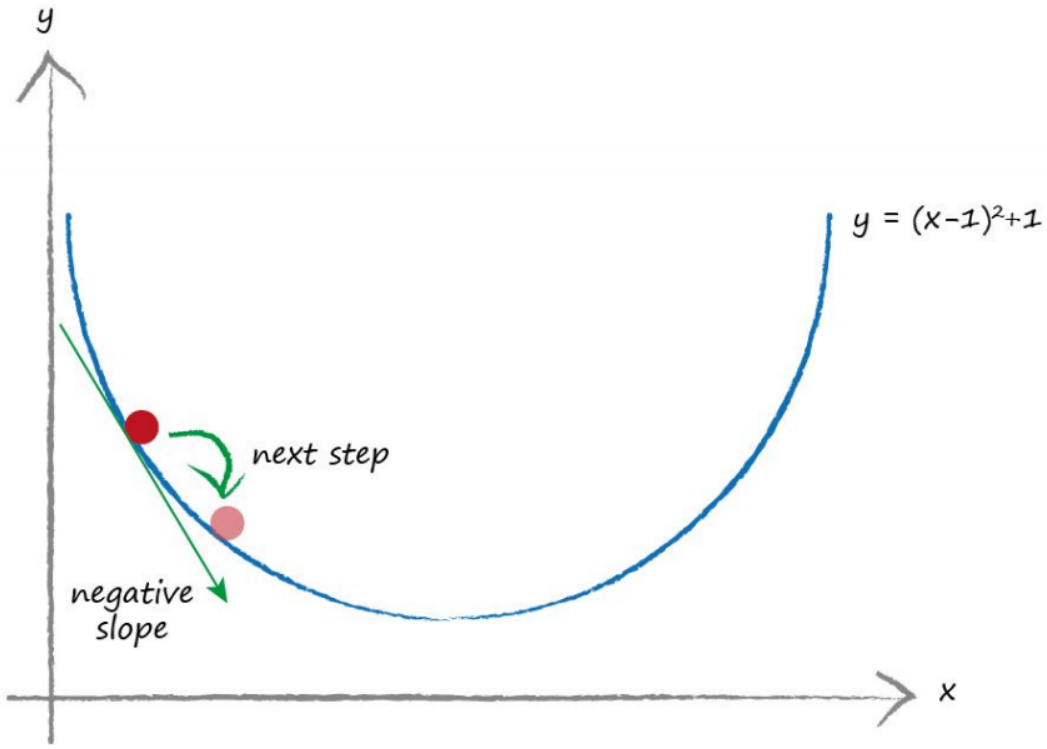
$$f(x + \varepsilon) \approx f(x) + \varepsilon f'(x)$$

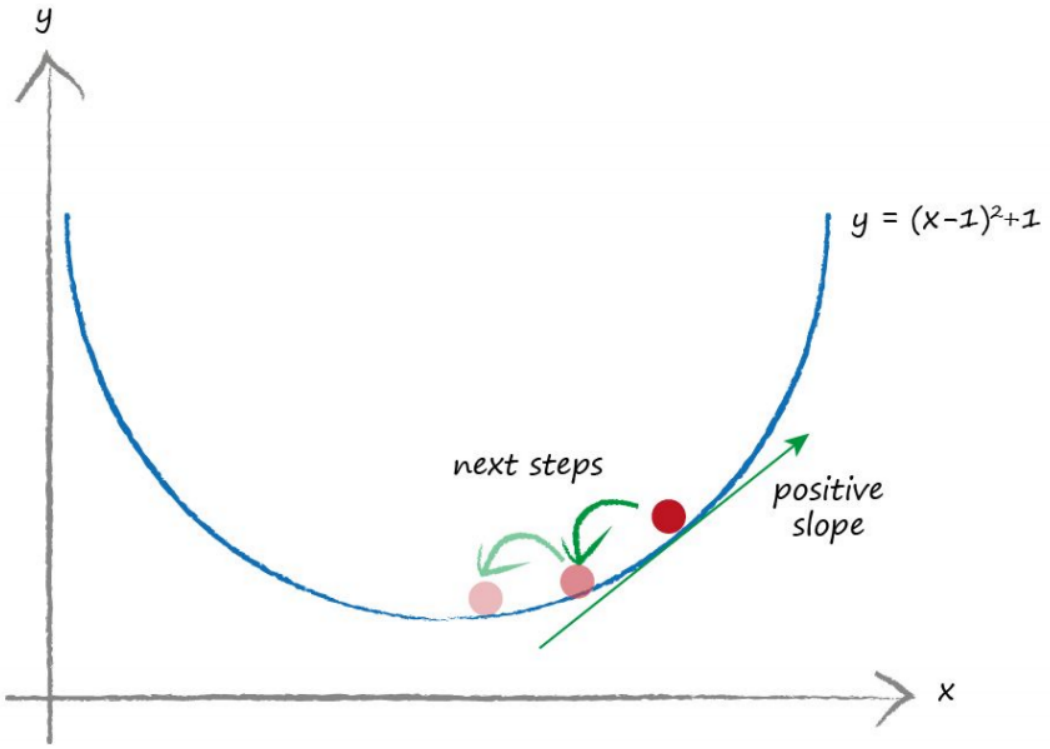
- We know that

$$f(x - \varepsilon \text{sign}(f'(x)))$$

  is less than $f(x)$ for small ε.
  - Thus we can reduce $f(x)$ by moving x in small steps with opposite sign of derivative
  - This technique is called gradient descent (Cauchy 1847)

Gradient Descent





The General Case

- Let us assume we a data dataset $X = \{x_1, \ldots, x_N\}$
- We have defined a neural network with the parameters θ
- We further have defined a cost function J:

$$J(\theta, \boldsymbol{x}, \boldsymbol{y})$$

  - Which give us per sample $\boldsymbol{x}$ and the true label $\boldsymbol{y}$, and the current parameters of the model a certain cost
    - If we, e.g., misclassify x we will get high costs, if we are correct, very low costs

The General Case

➢ For each object $\boldsymbol{x}_1$ we can now calculate the gradient:

$$\boldsymbol{g} = \nabla_{\boldsymbol{\theta}} \boldsymbol{J}\left(\boldsymbol{\theta}, \boldsymbol{x}_1, \boldsymbol{y}_1\right)$$

➢ This gradient tells us how to modify the weights in order to achieve a lesser weight, but is noisy and overly depended on $\boldsymbol{x}_1$

➢ Since we want to improve on the entire dataset, we calculate the overall gradient:

$$\boldsymbol{g} = \frac{1}{N} \sum_{X} \nabla_{\boldsymbol{\theta}} \boldsymbol{J}\left(\boldsymbol{\theta}, \boldsymbol{x}_i, \boldsymbol{y}_i\right) = \nabla_{\boldsymbol{\theta}} \boldsymbol{J}\left(\boldsymbol{\theta}, \boldsymbol{X}, \boldsymbol{y}\right)$$

A bit more formally

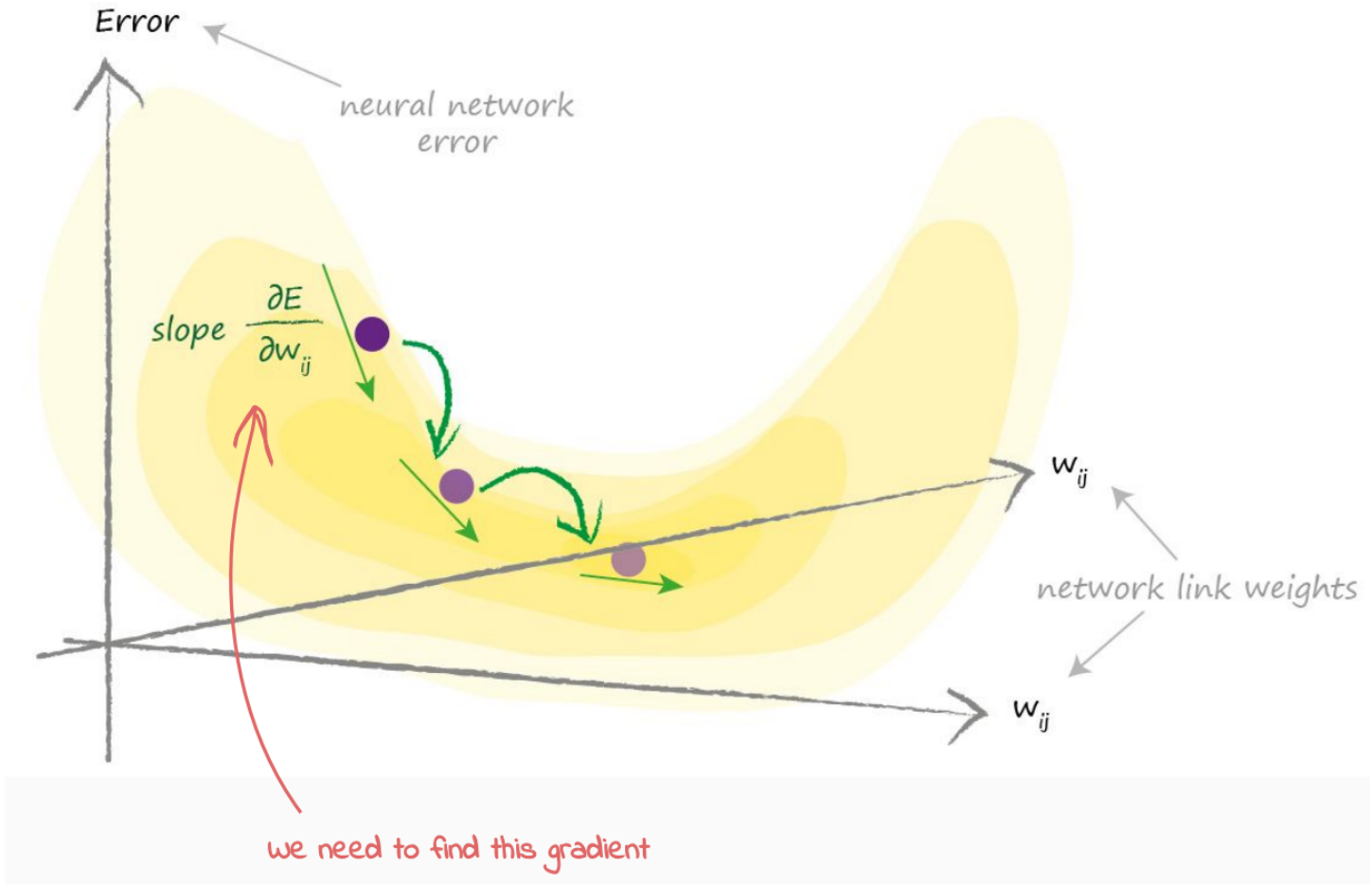
➢We now modify the weights according to the gradient descent method:

$$\boldsymbol{g} = \frac{1}{N} \sum_{X} \nabla_{\theta} \boldsymbol{J}\left(\boldsymbol{\theta}, \boldsymbol{x}_i, \boldsymbol{y}_i\right) = \nabla_{\theta} \boldsymbol{J}\left(\boldsymbol{\theta}, \boldsymbol{X}, \boldsymbol{y}\right)$$

$$\boldsymbol{\theta}_{\text{new}} = \boldsymbol{\theta}_{old} - \epsilon \boldsymbol{g}$$

- ➢ϵ is called the learning rate
- ➢Note, most of the time, we do not compute the gradient for all available data, but for randomly selected small portions of the dataset.
- ➢This is called **Stochastic Gradient Descent**

Gradient Descent



The Loss Function

➢We are often using the loss function called cross-entropy

$$H(p,q) = -\mathbb{E}[\log(q)]$$

➢For most of our (discrete) cases

$$H(p,q) = -\sum_{x \in X} p(x)\log\big(q(x)\big)$$

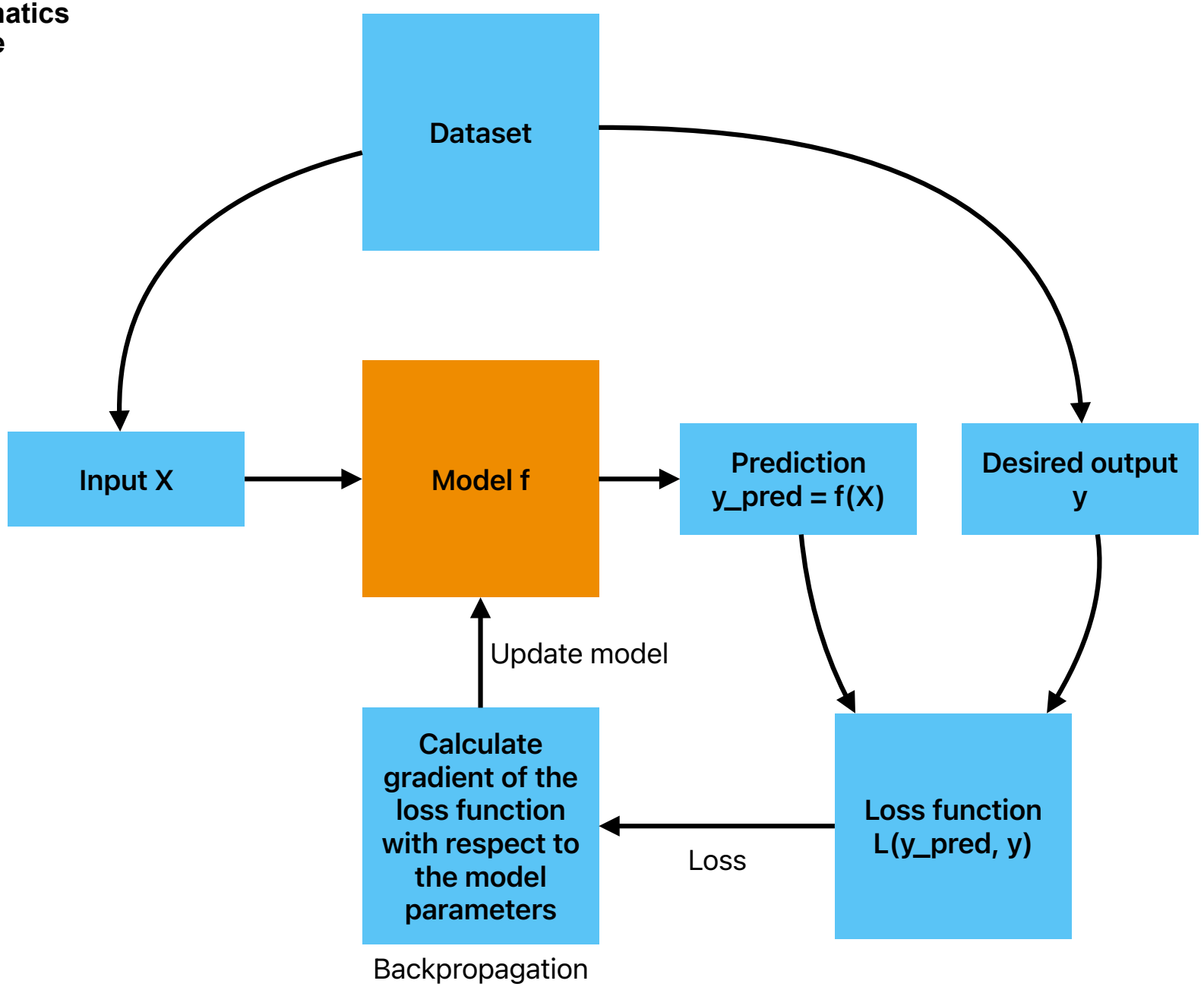
➢**Where $p(x)$ is the real data distribution, while $q(x)$ is our prediction**

➢Interpretation: The average number of bits needed to encode data distributed according to p(x), if you assume it follows q(x).

Keywords for this session

- Neural Networks
- Gradient-based Learning

Outlook


Dataset
Input X
Model f
Prediction
y_pred = f(X)
Desired output
y
Update model
Calculate
gradient of the
loss function
with respect to
the model
parameters
Loss function
L(y_pred, y)
Loss
Backpropagation

How’s my teaching?

https://www.admonymous.co/lukasgalke

Department of Mathematics
and Computer Science

sdu.dk

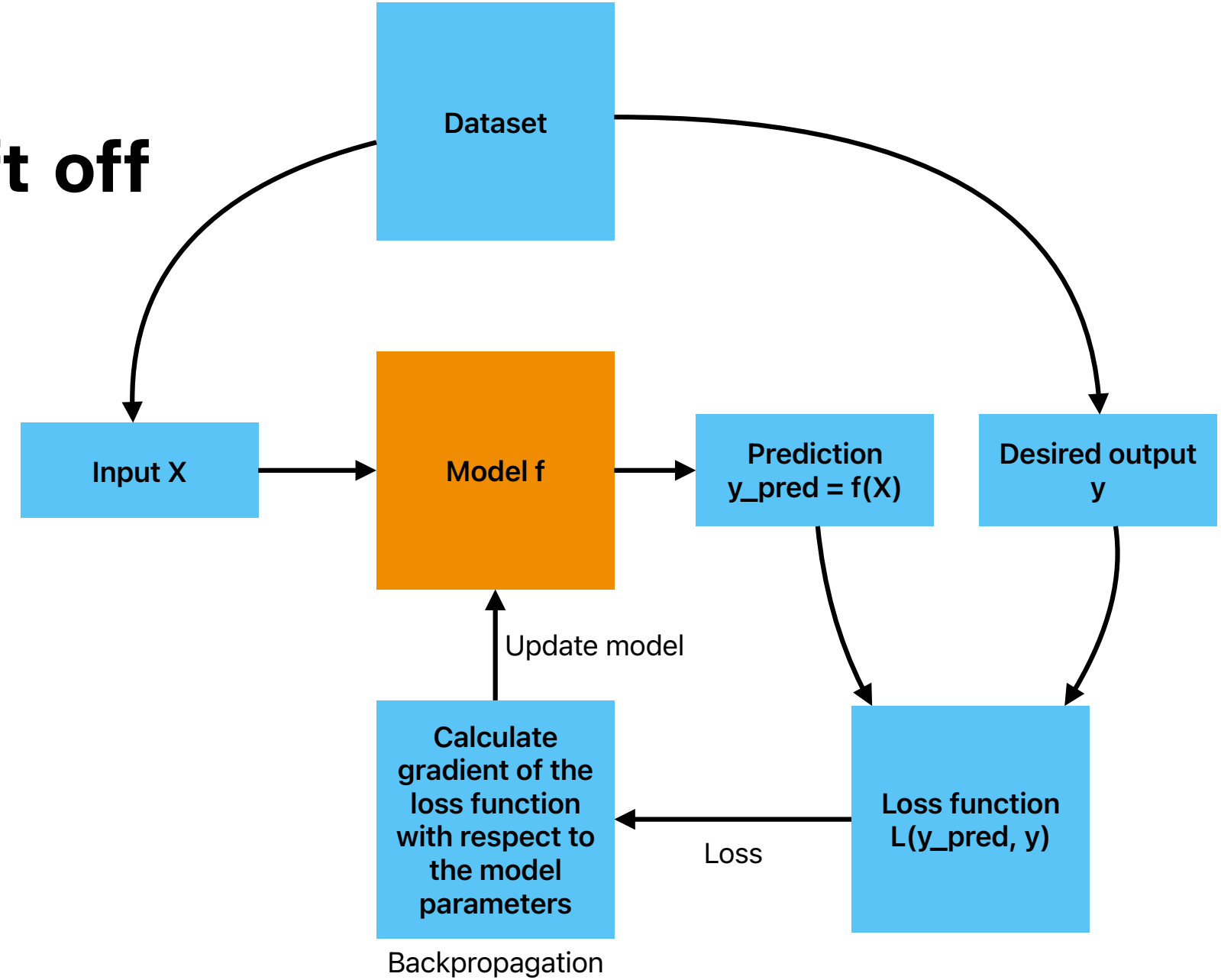
#sdudk

Advanced Machine Learning

Deep Feedforward Networks

Lukas Galke Poech • Spring 2026

Where we left off

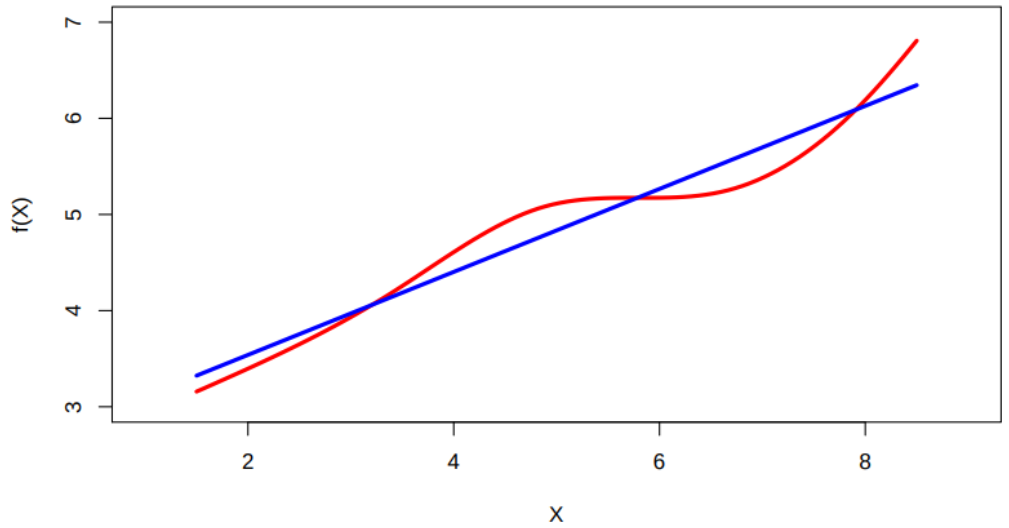


Keywords

- **Single-Layer Networks**
- Deep Neural Networks
- Universal Approximation Theorem
- Activation functions
- Output units

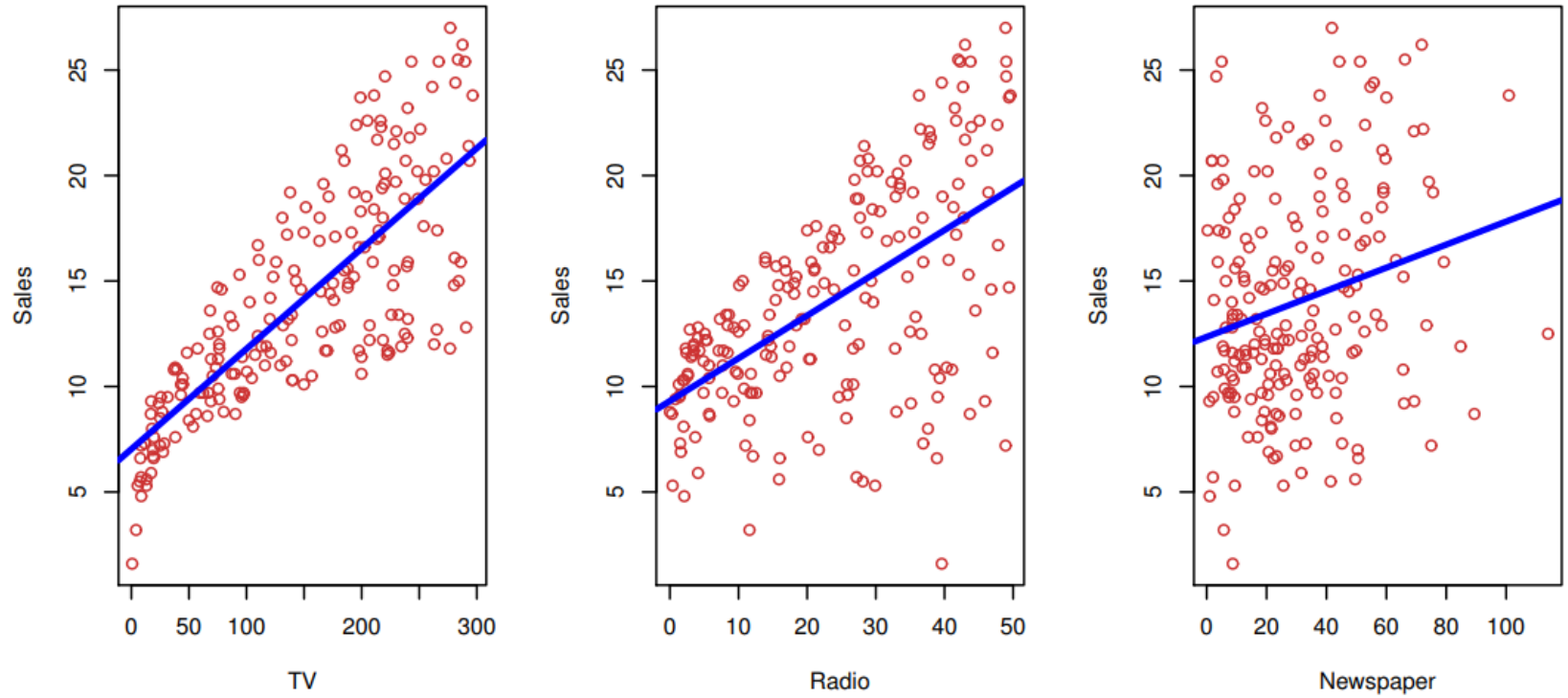
Linear Regression

- Linear regression is a simple approach to supervised learning. It assumes that the dependence of Y on $X_1, X_2, \ldots X_p$ is linear.
- True regression functions are never linear!



- Although it may seem overly simplistic, linear regression is extremely useful both conceptually and practically.

Example: Advertising data



- The Advertising data set consists of the sales of that product in 200 different markets, along with advertising budgets for the product in each of those markets for three different media: TV, radio, and newspaper.

Example: Advertising data

Questions we might ask:

- Is there a relationship between advertising budget and sales?
- How strong is the relationship between advertising budget and sales?
- Which media contribute to sales?
- How accurately can we predict future sales?
- Is the relationship linear?
- Is there synergy among the advertising media?

Simple Linear Regression.

We assume a model

$$Y = \beta_0 + \beta_1 X + \epsilon$$

where β_0 and β_1 are two unknown constants that represent the intercept and slope, also known as coefficients or parameters, and ϵ is the error term.

Given some estimates $\hat{\beta}_0$ and $\hat{\beta}_1$ model coefficients, we **predict** future sales using

$$\hat{y} = \hat{\beta}_0 + \hat{\beta}_1 x$$

Estimation of the Parameters by Least Squares

➢ Let $\hat{y}_i = \hat{\beta}_0 + \hat{\beta}_1 x_i$ be the prediction of Y for the ith observation.

➢ Then the **residual** is defined as

$$e_i = y_i - \hat{y}_i$$

➢ We define the **residual sum of squares** as

$$\text{RSS} = e_1^2 + e_2^2 + \ldots + e_n^2 =$$
$$\left(y_1 - \hat{\beta}_0 - \hat{\beta}_1 x_1\right)^2 + \left(y_2 - \hat{\beta}_0 - \hat{\beta}_1 x_2\right)^2 + \ldots + \left(y_n - \hat{\beta}_0 - \hat{\beta}_1 x_n\right)^2$$

➢ Our task is to find the $\hat{\beta}_0$ and $\hat{\beta}_1$ minimizing the RSS.

Estimation of the Parameters by Least Squares

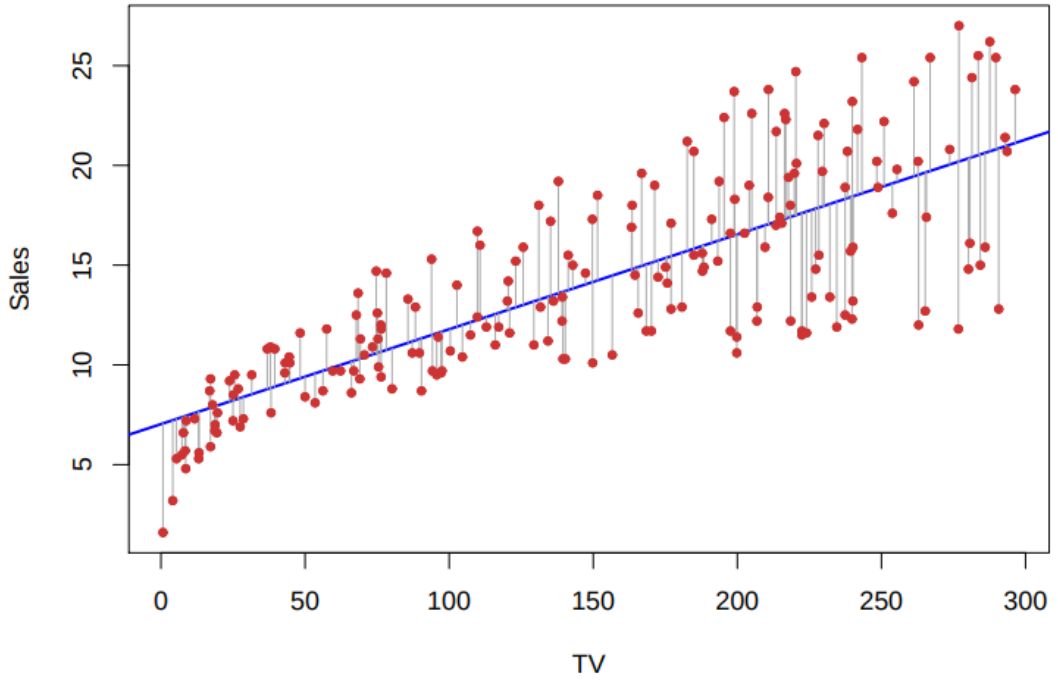
➢ With simple derivation of the RSS formula, we can show that

$$\hat{\beta}_1 = \frac{\sum_{i=1}^{n} \left(x_i - \bar{x}\right)\left(y_i - \bar{y}\right)}{\sum_{i=1}^{n} \left(x_i - \bar{x}\right)^2}$$

$$\hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}$$

➢ with the sample means $\bar{y} = \frac{1}{\mathrm{n}} \sum_{i=1}^{n} y_i$ and $\bar{x} = \frac{1}{\mathrm{n}} \sum_{i=1}^{n} x_i$

Example: Advertising data



- The least squares fit for the regression of sales onto TV.
- In this case a linear fit captures the essence of the relationship, although it is somewhat deficient in the left of the plot.

Multiple Linear Regression

➢ We can include several features or predictors into our model

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \ldots + + \beta_p X_p + \epsilon$$

➢ We interpret β_j as the average effect on Y of a one unit increase in X_j, holding all other predictors fixed. In the advertising example, the model becomes

$$\text{sales} = \beta_0 + \beta_1 \cdot \text{TV} + \beta_2 \cdot \text{radio} + \beta_3 \cdot \text{newspaper} + \epsilon$$

In Matrix Form

We can formulate the entire linear regression also in Matrixform

$\boldsymbol{y} = \boldsymbol{X}\boldsymbol{b} + \boldsymbol{e}$

$\boldsymbol{y} = (y_1, \ldots, y_n) \in \mathbb{R}^n$ is the $n \times 1$ response vector

$\boldsymbol{X} = [1_n, \boldsymbol{X}'] \in \mathbb{R}^{n \times (p+1)}$ is the $n \times (p+1)$ design matrix

1_n is an $n \times 1$ vector of ones

$\boldsymbol{X}' = \left[\boldsymbol{X_1}, \boldsymbol{X_2}, \ldots, \boldsymbol{X_p}\right] \in \mathbb{R}^{n \times p}$ the $n \times p$ predictor Matrix

$b = \left(\beta_0, \beta_1, \ldots, \beta_p\right) \in \mathbb{R}^{p+1}$ is regression coefficient vector

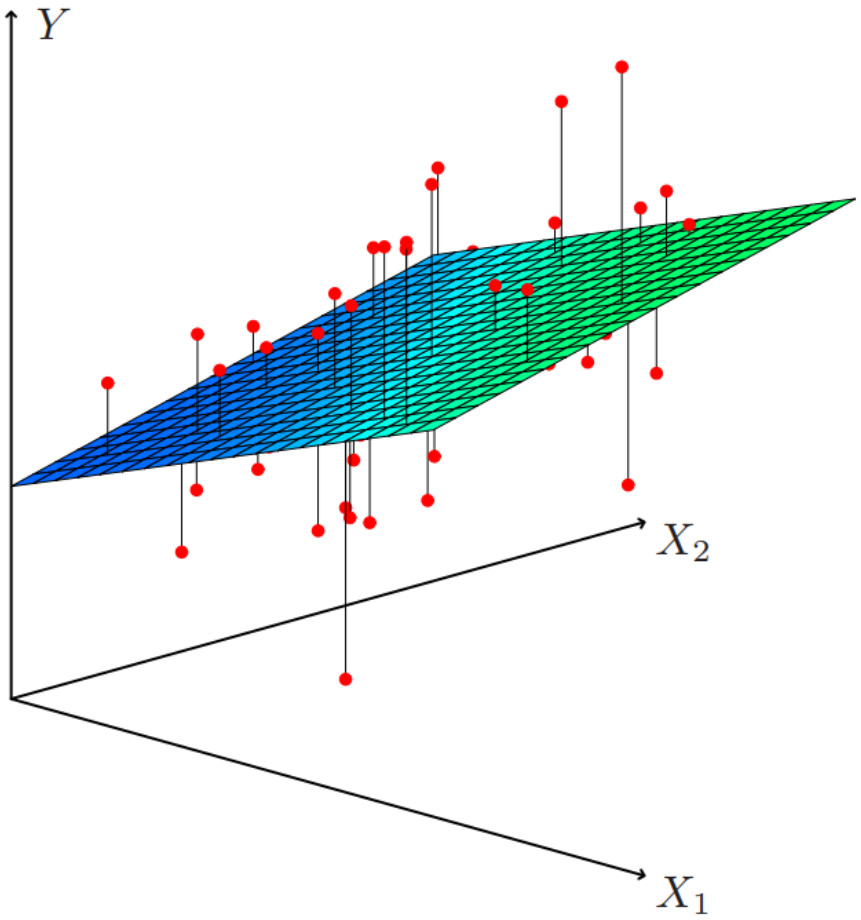
$e = (e_1, e_2, \ldots, e_3) \in \mathbb{R}^n$ is the $n \times 1$ error vector

Example

#	TV	Radio	News	Sales
1	230.1	37.8	69.2	**22.1**
2	44.5	39.3	45.1	**10.4**
3	17.2	45.9	69.3	**9.3**
4	151.5	41.3	58.5	**18.5**
5	180.8	10.8	58.4	**12.9**

$$\begin{pmatrix} 22.1 \\ 10.4 \\ 9.3 \\ 18.5 \\ 12.9 \end{pmatrix} = \begin{pmatrix} 1 & 230.1 & 37.8 & 69.2 \\ 1 & 44.5 & 39.3 & 45.1 \\ 1 & 17.2 & 45.9 & 69.3 \\ 1 & 151.5 & 41.3 & 58.5 \\ 1 & 180.8 & 10.8 & 58.4 \end{pmatrix} \begin{pmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \\ \beta_3 \end{pmatrix} + \begin{pmatrix} e_1 \\ e_2 \\ e_3 \\ e_4 \\ e_5 \end{pmatrix}$$

Linear Regression



Generalizations

There exist many different extensions and generalizations to the simple linear regression model:

- **Classification problems**: logistic regression, support vector machines
- **Non-linearity**: kernel smoothing, splines and generalized additive models; nearest neighbor methods.
- **Interactions**: Tree-based methods, bagging, random forests and boosting (these also capture non-linearities)
- **Regularized fitting**: Ridge regression and lasso

Logistic Regression

One type of generalized linear model

- **Logistic regression** models the log-odds of an event as a a linear combination of input variables
- How to get there? Apply sigmoid to the output to ensure it is between 0 and 1, estimating the probability of an event (rather than a quantity)

Sigmoid $\sigma(x) = \frac{1}{1+e^{-x}} = \frac{e^x}{1+e^x}$

Regression $p(X) = \beta_0 + \beta_1 X.$

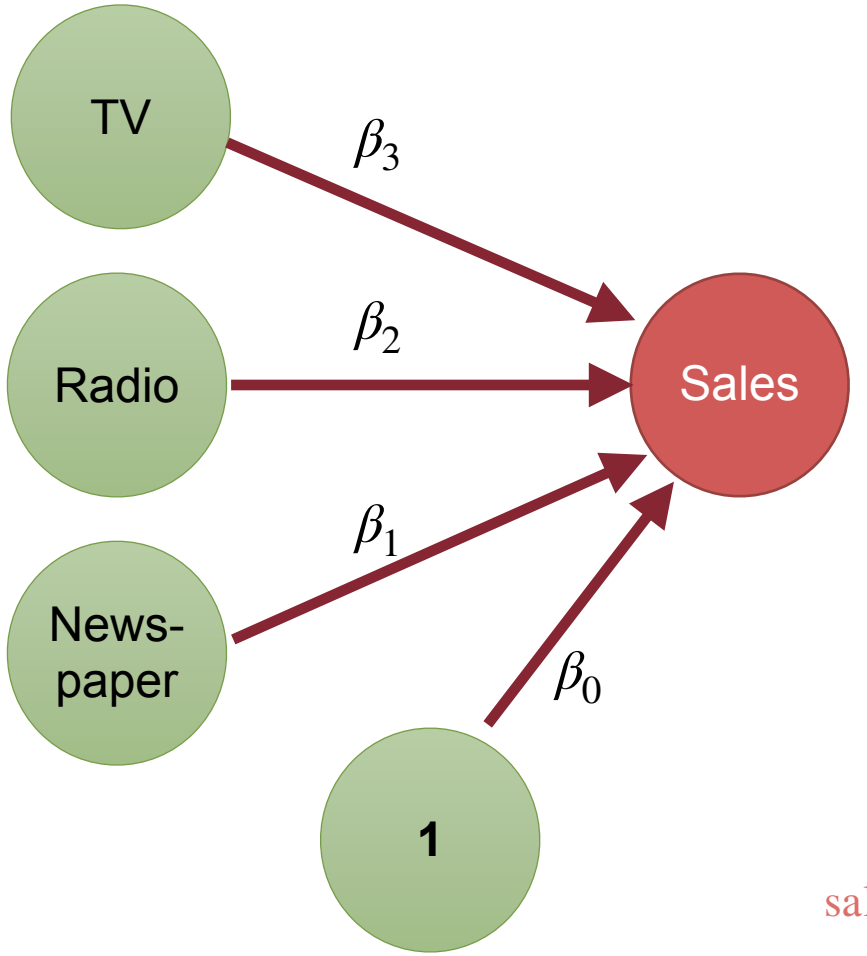
Apply sigmoid to RHS $p(X) = \frac{e^{\beta_0+\beta_1 X}}{1+e^{\beta_0+\beta_1 X}}.$

Rewrite to "Odds" $\frac{p(X)}{1-p(X)} = e^{\beta_0+\beta_1 X}.$

Log odds or "Logits" $\log\left(\frac{p(X)}{1-p(X)}\right) = \beta_0 + \beta_1 X.$

This terminology has been recycled for the raw outputs of a neural network

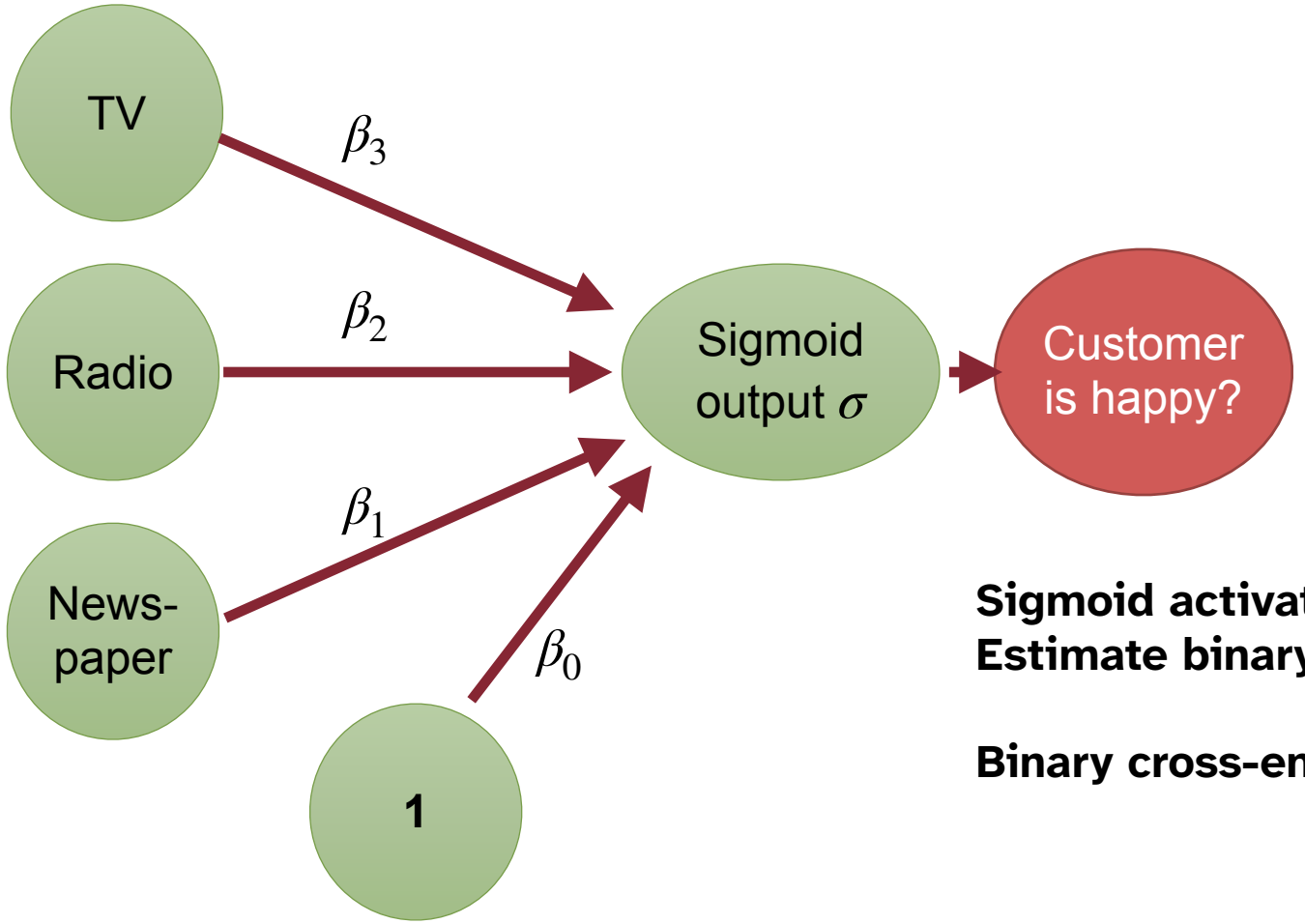
Single-layer neural network for linear regression



Mean-squared error loss

$$\text{sales} = \beta_0 + \beta_1 \cdot \text{TV} + \beta_2 \cdot \text{radio} + \beta_3 \cdot \text{newspaper} + \epsilon$$

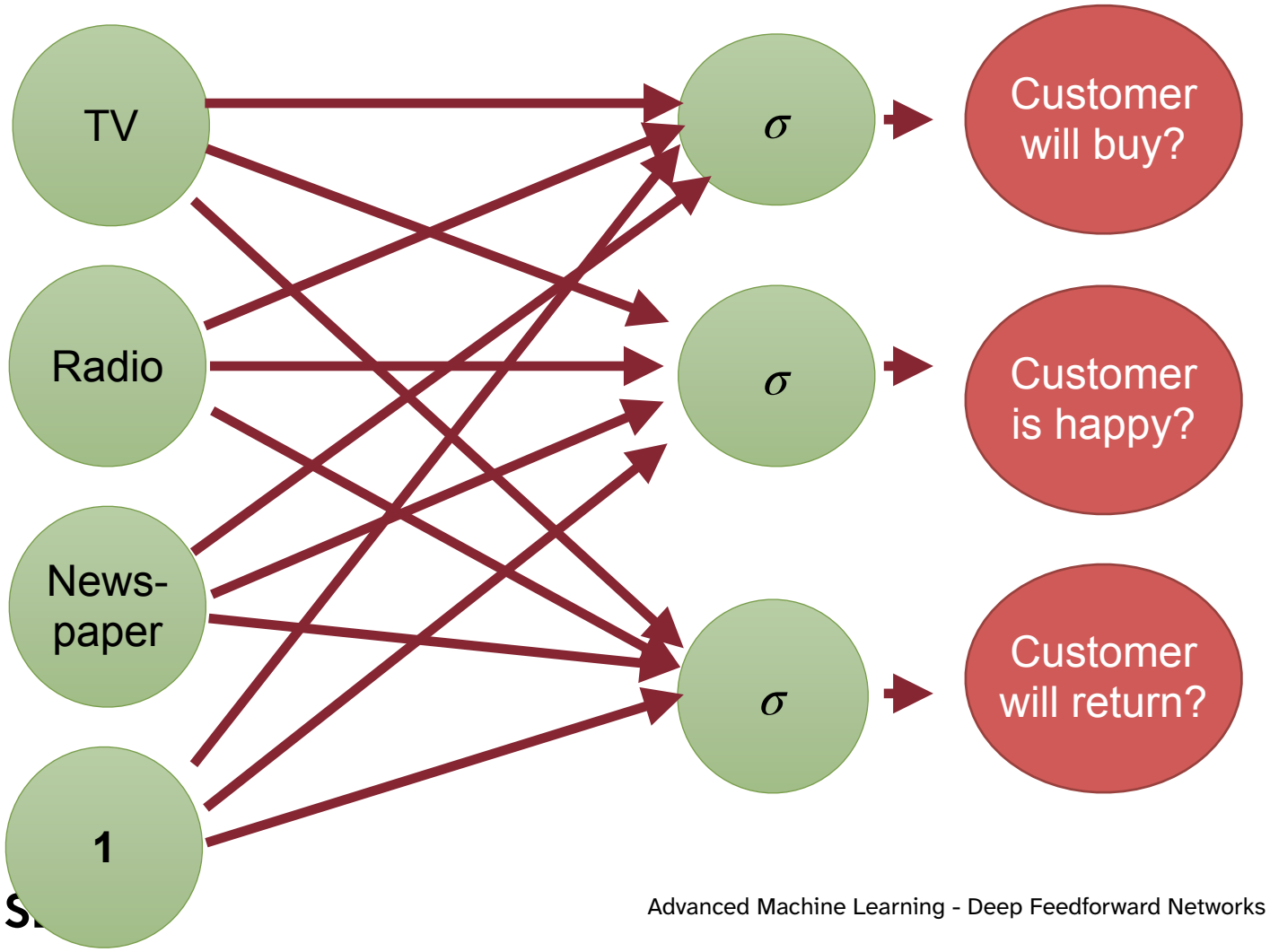
Single-layer neural network for logistic regression



Sigmoid activation,
Estimate binary probability

Binary cross-entropy loss

Single-layer NN doing multiple logistic regressions



Here, all the logistic regressions are independent from each other

Multi-layered networks can find shared factors of variation, similar to the interactions seen before

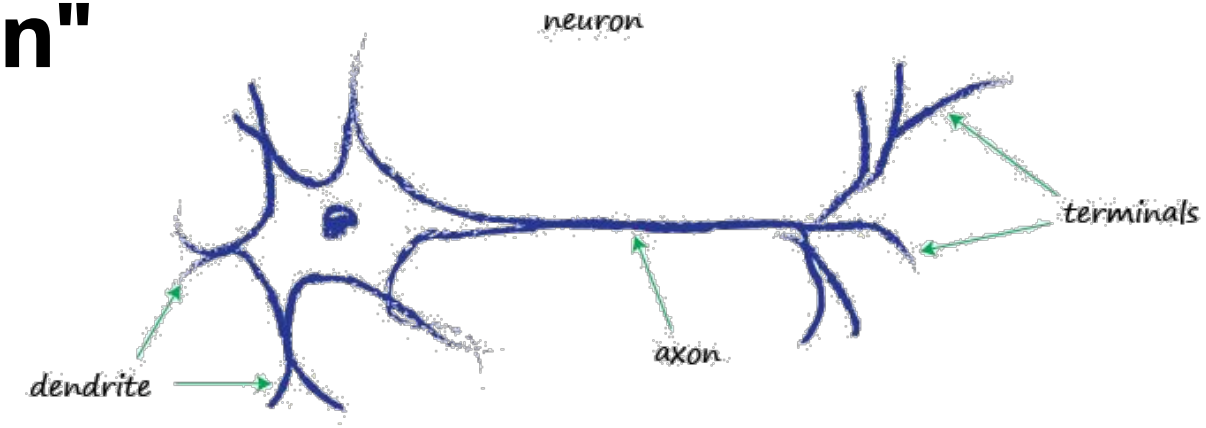
More scalable than modeling explicit interaction for every pair of variables

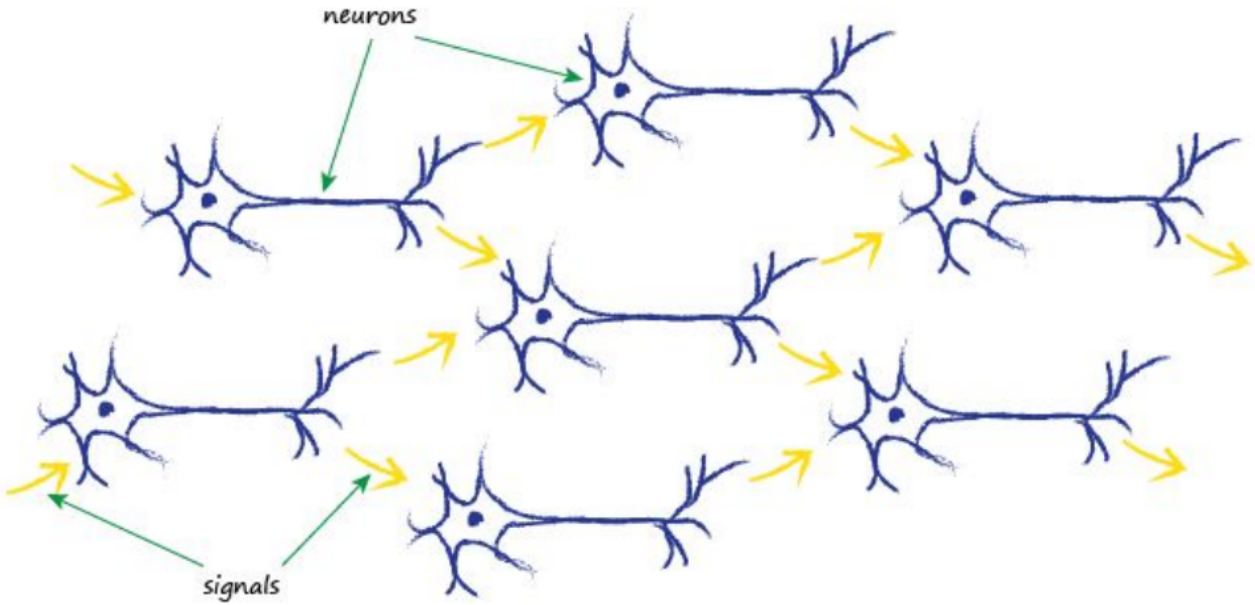
But less interpretable

Keywords

- Single-Layer Networks
- **Deep Neural Networks**
- Universal Approximation Theorem
- Activation functions
- Output units

From Neuron to "Brain"

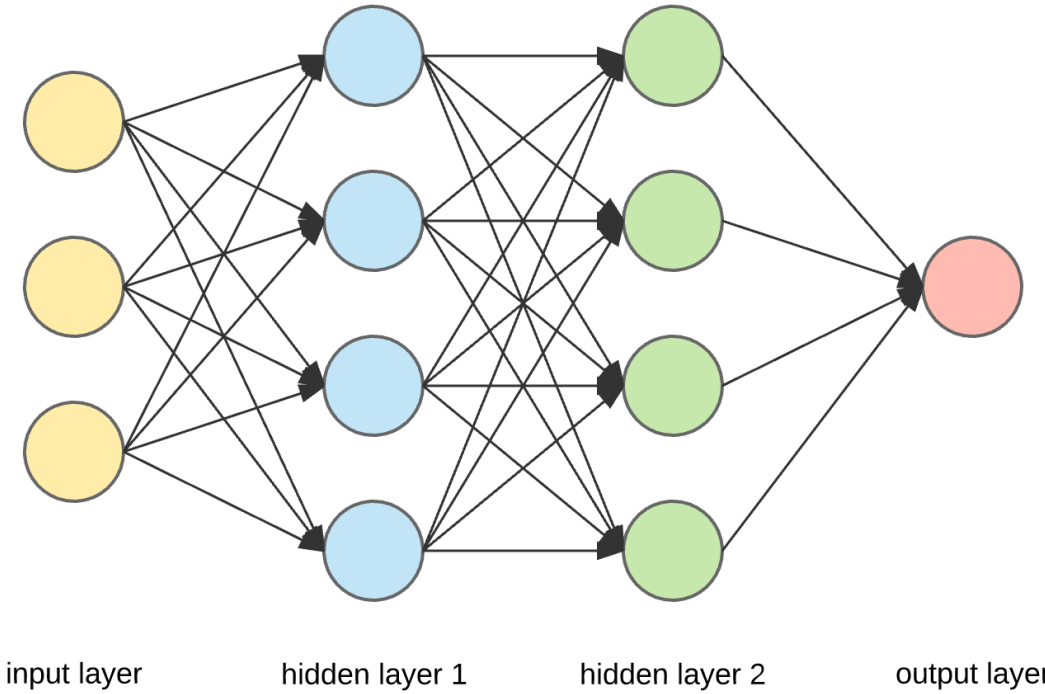




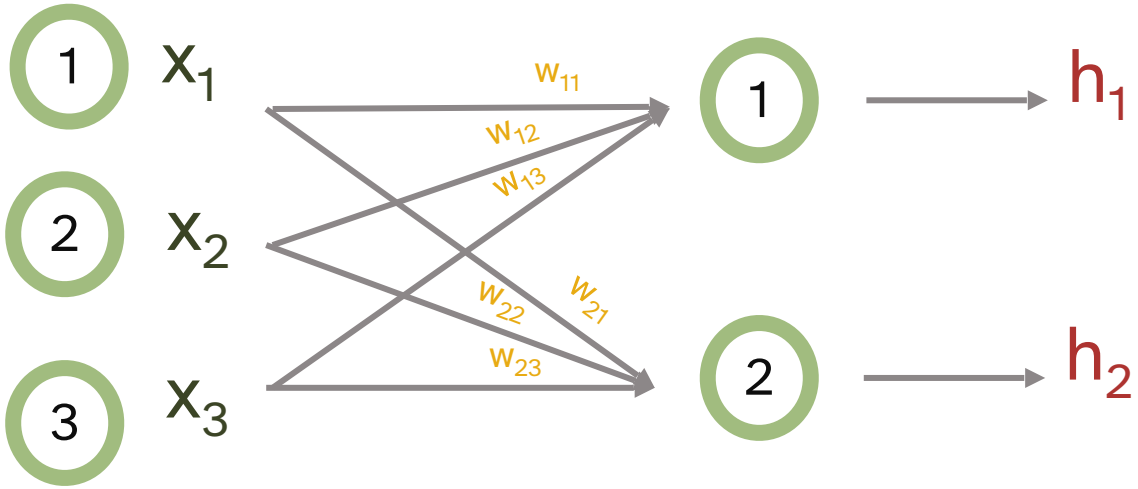
- We now can connect neurons
- The output of one neuron becomes the input of other neurons

In a more structured way

- We normally have more than one node
- Multiple Nodes are arranged in layers
- Each layer receives the generated output from the previous layer

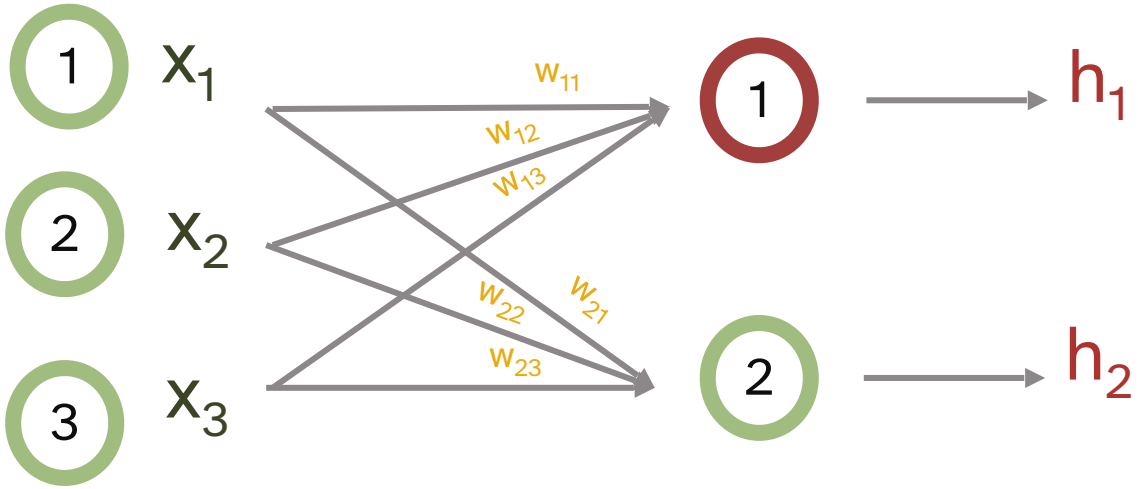


What does such a Network do?



- Remember from before:
  - We have a weighted sum
  - Then we have an "activation function" (which we ignore for now)

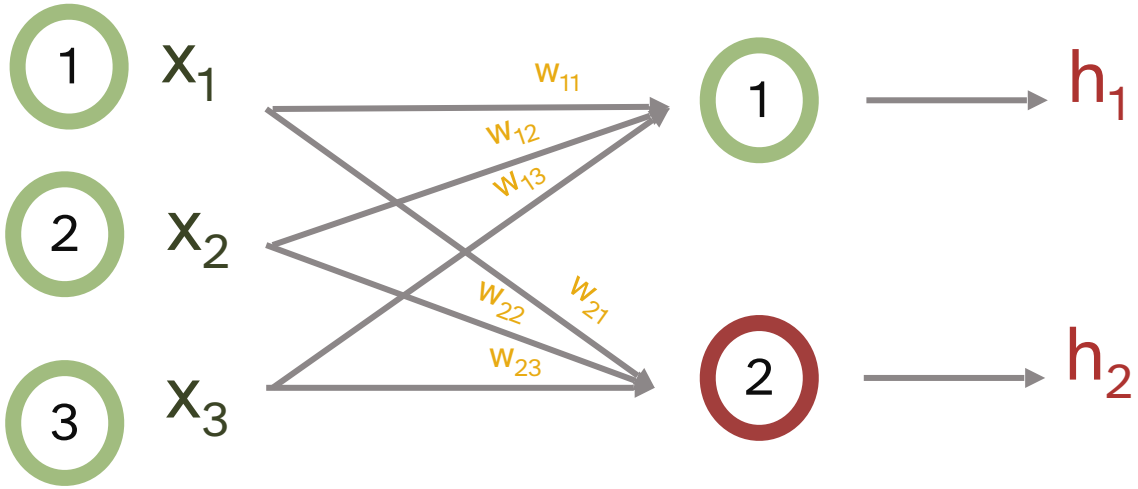
What does such a Network do?



- Remember from before:
  - We have a weighted sum
  - Then we have an "activation function" (which we ignore for now)

- For **Node 1**, we get: $h_1 = w_{11} \cdot x_1 + w_{12} \cdot x_2 + w_{13} \cdot x_3$

What does such a Network do?



- Remember from before:
  - We have a weighted sum
  - Then we have an "activation function" (which we ignore for now)

- For **Node 2**, we get: $h_2 = w_{21} \cdot x_1 + w_{22} \cdot x_2 + w_{23} \cdot x_3$

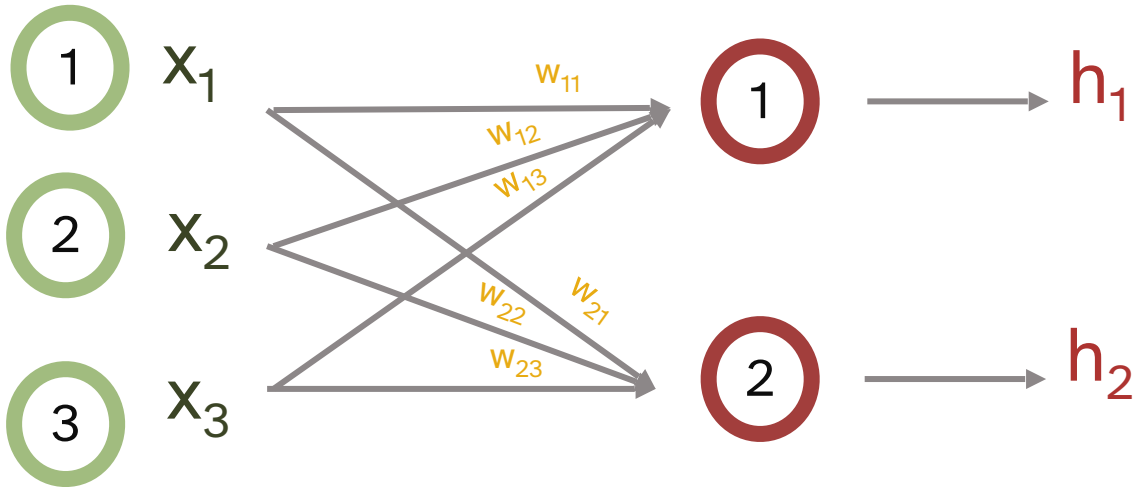
What does such a Network do?

Together:

$$h_1 = w_{11} \cdot x_1 + w_{12} \cdot x_2 + w_{13} \cdot x_3$$

$$h_2 = w_{21} \cdot x_1 + w_{22} \cdot x_2 + w_{23} \cdot x_3$$

Looks familiar, doesn't it?

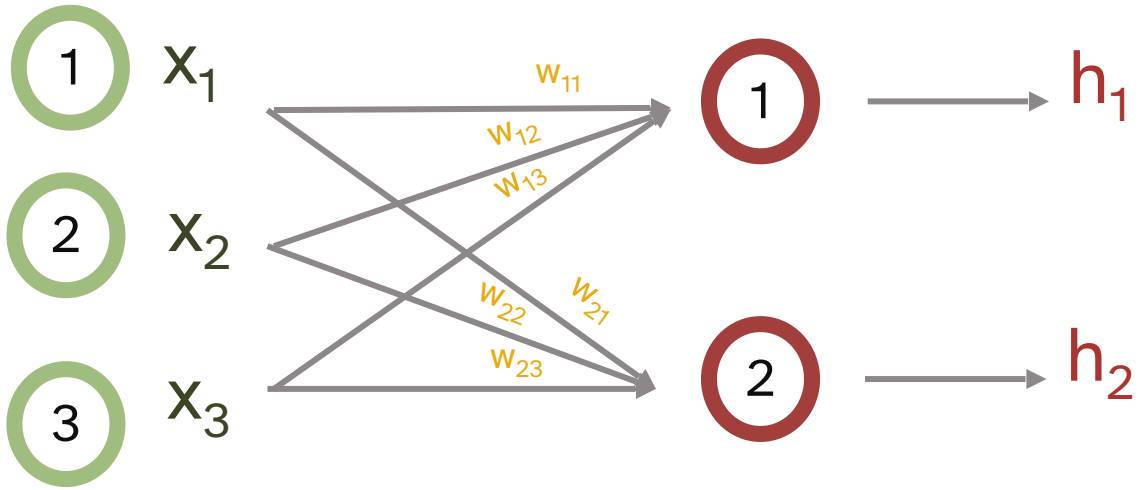


What does such a Network do?

Rewrite:

$$\begin{pmatrix} h_1 \\ h_2 \end{pmatrix} = \begin{pmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \end{pmatrix} \cdot \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix}$$

$$h = \boldsymbol{W}\boldsymbol{x}$$

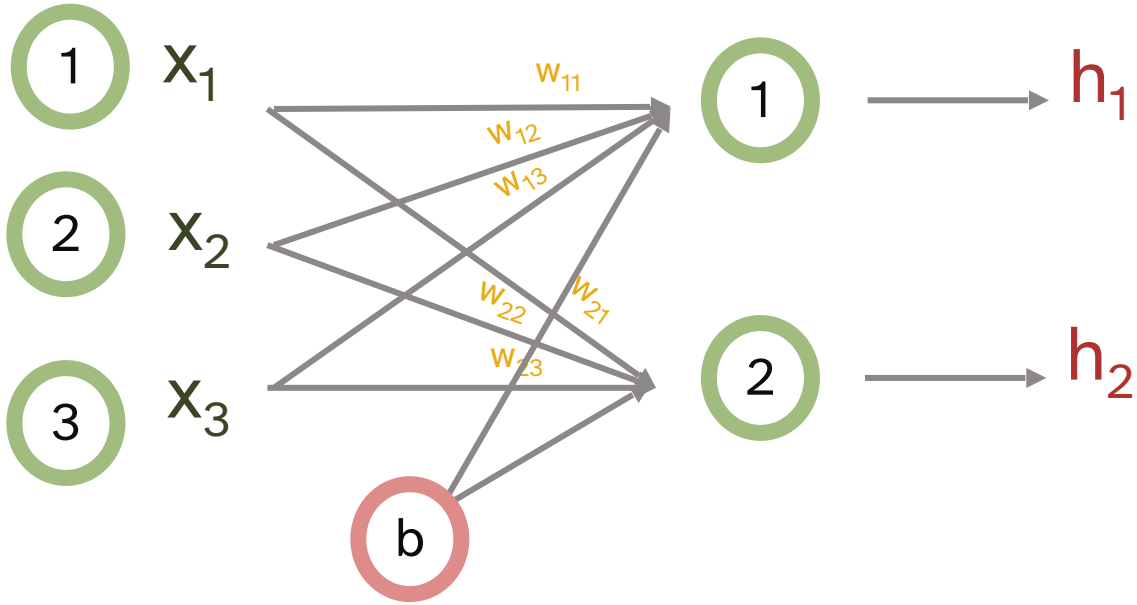


What does such a Network do?

Adding a **Bias**:

$$\begin{pmatrix} h_1 \\ h_2 \end{pmatrix} = \begin{pmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \end{pmatrix} \cdot \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} + \begin{pmatrix} b_1 \\ b_2 \end{pmatrix}$$

$$\boldsymbol{h} = \boldsymbol{W}\boldsymbol{x} + \boldsymbol{b}$$

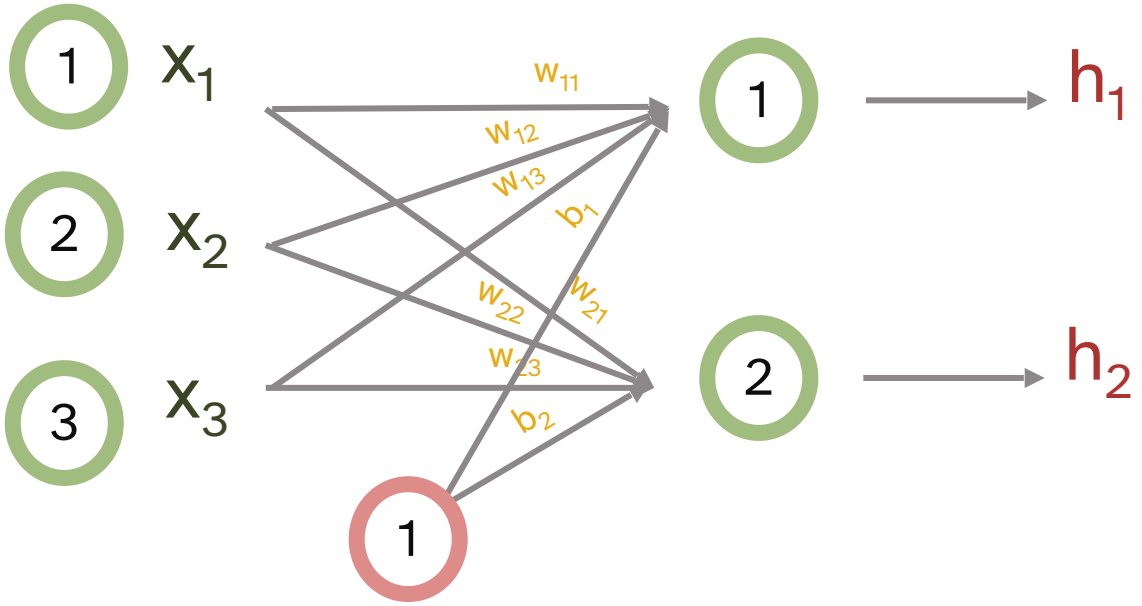


What does such a Network do?

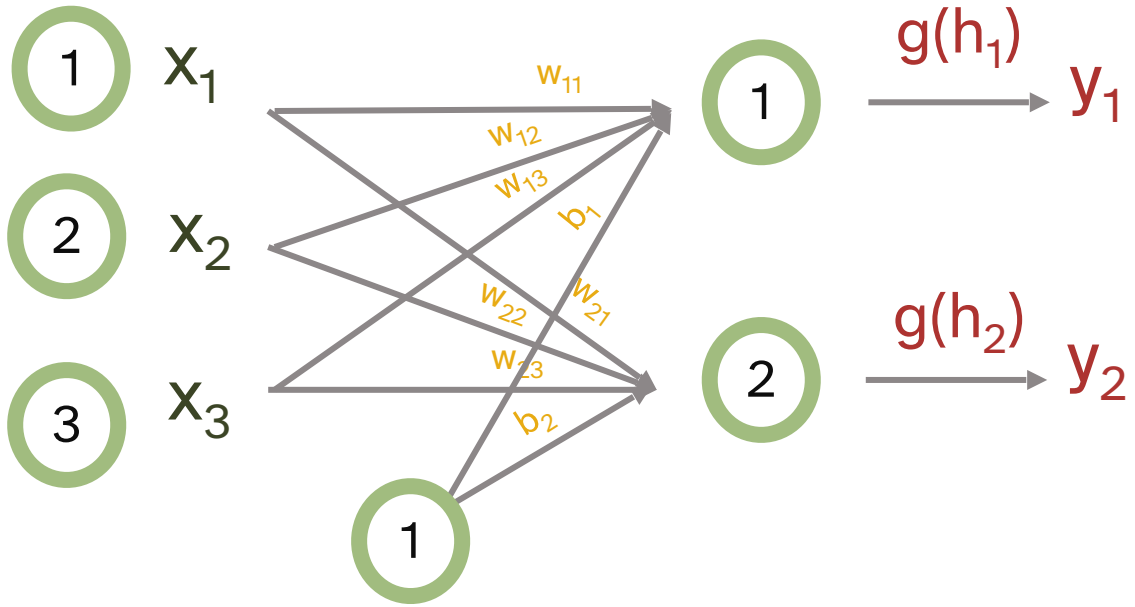
Adding a **Bias** (rewritten):

$$\begin{pmatrix} h_1 \\ h_2 \end{pmatrix} = \begin{pmatrix} w_{11} & w_{12} & w_{13} & b_1 \\ w_{21} & w_{22} & w_{23} & b_2 \end{pmatrix} \cdot \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ 1 \end{pmatrix}$$

$$\boldsymbol{h} = \boldsymbol{W}\boldsymbol{x}$$



What does such a Network do?

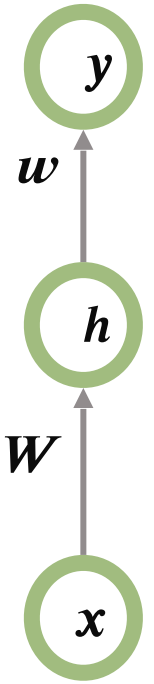


Adding the **activation function:**

$$\boldsymbol{y} = \boldsymbol{g}\left(\boldsymbol{W}\boldsymbol{x} + \boldsymbol{b}\right)$$

More Compact Representation

- We can summarize these networks to their essential parts: We receive the vector $\boldsymbol{x}$ and perform an affine transformation according to the weight matrix
- $\boldsymbol{h}' = \boldsymbol{W}\boldsymbol{x} + \boldsymbol{b}$
- Afterward, the internal representation is component wise feed through an activation function $h_i = g(h_i')$ to generate the input vector $\boldsymbol{h}$ for the next layer
- Please note the slight shift in notation. From now on:
  - $\boldsymbol{x}$ denotes the input
  - $\boldsymbol{W}$ denotes weight matrices, **w** denotes weight vectors
  - $\boldsymbol{h}$ denotes hidden layer "outputs"
  - $\boldsymbol{y}$ is the final output of the network



Composition of Complex Functions

Now let's look at a deeper Network:

E.g., functions $f^{(1)}$, $f^{(2)}$, $f^{(3)}$ connected in a chain to form

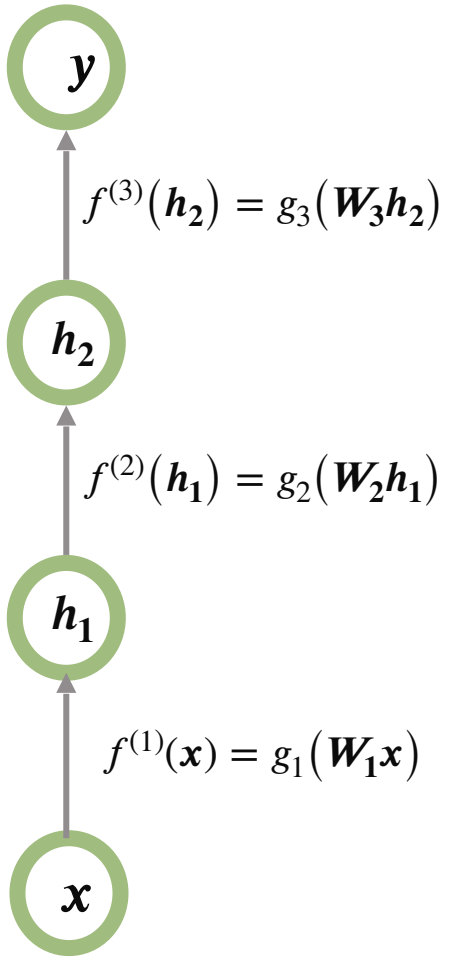
$$f(\boldsymbol{x}) = f^{(3)}\Big(f^{(2)}\big(f^{(1)}(\boldsymbol{x})\big)\Big)$$

$f^{(1)}$ is the first layer, $f^{(2)}$ the second, etc.

The length of the chain is the depth of the model

The name deep learning arises from this terminology

Final layer of a feedforward network, (here $f^{(3)}$) is the output layer



What are Hidden Layers

- Behavior of hidden layers is not directly specified by the data
- Learning algorithm must decide how to use those layers to produce a final value that is close to y
- Training data does not say what individual layers should do
  - This is one of the main tasks: Define the appropriate structure of the network and the hidden layers
- Since the desired output for these layers is not shown, they are called hidden layers

Extending Linear Models

- To represent non-linear functions of $\boldsymbol{x}$
- Apply linear model to transformed input $\phi(\boldsymbol{x})$ with non-linear ϕ

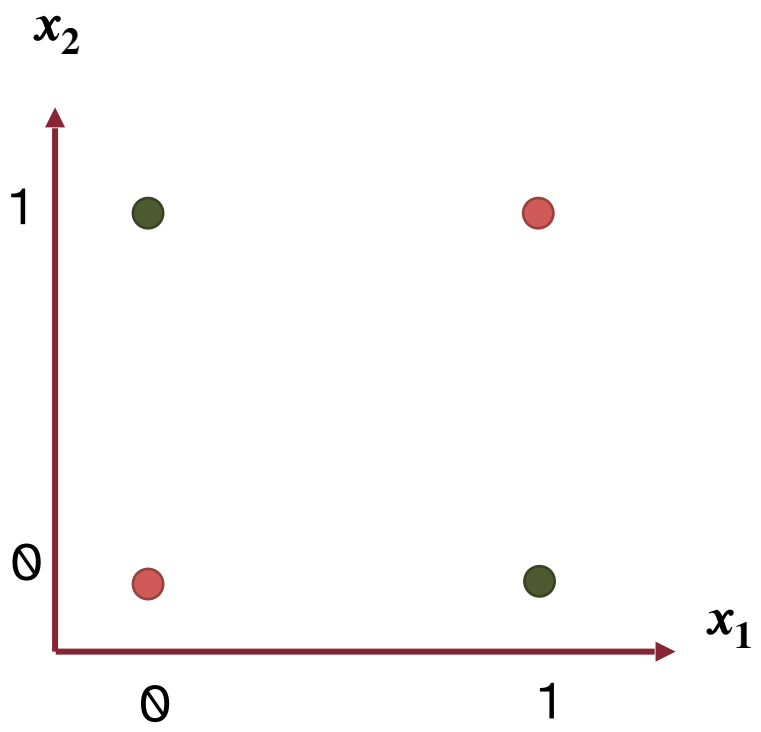
  Equivalently kernel trick of SVM obtains nonlinearity
- Function is nonlinear wrt x but linear wrt $\phi(x)$

  We can think of ϕ as providing a set of features resulting in a new representation for x
- Get ϕ
  - Generic functions: There exists a set of useful kernel functions
  - Manually engineer ϕ: Laborious and not transformable
  - **Deep Learning**: Learn ϕ

On the importance of Activation Functions

- The XOR Problem:
  - XOR is a logical function
  - It takes two inputs, x_1 and x_2

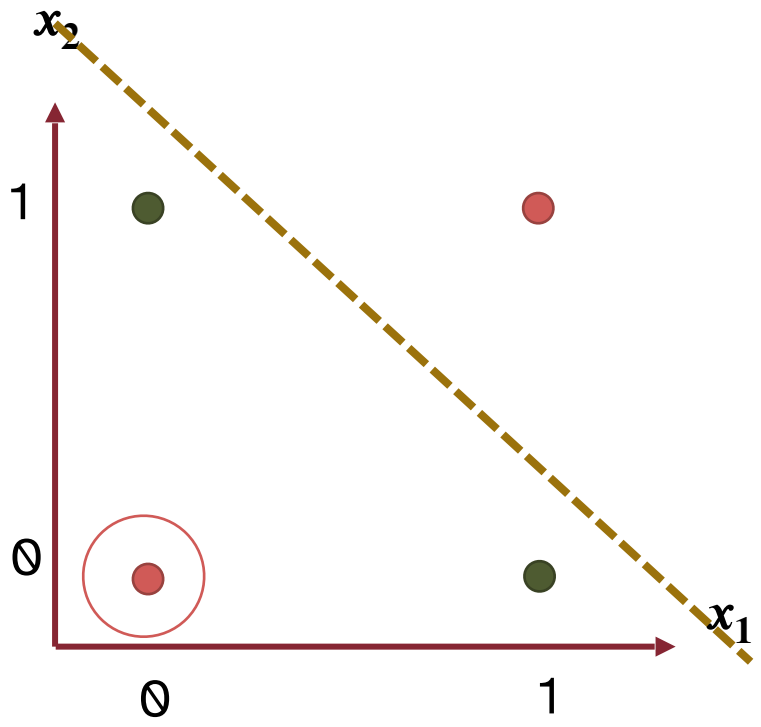
Input 1	Input 2	Output
0	0	0
0	1	1
1	0	1
1	1	0



On the importance of Activation Functions

- The XOR Problem:
  - XOR is a logical function
  - It takes two inputs, x_1 and x_2
- **The problem is not linearly separable**

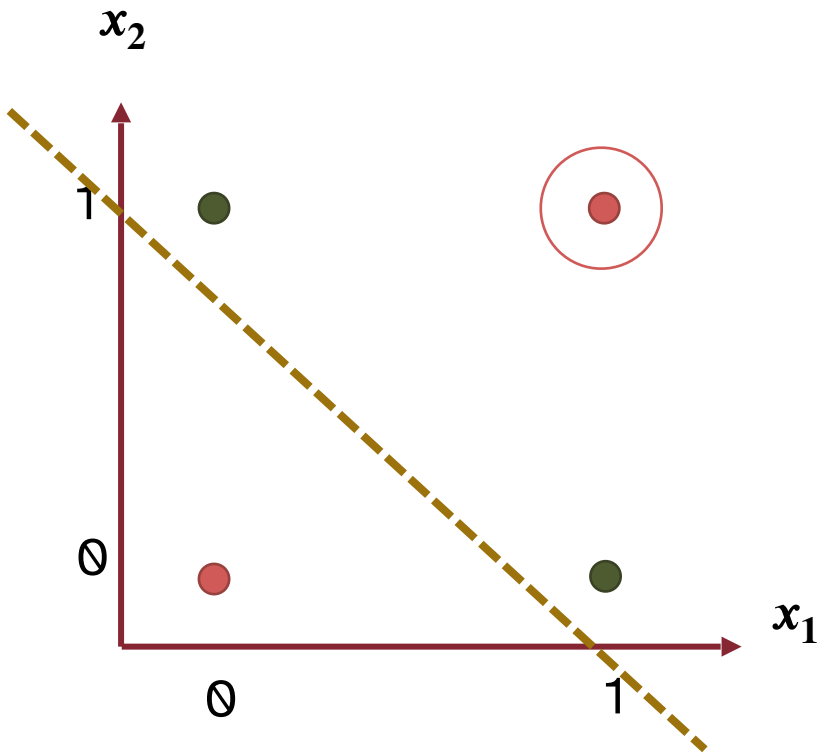
Input 1	Input 2	Output
0	0	0
0	1	1
1	0	1
1	1	0



On the importance of Activation Functions

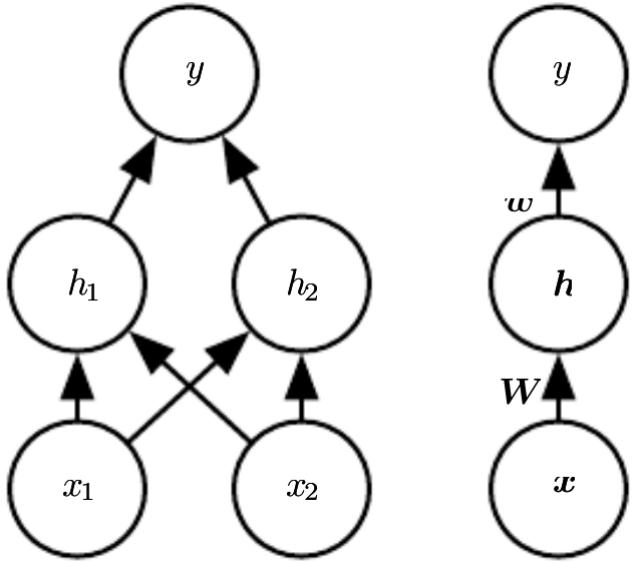
- The XOR Problem:
  - XOR is a logical function
  - It takes two inputs, x_1 and x_2
- **The problem is not linearly separable**

Input 1	Input 2	Output
0	0	0
0	1	1
1	0	1
1	1	0



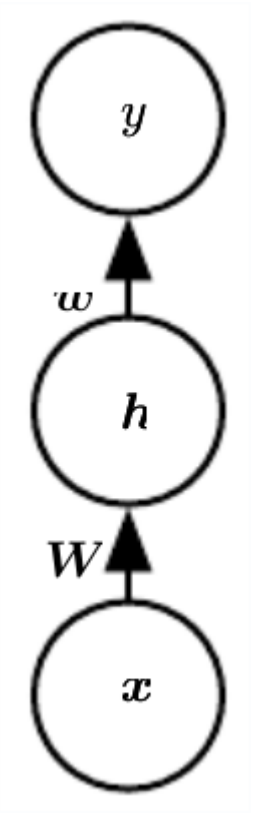
The XOR Problem

- We are trying to approximate the XOR function
  - Our training points: $X = \left\{ [0,0]^T, [1,0]^T, [0,1]^T, [1,1]^T \right\}$
  - As we have seen, linear separation is not possible!
- **Now lets use a FFN**
  - One hidden layer, containing two units
  - Both representations are equivalent
    - Matrix $\boldsymbol{W}$ describes mapping $\boldsymbol{x}$ to $\boldsymbol{h}$
    - Vector $\boldsymbol{w}$ describes mapping $\boldsymbol{h}$ to y
    - (biases omitted)



What is computed

- Layer 1 (hidden layer): vector of hidden units $\boldsymbol{h}$ computed by function $\boldsymbol{h} = f^{(1)}(\boldsymbol{x}; \boldsymbol{W}, c)$ with bias c
- Layer 2 (output layer) computes: $y = f^{(2)}(\boldsymbol{h}; \boldsymbol{w}, b)$
  - $\boldsymbol{w}$ are the weights (it is basically a linear regression)
  - Output is linear regression applied to $\boldsymbol{h}$ rather than to $\boldsymbol{x}$
- Complete model is $f(\boldsymbol{x}; \boldsymbol{W}, c, \boldsymbol{w}, b) = f^{(2)}(f^{(1)}(\boldsymbol{x}; \boldsymbol{W}, c), \boldsymbol{w}, b)$



Activation Function

- If we choose both $f^{(1)}$ and $f^{(2)}$ to be linear, the total function will still be linear; which is insufficient (as already seen)

- Therefore, we need an **Activation Function**:
- Remember, our function looks as follows:

$$y = \boldsymbol{x}^{\boldsymbol{T}}\boldsymbol{w} + b$$

- Activation function g is typically chosen to be applied element-wise:

$$h_i = g(x^T W_{:i} + c_i)$$

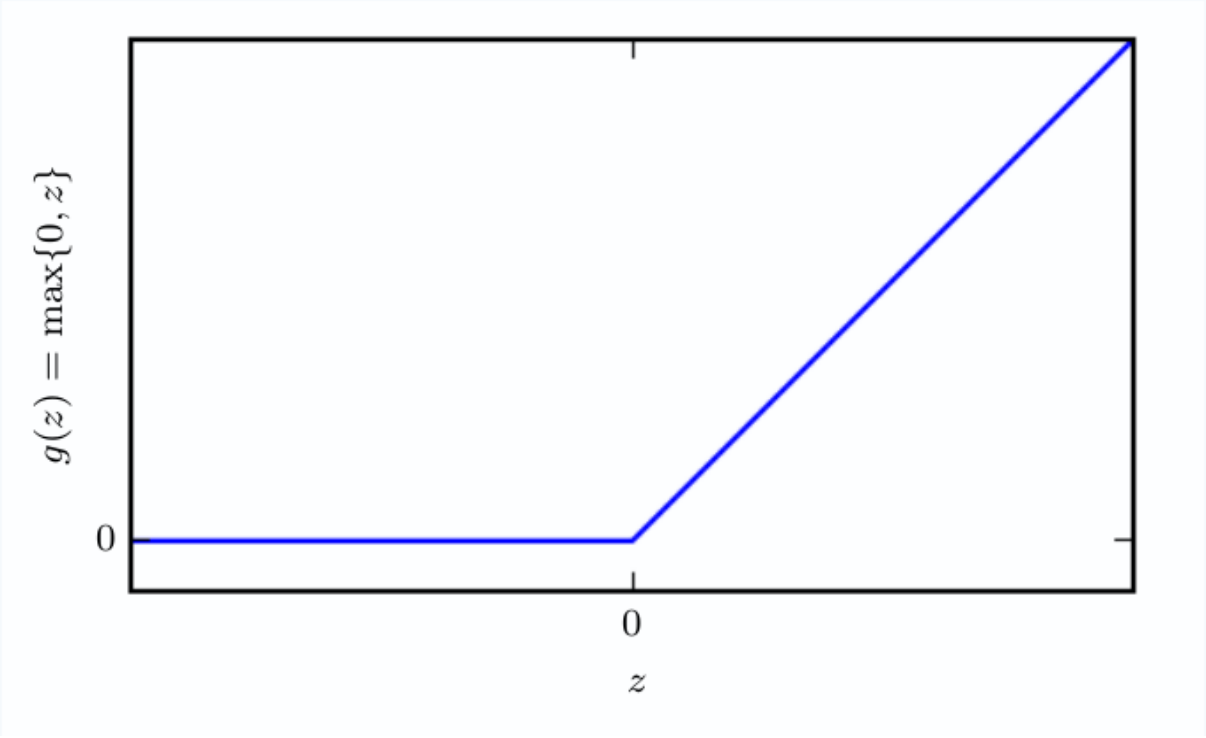
Default Activation Function: ReLU

➢ Activation function:

$$g(z) = \max\{0, z\}$$

✳ This already yields to a non-linear transformation

- ✳ But still piecewise linear
- ✳ Preserves couple of nice properties making gradient-based learning easy
- ✳ Generalizes well



Back to our XOR Problem

➢ The complete equation is now:

$$f(\boldsymbol{x}; \boldsymbol{W}, \boldsymbol{c}, \boldsymbol{w}, b) = \boldsymbol{w}^T \max\left\{0, \boldsymbol{W}^T \boldsymbol{x} + \boldsymbol{c}\right\} + b$$

Lets use the following weights:

$$\boldsymbol{W} = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}, \boldsymbol{c} = \begin{bmatrix} 0 \\ -1 \end{bmatrix}, \boldsymbol{w} = \begin{bmatrix} 1 \\ -2 \end{bmatrix}, b = 0 \text{ and } \boldsymbol{X} = \begin{bmatrix} 0101 \\ 0011 \end{bmatrix}^T$$

Back to our XOR Problem

➢ The complete equation is now:

$$f(\boldsymbol{x}; \boldsymbol{W}, \boldsymbol{c}, \boldsymbol{w}, b) = \boldsymbol{w}^T \max\{0, \boldsymbol{W}^T \boldsymbol{x} + \boldsymbol{c}\} + b$$

Lets use the following weights:

$$\boldsymbol{W} = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}, \boldsymbol{c} = \begin{bmatrix} 0 \\ -1 \end{bmatrix}, \boldsymbol{w} = \begin{bmatrix} 1 \\ -2 \end{bmatrix}, b = 0 \text{ and } \boldsymbol{X} = \begin{bmatrix} 0101 \\ 0011 \end{bmatrix}^T$$

$$\boldsymbol{XW} = \begin{bmatrix} 0 & 0 \\ 1 & 1 \\ 1 & 1 \\ 2 & 2 \end{bmatrix}$$

Back to our XOR Problem

➢ The complete equation is now:

$$f(\boldsymbol{x}; \boldsymbol{W}, \boldsymbol{c}, \boldsymbol{w}, b) = \boldsymbol{w}^T \max\{0, \boldsymbol{W}^T \boldsymbol{x} + \boldsymbol{c}\} + b$$

Lets use the following weights:

$$\boldsymbol{W} = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}, \boldsymbol{c} = \begin{bmatrix} 0 \\ -1 \end{bmatrix}, \boldsymbol{w} = \begin{bmatrix} 1 \\ -2 \end{bmatrix}, b = 0 \text{ and } \boldsymbol{X} = \begin{bmatrix} 0101 \\ 0011 \end{bmatrix}^T$$

$$\boldsymbol{XW} = \begin{bmatrix} 0 & 0 \\ 1 & 1 \\ 1 & 1 \\ 2 & 2 \end{bmatrix} \qquad \boldsymbol{XW} + \boldsymbol{c} = \begin{bmatrix} 0 & -1 \\ 1 & 0 \\ 1 & 0 \\ 2 & 1 \end{bmatrix}$$

Back to our XOR Problem

➢ The complete equation is now:

$$f(\boldsymbol{x}; \boldsymbol{W}, \boldsymbol{c}, \boldsymbol{w}, b) = \boldsymbol{w}^T \max\{0, \boldsymbol{W}^T \boldsymbol{x} + \boldsymbol{c}\} + b$$

Lets use the following weights:

$$\boldsymbol{W} = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}, \boldsymbol{c} = \begin{bmatrix} 0 \\ -1 \end{bmatrix}, \boldsymbol{w} = \begin{bmatrix} 1 \\ -2 \end{bmatrix}, b = 0 \text{ and } \boldsymbol{X} = \begin{bmatrix} 0101 \\ 0011 \end{bmatrix}^T$$

$$\boldsymbol{XW} = \begin{bmatrix} 0 & 0 \\ 1 & 1 \\ 1 & 1 \\ 2 & 2 \end{bmatrix}$$

$$\boldsymbol{XW} + \boldsymbol{c} = \begin{bmatrix} 0 & -1 \\ 1 & 0 \\ 1 & 0 \\ 2 & 1 \end{bmatrix}$$

$$\max\{0, \boldsymbol{XW} + \boldsymbol{c}\} = \begin{bmatrix} 0 & 0 \\ 1 & 0 \\ 1 & 0 \\ 2 & 1 \end{bmatrix}$$

Back to our XOR Problem

➢ The complete equation is now:

$$f(\boldsymbol{x}; \boldsymbol{W}, \boldsymbol{c}, \boldsymbol{w}, b) = \boldsymbol{w}^T \max\{0, \boldsymbol{W}^T \boldsymbol{x} + \boldsymbol{c}\} + b$$

Lets use the following weights:

$$\boldsymbol{W} = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}, \boldsymbol{c} = \begin{bmatrix} 0 \\ -1 \end{bmatrix}, \boldsymbol{w} = \begin{bmatrix} 1 \\ -2 \end{bmatrix}, b = 0 \text{ and } \boldsymbol{X} = \begin{bmatrix} 0101 \\ 0011 \end{bmatrix}^T$$

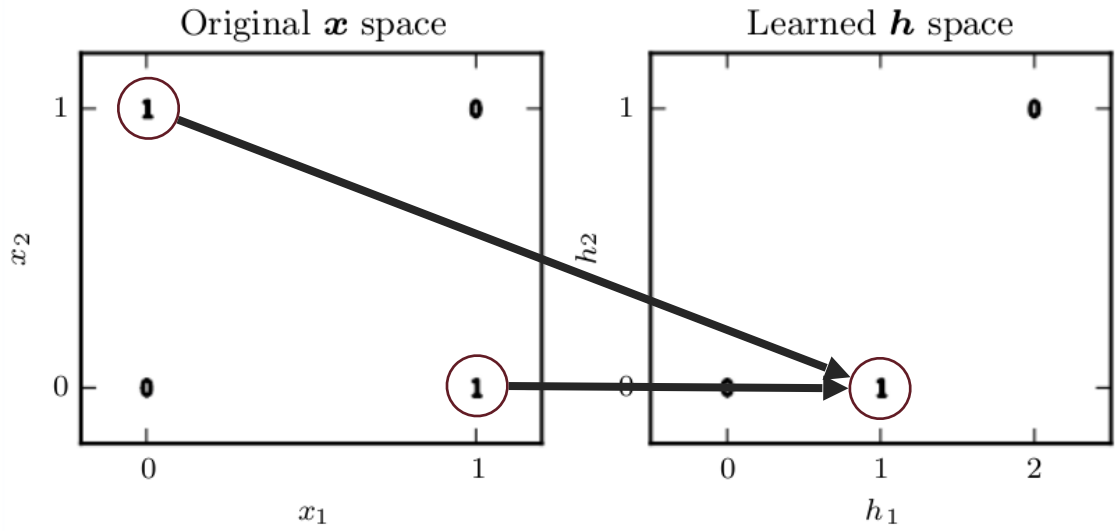
$$\boldsymbol{XW} = \begin{bmatrix} 0 & 0 \\ 1 & 1 \\ 1 & 1 \\ 2 & 2 \end{bmatrix}$$

$$\boldsymbol{XW} + \boldsymbol{c} = \begin{bmatrix} 0 & -1 \\ 1 & 0 \\ 1 & 0 \\ 2 & 1 \end{bmatrix}$$

$$\max\{0, \boldsymbol{XW} + \boldsymbol{c}\} = \begin{bmatrix} 0 & 0 \\ 1 & 0 \\ 1 & 0 \\ 2 & 1 \end{bmatrix}$$

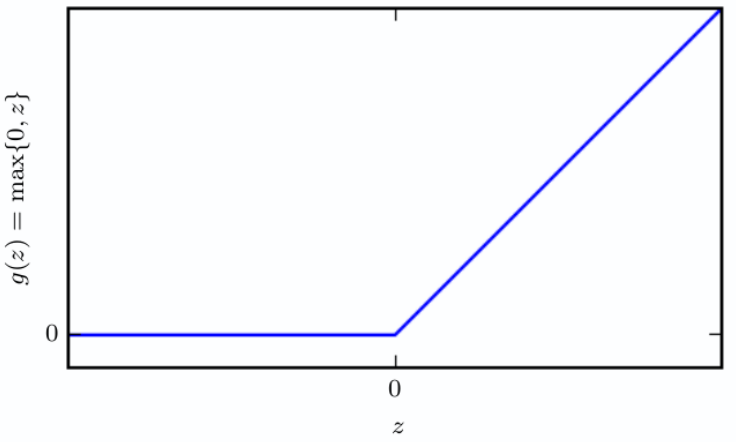
$$\begin{bmatrix} 0 & 0 \\ 1 & 0 \\ 1 & 0 \\ 2 & 1 \end{bmatrix} \boldsymbol{w} = \begin{bmatrix} \mathbf{0} \\ \mathbf{1} \\ \mathbf{1} \\ \mathbf{0} \end{bmatrix}$$

“Learned XOR”



- Two points that must have output 1 have been collapsed into one
- Can be separated in a linear model:
  - For fixed h_2 output has to increase in h_1
- Of course, we “cheated”: We have “guessed” the parameters correctly which lead to the desired result
- In reality: Millions of parameters which we have to actually learn

Interim Summary



- Linear models cannot solve the X-OR (exclusive-or) problem
- Composition of linear models are still Linear, due to associativity.

  In other words, we could rewrite $\mathbf{W}^{(2)}\mathbf{W}^{(1)}\mathbf{x}$ as we could set

  $\mathbf{W}' := \mathbf{W}^{(1)} \cdot \mathbf{W}^{(2)}$,

  s.th. $\mathbf{W}'x = \mathbf{W}^{(2)}\mathbf{W}^{(1)}\mathbf{x}$ are equivalent.
- Therefore, we need to add **nonlinear activation functions** in-between
- A simple but effective nonlinearity is the rectified linear unit (**ReLU**)

  ReLU(x) = max(0, x), i.e., x if x > 0, else 0

Keywords

- Single-Layer Networks
- Deep Neural Networks
- **Universal Approximation Theorem**
- Activation functions
- Output units

On the Power of Neural Networks

A linear model, mapping from features to outputs via matrix multiplication, can by definition represent only linear functions. But:

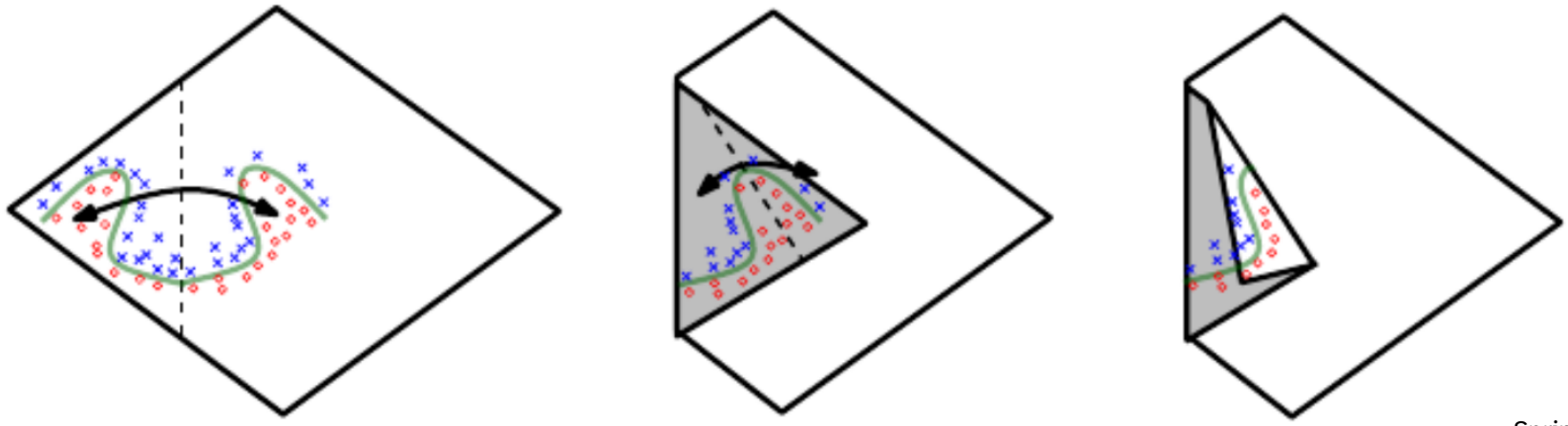
The **universal approximation theorem** states that a feedforward neural network with at least one hidden layer and a nonlinear activation function can approximate any compact, continuous function to an arbitrary degree of accuracy (depending on the hidden layer's width).

Implication of Theorem

- Whatever function we are trying to learn, a sufficiently large MLP will be able to represent it
- However we are not guaranteed that the training algorithm will learn this function
- No Free Lunch: There is no universal procedure for examining a training set of samples and choosing a function that will generalize to points not in training set

Advantage of deeper networks

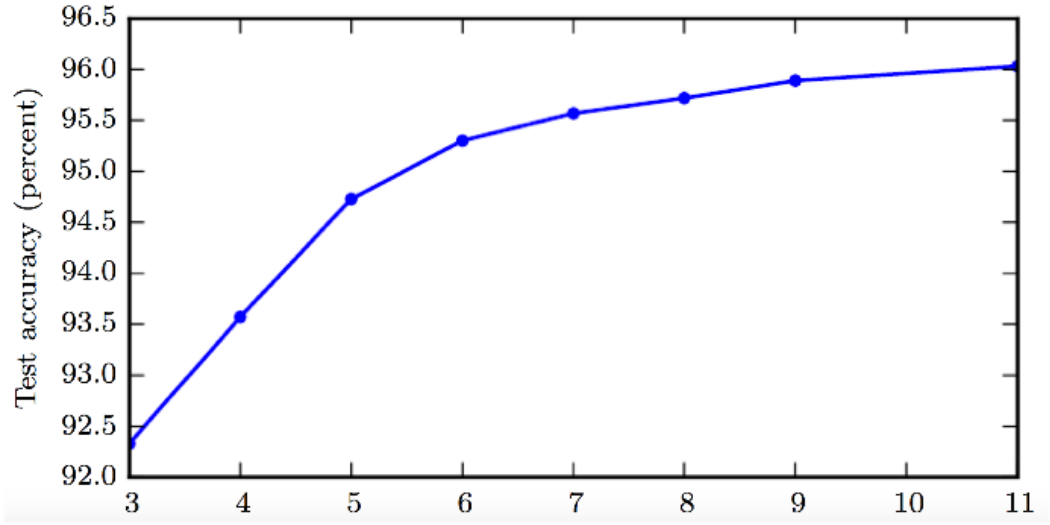
- Absolute value rectification creates mirror images of function computed on top of some hidden unit, wrt the input of that hidden unit.
- Each hidden unit specifies where to fold the input space to create mirror responses.
- By composing these folding operations, we obtain an exponentially large no. of piecewise linear regions which can capture all kinds of repeating patterns



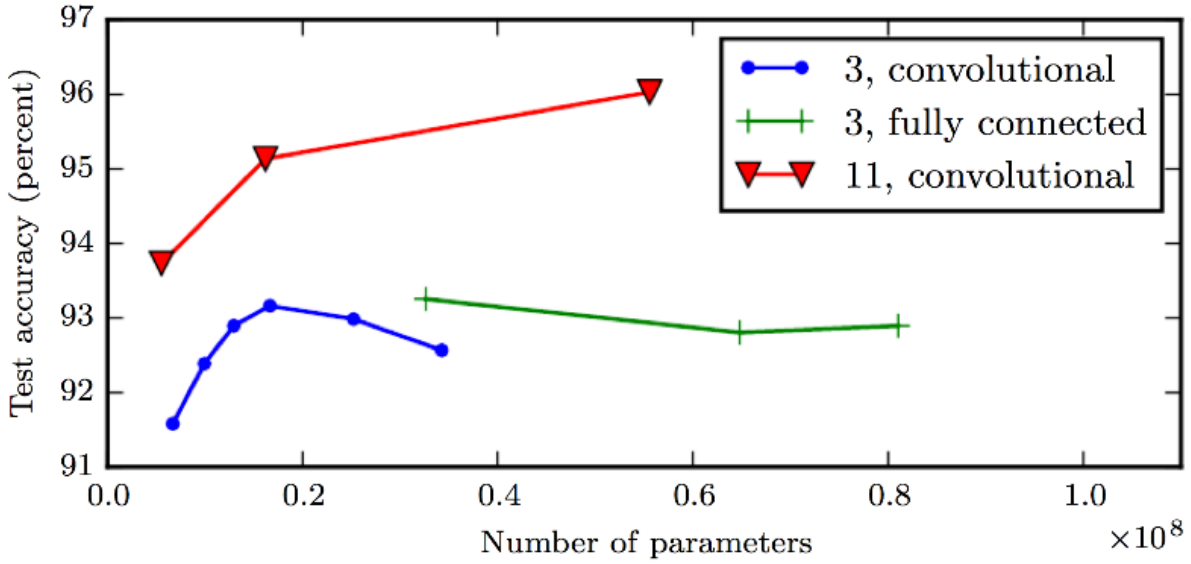
Intuition of Depth

- Any time we choose a ML algorithm we are implicitly stating a set of beliefs about what kind of functions that algorithm should learn
- Choosing a deep model encodes a belief that the function should be a composition several simpler functions
- The learning problem consists of discovering a set of underlying factors of variation that can in turn be described in terms of other, simpler underlying factors of variation.
- Empirically: Deeper Networks perform better ☺

Deeper Networks



Test accuracy consistently increases with depth



Increasing parameters without increasing depth is not as effective

Keywords

- Single-Layer Networks
- Deep Neural Networks
- Universal Approximation Theorem
- **Activation functions**
- Output units

General Construction of a Hidden Unit

- Accepts a vector of inputs $\boldsymbol{x}$ and computes an affine transformation $\boldsymbol{h}' = \boldsymbol{W}\boldsymbol{x} + \boldsymbol{b}$
- Computes an element-wise non-linear function $g(\boldsymbol{h}')$
- Most hidden units are distinguished from each other by the choice of activation function $g(\boldsymbol{h}')$
  - We look at: ReLU, Sigmoid and tanh, and other hidden units

Choice of Hidden Unit

- We now look at how to choose the type of hidden unit in the hidden layers of the model
- Design of hidden units is an active research area that does not have many definitive guiding theoretical principles
- ReLU is an excellent default choice
- But there are many other types of hidden units available
- When to use which kind?
  - Hard to predict in advance which will work best for a problem at hand
  - Design process is trial and error

Logistic Sigmoid

- Prior to introduction of ReLU, most neural networks used logistic sigmoid activation

$$g(z) = \sigma(z) = \frac{1}{1 + e^{-x}}$$

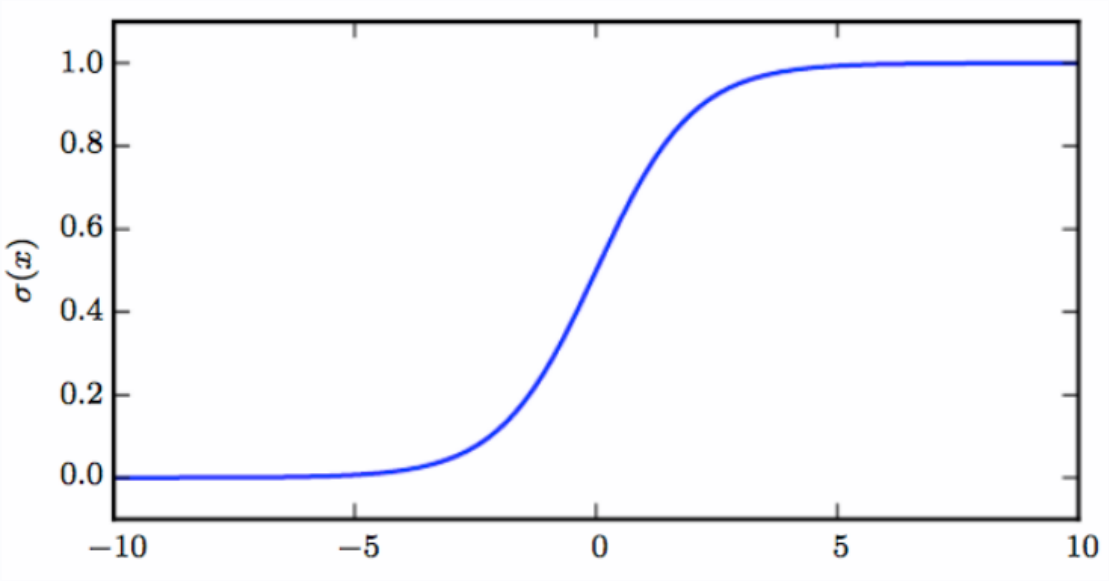
- Or the hyperbolic tangent

$$g(z) = \tanh(z)$$

- These activation functions are closely related because

$$\tanh(z) = 2\sigma(2z) - 1$$

- Sigmoid units are used to predict probability that a binary variable is 1



Other Activation Functions

✳ Cosine

$$\boldsymbol{h} = \cos\left(\boldsymbol{W}\boldsymbol{x} + \boldsymbol{b}\right)$$

- ✳ Feedforward networks on MNIST obtained error rate of less than 1%

✳ Radial Basis

$$h_i = \exp\left(-\frac{1}{\sigma^2}\left\|\boldsymbol{W}_{:j} - \boldsymbol{x}\right\|^2\right)$$

- ✳ Becomes more active as $\boldsymbol{x}$ approaches a template $W_{:,i}$

✳ Softplus

$$g(a) = \zeta(a) = \log\left(1 + e^a\right)$$

- ✳ Smooth version of the rectifier

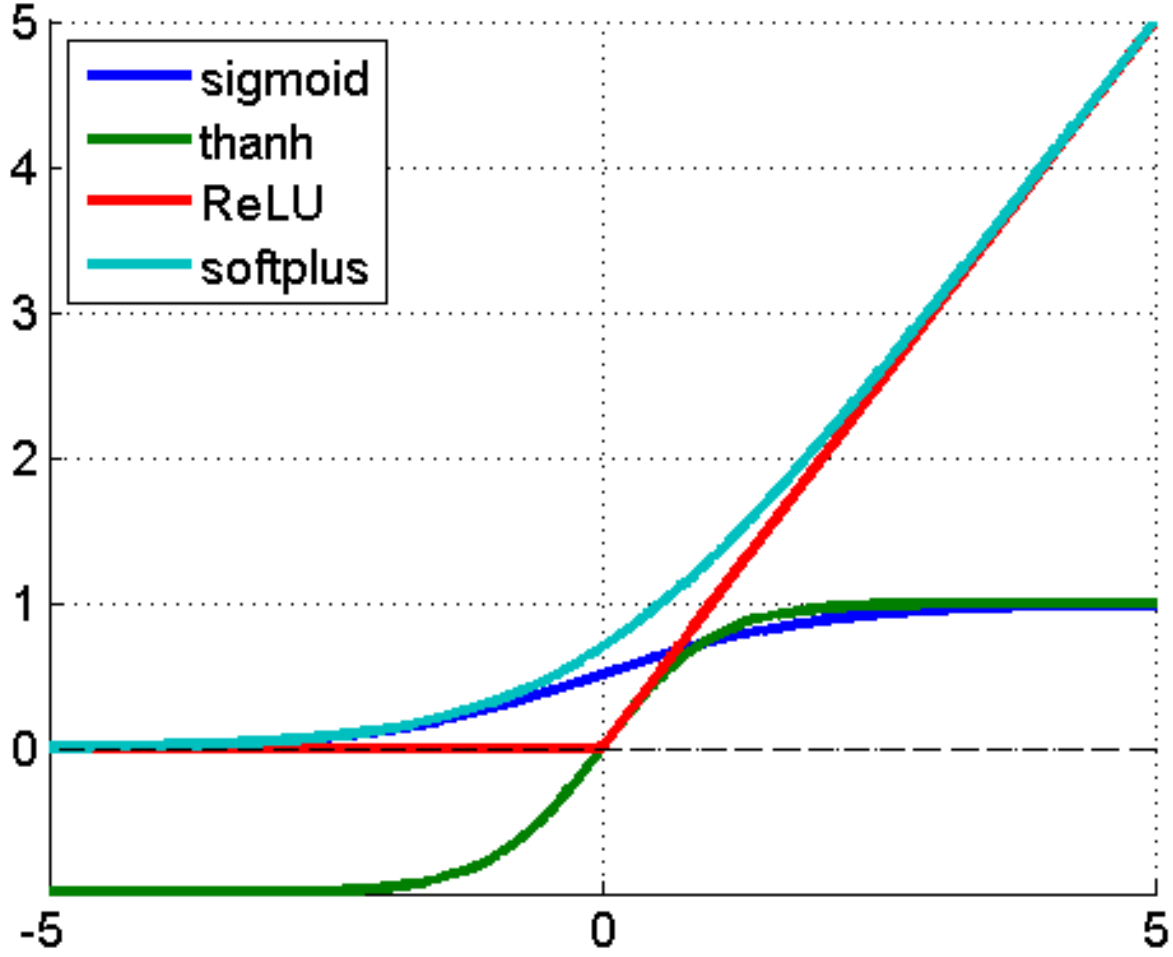
✳ Hard tanh

$$g(a) = \max(-1, \min(1, a))$$

- ✳ Shaped similar to tanh and the rectifier but it is bounded

Sigmoid Saturation

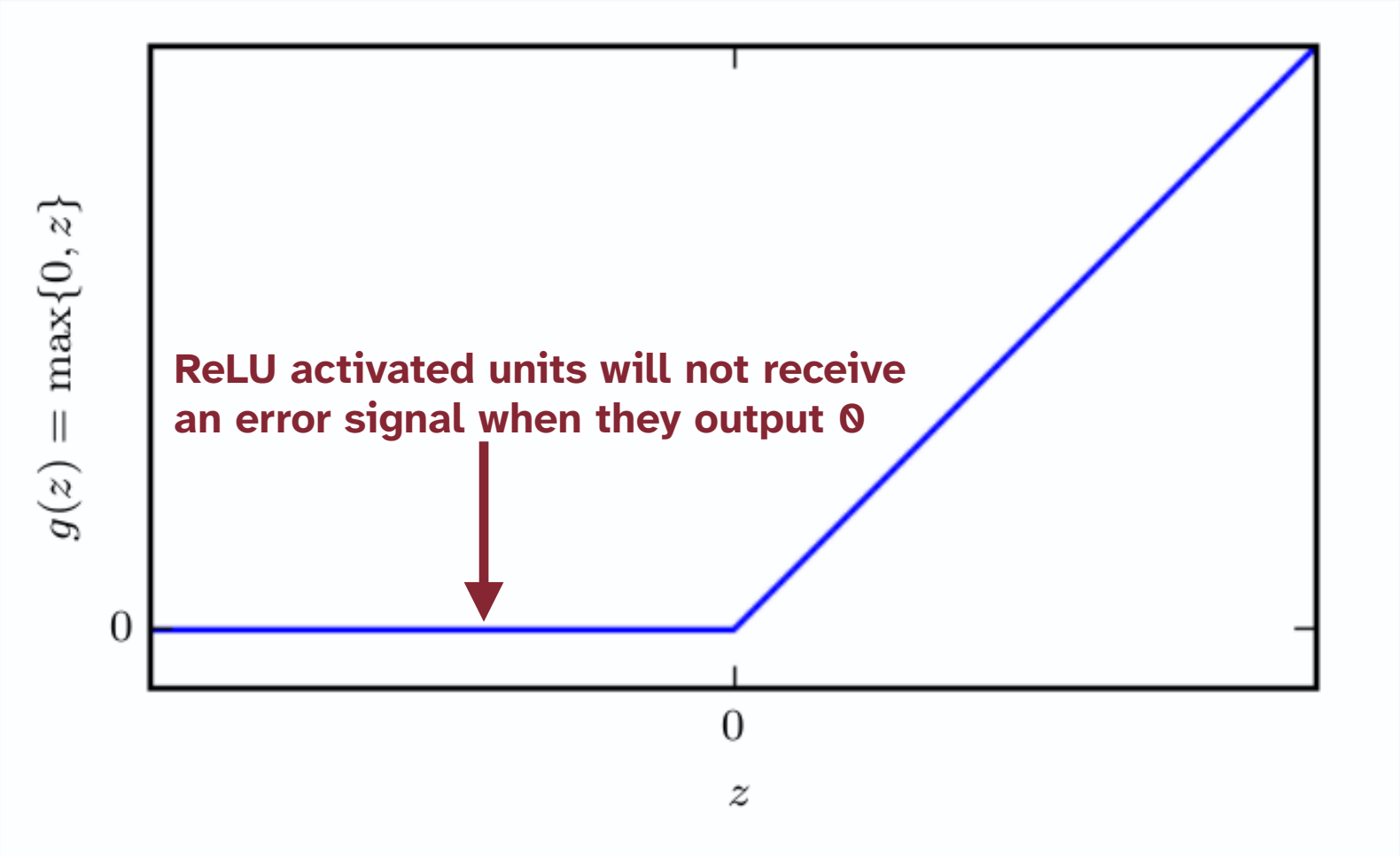
- Sigmoidals saturate
- ReLU and softplus increase for input >0
- As networks get very deep, this linear increase of the ReLU in the positive domain is important, such that the error signals don't get too small in the backward pass. Sigmoid units would learn **much** slower.



But is ReLU the end of the story?

$$\boldsymbol{h} = g\left(\boldsymbol{W}^T\boldsymbol{x} + \boldsymbol{b}\right)$$

$$g(x) = \max\{0, x\}$$



Generalizations of ReLU

- Three methods based on using a non-zero slope α_i when $z_i < 0$:

  $h_i = g(\boldsymbol{z}, \boldsymbol{\alpha})_i = \max(0, z_i) + \alpha_i \min(0, z_i)$

  or equivalently: $g(z, \alpha)_i = z_i$ **if** $z_i > 0$ **else** $\alpha_i \cdot z_i$

- Absolute-value rectification:
  - fixes $\alpha_i = -1$ to obtain $g(z) = |z|$
- Leaky ReLU:
  - fixes α_i to a small value like 0.01
- Parametric ReLU or PReLU
  - treats α_i as a learnable parameter

Dead ReLU's and how to do better: ELU

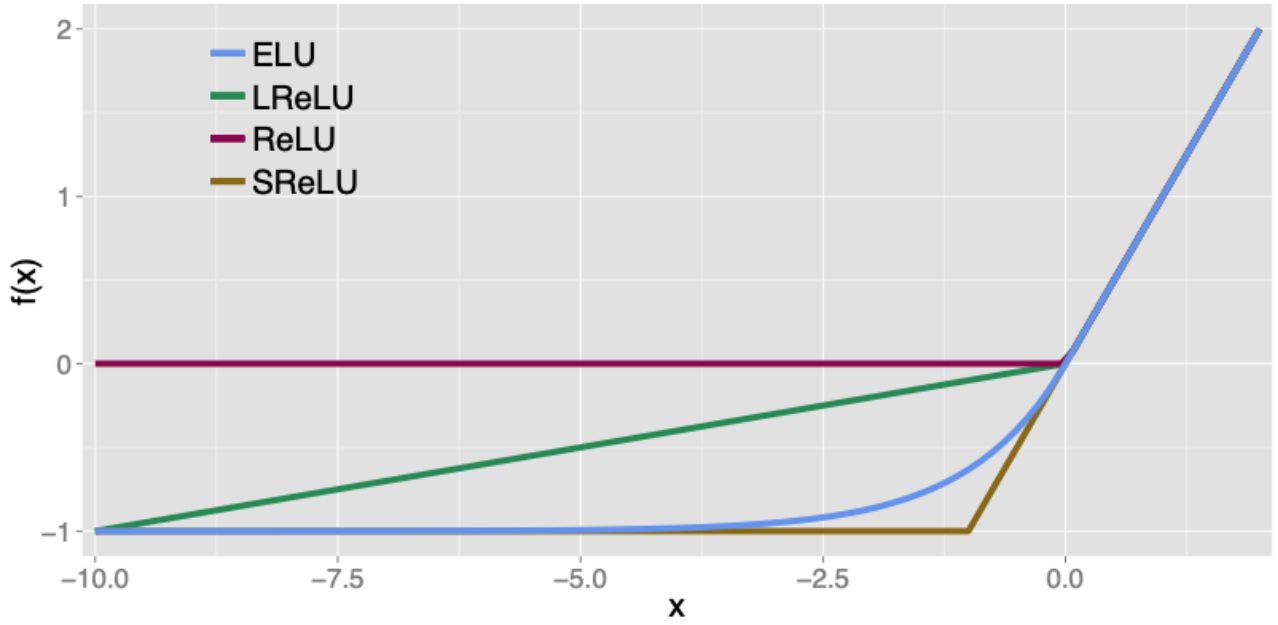
- **ReLU is a good default choice**
- **But**: ReLU's don't receive any gradient (error signal) when they output 0
- It can happen that they output 0 for all possible inputs – then you have **dead ReLU's**
- Variants that mitigate this problem:
  - **LeakyReLU**:

    $h_i = z_i$ if $z_i > 0$ else αz_i

    (with small positive α)
  - **Exponential Linear Unit (ELU)**, better than LeakyReLU

The *exponential linear unit* (ELU) with $0 < \alpha$ is

$$f(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha\,(\exp(x) - 1) & \text{if } x \leq 0 \end{cases}$$

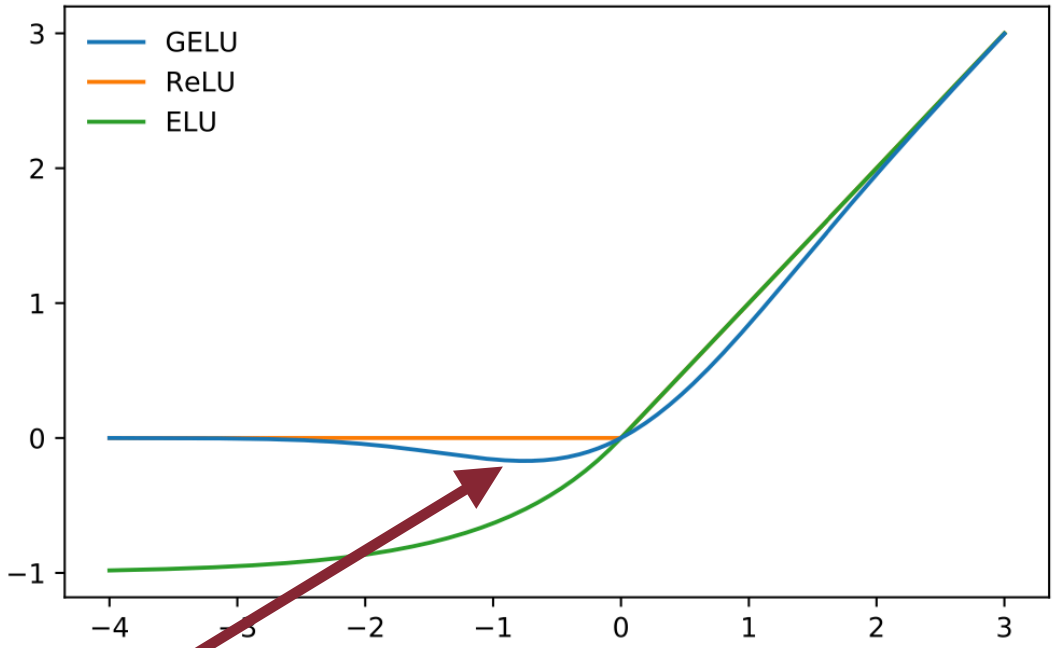


From arXiv:1511.07289v5

Dead ReLU's and how to do even better: GELU

- Popular in Transformers is the Gaussian Error Linear Unit (GELU)
- Exceeds performance of ReLU and ELU

From arXiv:1606.08415v5



Idea: GELU scales activations by how much larger they are than other activations, assuming N(0,1) distribution

$$\mathrm{GELU}(x) = xP(X \leq x) = x\Phi(x) = x \cdot \frac{1}{2}\left[1 + \mathrm{erf}(x/\sqrt{2})\right]$$

We can approximate the GELU with

$$0.5x(1 + \tanh[\sqrt{2/\pi}(x + 0.044715x^3)])$$

or

$$x\sigma(1.702x),$$

Leads to a smooth "bump" just before zero

Pattern also seen in other recent activation functions: x multiplied by logistic sigmoid of x

Summary of activation functions

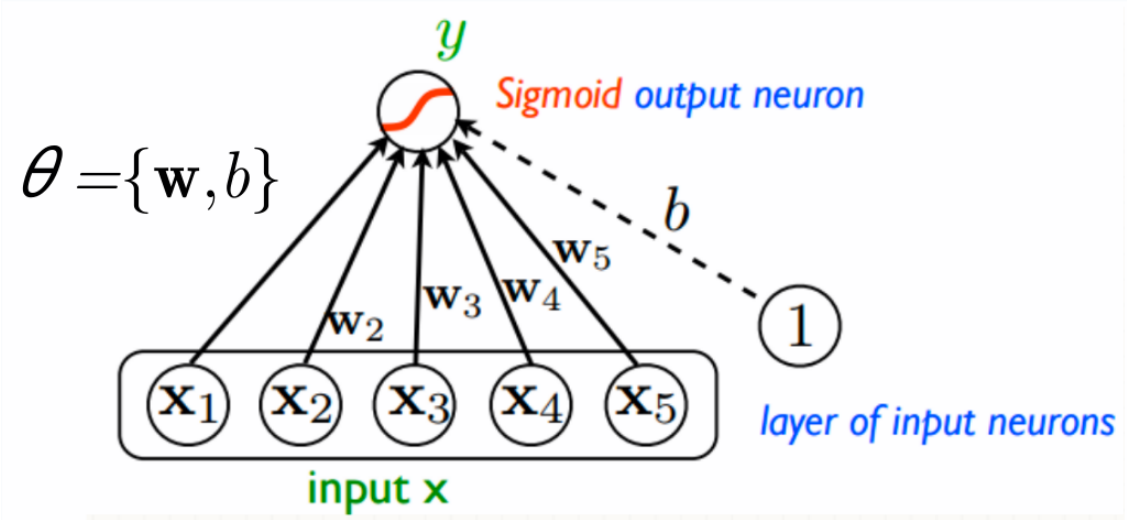
- Sigmoidal functions were used traditionally
- **But** they saturate and make learning difficult – **Sigmoid Saturation**
- **Therefore** ReLU's, which don't saturate on the positive side
- **But** ReLU's don't receive an update when they output 0 – **Dead ReLU's**
- **Therefore** do allow small, downscaled negative values (e.g., **LeakyReLU**)
- **But** we don't want them to become too negative, else they will have difficulty to re-activate
- **Therefore** have them saturate on the negative range (e.g., **ELU**)
- **And** often used recently, introducing a small bump close to zero (e.g., **GELU**)

Keywords

- Single-Layer Networks
- Deep Neural Networks
- Universal Approximation Theorem
- Activation functions
- **Output units**

Output Units

- The choice of cost function is **tightly** coupled with the choice of output unit (later...)
- Most of the time we use cross-entropy between data distribution and model distribution (later ...)
- Choice of how to represent the output then determines the form of the cross-entropy function



Role of an Output Unit

- Any output unit is in principle also usable as a hidden unit
- A feedforward network provides a hidden set of features

$$\boldsymbol{h} = f\left(\boldsymbol{x}; \boldsymbol{\theta}\right)$$

- Role of output layer is to provide some additional transformation from the features to the task that network must perform
- Common Output Units
  - **Linear units**: no non-linearity (used for Regressions)
  - **Sigmoid units** each individual output is between 0 and 1 (used for many-out-of-K classification)
  - **Softmax units** each individual output is between 0 and 1 **and** all outputs together sum up to 1 (e.g., used for one-out-of-K classification)

Linear Units for Regressions

- Linear unit: simple output based on affine transformation with no nonlinearity
- Given features h, a layer of linear output units produces a vector
  $\hat{\boldsymbol{y}} = \boldsymbol{W}^T \boldsymbol{h} + \boldsymbol{b}$
- Linear units are often used to produce mean $\hat{\boldsymbol{y}}$ of a conditional Gaussian distribution
- $P(\boldsymbol{y} \mid \boldsymbol{x}) = N(\boldsymbol{y}; \hat{\boldsymbol{y}}, \sigma^2)$
  We will see later why this is interesting

Most commonly used for regression:
Mean squared error loss (MSELoss)

Sigmoid Units for Binary Classification

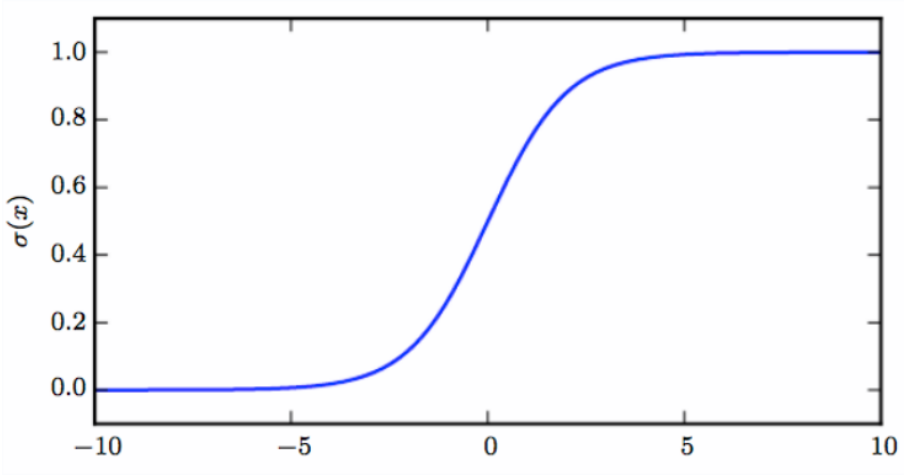
- Task of predicting value of binary variable y
  - Classification problem with two classes
- Maximum likelihood approach is to define a Bernoulli distribution over y conditioned on $\boldsymbol{x}$
- Neural net needs to predict $p(y = 1 \mid \boldsymbol{x}) \in [0,1]$:

$$P(y = 1 \mid \boldsymbol{x}) = \max\left\{0, \min\left\{1, \boldsymbol{w}^T \boldsymbol{h} + b\right\}\right\}$$

  - We would define a valid conditional distribution, but cannot train it effectively with gradient descent
  - A gradient of 0: learning algorithm cannot be guided

Binary Cross-Entropy

Sigmoid Units

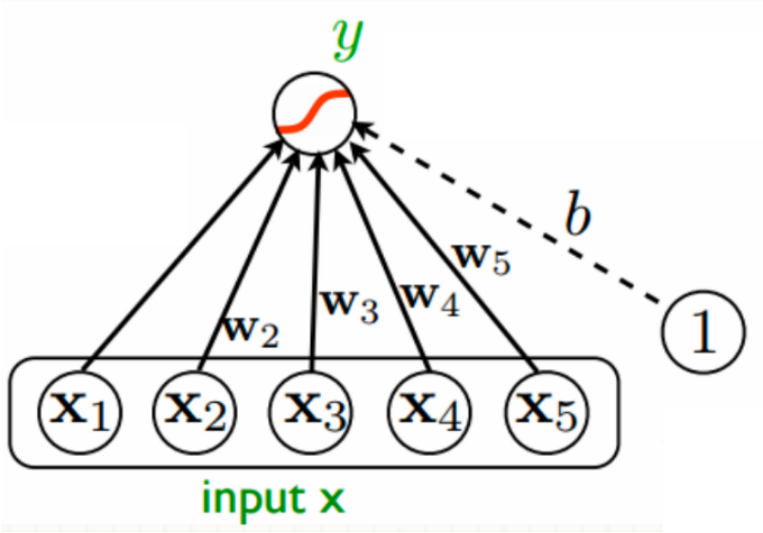


➢ Sigmoid always gives a gradient

$\hat{y} = \sigma(\boldsymbol{w}^T \boldsymbol{h} + b)$

with

$$\sigma(x) = \frac{1}{1 + \exp(-x)} = \frac{e^x}{1 + e^x}$$



➢ Sigmoid Unit has two components:

- ➢ A linear layer to compute

$z = \boldsymbol{w}^T \boldsymbol{h} + b$

- ➢ A sigmoid activation function to convert z into a probability

➢ You will understand these slides later ☺.

Softmax Units for General Classification

➢ Any time we want a probability distribution over a discrete variable with n values we may use the softmax function

➢ Softmax most often used for output of classifiers to represent distribution over n classes

➢ Also, inside the model itself when we wish to choose between one of n options

➢ For the Binary case we have produced a single number

$$\hat{y} = P(y = 1 \mid \boldsymbol{x})$$

➢ Now, we have to produce a vector $\hat{\boldsymbol{y}}$

$$\hat{y}_i = P(y = i \mid \boldsymbol{x})$$

Cross-entropy Loss

➢ You will understand these slides later ☺.

Softmax Defintion

➢ The **softmax function** (aka normalized exponential), is defined as follows:

$$\text{softmax}\left(a_1, \ldots, a_K\right) = \left(\frac{\exp a_1}{\sum_j^K \exp a_j}, \frac{\exp a_2}{\sum_j^K \exp a_j}, \ldots, \frac{\exp a_K}{\sum_j^K \exp a_j} \right)$$

➢ We will learn later more about this function

➢ But it should be visible that it creates a valid distribution
all values are between 0 and 1 and all values sum up to 1

➢ You will understand these slides later ☺.

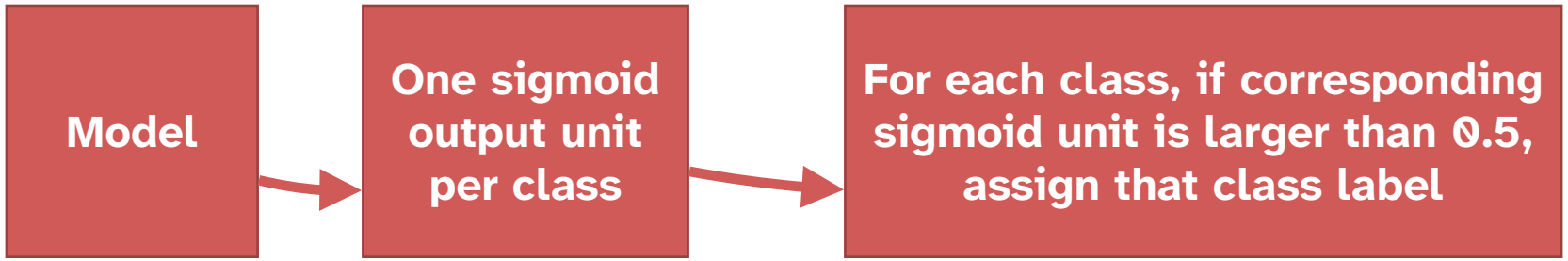
Softmax, even for binary classification

Softmax way often works better in practice!

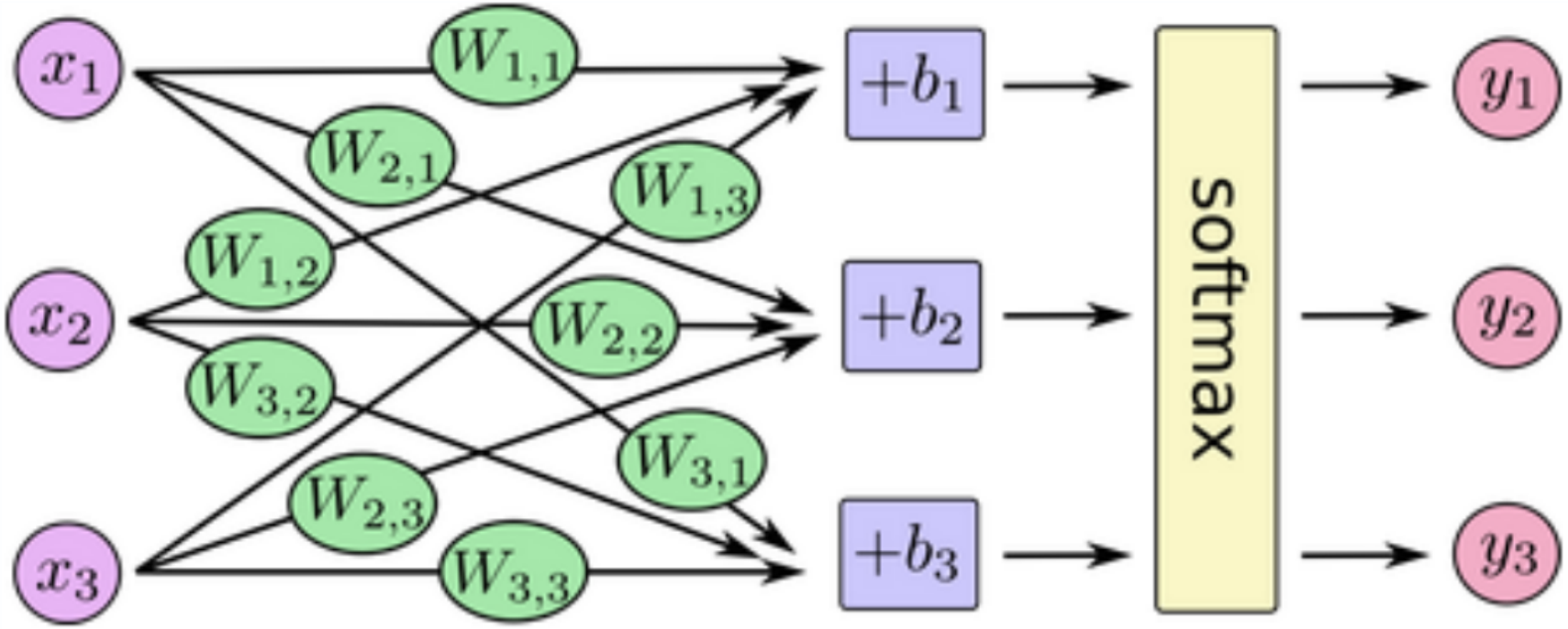
But when do I need sigmoid output units then?

- **Prime example: Multi-label classification**
- Consider the case that you have **M** possible classes.
- And multiple class labels can be assigned to a single example.
- For example, you have a news article, and you need to classify it into topics {Sports, Football, Basketball, Europe, Denmark, Germany, Politics, Economics, Science}

  → **multiple can be correct**
- "A **scientific** investigation of the **football** match between **Denmark** and **Germany**"



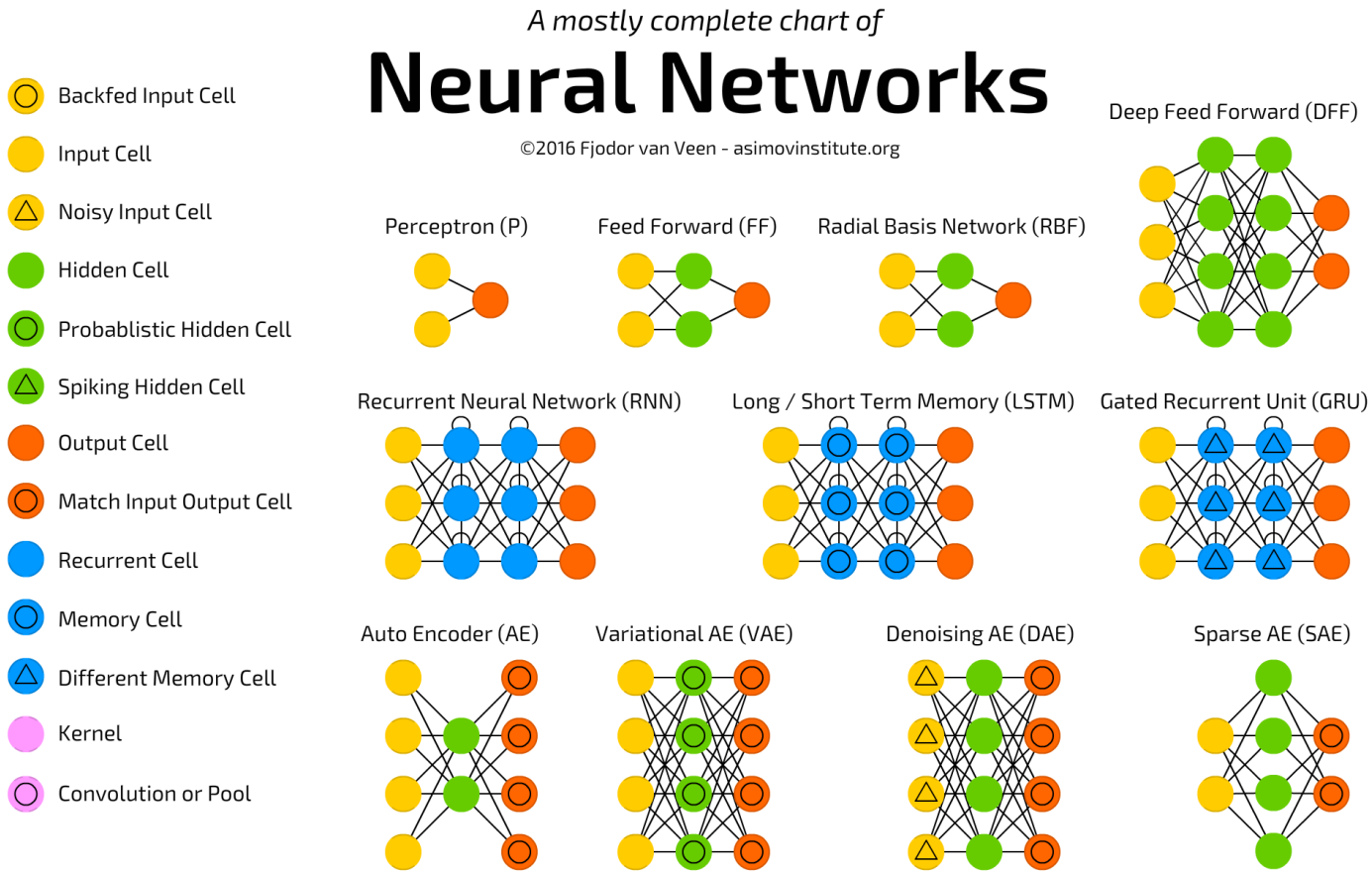
Softmax Regression



Keywords

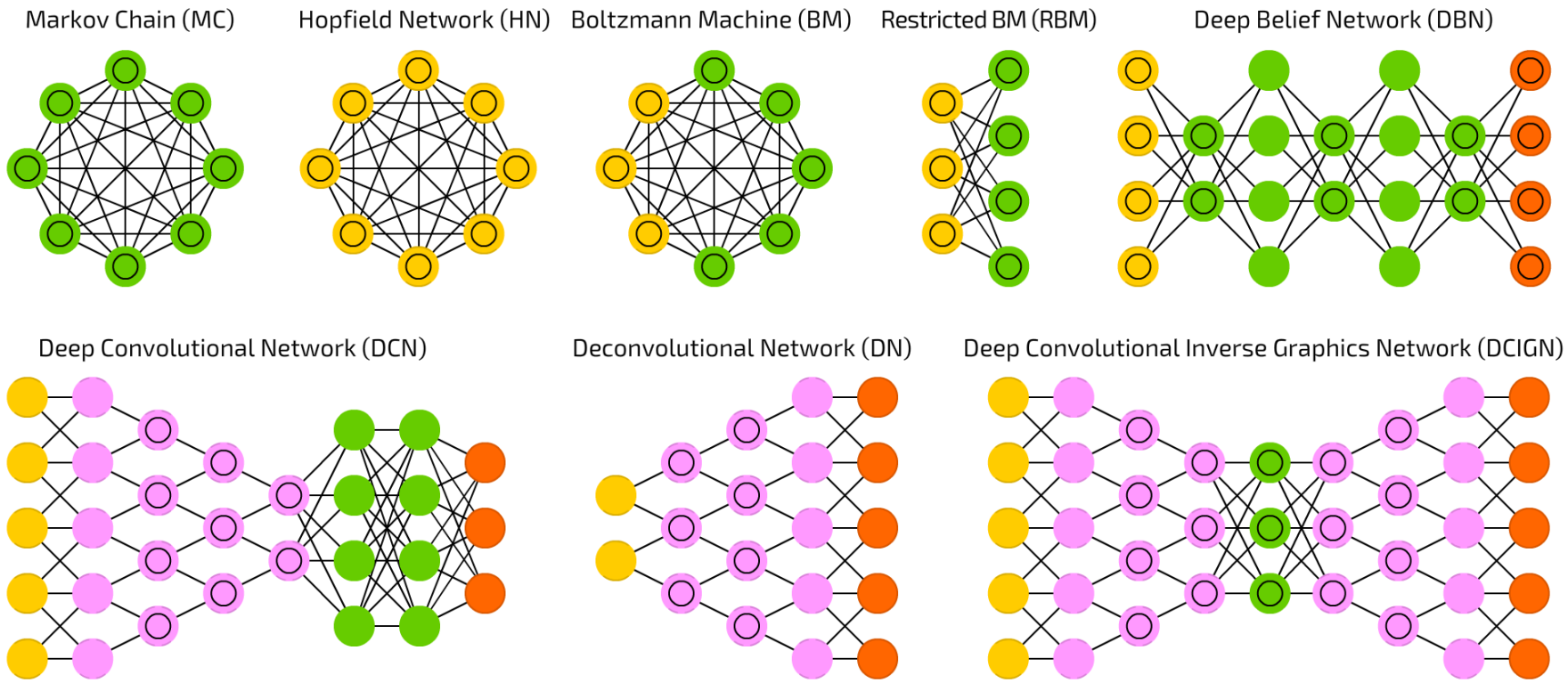
- Single-Layer Networks
- Deep Neural Networks
- Universal Approximation Theorem
- Activation functions
- Output units

A mostly complete chart of Neural Networks



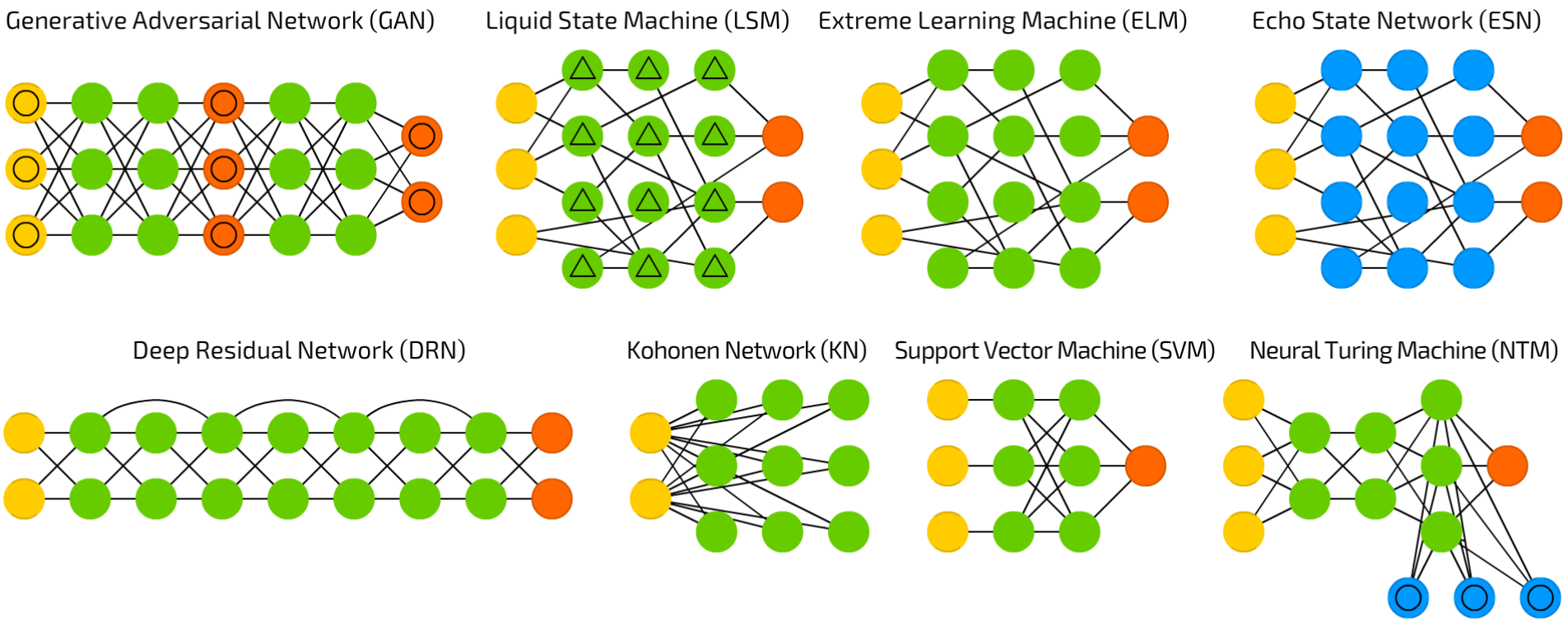
➢ https://towardsdatascience.com/the-mostly-complete-chart-of-neural-networks-explained-3fb6f2367464

A mostly complete chart of Neural Networks



➢ https://towardsdatascience.com/the-mostly-complete-chart-of-neural-networks-explained-3fb6f2367464

A mostly complete chart of Neural Networks

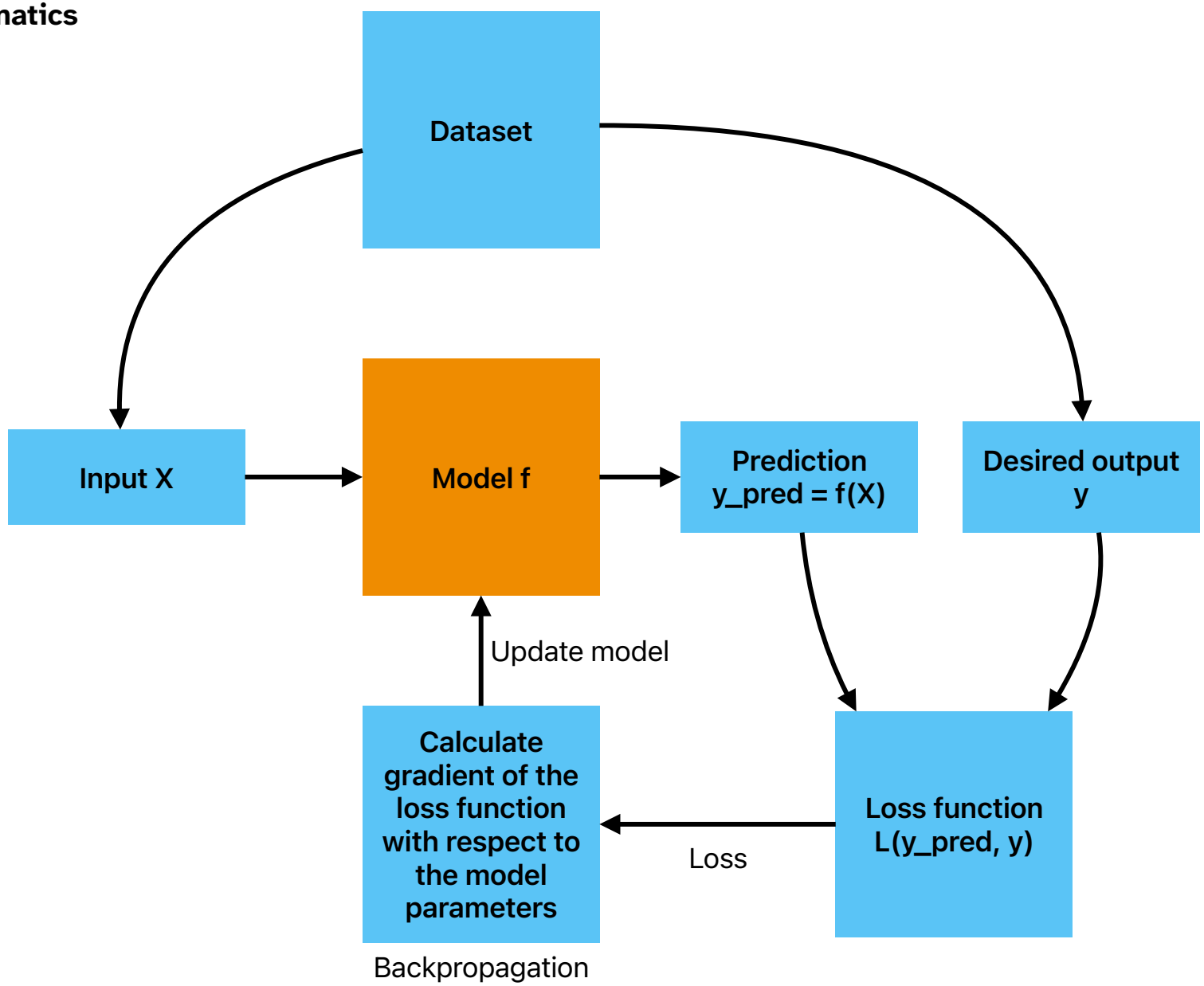


➢ https://towardsdatascience.com/the-mostly-complete-chart-of-neural-networks-explained-3fb6f2367464

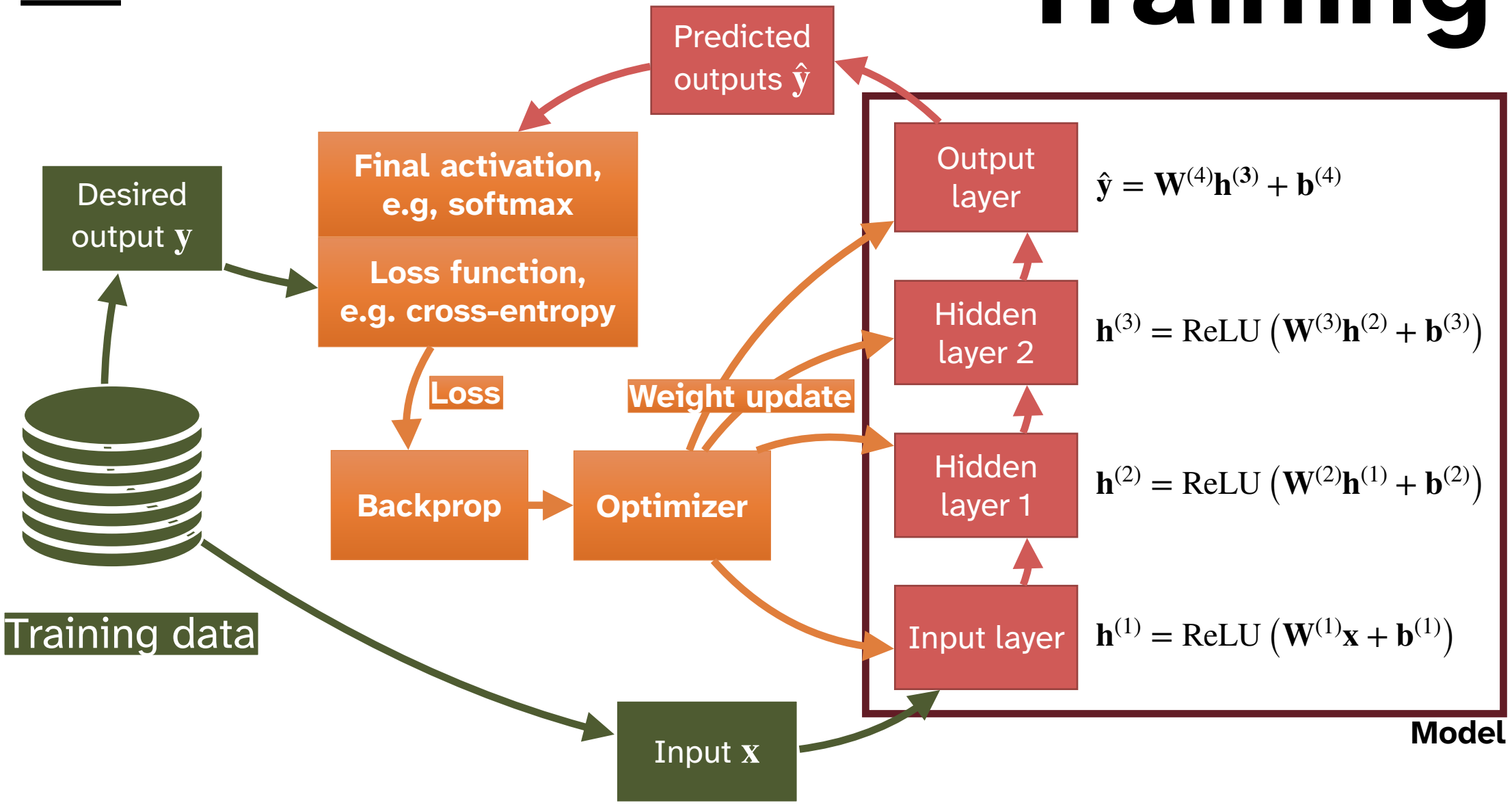
Main Architectural Considerations

1. Choice of depth of network
2. Choice of width of each layer

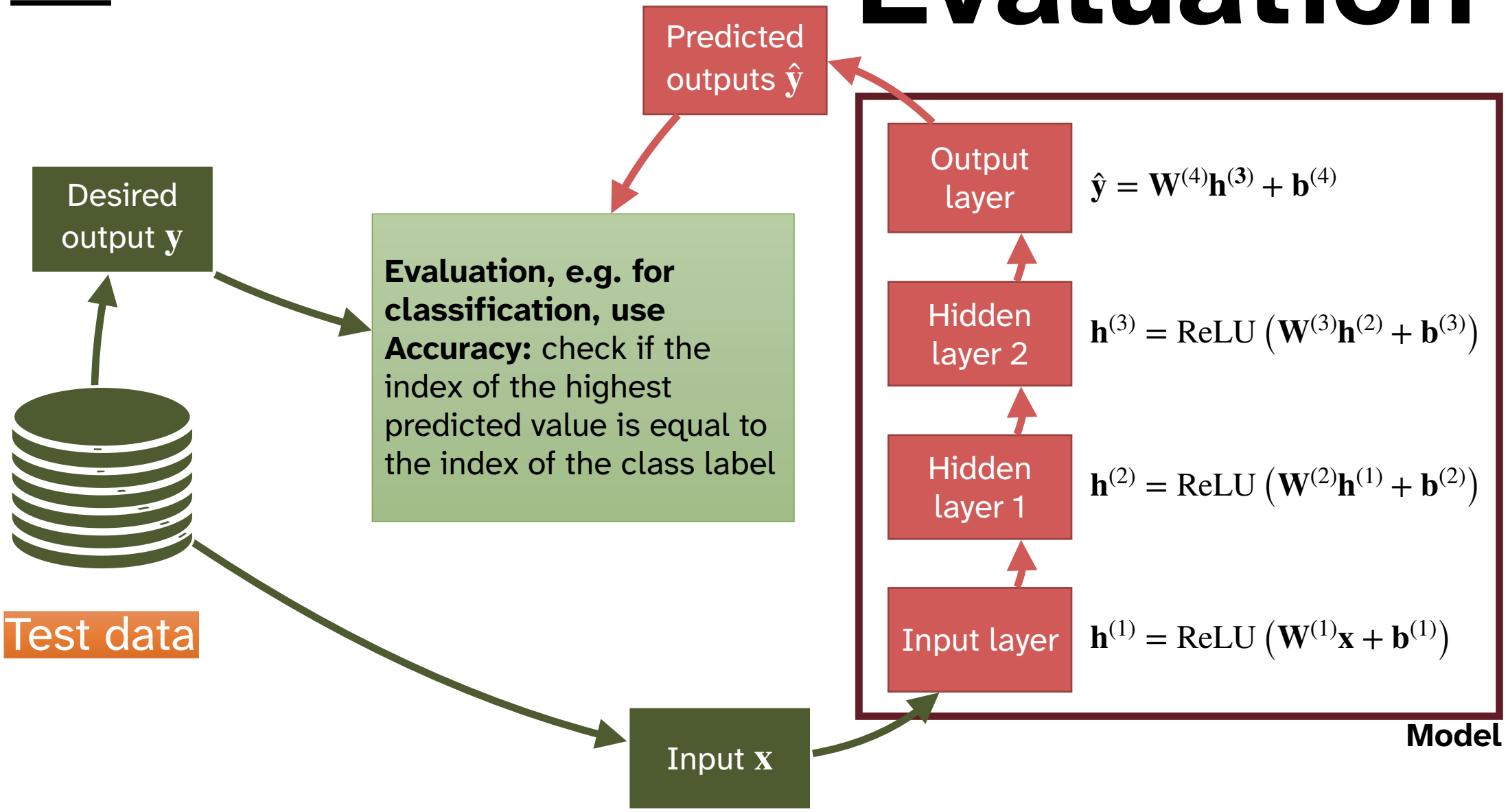
- Deeper networks have
  - Far fewer units in each layer
  - Far fewer parameters
  - Often generalize well to the test set
  - But are often more difficult to optimize
- Ideal network architecture must be found via experimentation guided by validation set error


Dataset
Input X
Model f
Prediction
y_pred = f(X)
Desired output
y
Update model
Calculate
gradient of the
loss function
with respect to
the model
parameters
Loss function
L(y_pred, y)
Loss
Backpropagation

Training


Predicted outputs $\hat{\mathbf{y}}$
Final activation, e.g, softmax
Loss function, e.g. cross-entropy
Desired output $\mathbf{y}$
Loss
Weight update
Backprop
Optimizer
Training data
Input $\mathbf{x}$
Output layer
Hidden layer 2
Hidden layer 1
Input layer
$\hat{\mathbf{y}} = \mathbf{W}^{(4)}\mathbf{h}^{(3)} + \mathbf{b}^{(4)}$
$\mathbf{h}^{(3)} = \mathrm{ReLU}\left(\mathbf{W}^{(3)}\mathbf{h}^{(2)} + \mathbf{b}^{(3)}\right)$
$\mathbf{h}^{(2)} = \mathrm{ReLU}\left(\mathbf{W}^{(2)}\mathbf{h}^{(1)} + \mathbf{b}^{(2)}\right)$
$\mathbf{h}^{(1)} = \mathrm{ReLU}\left(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}\right)$
Model

Evaluation


Predicted outputs $\hat{\mathbf{y}}$
Desired output $\mathbf{y}$
Evaluation, e.g. for classification, use Accuracy: check if the index of the highest predicted value is equal to the index of the class label
Output layer
$\hat{\mathbf{y}} = \mathbf{W}^{(4)}\mathbf{h}^{(3)} + \mathbf{b}^{(4)}$
Hidden layer 2
$\mathbf{h}^{(3)} = \text{ReLU}\left(\mathbf{W}^{(3)}\mathbf{h}^{(2)} + \mathbf{b}^{(3)}\right)$
Hidden layer 1
$\mathbf{h}^{(2)} = \text{ReLU}\left(\mathbf{W}^{(2)}\mathbf{h}^{(1)} + \mathbf{b}^{(2)}\right)$
Input layer
$\mathbf{h}^{(1)} = \text{ReLU}\left(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}\right)$
Model
Test data
Input $\mathbf{x}$

Training loop (pseudo-PyTorch)


```
# […] model & training_set defined
loss_function = nn.CrossEntropyLoss() # define the loss
function, Softmax activation included
optimizer = optim.SGD(model.parameters(), lr=0.001)  #
Connects your optimizer with parameters
# Inside training loop
For each x with label y in training_set:
  model.zero_grad() # Resets gradient to zero - important!
  y_hat = model(x)  # Feed data into the model
  loss = loss_function(y_hat, y) # Calculate the loss
  loss.backward()  # Back-propagate the error
  optimizer.step()  # Update model weights
```

Coding example

✳ example_xor.py

Take-home

- Linear models – even multi-layered – are not enough to solve a simple XOR problem.
- **Universal Approximation**; already with a few hidden layers and nonlinearities, a neural network model can **approximate any compact function to an arbitrary degree of accuracy**
- **Loss function** and **output units** are tightly coupled and typically determined by the task
- **Hidden units**: Rectified Linear Units are a good choice, yet there are some variants that tackle its shortcomings (dead ReLU's)
- **Depth** and **Width** of a neural network are important for its performance
- **The basic training loop – will be picked up again in the exercises**

Keywords

- Single-Layer Networks
- Deep Neural Networks
- Universal Approximation Theorem
- Activation functions
- Output units

How’s my teaching?

https://www.admonymous.co/lukasgalke

Department of Mathematics
and Computer Science

sdu.dk

#sdudk

Advanced Machine Learning

Regularization

Lukas Galke Poech

Spring 2026

SDU

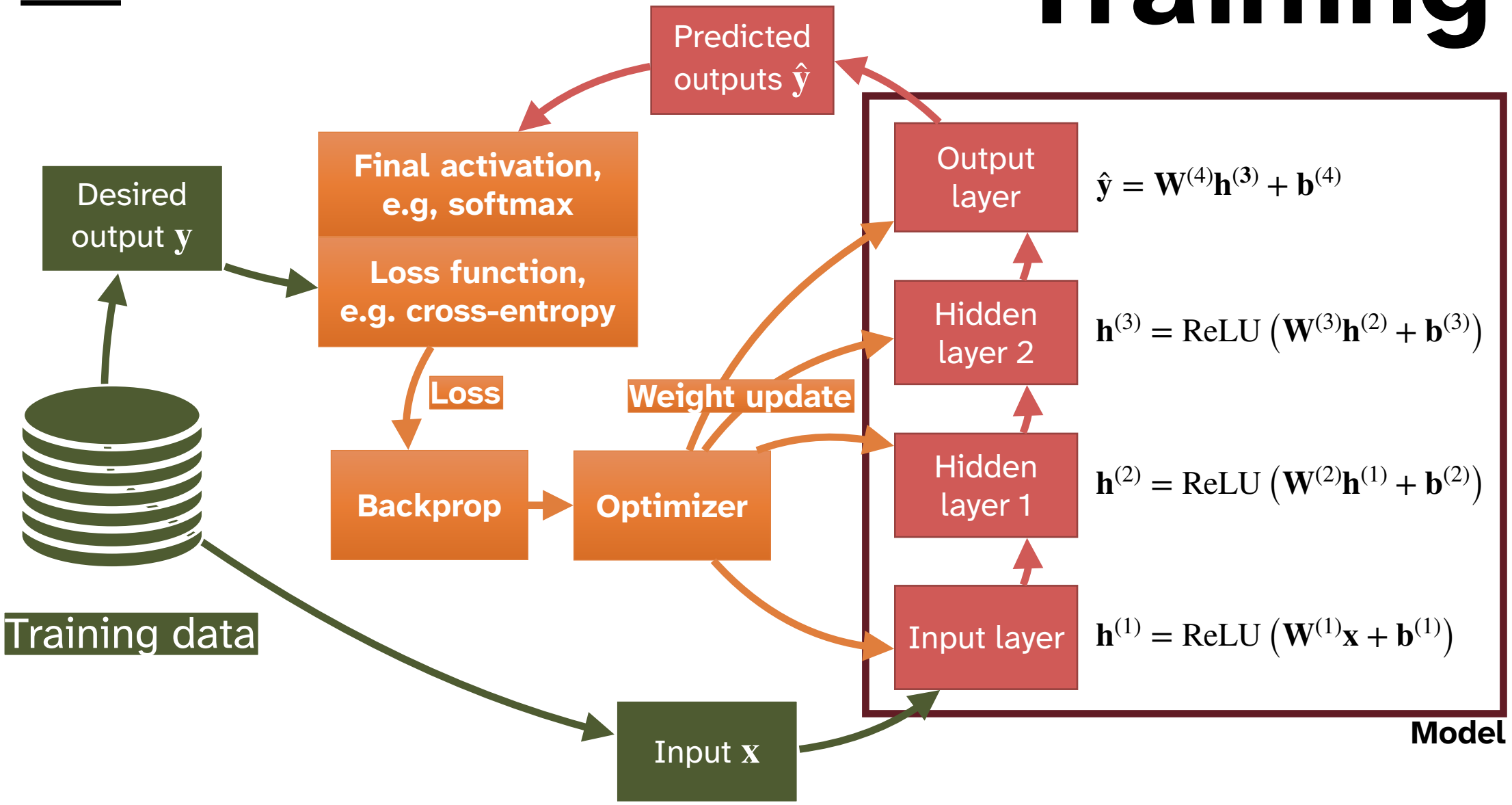
Keywords

- Bias-Variance Trade-off
- Regularization
- Model Capacity
- Invariance/equivariance
- Dropout

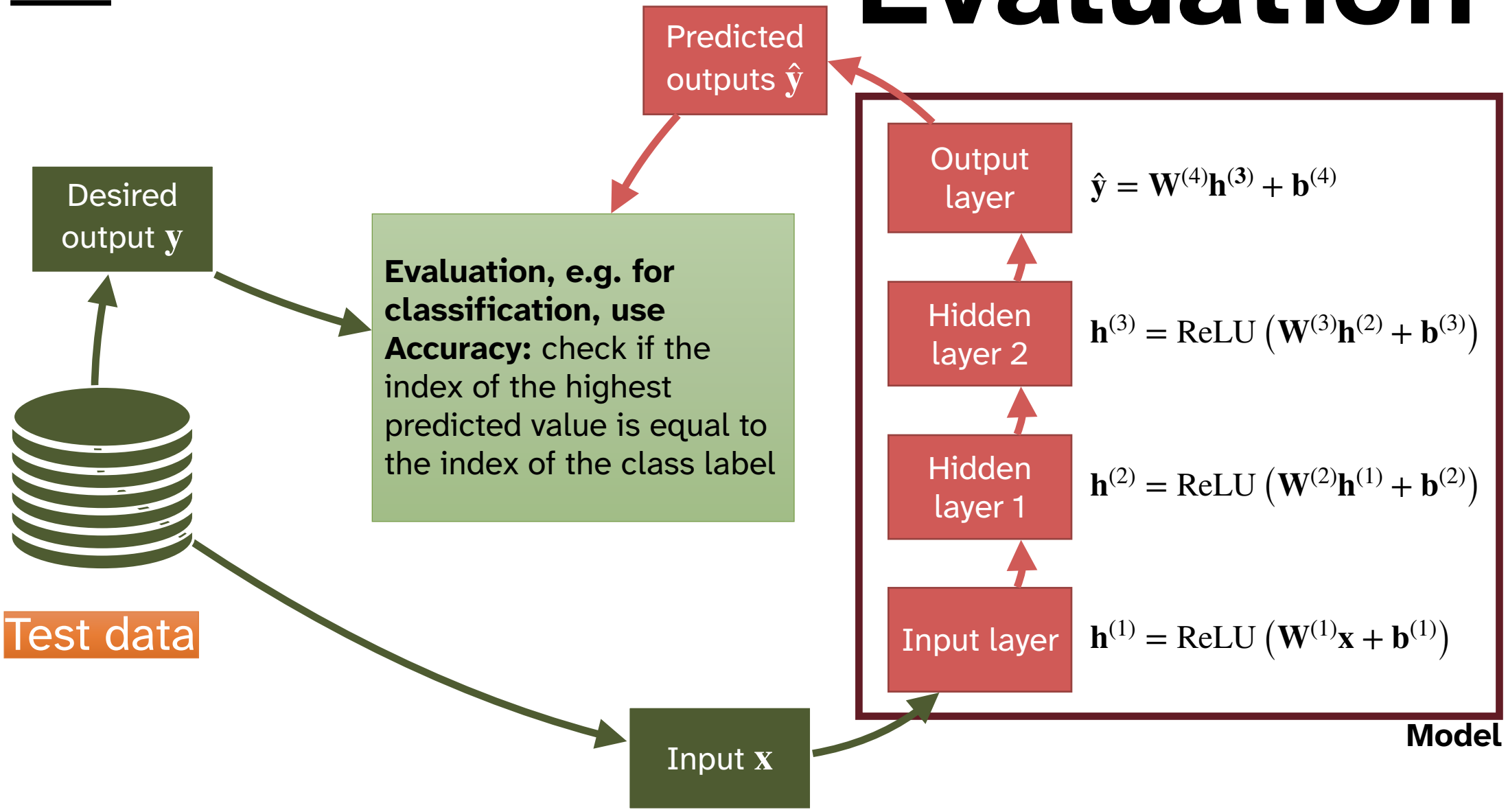
Recap

- Linear models – even multi-layered – are not enough to solve a simple XOR problem.
- **Nonlinear activation functions** allow us to model more complex functions
- **Universal Approximation**; already with a few hidden layers and nonlinearities, a neural network model can **approximate any compact function to an arbitrary degree of accuracy, depending on the width of the network.**
- So, can we just make models very large and we're good?

Training


Predicted outputs $\hat{\mathbf{y}}$
Final activation, e.g, softmax
Loss function, e.g. cross-entropy
Desired output $\mathbf{y}$
Loss
Weight update
Backprop
Optimizer
Training data
Input $\mathbf{x}$
Output layer
Hidden layer 2
Hidden layer 1
Input layer
$\hat{\mathbf{y}} = \mathbf{W}^{(4)}\mathbf{h}^{(3)} + \mathbf{b}^{(4)}$
$\mathbf{h}^{(3)} = \mathrm{ReLU}\left(\mathbf{W}^{(3)}\mathbf{h}^{(2)} + \mathbf{b}^{(3)}\right)$
$\mathbf{h}^{(2)} = \mathrm{ReLU}\left(\mathbf{W}^{(2)}\mathbf{h}^{(1)} + \mathbf{b}^{(2)}\right)$
$\mathbf{h}^{(1)} = \mathrm{ReLU}\left(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}\right)$
Model

Evaluation


Predicted outputs $\hat{\mathbf{y}}$
Desired output $\mathbf{y}$
Evaluation, e.g. for classification, use Accuracy: check if the index of the highest predicted value is equal to the index of the class label
Output layer
$\hat{\mathbf{y}} = \mathbf{W}^{(4)}\mathbf{h}^{(3)} + \mathbf{b}^{(4)}$
Hidden layer 2
$\mathbf{h}^{(3)} = \text{ReLU}\left(\mathbf{W}^{(3)}\mathbf{h}^{(2)} + \mathbf{b}^{(3)}\right)$
Hidden layer 1
$\mathbf{h}^{(2)} = \text{ReLU}\left(\mathbf{W}^{(2)}\mathbf{h}^{(1)} + \mathbf{b}^{(2)}\right)$
Input layer
$\mathbf{h}^{(1)} = \text{ReLU}\left(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}\right)$
Model
Test data
Input $\mathbf{x}$

~Recap: Cross Entropy Loss

Cross-Entropy measures how well one probability distribution approximates

$$H(p \mid q) = -\sum_{i}^{K} p(y_i) \cdot \log q(y_i)$$

Softmax normalizes an arbitrary vector into a probability distribution:

$$\operatorname{softmax}(\mathbf{z})_i = \frac{e^{z_i}}{\sum_j^K e^{z_j}}$$

Concretely: First **Softmax**, then **Cross-Entropy** against ground-truth.

Essential tool: KL Divergence

$$D_{\mathrm{KL}}(P\,||\,Q) = \sum_{x \in \mathcal{X}} P(x) \cdot \log \frac{P(x)}{Q(x)}$$

- EXPECTATION of the LOGARITHMIC DIFFERENCE between TWO PROBABILITY DISTRIBUTIONS P and Q, where EXPECTATION is taken over P
- Note the similarity to Cross-Entropy. *More on that later!*

KL-Bias and KL-Variance

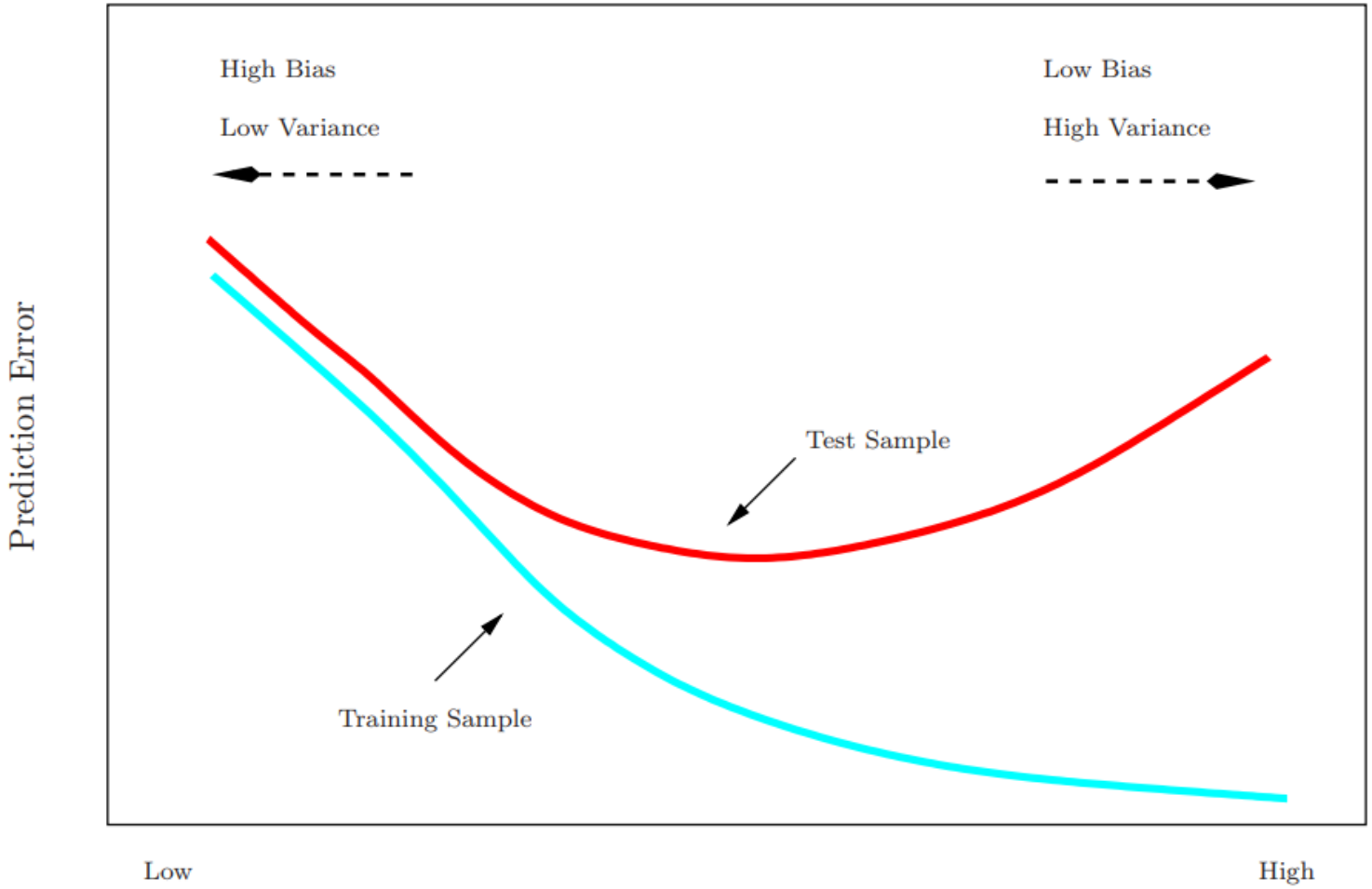
$$\underbrace{\mathbb{E}_\varepsilon\left[\mathrm{CE}(y, f_\varepsilon)\right]}_{\textsc{Error}} = \mathbb{E}_\varepsilon\left[\sum_{c=1}^{C} y[c]\log(f_\varepsilon[c])\right] = \underbrace{D_{\mathrm{KL}}\left(y\|\bar{f}\right)}_{\textsc{Bias}^2} + \underbrace{\mathbb{E}_\varepsilon\left[D_{\mathrm{KL}}(\bar{f}\|f_\varepsilon)\right]}_{\textsc{Variance}},$$

Here: f_ϵ is the model and ϵ denotes the randomness from training or from other sources of randomness at test time. $\bar{f}$ denotes the average prediction under this randomness.

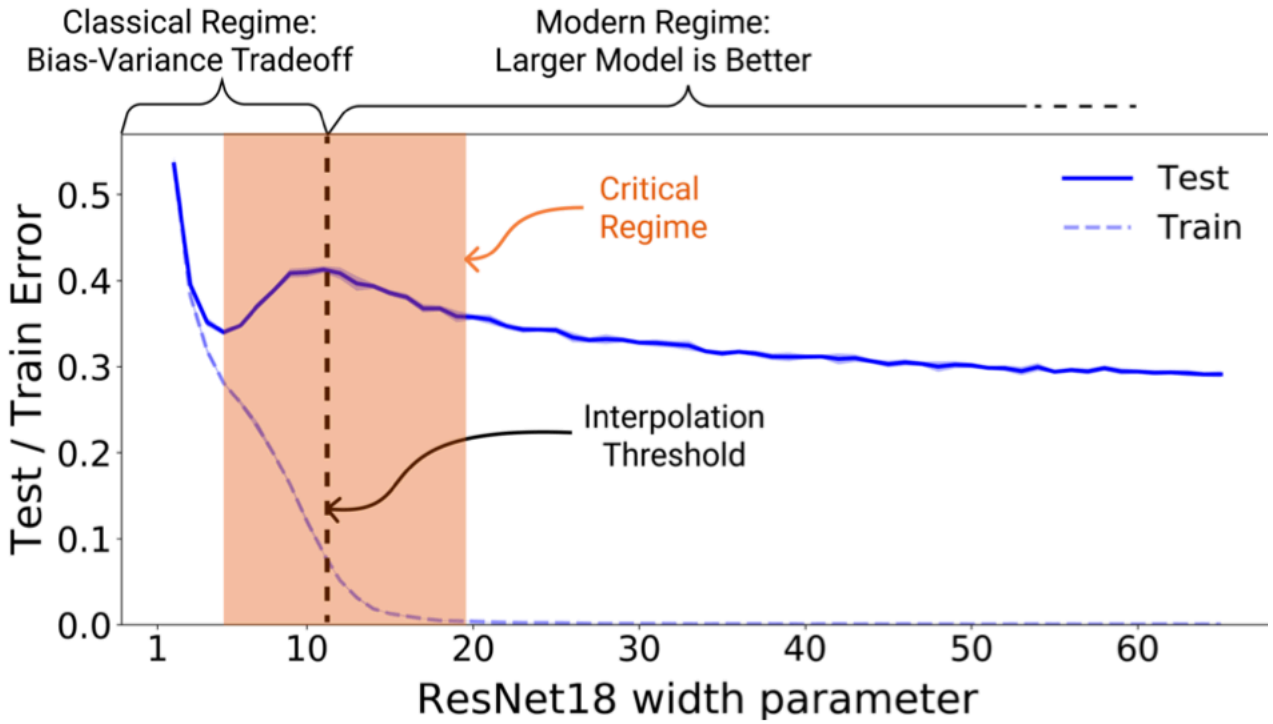
tldr; Error = Bias^2 + Variance

Hägele et al. (2026) https://arxiv.org/abs/2601.23045

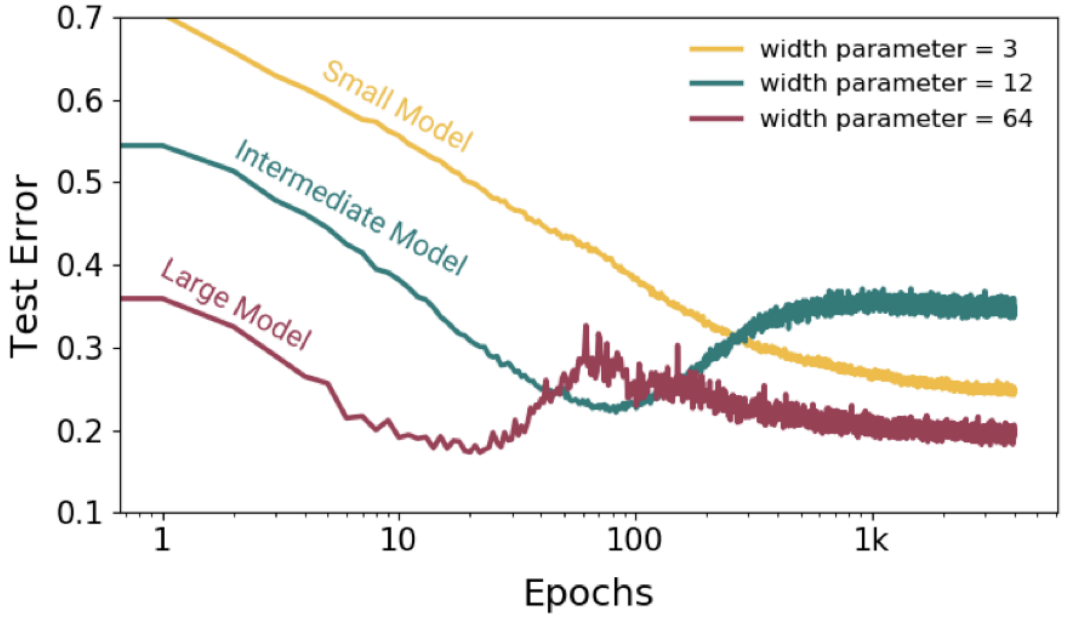
Bias Variance Trade-off



Deep Double Descent



~Model Capacity



~Training time

Source: Deep Double Descent, https://arxiv.org/abs/1912.02292

Generalization

- Central challenge of ML is that the algorithm must perform well on new, **previously unseen** inputs
- Ability to perform well on previously unobserved inputs is called **generalization**
- Normally we have a training and test dataset
- Difference to normal optimization
  - We try to minimize the error on unseen data not on the training data, e.g. in the case of a linear regression: $\hat{y} = \boldsymbol{w}^T \boldsymbol{x} + b$

$$\frac{1}{m^{(\text{test})}} \left\| X^{(\text{test})} \boldsymbol{w} - \boldsymbol{y}^{(\text{test})} \right\|_2^2$$

Estimating the generalization error

➢We optimize our parameters by minimizing the training error

$$\frac{1}{m^{(\text{train})}}\left\| X^{(\text{train})}\boldsymbol{w} - \boldsymbol{y}^{(\text{train})}\right\|_2^2$$

➢**BUT**: We are actually interested in the test error:

$$\frac{1}{m^{(\text{test})}}\left\| X^{(\text{test})}\boldsymbol{w} - \boldsymbol{y}^{(\text{test})}\right\|_2^2$$

➢**How can we affect performance when we observe only the training set?**

Under- and Over-fitting

- Factors determining how well an ML algorithm will perform are its ability to:
  1. Make the training error small
  2. Make gap between training and test errors small
- **Underfitting**
  - Inability to obtain low enough error rate on the training set
- **Overfitting**
  - Gap between training error and testing error is too large
  - We can control whether a model is more likely to overfit or underfit by altering its capacity

Capacity of a model

- Model capacity is ability to **fit variety of functions**
  - Model with low-capacity struggles to fit training set
  - A high-capacity model can overfit by memorizing properties of training set which are not useful on test set
- When a model has higher capacity, it may overfit
  - One way to control capacity of a learning algorithm is by choosing the hypothesis space
    - i.e., set of functions that the learning algorithm is allowed to select as being the solution
    - e.g., the linear regression algorithm has the set of all linear functions of its input as the hypothesis space
  - **Regularization**

Representational and Effective Capacity

Representational capacity:

➢Specifies family of functions learning algorithm can choose from

Effective capacity:

➢Imperfections in optimization algorithm can limit representational capacity

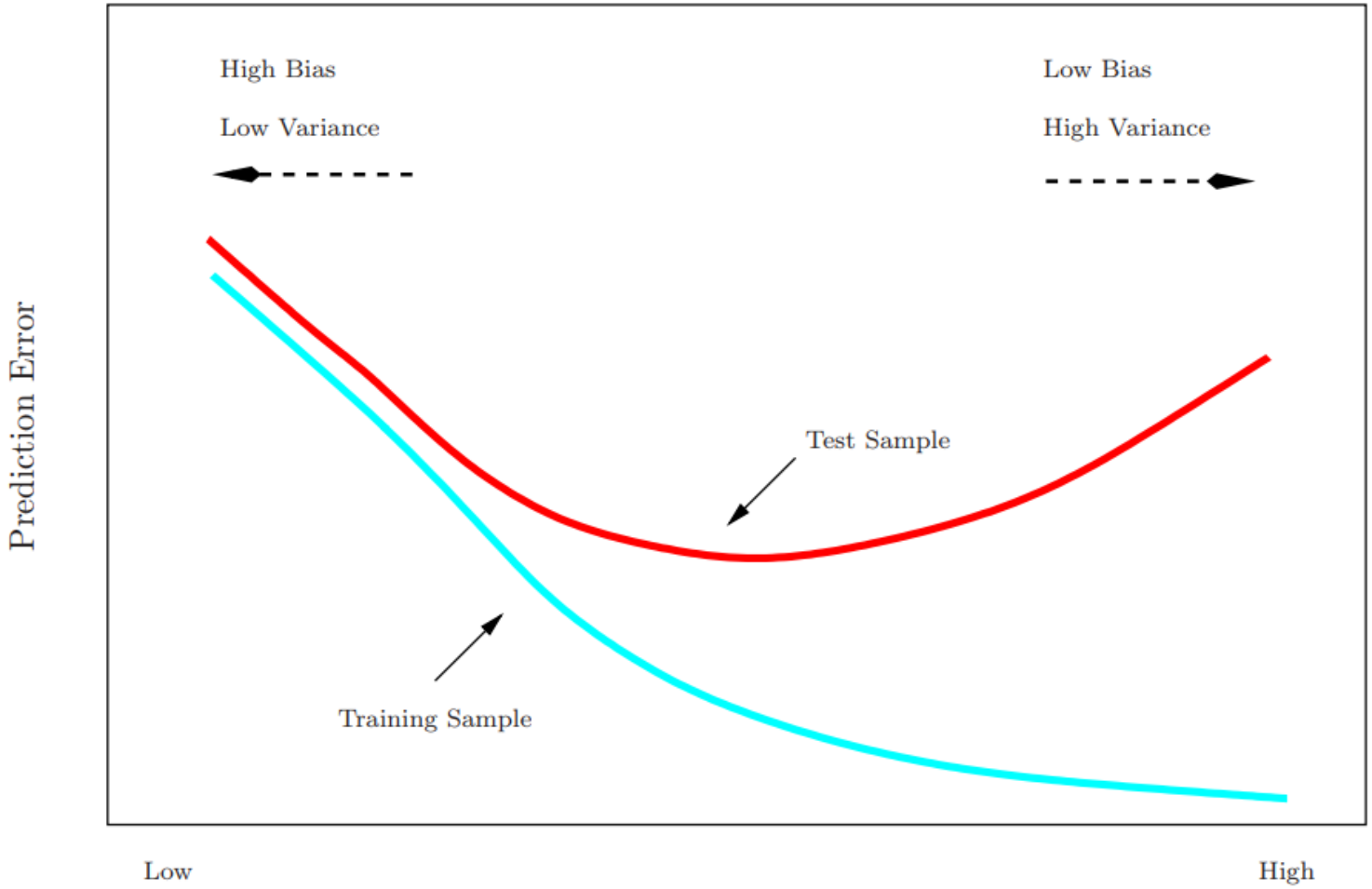
Occam's razor:

➢Among competing hypotheses that explain known observations equally well, choose the simplest one

Training Error versus Test error

- Recall the distinction between the **test error** and the **training error**:
  - The test error is the average error that results from using a statistical learning method to predict the response on a new observation, one that was not used in training the method.
  - In contrast, the training error can be easily calculated by applying the statistical learning method to the observations used in its training.
  - **But the training error rate often is quite different from the test error rate, and in particular the former can dramatically underestimate the latter.**

Training- versus Test-Set Performance



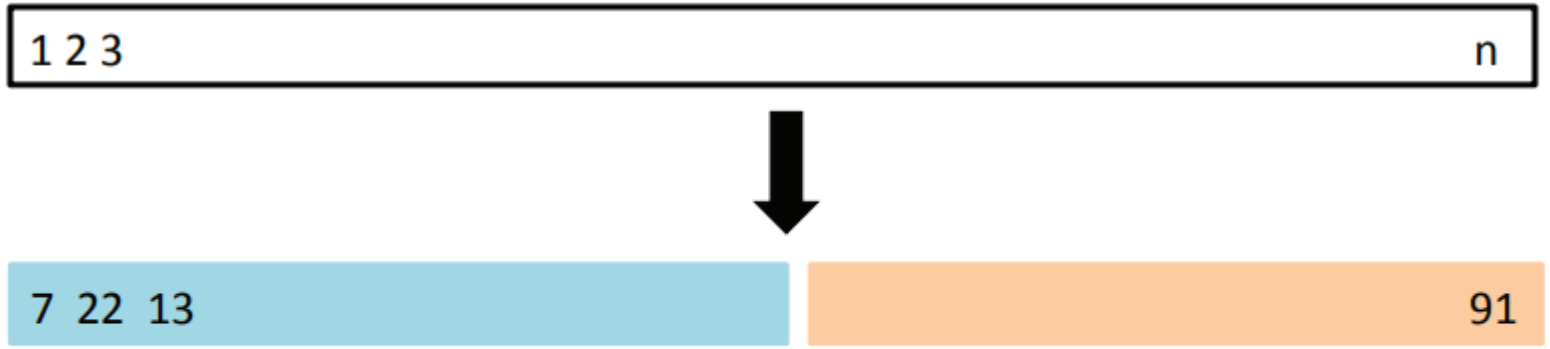
Validation-set approach

- Here we randomly divide the available set of samples into two parts: a **training set** and a **validation** or **hold-out set**.

- The model is fit on the training set, and the fitted model is used to predict the responses for the observations in the validation set.

- The resulting validation-set error provides an estimate of the test error. This is typically assessed using MSE in the case of a quantitative response and misclassification rate in the case of a qualitative (discrete) response.

Validation-set approach



A random splitting into two halves: left part is training set, right part is validation set

Drawbacks of Validation Set Approach

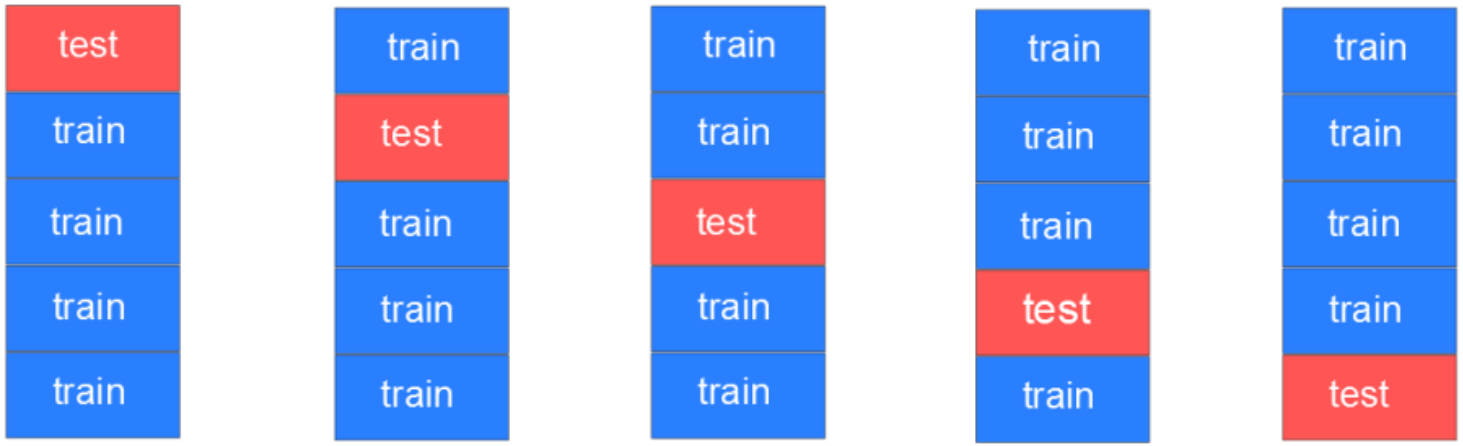
- The validation estimate of the test error can be highly variable, depending on precisely which observations are included in the training set and which observations are included in the validation set.

- In the validation approach, only a subset of the observations - those that are included in the training set rather than in the validation set - are used to fit the model.

- This suggests that the validation set error may tend to overestimate the test error for the model fit on the entire data set.

Cross-Validation

- When data set is too small, a fixed test set is problematic
- k-fold cross-validation:
  - All available data is partitioned into k groups
  - $k-1$ groups are used to train and evaluated on remaining group
  - Repeat for all k choices of held-out group
  - Performance scores from k runs are averaged



Keywords

- Bias-Variance Trade-off
- **Regularization**
- Model Capacity
- Invariance/equivariance
- Dropout

What we have seen so far

- Models can overfit easily, especially when they have high capacity
  - We have seen, that we can change the capacity of the simple models
  - E.g., just fit polynomials of lower degree

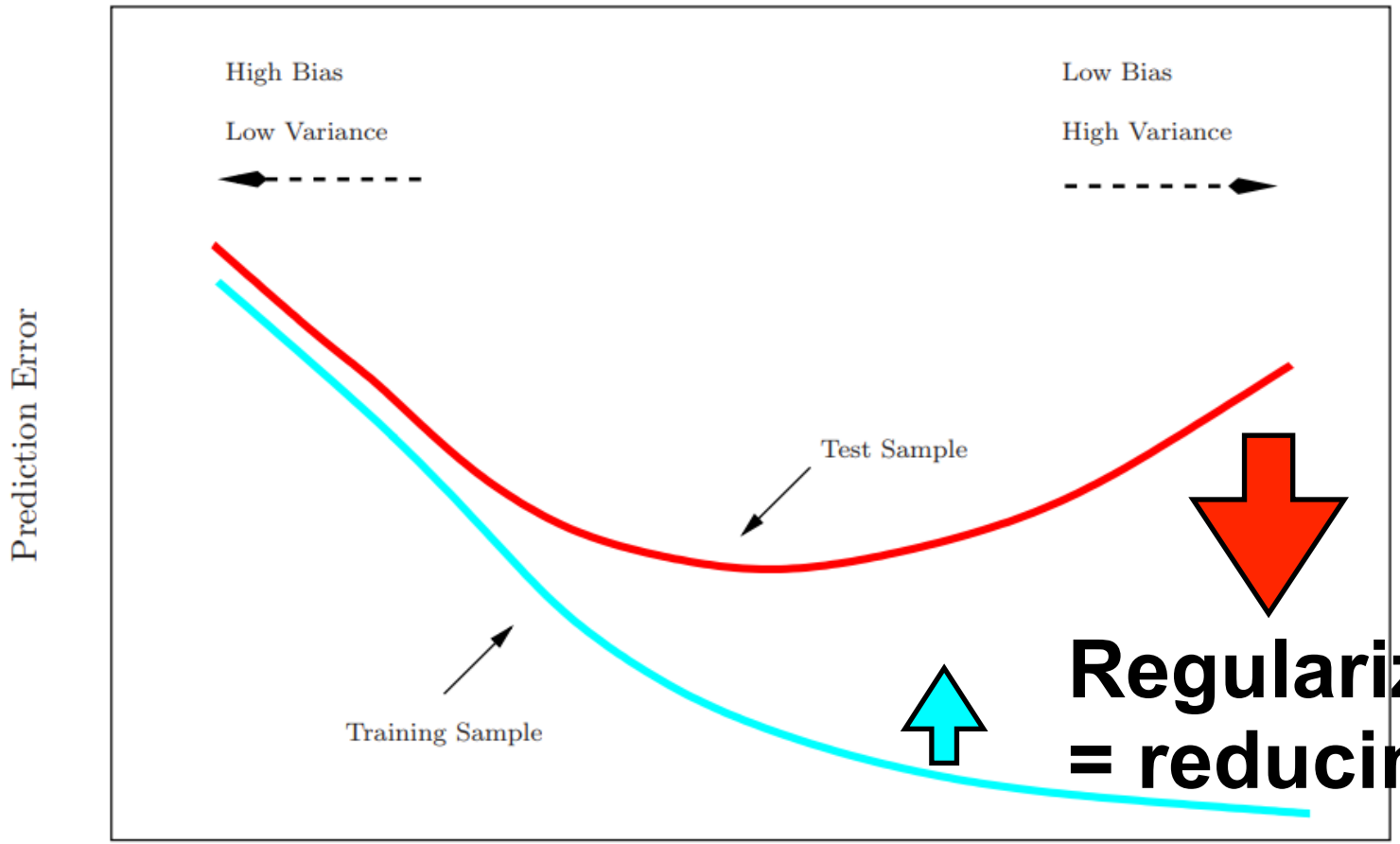
- Deep Neural Networks have very high capacity and thus we need measures to tame them → **Regularization**

What is Regularization?

- Central problem of ML is to design algorithms that will perform well not just on training data but on new inputs as well → **We want generalization!**
- **Regularization** is:
  - "any modification we make to a learning algorithm to reduce its generalization error but not its training error"
  - **Reduce test error even at the expense of (slightly) increasing training error**

- Some goals of regularization
  1. Encode prior knowledge
  2. Express preference for simpler model
  3. Needed to make underdetermined problem determined

Regularization



Regularization
= reducing overfitting

Model Types and Regularization

➢Three regimes of models:

1. Underfitting: Not capturing the data distribution well (high bias)
2. Matches the true data generating process
3. Overfitting: Capturing the data distribution too well

➢Goal of regularization is to take model from third regime to second

➢Best fitting model obtained not by finding the right number of parameters

➢Instead, **best fitting model is a large model that has been regularized appropriately**

Keywords

- Bias-Variance Trade-off
- Regularization
- **Model Capacity**
- Invariance/equivariance
- Dropout

Limiting Model Capacity

- Regularization has been used for decades prior to advent of Deep Learning
- Linear- and logistic-regression allow simple, straightforward and effective regularization strategies:
  - Adding a parameter norm penalty $\Omega(\theta)$ to the objective function J:

$$\tilde{J}(\boldsymbol{\theta}, \boldsymbol{X}, \boldsymbol{y}) = J(\boldsymbol{\theta}, \boldsymbol{X}, \boldsymbol{y}) + \alpha\Omega(\boldsymbol{\theta})$$

- with α is a hyperparameter that scales the relative contribution of the norm penalty term Ω

- Setting α to 0 results in no regularization. Larger values correspond to more regularization

Norm Penalty

- When our training algorithm minimizes the regularized objective function J and some measure of the size of the parameters θ

- Different choices of the parameter norm Ω can result in different solutions preferred

- Norm penalty Ω penalizes only weights at each layer and leaves biases unregularized
  - Biases require less data to fit than weights
  - Each bias controls only a single variable

- Let $\boldsymbol{w}$ indicate all weights affected by norm penalty, θ denotes both $\boldsymbol{w}$ and biases

L^2 parameter Regularization

- Simplest and most common kind
- Called Weight decay
- Drives weights closer to the origin by adding a regularization term to the objective function

$$\Omega(\boldsymbol{\theta}) = \frac{1}{2}\|\boldsymbol{w}\|_2^2$$

- In other communities also known as **ridge regression** or Tikhonov regularization

A closer look

➢Objective function (with no bias parameter)

$$\tilde{J}(\boldsymbol{w}; \boldsymbol{X}, \boldsymbol{y}) = \frac{\alpha}{2}\boldsymbol{w}^T\boldsymbol{w} + J(\boldsymbol{w}; \boldsymbol{X}, \boldsymbol{y})$$

➢Corresponding Gradient:

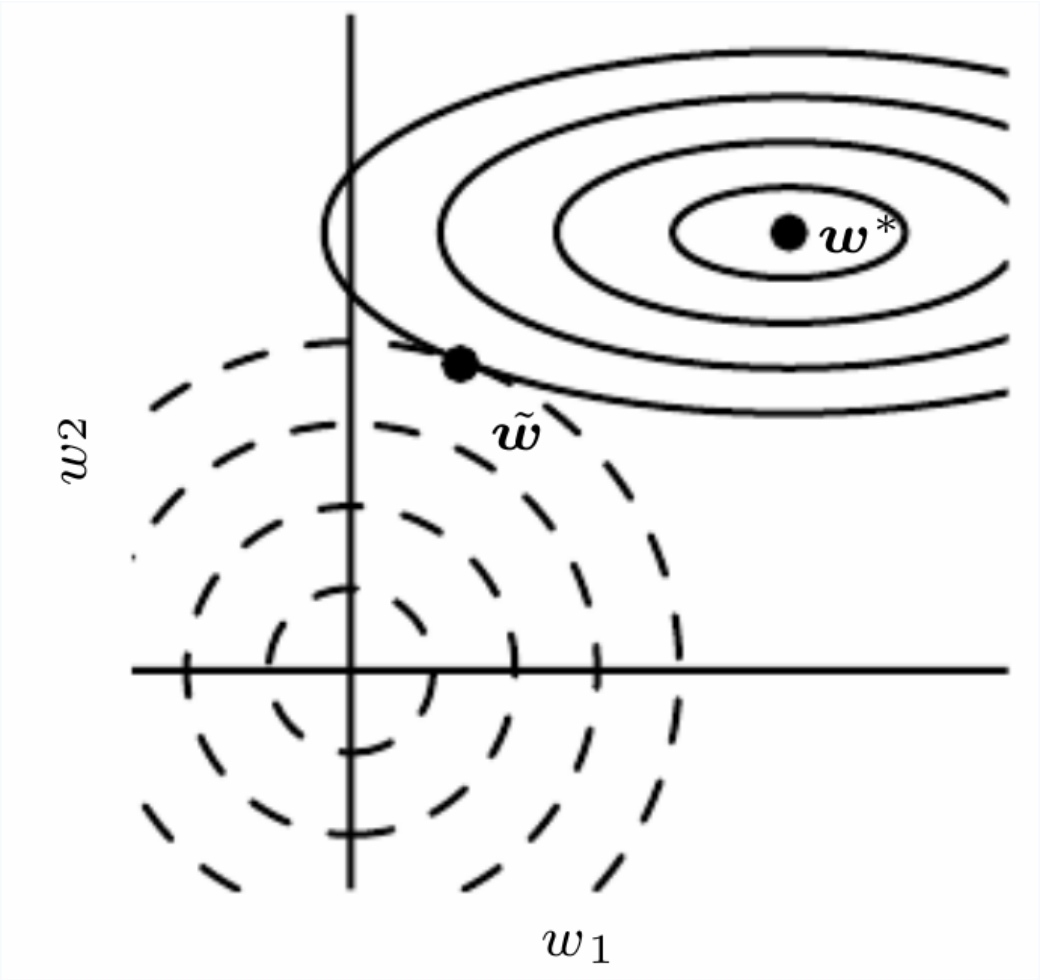
$$\nabla_{\boldsymbol{w}}\tilde{J}(\boldsymbol{w}; \boldsymbol{X}, \boldsymbol{y}) = \alpha\boldsymbol{w} + \nabla_{\boldsymbol{w}}J(\boldsymbol{w}; \boldsymbol{X}, \boldsymbol{y})$$

➢To perform single gradient step, perform update:

$$\boldsymbol{w}' \leftarrow (1 - \epsilon\alpha)\boldsymbol{w} - \epsilon\nabla_{\boldsymbol{w}}J(\boldsymbol{w}; \boldsymbol{X}, \boldsymbol{y})$$

➢We have modified learning rule to shrink $\boldsymbol{w}$ by constant factor $1 - \epsilon\alpha$ at each step

An illustration of the effect of weight decay



L^1 Regularization

➢While L^2 weight decay is the most common form of weight decay there are other ways to penalize the size of model parameters

➢L^1 regularization is defined as

$$\Omega(\boldsymbol{\theta}) = \|\boldsymbol{w}\|_1 = \sum_i |w_i|$$

➢Which is the sum of the absolute values of the individual parameters

A closer look

➢Objective function (with no bias parameter)

$$\tilde{J}(\boldsymbol{w}; \boldsymbol{X}, \boldsymbol{y}) = \alpha \|\boldsymbol{w}\|_1 + J(\boldsymbol{w}; \boldsymbol{X}, \boldsymbol{y})$$

➢Corresponding Gradient:

$$\nabla_{\boldsymbol{w}} \tilde{J}(\boldsymbol{w}; \boldsymbol{X}, \boldsymbol{y}) = \alpha \text{sign}(\boldsymbol{w}) + \nabla_{\boldsymbol{w}} J(\boldsymbol{w}; \boldsymbol{X}, \boldsymbol{y})$$

➢Where $\text{sign}(\boldsymbol{w})$ is simply the element-wise sign of $\boldsymbol{w}$

- ➢Always a strong gradient unless $w_i = 0$
- ➢Leads to a sparse solution

Sparsity and Feature Selection

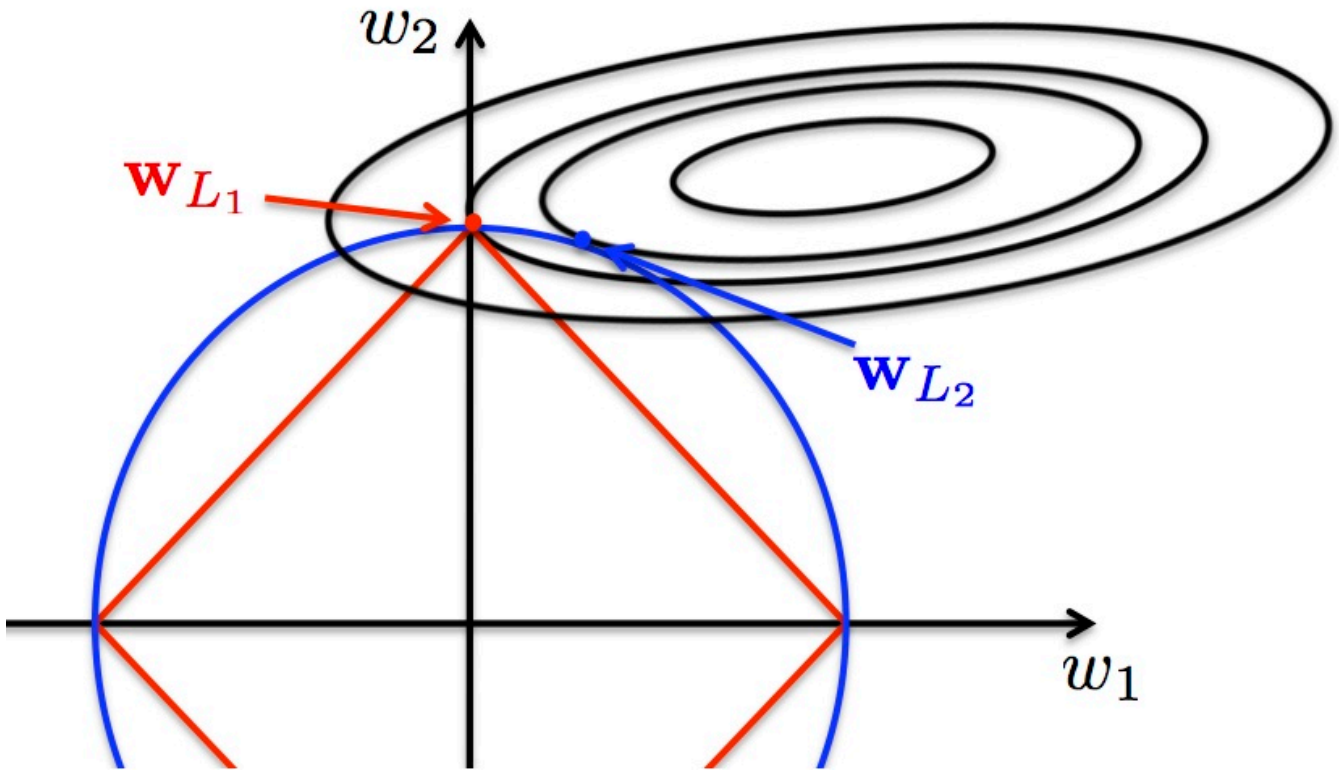
- The sparsity property induced by L^1 regularization has been used extensively as a feature selection mechanism

- Feature selection simplifies an ML problem by choosing subset of available features

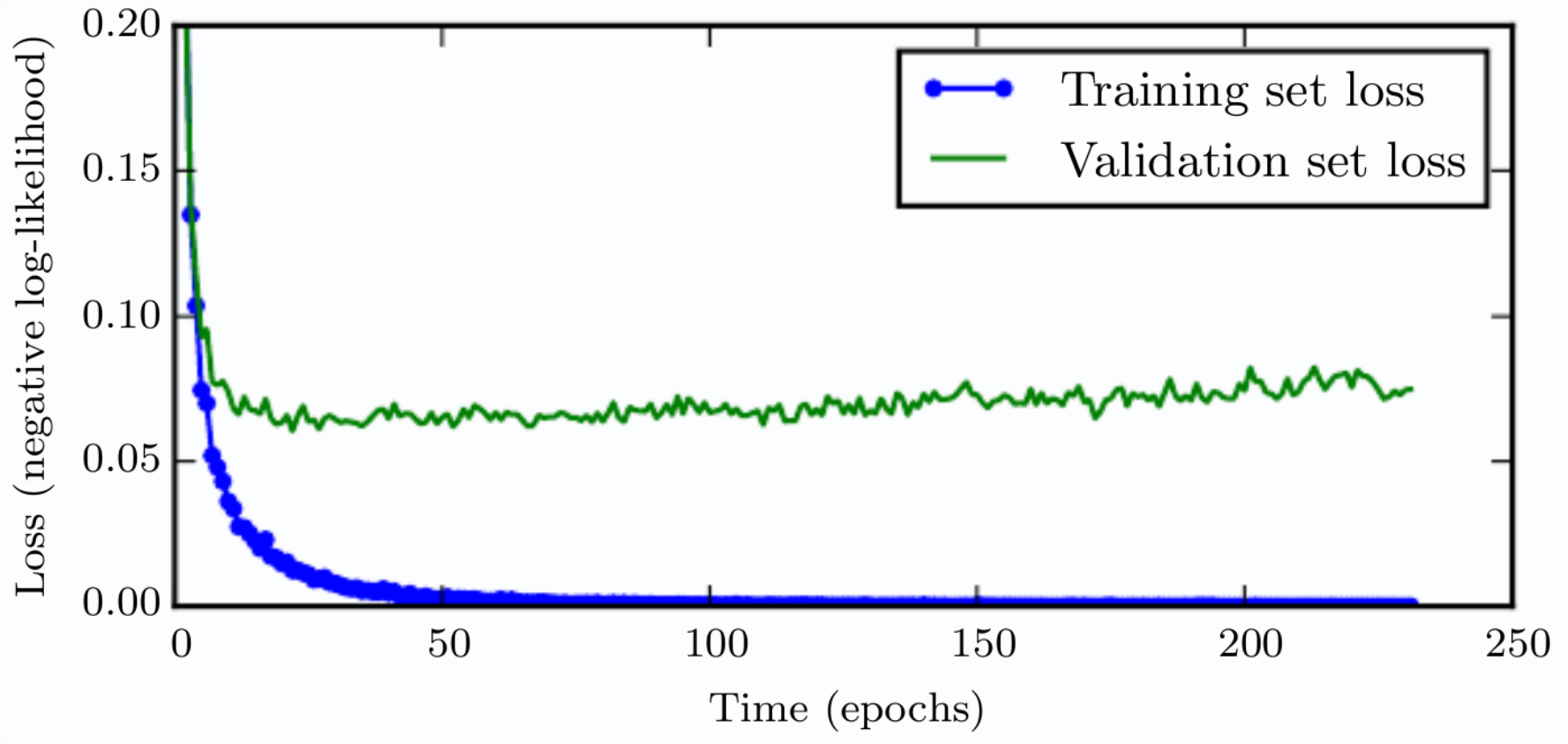
- LASSO (Least Absolute Shrinkage and Selection Operator) integrates an L^1 penalty with a linear model and least squares cost function

- The L^1 penalty causes a subset of the weights to become zero, suggesting that those features can be discarded

L^1 vs. L^2 Regularization



Typical Learning Curves



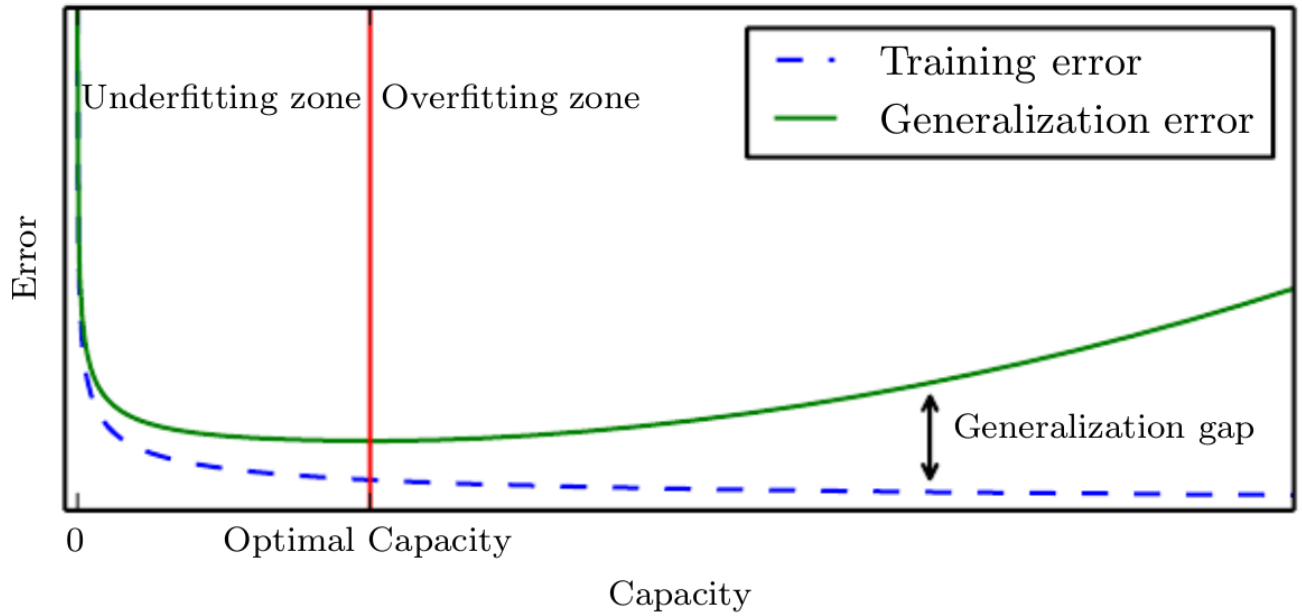
➢ In this example, we train a maxout network on MNIST. Observe that the training objective decreases consistently over time, but the validation set average loss eventually begins to increase again, forming an asymmetric U-shaped curve.

Early Stopping

- We can obtain a model with better validation set error (and thus better test error) by returning to the parameter setting at the point of time with the lowest validation set error
- Every time the error on the validation set improves, we store a copy of the model parameters.
- When the training algorithm terminates, we return these parameters, rather than the latest set
- It is the most common form of regularization in Advanced Machine Learning due to its effectiveness and its simplicity

Early Stopping as Hyperparameter Selection

- We can think of early stopping as a hyperparameter selection algorithm
  - In this view no. of training steps is just a hyperparameter
  - This hyperparameter has a U-shaped validation set performance curve
  - Most hyperparameters have such a U-shaped validation set performance curve, as seen below



Costs of Early Stopping

- Cost of this hyperparameter is running validation evaluation periodically during training
  - Ideally done in parallel to training process on a separate machine
  - Separate CPU or GPU from main training process, Or using small validation set or validating set less frequently

- Need to maintain a copy of the best parameters
  - This cost is negligible because they can be stored on a slower, larger memory
  - E.g., training in GPU, but storing the optimal parameters in host memory or on a disk drive

Early Stopping as Regularization

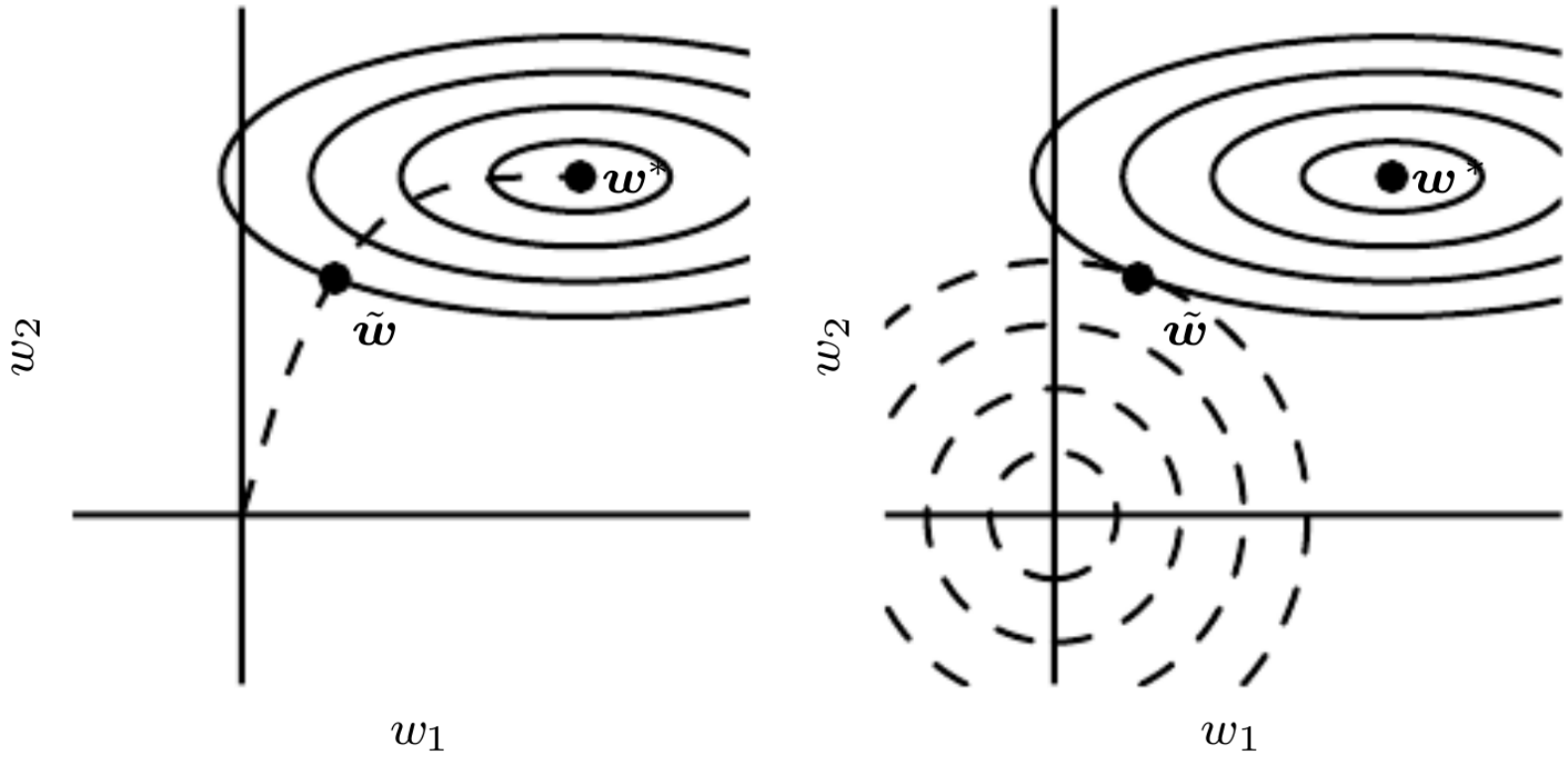
- Early stopping is an unobtrusive form of regularization
- It requires almost no change to the underlying training procedure, the objective function, or the set of allowable parameter values
- So it is easy to use early stopping without damaging the learning dynamics
- In contrast to weight decay, where we must be careful not to use too much weight decay
  - Otherwise we trap the network in a bad local minimum corresponding to pathologically small weights

Use of a second training step

- Early stopping requires a validation set
  - Thus some training data is not fed to the model
- To best exploit this extra data, one can perform extra training after the initial training with early stopping has completed.

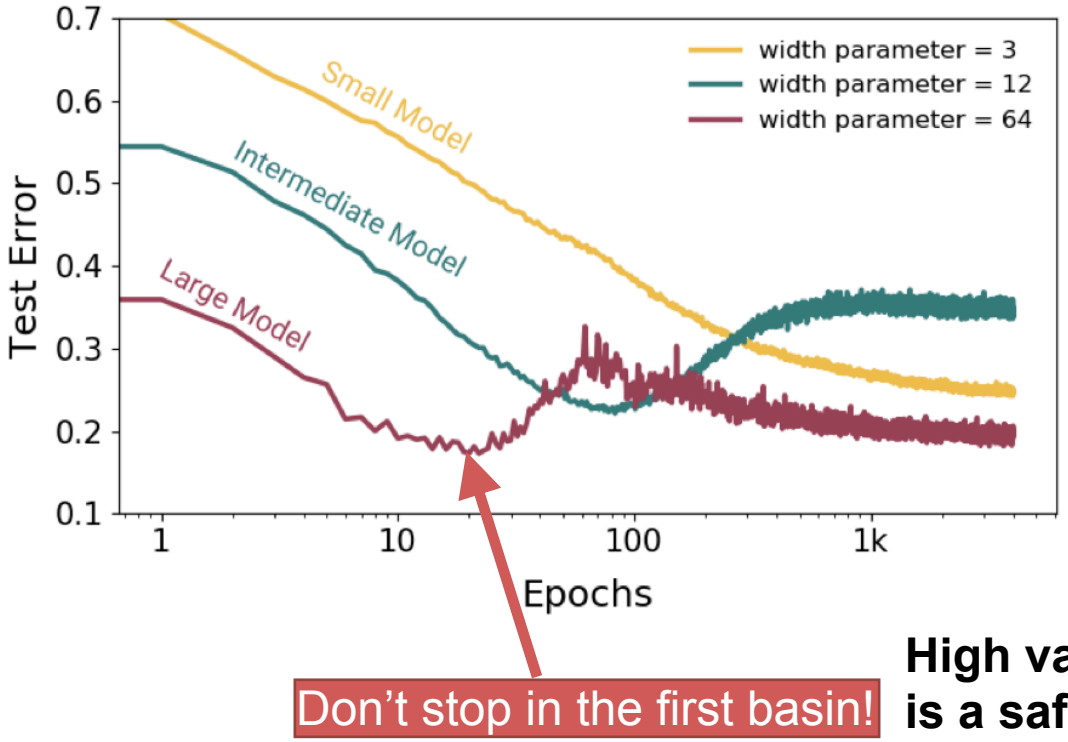
- There are two basic strategies for including the validation data
  1. Reinitialize the model to the untrained state and re-train on training AND validation sets, with the identified optimal hyperparameters – including how long to train the model (number of epochs).
  2. Continue training with the final model on the validation set (e.g., if dealing with very large models)

Early Stopping vs L^2 Regularization



- We now restrict the “distance” the weights can travel to find an optimal solution

Be patient with early stopping



High value for patience is a safe choice!

Keywords

- Bias-Variance Trade-off
- Regularization
- Model Capacity
- **Invariance/equivariance**
- Dropout

More Data is Better

- Best way to make a ML model to generalize better is to train it on more data
- In practice amount of data is limited
- **Get around the problem by creating synthesized data**
- For some ML tasks it is straightforward to synthesize data
- Especially popular for classification/object recognition

Data Augmentation for Classification

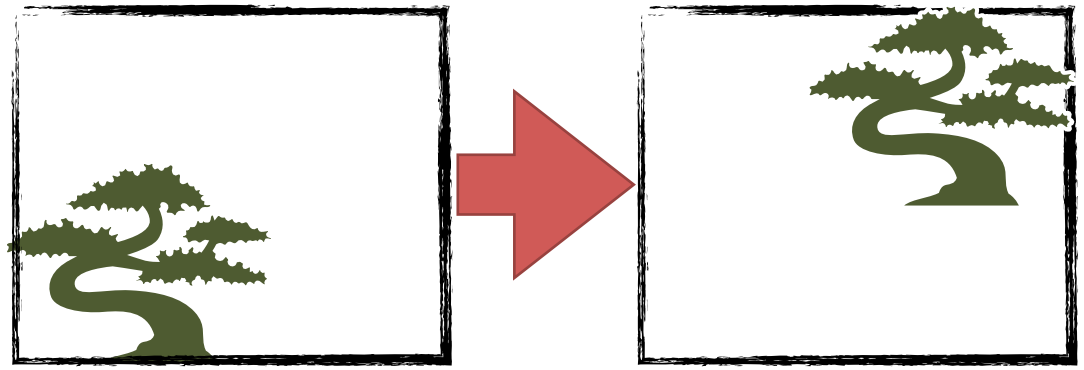
- Data augmentation is easiest for classification
  - Classifier takes high-dimensional input $\boldsymbol{x}$ and summarizes it with a single category identity y
  - Main task of classifier is to be **invariant** to a wide variety of transformations

- Generate new samples $(\boldsymbol{x}, y)$ just by transforming inputs
  - Good example: Images are high-dimensional and include a variety of variations, may easily simulated
  - Translating the images a few pixels can greatly improve performance
  - Rotating and scaling are also effective
  - It may be more difficult in other area (e.g., language)

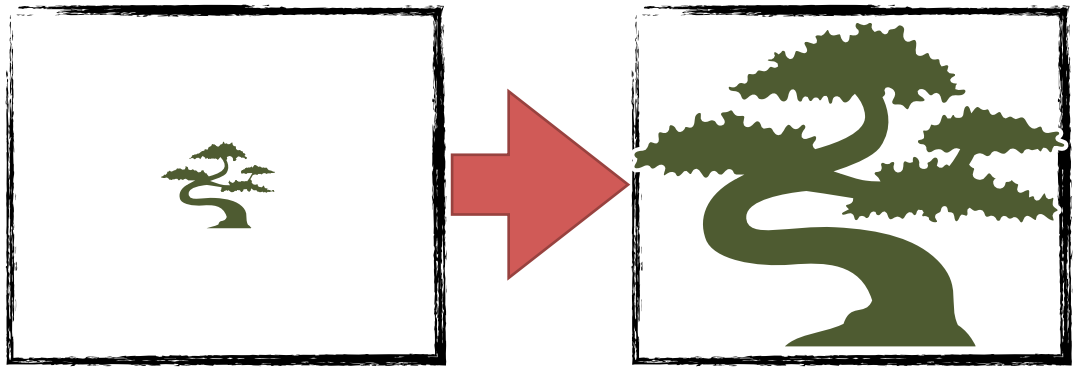
Invariance

Model output remains unchanged with respect to some transformation of the input. **S(T(I)) = S(I)**

Translation invariance



Scale invariance



Equivariance

S(T(I)) = T(S(I))

Model output changes in the same way, regardless of whether the transformation is applied before or after the model is applied.

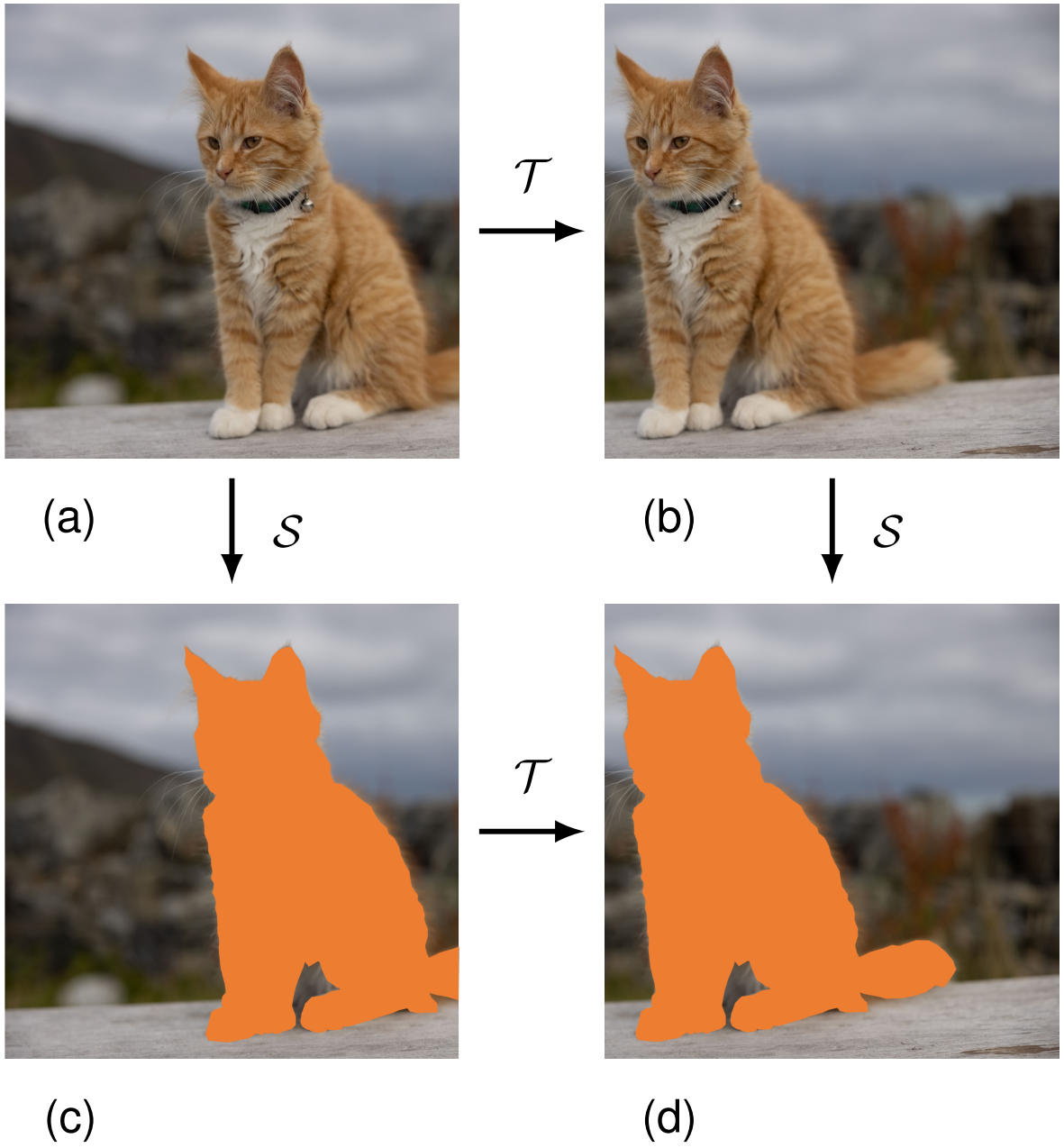
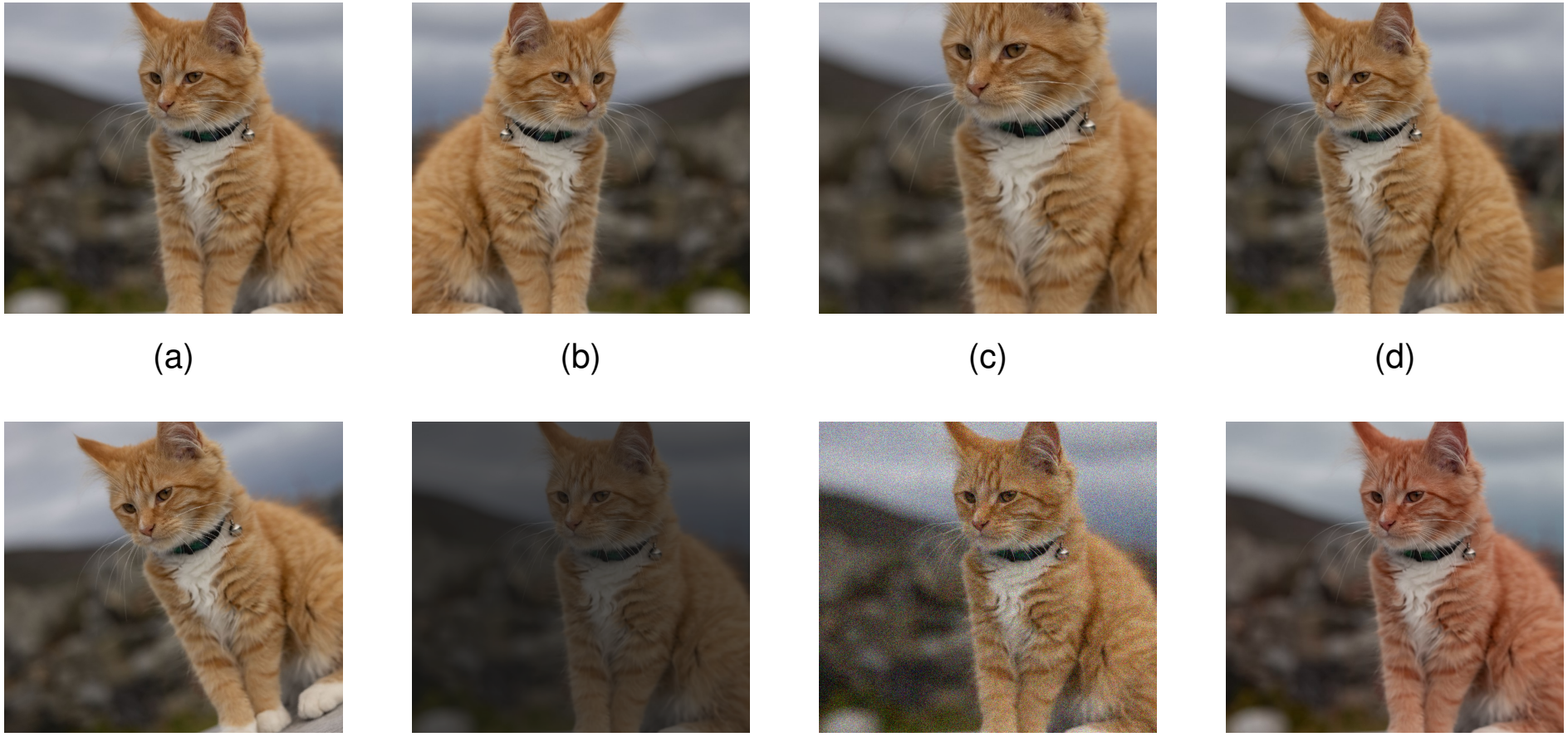


Figure from Bishop & Bishop (2024)

Injecting Noise

- Injecting noise into the input of a neural network can be seen as data augmentation
- **Neural networks are not robust to noise**
- To improve robustness, train them with random noise applied to their inputs
- Noise can also be applied to hidden units
- Dropout, a powerful regularization strategy, can be viewed as constructing new inputs by multiplying by noise (*more later*)

Commonly applied augmentation on images



(a) (b) (c) (d)

(e) (f) (g) (h)

Figure from Bishop & Bishop (2024)

Keywords

- Bias-Variance Trade-off
- Regularization
- Model Capacity
- Invariance/equivariance
- **Dropout**

What is Bagging (Bootstrap Aggregating)

- It is a technique for reducing generalization error by combining several models
  - Idea is to train several models separately, then have all the models vote on the output for test examples
- This strategy is called model averaging
- Techniques employing this strategy are known as ensemble methods
- Model averaging works because different models will not make the same mistake

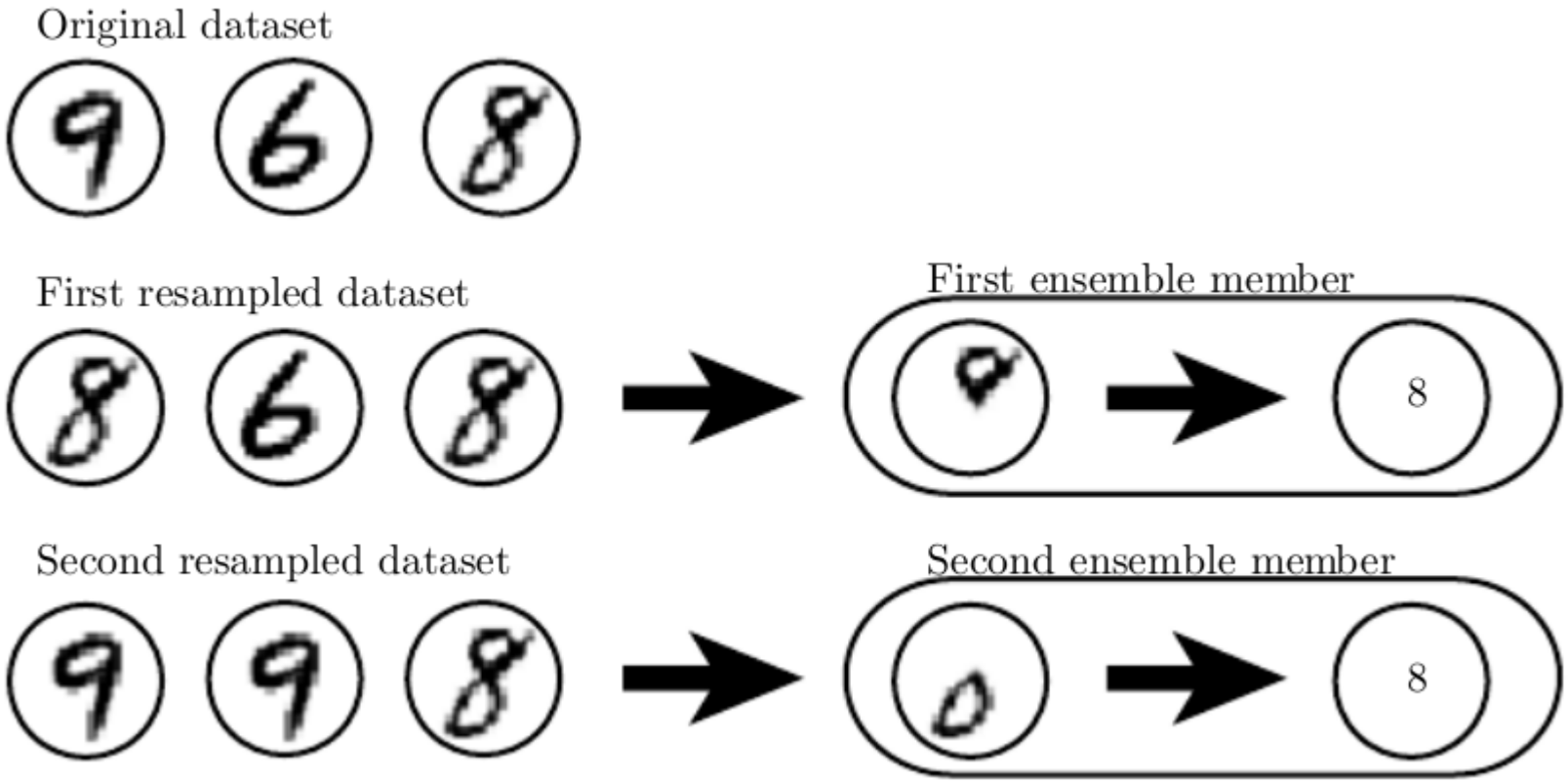
Ensemble vs Bagging

- Different ensemble methods construct the ensemble of models in different ways
- Example: each member of ensemble could be formed by training a completely different kind of model using a different algorithm or objective function
- Bagging is a method that allows the same kind of model, training algorithm and objective function to be reused several times

The Bagging Technique

- Given training set D of size N, generate k data sets of same no of examples as original by sampling with replacement
- Some observations may be repeated in D_i, some others are missing. This is known as a bootstrap sample.
- The differences in examples will result in differences between trained models
- The k models are combined by averaging the output (for regression) or voting (for classification)

Example


Original dataset
First resampled dataset
First ensemble member
8
Second resampled dataset
Second ensemble member
8

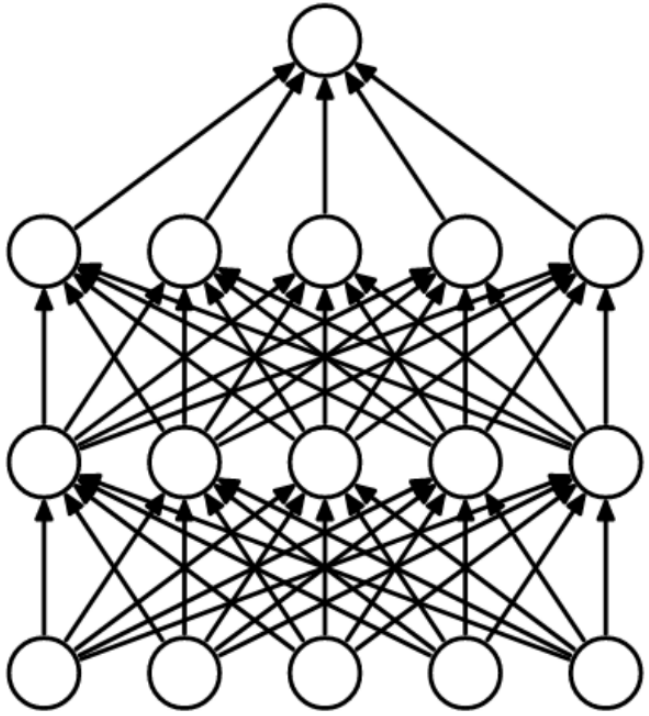
Bagging in Neural Nets

- Neural nets reach a wide variety of solution points
  - Thus, they benefit from model averaging when trained on the same dataset
  - Differences in:
    - random initializations
    - random selection of minibatches, in hyperparameters,
  - cause different members of the ensemble to make partially independent errors

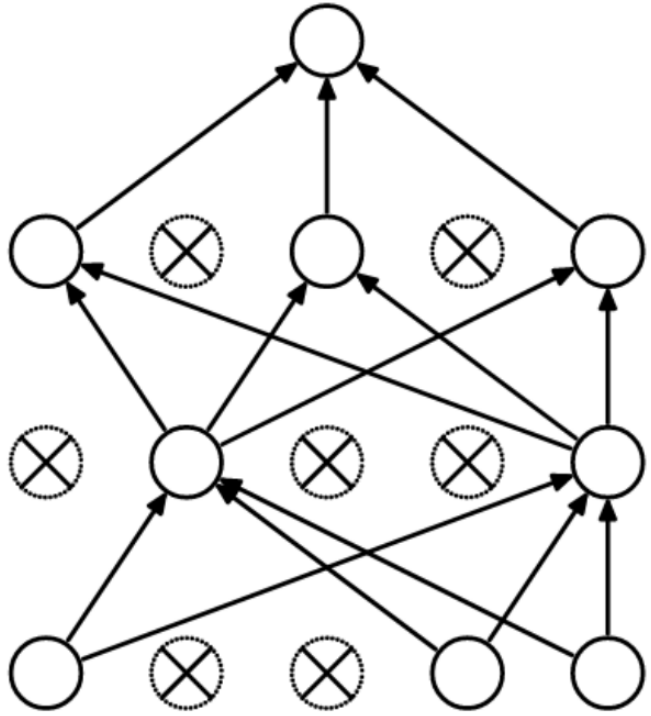
- Model averaging is a reliable method for reducing generalization error
  - Machine learning contests are usually won by model averaging over dozens of models, e.g., the Netflix grand prize
- But comes with significant computational costs

Creating new Models by Removing Units

- Dropout trains an ensemble of subnetworks
  - formed by removing non-output units from an underlying base network
- We can effectively remove units by multiplying its output value by zero



(a) Standard Neural Net



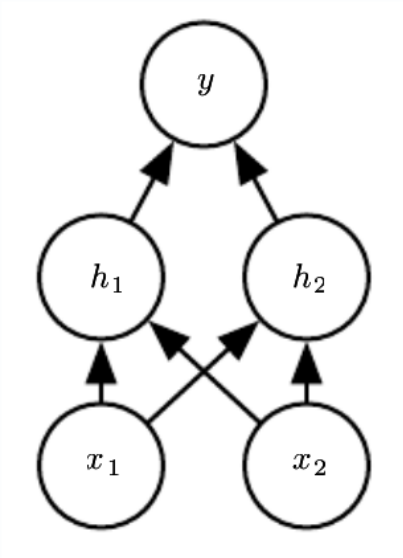
(b) After applying dropout.

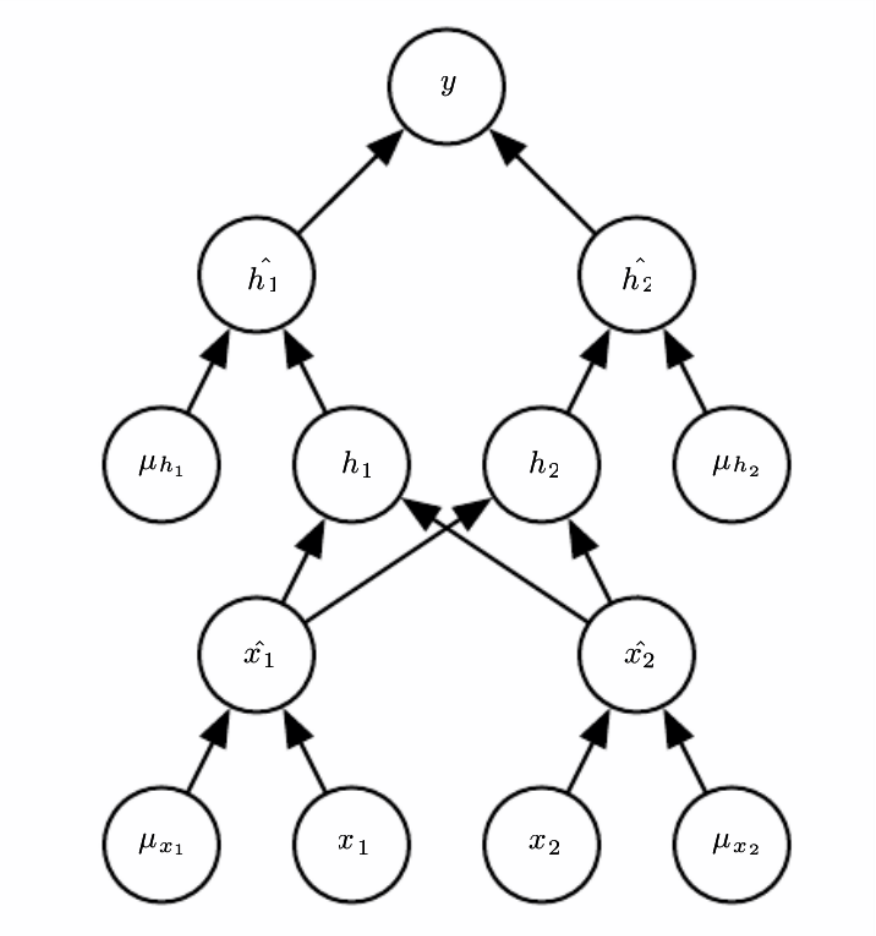
Mask for dropout training

- To train with dropout we use minibatch based learning algorithm that takes small steps such as SGD
- At each step randomly sample a binary mask
  - Probability of including a unit is a hyperparameter
  - Normally: 0.5 for hidden units and 0.8 for input units
- We run forward & backward propagation as usual
- Equivalent to randomly selecting a subnetwork of the entire network

Example

- Modified network incorporating a binary vector $\boldsymbol{\mu}$
- Training and evaluation as normal





Formal Description

➢Suppose that mask vector μ specifies which units to include

➢Cost of the model is specified by

$J(\theta, \mu)$

➢Dropout training consists of minimizing

$\mathbb{E}_{\mu}\left(J(\theta, \mu)\right)$

➢Expected value contains exponential no. of terms

➢We can get an unbiased estimate of its gradient by sampling values of μ

Dropout Prediction

- Submodel defined by mask vector $\boldsymbol{\mu}$ defines a probability distribution $p(y \mid \boldsymbol{x}, \boldsymbol{\mu})$
- At test time after training has finished, we would ideally like to find a sample average of all possible 2^n dropped-out networks
  - This is unfeasible for any realistic number n
- Approximation:
  - Each node's output scaled by a factor of p during training, so that the expected value of the output of any node is the same as in the training stages.
  - This is the biggest contribution of the dropout method:
    - **although it simulates 2^n different neural nets, at test time only a single network needs to be evaluated and tested**

Dropout

CLASS torch.nn.Dropout(*p=0.5*, *inplace=False*) [SOURCE]

During training, randomly zeroes some of the elements of the input tensor with probability `p`.

The zeroed elements are chosen independently for each forward call and are sampled from a Bernoulli distribution.

Each channel will be zeroed out independently on every forward call.

This has proven to be an effective technique for regularization and preventing the co-adaptation of neurons as described in the paper Improving neural networks by preventing co-adaptation of feature detectors .

Furthermore, the outputs are scaled by a factor of $\frac{1}{1-p}$ during training. This means that during evaluation the module simply computes an identity function.

Parameters

- **p** (*float*) – probability of an element to be zeroed. Default: 0.5
- **inplace** (*bool*) – If set to `True`, will do this operation in-place. Default: `False`

Always check if p is p(keep) or p(drop).

Scale down during training to implement Dropout, such that no action needs to be done at test time

Make sure to properly set your model into .train() and .eval() mode!

Source: PyTorch documentation, Oct 10, 2024

Another Interpretation of Dropout

- So far we have described dropout purely as a means of performing efficient, approximate an ensemble method

- **Dropout trains an ensemble of models that share hidden units**
  - Each hidden unit must be able to perform well regardless of which other hidden units are in the model
  - Hidden units must be prepared to be swapped and interchanged between models

- **Dropout thus regularizes each hidden unit to be not merely a good feature but a feature that is good in many contexts.**

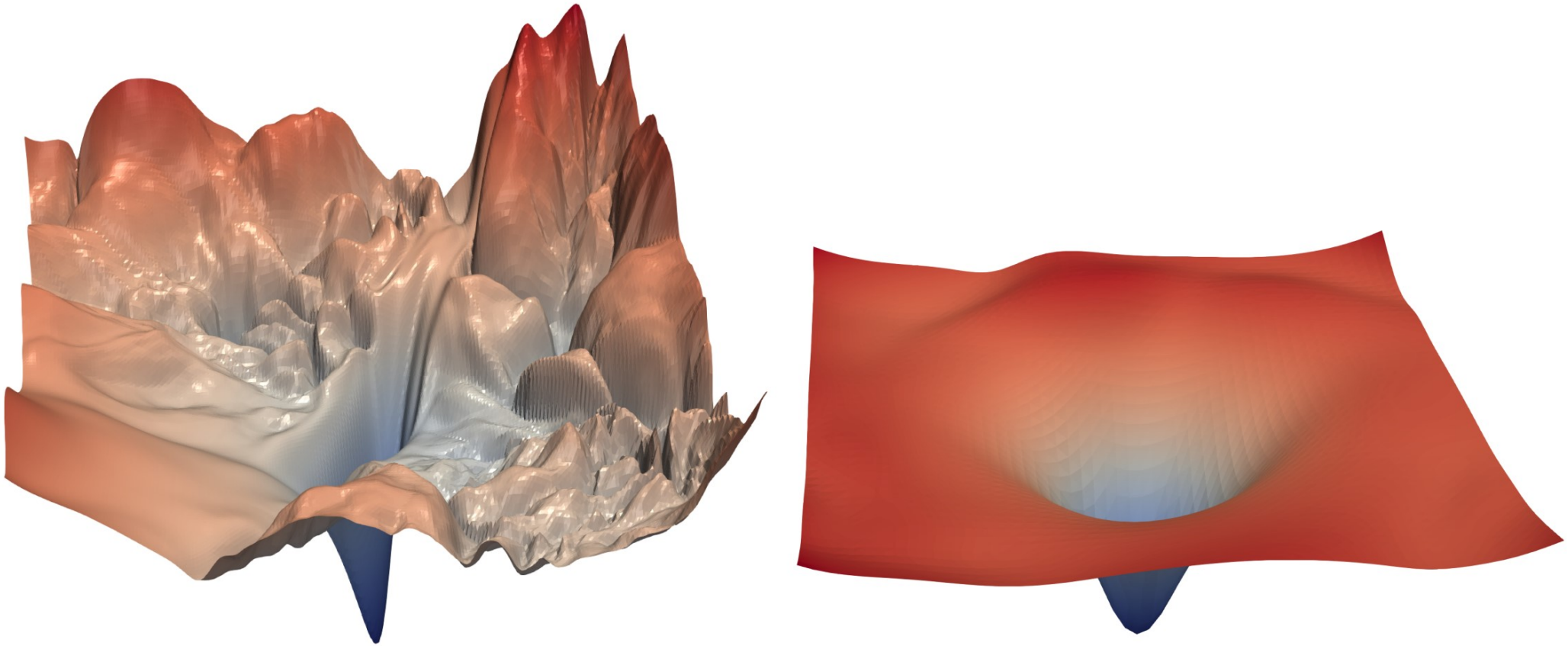
Take-home

- Best way to improve your model is to gather more data
- If that's not possible, **data augmentation** depending on domain knowledge (e.g., invariance)
- An L2 penalty (weight decay) may help to generalize better, but it shouldn't be too strong
- An L1 penalty on the weights nudges your model to be sparse (higher fraction of weights being 0)
- Early-stopping can help but be careful to not stop too early (use a very high patience to be safe)
- Ensembles/Bagging (averaging many models) usually works but is very expensive with large neural nets
- **Dropout** (randomly zeroing out activations) is an efficient way to implement bagging with neural nets

Keywords

- Bias-Variance Trade-off
- Regularization
- Model capacity
- Invariance/equivariance
- Dropout

Outlook: Residual Connections



(a) (b)

How’s my teaching?

https://www.admonymous.co/lukasgalke

Department of Mathematics
and Computer Science

sdu.dk

#sdudk

Advanced Machine Learning

Convolutional Networks

Lukas Galke

Spring 2026

SDU

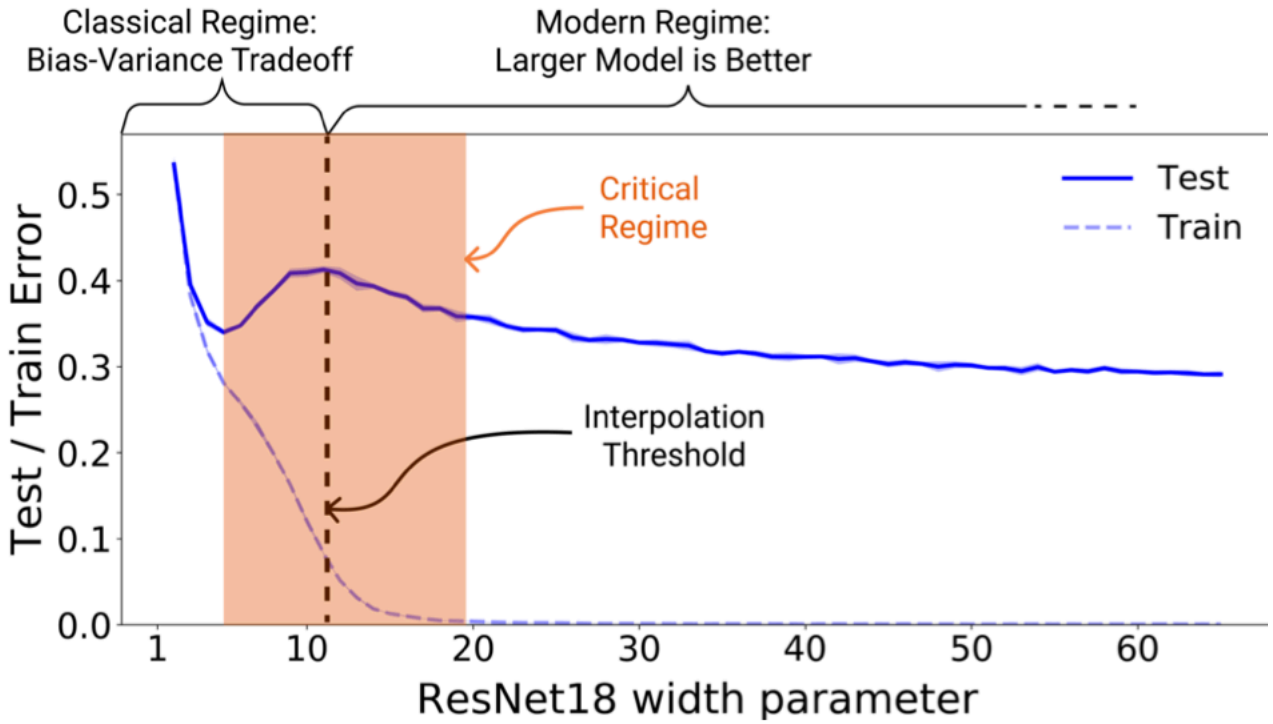
Keywords

- Convolution
- Parameter Sharing
- Pooling
- Normalization
- Residual connections

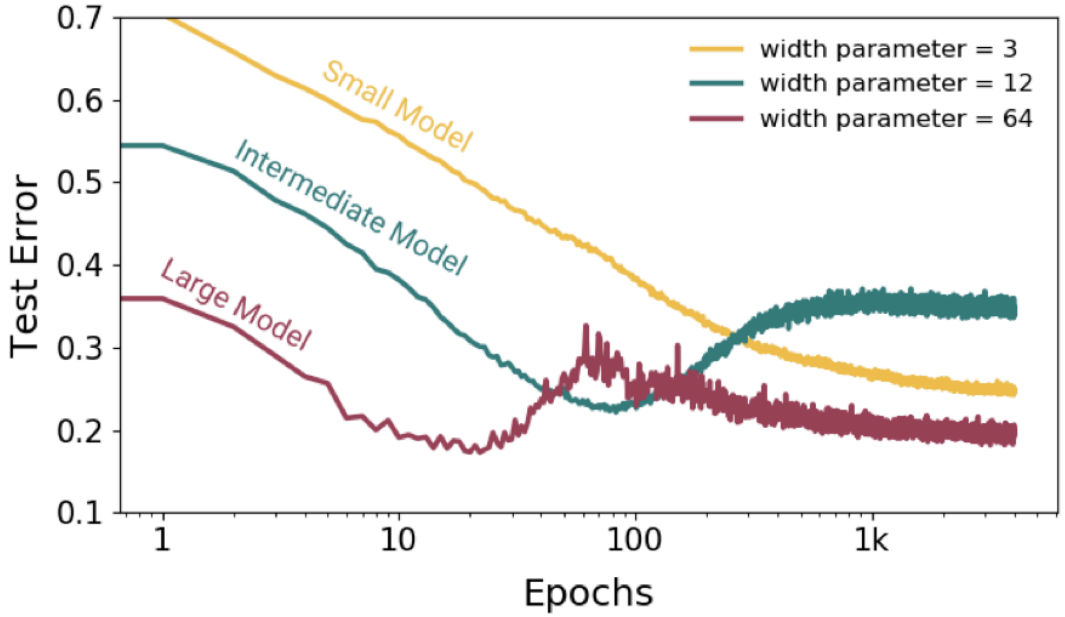
What have we learned so far?

- We know how to handle normal feed-forward networks
  - How to design them
  - How to define the error function
  - How to train them
- BUT: So far, our inputs have been rather small, i.e., we had just a few dimensions as input
- What if we now want to treat large inputs, like a full-blown high-resolution image

Recap: Deep Double Descent



~Model Capacity



~Training time

Source: Deep Double Descent,
https://arxiv.org/abs/1912.02292

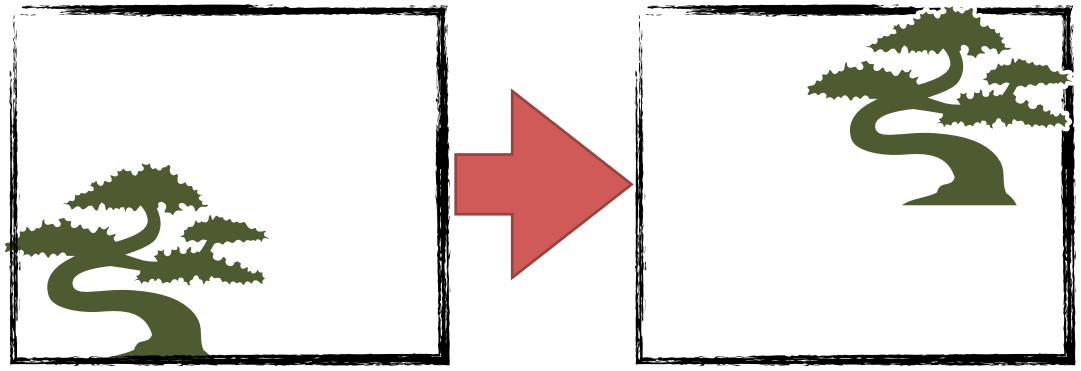
Recap from last lecture (Regularization)

- Best way to improve your model is to gather more data
- If that's not possible, **data augmentation** depending on domain knowledge (e.g., invariance)
- An L2 penalty (weight decay) may help to generalize better, but it shouldn't be too strong
- An L1 penalty on the weights nudges your model to be sparse (higher fraction of weights being 0)
- Early-stopping can help but be careful to not stop too early (use a very high patience to be safe)
- Ensembles Bagging (averaging many models) usually works but is very expensive with large neural nets
- **Dropout** (randomly zeroing out activations) is an efficient way to implement bagging with neural nets
- Adversarial training makes your model more robust to slight variations of the input

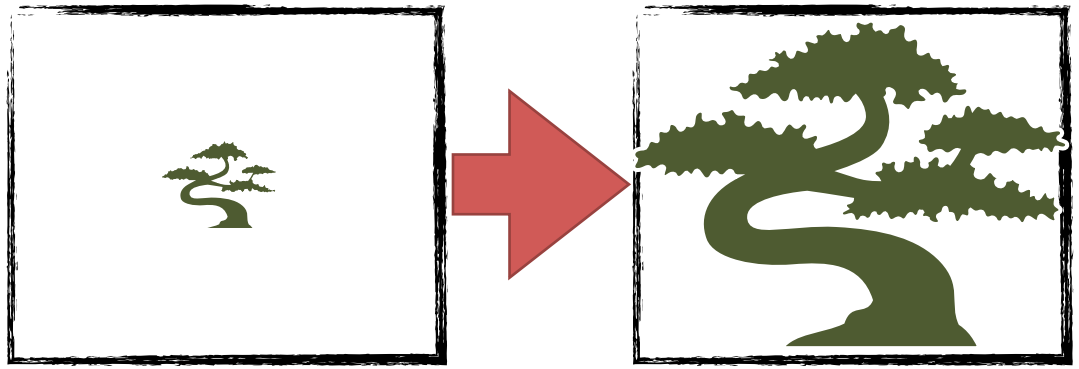
Recap: Symmetry and Invariance

Invariance:
S(I) = S(T(I))

Translation invariance



Scale invariance



Invariance: Model output is unchanged with respect to some transformation.

Symmetries: The transformations that leave other properties unchanged

Recap: Equivariance

S(T(I)) = T(S(I))

Model output changes in the same way with respect to some transformation

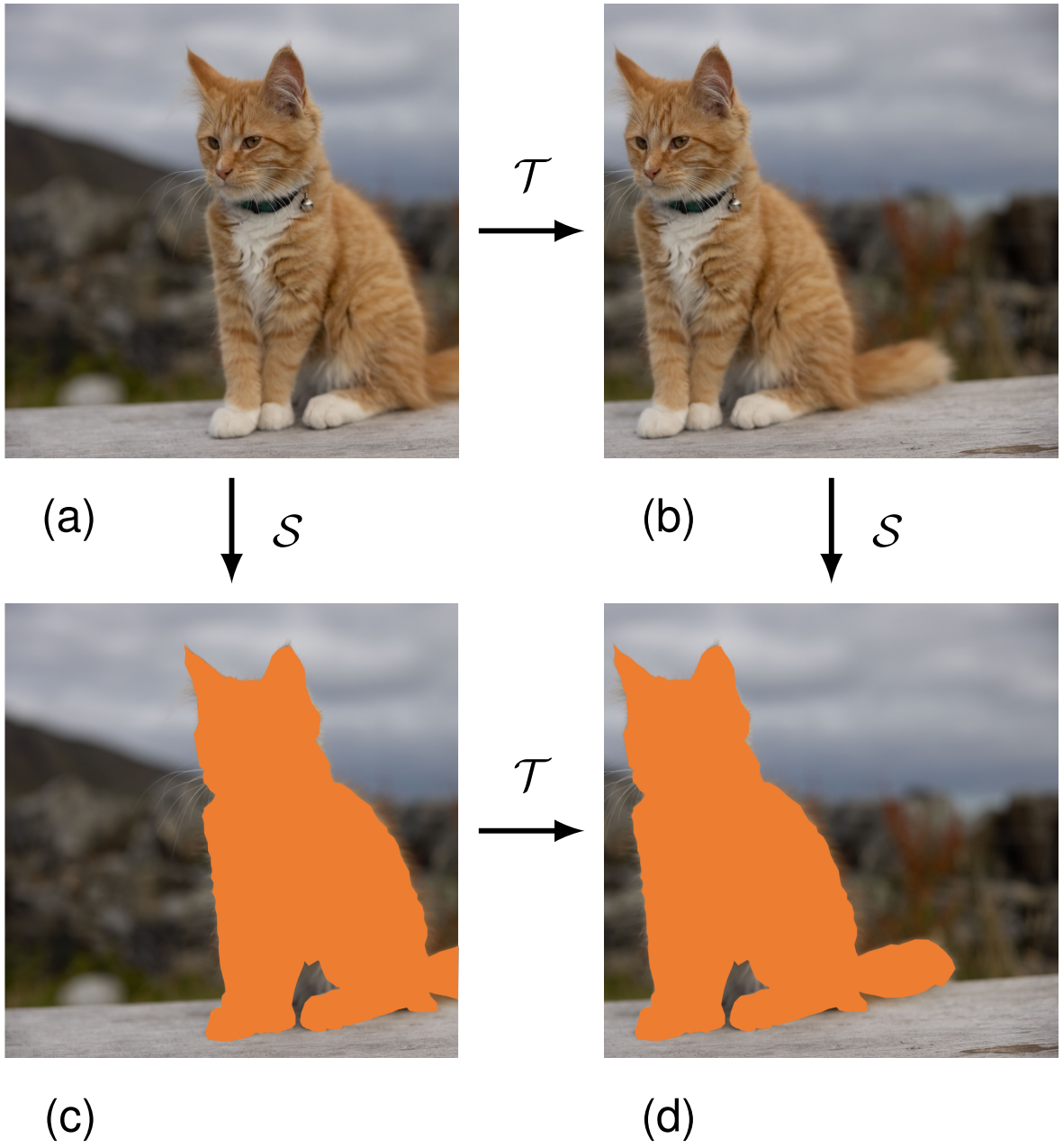


Figure from Bishop & Bishop (2024)

Problems when using large inputs

- Assume a standard mobile phone image with 8 Mega pixels
- Each pixel consists of 3 color channels, i.e., we end up with 24 million input "dimensions"
- Now we want to detect a face. We build a fully-connected layer where each node represents the presence of a certain feature, for instance:
  - One for eyes
  - One for hair
  - One for lips
  - …

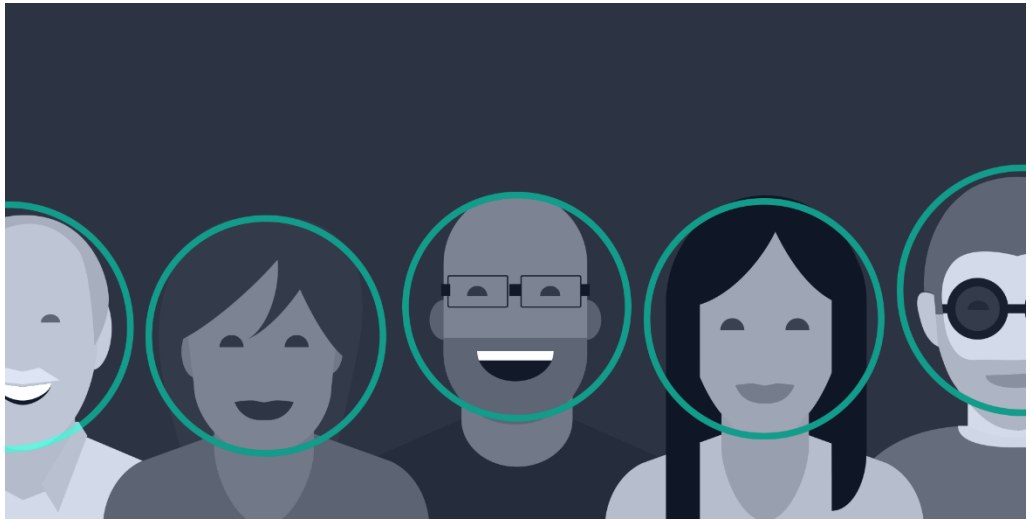
Problems when using large inputs

- Now we want to detect a face. We build a fully-connected layer where each node represents the presence of a certain feature, for instance:
  - One for eyes
  - One for hair
  - One for lips
  - …
- So that means even for a moderate number of features we aim to extract (in case we could so clearly define the purpose of the hidden layer). Extracting just 10 features, we end up with

240 million weights

Problems when using large inputs

- Even if we could just train those networks, we have another problem:
  - Images of faces for example might look vastly different
  - The head can be tilted, rotated, viewed from the side
  - But we want our network to be able to detect, e.g., an eye regardless of the position of the eye in the image:



Problems when using large inputs

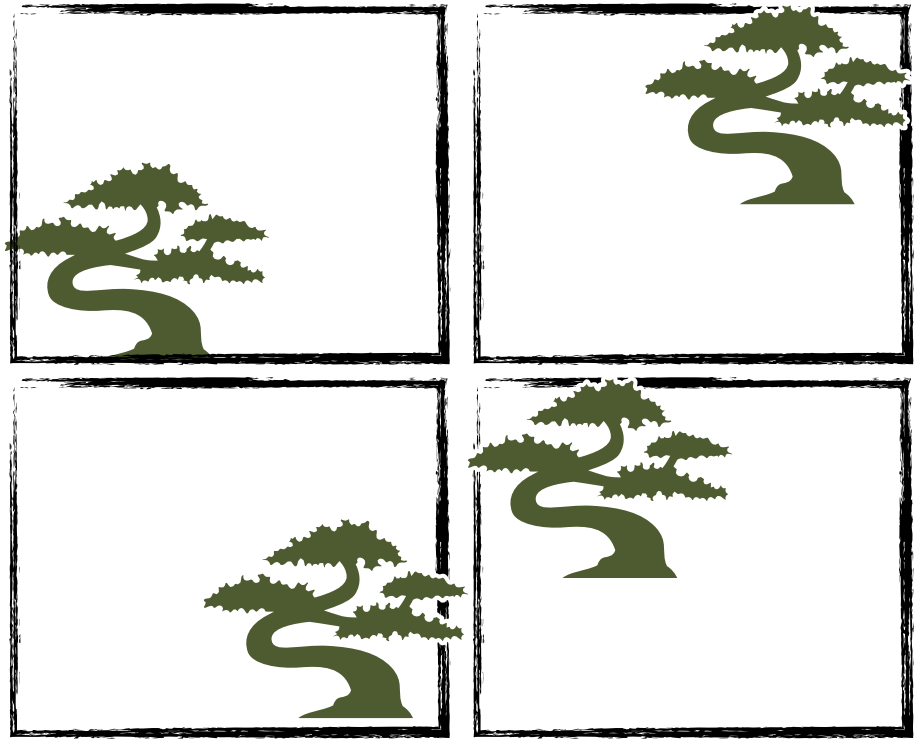
- Bottomline is: If a fully connected layer has never seen an eye at a certain position in the image, it had never the possibility to train the weights accordingly
- We would not only need to train an insane amount of data, but we additionally would also need to train with an insane amount data
- **We need something else which**
  - **keeps the number of weights in check**
  - **is translation invariant, i.e., we can detect the presence of a feature regardless where it is located within the feature**

The key ideas we want to exploit

- **Image statistics are translation invariant (objects and viewpoint translates)**
  - build this translation invariance into the model (rather than learning it)
  - tie lots of the weights together in the network
  - reduces number of parameters
- **Expect learned low-level features to be local (e.g. edge detector)**
  - build this into the model by allowing only local connectivity
  - this reduces the numbers of parameters further
- **Expect high-level features learned to be coarser**
  - build this into the model by subsampling more and more up the hierarchy
  - reduces the number of parameters again

Another perspective: Sample Complexity

➢ **Sample Complexity**: The number of training example it needs to successfully learn a target function.



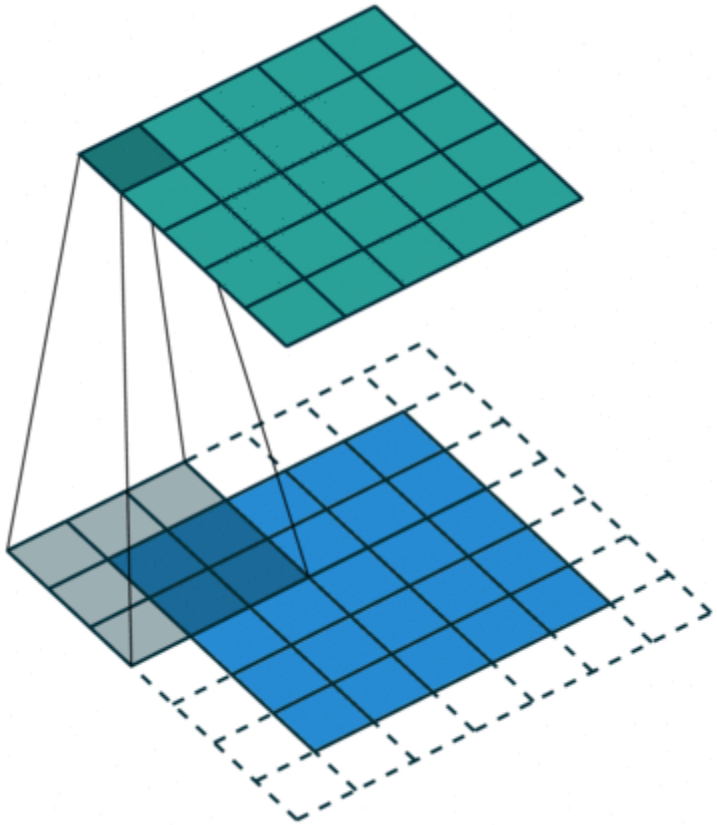
Not translation invariant
High sample complexity
Many data points needed
to generalize

Translation invariant
Low sample complexity
Little data points needed
to generalize

Keywords

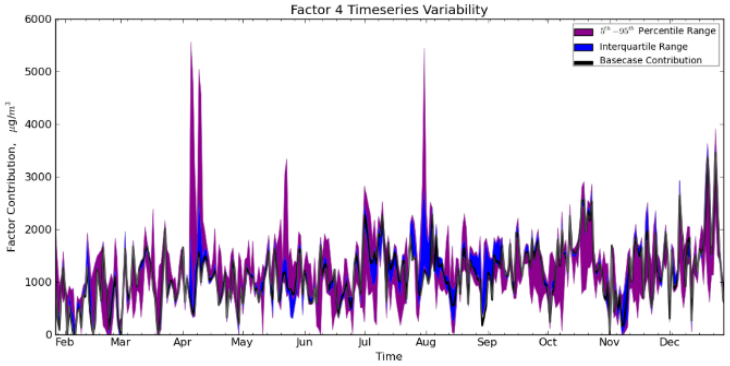
- **Convolution**
- Parameter Sharing
- Pooling
- Normalization
- Residual connections

Convolution Visualized

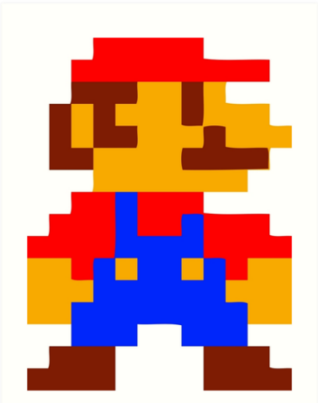


Convolutional Networks

- Convolutional neural networks (CNNs) are a specialized kind of neural network
  - Neural networks that use **convolution** in place of general matrix multiplication in at least one of their layers
- Well suited for data with a grid-like topology
- But: Convolution can be viewed as multiplication by a matrix



E.g. Ex: time-series data, which is a 1-D grid, taking samples at intervals



More obviously: Image data, which are 2-D grid of pixels

What is Convolution?

- Convolution is an operation on two functions of a real-valued argument
- Examples of the two functions
  - Tracking location of a spaceship by a laser sensor
    - A laser sensor provides a single output $x(t)$, the position of spaceship at time t
  - w a function of a real-valued argument
    - If laser sensor is noisy, we want a weighted average that gives more weight to recent observations
    - Weighting function is $w(a)$ where a is age of measurement
- Convolution is the smoothed estimate of the position of the spaceship

$$s(t) = \int x(a)w(t-a)da$$

What is Convolution?

- One-dimensional continuous case
  - Input $f(t)$ is convolved with a kernel $g(t)$

$$\big(f * g\big)(t) = \int_{-\infty}^{\infty} f(\tau)g(t-\tau)d\tau$$

- Note that $\big(f * g\big)(t) = (g * f)(t)$
- Terminology:
  - The first argument (here the function f) to the convolution is often referred to as the **input** and
  - The second argument (in this example, the function g) as the **kernel**.
  - The output is sometimes referred to as the **feature map**.

Convolution with Discrete Variables

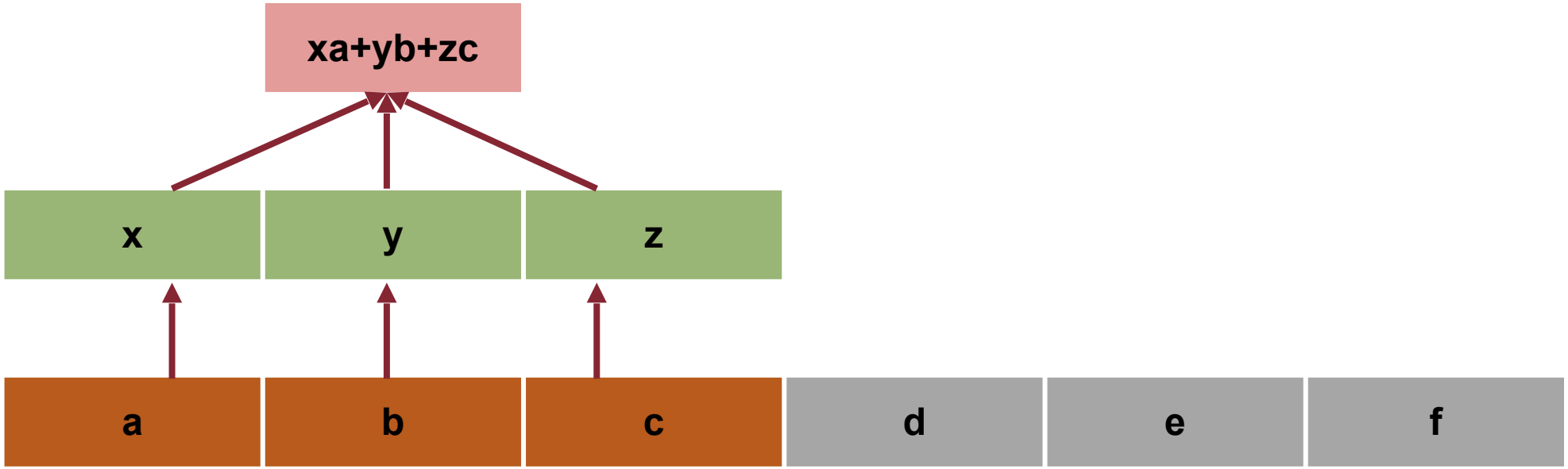
- Laser sensor may only provide data at regular intervals
- Time index t can take on only integer values
  - x and w are defined only on integer t

$$s(t) = (x * w)(t) \sum_{a=-\infty}^{\infty} x(a)w(t-a)$$

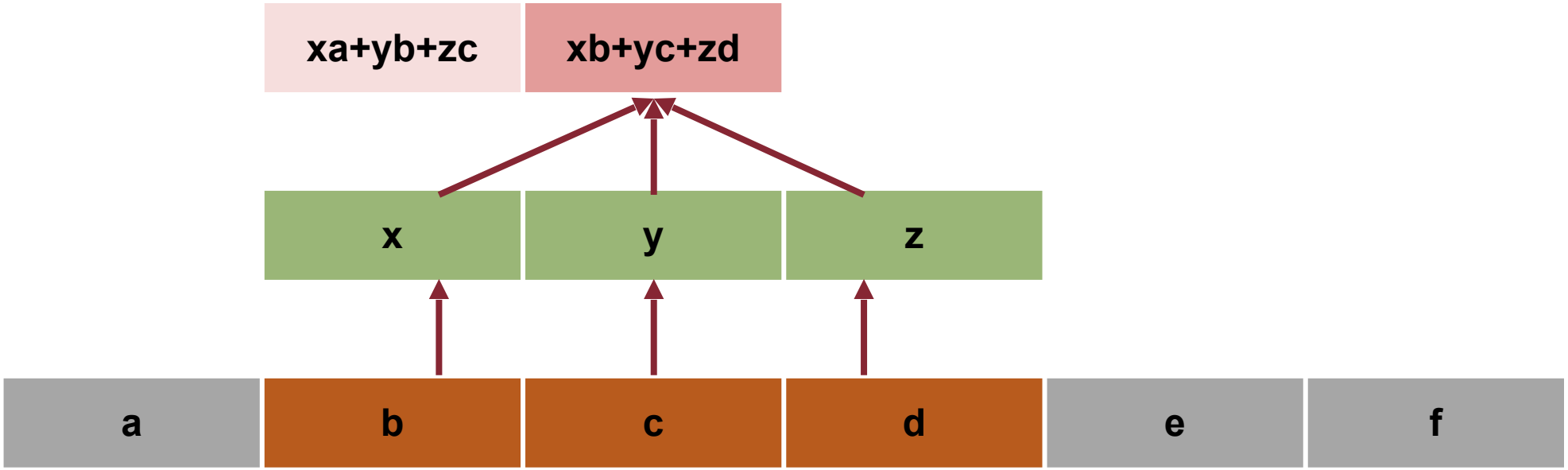
- In ML applications, input is a multidimensional array of data and the kernel is a multidimensional array of parameters that are adapted by the learning algorithm

- Input and kernel are explicitly stored separately

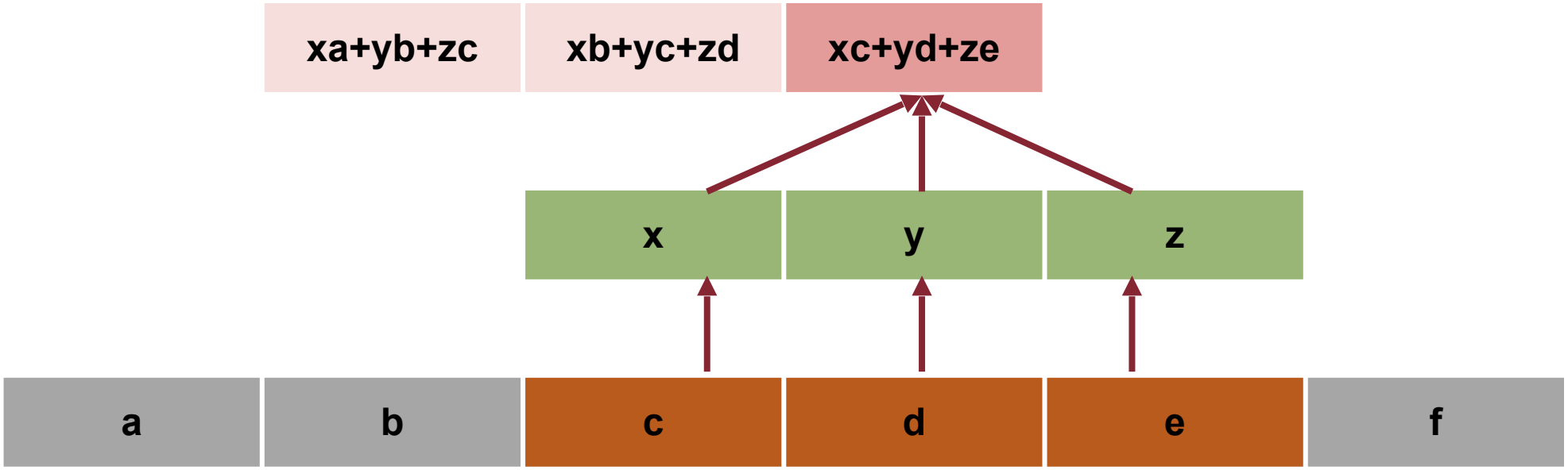
Discrete 1-Dimensional Case



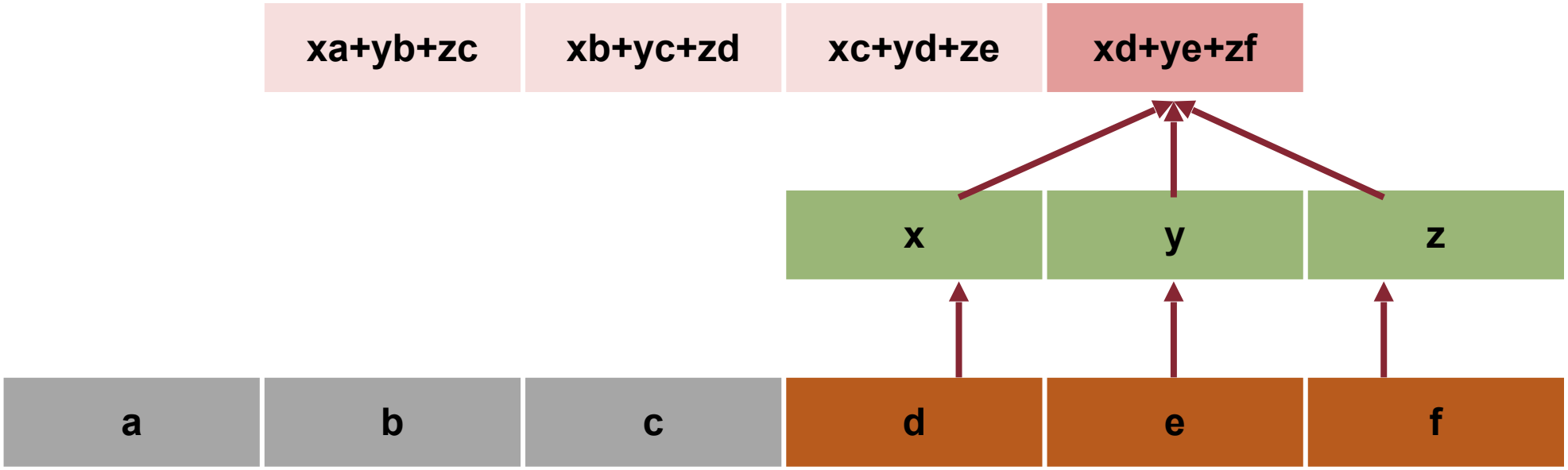
Discrete 1-Dimensional Case



Discrete 1-Dimensional Case

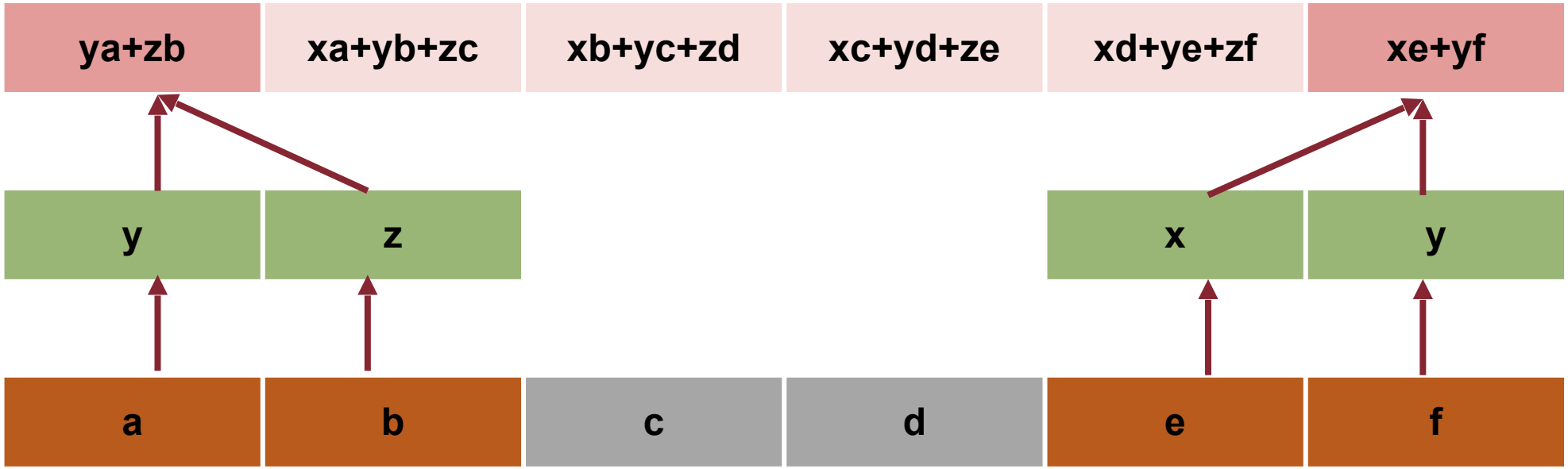


Discrete 1-Dimensional Case

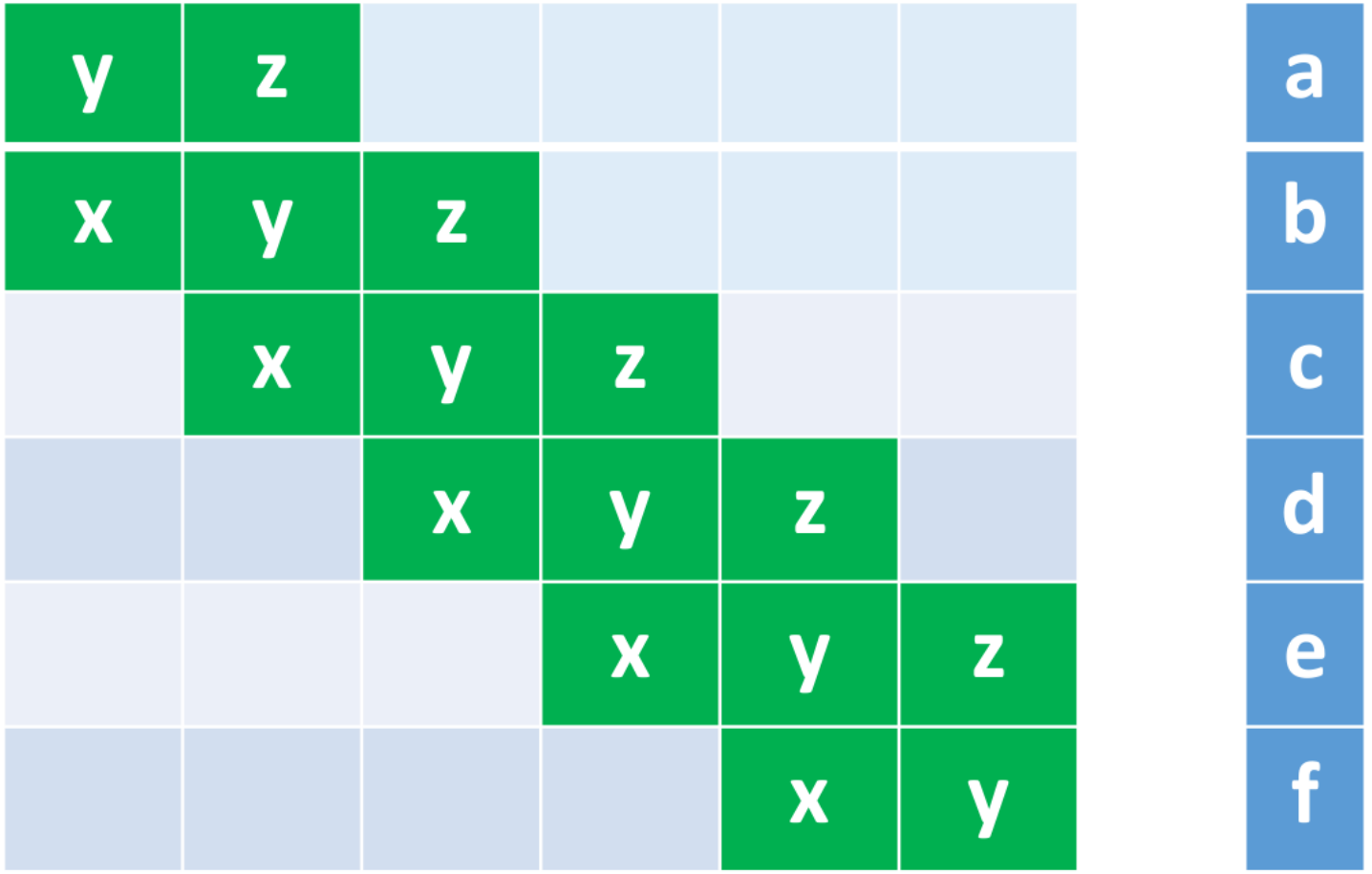


Discrete 1-Dimensional Case: Border Cases

- Border require special treatment
- Most commonly they are ignored
- Sometimes a smaller kernel is used then

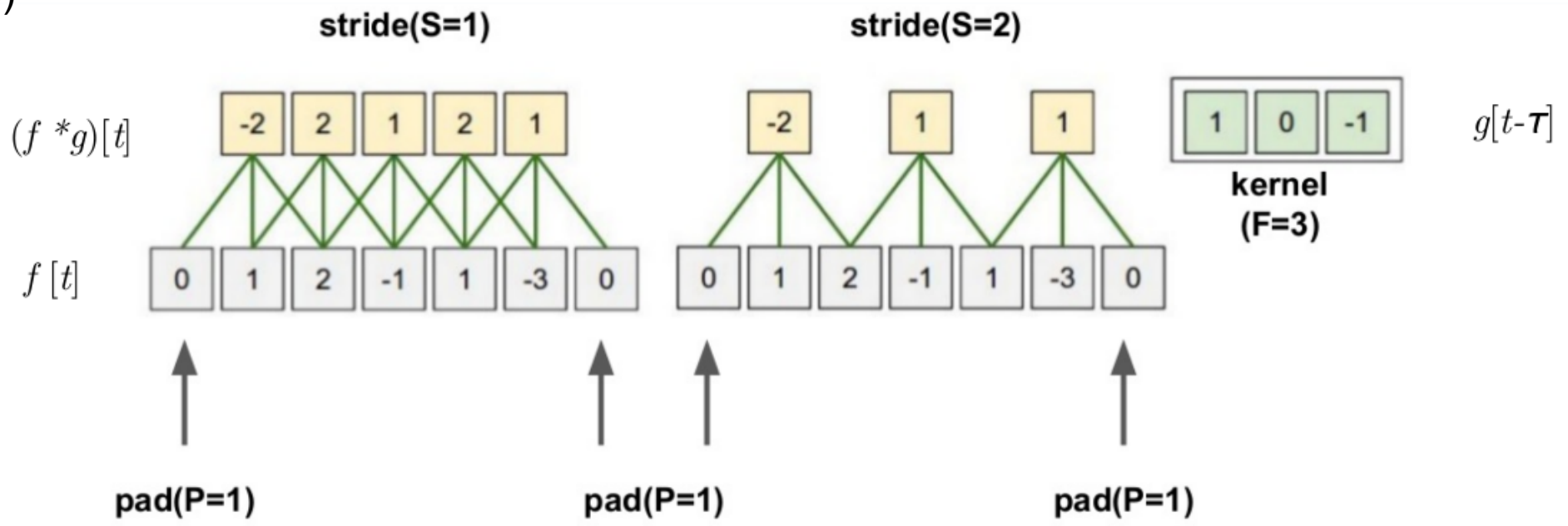


Seen as a Matrix Multiplication



Parameters of Convolution

- Kernel Size (F)
- Padding (P)
- Stride (S)



Two-dimensional Convolution

➢ If we use a 2D image I as input and use a 2D kernel K we have

$$S(i,j) = (I * K)(i,j) = \sum_m \sum_n I(m,n)\, K(i-m, j-n)$$

➢ Since convolution is commutative, we can also write:

$$S(i,j) = (K * I)(i,j) = \sum_m \sum_n I(i-m, j-n) K(m,n)$$

➢ Commutativity arises because we have flipped the kernel relative to the input

➢ As m increases, index to the input increases, but index to the kernel decreases

Cross-Correlation

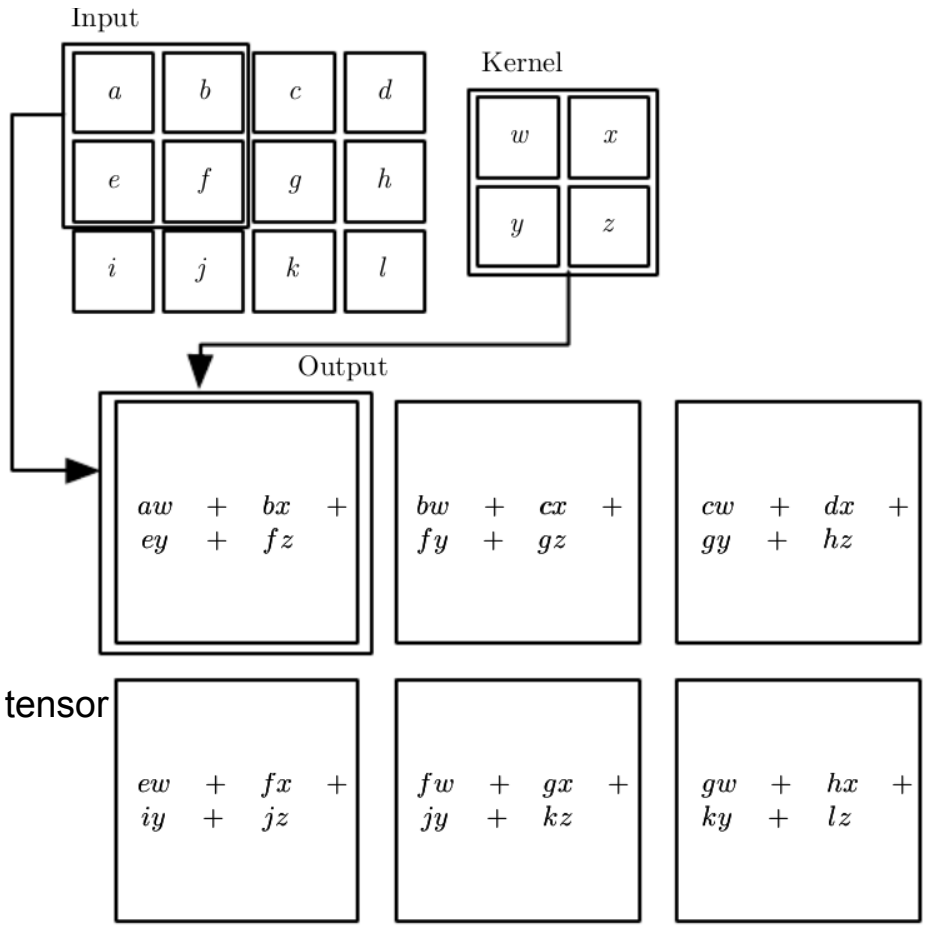
➢ Same as convolution, but without flipping the kernel

$$S(i,j) = (K * I)(i,j) = \sum_m \sum_n I(i+m, j+n) K(m,n)$$

➢ Both referred to as convolution, whether kernel is flipped or not

➢ The learning algorithm will learn appropriate values of the kernel in the appropriate place

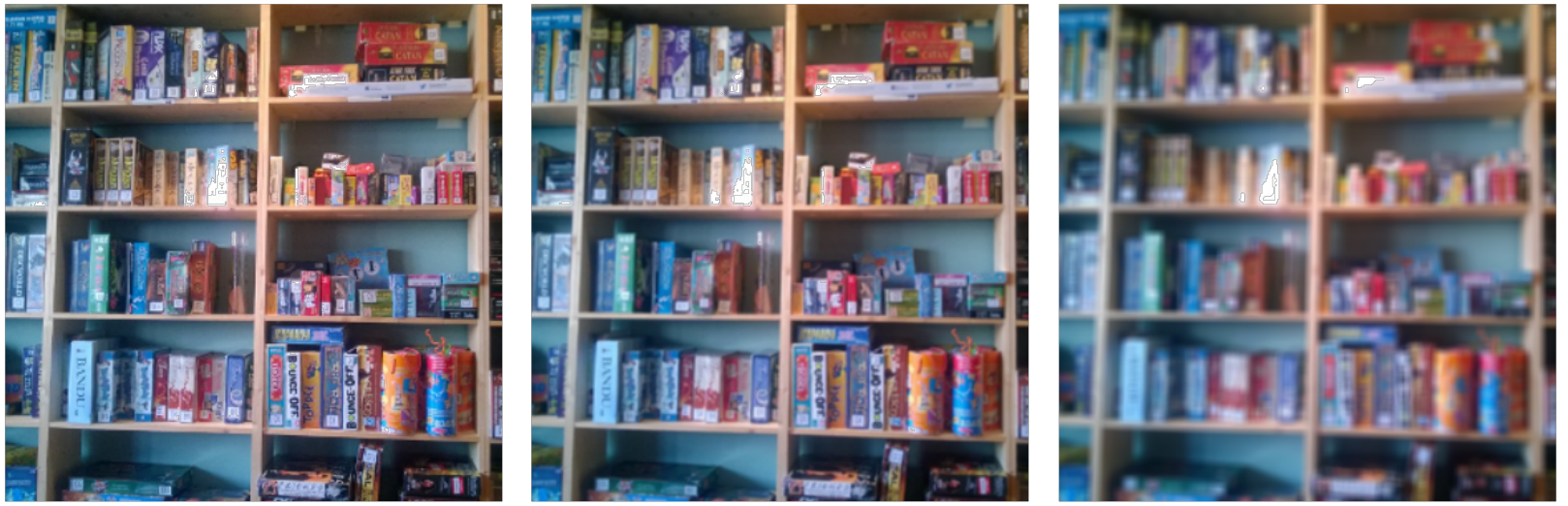
Example of 2D convolution



- Convolution without kernel flipping applied to a 2D tensor
- Output is restricted to case where kernel is situated entirely within the image
- Arrows show how upper-left of input tensor is used to form upper-left of output tensor

Classic Examples: Blurring Filter

$$K_{box} = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} \qquad \hat{K}_{box} = \begin{bmatrix} \frac{1}{9} & \frac{1}{9} & \frac{1}{9} \\ \frac{1}{9} & \frac{1}{9} & \frac{1}{9} \\ \frac{1}{9} & \frac{1}{9} & \frac{1}{9} \end{bmatrix}$$



the same image without any filtering, after a 3x3 box blur and after a 9x9 box blur

➢ https://www.taylorpetrick.com/blog/post/convolution-part3

Classic Examples: Edge Detection

➢ Simple

$$\begin{pmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{pmatrix} \quad \begin{pmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ -1 & -1 & -1 \end{pmatrix}$$

➢ Prewitt

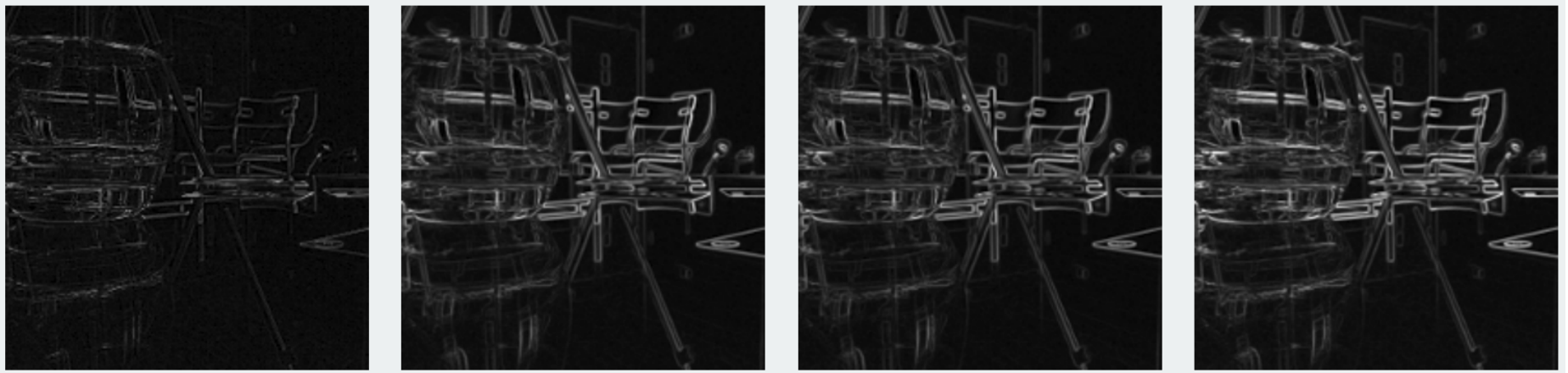
$$\mathbf{G_x} = \begin{bmatrix} -1 & 0 & +1 \\ -1 & 0 & +1 \\ -1 & 0 & +1 \end{bmatrix} * \mathbf{A} \quad \text{and} \quad \mathbf{G_y} = \begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ +1 & +1 & +1 \end{bmatrix} * \mathbf{A}$$

➢ Sobel

$$\mathbf{G}_x = \begin{bmatrix} +1 & 0 & -1 \\ +2 & 0 & -2 \\ +1 & 0 & -1 \end{bmatrix} * \mathbf{A} \quad \text{and} \quad \mathbf{G}_y = \begin{bmatrix} +1 & +2 & +1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix} * \mathbf{A}$$

➢ Kirsch (8 directions)

$$\mathbf{g}^{(1)} = \begin{bmatrix} +5 & +5 & +5 \\ -3 & 0 & -3 \\ -3 & -3 & -3 \end{bmatrix}, \mathbf{g}^{(2)} = \begin{bmatrix} +5 & +5 & -3 \\ +5 & 0 & -3 \\ -3 & -3 & -3 \end{bmatrix}, \mathbf{g}^{(3)} = \begin{bmatrix} +5 & -3 & -3 \\ +5 & 0 & -3 \\ +5 & -3 & -3 \end{bmatrix}$$



comparison of simple, sobel, prewitt and kirsch edge detection filters

Classic Examples: Sharpen

$$K_{sharp} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix} + \begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix} * amount$$

comparison of unfiltered image and sharpened images with amount=2 and amount=8

Classic Examples: Sharpen

Main Idea of convolution: Learn these weights

$$K_{sharp} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix} + \begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix} * amount$$

comparison of unfiltered image and sharpened images with amount=2 and amount=8

Motivation for using convolution networks

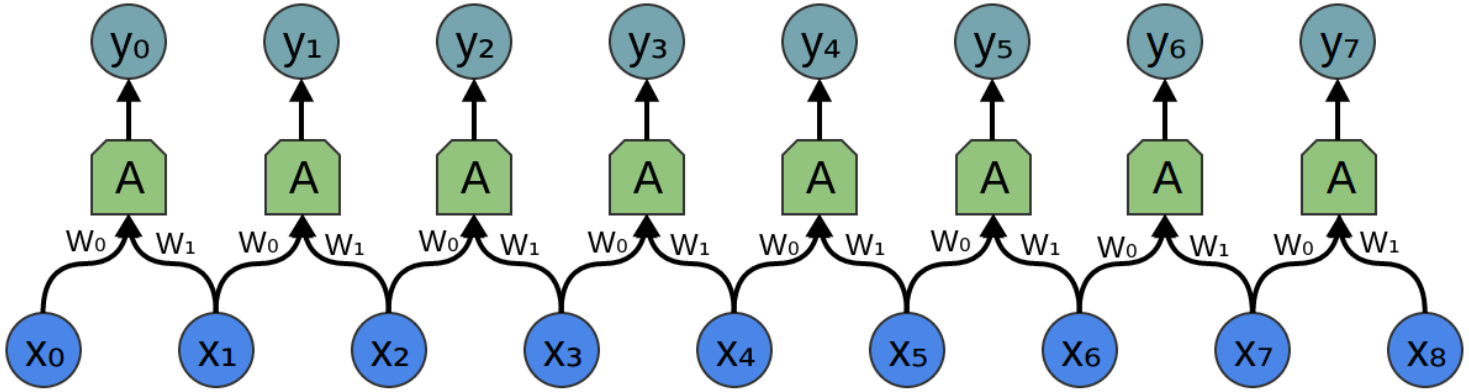
➢ Convolution leverages three important ideas to improve ML systems:

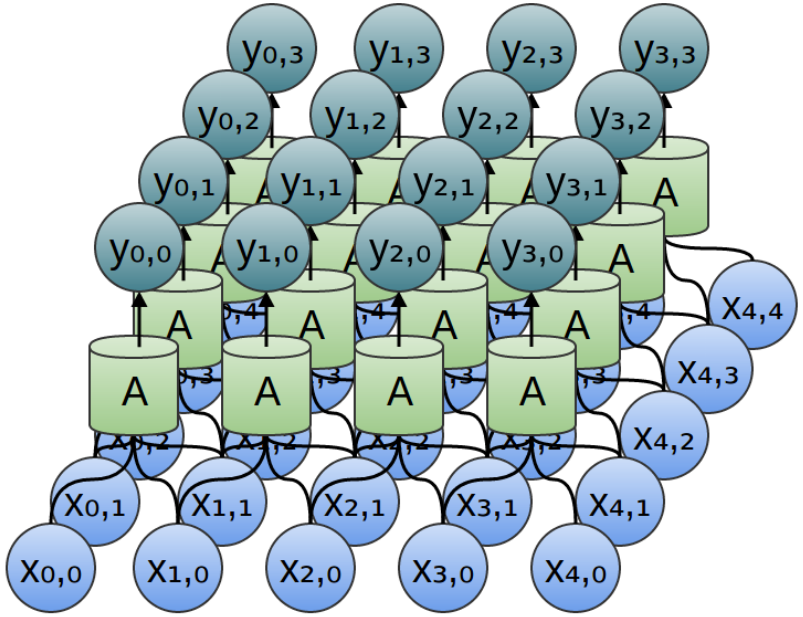
1. Sparse interactions
2. Parameter sharing
3. Equivariant representations

➢ Convolution also allows for working with inputs of variable size

Sparse connectivity due to Convolution

➢ Instead of learning a full matrix, only a couple of weights of the kernel need to be learned



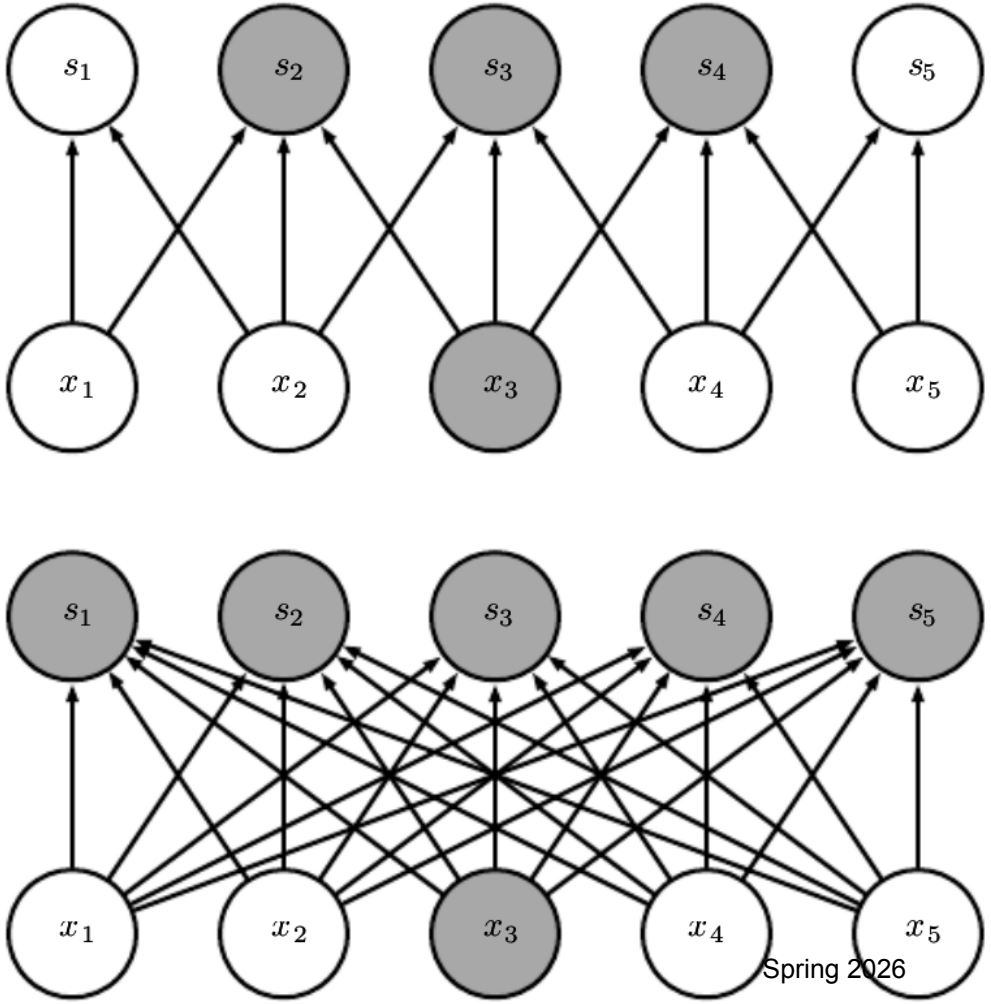


Sparse Connectivity: Input Perspective

Highlight one input x_3 and output units s affected by it

➢ Top: when s is formed by convolution with a kernel of width 3, only three outputs are affected by x_3

➢ Bottom: when s is formed by matrix multiplication connectivity is no longer sparse
=> All outputs are affected by x_3

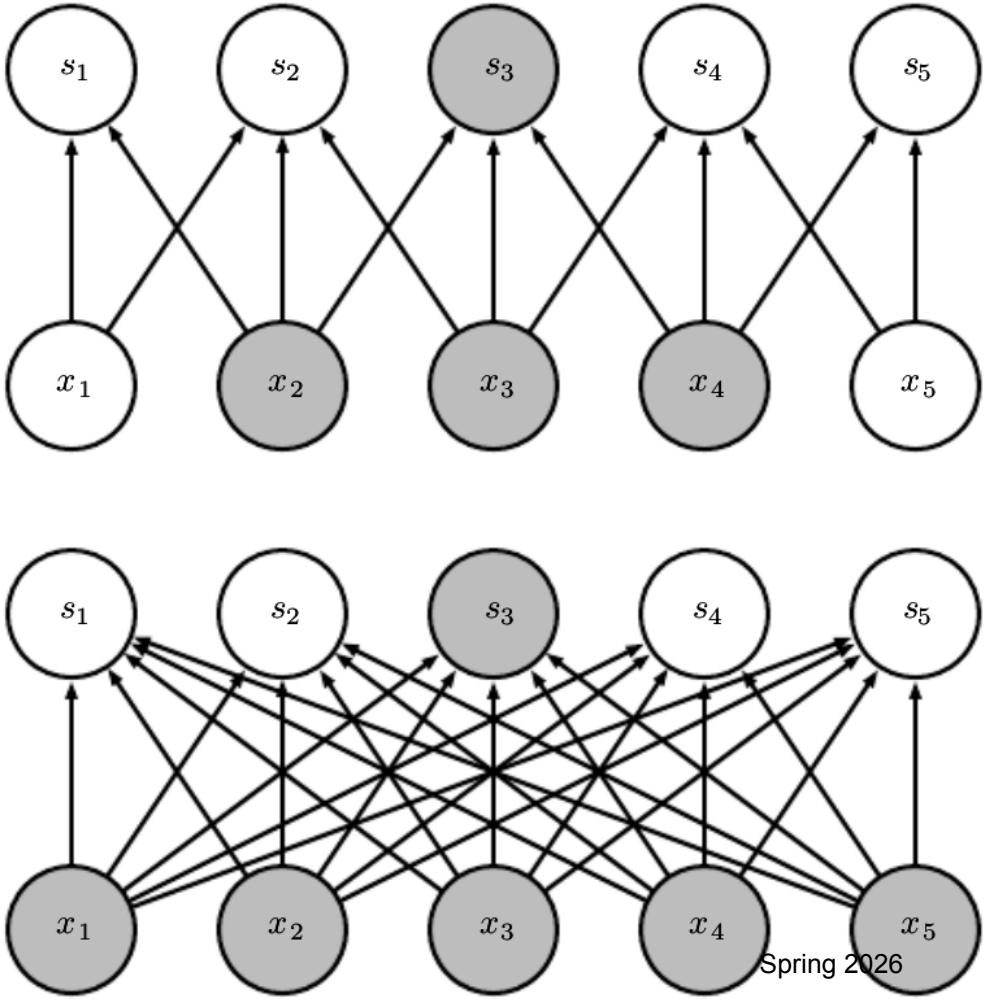


Sparse Connectivity: Output Perspective

Highlight one output s_3 and input units x affecting it:

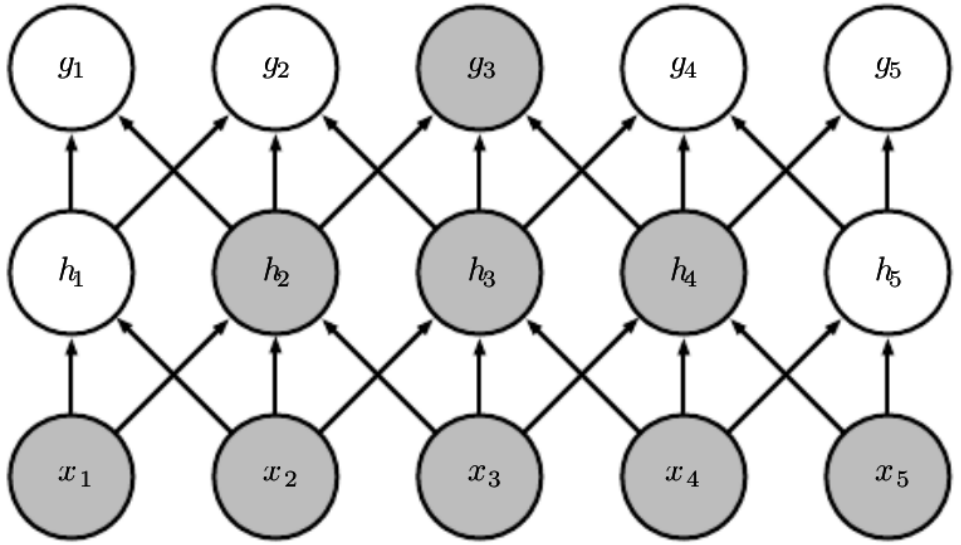
➢ Top: when s_3 is formed by convolution with a kernel of width 3, only three inputs are defining output s_3

➢ Bottom: when s_3 is formed by a matrix multiplication connectivity is no longer sparse
=> All inputs are defining output s_3



Performance with Reduced Connections

- It is possible to obtain good performance while keeping k several magnitudes lower than m
- In a deep neural network, units in deeper layers may indirectly interact with a larger portion of the input: Depth is the key
- This allows the network to efficiently describe complicated interactions between many variables from simple building blocks that only describe sparse interactions



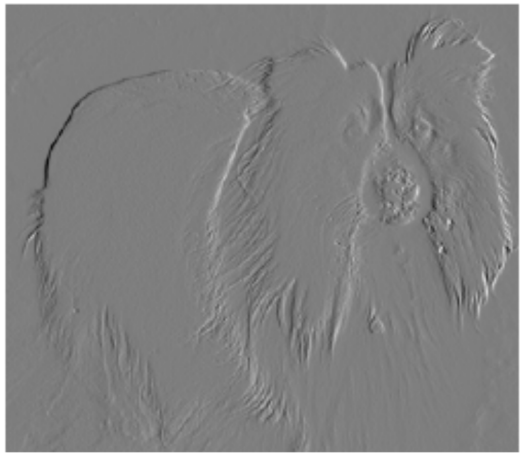
Keywords

- Convolution
- **Parameter Sharing**
- Pooling
- Normalization
- Residual connections

Parameter Sharing

- Parameter sharing refers to using the same parameter for more than one function in a model
- In a traditional neural net each element of the weight matrix is used exactly once when computing the output of a layer
  - It is multiplied by one element of the input and never revisited
- Parameter sharing is synonymous with tied weights
  - Value of the weight applied to one input is tied to a weight applied elsewhere
- In a Convolutional net, each member of the kernel is used in every position of the input (boundaries might be different)

Example: Efficiency of Convolution for Edge Detection

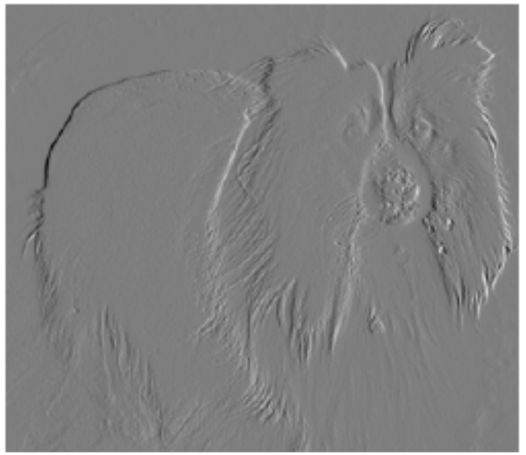


- Image on right formed by taking each pixel of input image and subtracting the value of its neighboring pixel on the left
- Input image is 320x280, output is 319x280
- Computational effort:

$$319 \cdot 320 \cdot 3 = 267{,}960 \text{ flops}$$

  - Each output pixel is calculated by a convolutional kernel with 2 entries, i.e. 2 multiplications, one add

Example: Efficiency of Convolution for Edge Detection



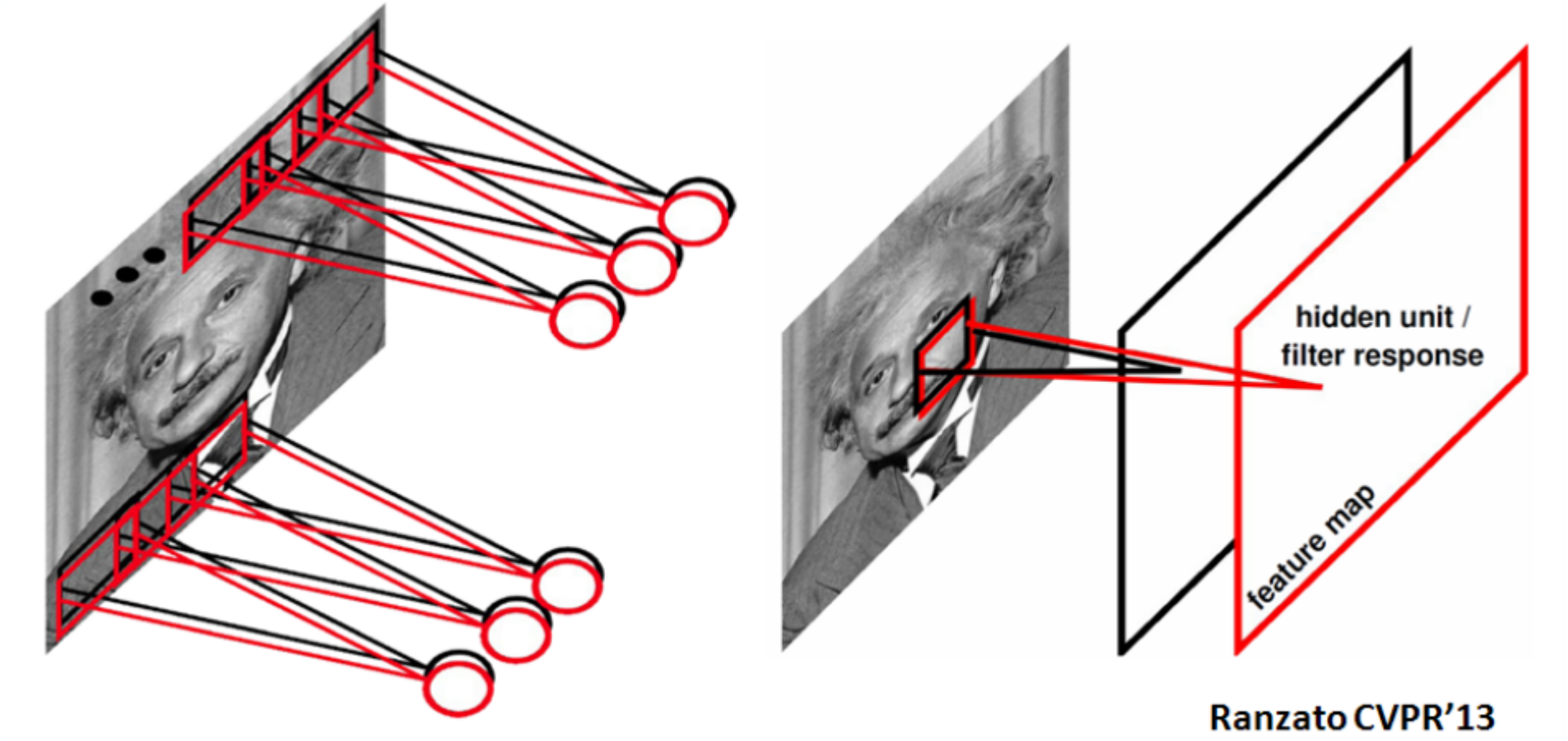
➢ Same operation with a full matrix with

$320 \cdot 280 \cdot 319 \cdot 280 = 8$ billion entries

➢ Storing the matrix (double vals): 8 billion * 8byte $\approx$ 60 Gbyte

➢ Performing the calculation: roughly 60,000 times less efficient computationally

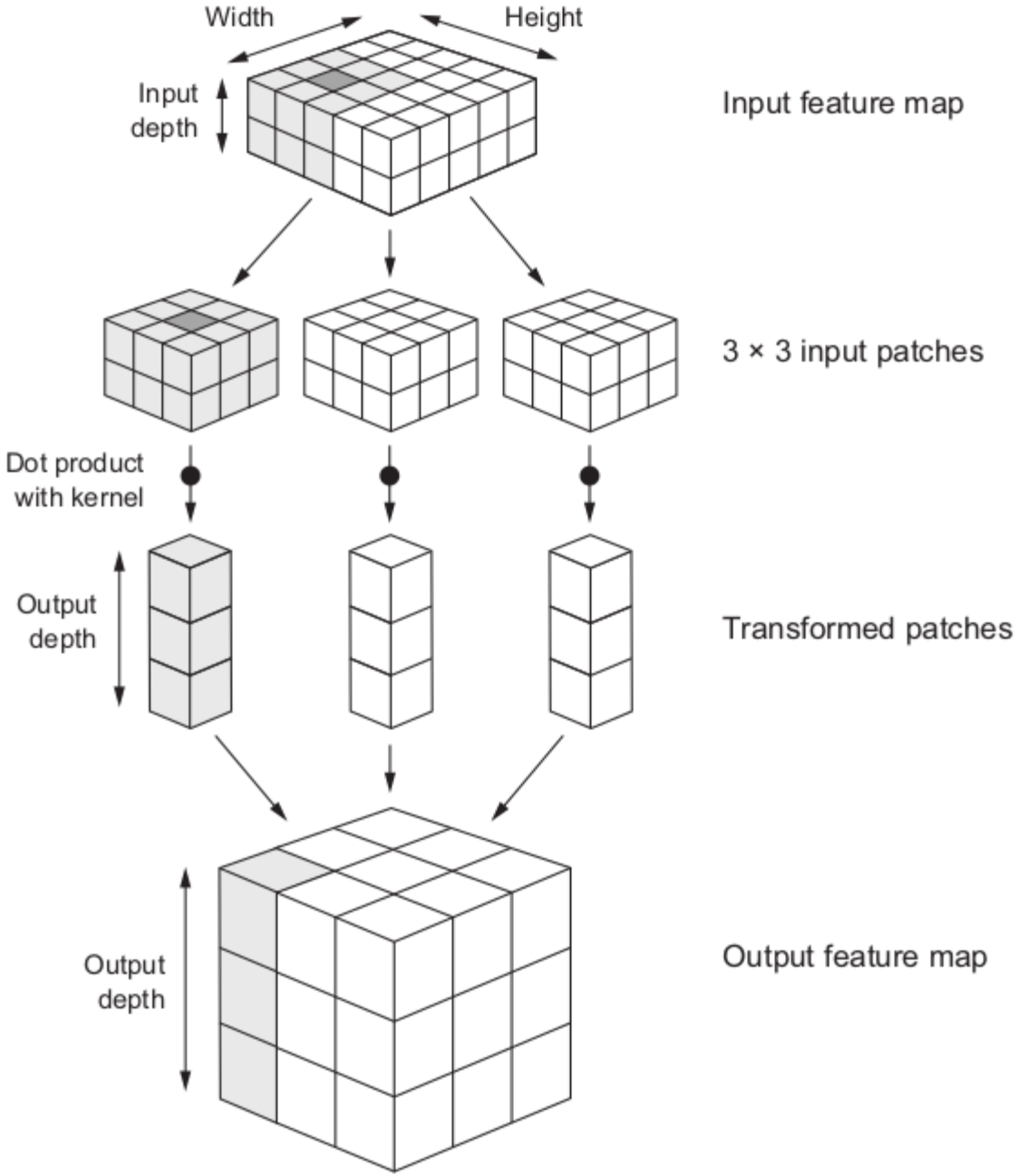
Depth of the Output Volume



Depth of the Output Volume

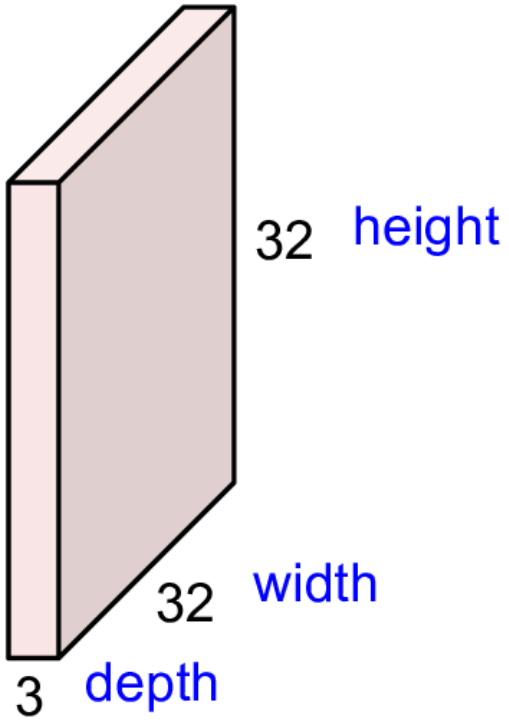
- The depth of the output volume is a hyperparameter
- It corresponds to the number of filters we would like to use, each learning to look for something different in the input.
- For example, if the first Convolutional Layer takes as input the raw image, then different neurons along the depth dimension may activate in presence of various oriented edges, or blobs of color.
- We will refer to a set of neurons that are all looking at the same region of the input as a depth column (some people also prefer the term fiber).

Depth of the Output Volume



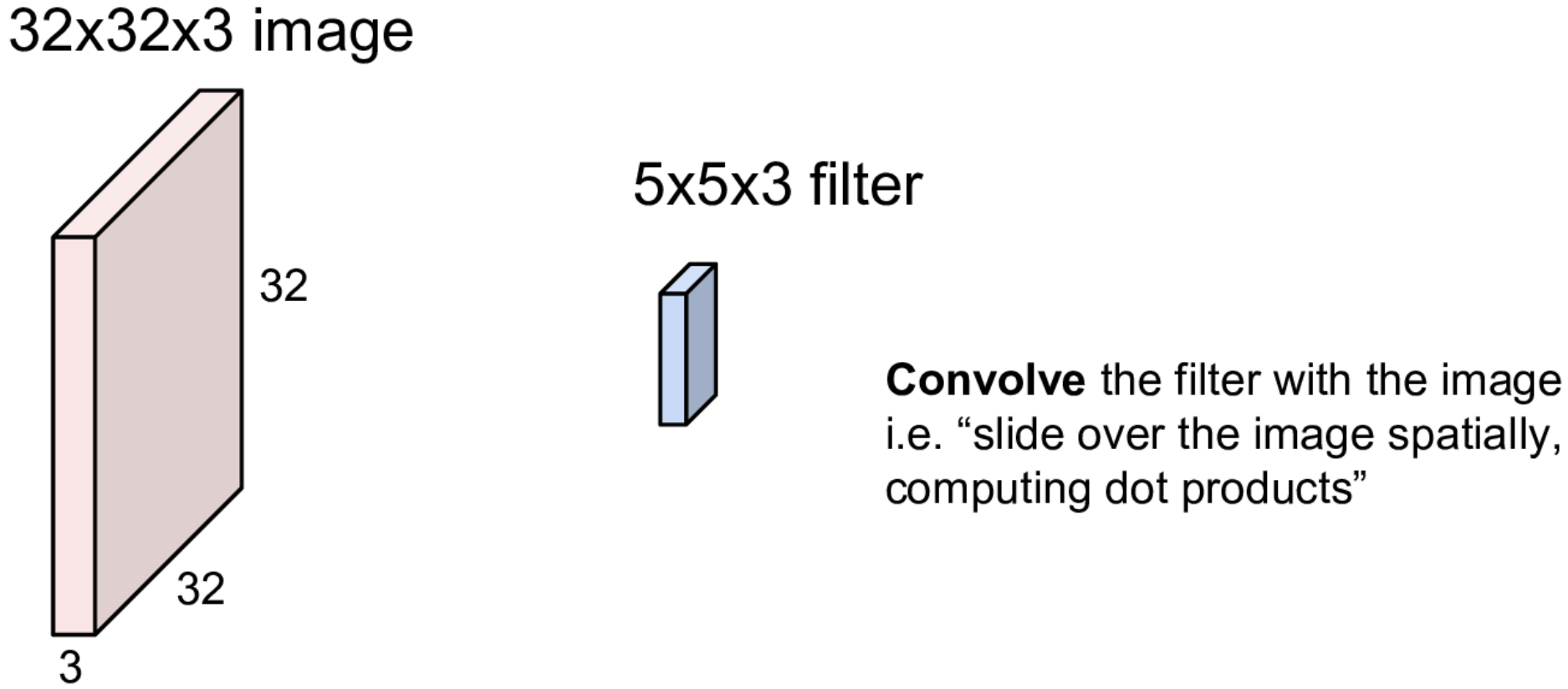
How Multiple Kernels Work

32x32x3 image -> preserve spatial structure



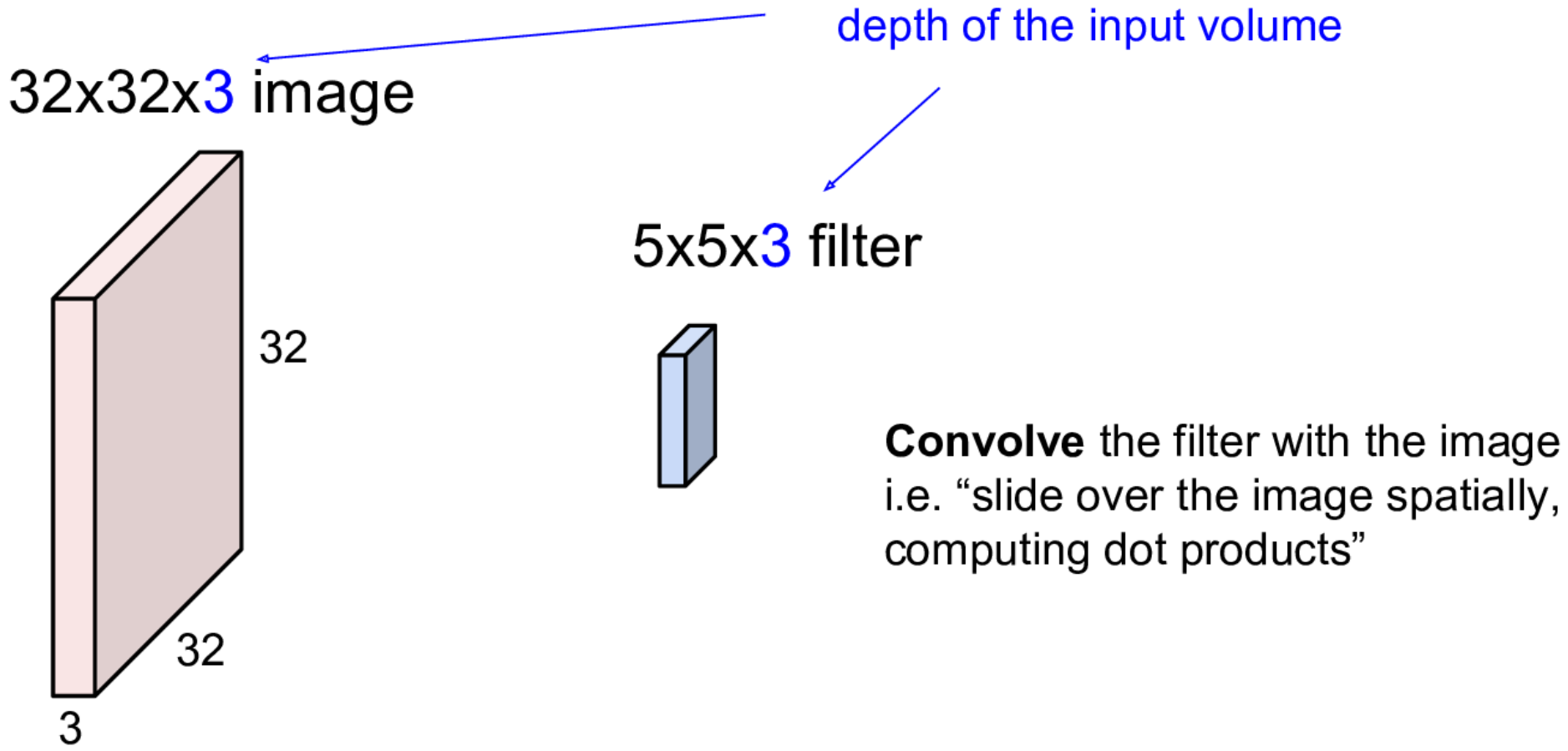
➢ http://cs231n.stanford.edu/slides/2017/cs231n_2017_lecture5.pdf

How Multiple Kernels Work



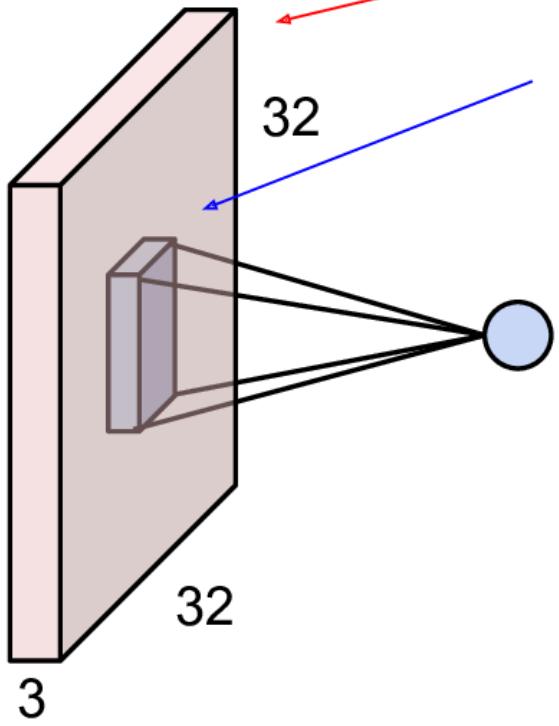
➢ http://cs231n.stanford.edu/slides/2017/cs231n_2017_lecture5.pdf

How Multiple Kernels Work



➢ http://cs231n.stanford.edu/slides/2017/cs231n_2017_lecture5.pdf

How Multiple Kernels Work

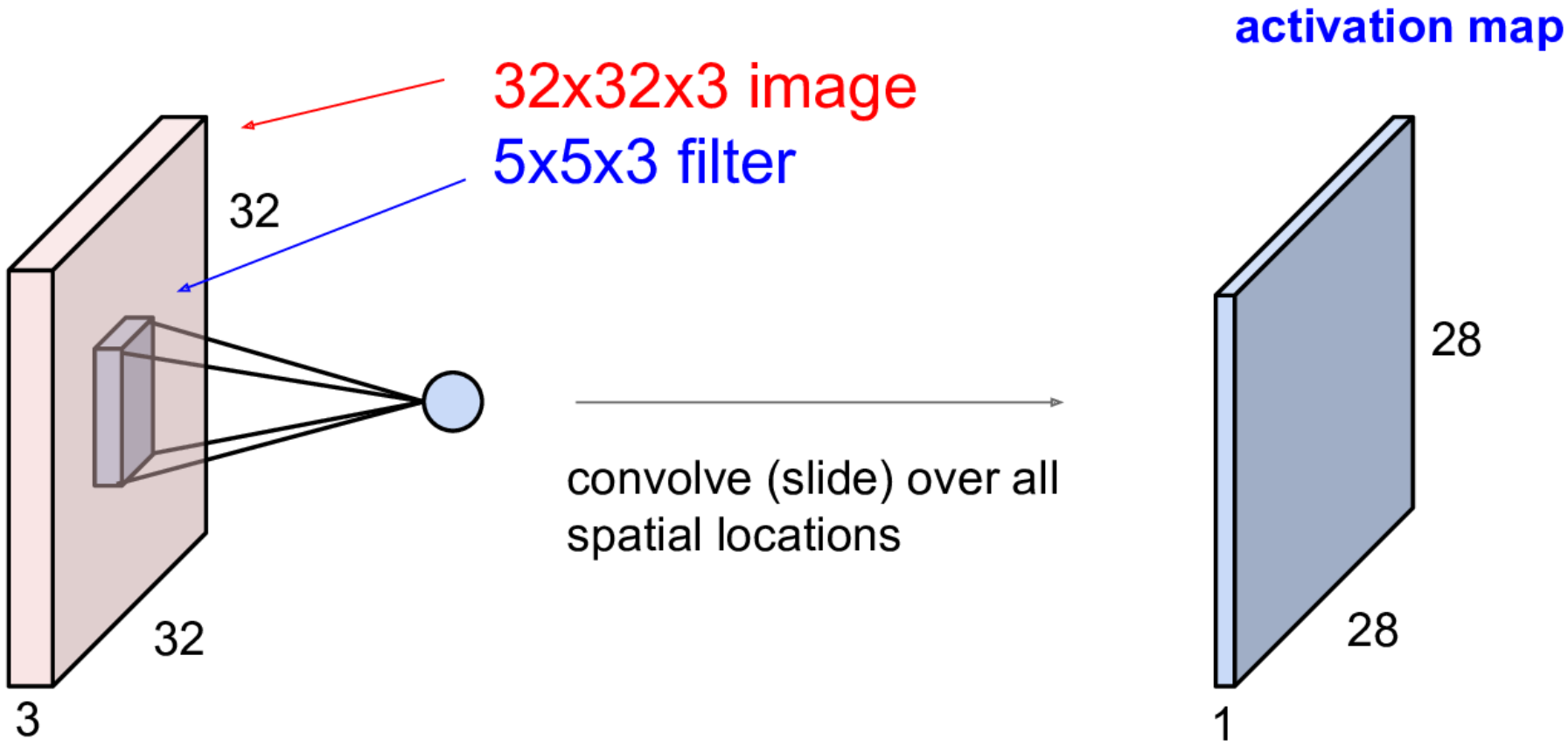


1 number:
the result of taking a dot product between the filter and a small 5x5x3 chunk of the image
(i.e. 5*5*3 = 75-dimensional dot product + bias)

$$w^T x + b$$

➢ http://cs231n.stanford.edu/slides/2017/cs231n_2017_lecture5.pdf

How Multiple Kernels Work

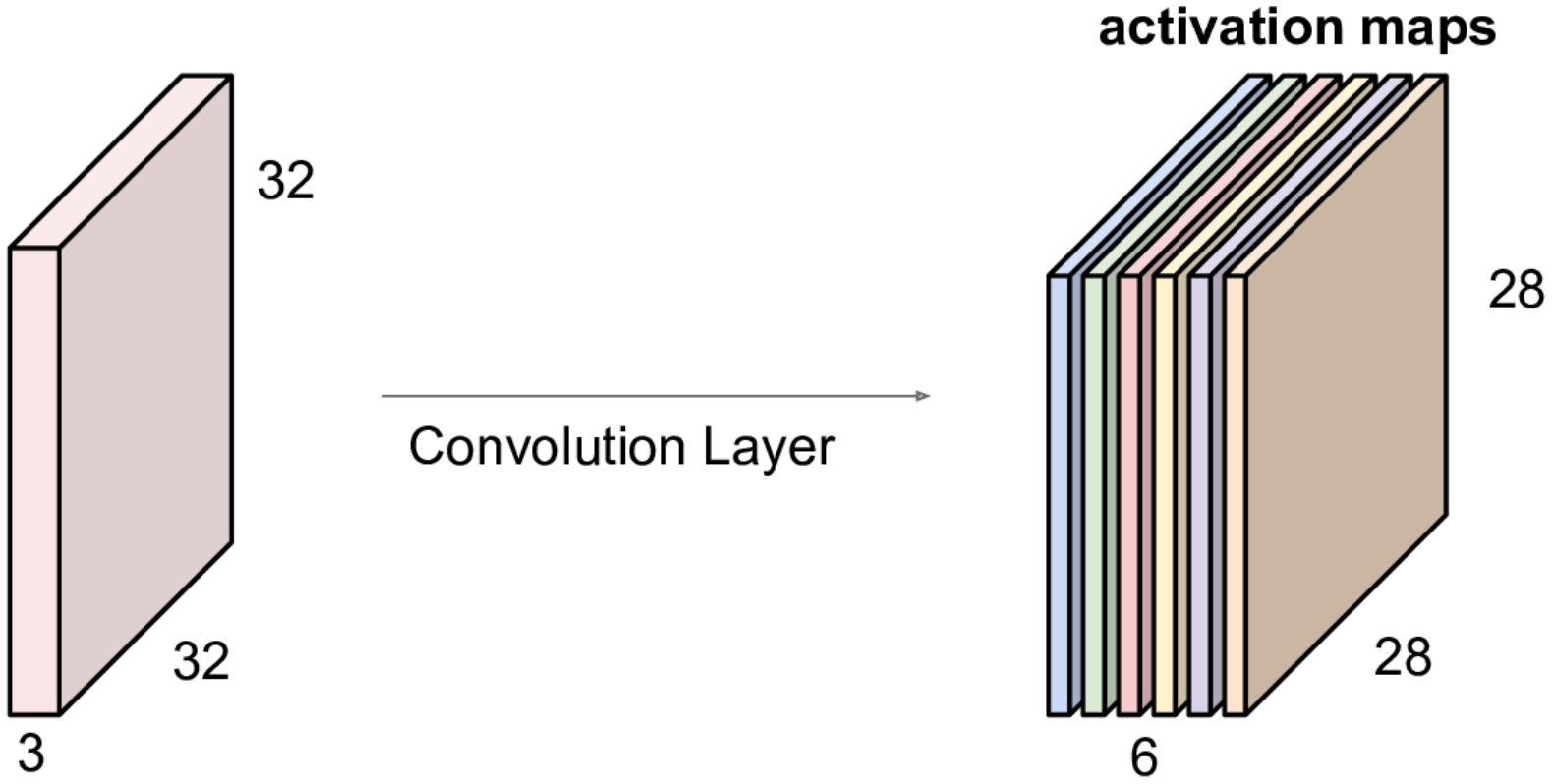


➢ http://cs231n.stanford.edu/slides/2017/cs231n_2017_lecture5.pdf

How Multiple Kernels Work

➢ http://cs231n.stanford.edu/slides/2017/cs231n_2017_lecture5.pdf

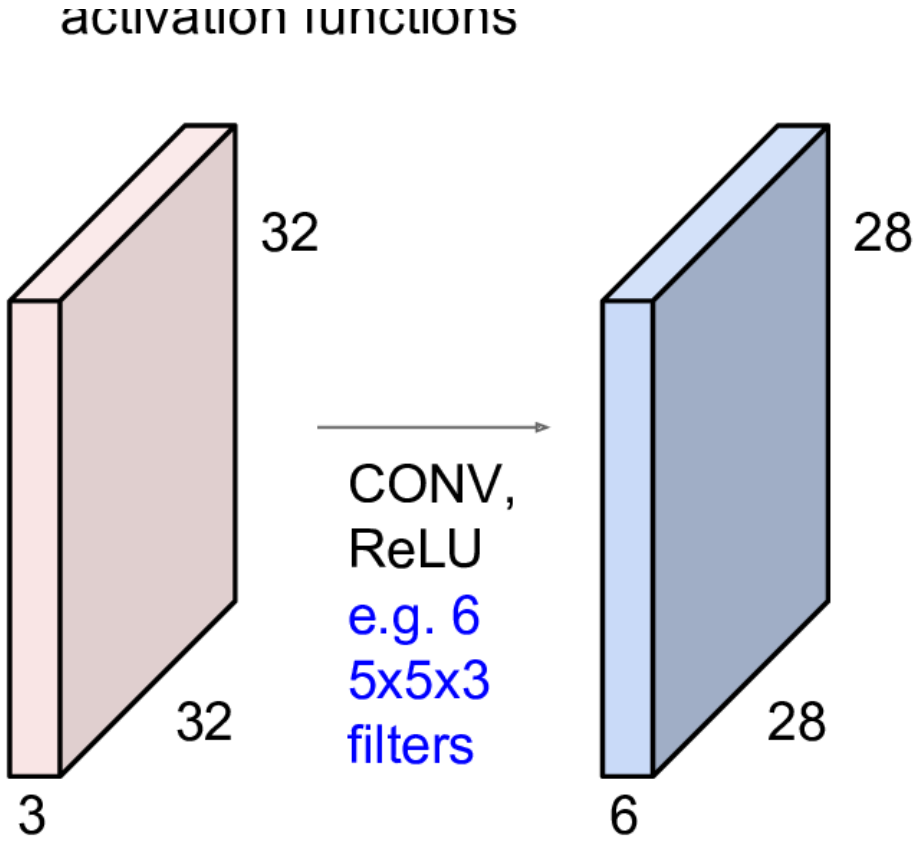
How Multiple Kernels Work



We stack these up to get a “new image” of size 28x28x6!

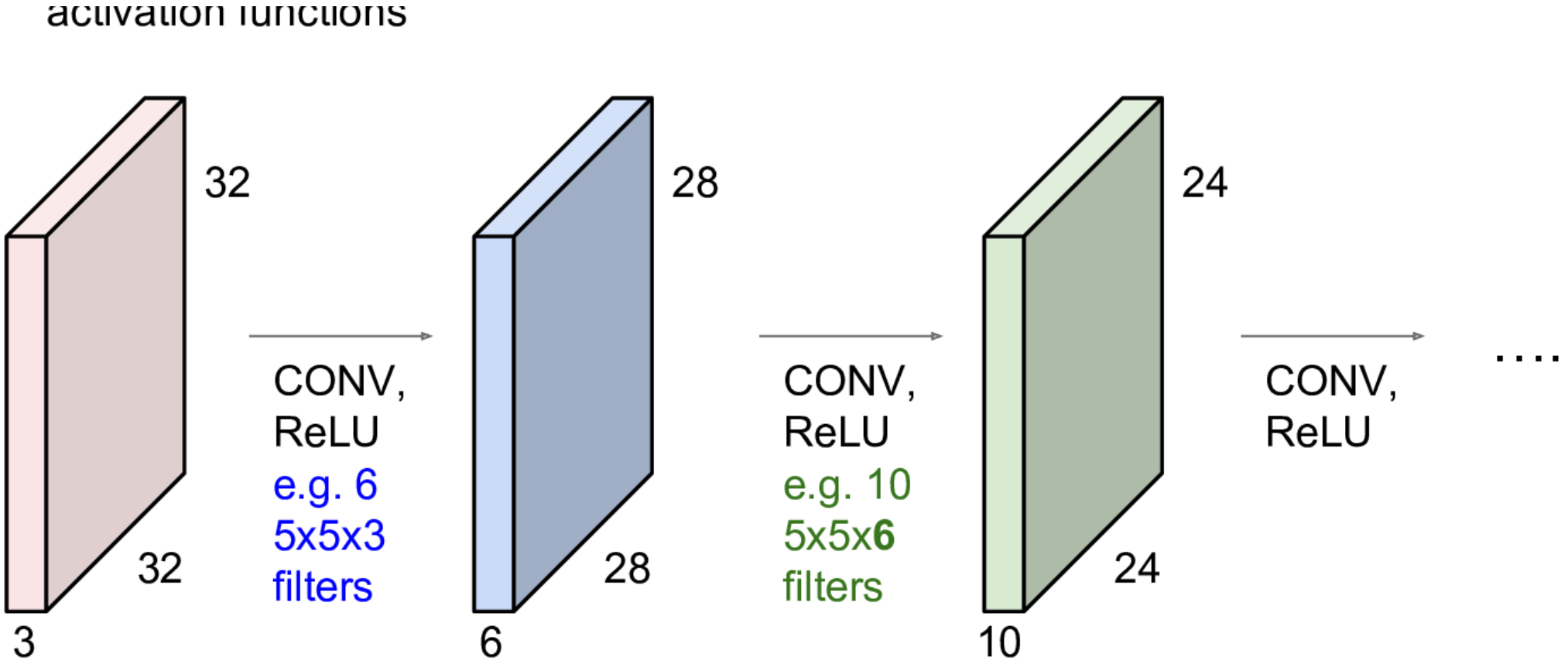
➢ http://cs231n.stanford.edu/slides/2017/cs231n_2017_lecture5.pdf

How Multiple Kernels Work



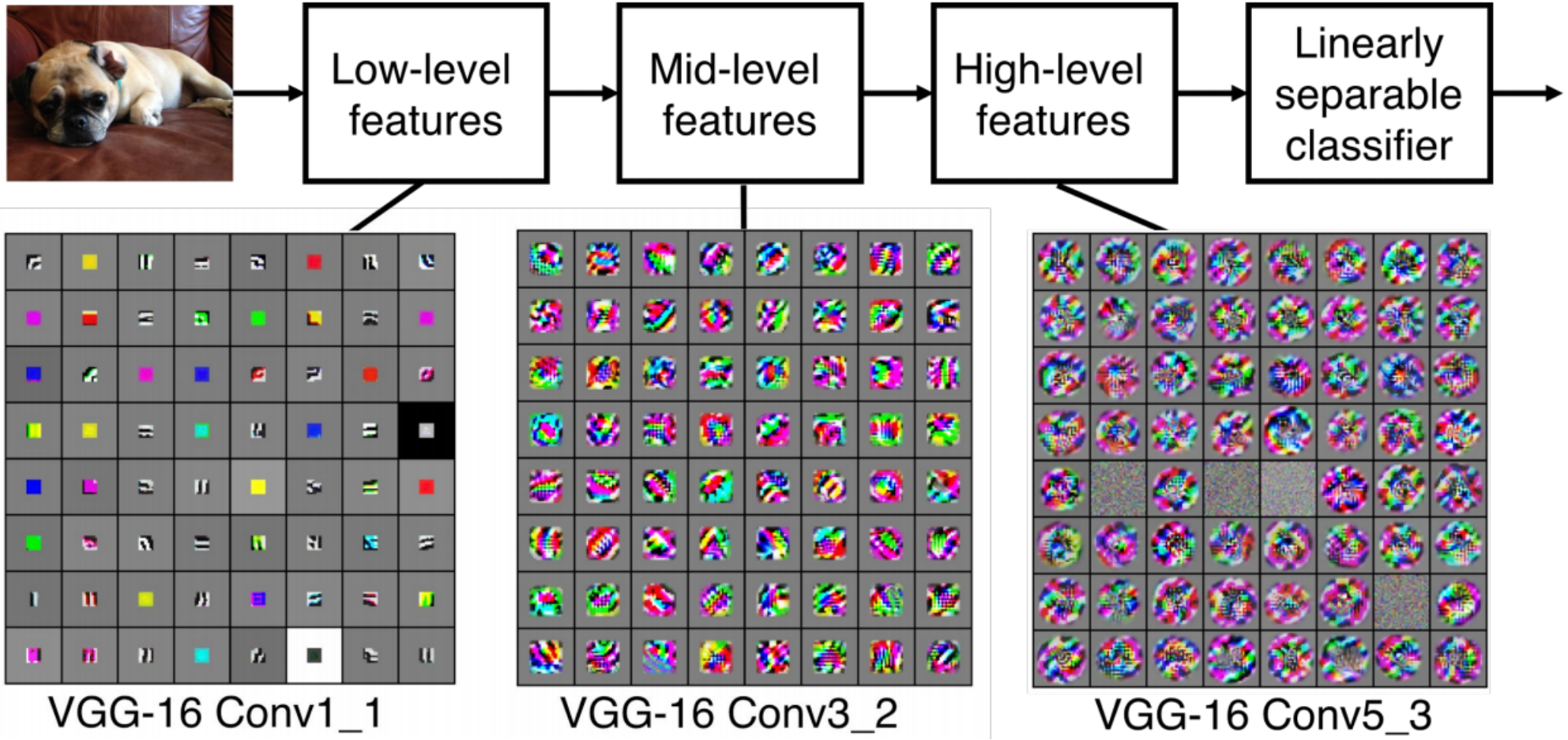
➢ http://cs231n.stanford.edu/slides/2017/cs231n_2017_lecture5.pdf

How Multiple Kernels Work



➢ http://cs231n.stanford.edu/slides/2017/cs231n_2017_lecture5.pdf

How Multiple Kernels Work



Equivariance of Convolution to Translation

- The particular form of parameter sharing leads to equivariance to translation
  - A function $f(x)$ is equivariant to a function g if $f\big(g(x)\big) = g\left(f(x)\right)$
  - Equivariant means that if the input changes, the output changes in the same way

- If g is a function that translates the input, i.e., that shifts it, then the convolution function is equivariant to g
  - $I(x, y)$ is image brightness at point (x, y)
  - $I' = g(I)$ is image function with $I'(x, y) = I(x - 1, y)$
  - If we apply g to I and then apply convolution, the output will be the same as if we applied convolution to I, then applied transformation g to the output

- Convolution enables building equivariant models
- But what about translation invariance?

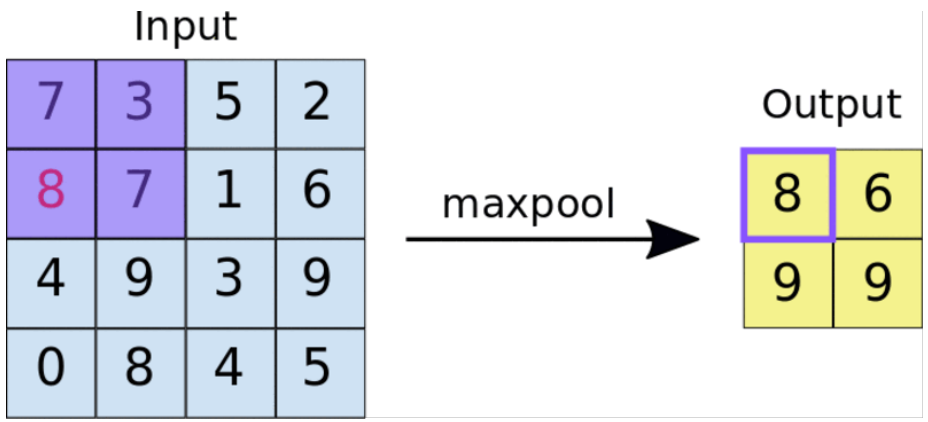
Keywords

- Convolution
- Parameter Sharing
- **Pooling**
- Normalization
- Residual connections

What else do we need?

- We have seen how convolution with multiple filters works and creates a feature-map
- Where do we put our activation functions? Do we need some in the first place?
- We also believe that high-order features are expected to be coarser than the low-level features
- How can we encode that?

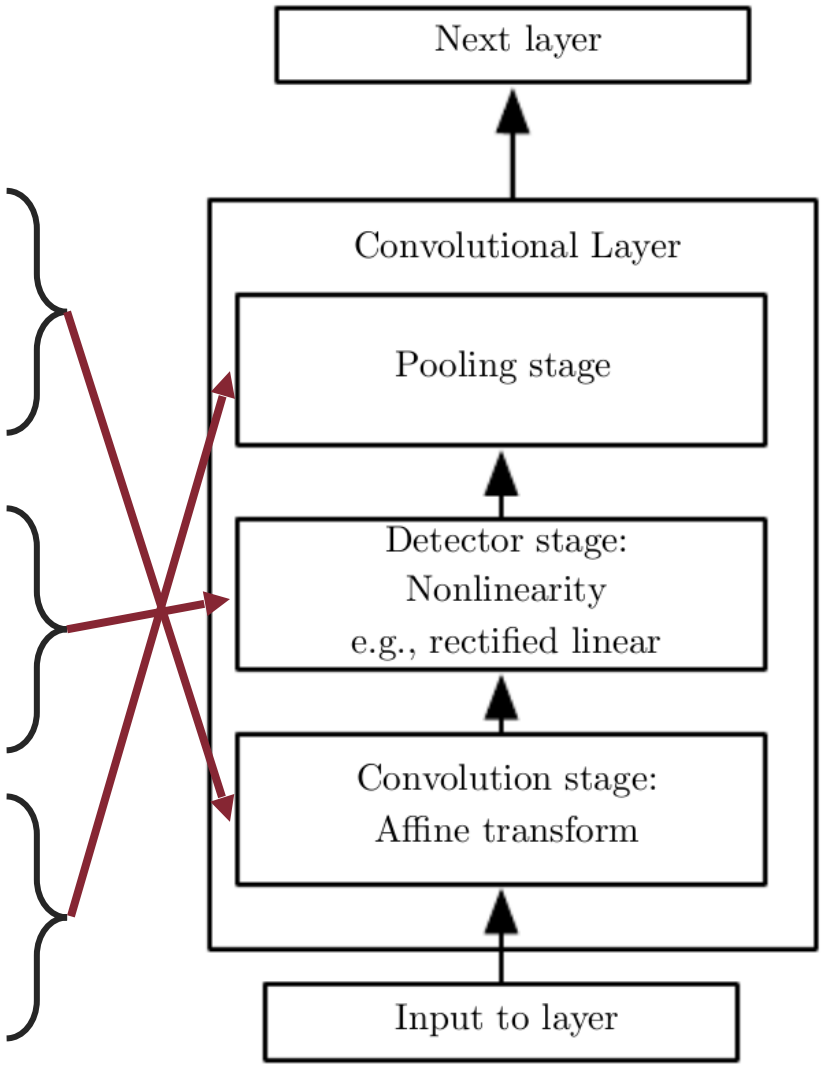
Max Pooling Example



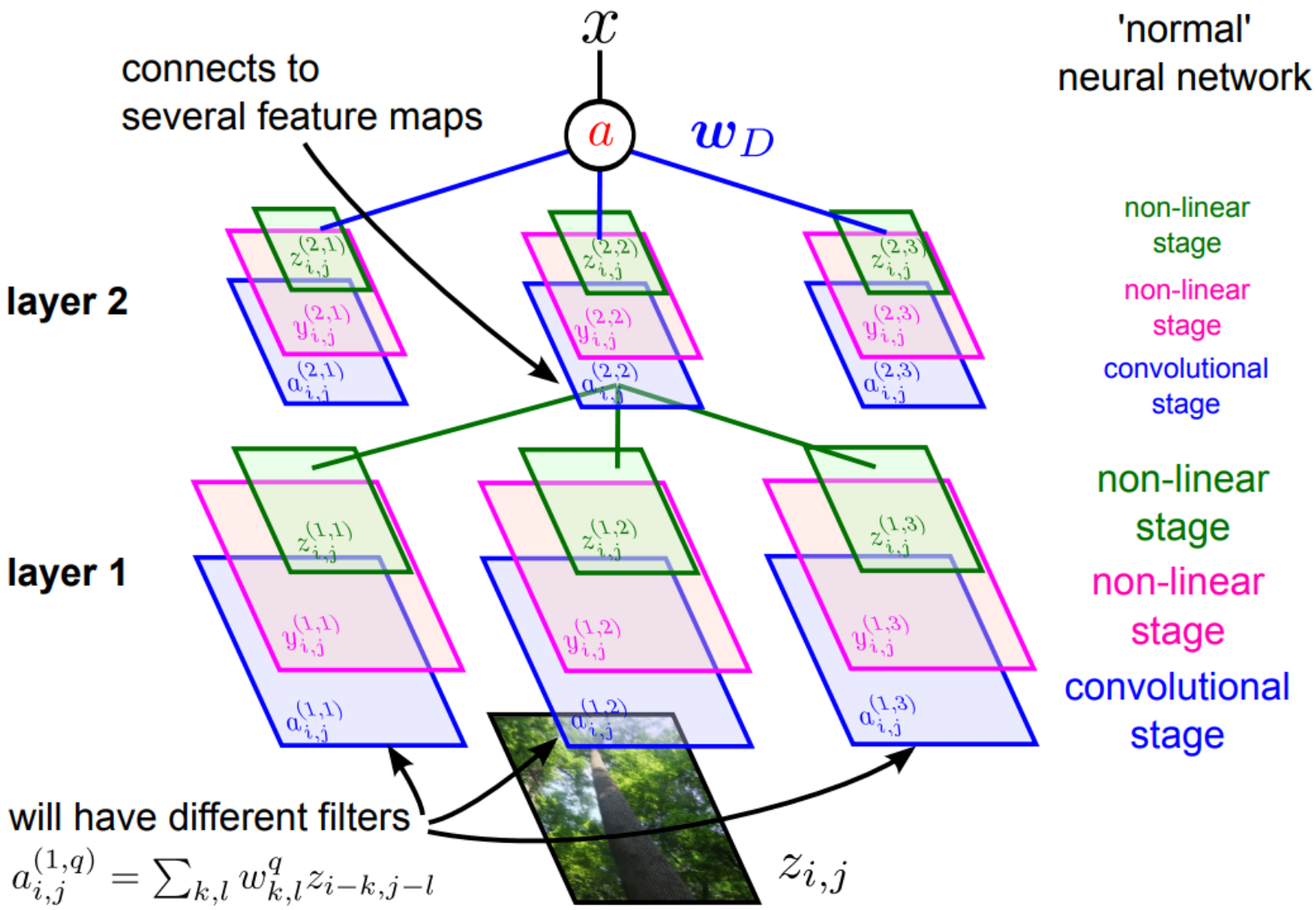
Pooling

Typical layer of a CNN consists of three stages:

- Stage 1 (**Convolution**):
  - Perform several convolutions in parallel to produce a set of linear activations
- Stage 2 (**Detector**):
  - Each linear activation is run through a nonlinear activation function (e.g. ReLU)
- Stage 3 (**Pooling**):
  - Use a pooling function to modify output of the layer further



Classic Setup for a Convolutional Network



Types of Pooling functions

➢ A pooling function replaces the output of the net at a certain location with a summary statistic of the nearby inputs

Popular pooling functions:

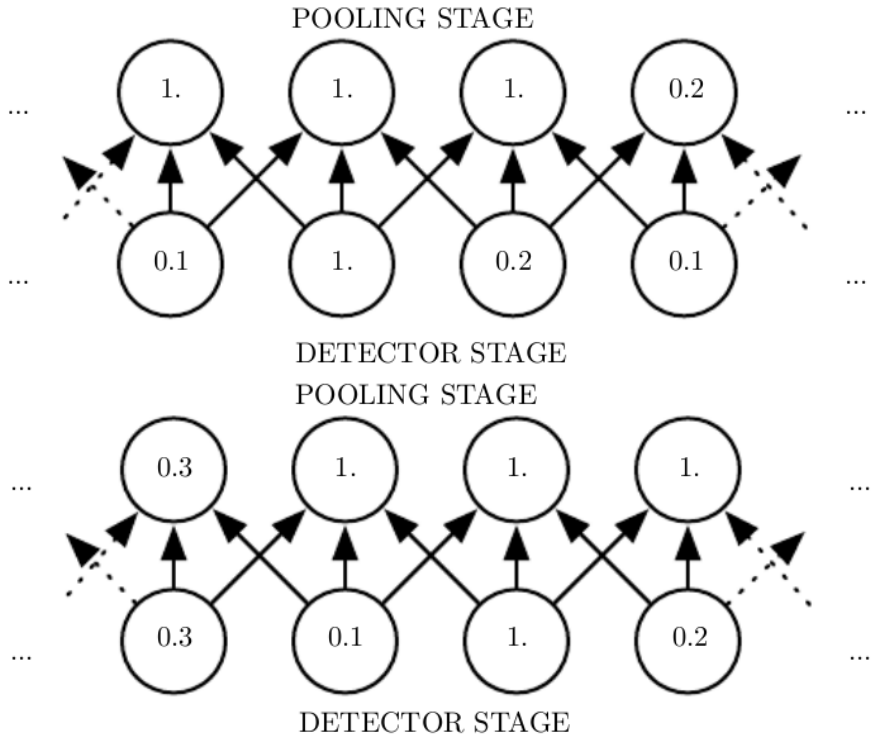
1. Max pooling operation reports the maximum output within a rectangular neighborhood
2. Average of a rectangular neighborhood
3. L^2 norm of a rectangular neighborhood
4. Weighted average based on the distance from the central pixel

Why Pooling?

- In all cases, pooling helps make the representation become approximately invariant to small translations of the input
- If we translate the input by a small amount values of most of the outputs does not change
- **Pooling can be viewed as adding a strong prior that the function the layer learns must be invariant to small translations**

Max pooling introduces invariance to translation

- View of middle of output of a convolutional layer
- Same network after the input has been shifted by one pixel
- Every input value has changed, but only half the values of output have changed



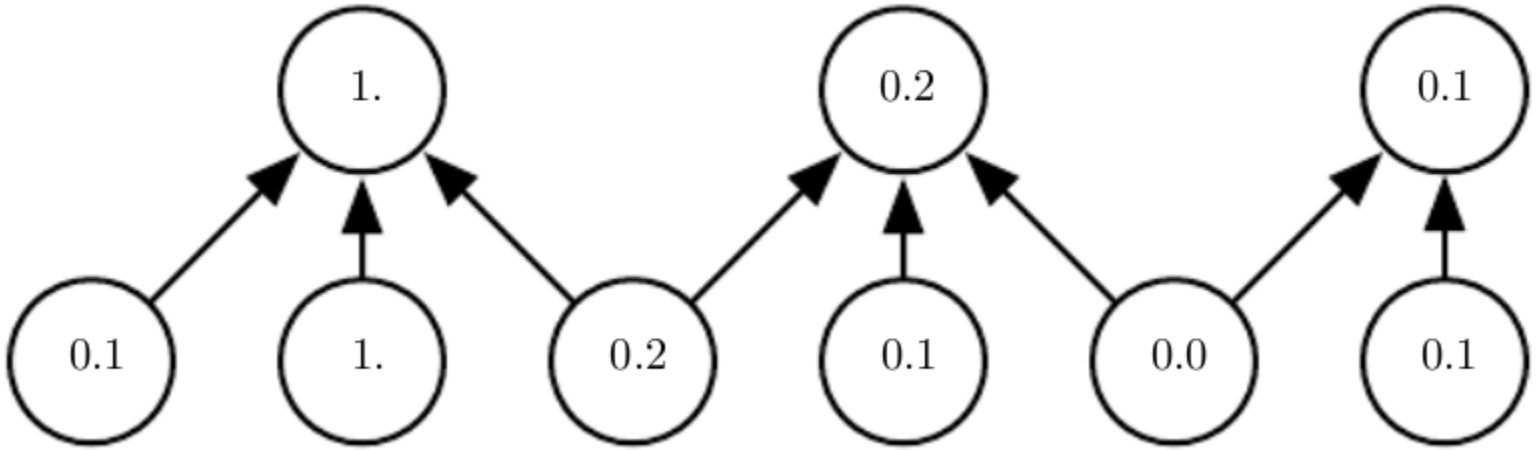
Importance of Translation Invariance

- Invariance to translation is important if we care about whether a feature is present rather than exactly where it is
  - For detecting a face, we just need to know that an eye is present in a region, not its exact location

- In other contexts, it is more important to preserve location of a feature
  - E.g., to determine a corner, we need to know whether two edges are present and test whether they meet

Pooling with Down-Sampling

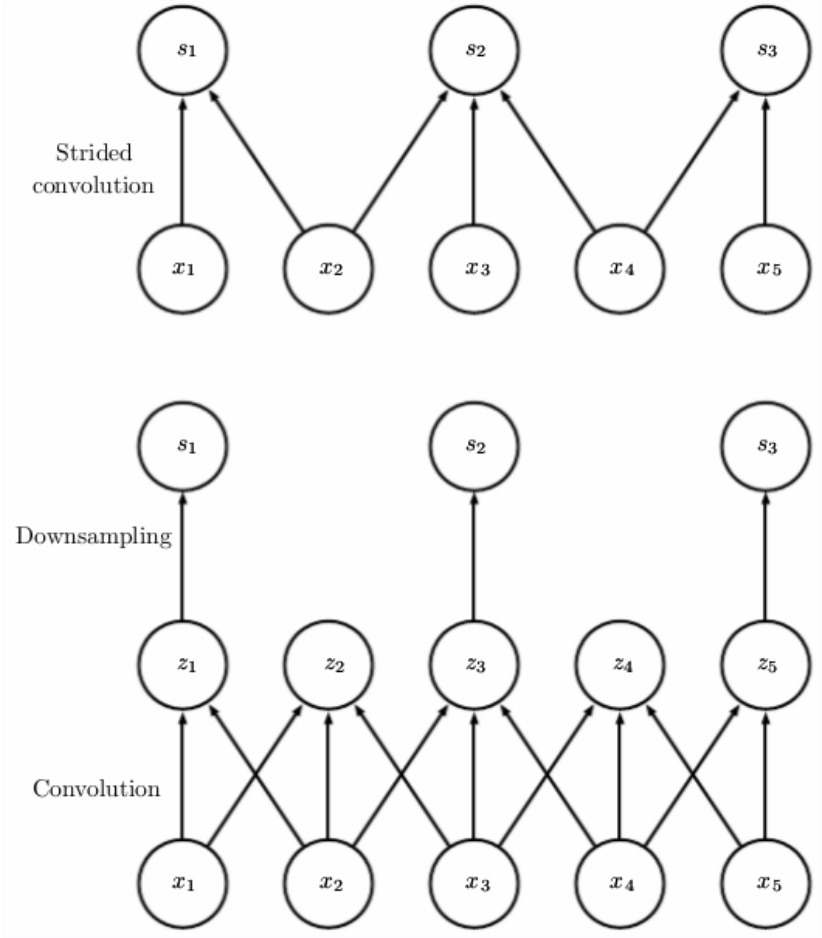
- Because pooling summarizes the responses over a whole neighborhood, it is possible to use fewer pooling units than detector units
  - By reporting summary statistics for pooling regions spaced k pixels apart rather than one pixel apart
  - Next layer has k times fewer inputs to process



Keywords

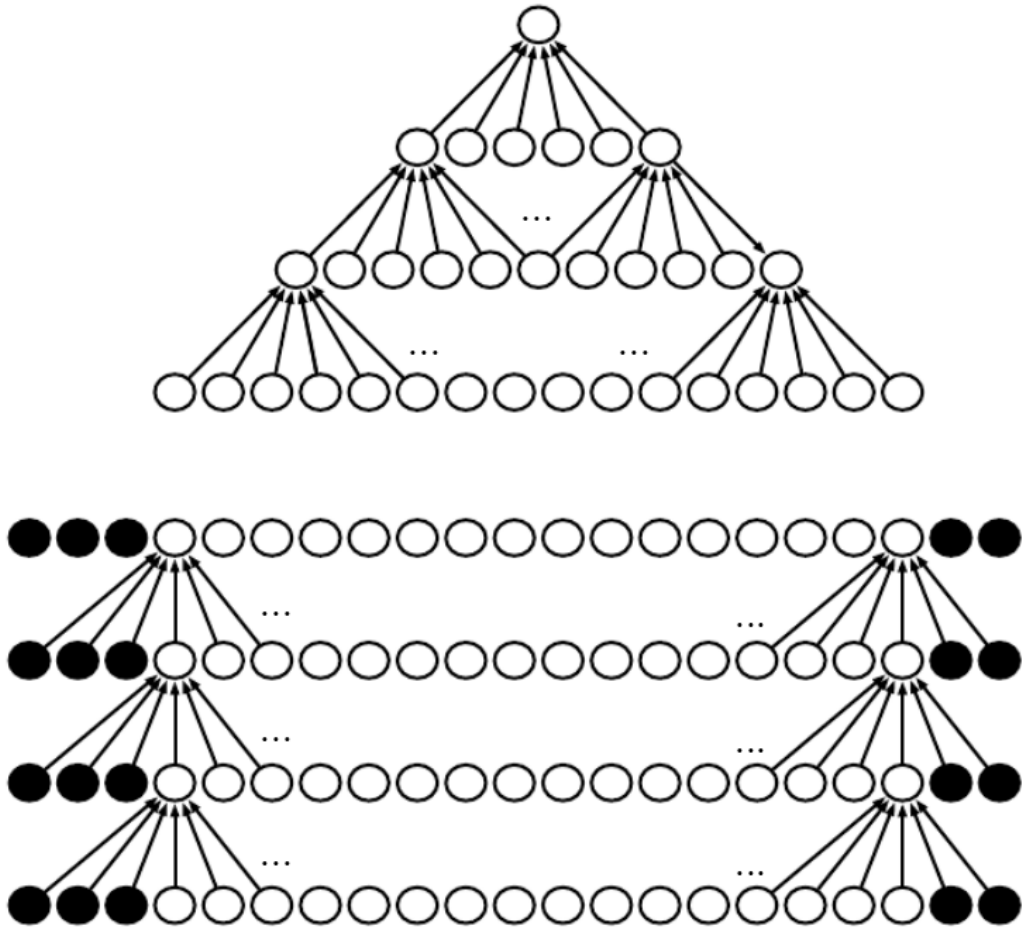
- **Convolution (variants)**
- Parameter Sharing
- Pooling
- Normalization
- Residual connections

Convolution with a stride: Implementation

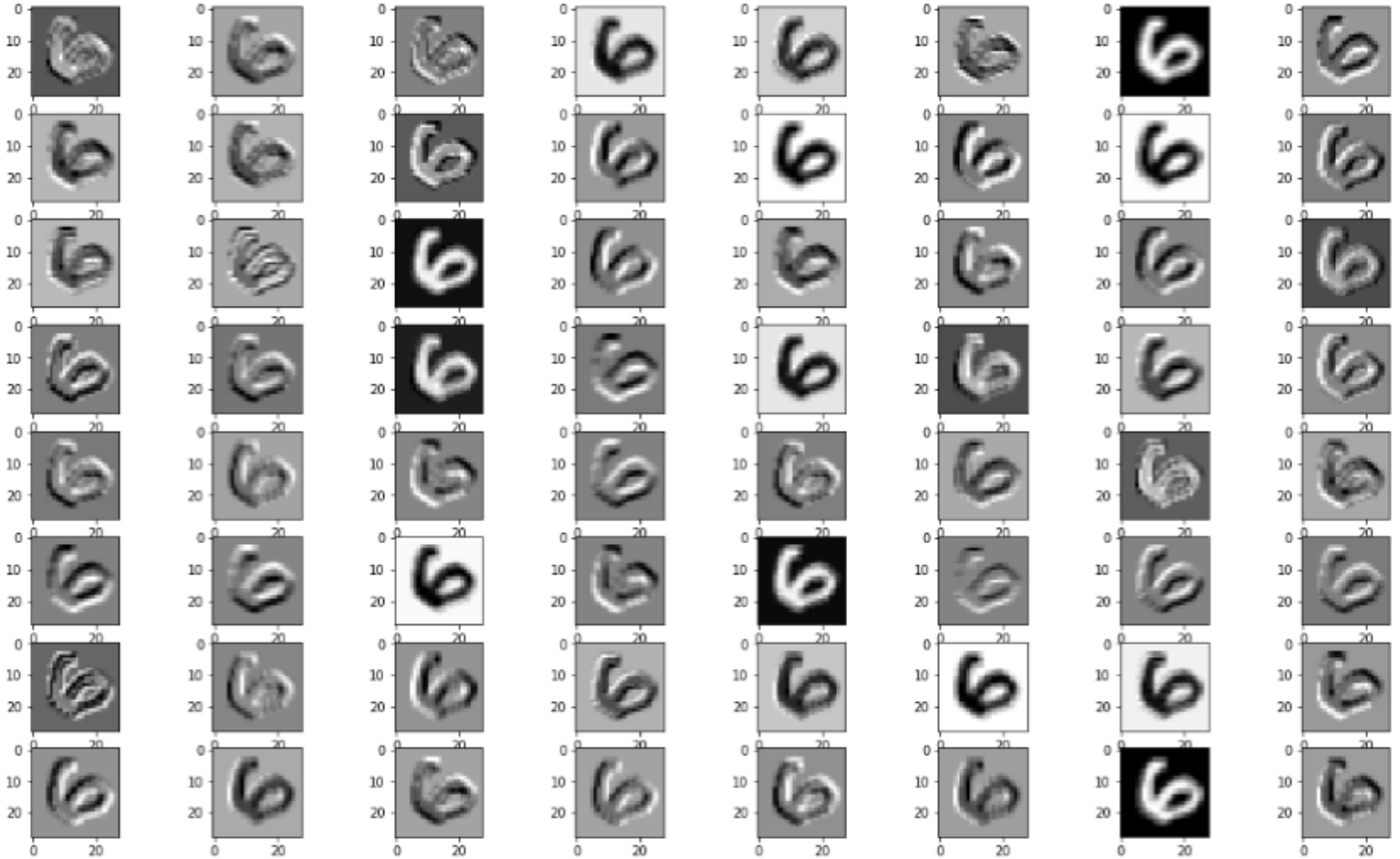


- Strided convolution is equivalent to normal convolution followed by a downsampling
- Second approach is computationally wasteful

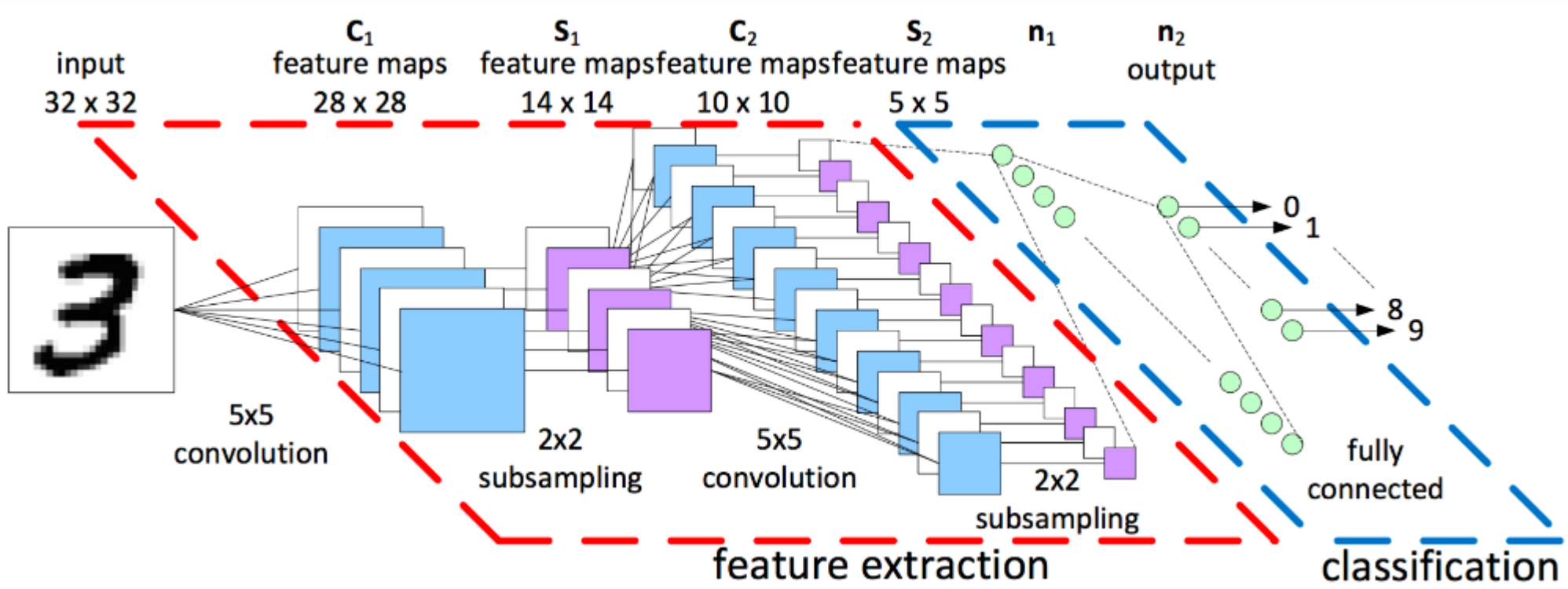
Effect of Zero-padding on network size



Visualize Activations

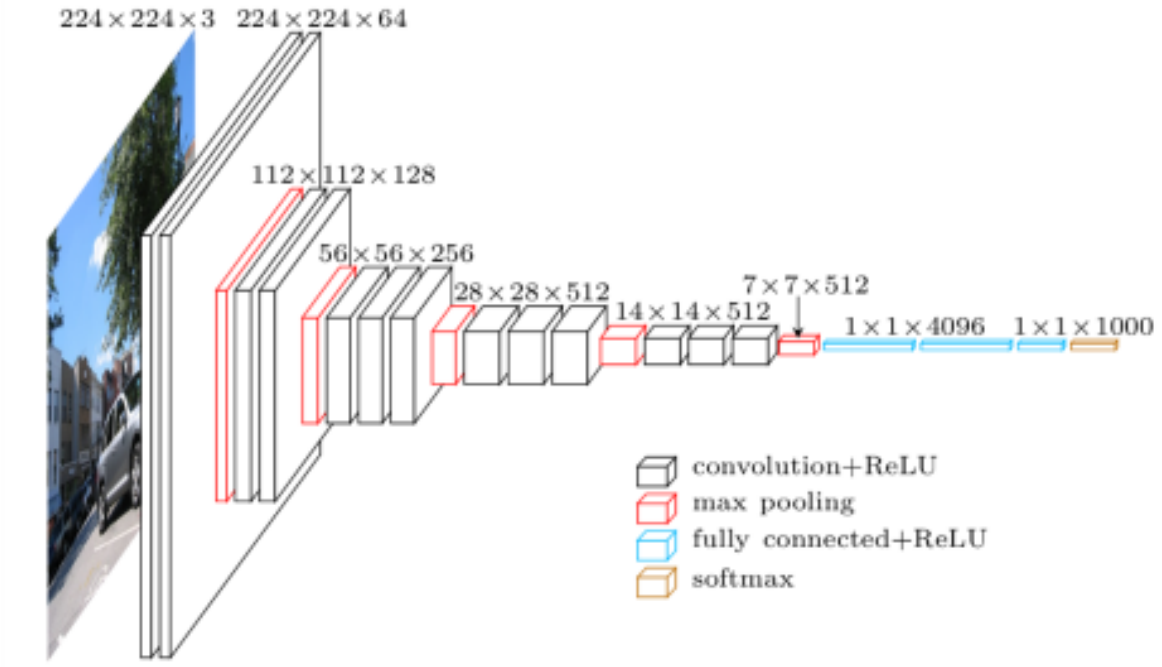


Example for a Convolutional Network



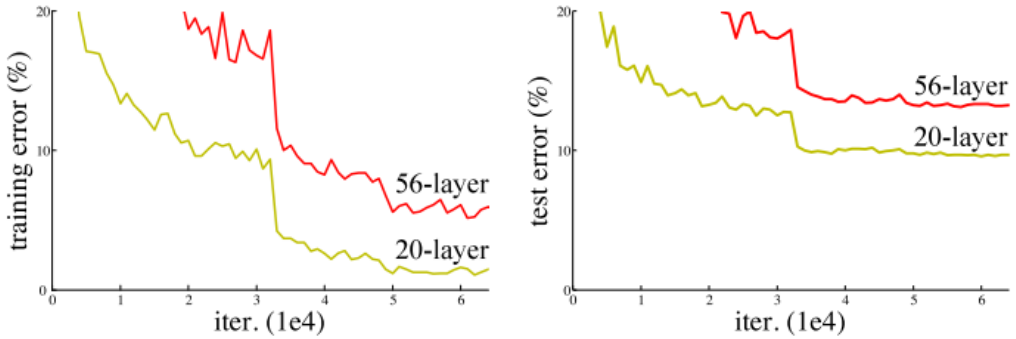
VGG Net

- VGG is a convolutional neural network model
  - "Very Deep Convolutional Networks for Large-Scale Image Recognition"
- The model achieves 93% top-5 test accuracy in ImageNet which is a dataset of over 14 million images belonging to 1000 classes.



Problems with Deep Networks

- Generally observed:
  - Deeper networks become increasingly hard to train
  - Gradients vanish or explode
  - Information is required to travel through a long distance

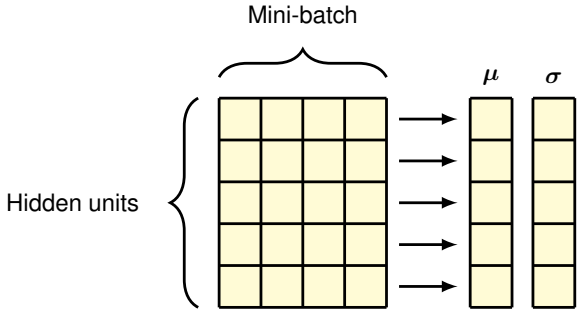


- Solution:
  - Ease training of deep network with more direct influence onto the gradient

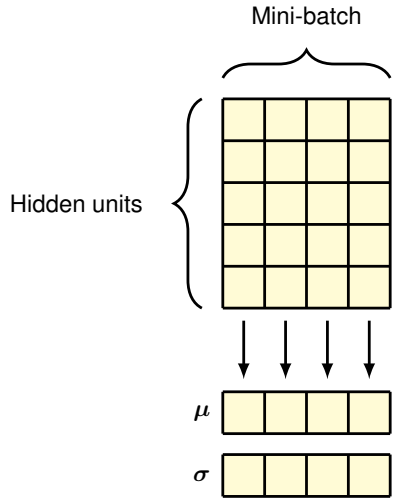
Keywords

- Convolution
- Parameter Sharing
- Pooling
- **Normalization**
- Residual connections

Normalization



- Normalizing the input data is important for NN training.
- We can also normalize the activations after each layer
- helps stabilize training of deep architectures
- **BatchNorm**: Calculate mean and SD over batch dimension
- **LayerNorm**: Calculate mean and SD over activation dimension
- Then subtract mean from activations and scale them with SD



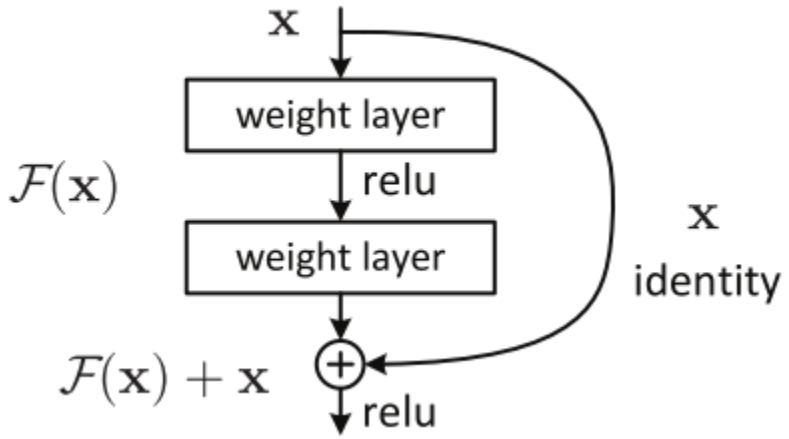
- **Caveat**: BatchNorm depends on batch size, can be noisy for small batches

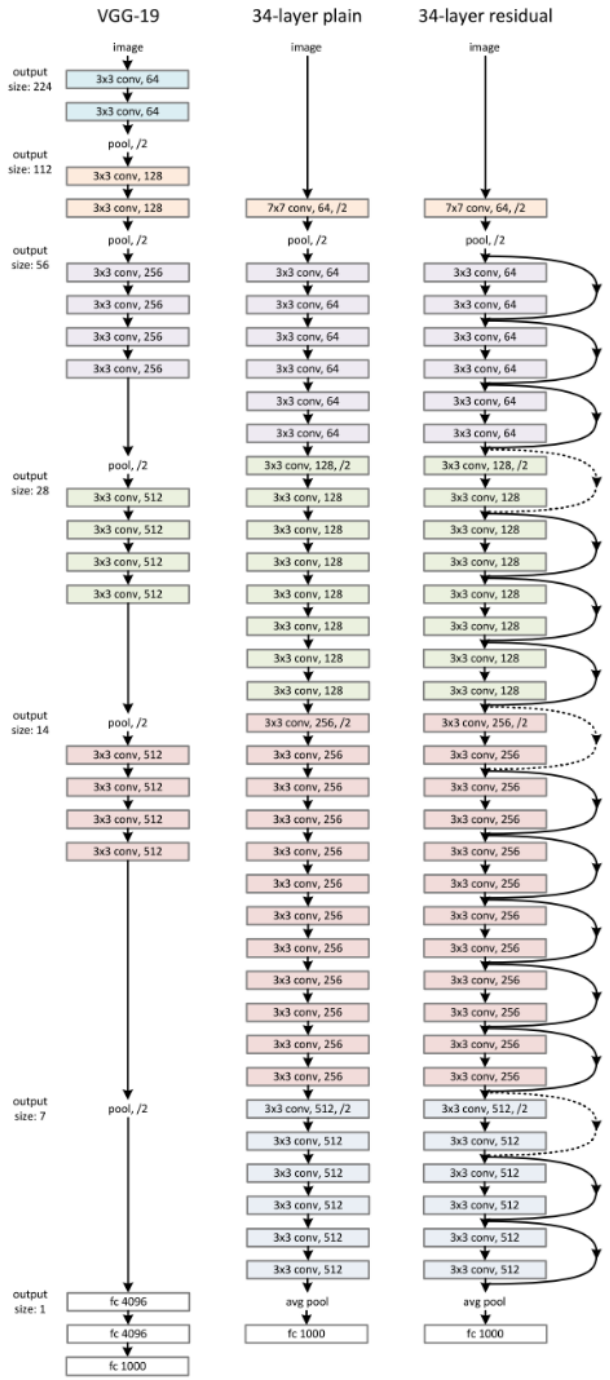
Keywords

- Convolution
- Parameter Sharing
- Pooling
- Normalization
- **Residual connections**

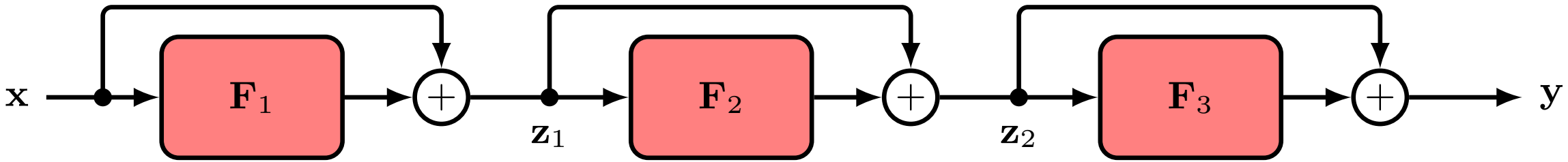
Idea of ResNet

- The network is constructed in blocks
- Introduces the identity shortcut connections

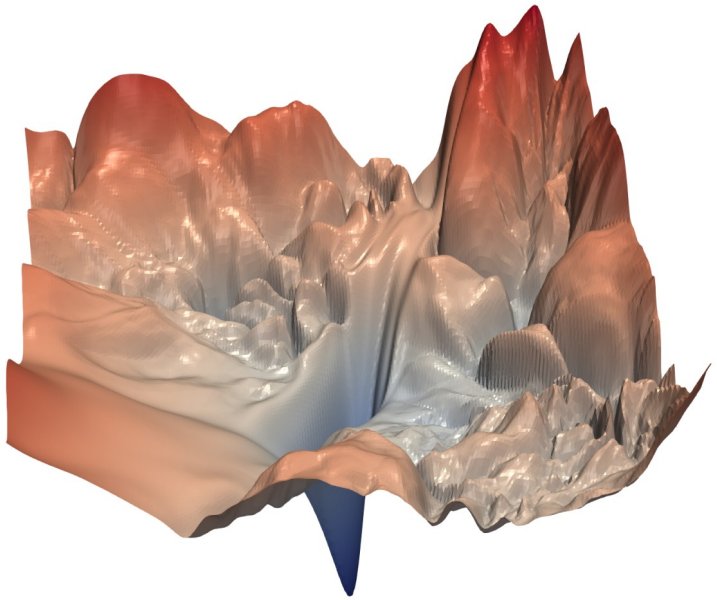




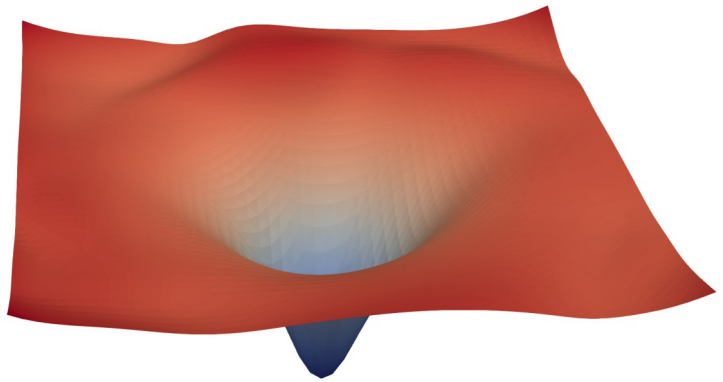
Residual Connections as a Regularizer



Loss landscape without residual connections



(a)

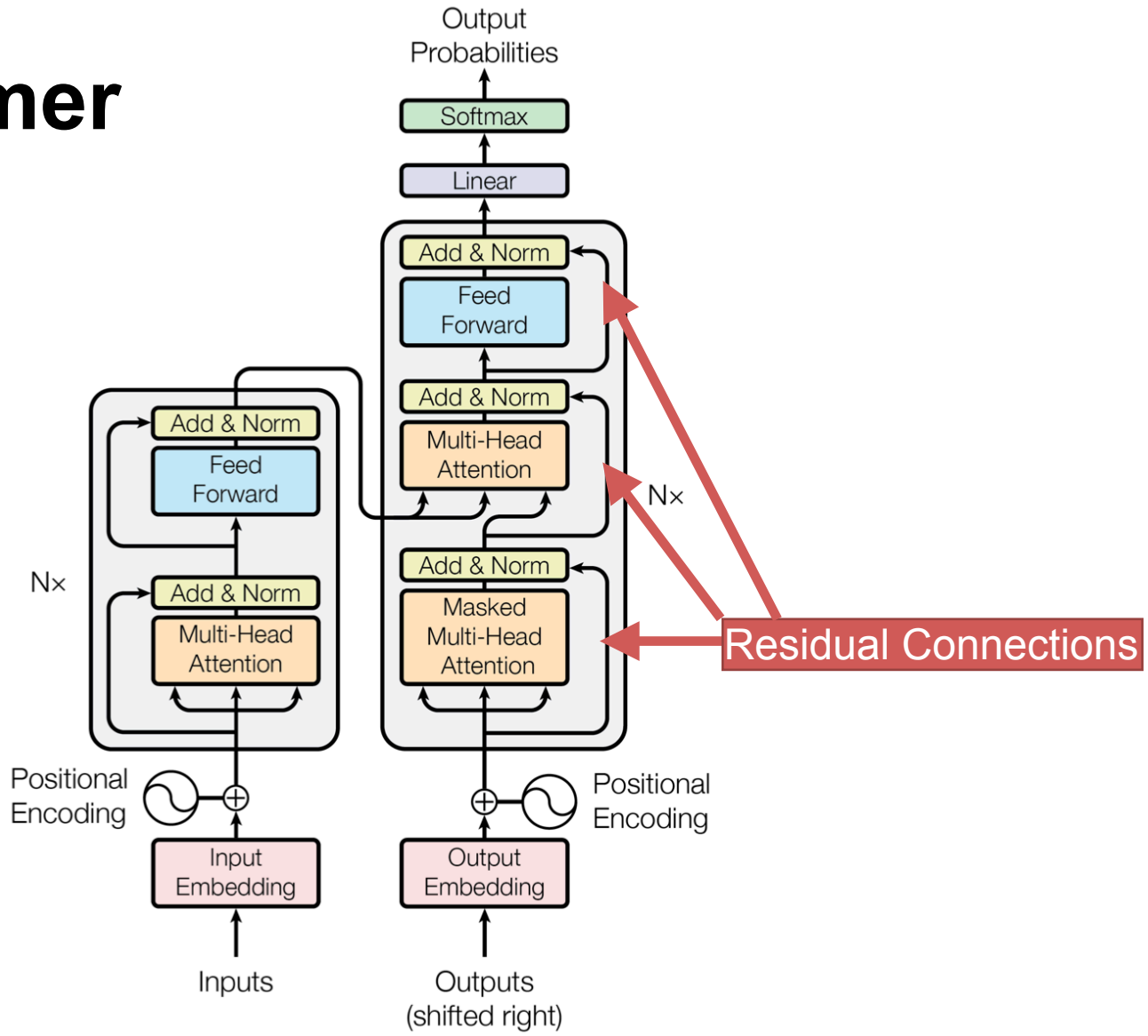


(b)

Loss landscape **with** residual connections

Preview: The Transformer

Vaswani et al. (2017)



Keywords

- Convolution
- Parameter Sharing
- Pooling
- Normalization
- Residual connections

How’s my teaching?

https://www.admonymous.co/lukasgalke

Department of Mathematics
and Computer Science

sdu.dk

#sdudk

Advanced Machine Learning

Gradient-based Learning & Backpropagation

Lukas Galke Poech

Spring 2026

SDU

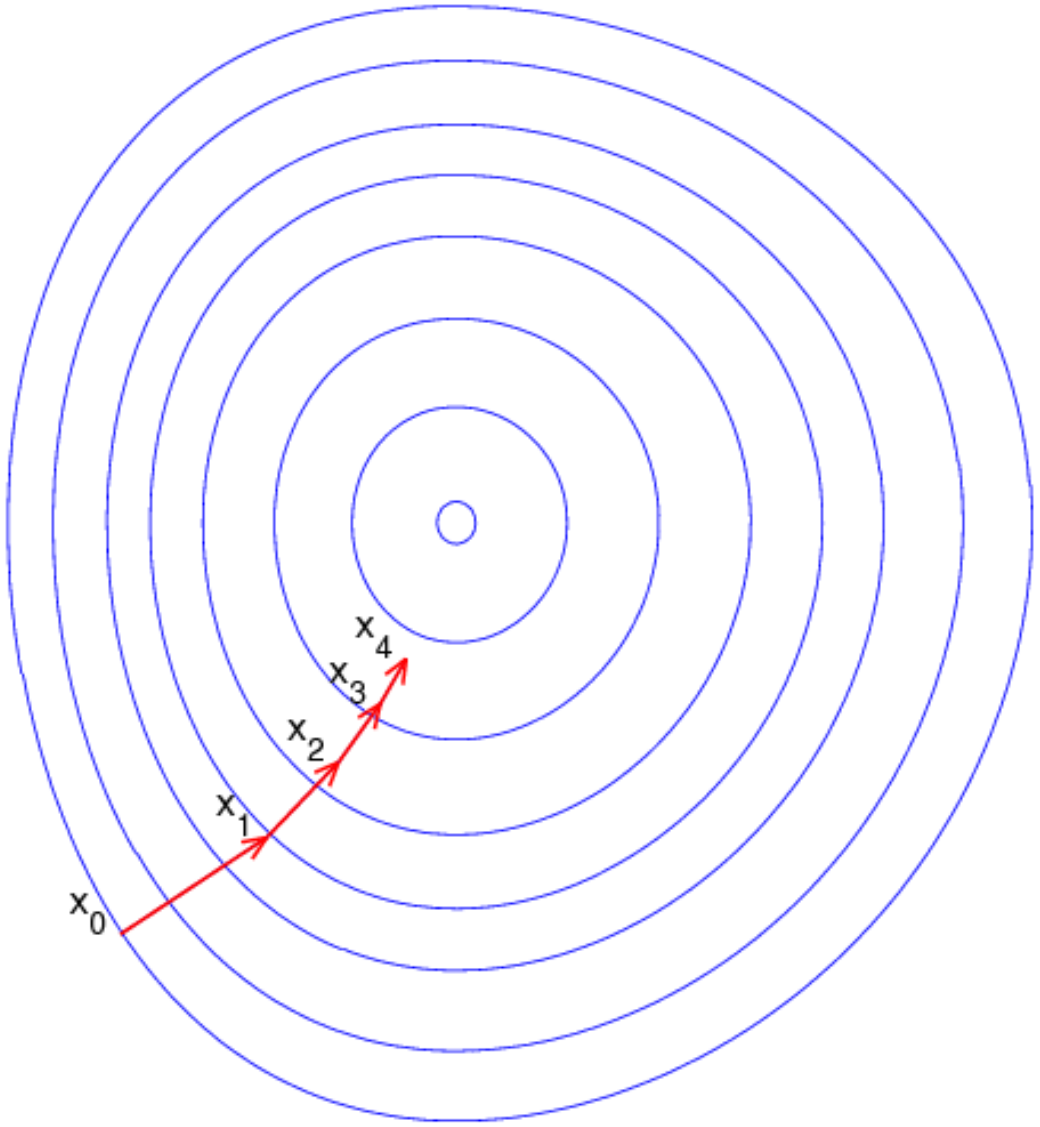
Keywords

- **Gradient-based learning**
- Stochastic gradient descent
- Challenges of optimization
- Backpropagation

Gradient Based Learning

The Central Idea

- Update the model parameters following the steepest slope



More Mathematically

- Suppose function $y = f(x)$
- Derivative of function denoted: $f'(x)$ or as dy/dx
  - Derivative $f'(x)$ gives the slope of $f(x)$ at point x
  - It specifies how to scale a small change in input to obtain a corresponding change in the output:

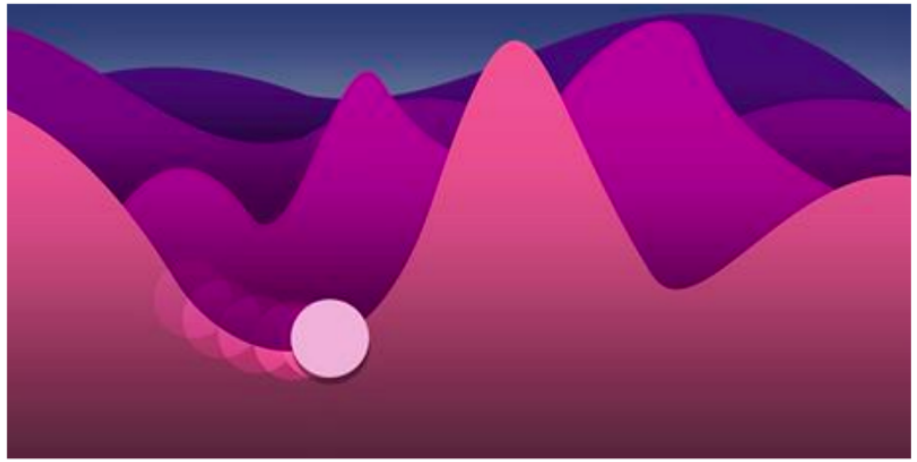
$$f(x + \varepsilon) \approx f(x) + \varepsilon f'(x)$$

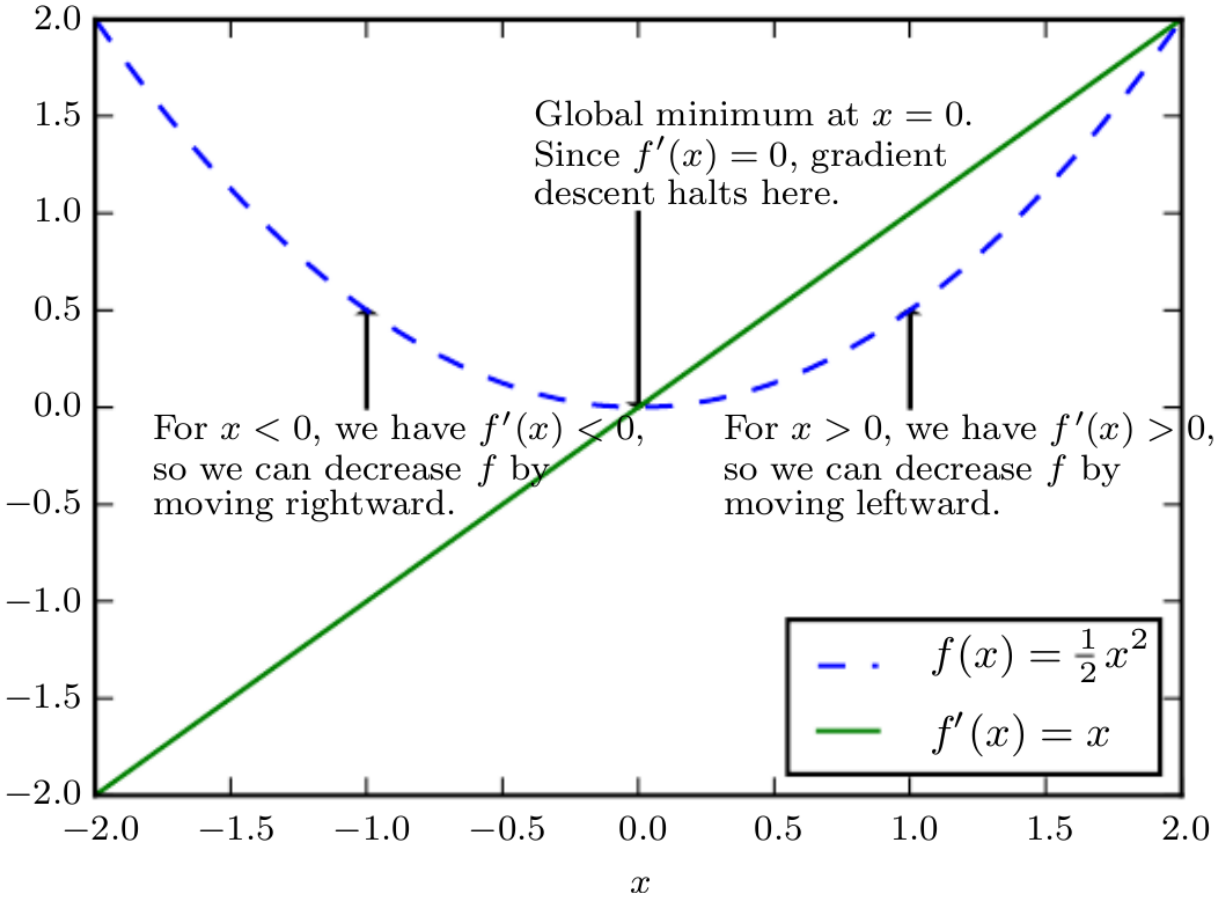
- We know that

$$f(x - \varepsilon \text{sign}(f'(x)))$$

  is less than $f(x)$ for small ε.
  - Thus we can reduce f (x) by moving x in small steps with opposite sign of derivative
  - This technique is called gradient descent (Cauchy 1847)

Usage of Derivates





Functions with multiple inputs

- Need partial derivatives

$$\frac{\partial}{\partial x_i} f(\boldsymbol{x})$$

- Measures how f changes as only variable x_i increases at point $\boldsymbol{x}$

- Gradient is vector containing all of the partial derivatives denoted with $\nabla_{\boldsymbol{x}} f(\boldsymbol{x})$
  - Element i of the gradient is the partial derivative of f wrt x_i
  - Critical points are where every element of the gradient is equal to zero
  - A function can be minimized when moving in the direction opposite to the gradient

Functions with multiple inputs

➢ We can decrease f by moving in the direction of the negative gradient vector

➢ Steepest descent proposes a new point

$$\boldsymbol{x}' = \boldsymbol{x} - \epsilon \nabla_{\boldsymbol{x}} f(\boldsymbol{x})$$

➢ With ϵ being the learning rate (there are many methods of defining ϵ)

➢ Ascending an objective function of discrete parameters is called hill climbing

Keywords

- Gradient-based learning
- **Stochastic gradient descent**
- Challenges of optimization
- Backpropagation

Stochastic Gradient Descent (SGD)

- A recurring problem in machine learning:
  - large training sets are necessary for good generalization
  - but large training sets are also computationally expensive

- Nearly all deep learning is powered by one very important algorithm: **Stochastic Gradient Descent**

Insight of SGD

- Insight: Gradient descent based on only a sample (we don't have the universe as data) is an **expectation**
  - Expectation may be approximated using small set of samples

- In each step of SGD we can sample a minibatch of examples

$$B = \left\{ \boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(m')} \right\}$$

  - drawn uniformly from the training set
  - Minibatch size m' is typically chosen to be small: 1 to a hundred
  - Crucially m' is held fixed even if sample set is in billions
  - We may fit a training set with billions of examples using updates computed on only a hundred examples

SGD Estimate on minibatch

➢ Estimate of gradient is formed as

$$\boldsymbol{g} = \frac{1}{m'} \nabla_{\boldsymbol{\theta}} \sum_{i=1}^{m'} L\left(\boldsymbol{x}^{(i)}, y^{(i)}, \boldsymbol{\theta}\right)$$

using only the examples of the minibatch

➢ SDG then simply follows the estimated gradient downhill

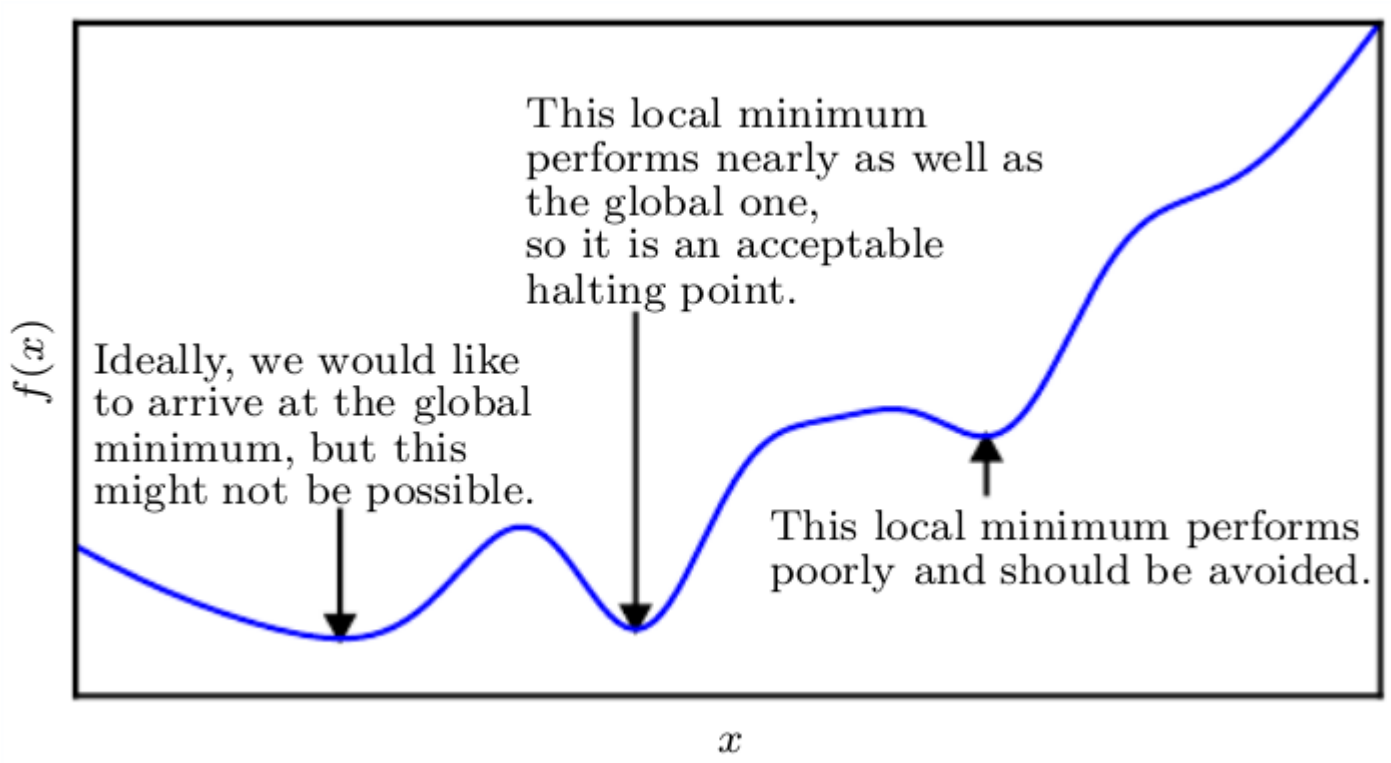
$$\boldsymbol{\theta}' = \boldsymbol{\theta} - \epsilon \boldsymbol{g}$$

How good is SGD?

- In the past gradient descent was regarded as slow and unreliable
- Application of gradient descent to non-convex optimization problems was regarded as unprincipled
- SGD is not guaranteed to arrive at even a local minimum in reasonable time

- **But it often finds a very low value of the cost function quickly enough**
- As $m \to \infty$ the model will eventually converge to its best possible test error before SGD has sampled every example

Good Enough in Practice



Keywords

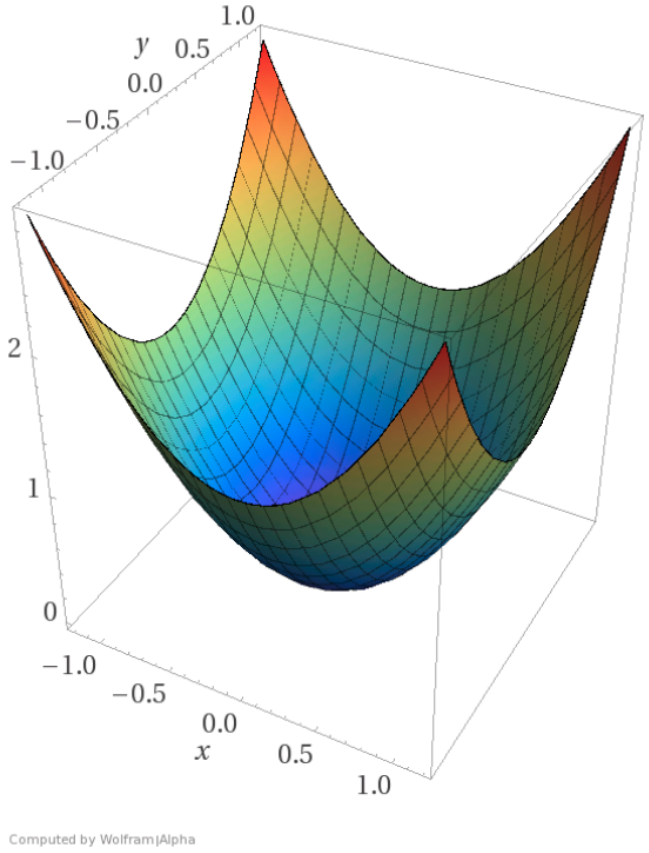
- Gradient-based learning
- Stochastic gradient descent
- **Challenges of optimization**
- Backpropagation

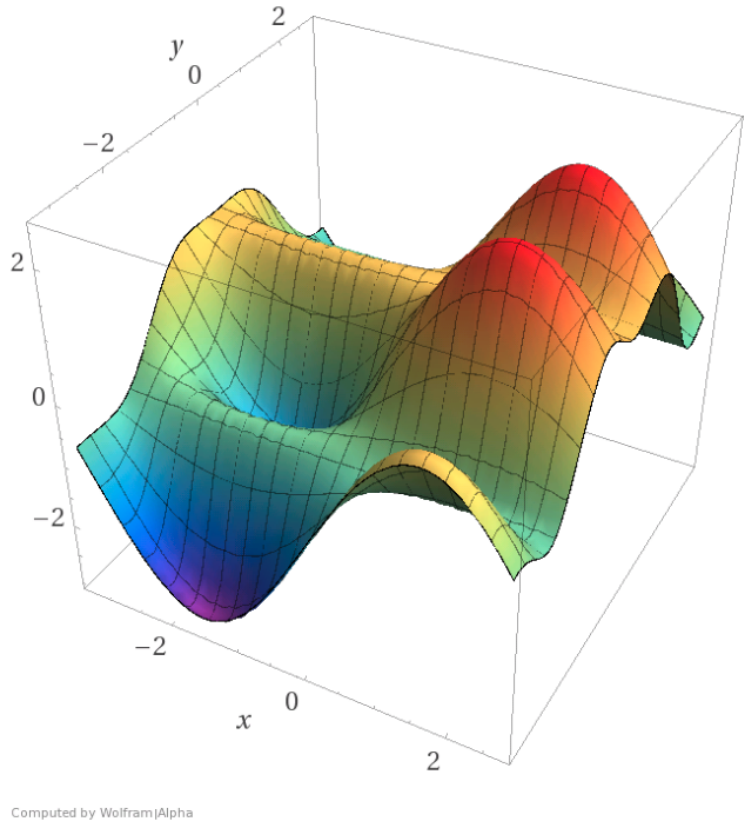
Specialties of Deep Learning

- Neural Network training not different from ML models with gradient descent. The components are needed:
  1. optimization procedure, e.g., gradient descent
  2. cost function, e.g., MLE
  3. model family, e.g., linear with basis functions

- Difference: **nonlinearity causes non-convex loss**
  - Use iterative gradient-based optimizers that merely drives cost to low value
  - No guarantees in comparison to convex optimizations
  - The initialization matters

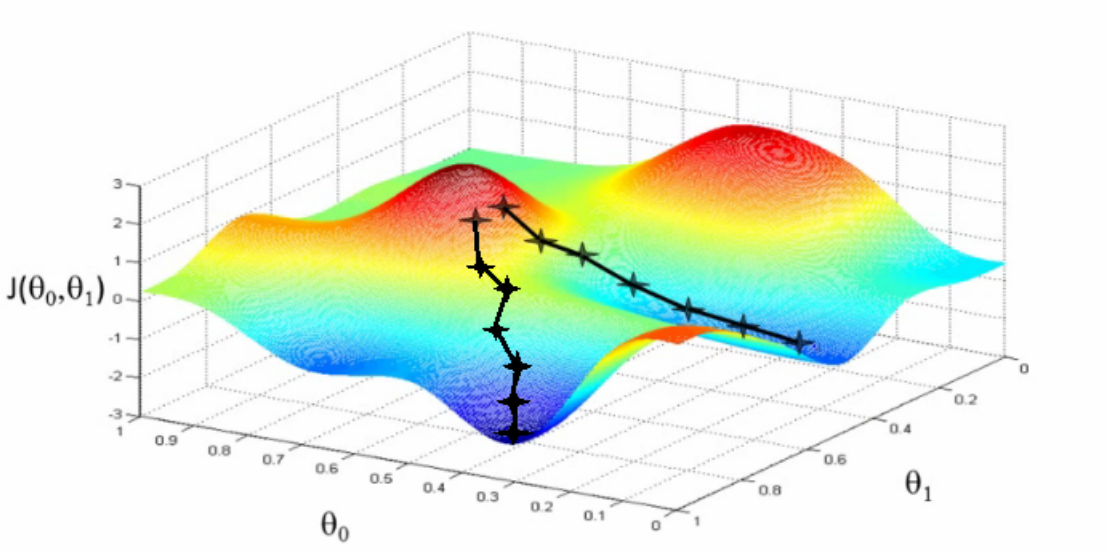
Convex vs. Non-Convex





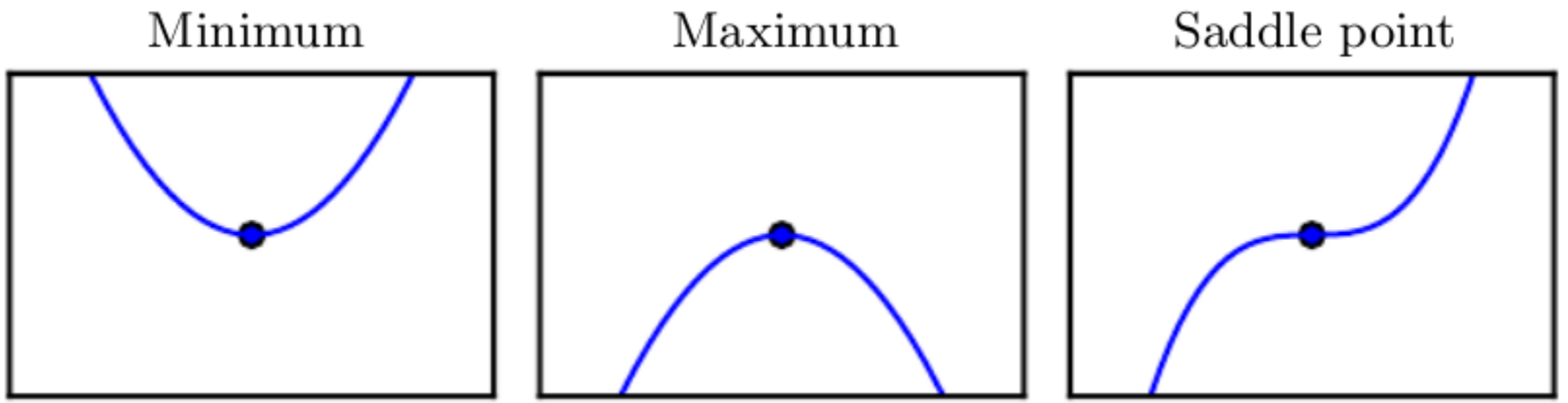
➢ https://www.matroid.com/blog/post/the-hard-thing-about-deep-learning

Problem: We can end-up in local minima



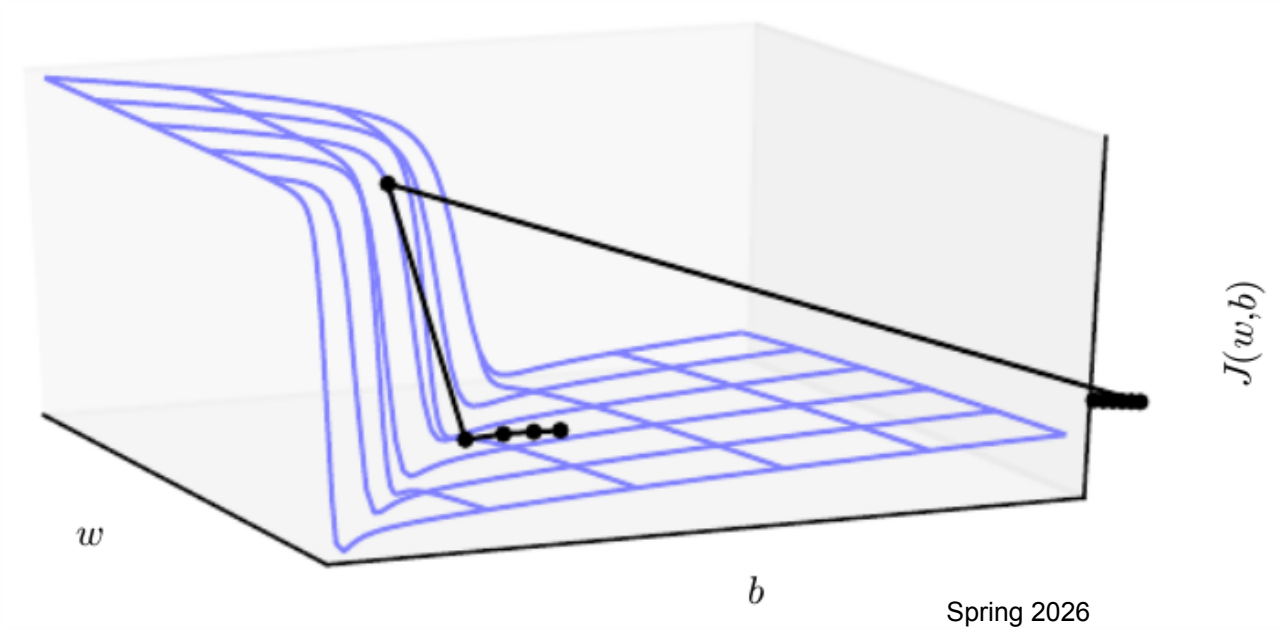
Problem: Stationary points, Local Optima

- When $f'(x) = 0$ derivative provides no information about direction of move
- Points where $f'(x) = 0$ are known as stationary or critical points
  - **Local minimum/maximum**: a point where $f(x)$ lower/ higher than all its neighbors
  - **Saddle Points**: neither maxima nor minima



Cliffs and Exploding Gradients

- Neural networks with many layers have steep regions i.e., cliffs
  - Result from multiplying several large weights
  - E.g., RNNs with many factors at each time step
- Gradient update step can move parameters extremely far, jumping off cliff altogether
- Cliffs dangerous from either direction
- Gradient clipping heuristics can be used



Inexact Gradients

- Optimization algorithms assume we have access to exact gradient or Hessian matrix

- In practice we have a noisy or biased estimate
  - Every deep learning algorithm relies on sampling-based estimates
    - In using minibatch of training examples
  - In other case, objective function is intractable
    - In which case gradient is intractable as well

Poor Correspondence between Local and Global Structure

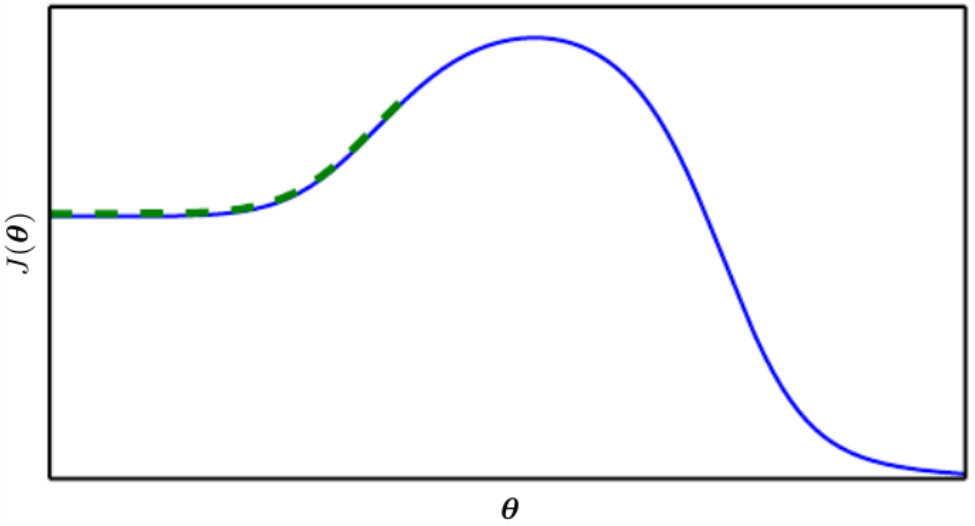
- It can be difficult to make a single step if:
  - $J(\theta)$ is poorly conditioned at the current point θ
  - θ lies on a cliff
  - θ is a saddle point hiding the opportunity to make progress downhill from the gradient

- It is possible to overcome all these problems and still perform poorly:
  - if the direction that makes most improvement locally does not point towards distant regions of much lower cost

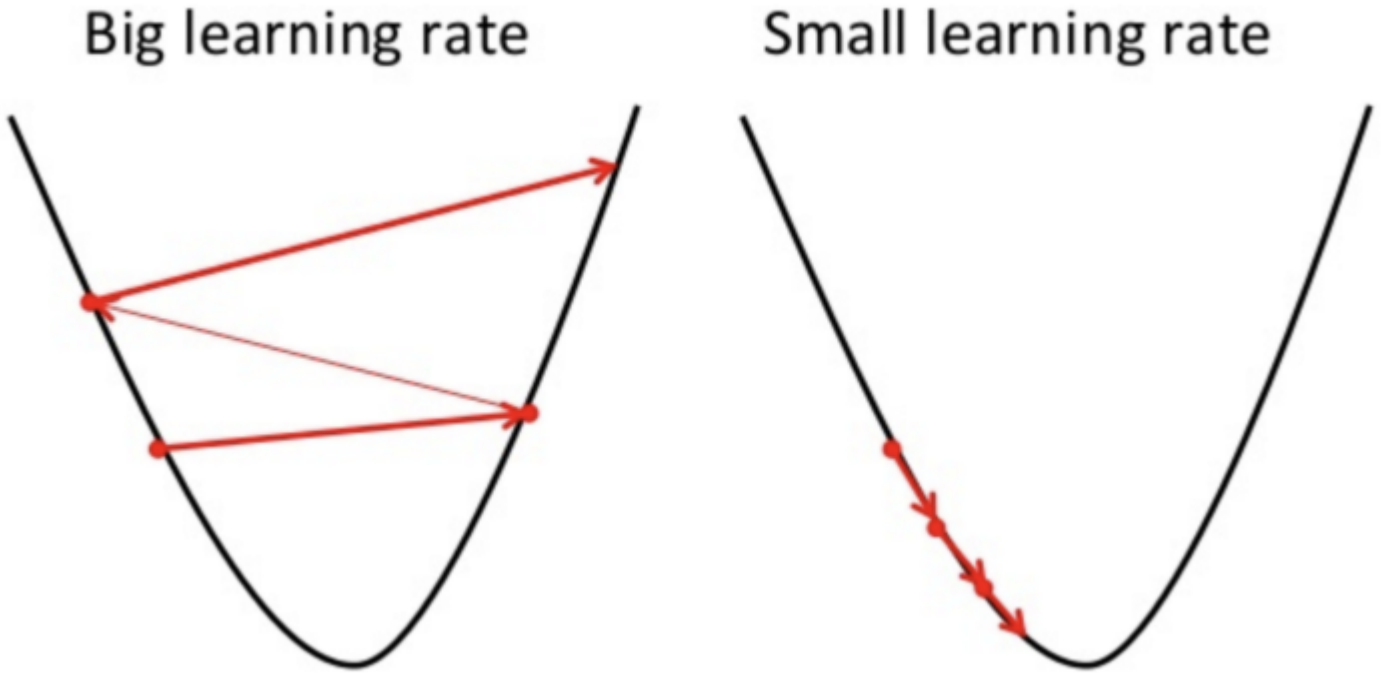
Need for good initial points

- Optimization based on local downhill moves can fail if local surface does not point towards the global solution

- Research directions are aimed at finding good initial points for problems with a difficult global structure
  - For example: Trajectory of circumventing mountains may be long and result in excessive training time



Problem: The Learning Rate



Convergence of Steepest Descent

- Steepest descent converges when every element of the gradient is zero

- Pure math way of life:
  - Find literally the smallest value of $f(x)$
  - Or maybe: find some critical point of $f(x)$ where the value is locally smallest

- **Deep learning way of life:**
  - **Decrease the value of $f(x)$ a lot**
  - **But we have a highly non-convex problem (because of the activation functions) => No guarantees!**

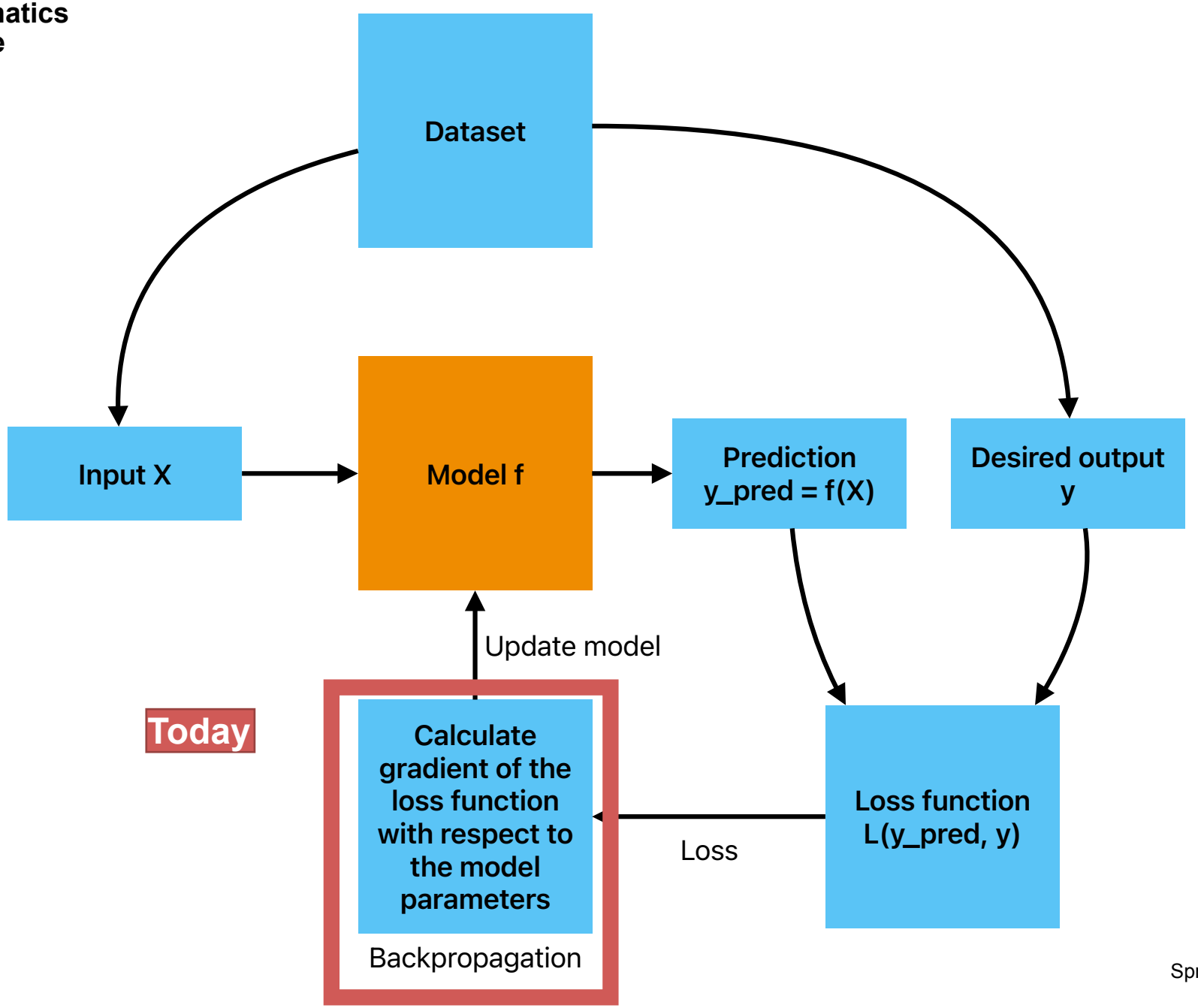
About the Gradient

- Gradient must be large and predictable enough to serve as good guide to the learning algorithm
- Functions that **saturate** (become very flat) undermine this
  - Because the gradient becomes very small
  - Happens when activation functions producing output of hidden/output units saturate

- **Negative log-likelihood** helps avoid saturation problem for many models
  - Many output units involve exp functions that saturate when its argument is very negative
  - Log function in Negative log likelihood cost function undoes exp of some units

Keywords

- Gradient-based learning
- Stochastic gradient descent
- Challenges of optimization
- **Backpropagation**


Dataset
Input X
Model f
Prediction
y_pred = f(X)
Desired output
y
Update model
Today
Calculate
gradient of the
loss function
with respect to
the model
parameters
Backpropagation
Loss
Loss function
L(y_pred, y)

Backprop in the training loop

```
for i in range(n_epochs):
        model.train()
        for x, y in train_loader:
            optimizer.zero_grad()
            y_hat = model(x)
            loss = criterion(y_hat, y)
            loss.backward()
            optimizer.step()
```

Reset grad

Forward pass

Calc. loss

Backprop

Update parameters

Backprop, 1986

Learning representations by back-propagating errors

David E. Rumelhart*, Geoffrey E. Hinton†
& Ronald J. Williams*

* Institute for Cognitive Science, C-015, University of California, San Diego, La Jolla, California 92093, USA
† Department of Computer Science, Carnegie-Mellon University, Pittsburgh, Philadelphia 15213, USA

We describe a new learning procedure, back-propagation, for networks of neurone-like units. The procedure repeatedly adjusts the weights of the connections in the network so as to minimize a measure of the difference between the actual output vector of the net and the desired output vector. As a result of the weight adjustments, internal 'hidden' units which are not part of the input or output come to represent important features of the task domain, and the regularities in the task are captured by the interactions of these units. The ability to create useful new features distinguishes back-propagation from earlier, simpler methods such as the perceptron-convergence procedure[1].

sdu.dk
#sdudk

Backprop, 1986

Learning representations by back-propagating errors

David E. Rumelhart*, Geoffrey E. Hinton†
& Ronald J. Williams*

The backward pass starts by computing $\partial E/\partial y$ for each of the output units. Differentiating equation (3) for a particular case, c, and suppressing the index c gives

$$\partial E/\partial y_j = y_j - d_j \qquad (4)$$

We can then apply the chain rule to compute $\partial E/\partial x_j$

$$\partial E/\partial x_j = \partial E/\partial y_j \cdot \mathrm{d}y_j/\mathrm{d}x_j$$

Chain Rule of Calculus

➢ If g is differentiable at x and f is differentiable at $g(x)$, then the composite function $F = f \circ g$ defined by $F(x) = f\big(g(x)\big)$ is differentiable at x and F' is given by

$$F' = f'\big(g(x)\big) \cdot g'(x)$$

➢ In Leibnitz notation, if $y = f(u)$ and $u = g(x)$, then

$$\frac{dy}{dx} = \frac{dy}{du}\frac{du}{dx}$$

A chain rule example

- $F(x) = \left(3x - 5\right)^2$
- Chain rule: $F(x) = f'\left(g(x)\right) \cdot g'(x)$
- Set $g(x) = 3x - 5$, and then the outer function is $f(g(x)) = g(x)^2$
- Calculate outer derivative $f'(g(x)) = 2 \cdot g(x)$, and multiply by inner derivative $g'(x) = 3$
- Substitute g(x) to obtain $F'(X) = 2 \cdot \left(3x - 5\right) \cdot 3 = \left(6x - 10\right) \cdot 3 = 18x - 30$

Decomposing a neural net into functions

- Consider the function $f(\mathbf{x}) = \mathbf{W}^{(2)}\,\mathrm{ReLU}\left(\mathbf{W}^{(1)}\mathbf{x}\right)$ representing a 2-layer neural net
- Can be decomposed into:
  - $\mathbf{a} = \mathbf{W}^{(1)}\mathbf{x}$
  - $\mathbf{z} = \mathrm{ReLU}\,(\mathbf{a})$
  - $\mathbf{y} = \mathbf{W}^{(2)}\mathbf{h}$
- And then we have the loss $E\left(f(\mathbf{x}), y\right)$

Or in function notation:

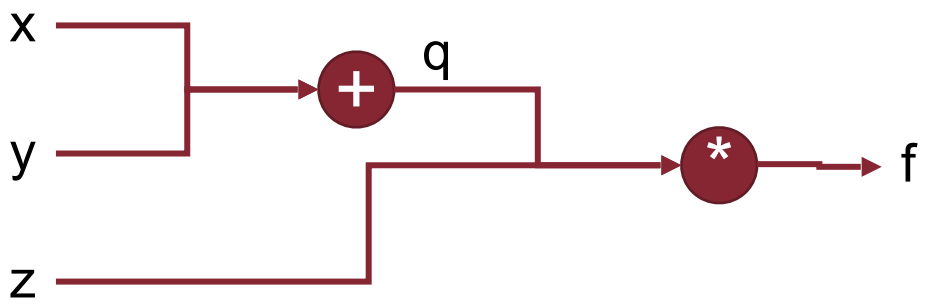
- $a(\mathbf{x}) = \mathbf{W}^{(1)}\mathbf{x}$
- $z(\mathbf{x}) = \mathrm{ReLU}(\mathbf{x})$
- $y(\mathbf{x}) = W^{(2)}x$
- Then, $f = y \circ z \circ a$
- Or: $f(x) = y\left(z\left(a(x)\right)\right)$

Differentiating the individual functions

- One hidden unit's weights: $a(\mathbf{z}) = \mathbf{w}^\top \mathbf{z} = w_1 z_1 + w_2 z_2 + \ldots + w_n z_n$
  - $\frac{da}{dw_i} = z_i \quad \frac{da}{dz_i} = w_i$
- Rectified linear unit: $\mathrm{ReLU}(x) = \max\{0, x\}$
  - $\frac{d\,\mathrm{ReLU}}{dx} := 1$ if $x > 0$ and 0 otherwise (*technically not differentiable at 0 but we just define*)
- Loss function: $\mathrm{E}(y, t) = \frac{1}{2}\left(y - t\right)^2$ (*assuming 1d output)*
  - $\frac{d\,\mathrm{E}}{dy} = y - t$

Backpropagation: A Simple Example

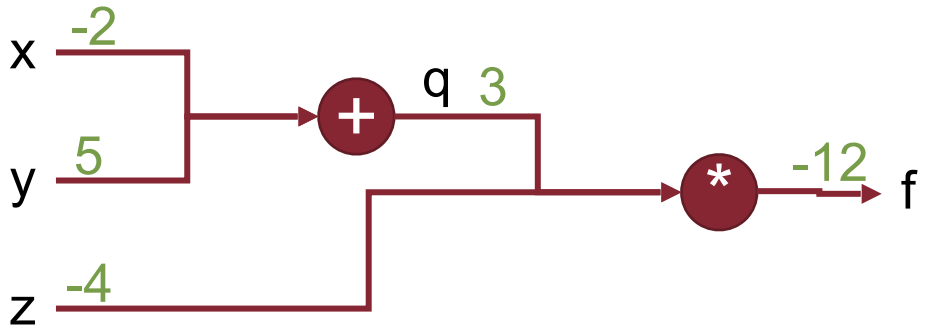
$$f(x, y, z) = (x + y)z$$



Backpropagation: A Simple Example

$$f(x, y, z) = (x + y)z$$

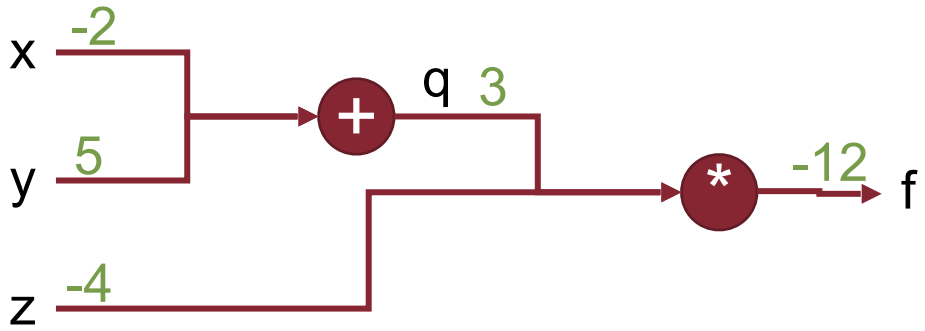
- With
  - $x = -2$
  - $y = 5$
  - $z = -4$



Backpropagation: A Simple Example

$$f(x, y, z) = (x + y)z$$

- With
  - $x = -2$
  - $y = 5$
  - $z = -4$



$$q = x + y; \quad \frac{\partial q}{\partial x} = 1; \quad \frac{\partial q}{\partial y} = 1;$$

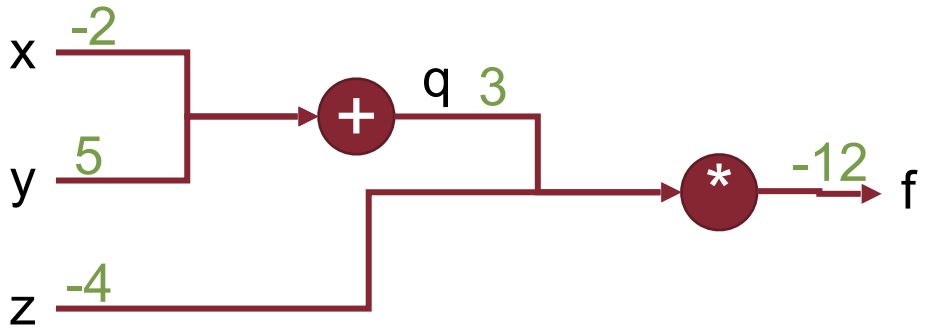
$$f = qz; \quad \frac{\partial f}{\partial q} = z; \quad \frac{\partial f}{\partial z} = q;$$

Backpropagation: A Simple Example

$$f(x, y, z) = (x + y)z$$

➢ With

- ➢ $x = -2$
- ➢ $y = 5$
- ➢ $z = -4$



$$q = x + y; \quad \frac{\partial q}{\partial x} = 1; \quad \frac{\partial q}{\partial y} = 1;$$

$$f = qz; \quad \frac{\partial f}{\partial q} = z; \quad \frac{\partial f}{\partial z} = q;$$

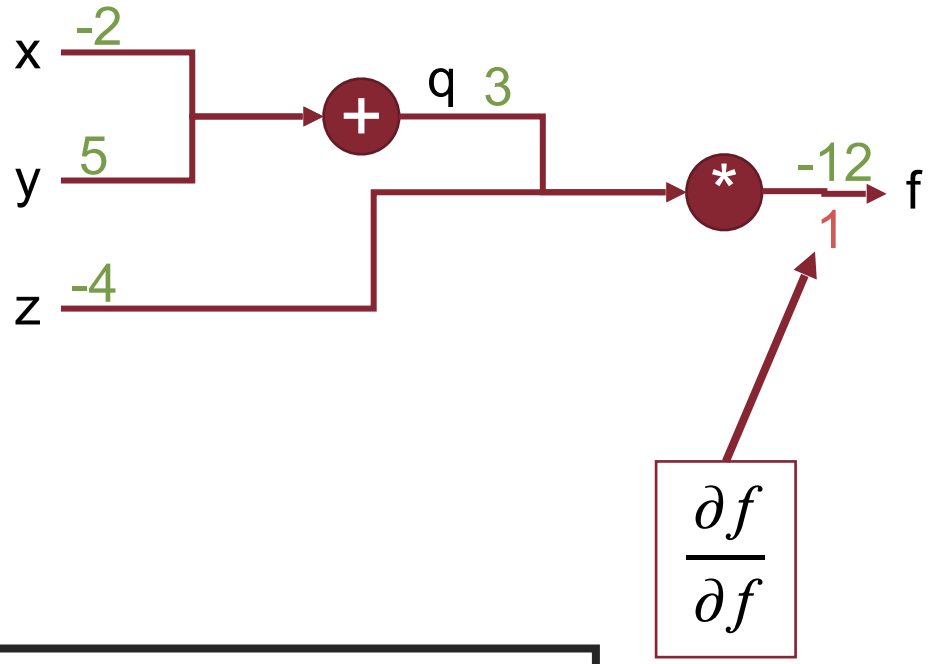
Looking for:

$$\frac{\partial f}{\partial x} \quad \frac{\partial f}{\partial y} \quad \frac{\partial f}{\partial z}$$

Backpropagation: A Simple Example

$$f(x, y, z) = (x + y)z$$

➢ With
- ➢ $x = -2$
- ➢ $y = 5$
- ➢ $z = -4$



$$q = x + y; \quad \frac{\partial q}{\partial x} = 1; \quad \frac{\partial q}{\partial y} = 1;$$

$$f = qz; \quad \frac{\partial f}{\partial q} = z; \quad \frac{\partial f}{\partial z} = q;$$

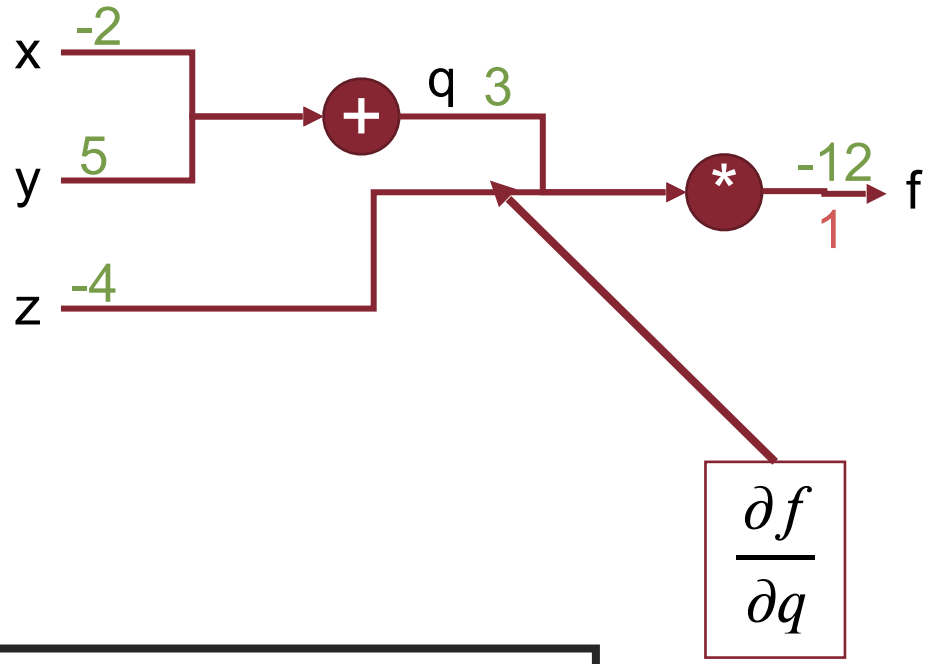
Looking for:

$$\frac{\partial f}{\partial x} \quad \frac{\partial f}{\partial y} \quad \frac{\partial f}{\partial z}$$

Backpropagation: A Simple Example

$$f(x, y, z) = (x + y)z$$

➢ With
- ➢ $x = -2$
- ➢ $y = 5$
- ➢ $z = -4$



$$q = x + y; \quad \frac{\partial q}{\partial x} = 1; \quad \frac{\partial q}{\partial y} = 1;$$

$$f = qz; \quad \frac{\partial f}{\partial q} = z; \quad \frac{\partial f}{\partial z} = q;$$

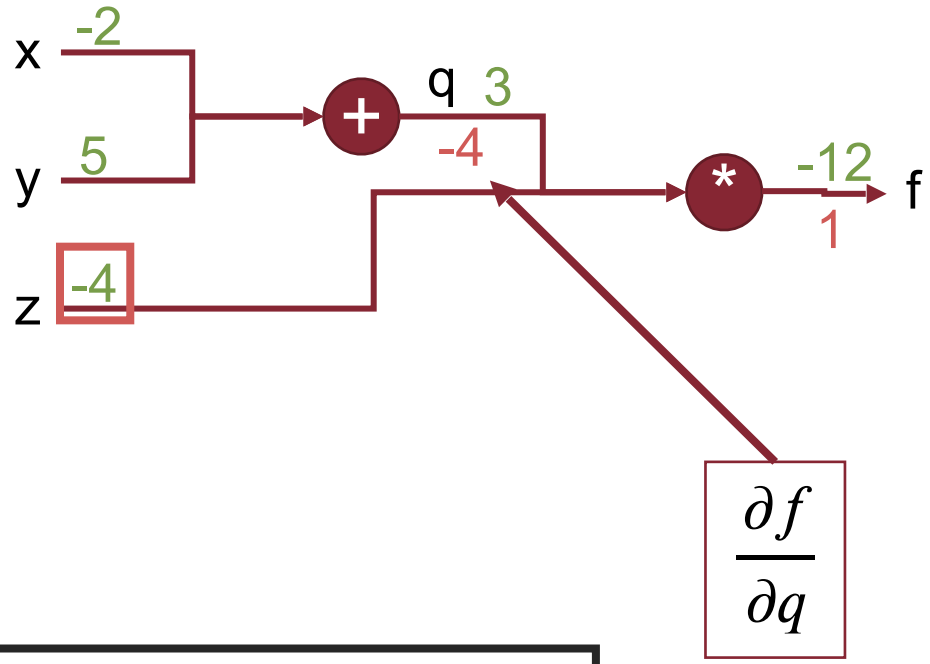
Looking for:

$$\frac{\partial f}{\partial x} \quad \frac{\partial f}{\partial y} \quad \frac{\partial f}{\partial z}$$

Backpropagation: A Simple Example

$$f(x, y, z) = (x + y)z$$

➢ With
- ➢ $x = -2$
- ➢ $y = 5$
- ➢ $z = -4$



$$q = x + y; \quad \frac{\partial q}{\partial x} = 1; \quad \frac{\partial q}{\partial y} = 1;$$

$$f = qz; \quad \boxed{\frac{\partial f}{\partial q} = z;} \quad \frac{\partial f}{\partial z} = q;$$

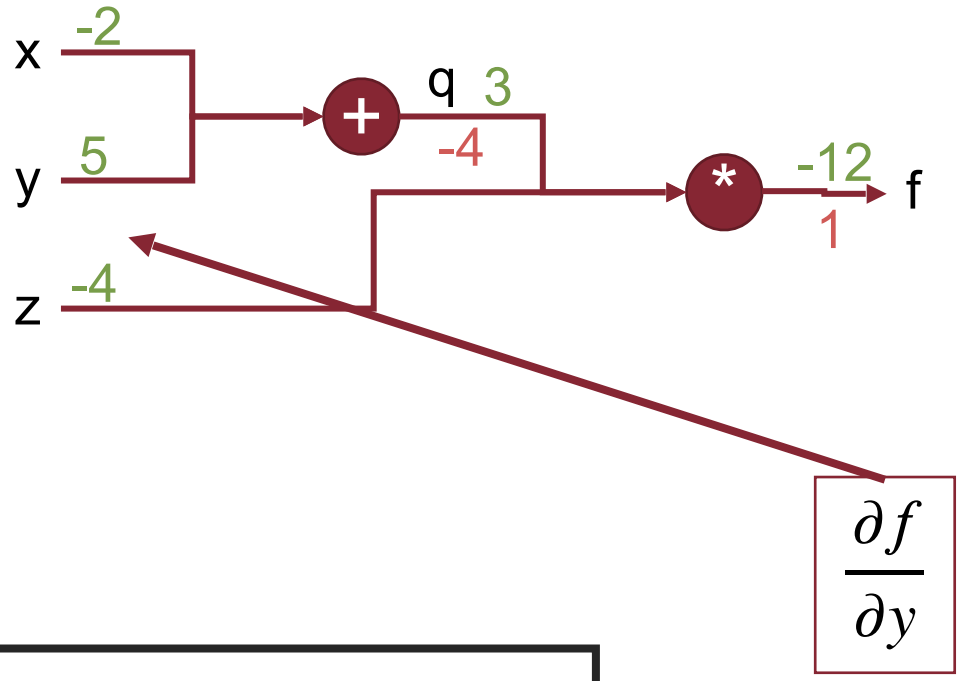
Looking for:

$$\frac{\partial f}{\partial x} \qquad \frac{\partial f}{\partial y} \qquad \frac{\partial f}{\partial z}$$

Backpropagation: A Simple Example

$$f(x, y, z) = (x + y)z$$

- With
  - $x = -2$
  - $y = 5$
  - $z = -4$



$$q = x + y; \quad \frac{\partial q}{\partial x} = 1; \quad \frac{\partial q}{\partial y} = 1;$$

$$f = qz; \quad \frac{\partial f}{\partial q} = z; \quad \frac{\partial f}{\partial z} = q;$$

Looking for:

$$\frac{\partial f}{\partial x} \qquad \frac{\partial f}{\partial y} \qquad \frac{\partial f}{\partial z}$$

Backpropagation: A Simple Example

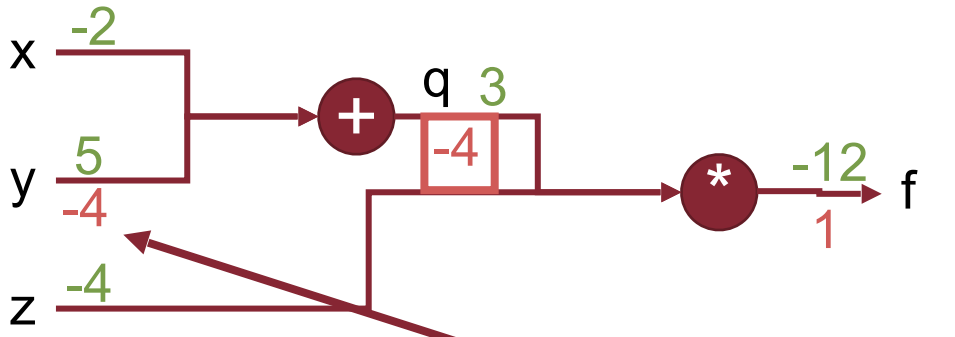
$$f(x, y, z) = (x + y)z$$

➢ With

- ➢ $x = -2$
- ➢ $y = 5$
- ➢ $z = -4$

$$q = x + y; \quad \frac{\partial q}{\partial x} = 1; \boxed{\frac{\partial q}{\partial y} = 1;}$$

$$f = qz; \quad \frac{\partial f}{\partial q} = z; \quad \frac{\partial f}{\partial z} = q;$$



Chain Rule:

$$\frac{\partial f}{\partial y} = \frac{\partial f}{\partial q}\frac{\partial q}{\partial x}$$

$$\frac{\partial f}{\partial y}$$

Looking for:

$$\frac{\partial f}{\partial x} \quad \frac{\partial f}{\partial y} \quad \frac{\partial f}{\partial z}$$

Backpropagation: A Simple Example

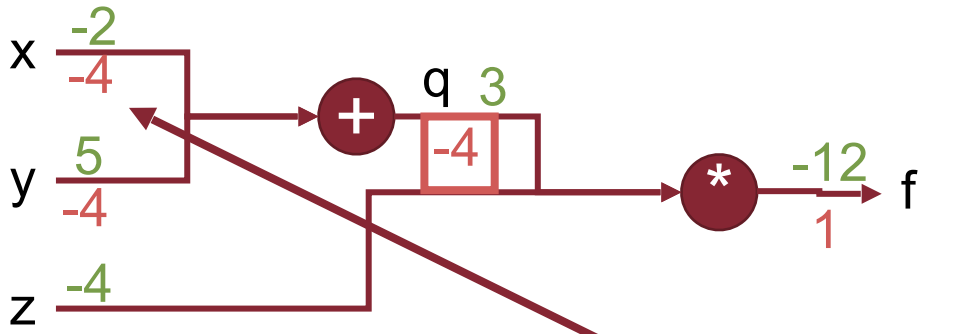
$$f(x, y, z) = (x + y)z$$

➢ With

- ➢ $x = -2$
- ➢ $y = 5$
- ➢ $z = -4$

$$q = x + y; \quad \frac{\partial q}{\partial x} = 1; \quad \frac{\partial q}{\partial y} = 1;$$

$$f = qz; \quad \frac{\partial f}{\partial q} = z; \quad \frac{\partial f}{\partial z} = q;$$



Chain Rule:

$$\frac{\partial f}{\partial y} = \frac{\partial f}{\partial q}\frac{\partial q}{\partial x}$$

$$\frac{\partial f}{\partial x}$$

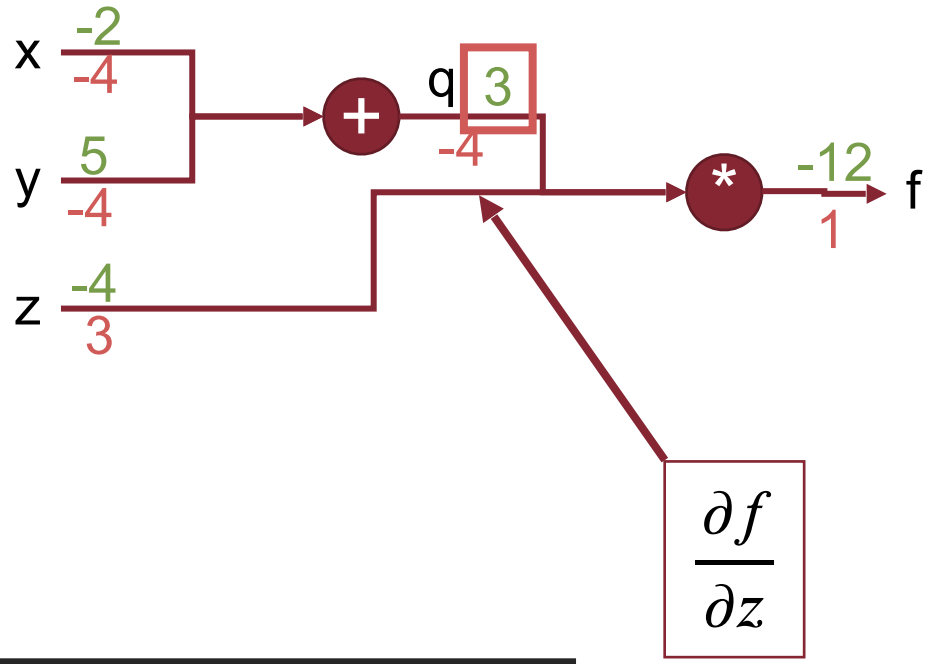
Looking for:

$$\frac{\partial f}{\partial x} \quad \frac{\partial f}{\partial y} \quad \frac{\partial f}{\partial z}$$

Backpropagation: A Simple Example

$$f(x, y, z) = (x + y)z$$

- With
  - $x = -2$
  - $y = 5$
  - $z = -4$



$$q = x + y; \quad \frac{\partial q}{\partial x} = 1; \quad \frac{\partial q}{\partial y} = 1;$$

$$f = qz; \quad \frac{\partial f}{\partial q} = z; \quad \boxed{\frac{\partial f}{\partial z} = q;}$$

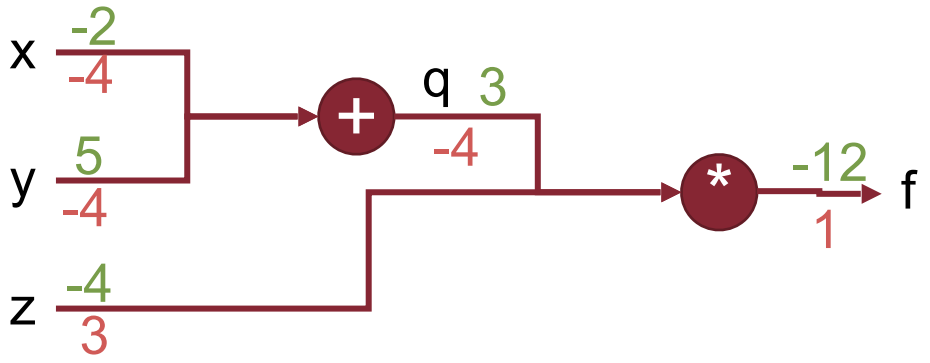
Looking for:

$$\frac{\partial f}{\partial x} \qquad \frac{\partial f}{\partial y} \qquad \frac{\partial f}{\partial z}$$

Backpropagation: A Simple Example

$$f(x, y, z) = (x + y)z$$

➢ With
- ➢ $x = -2$
- ➢ $y = 5$
- ➢ $z = -4$



$$q = x + y; \quad \frac{\partial q}{\partial x} = 1; \quad \frac{\partial q}{\partial y} = 1;$$

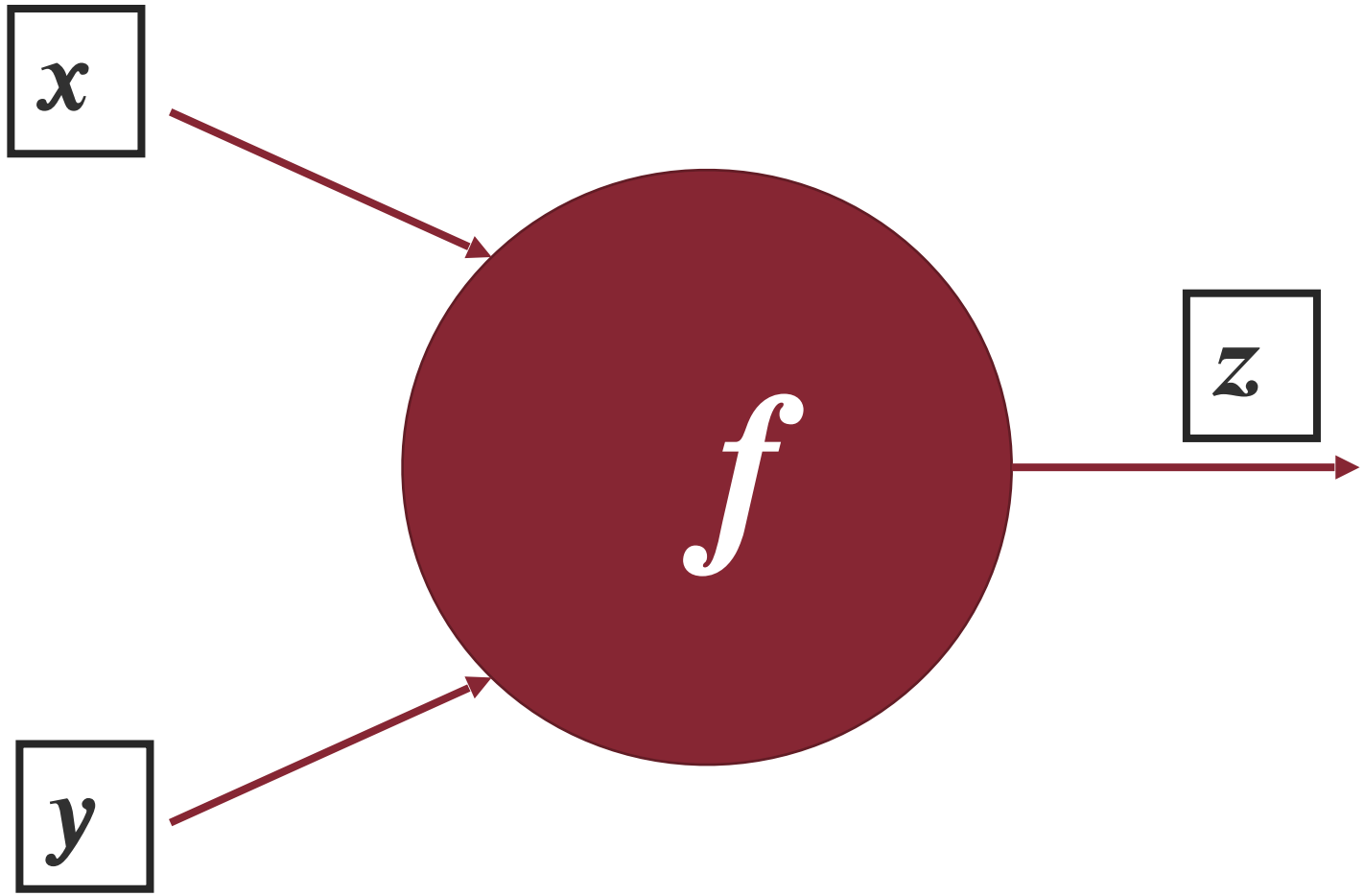
$$f = qz; \quad \frac{\partial f}{\partial q} = z; \quad \frac{\partial f}{\partial z} = q;$$

Looking for:

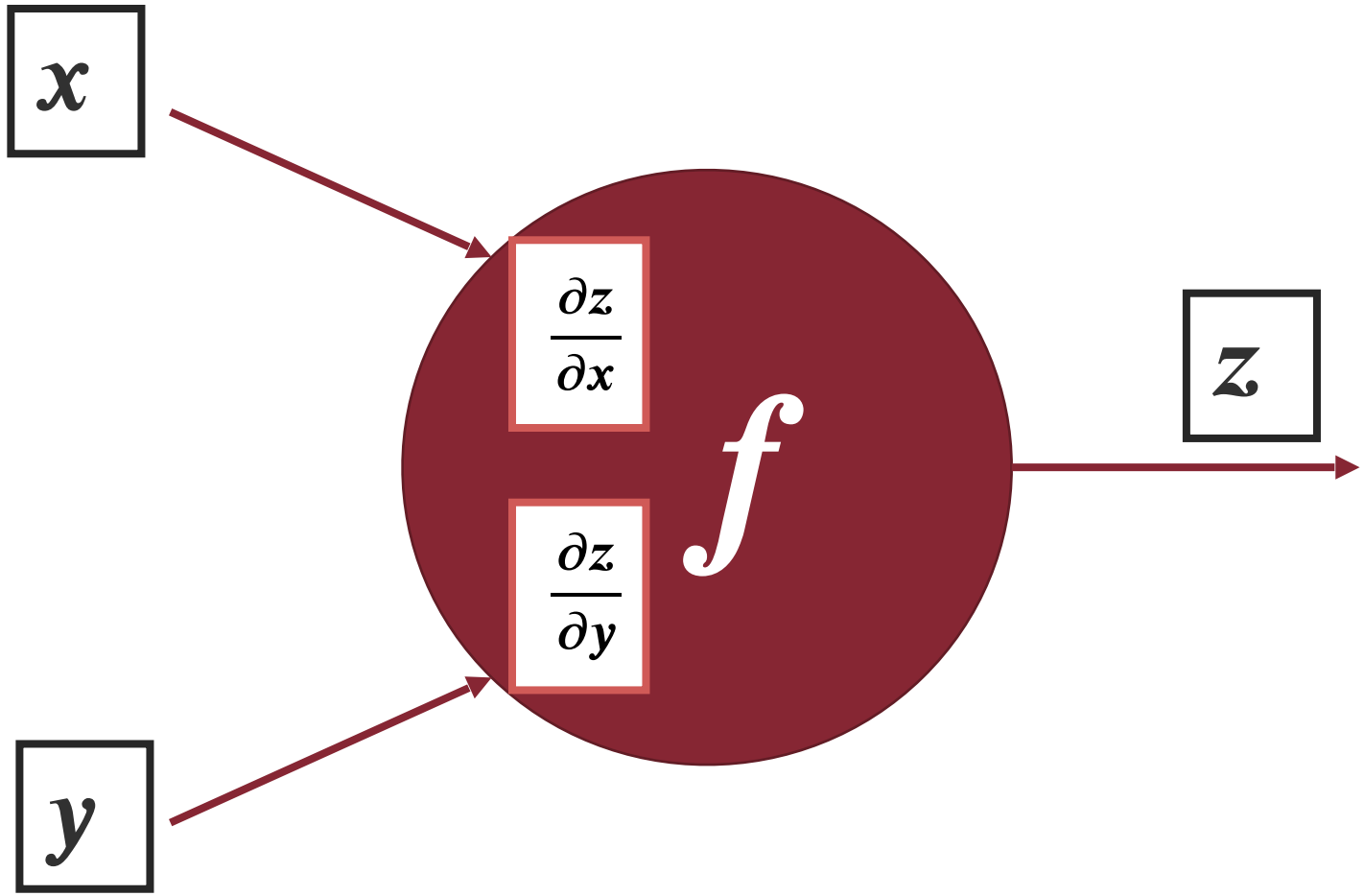
$$\frac{\partial f}{\partial x} \quad \frac{\partial f}{\partial y} \quad \frac{\partial f}{\partial z}$$

$$\nabla_{x,y,z} f = \begin{pmatrix} -4 \\ -4 \\ 3 \end{pmatrix}$$

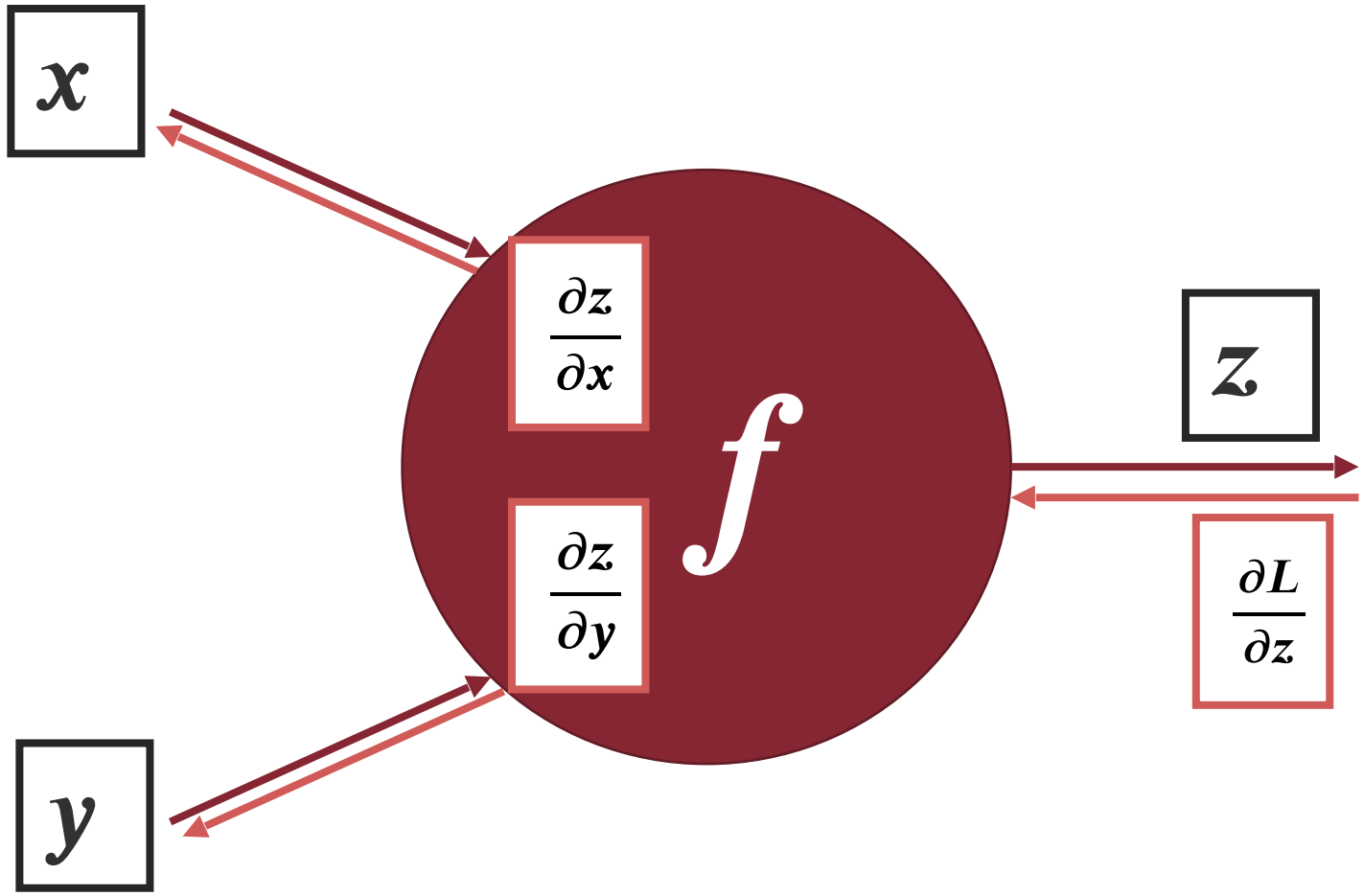
General Principle



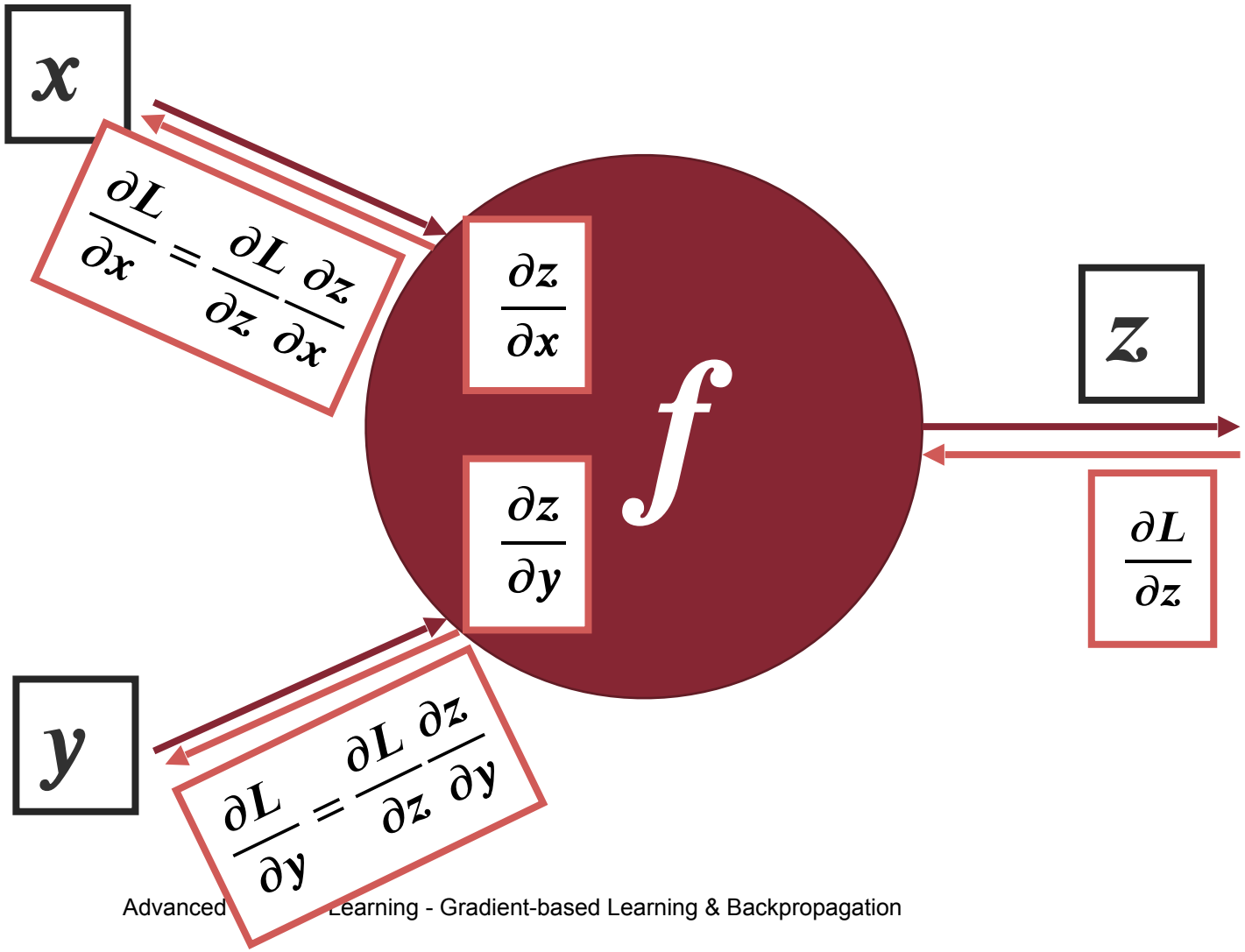
General Principle



General Principle



General Principle



Department of Mathematics
and Computer Science

Single-layer neural networks

- Linear model with multiple inputs x_i and multiple outputs y_i

$$y_k = \sum_i w_{ki} \cdot x_i$$

- Sum-of-squares error function for one data point n

$$E_n = \frac{1}{2} \sum_k \left(y_{nk} - t_{nk} \right)^2$$

- Gradient

$$\frac{dE_n}{dw_{ji}} = \left(y_{nj} - t_{nj} \right) \cdot x_{ni}$$

Error signal for connection between input i and output j

There is already a chain rule application here:

$$\frac{dy_k}{dw_{ki}} = x_i \qquad \frac{dE_n}{y_{nk}} = y_{nk} - t_{nk}$$

$$\frac{dE_n}{dw_{ij}} = \frac{dE_n}{dy_{nj}} \cdot \frac{dy_{nj}}{dw_{ji}}$$

Department of Mathematics
and Computer Science

General feedforward networks

Set of units, each computing the weighted sum of its inputs:

$$a_j = \sum_i w_{ji} z_i$$

Then we feed this pre-activation into a nonlinear activation function

$$z_j = h(a_j)$$

We assume that the forward pass (=feeding the data through the network) has already happened, and we have all the activations and the loss

Applying chain rule for partial derivatives gives

$$\frac{\partial E_n}{\partial w_{ij}} = \frac{\partial E_n}{\partial a_j} \frac{\partial a_j}{\partial w_{ji}}$$

Useful notation for the '**errors**': $\delta_j = \frac{\partial E_n}{\partial a_j}$

Now we know that:

$$\frac{\partial a_j}{\partial w_{ji}} = z_i$$

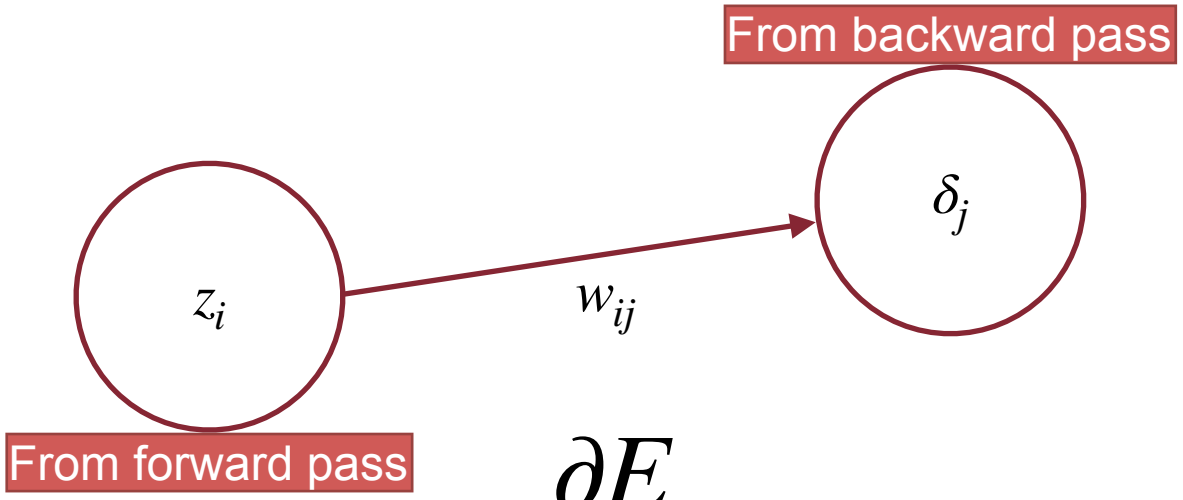
And by substitution we get

$$\frac{\partial E_n}{\partial w_{ji}} = \delta_j z_i$$

Department of Mathematics
and Computer Science

From errors to derivatives for weights

- When we have the error δ_j for a hidden/ output unit, we can derive the partial derivatives for its inbound weights by multiplying the error at the end of the connection with the activation at the start.



$$\frac{\partial E_n}{\partial w_{ji}} = \delta_j z_i$$

Department of Mathematics
and Computer Science

General feedforward networks (cont.)

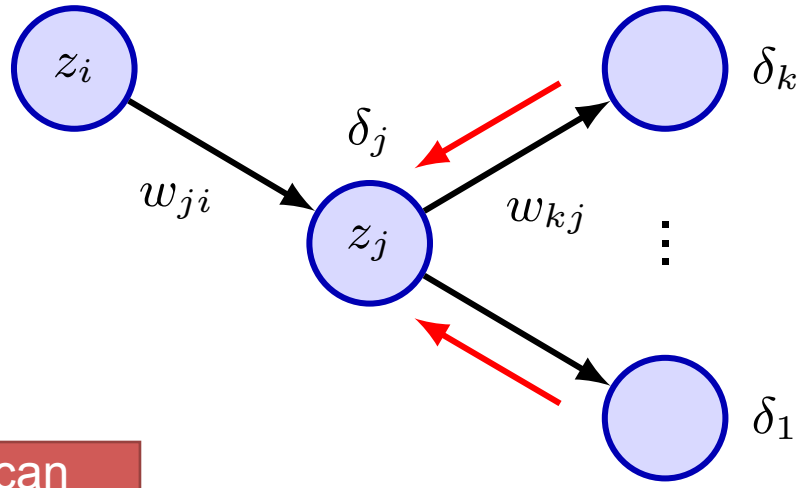
- For the final layer, we get the error from the loss function – just as in the single layer case:

$$\delta_k = y_k - t_k$$

- To get the deltas for the hidden units, apply chain rule

$$\delta_j = \frac{\partial E_n}{\partial a_j} = \sum_k \frac{\partial E_n}{\partial a_k} \frac{\partial a_k}{\partial a_j}$$

The sum runs over all units k to which j sends something



- Now substitute definition of $\delta_k = \frac{\partial E_n}{\partial a_k}$ in there

$$\delta_j = h'(a_j) \sum_k w_{kj} \delta_k$$

This tells us how we can propagate the errors back, starting from the output

Derivative of activation function

Department of Mathematics
and Computer Science

Backprop in a feedforward network

- Consider the network

$$y = W^{(2)} \tanh\left(W^{(1)} x\right)$$

- And a mean-squared error loss function

$$E_n = \frac{1}{2} \sum_{k=1}^{K} \left(y_k - t_k\right)^2$$

- Forward pass:

$$a_j = \sum_{i=0}^{D} w_{ji} x_i$$

$$z_j = \tanh(a_j)$$

$$y_k = \sum_{j=0}^{M} w_{kj}^{(2)} z_j$$

Derivative of tanh

$$\frac{\partial z_j}{\partial a_j} = \left(1 - z_j\right)^2$$

- Get output error

$$\delta_k = y_k - t_k$$

- Backpropagate errors

$$\delta_j = \left(1 - z_j\right)^2 \sum_{k=1}^{K} w_{kj}^{(2)} \delta_k$$

- Calculate the derivatives

$$\frac{\partial E_n}{\partial w_{ji}^{(1)}} = \delta_j x_i$$

$$\frac{\partial E_n}{\partial w_{kj}^{(2)}} = \delta_k z_j$$

Backprop in a feedforward network with ReLU

- Consider the network

$$y = W^{(2)}\,\mathrm{ReLU}\left(W^{(1)}x\right)$$

- And a mean-squared error loss function

$$E_n = \frac{1}{2}\sum_{k=1}^{K}\left(y_k - t_k\right)^2$$

- Forward pass:

$$a_j = \sum_{i=0}^{D} w_{ji}x_i$$

$$z_j = \mathrm{ReLU}(a_j)$$

$$y_k = \sum_{j=0}^{M} w_{kj}^{(2)}z_j$$

Derivative of ReLU

$$\frac{\partial z_j}{\partial a_j} = \begin{cases} 1 & \text{if } a_j > 0 \\ 0 & \text{otherwise} \end{cases}$$

- Get output error

$$\delta_k = y_k - t_k$$

- Backpropagate errors

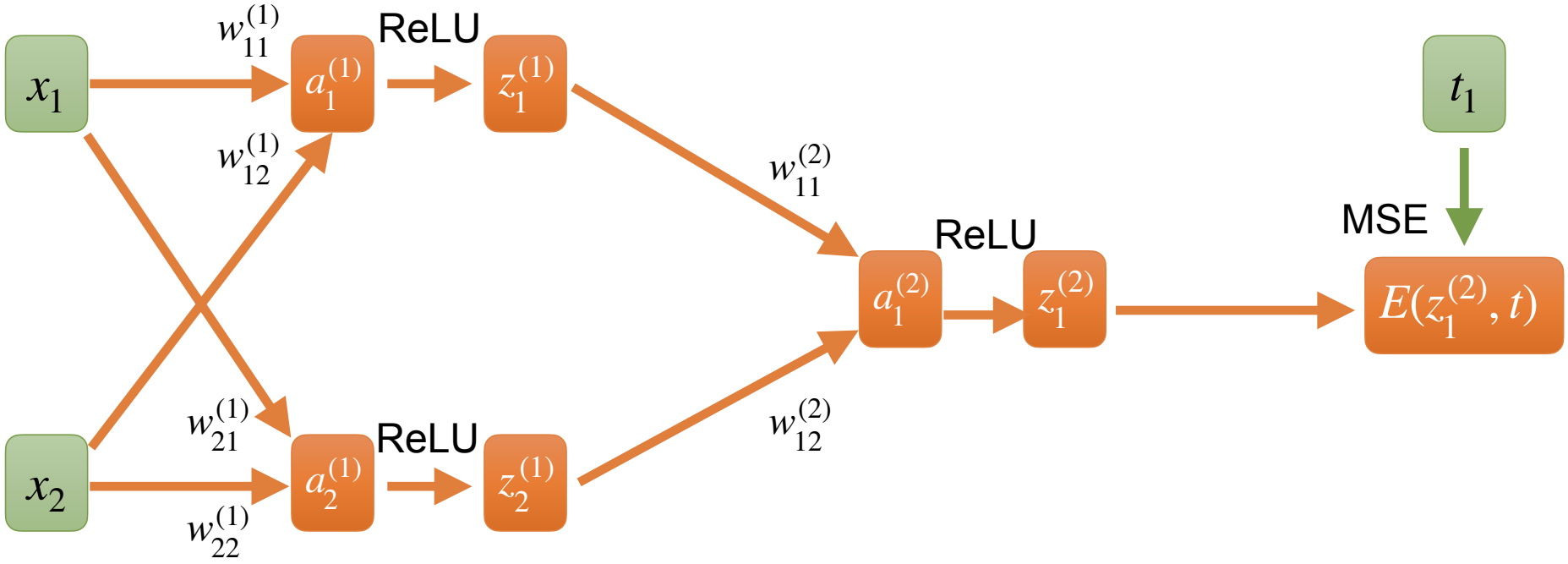
$$\delta_j = \begin{cases} \sum_{k=1}^{K} w_{kj}^{(2)}\delta_k & \text{if } a_j > 0 \\ 0 & \text{otherwise} \end{cases}$$

- Calculate the derivatives

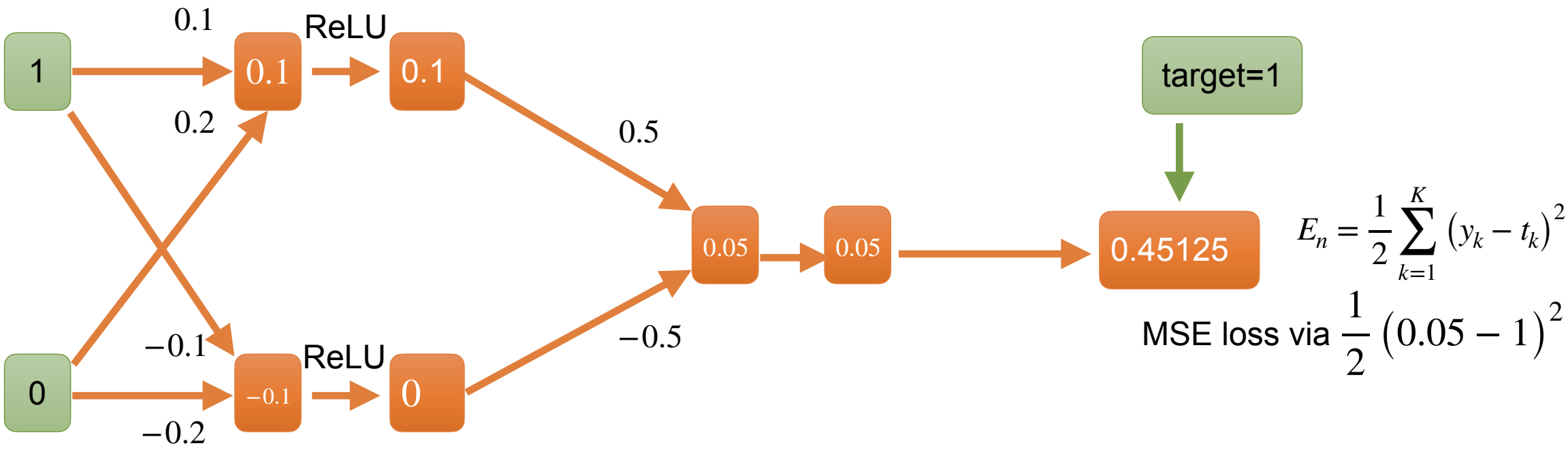
$$\frac{\partial E_n}{\partial w_{ji}^{(1)}} = \delta_j x_i$$

$$\frac{\partial E_n}{\partial w_{kj}^{(2)}} = \delta_k z_j$$

Let's try it out: 2 inputs, 2 hidden units, 1 output

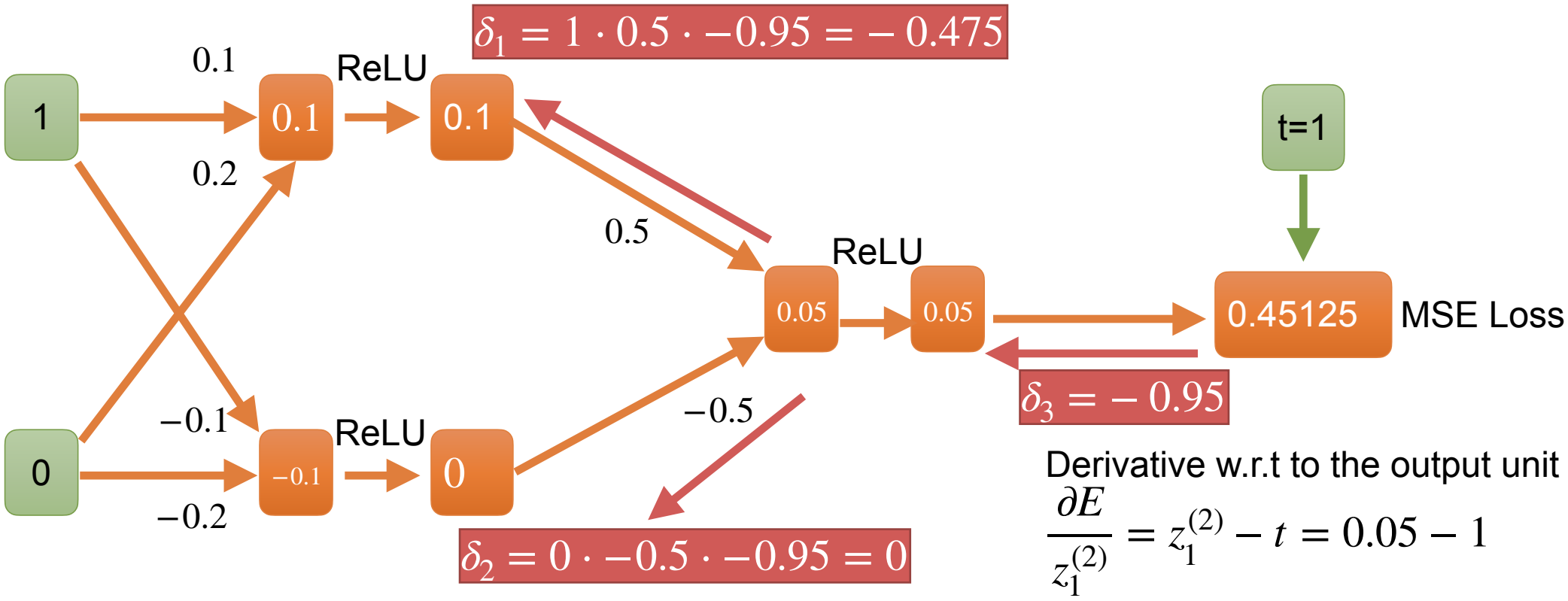


Example forward pass



Example backward pass

$$\delta_j = h'(a_j) \sum_k w_{kj} \delta_k$$

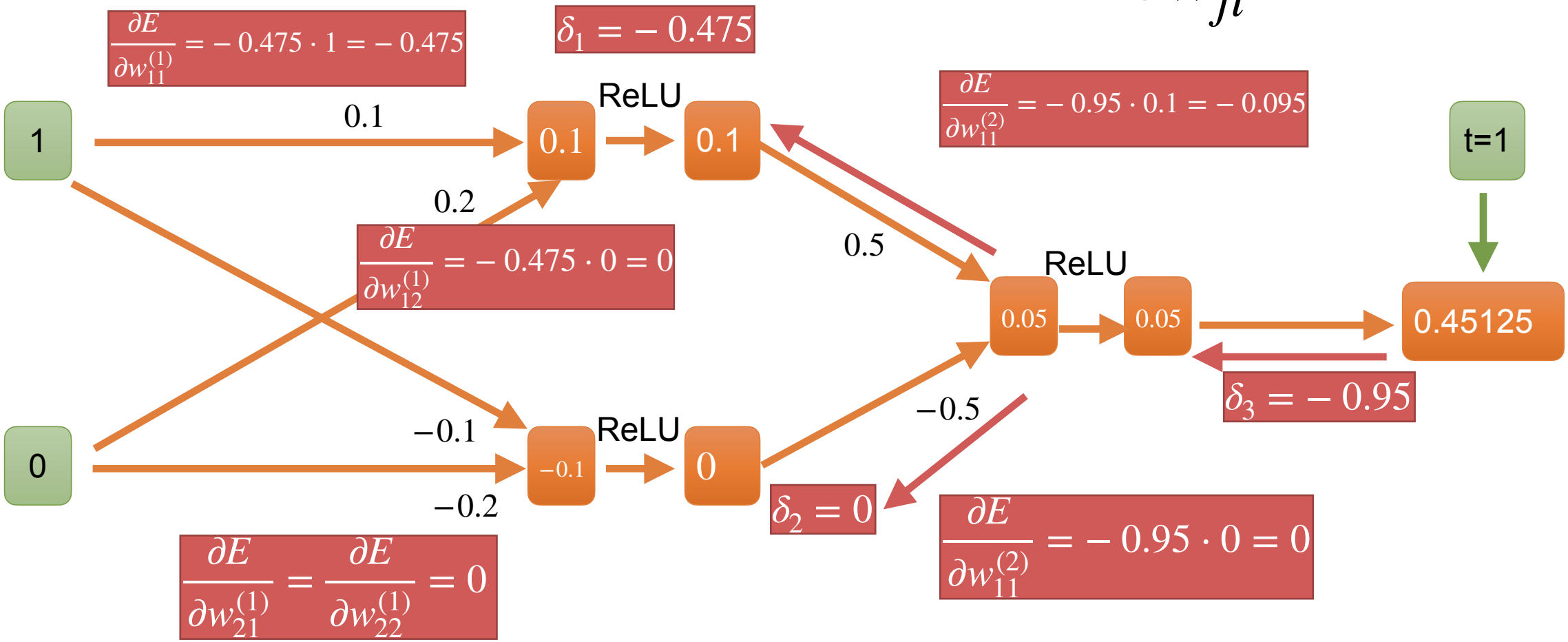


Derivative w.r.t to the output unit

$$\frac{\partial E}{z_1^{(2)}} = z_1^{(2)} - t = 0.05 - 1$$

Example gradient

$$\frac{\partial E_n}{\partial w_{ji}} = \delta_j z_i$$



Backprop Algorithm

Algorithm 8.1: Backpropagation

Input: Input vector $\mathbf{x}_n$
Network parameters $\mathbf{w}$
Error function $E_n(\mathbf{w})$ for input x_n
Activation function $h(a)$

Output: Error function derivatives $\{\partial E_n/\partial w_{ji}\}$

// Forward propagation

for $j \in$ all hidden and output units **do**

  $a_j \leftarrow \sum_i w_{ji} z_i$ // $\{z_i\}$ includes inputs $\{x_i\}$

  $z_j \leftarrow h(a_j)$ // activation function

end for

// Error evaluation

for $k \in$ all output units **do**

  $\delta_k \leftarrow \dfrac{\partial E_n}{\partial a_k}$ // compute errors

end for

// Backward propagation, in reverse order

for $j \in$ all hidden units **do**

  $\delta_j \leftarrow h'(a_j) \sum_k w_{kj} \delta_k$ // recursive backward evaluation

  $\dfrac{\partial E_n}{\partial w_{ji}} \leftarrow \delta_j z_i$ // evaluate derivatives

end for

return $\left\{ \dfrac{\partial E_n}{\partial w_{ji}} \right\}$

Notes on backprop

- How to deal with multiple examples in a batch?
  - Sum the error up (textbook) or average the error over examples (default in practice)
- Backprop is not a learning algorithm by itself.
  - The purpose of backprop is to calculate gradients.
  - Needs to be coupled with a learning algorithm, e.g. stochastic gradient descent
- Backprop is not specific to deep learning, it can be used elsewhere.
- Generally, there are alternatives to backprop to calc. gradients (e.g. forward accumulation)
  - But backprop is most suitable for deep learning (because high number of parameters)

Keywords

- Gradient-based learning
- Stochastic gradient descent
- Challenges of optimization
- Backpropagation

How’s my teaching?

https://www.admonymous.co/lukasgalke

Department of Mathematics
and Computer Science

sdu.dk

#sdudk

Advanced Machine Learning

Loss Functions

Lukas Galke Poech

Spring 2026

SDU

Keywords

- Generative Models
- Discriminative Models & Logistic Regression
- Information Theory
- Well Behaved Gradients

One Example

Decanter

Police uncover Italian wine fraud

Maggie Rosen
August 23, 2007

3 shares

Police have broken up an international counterfeit wine racket involving top Italian wines.

German and Italian police forces have uncovered a cross-border scam involving table wine from Puglia and Piedmont sold as Barolo, Brunello di Montalcino, Amarone and Chianti.

Unlabelled wine was brought into Germany, where it was given fake DOC and DOCG seals and phoney labels from well-known producers as well as non-existent wineries.

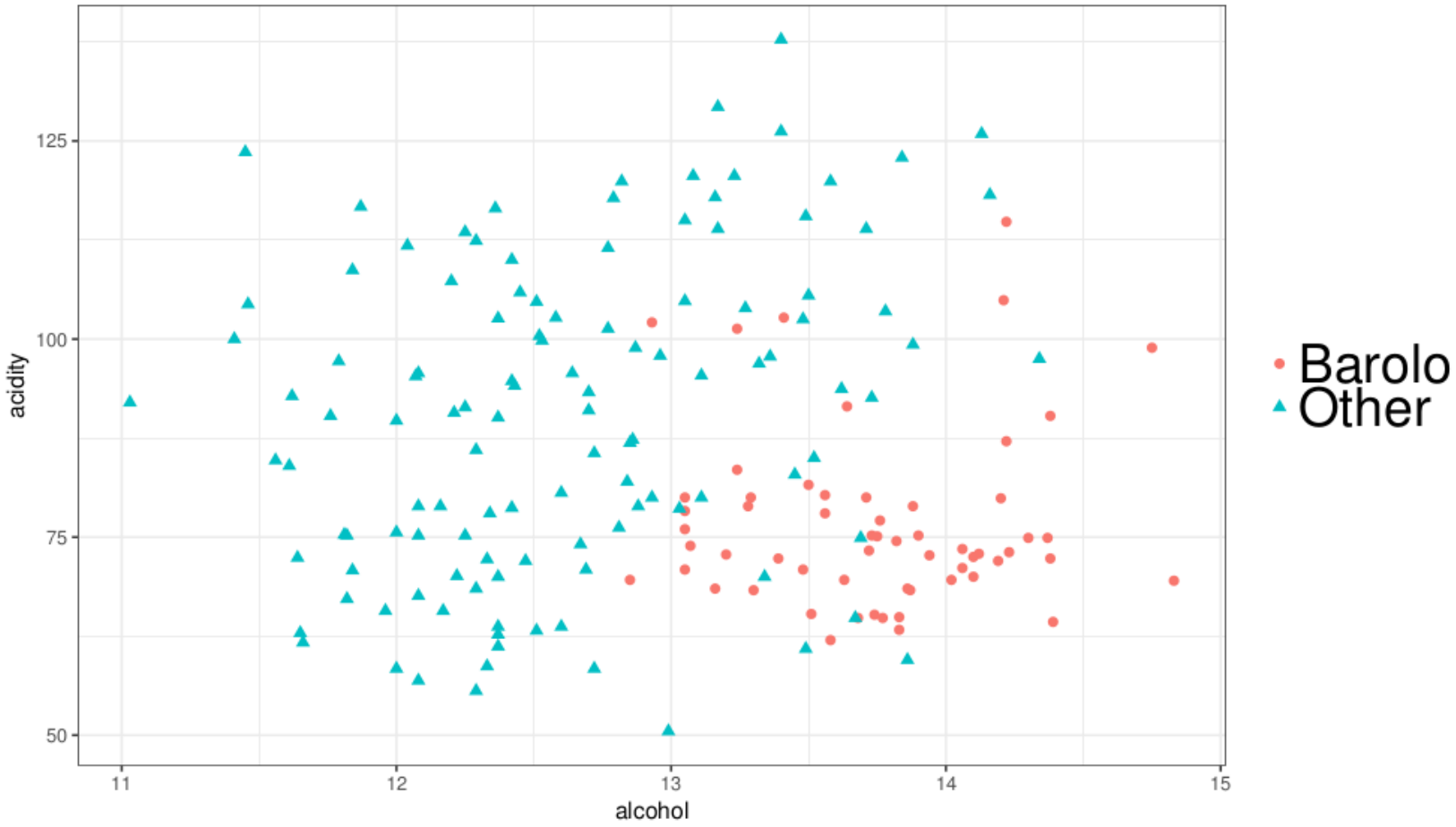
Ten people have been charged. It is thought the con could be worth €750,000.

➢ The following slides are adapted from Pierre-Alexandre Mattei and his "Machine Learning" course

One Example

- Would it be possible to create an AI device that learns to estimates the probability that a bottle of wine is fake or not?
- The device would need some information about the wine, for example some chemical properties:
  - alcohol content
  - acidity
- One of the wines the bad guys counterfeited was from the Barolo region.
  - Those wines have "pronounced tannins and acidity", and "moderate to high alcohol levels (Minimum 13%)". (Wikipedia)
  - So, can we use a computer to do so?

Our Dataset



Probabilistic Model for Classification

- There are two schools of modelling when it comes to building a probabilistic model of our data $\{x_n, t_n\}$.
- Both assume that the data are sampled from a distribution $p(x, t)$, or equivalently $p(x,\ C_k)$.
- The **generative school** builds models for the joint distribution $p(x, t)$.
  - This is quite hard, because this means we'll have in particular to learn the distribution of the data $p(x)$, which can be very complicated.
- The **discriminative school** builds models for the conditional distribution $p(t \mid x)$.
  - This is much easier, because we don't care about $p(x)$, but this is also often less powerful (in particular, all the training set needs to be labelled).

A Generative Model

➢ So, we want to define a distribution $p(x, C_k)$ Using the product rule, we can write

$$p(x, C_k) = p(x \mid C_k) p(C_k)$$

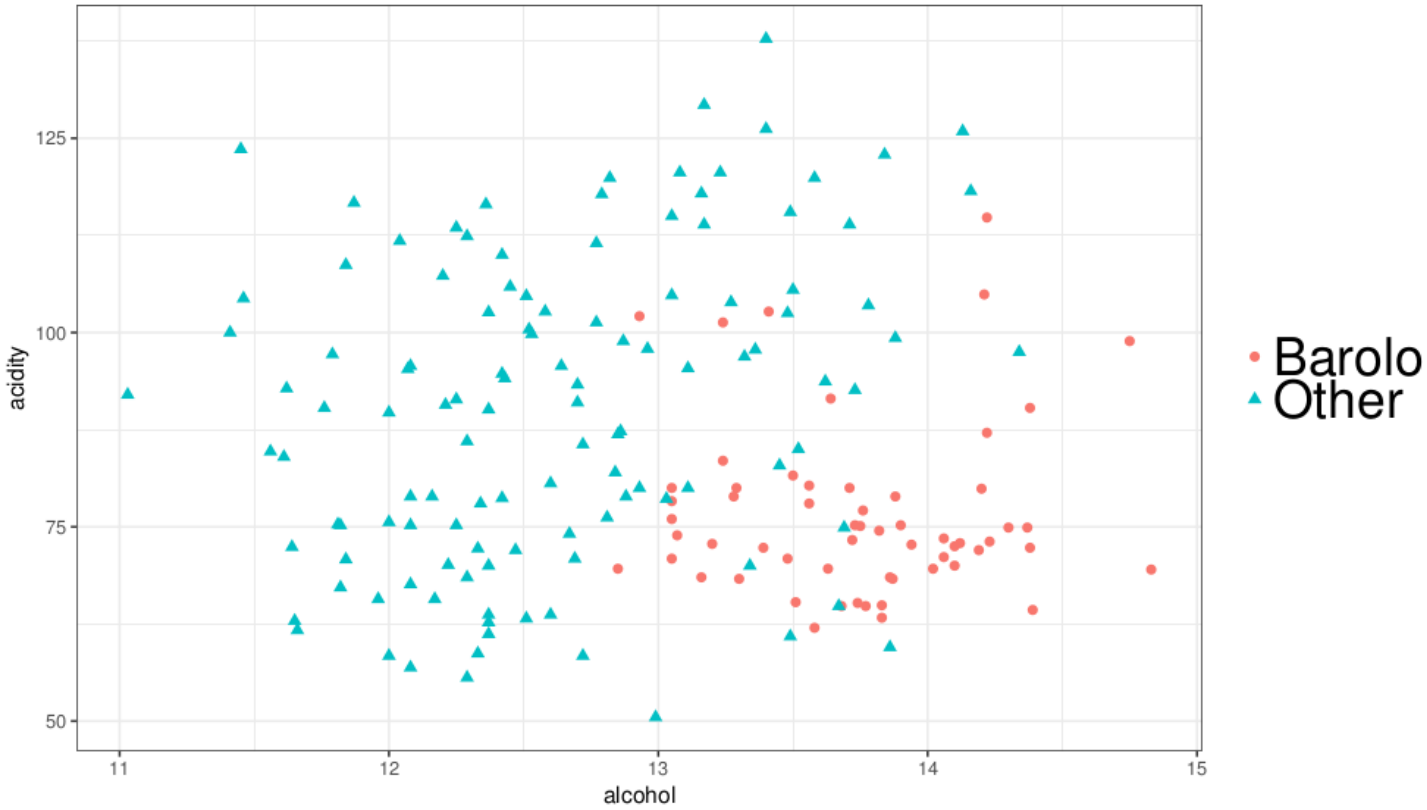
➢ This involve two terms:

- ➢ The class conditional densities $p(x \mid C_k)$, which are the K different distributions $p(x \mid C_1), \ \ldots, \ (x \mid C_K)$ of the K classes.
- ➢ The class prior probabilities $p(C_k)$, which are the proportions $p(C_1), \ \ldots, \ (C_K)$ of the classes in the population.

➢ Of course, both $p(x \mid C_k)$ and $p(C_k)$ will need to be learned, but let's assume that we just know them.

A Generative Model

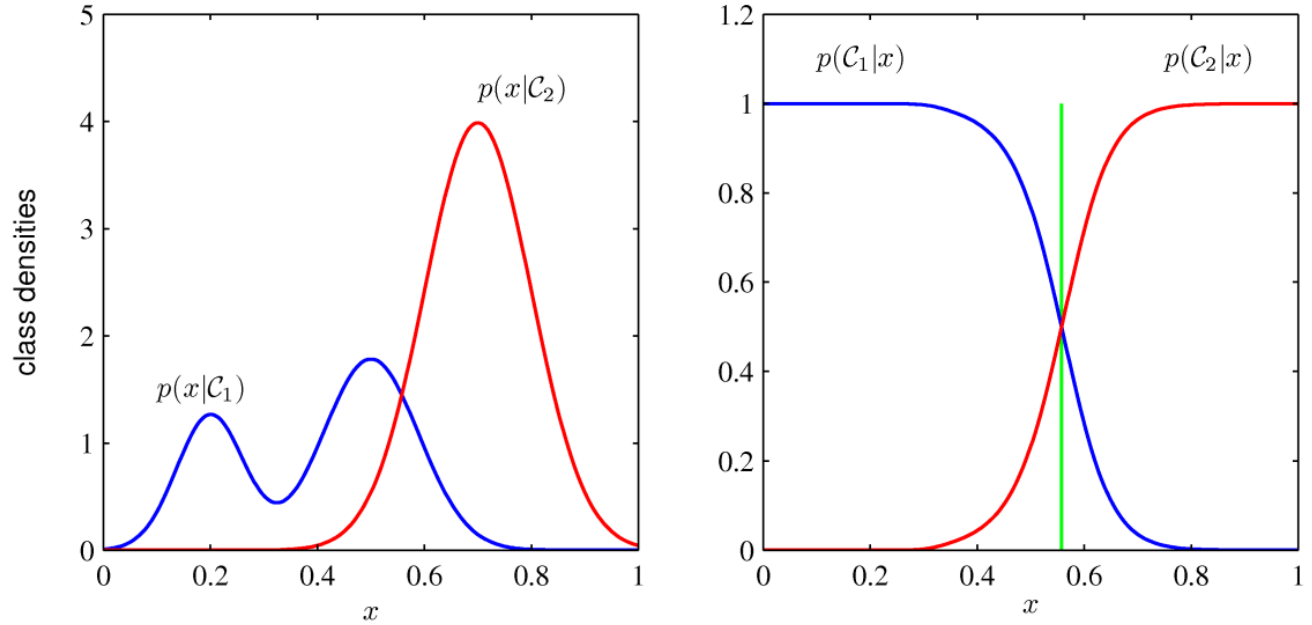
➢ Let's assume we have learned $p(x \mid C_1)$ (density of the Barolos), $p(x \mid C_2)$ (density of the other wines), $p(C_1)$ (proportion of Barolos), $p(C_2)$ (proportion of the other wines).



Estimate Classes

➢ Our main goal is to estimate the probability that the bottle is truly a Barolo. Using Bayes:

$$p(C_1 \mid \boldsymbol{x}) = \frac{p(\boldsymbol{x} \mid C_1)p(C_1)}{p(\boldsymbol{x})} = \frac{p(\boldsymbol{x} \mid C_1)p(C_1)}{p(\boldsymbol{x} \mid C_1)p(C_1) + p(\boldsymbol{x} \mid C_2)p(C_2)}$$



➢ Image: Simplified view on only one variable

Closer Look to the Formula

$$p(C_1 \mid \boldsymbol{x}) = \frac{p(\boldsymbol{x} \mid C_1)p(C_1)}{p(\boldsymbol{x})} = \frac{p(\boldsymbol{x} \mid C_1)p(C_1)}{p(\boldsymbol{x} \mid C_1)p(C_1) + p(\boldsymbol{x} \mid C_2)p(C_2)}$$

➢ It only involves the class-conditional densities and the class proportions. Rewrite:

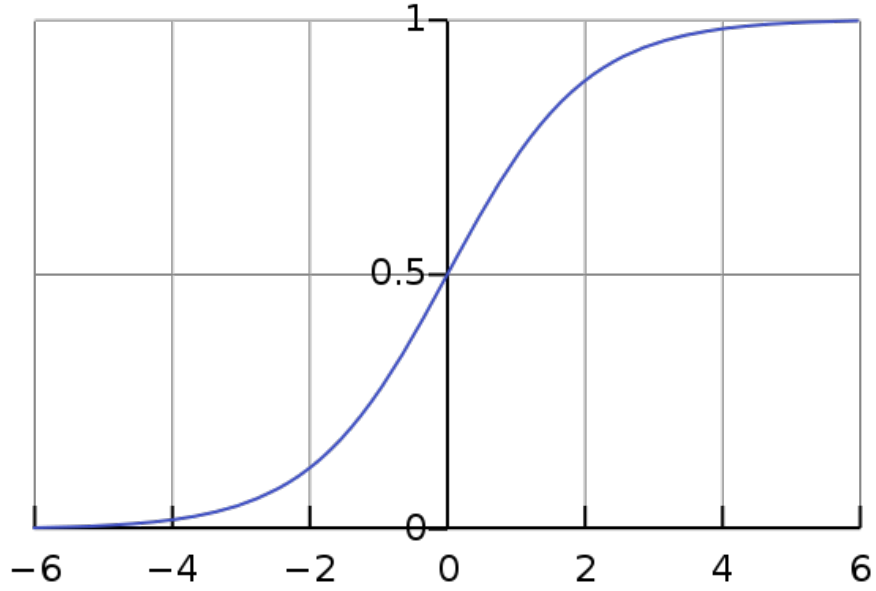
$$p(C_1 \mid \boldsymbol{x}) = \sigma(a(\boldsymbol{x})), \text{ with } a(\boldsymbol{x}) = \ln \frac{p(\boldsymbol{x} \mid C_1)p(C_1)}{p(\boldsymbol{x} \mid C_2)p(C_2)} = \ln \frac{p(\boldsymbol{x}, C_1)}{p(\boldsymbol{x}, C_2)}$$

Closer Look to the Formula

$$p(C_1 \mid \boldsymbol{x}) = \sigma\big(a(\boldsymbol{x})\big), \text{ with } a(\boldsymbol{x}) = \ln \frac{p(\boldsymbol{x} \mid C_1)p(C_1)}{p(\boldsymbol{x} \mid C_2)p(C_2)} = \ln \frac{p(\boldsymbol{x}, C_1)}{p(\boldsymbol{x}, C_2)}$$

➢ With σ being the logistic sigmoid function

$$\sigma(a) = \frac{1}{1 + \exp(-a)} = \frac{e^x}{e^x + 1}$$



The Sigmoid Function

- Why on earth do we need this weird sigmoid function?

- Interpretation: we can interpret $a(x)$ as the amount of evidence about x being a true Barolo, because it's the logarithm of the ratio of probabilities of being true and being fake:
  - Using Bayes's rule, we can see that $a(\boldsymbol{x}) = \ln \frac{p(C_1 \mid \boldsymbol{x})}{p(C_2 \mid \boldsymbol{x})}$

- Unification of the generative and discriminative schools: using the logistic function makes a bridge between the generative methods, and the discriminative ones and neural networks … you will see soon

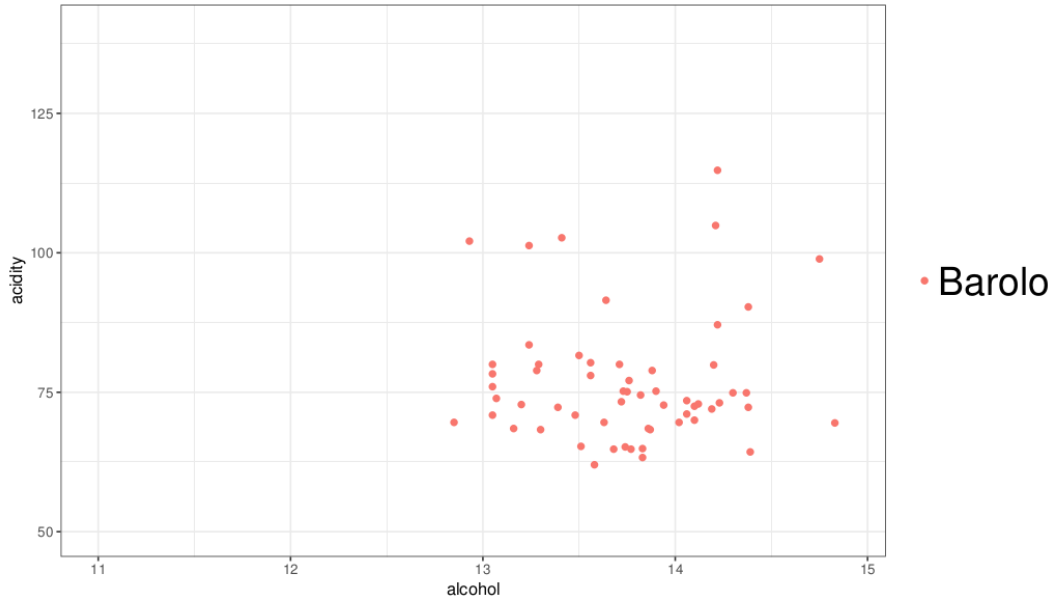
What we have done, what we still have to do...

- **What we have done**.
  - If we know the class-conditional densities $p(x \mid C_k)$ and the class proportions $p(C_k)$, we can compute the posterior probabilities $p(C_k \mid x)$, and use them to classify data.

- **What we still have to do**. How can we find the class-conditional densities and the proportions?
  - We can choose a parametrized model family
  - For that we could use for instance Gaussian Mixture Models
  - Then use MLE to find the best parameters for a Gaussian

Maximum Likelihood Estimators

➢ Let's look at the Barolos:



➢ We now need to fit a Gaussian that describes the data best:

$$N(\boldsymbol{x};\boldsymbol{\mu},\boldsymbol{\Sigma}) = \sqrt{\frac{1}{(2\pi)^2\,|\boldsymbol{\Sigma}|}}\exp\left(-\frac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu})^T\boldsymbol{\Sigma}^{-1}(\boldsymbol{x}-\boldsymbol{\mu})\right)$$

Maximum Likelihood Estimation

- The Maximum Likelihood Estimation (MLE) is a method of estimating the parameters of a model. This estimation method is one of the most widely used.

- The method of maximum likelihood selects the set of values of the model parameters that maximizes the likelihood function. Intuitively, this maximizes the "agreement" of the selected model with the observed data.

- The Maximum-likelihood Estimation gives an unified approach to estimation.

What is a Maximum Likelihood Estimator?

➢ The likelihood that one datapoint was generated by any distribution function is given by

$$l\left(\boldsymbol{x}, \boldsymbol{\theta}\right) = p\left(\boldsymbol{x}; \boldsymbol{\theta}\right)$$

➢ For the entire dataset, we have the likelihood

$$L\left(X, \boldsymbol{\theta}\right) = \prod_{i=1}^{n} p(\boldsymbol{x}^{(i)}; \boldsymbol{\theta})$$

What is a Maximum Likelihood Estimator?

➢ Goal of the MLE is to find θ in such a way that $L\left(X, \theta\right)$ is maximized

$$\theta_{\text{MLE}} = \text{argmax}_{\theta}\, L\left(X, \theta\right)$$

➢ Very often (for simplicity), the log likelihood is used

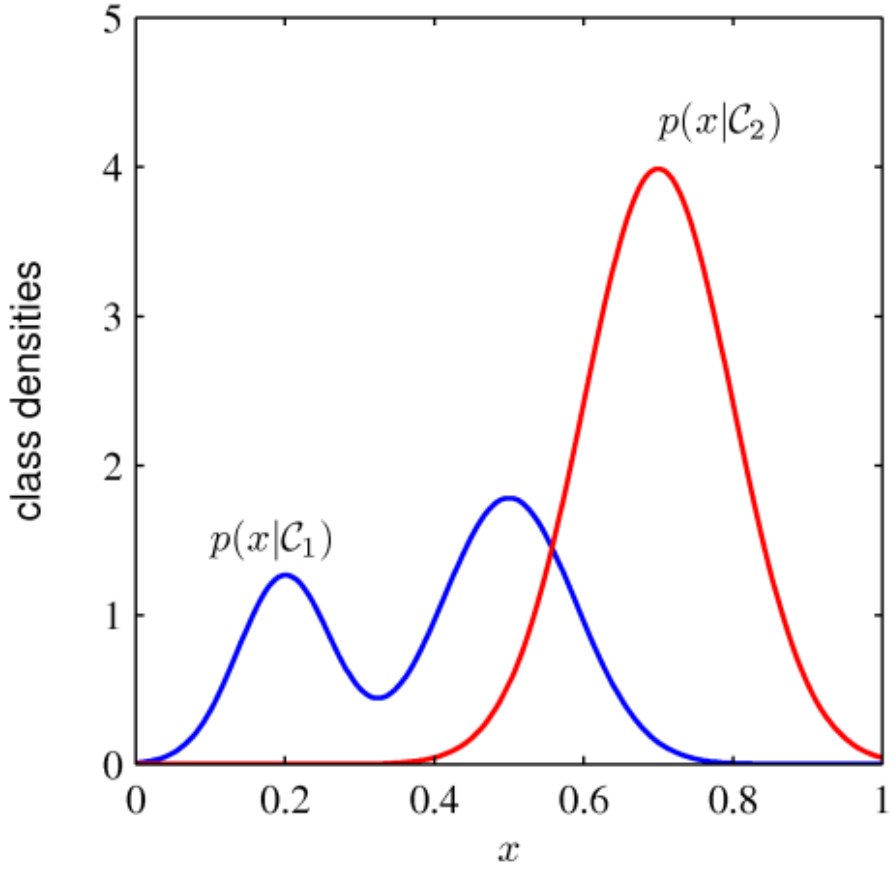
$$\theta_{\text{MLE}} = \text{argmax}_{\theta} \log \prod_{i=1}^{n} p(\boldsymbol{x}^{(i)}; \boldsymbol{\theta}) = \ = \text{argmax}_{\theta} \sum_{i=1}^{n} \log p(\boldsymbol{x}^{(i)}; \boldsymbol{\theta})$$

- ➢ This doesn't change the maximum of the function
- ➢ Eases many calculations

Example: MLE for a Gaussian

- For the sake of simplicity, we only approximate a one-dimensional gaussian for the Barolo, i.e., we just concentrate on one feature

$$\frac{1}{\sigma\sqrt{2\pi}}\exp\left[-\frac{(x-\mu)^2}{2\sigma^2}\right]$$



Example: MLE for a Gaussian

➢ Target: Estimate μ and σ for a Gaussian. The likelihood is

$$L(X\,|\,\theta) = \prod_{i=1}^{n} \frac{1}{\sigma\sqrt{2\pi}} \exp\left[-\frac{\left(x-\mu\right)^2}{2\sigma^2}\right]$$

➢ Taking the log results in an easier version:

$$LL(X\,|\,\theta) = -\frac{1}{2} n \log 2\pi - n\log\sigma - \sum_{i=1}^{n} \frac{\left(x-\mu\right)^2}{2\sigma^2}$$

Example: MLE for a Gaussian

➢ Derivate for μ:

$$\frac{\partial LL}{\partial \mu} = \frac{1}{\sigma^2} \sum_{i=1}^{n} \left(x_i - \mu \right)$$

➢ And for σ^2:

$$\frac{\partial LL}{\partial \sigma^2} = \frac{1}{2\sigma^2} \left[\frac{1}{\sigma^2} \sum_{i=1}^{n} \left(x_i - \mu \right)^2 - n \right]$$

➢ These derivates are 0 if

$$\mu = \frac{1}{n} \sum_{i=1}^{n} x_i \qquad \text{and} \qquad \sigma = \frac{1}{n} \sum_{i=1}^{n} \left(x_i - \mu \right)^2$$

Result

- The Estimator works the same in the multi-dimensional case, except the math gets a bit more iffy (Matrices)

- Now, we have a model for the Barolos
  - We Could now fit a Gaussian over the other wines
  - We calculate our priors, i.e., class prior probabilities, which is simply the ratio of Barolos of all wines in our dataset

- We have everything to compute:

$$p(C_1 \mid \boldsymbol{x}) = \frac{p(\boldsymbol{x} \mid C_1)p(C_1)}{p(\boldsymbol{x})} = \frac{p(\boldsymbol{x} \mid C_1)p(C_1)}{p(\boldsymbol{x} \mid C_1)p(C_1) + p(\boldsymbol{x} \mid C_2)p(C_2)}$$

Keywords

- Generative Models
- Discriminative Models & Logistic Regression
- Information Theory
- Well Behaved Gradients

From generative to discriminative models

➢ We defined a distribution $p(\boldsymbol{x}, C_k)$ over features and classes. We saw how to learn it using the decomposition

$$p(\boldsymbol{x},\ C_k) = p(\boldsymbol{x} \mid C_k)p(C_k)$$

➢ Using this distribution, we were able to compute the posterior probabilities of the classes $p(C_k \mid \boldsymbol{x})$, and to use them for classification.

➢ But, the only thing that we actually need for classification are those posterior probabilities.

➢ **So, could we avoid having to compute $p(x, C_k)$ and directly target the posterior probabilities?**

From generative to discriminative models

➢ The answer to this question is the general rationale of discriminative models:

Learn a way to transform any input vector x into its posterior class distributions

$$p(C_1|x), \ldots, p(C_K|x)$$

From generative to discriminative models

➢ So, we're looking for a way to transform any input vector x into its posterior class distributions $p(C_1 \mid x), \ldots, p(C_K \mid x)$.

➢ For simplicity, let's start with the binary case ($K = 2$).

➢ We have already seen:

$$p(C_1 \mid \boldsymbol{x}) = \sigma(a(\boldsymbol{x})), \text{ with } a(\boldsymbol{x}) = \ln \frac{p(x, C_1)}{p(x, C_2)}$$

From generative to discriminative models

➢ So, with generative models, we have the formula $p(C_1 \mid x) = \sigma\big(a(x)\big)$, where the function $a(x)$ comes from the class proportions and the model $p\big(x, C_k\big)$.

➢ The main idea of discriminative modelling will be to forget that $a(x)$ comes from a generative model, and to rather treat $a(x)$ as an unknown function to be learned from the data.

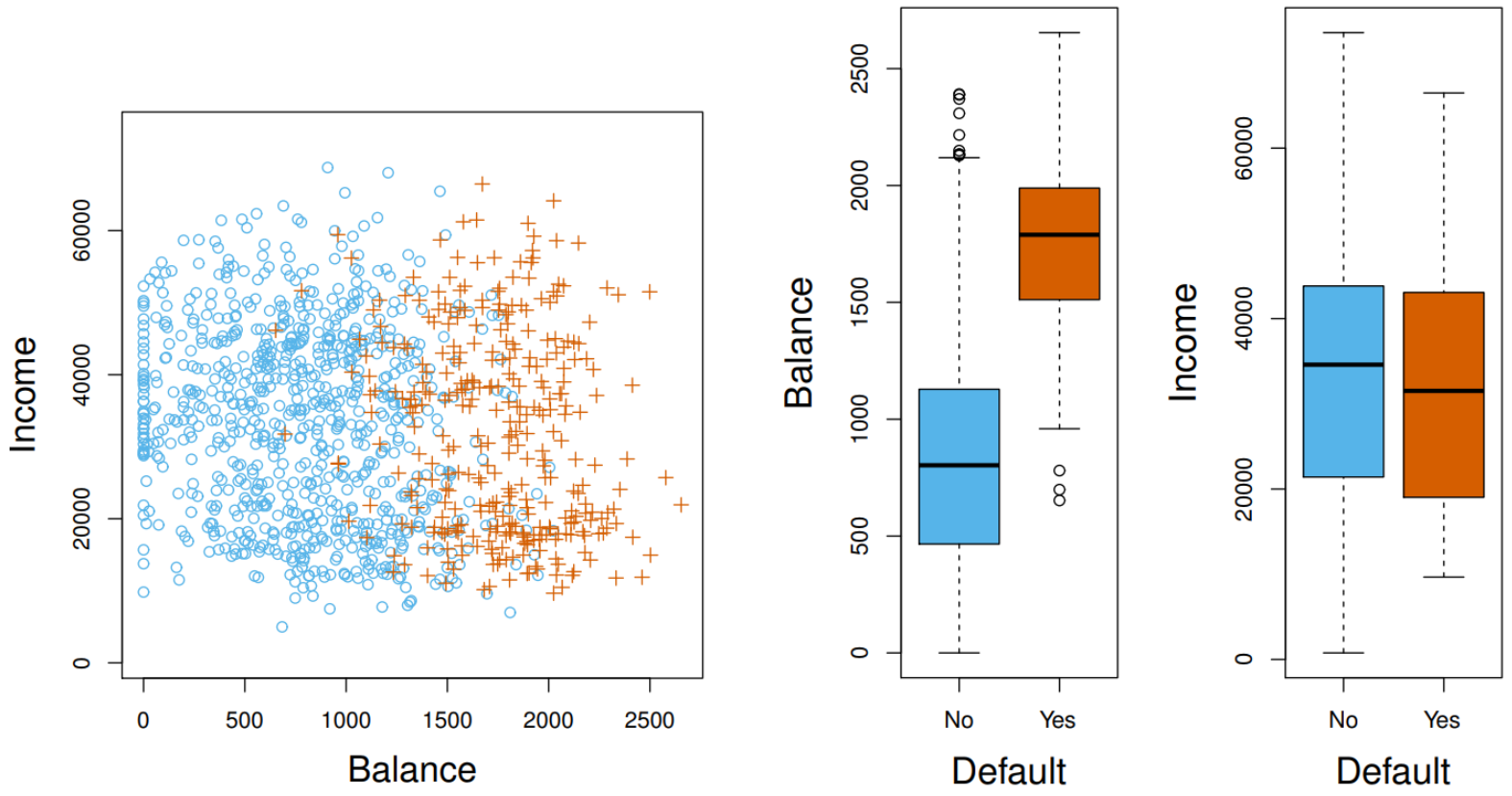
Say Hi to the Big Family of Discriminative Models

➢ The General model is

$$p(C_1 \mid \boldsymbol{x}) = \sigma(a(\boldsymbol{x}))$$

➢ it would be impractically hard to learn any kind of function without making additional assumptions:
- ➢ **Logistic regression** assumes that the function a is linear
- ➢ **Advanced Machine Learning** assumes that the function a is a composition of many linear and nonlinear functions
- ➢ **Gaussian processes** and **nonparametric methods** more generally, try to make as few assumptions as possible regarding a

Example for Logistic Regression: Credit Card Default



Logistic Regression

➢ In Logistic regression we model $a(\boldsymbol{x})$ as a linear regression

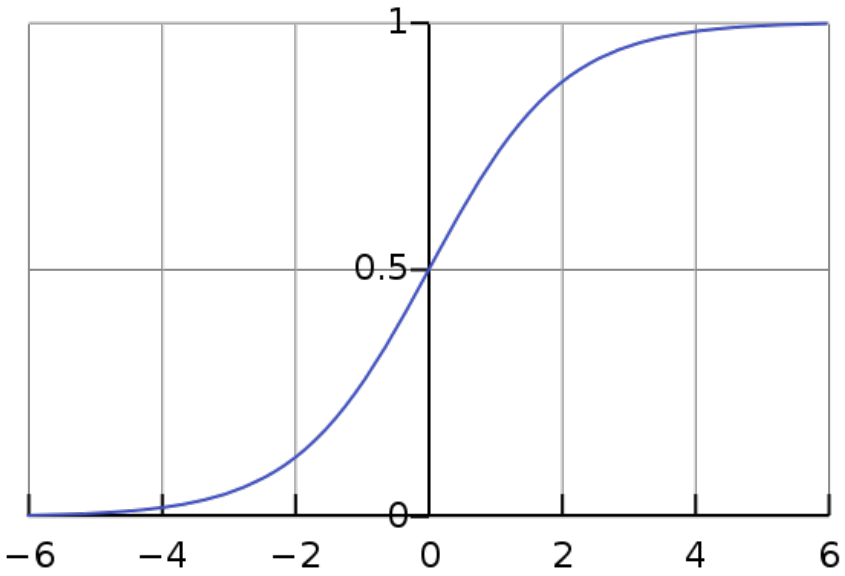
$\beta_0 + \beta_1 x$

➢ Put everything together, with $\boldsymbol{\theta} = \left(\beta_0, \beta_1\right)^T$:

$$h_{\boldsymbol{\theta}}(\boldsymbol{x}) = p\left(y \middle| \boldsymbol{x}, \boldsymbol{\theta}\right) = \frac{e^{\beta_0 + \beta_1 X}}{1 + e^{\beta_0 + \beta_1 X}}$$

Learning Goal

$$\sigma(x) = \frac{e^x}{e^x + 1}$$

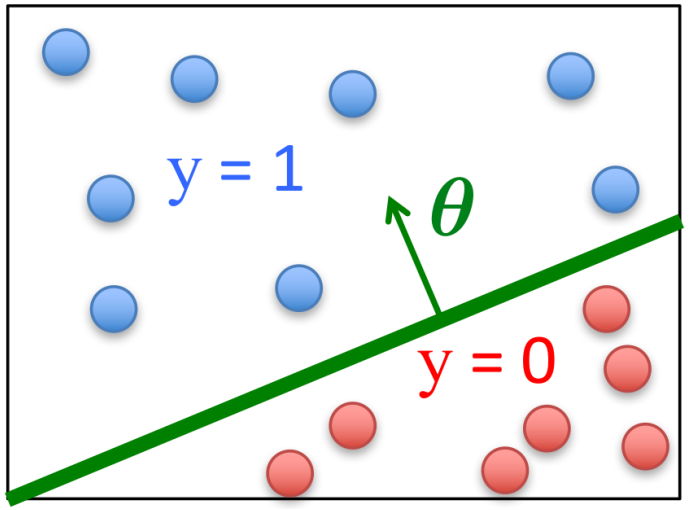


➢ Our Model:

$h_\theta(\boldsymbol{x}) = p\left(y \middle| \boldsymbol{x}, \boldsymbol{\theta}\right) = \sigma(\boldsymbol{\theta}^T \boldsymbol{x})$

➢ $\boldsymbol{\theta}^T \boldsymbol{x}$ should be large negative for negative instances

➢ $\boldsymbol{\theta}^T \boldsymbol{x}$ should be large positive for positive instances



➢ Select a threshold t and
  ➢ Predict $y = 1$ if $p\left(y \middle| \boldsymbol{x}, \boldsymbol{\theta}\right) \geq t$
  ➢ Predict $y = 0$ if $p\left(y \middle| \boldsymbol{x}, \boldsymbol{\theta}\right) < t$

sdu.dk
#sdudk

Logistic Regression Objective Function

➢ We cannot just use the least squared approach as with linear regression

$$J(\theta) = \frac{1}{n} \sum_{i}^{n} \left(h_\theta\left(x^{(i)}\right) - y^{(i)} \right)^2$$

➢ Since the logistic regression model with the sigmoid function results in a non-convex optimization problem.
➢ And due to other problems would not lead to the optimal parameter set

Remember MLE

- The Maximum Likelihood Estimation (MLE) is a method of estimating the parameters of a model. This estimation method is one of the most widely used.

- The method of maximum likelihood selects the set of values of the model parameters that maximizes the likelihood function. Intuitively, this maximizes the "agreement" of the selected model with the observed data.

- And it worked like a charm with Gaussian ☺

Computing the Likelihood

➢ Let's calculate the log-likelihood:

$$\ell(a) = \sum_{i=1}^{n} \log p\left(y^{(i)} \middle| \boldsymbol{x}^{(i)}; \boldsymbol{\theta}\right) = \; = \sum_{i=1}^{n} \log \left[h_\theta\left(\boldsymbol{x}^{(i)}\right)^{y^{(i)}} \cdot \left(1 - h_\theta\left(\boldsymbol{x}^{(i)}\right)\right)^{1-y^{(i)}} \right] =$$

$$= \sum_{i=1}^{n} \left[y^{(i)} \log h_\theta\left(\boldsymbol{x}^{(i)}\right) + \left(1 - y^{(i)}\right) \log\left(1 - h_\theta\left(\boldsymbol{x}^{(i)}\right)\right) \right]$$

Computing the Likelihood

➢ We have

$$\sum_{i=1}^{n}\left[y^{(i)}\log h_\theta\left(\boldsymbol{x}^{(i)}\right)+\left(1-y^{(i)}\right)\log\left(1-h_\theta\left(\boldsymbol{x}^{(i)}\right)\right)\right]$$

➢ We want to maximize this function, which is equivalent to minimizing its opposite, which is called the cross-entropy loss:

$$J(\theta)=-\sum_{i=1}^{n}\left[y^{(i)}\log h_\theta\left(\boldsymbol{x}^{(i)}\right)+\left(1-y^{(i)}\right)\log\left(1-h_\theta\left(\boldsymbol{x}^{(i)}\right)\right)\right]$$

➢ The cross-entropy loss is the most commonly used loss for neural networks, and is a way of doing maximum likelihood without necessarily saying it.

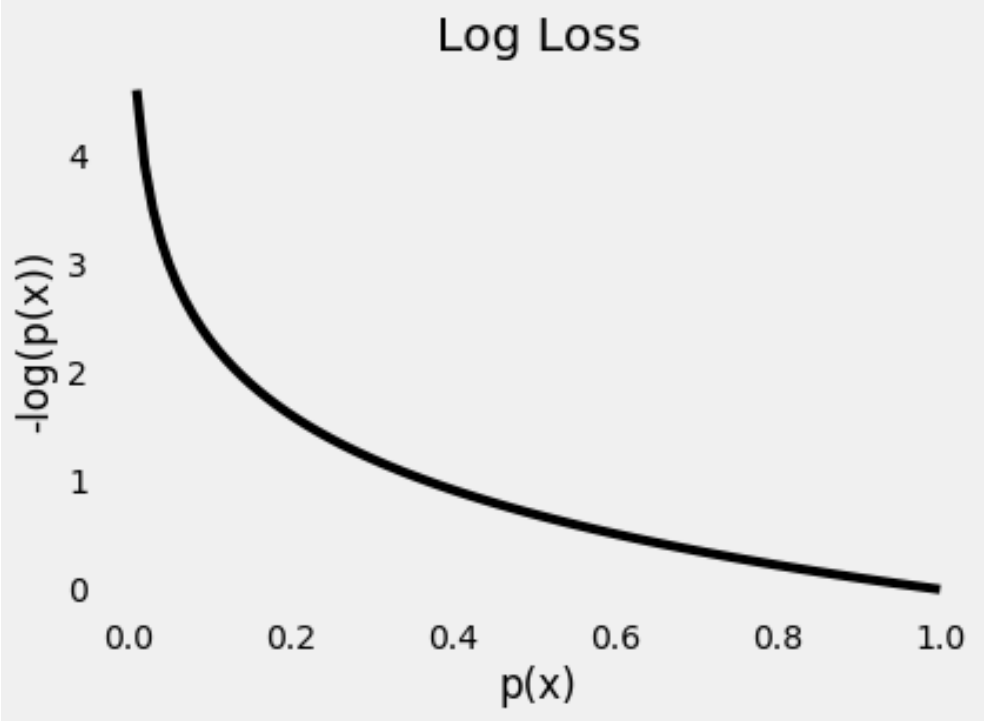
Intuition Behind the Objective

$$J(\theta) = -\sum_{i=1}^{n}\left[y^{(i)}\log h_\theta\left(\boldsymbol{x}^{(i)}\right) + \left(1-y^{(i)}\right)\log\left(1-h_\theta\left(\boldsymbol{x}^{(i)}\right)\right)\right]$$

- Depending on y, the loss function assumes these values:

$$J(\theta) = \begin{cases} -\log\left(h_\theta(\boldsymbol{x})\right) & \text{for} \quad y=1 \\ -\log\left(1-h_\theta(\boldsymbol{x})\right) & \text{for} \quad y=0 \end{cases}$$

- Effect: Extremely negative with wrong classifications
- Trained with gradient descent methods



Keywords

- Generative Models
- Discriminative Models & Logistic Regression
- Information Theory
- Well Behaved Gradients

Cross Entropy Loss

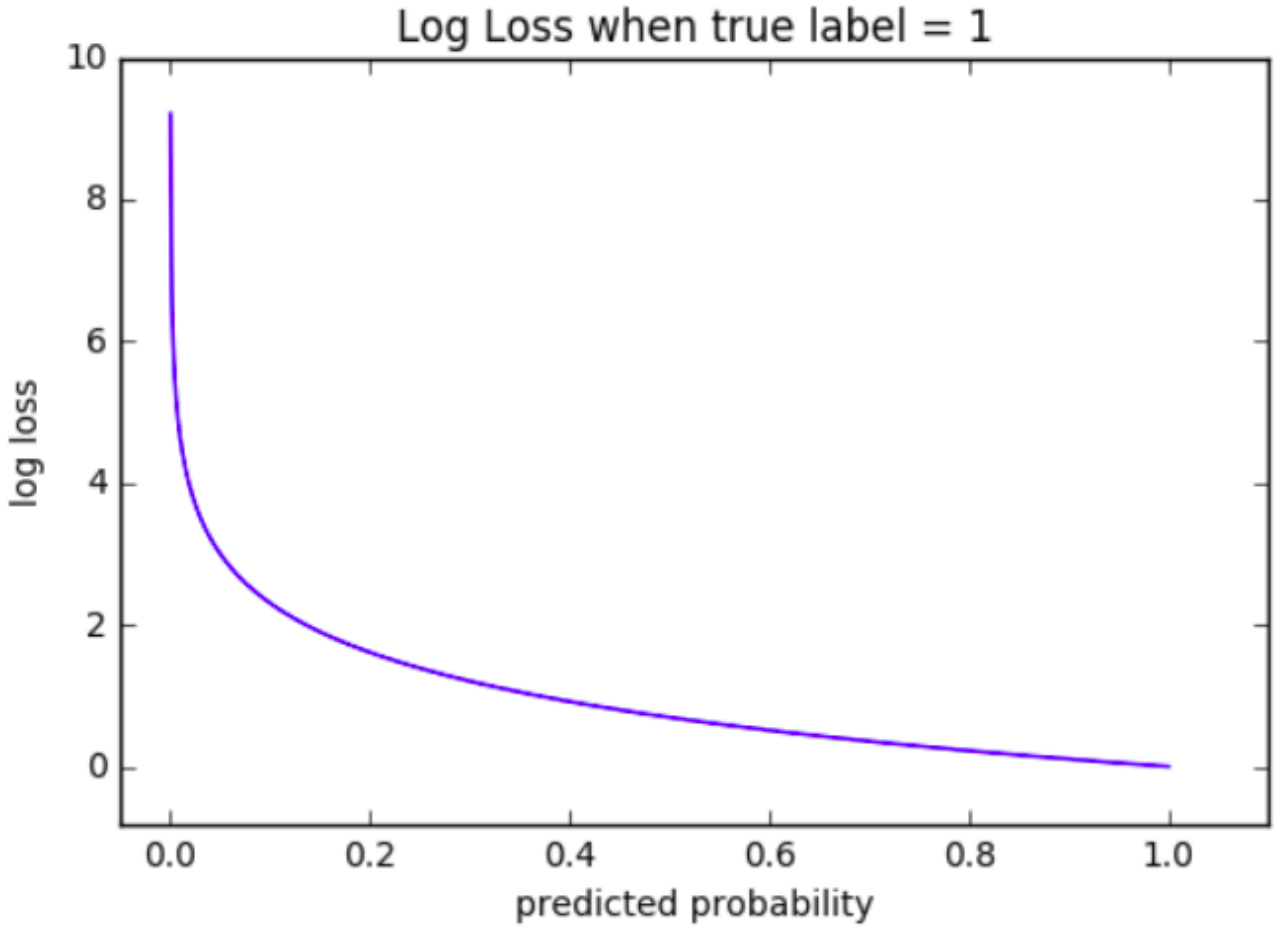
➢ Minimizing the cross-entropy loss corresponds to maximize the log likelihood (but comes from a different theoretical background):

$$J(\theta) = -\mathbb{E}\left[p(y \mid \boldsymbol{x}; \boldsymbol{\theta})\right]$$

➢ In case of our 2-case logistic regression:

$$-\frac{1}{n}\sum_{i=1}^{n}\left[y^{(i)}\log h_{\boldsymbol{\theta}}(\boldsymbol{x}) + \left(1 - y^{(i)}\right)\log\left(1 - h_{\boldsymbol{\theta}}(\boldsymbol{x})\right)\right]$$

Cross Entropy Loss



Information Theory

- To fully understand the origins of the most common loss functions, we have to resort back to the basics of statistical learning

- The key assumption is that our dataset $X = \{x^{(1)}, \ldots, x^{(m)}\}$ is drawn independently from the true but unknown data generating distribution p_{data}

- Now, let $p_{model}(\boldsymbol{x}; \boldsymbol{\theta})$ be a parametric family of probability distributions

Information Theory

- Intuition
  - The occurrence of an extremely unlikely event is more informative than the occurrence of a very likely event
    - There is no pandemic outbreak – is not very informative
    - There is a pandemic outbreak and you should stay home => very informative
- To quantify this intuition and need a function fulfilling:
  - Likely events should have low information content
    - Extreme case: Guaranteed events shall have no information content
  - Less likely events should have higher information content.
  - Independent events should have additive information.
    - Finding out that a tossed coin has come up as heads twice should convey twice as much information as finding out that a tossed coin has come up as heads once.

Self-Information

➢ In order to satisfy all three of these properties, we define the self-information of an event $X = x$ to be

$$-\log P(X = x)$$

➢ The information content is measured in
- ➢ **nats** … when using the base e
- ➢ **bits** or **shannons** … when using the base 2

➢ To measure the uncertainty for an entire distribution **(Shannon Entropy)**

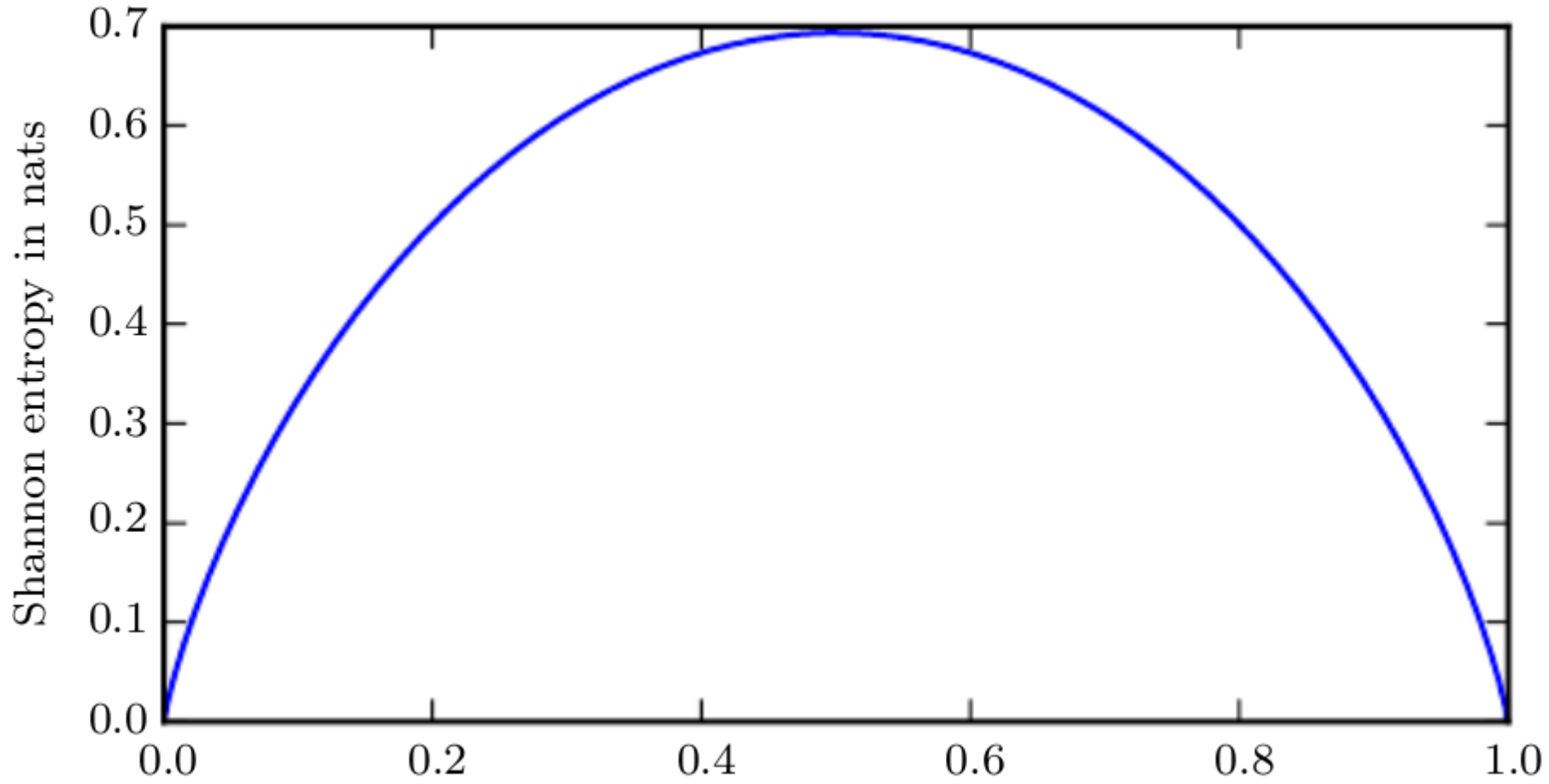
$$H(x) = \mathbb{E}_{x\sim P}\big[I(x)\big] = -\mathbb{E}_{x\sim P}[\log P(X = x)]$$

Shannon Entropy

$$H(x) = \mathbb{E}_{x\sim P}\big[I(x)\big] = -\mathbb{E}_{x\sim P}[\log P(X = x)]$$

- Expected amount of information of an event drawn from P
- It gives a lower bound on the number of bits (if the logarithm is base 2, otherwise the units are different) needed on average to encode symbols drawn from a distribution P.
- Distributions that are nearly deterministic (where the outcome is nearly certain) have low entropy; distributions that are closer to uniform have high entropy.

Shannon Entropy



The Maximum Likelihood Estimator

➢ The maximum likelihood estimator for θ is then defined as

$$\theta_{ML} = \text{argmax}_{\theta} p_{model}\left(X; \theta\right)$$
$$= \text{argmax}_{\theta} \prod_{i=1}^{m} p_{model}(\boldsymbol{x}^{(i)}; \boldsymbol{\theta})$$

➢ The argmax function is invariant to rescaling, thus

$$\theta_{ML} = \text{argmax}_{\theta} \sum_{i}^{m} \log p_{model}(\boldsymbol{x}^{(i)}; \boldsymbol{\theta})$$
$$= \text{argmax}_{\theta} \mathbb{E}_{\boldsymbol{x} \sim \hat{p}_{data}} \log p_{model}(\boldsymbol{x}; \boldsymbol{\theta})$$

Kullback-Leibler Divergence

- If we have two separate probability distributions $P(x)$ and $Q(x)$ over the same random variable x, we can measure how different these two distributions are using the Kullback-Leibler (KL) divergence:

$$D_{KL}\big(P||Q\big) = \mathbb{E}_{x\sim P}\left[\log\frac{P(x)}{Q(x)}\right] = \mathbb{E}_{x\sim P}\big[\log P(x) - \log Q(x)\big]$$

  - In discrete case it can be read as the extra amount of information needed to send a message drawn from P, when we use a code designed for Q.

- Properties:
  - Non-negative
  - 0 if and only if P and Q are the same function
  - Not symmetric! (=> no true distance measure)

Cross Entropy

- Closely related to the KL divergence is the cross-entropy

$$H(P,Q) = H(P) + D_{KL}(P||Q) = -\mathbb{E}_{x\sim P}\log Q(x)$$

- Interpretation very similar to KL divergence:
  - **The cross entropy is the average number of bits needed to encode data coming from a source with distribution p when we use model q**
- While as the KL divergence:
  - **measures the number of EXTRA bits required to encode data coming from a source with distribution p when we use model q**
- Minimizing the cross-entropy with respect to Q is equivalent to minimizing the KL divergence, because Q does not participate in the omitted term.

All Together

➢ Different point of view:

➢ Minimize the dissimilarity between the empirical distribution $\hat{p}_{data}$ (defined by the data) and the model with Kullback-Leibler (KL) distance:

$$D_{KL}\left(\hat{p}_{data},\ p_{model}\right) = \mathbb{E}_{\boldsymbol{x}\sim\hat{p}_{data}}\left[\log\frac{\hat{p}_{data}}{p_{model}(\boldsymbol{x};\boldsymbol{\theta})}\right] = \mathbb{E}_{\boldsymbol{x}\sim\hat{p}_{data}}\left[\log\hat{p}_{data} - \log p_{model}(\boldsymbol{x};\boldsymbol{\theta})\right]$$

➢ We have just seen, that minimizing this term is equivalent to minimize the cross-entropy:

$$-\mathbb{E}_{\boldsymbol{x}\sim\hat{p}_{data}}\left[\log p_{model}(\boldsymbol{x};\boldsymbol{\theta})\right]$$

➢ Minimizing **KL** is equivalent to minimizing the **cross-entropy** or maximizing the **log-likelihood**

About the Gradient

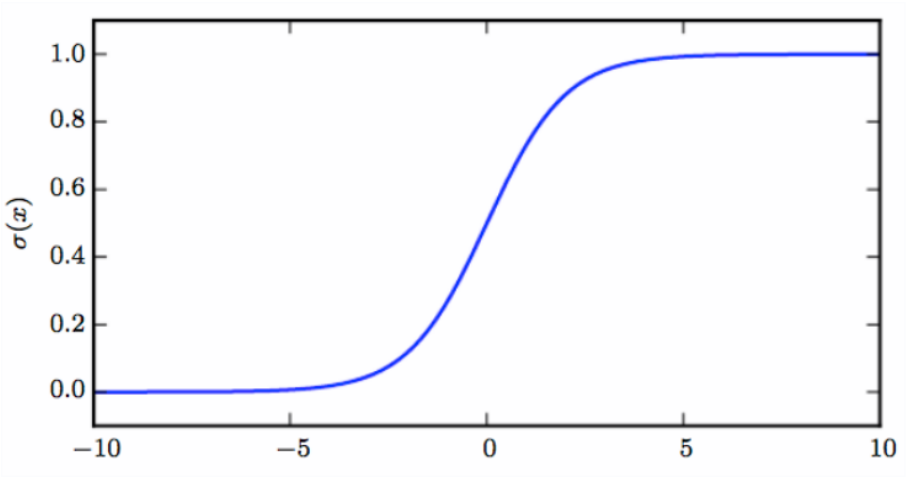
- Gradient must be large and predictable enough to serve as good guide to the learning algorithm
- Functions that **saturate** (become very flat) undermine this
  - Because the gradient becomes very small
  - Happens when activation functions producing output of hidden/output units saturate

- **Negative log-likelihood** helps avoid saturation problem for many models
  - Many output units involve exp functions that saturate when its argument is very negative
  - Log function in Negative log likelihood cost function undoes exp of some units

Keywords

- Generative Models
- Discriminative Models & Logistic Regression
- Information Theory
- Well Behaved Gradients

Revisit: Sigmoid Units

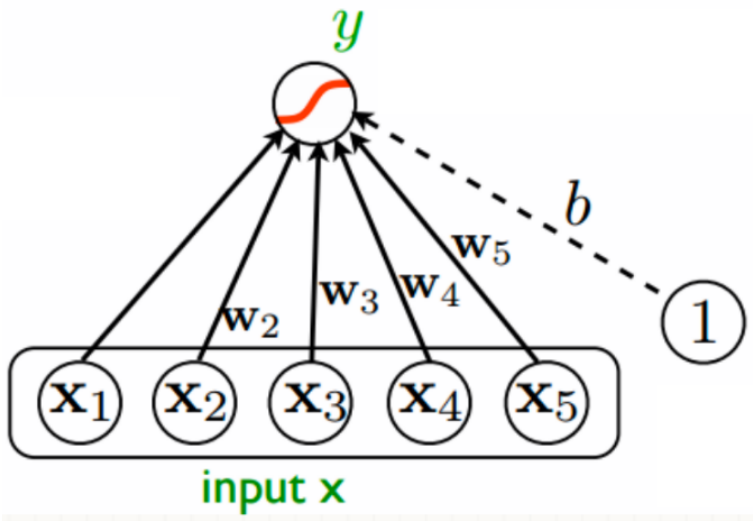


➢ Sigmoid always gives a gradient

$\hat{y} = \sigma(\boldsymbol{w}^T \boldsymbol{h} + b)$

with

$$\sigma(x) = \frac{1}{1 + \exp(-x)}$$



➢ Sigmoid Unit has two components:

- ➢ A linear layer to compute

$z = \boldsymbol{w}^T \boldsymbol{h} + b$

- ➢ A sigmoid activation function to convert z into a probability

Cross-Entropy

➢ Using sigmoid directly:

$$p(y|\theta) = \prod_{n=1}^{N} \sigma(\theta^T x_n)^{y_n} \cdot \left(1 - \sigma(\theta^T x_n)\right)^{1-y_n}$$

➢ With the loss function:

$$J(\theta) = -\ln p(y|\theta) = -\sum_{n=1}^{N} \left(y_n \ln\left(\sigma(\theta^T x_n)\right) + (1 - y_n) \ln\left(1 - \sigma(\theta^T x_n)\right) \right)$$

➢ Problem:
- ➢ Sigmoid saturates to 0 or 1 when z becomes very negative/positive
- ➢ Gradient can shrink to too small to be useful for learning, whether model has correct or incorrect answer

Different Formulation

➢ At the end of the day we need to produce a number which represents $\hat{y} = P(y = 1 \mid \boldsymbol{x})$; thus lies between 0 and 1

➢ Therefore, let's predict a number

$z = \log \tilde{P}(y = 1 \mid \boldsymbol{x})$

with $\tilde{P}$ being an unnormalized random distribution:

$\log \tilde{P}(y) = yz$ $\qquad\qquad$ $\tilde{P}(y) = \exp(yz)$

➢ Normalizing leads to a real distribution:

$$P(y) = \frac{\exp(yz)}{\sum_{y'=\{0,1\}} \exp(y'z)} \qquad\qquad P(y) = \sigma\big((2y-1)z\big)$$

Different Formulation

➢ At the end of the day we need to produce a number which represents $\hat{y} = P(y = 1 \mid \boldsymbol{x})$; thus lies between 0 and 1

➢ Therefore, let's predict a number

$$z = \log \tilde{P}(y = 1 \mid \boldsymbol{x})$$

with $\tilde{P}$ being an unnormalized random distribution:

$$\log \tilde{P}(y) = yz \qquad \tilde{P}(y) = \exp(yz)$$

➢ Normalizing leads to a real distribution:

$$P(y) = \frac{\exp(yz)}{\sum_{y'=\{0,1\}} \exp(y'z)} \qquad P(y) = \sigma\big((2y-1)z\big)$$

$$\log \tilde{P}(y=1) = z$$
$$\log \tilde{P}(y=0) = 0$$

$$\tilde{P}(y=1) = e^z$$
$$\tilde{P}(y=0) = e^0 = 1$$

$$P(y=1) = \frac{e^z}{1+e^z}$$
$$\mathrm{P}(y=0) = \frac{1}{1+e^z}$$

$$\begin{cases} \sigma(z) = \frac{1}{1+e^{-z}} = \frac{e^z}{1+e^z} & \text{for } y = 1 \\ \sigma(-z) = \frac{1}{1+e^z} & \text{for } y = 0 \end{cases}$$

Sigmoid with Softplus

➢ Using

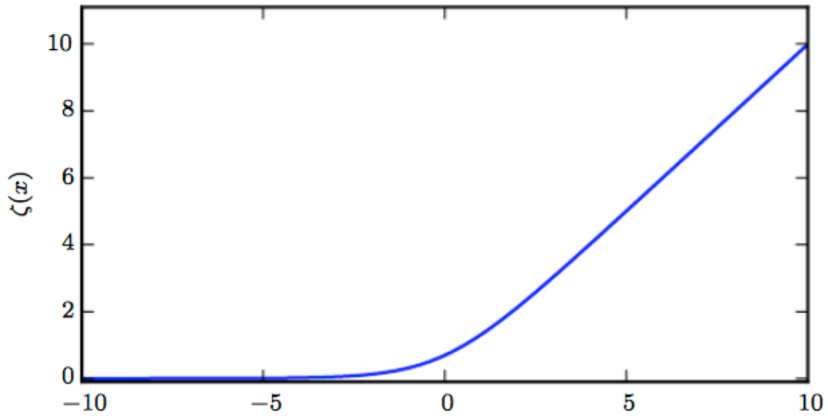
$$P(y \mid z) = \sigma\Big((2y-1)z\Big)$$

➢ The Loss function can be defined as:

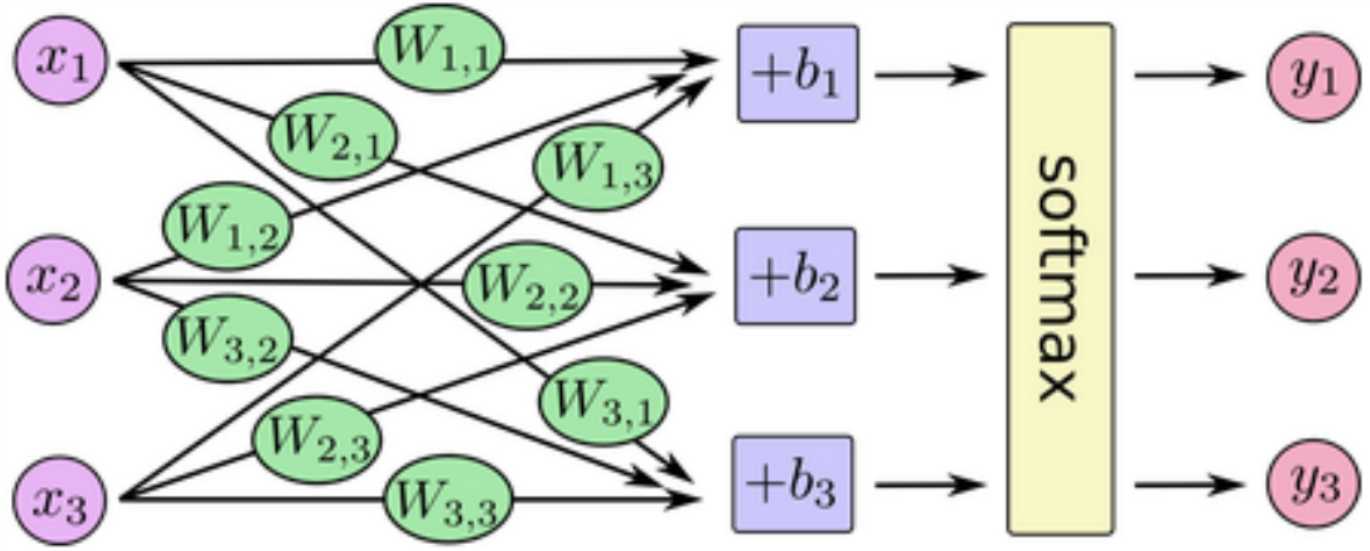
$$J(\theta) = -\log P(y \mid \boldsymbol{x}) = -\log\sigma\Big((2y-1)z\Big) = \zeta\Big((1-2y)z\Big)$$

$$\zeta(x) = \log\big(1+\exp(x)\big) \approx x^{+} = \max(0,x)$$

- ➢ With ζ being the softplus function
- ➢ only when saturates when $(1-2y)z$ is very negative
- ➢ That only happens, when the model already has the right answer



Revisit the Softmax Function



Log-likelihood of the Softmax Function

- As a general rule of thumb, exponentiations should be undone by the cost function
- Thus, the log-likelihood is a natural candidate for the loss function

$$\text{logsoftmax}(\boldsymbol{z})_i = z_i - \log \sum_j \exp\left(z_j\right)$$

- Input z_i always has a direct contribution and can never saturate
  - This first term encourages z_i being pushed up
  - The second term encourages entire $\boldsymbol{z}$ being pushed down

Second Term

➢ We consider the term

$$\log \sum_j \exp\left(z_j\right)$$

➢ Can be approximated to $\max_j z_j$ (when largest z_j is significantly larger than all other z_k)

➢ Costs penalizes most active incorrect prediction

➢ In case of a correct prediction, then z_i and $\log \sum_j \exp\left(z_j\right)$ will roughly cancel out.

More about the Softmax Function

➢ Overall, unregularized maximum likelihood will drive the model to learn parameters that drive the softmax to predict the fraction of counts of each outcome observed in the training set

$$\text{softmax}\left(\boldsymbol{z}\left(\boldsymbol{x};\boldsymbol{\theta}\right)\right)_i \approx \frac{\sum_{j=1}^{m} \mathbf{1}_{y^{(j)}=i,\boldsymbol{x}^{(j)}=\boldsymbol{x}}}{\sum_{j=1}^{m} \mathbf{1}_{\boldsymbol{x}^{(j)}=\boldsymbol{x}}}$$

➢ Objective functions that do not use a log to undo the $\exp$ of softmax fail to learn when argument of $\exp$ becomes very negative, causing gradient to vanish

➢ Especially the squared error is a poor loss function for softmax units

Other Output Types

- Linear, Sigmoid and Softmax output units are the most common
- Neural networks can generalize to any kind of output layer
- Principle of maximum likelihood provides a guide for how to design a good cost function for any output layer
- If we define conditional distribution $p(\boldsymbol{y} \mid \boldsymbol{x}; \boldsymbol{\theta})$, principle of maximum likelihood suggests we use $-\log p(\boldsymbol{y} \mid \boldsymbol{x}; \boldsymbol{\theta})$ for our cost function

Keywords

- Generative Models
- Discriminative Models & Logistic Regression
- Information Theory
- Well Behaved Gradients

How’s my teaching?

https://www.admonymous.co/lukasgalke

Department of Mathematics
and Computer Science

sdu.dk

#sdudk

Advanced Machine Learning

Optimizers in Brief

Lukas Galke Poech

Spring 2026

SDU

Problems with optimizing deep nets

- Non-convex: We can end-up in local minima
- Inexact: Our gradient is not exact but only an estimate
- Poor correspondence between local and global structure (cliffs etc)
- The starting point matters
- The learning rate matters

Now, why are there different optimizers?

- Different optimizers try to avoid or minder the aforementioned problems
- They differentiate in the way they are doing it, often the differences are rather subtle
  - SGD
  - RMSProp
  - Adagrad
  - Adadelta
  - Adam
  - Adamax
  - Nadam
  - AdamW
  - **...**

Tricks they use: Decreasing Learning Rate

- Batch* gradient descent (i.e. using all samples) can use a fixed learning rate
  - Since true gradient becomes small and then 0

- SGD has a source of error
  - Random sampling of m training samples
  - Sufficient condition for SGD convergence

    $$\sum \varepsilon = \infty; \ \sum \varepsilon^2 < \infty$$

  - Common to decay learning rate linearly until iteration

    τ: $\varepsilon_k = (1 - \alpha)\varepsilon_0 + \alpha\varepsilon_\tau$ with $\alpha = \frac{k}{\tau}$.

  - After iteration τ, it is common to leave ε constant

- *This is unfortunately a very confusing terminology: Methods which use the entire dataset are called Batch-methods. Do not confuse with the term "minibatch" from SDG.

Tricks they use: Momentum

- Momentum helps accelerate Gradient Descent(GD) when we have
  - Surfaces that curve more steeply in one direction than in another direction
  - Facing high curvature
  - Small but consistent gradients
  - Noisy gradients

- Dampens oscillation

- Algorithm accumulates moving average of past gradients and move in that direction

Tricks they use: Momentum

- Introduce variable v, or velocity
  - It is the direction and speed at which parameters move through parameter space
  - Momentum is physics is mass times velocity
  - The momentum algorithm assumes unit mass
  - A hyperparameter $\alpha \in [0,1)$ determines exponential decay

$$\boldsymbol{v} \leftarrow \alpha \boldsymbol{v} - \varepsilon \nabla_{\theta} \left(\frac{1}{m} \sum_{i=1}^{m} L\left(f(x^{(i)}; \theta), y^{(i)} \right) \right)$$

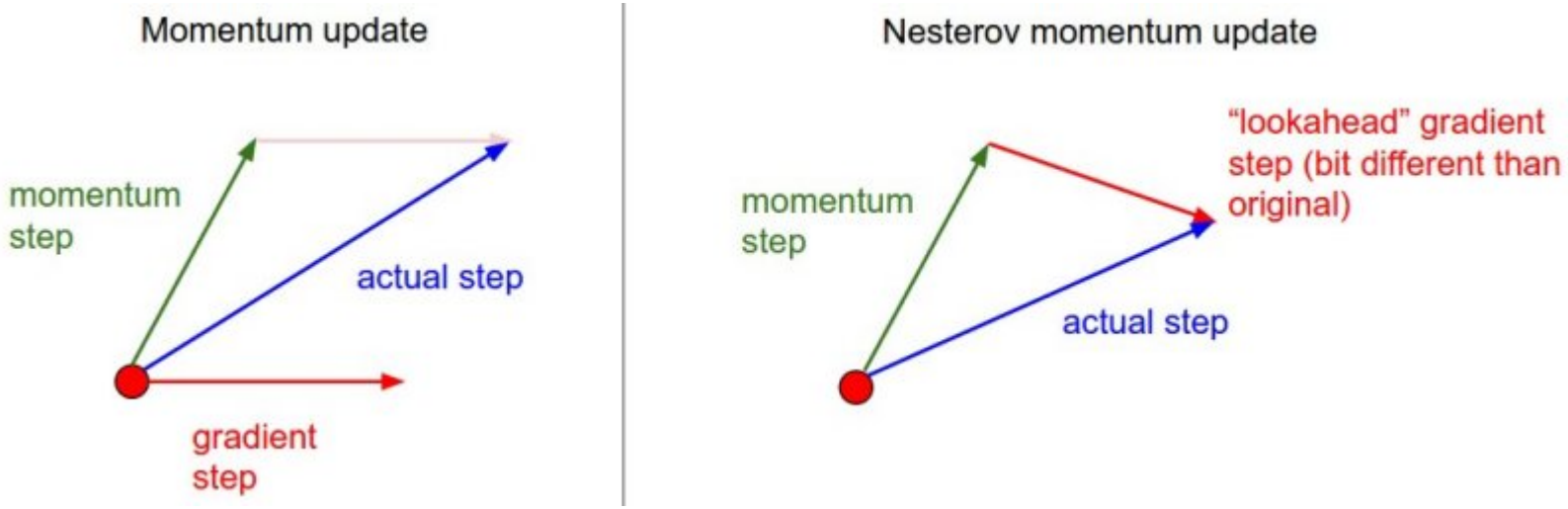
$$\theta \leftarrow \theta + \boldsymbol{v}$$

  - The larger α is relative to ε, the more previous gradients affect the current direction

Tricks they use: Nesterov Accelerated Gradient

- Nesterov acceleration optimization is like a ball rolling down the hill but knows exactly when to slow down before the gradient of the hill increases again.
- We calculate the gradient not with respect to the current step but with respect to the future step.
- We evaluate the gradient of the look-ahead and based on the importance then update the weights.

Tricks they use: Nesterov Accelerated Gradient



Tricks they use: Adaptive Learning Rate

- We have discussed how important the learning rate is
  - Set too small, we train very slow
  - Set too large, we might wildly jump above minima

- General Idea of adaptive learning:
  - Modify the learning rate **for individual parameters**
  - Based on their history
  - The different optimizers mainly distinguish each other how the adapt the learning rate.

- This is were the "ADA" comes from in most optimizers

Back to the Optimizers

- **SGD**
- RMSProp
- Adagrad
- Adadelta
- **Adam**
- Adamax
- Nadam
- **AdamW**

Stochastic Gradient Descent

- Can be enhanced by momentum
- Can use Nesterov look-ahead
- Can be combined with learning rate schedulers
- But not adaptive

input : γ (lr), θ_0 (params), $f(\theta)$ (objective), λ (weight decay),
μ (momentum), τ (dampening), *nesterov, maximize*

for $t = 1$ **to** $\ldots$ **do**
  $g_t \leftarrow \nabla_\theta f_t(\theta_{t-1})$
  if $\lambda \neq 0$
    $g_t \leftarrow g_t + \lambda\theta_{t-1}$
  if $\mu \neq 0$
    if $t > 1$
      $\mathbf{b}_t \leftarrow \mu\mathbf{b}_{t-1} + (1-\tau)g_t$
    else
      $\mathbf{b}_t \leftarrow g_t$
    if *nesterov*
      $g_t \leftarrow g_t + \mu\mathbf{b}_t$
    else
      $g_t \leftarrow \mathbf{b}_t$
  if *maximize*
    $\theta_t \leftarrow \theta_{t-1} + \gamma g_t$
  else
    $\theta_t \leftarrow \theta_{t-1} - \gamma g_t$

return θ_t

Adam

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$
$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

Bias corrected momentums

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t} \qquad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t$$

- Adaptive Moment Estimation
- Next to the exp. dec. avg. of past gradients m_t Adam also keeps an exp. dec. avg. of **squared** past gradients v_t
- Corresponding to estimated mean and ~variance of the gradient
- Larger steps when gradients are small, smaller steps when gradients are large
- Whereas momentum can be seen as a ball running down a slope, Adam behaves like a heavy ball with friction, which thus prefers flat minima in the error surface.

AdamW

- For SGD, L2 regularization is equivalent to weight decay
- **For Adam, L2 regularization is not equivalent to weight decay**
- It can be reconciled: AdamW
- AdamW removes the weight decay term from the initial gradient, but instead adds it when 1st and 2nd moment are consolidated into the parameter update.

Algorithm 1 SGD with L_2 regularization and SGD with decoupled weight decay (SGDW), both with momentum

given initial learning rate $\alpha \in \mathbb{R}$, momentum factor $\beta_1 \in \mathbb{R}$, weight decay/L_2 regularization factor $\lambda \in \mathbb{R}$
initialize time step $t \leftarrow 0$, parameter vector $\boldsymbol{\theta}_{t=0} \in \mathbb{R}^n$, first moment vector $\boldsymbol{m}_{t=0} \leftarrow \boldsymbol{0}$, schedule multiplier $\eta_{t=0} \in \mathbb{R}$
repeat
  $t \leftarrow t + 1$
  $\nabla f_t(\boldsymbol{\theta}_{t-1}) \leftarrow \text{SelectBatch}(\boldsymbol{\theta}_{t-1})$ ▷ select batch and return the corresponding gradient
  $\boldsymbol{g}_t \leftarrow \nabla f_t(\boldsymbol{\theta}_{t-1}) + \lambda \boldsymbol{\theta}_{t-1}$
  $\eta_t \leftarrow \text{SetScheduleMultiplier}(t)$ ▷ can be fixed, decay, be used for warm restarts
  $\boldsymbol{m}_t \leftarrow \beta_1 \boldsymbol{m}_{t-1} + \eta_t \alpha \boldsymbol{g}_t$
  $\boldsymbol{\theta}_t \leftarrow \boldsymbol{\theta}_{t-1} - \boldsymbol{m}_t - \eta_t \lambda \boldsymbol{\theta}_{t-1}$
until *stopping criterion is met*
return optimized parameters $\boldsymbol{\theta}_t$

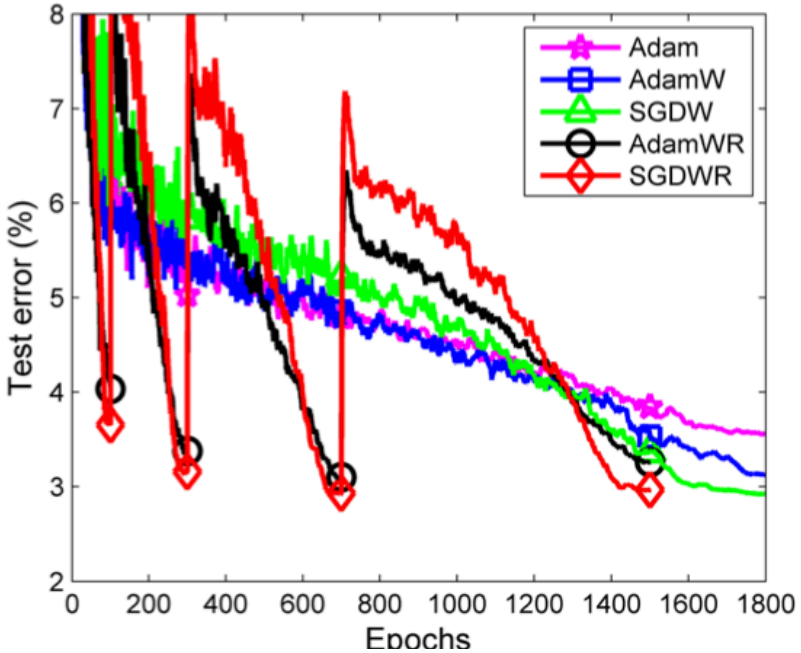
Algorithm 2 Adam with L_2 regularization and Adam with decoupled weight decay (AdamW)

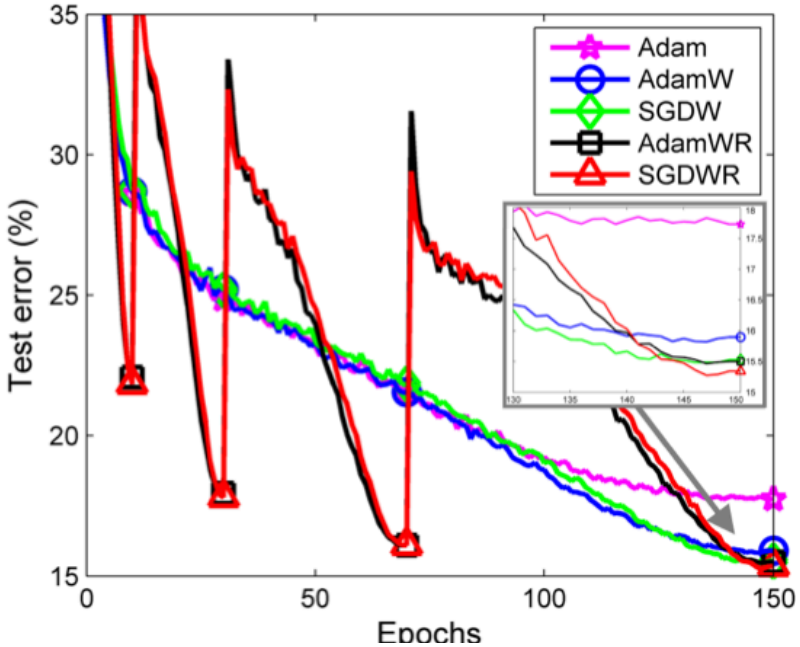
given $\alpha = 0.001, \beta_1 = 0.9, \beta_2 = 0.999, \epsilon = 10^{-8}, \lambda \in \mathbb{R}$
initialize time step $t \leftarrow 0$, parameter vector $\boldsymbol{\theta}_{t=0} \in \mathbb{R}^n$, first moment vector $\boldsymbol{m}_{t=0} \leftarrow \boldsymbol{0}$, second moment vector $\boldsymbol{v}_{t=0} \leftarrow \boldsymbol{0}$, schedule multiplier $\eta_{t=0} \in \mathbb{R}$
repeat
  $t \leftarrow t + 1$
  $\nabla f_t(\boldsymbol{\theta}_{t-1}) \leftarrow \text{SelectBatch}(\boldsymbol{\theta}_{t-1})$ ▷ select batch and return the corresponding gradient
  $\boldsymbol{g}_t \leftarrow \nabla f_t(\boldsymbol{\theta}_{t-1}) + \lambda \boldsymbol{\theta}_{t-1}$
  $\boldsymbol{m}_t \leftarrow \beta_1 \boldsymbol{m}_{t-1} + (1 - \beta_1) \boldsymbol{g}_t$ ▷ here and below all operations are element-wise
  $\boldsymbol{v}_t \leftarrow \beta_2 \boldsymbol{v}_{t-1} + (1 - \beta_2) \boldsymbol{g}_t^2$
  $\hat{\boldsymbol{m}}_t \leftarrow \boldsymbol{m}_t / (1 - \beta_1^t)$ ▷ β_1 is taken to the power of t
  $\hat{\boldsymbol{v}}_t \leftarrow \boldsymbol{v}_t / (1 - \beta_2^t)$ ▷ β_2 is taken to the power of t
  $\eta_t \leftarrow \text{SetScheduleMultiplier}(t)$ ▷ can be fixed, decay, or also be used for warm restarts
  $\boldsymbol{\theta}_t \leftarrow \boldsymbol{\theta}_{t-1} - \eta_t \left(\alpha \hat{\boldsymbol{m}}_t / (\sqrt{\hat{\boldsymbol{v}}_t} + \epsilon) + \lambda \boldsymbol{\theta}_{t-1} \right)$
until *stopping criterion is met*
return optimized parameters $\boldsymbol{\theta}_t$

sdu.dk
#sdudk

AdamW

➢ TLDR; if you want to use weight decay, always use AdamW instead of Adam





How’s my teaching?

https://www.admonymous.co/lukasgalke

Loss Functions and the Deep Learning Intuition

Lukas Galke Poech, AI506, Spring 2026

Keywords

Loss functions

- Information Theory
- Maximum Likelihood Estimation
- Binary/Categorical Cross-entropy
- KL divergence
- Mean squared error

Deep Learning Intuition

- Shortcut Learning & Simplicity Bias
- Lottery Ticket Hypothesis
- Residual connections as ensembles of simple functions
- Deep Double Descent & Grokking

Loss functions (continued)

Where we left of: Likelihood

The likelihood of a set of parameters is the probability of observing the data under the model.

Single example: $l(x, \theta) = p(x|\theta)$

Entire dataset: $L(x, \theta) = \prod_{x \in \mathcal{X}} p(x|\theta)$

In log space: $\log L(X, \theta) = \sum_{x \in \mathcal{X}} \log p(x|\theta)$

Maximum likelihood estimator

$$\theta_{\mathrm{MLE}} = \arg\max_\theta L(X, \theta) =$$
$$\arg\max_\theta \log L(X, \theta)$$

We have seen that we can calculate the argmax analytically for a Gaussian.

Generative vs. discriminative schools

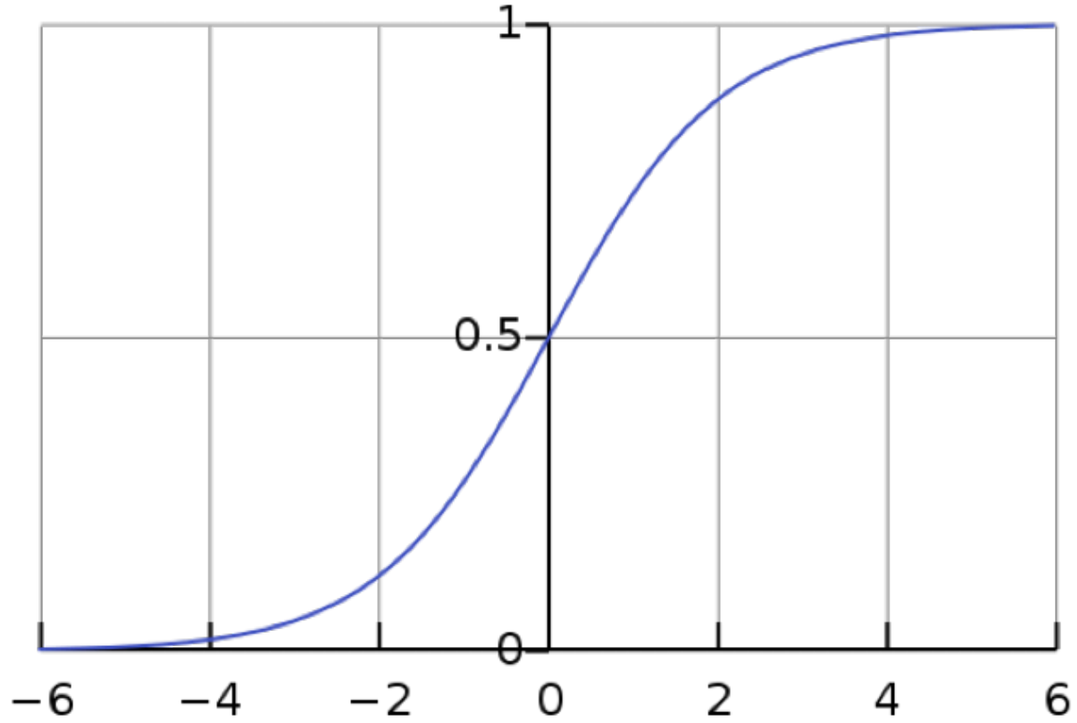
Generative: We must model $p(c, x)$, then we can derive $p(c|x) = \frac{p(x|c) \cdot p(c)}{p(x)}$

Discriminative: Let's estimate $p(c|x)$ directly from an arbitrary input vector!

Discriminative models

$$p(y|x) = \sigma(m(x))$$

with σ the logistic sigmoid function $\sigma(x) = \frac{e^x}{1+e^x}$



Why the logistic sigmoid function?

Maps $(-\infty, +\infty)$ to $(0, 1)$.

Smooth function, simple derivative.

It is mathematically derived from the log odds.

$$\log \text{odds} = \log \frac{p(x)}{1-p(x)}$$

$\sigma(x) = \frac{e^x}{1+e^x}$ and substitute in the log odds for x:

$$\sigma(x) = \frac{\frac{p(x)}{1-p(x)}}{1+\frac{p(x)}{1-p(x)}} = \frac{\frac{p(x)}{1-p(x)}}{\frac{(1-p(x))+p(x)}{1-p(x)}} = \frac{p(x)}{(1-p(x))+p(x)} = p(x)$$

Intermezzo: Information Theory

Shannon's information theory (1948)

Desiderata

- Continuity
- Symmetry
- Maximal value
- Additive

Surprisal

Given a distribution $P(X)$, how can we characterize how surprised we are when we observe a particular event x?

Intution: the smaller the probability of the event, the larger the surprise.

$$\text{Surprisal}(x) = \frac{1}{p(x)}$$

Shannon information as surprisal

$$h(x) = \log \frac{1}{p(x)} = -\log p(x)$$

Key advantage: Additive. Now, you can just add the surprisal of two events.

If $\log_2$ is used, the unit is called *bits*

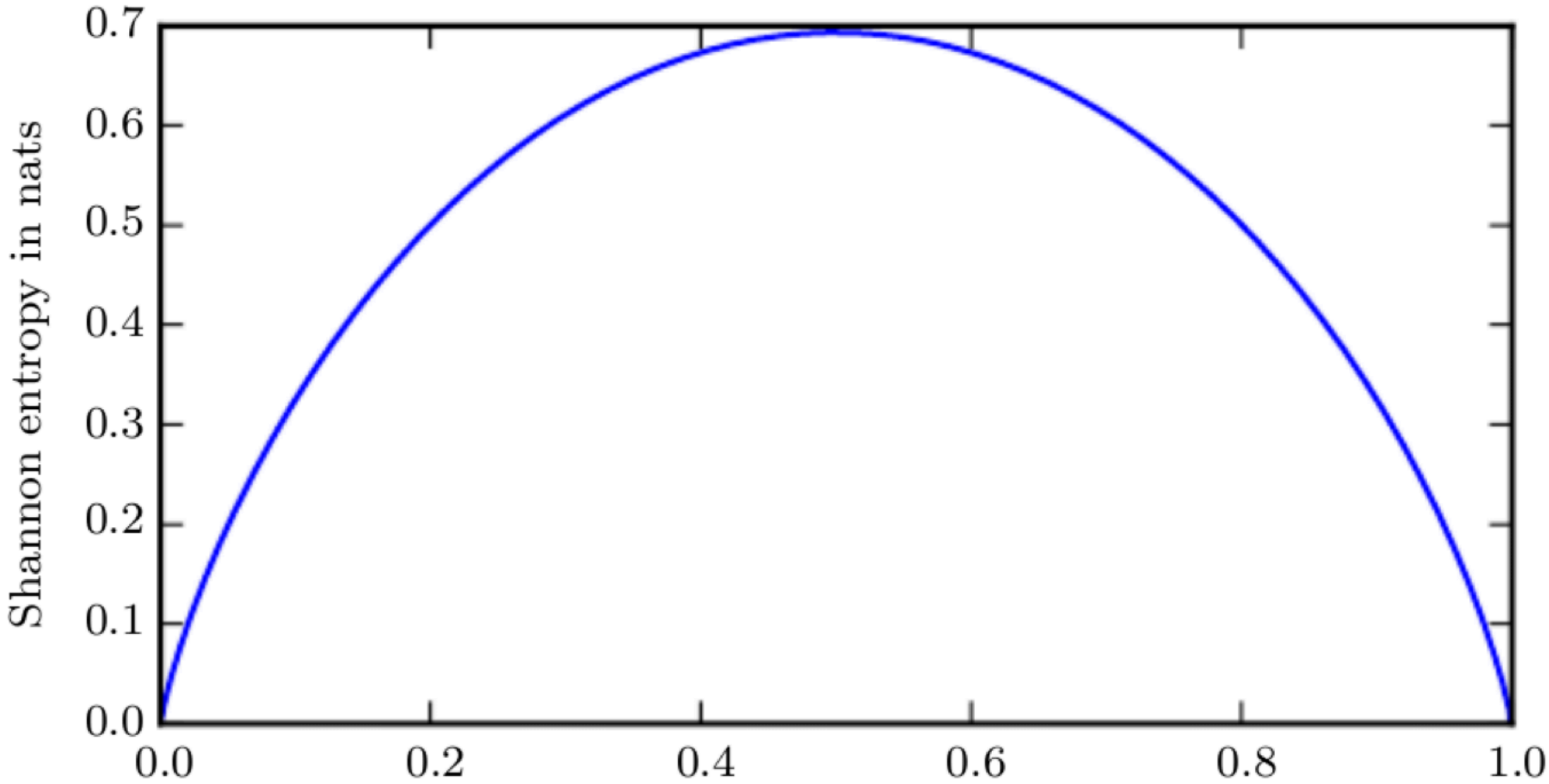
If base e is used, the unit is called *nats*

Entropy = Average Shannon Information

$$H(X) = \mathbb{E}[h(x)] = -\sum p(x) \log p(x)$$

Or, *uncertainty of a random variable.*

Entropy of a biased coin with probability p



Interpretation of Entropy

A variable with an entropy of $H(X)$ bits provides enough Shannon information to choose between $2^{H(X)}$ equally probable alternatives.

Cross-entropy

How surprised are you on average, if you assume a distribution Q, when it's actually P.

$$H(P, Q) = \mathbb{E}_p[h(x)] = -\sum p(x) \log q(x)$$

Back to our loss functions

Let's start with logistic regression

In logistic regression, m is just a linear model

$$m(x) = \theta^T x = \theta_0 + \theta_1 x$$

Then $\sigma(m(x)) = \frac{e^{\theta_0 + \theta_1 x}}{1 + e^{\theta_0 + \theta_1 x}}$

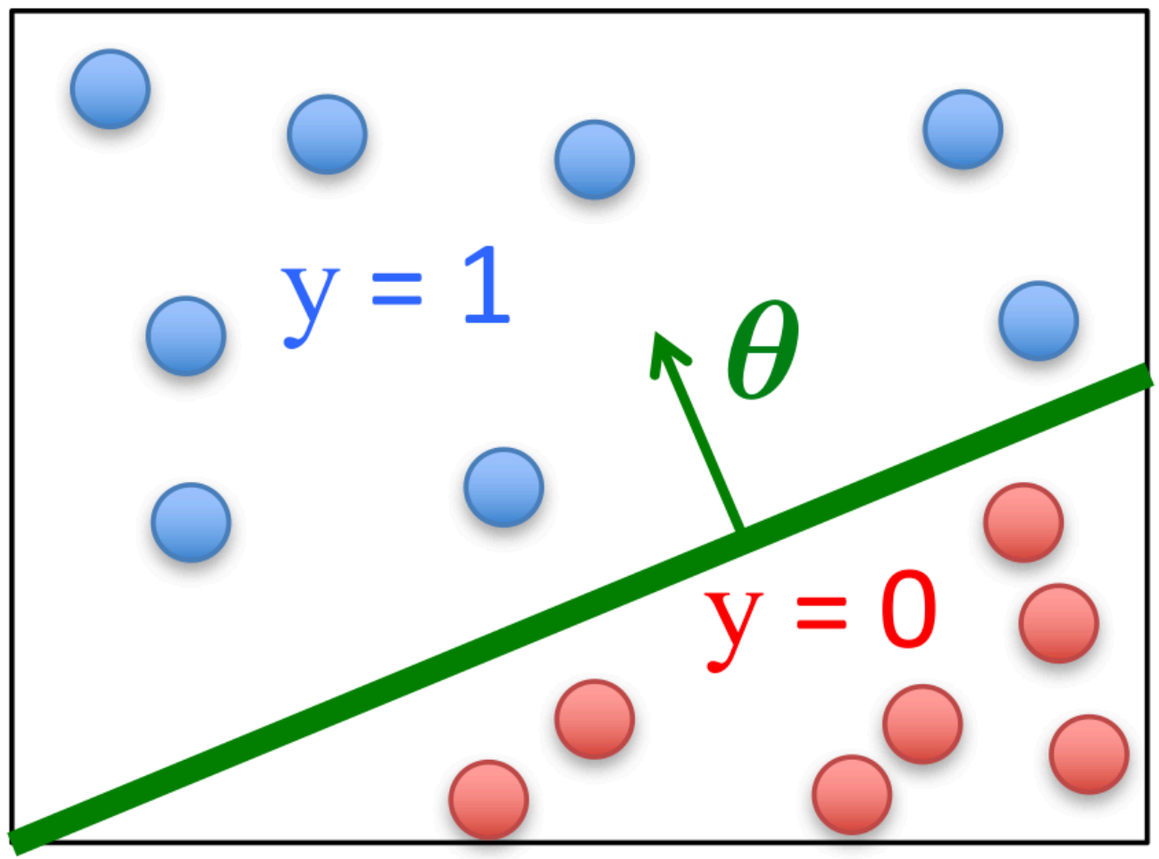
Learning goal

Our model: $p(y|x, \theta) = \sigma(m_\theta(x))$

$m(x)$ should be very negative for negative instances

$m(x)$ should be very positive for positive instances

Select a threshold on $\sigma(m(x))$ to make the decision. Often 0.5.



With loss function is suitable for classification?

Mean squared error is not suitable (already non-convex with a simple logistic regression).

Can we get a maximum likelihood estimate?

Binary Cross-Entropy Loss

$$\log L(\theta) = \sum_i y_i \log \sigma(m(x_i)) + (1 - y_i) \log(1 - \sigma(m(x_i)))$$

Maximizing likelihood is equivalent to minimizing cross entropy:

$$\mathcal{L}_{CE}(\theta) = -\log L(\theta) = -\sum_i [y_i \log \sigma(m(x_i)) + (1 - y_i) \log(1 - \sigma(m(x_i)))]$$

where $y_i \in 0, 1$ gives our estimate for $p(x)$ and $q(x)$ comes from the model.

Relationship between KL Divergence and Cross Entropy

$$D_{KL}(P\|Q) = H(P,Q) - H(P)$$

$$D_{KL}(P\|Q) = \sum_i p(x_i) \log \frac{p(x_i)}{q(x_i)} = -\sum_i p(x_i) \log q(x_i) + \sum_i p(x_i) \log p(x_i)$$

Key message: minimizing cross-entropy over model parameters is the same as minimizing KL divergence, since $H(P)$ is constant with respect to θ.

In practice: X-ent is easier to optimize for than KL-div.

Softmax loss

For multi-class classification with K classes, assume targets follow a Categorical distribution

The model outputs a probability vector via softmax:

$$\sigma(m(x))_k = \frac{e^{m(x)_k}}{\sum_{j=1}^{K} e^{m(x)_j}}$$

The likelihood of a single observation with one-hot label $y_i \in \{0,1\}^K$

$$p(y_i|x_i,\theta) = \prod_{k=1}^{K} \sigma(m(x_i))_k^{y_{ik}}$$

Log-likelihood over all observations:

$$\log L(\theta) = \sum_i \sum_k y_{ik} \log \sigma(m(x_i))_k$$

Negating gives cross-entropy loss:

$$\mathcal{L}_{CE}(\theta) = -\sum_i \sum_k y_{ik} \log \sigma(m(x_i))_k$$

Mean-squared error loss

Assume targets are Gaussian: $y_i \sim \mathcal{N}(m(x_i), \sigma^2)$

$$p(y_i|x_i, \theta) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y_i - m(x_i))^2}{2\sigma^2}\right)$$

Log likelihood over all observations:

$$\log L(\theta) = -\frac{n}{2}\log(2\pi\sigma^2) - \frac{1}{2\sigma^2}\sum_i (y_i - m(x_i))^2$$

Maximizing log likelihood is equivalent to minimizing

$$\mathcal{L}_{MSE} = \sum_i (y_i - m(x_i))^2$$

(because constants can be ignored for minimum)

Keywords

Loss functions

- Information Theory
- Maximum Likelihood Estimation
- Binary/Categorical Cross-entropy
- KL divergence
- Mean squared error

Deep Learning Intuition

- Shortcut Learning & Simplicity Bias
- Lottery Ticket Hypothesis
- Residual connections as ensembles of simple functions
- Deep Double Descent & Grokking

Deep Learning intuition

Shortcut Learning in Deep Neural Networks[1]

1. Geirhos et al., Nature Machine Intelligence 2020 ↩

Shortcut Learning Implications

Deep neural networks tend to find the simplest possible solution, even if it is a shortcut.

Conversely, overparametrization exerts a **simplicity bias**.

The Lottery Ticket Hypothesis[1]

Deep neural networks contain small and sparse subnetworks, that could be trained in isolation to ~same performance as the full network

Depth and overparametrization gives you exponentially many rolls (=tickets), such that it is more likely to find a good subnetwork + initial weights (= a winning ticket).

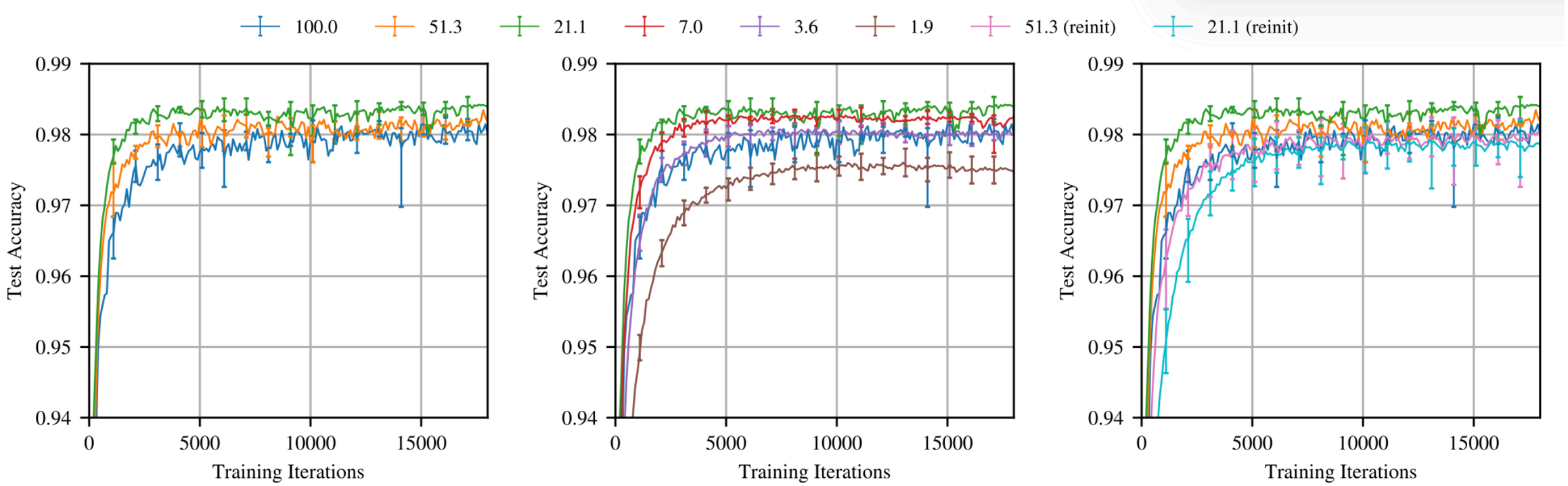
1. Frankle & Carbin, 2019 ICLR ↩

The Lottery Ticket Hypothesis (cont'd)

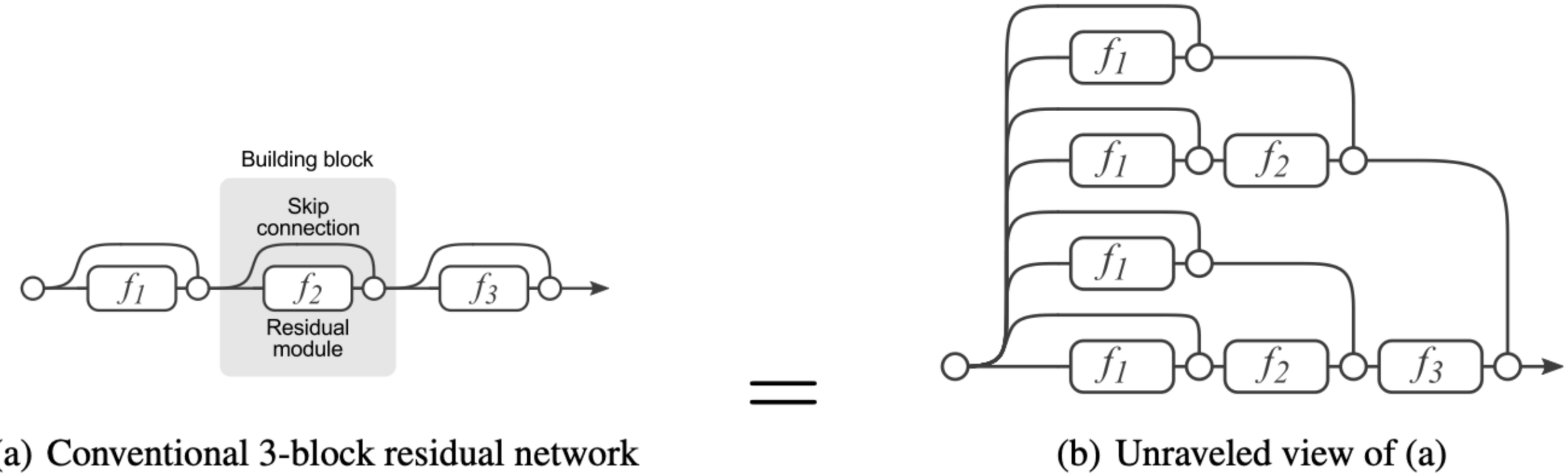
Algorithm for finding winning tickets:

1. Train til convergence.
2. Prune the lowest magnitude weights.
3. Restart with subnetwork@init from Step 1.

The Lottery Ticket Hypothesis (cont'd)



Residual Networks Behave Like Ensembles of Relatively Shallow Networks[1]

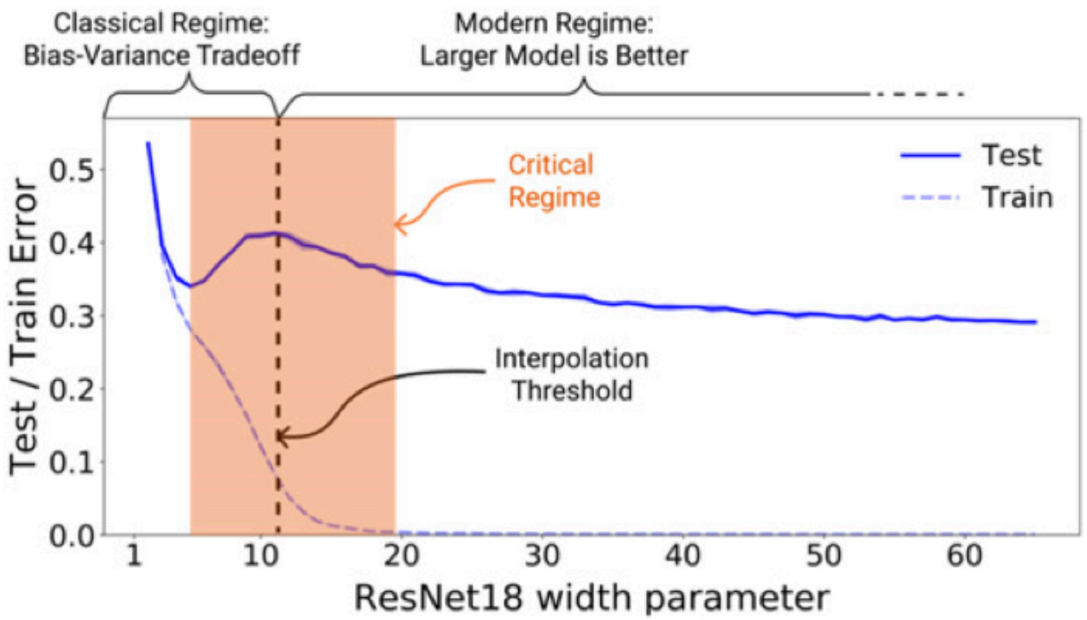


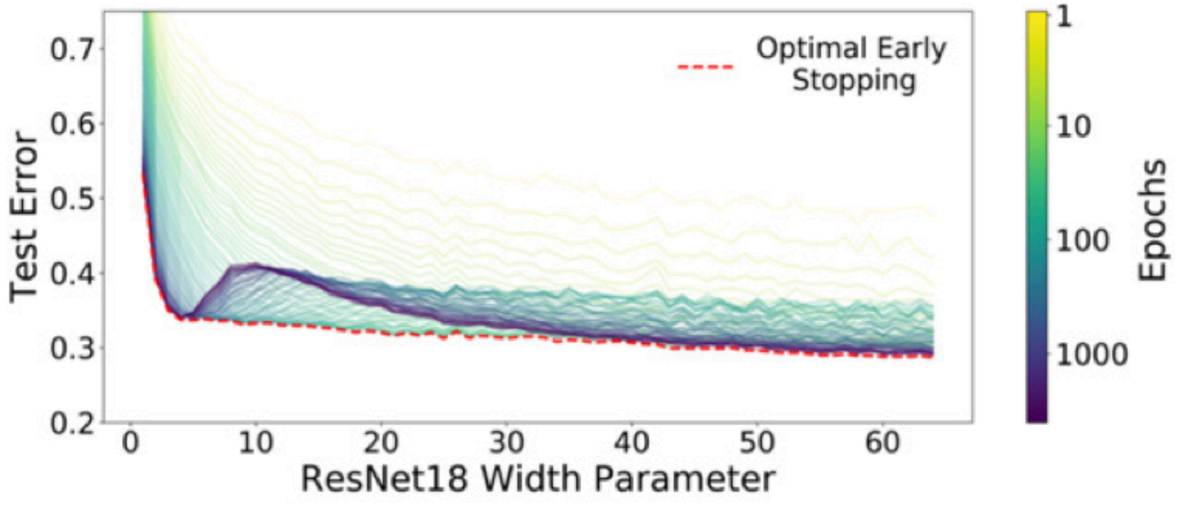
(a) Conventional 3-block residual network

(b) Unraveled view of (a)

1. Veit, Vilber, & Belongie (2016 NeurIPS ↩

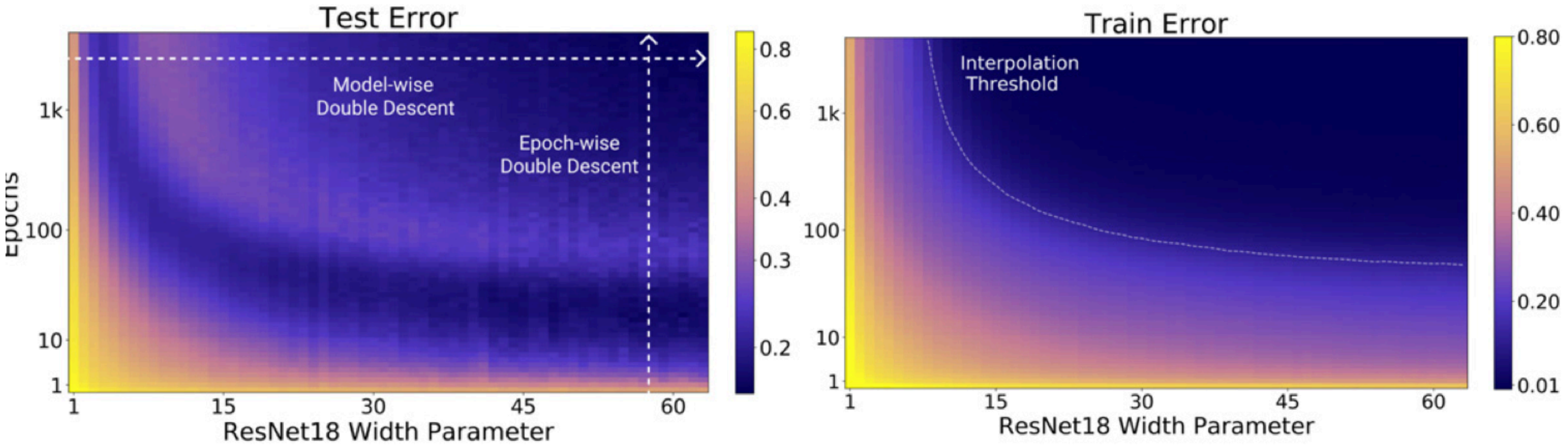
Deep Double Descent[1]





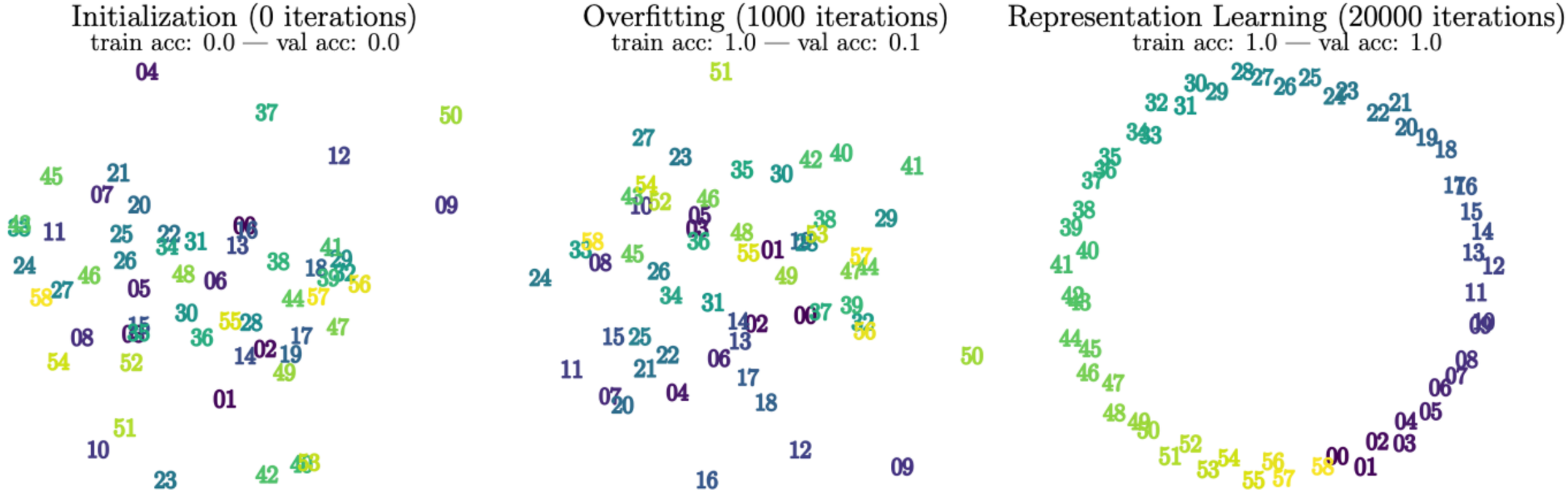
1. Nakkiran et al. (2021) ↩

Deep Double Descent from a different perspective



Grokking[1]

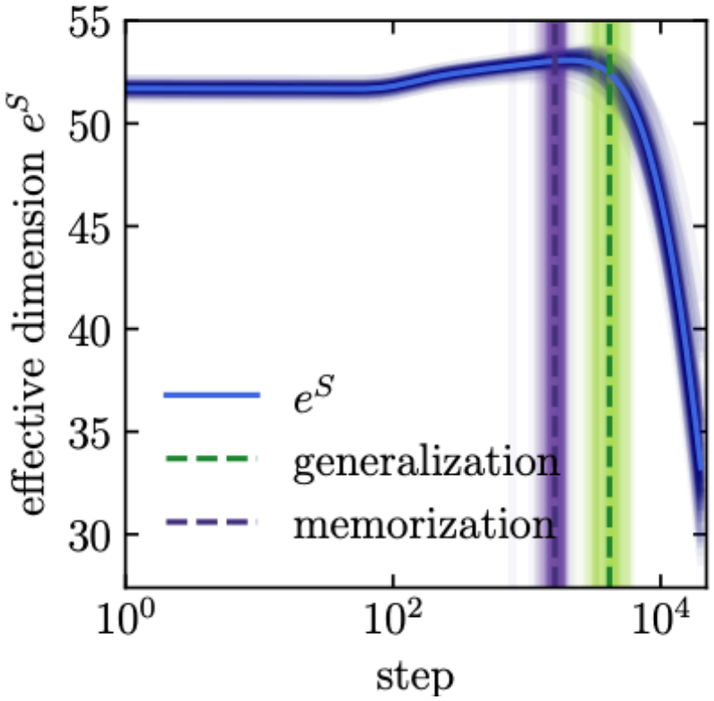
Grokking: a model first memorizes the training data, then — long after — suddenly learns to generalize

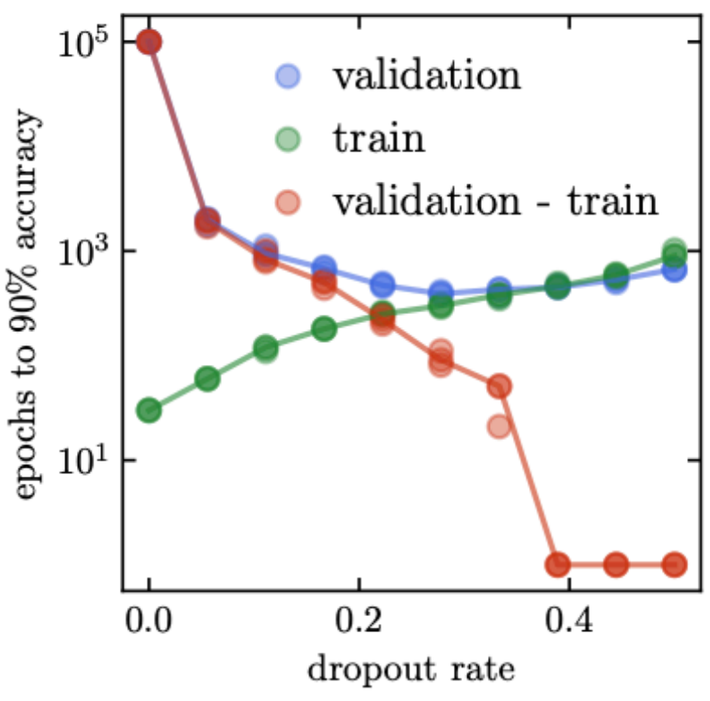


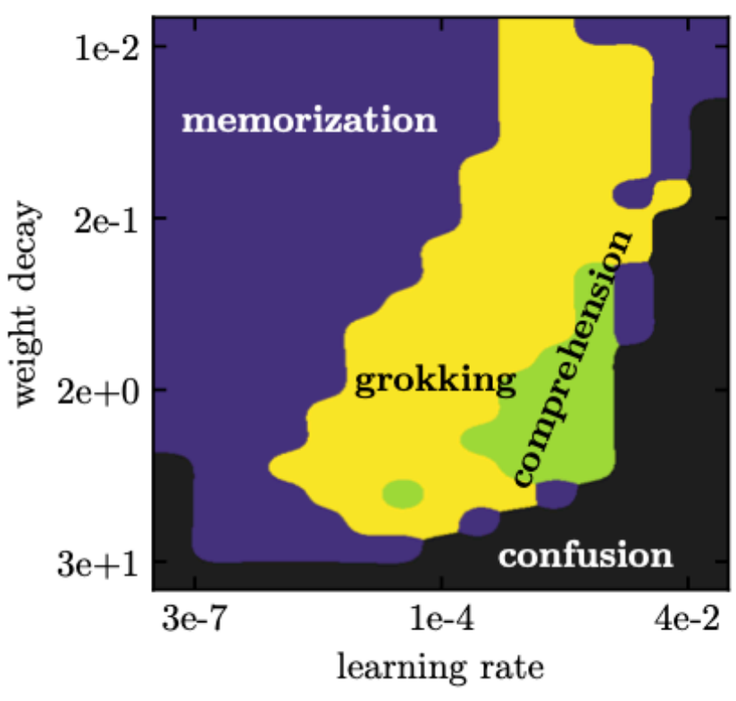
1. Liu et al. (2022 NeurIPS) ↩

More Grokking

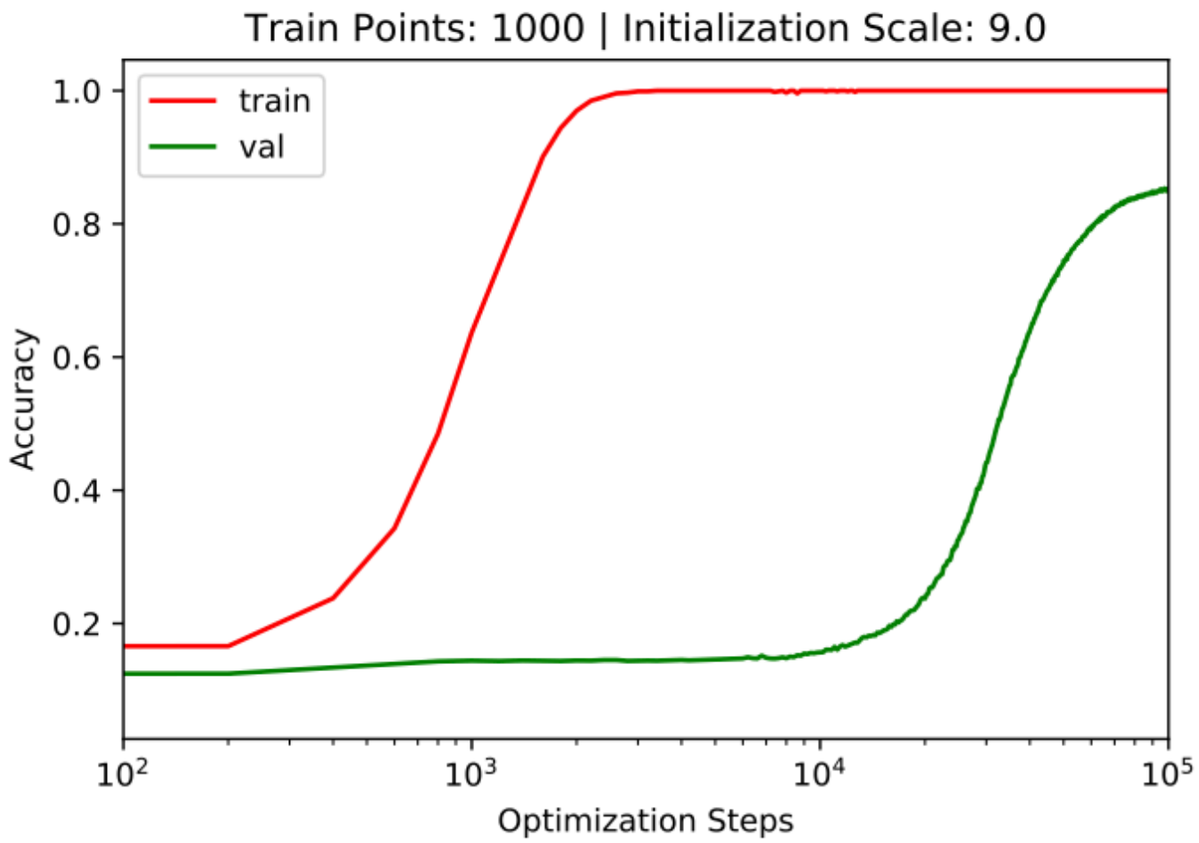
Generalization can happen after overfitting.



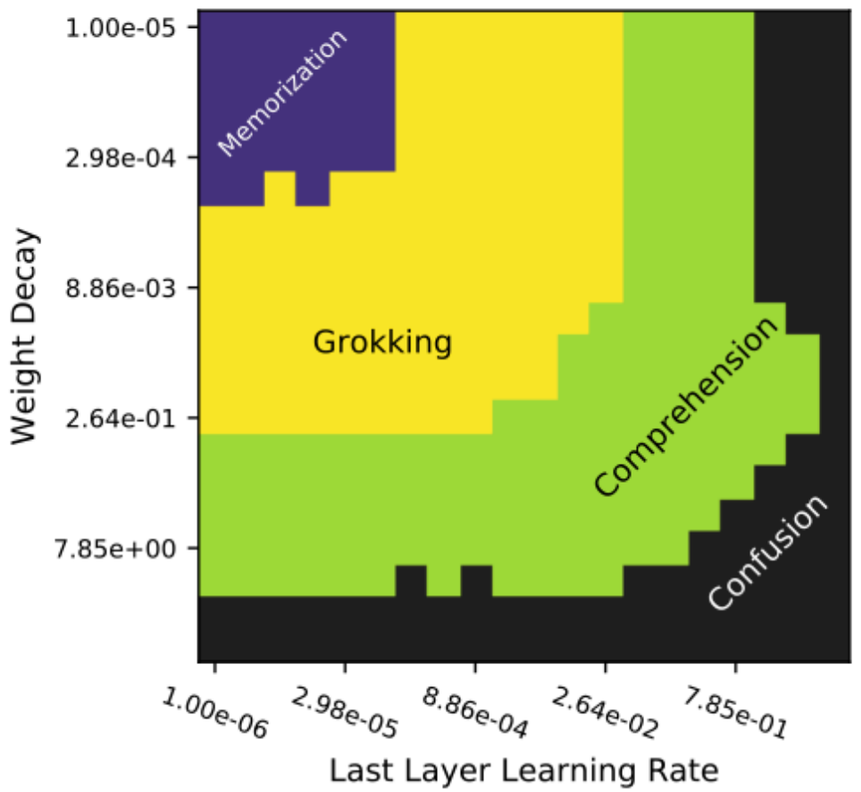




Even More Grokking 2



(a)



(b)

Keywords

Loss functions

- Information Theory
- Maximum Likelihood Estimation
- Binary/Categorical Cross-entropy
- KL divergence
- Mean squared error

Deep Learning Intuition

- Shortcut Learning & Simplicity Bias
- Lottery Ticket Hypothesis
- Residual connections as ensembles of simple functions
- Deep Double Descent & Grokking

How's my teaching?

Department of Mathematics
and Computer Science

sdu.dk

#sdudk

Advanced Machine Learning

Recurrent Neural Networks

Lukas Galke Poech

Spring 2026

SDU

Keywords

- **Word Embeddings**
- Recurrent Neural Networks
- Backpropagation-through-time
- Teacher forcing
- Long-term dependencies
- Long short-term memory
- Gating

About Text

➢Text is one of the most widespread forms of sequence data.

➢Can be interpreted as a sequence of words (or **tokens**).

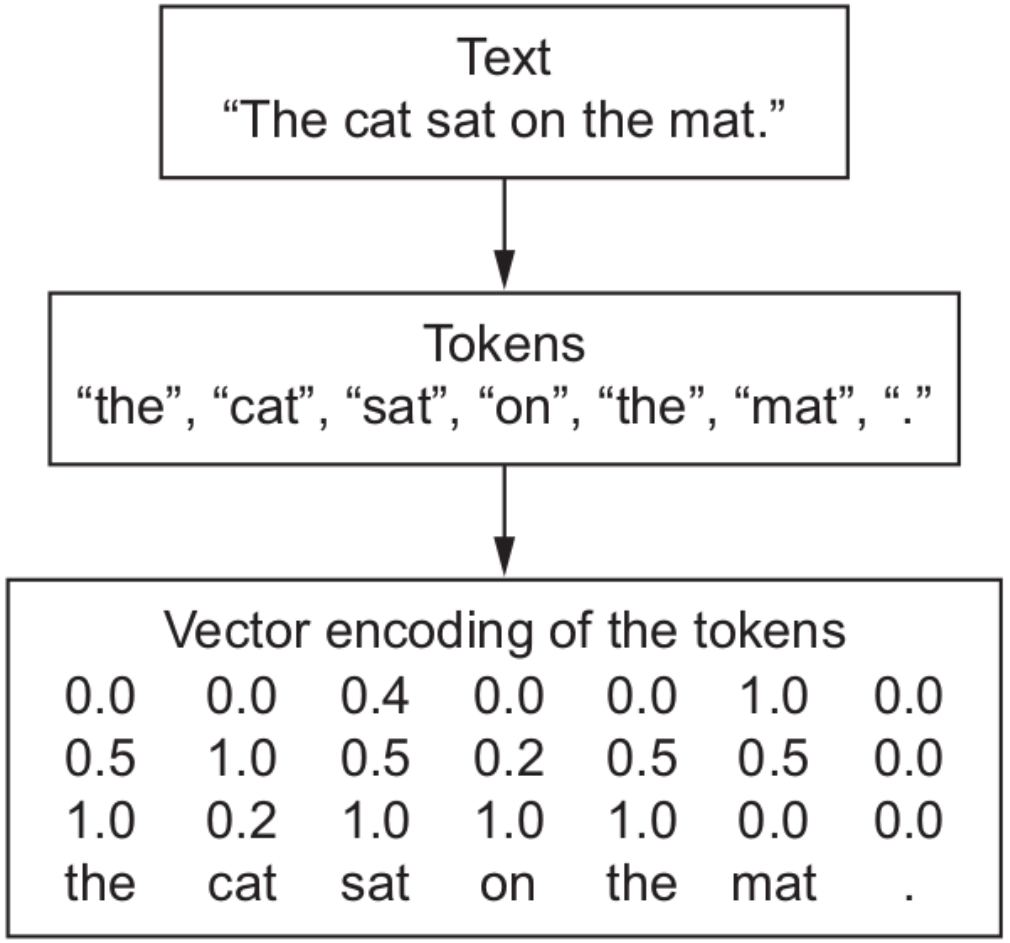
➢Classic NLP tasks: Text categorization, sentiment analysis, translation

➢Large language models as very powerful general-purpose methods

Text as Input

- We cannot feed text directly into networks.
- They only “eat” numerical tensors
- We need to **vectorize** text into numeric tensors
  - **Segment** text into **tokens** (words, subwords, characters)
  - and convert each token into a **vector**.

Example


Text
“The cat sat on the mat.”
Tokens
“the”, “cat”, “sat”, “on”, “the”, “mat”, “.”
Vector encoding of the tokens
0.0 0.0 0.4 0.0 0.0 1.0 0.0
0.5 1.0 0.5 0.2 0.5 0.5 0.0
1.0 0.2 1.0 1.0 1.0 0.0 0.0
the cat sat on the mat .

One-hot Encoding

- Associate a unique integer index with every token
- Turn the integer index **i** into a binary vector of size N (the size of the vocabulary); **the vector is all 0s except for the i_{th} entry, which is 1.**

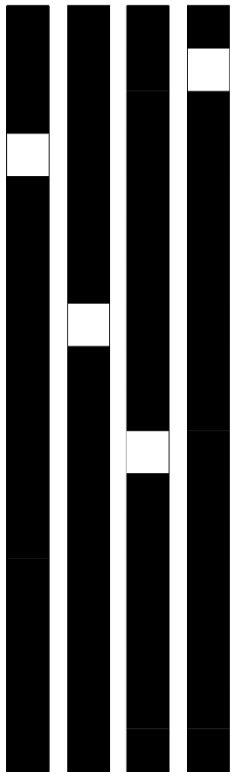
One-hot Encoding Example

- Let's look at the sentences with almost the same meaning:
  - "Have a good day"
  - "Have a great day"
- Using our vocabulary V = {*Have*, *a*, *good*, *great*, *day*} and one-hot embedding, we end with, for instance:
  - *Have* = [1,0,0,0,0]; *a*=[0,1,0,0,0]; *good*=[0,0,1,0,0]; *great*=[0,0,0,1,0]; *day*=[0,0,0,0,1]

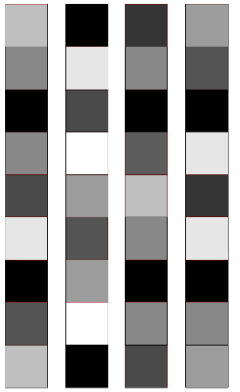
Word Embeddings

- Vectors obtained through one-hot encoding are binary, sparse (mostly made of zeros), and very high-dimensional (same dimensionality as the number of words in the vocabulary)

- Word embeddings are low-dimensional floating-point vectors (that is, dense vectors, as opposed to sparse vectors)



One-hot word vectors:
- Sparse
- High-dimensional
- Hard-coded



Word embeddings:
- Dense
- Lower-dimensional
- Learned from data

How to obtain Embeddings?

- Old way: load pre-computed word embeddings (e.g. word2vec, fastText)
- **Modern way: Learn word embeddings jointly with the main task**
  - Start with random word embeddings and then learn word embeddings in the same way you learn the other weights of the neural network.

Word Embeddings as Look-up Tables

- Segment sequence into tokens
- For each token, look up its index in the vocabulary
- Fetch the row corresponding to the index from the embedding table
- And proceed with the vector found in the embedding table

Embedding Table

[EOS]	[0.2, -0.1, 0.0, 0.5, 0.1, 0.0, 0.0, 0.2, 0.01, 0.0]	**0**
the	[0.2, -0.1, 0.1, -0.5, 0.1, -0.08, 0.04, 0.2, 0.01, -0.3]	**1**
Have	[0.2, -0.1, 0.45, -0.5, 0.1, 0.8, 0.4, 0.2, 0.01, -0.3]	**2**
a	[0.2, -0.1, -0.9, 0.5, 0.1, -0.3, 0.4, -0.4, 0.01, -0.3]	**3**
good	[0.2, -0.1, 0.9, 0.5, 0.1, 0.8, 0.4, 0.2, 0.01, -0.3]	**4**
great	[0.2, -0.1, 0.9, 0.5, 0.1, 1.0, 0.4, 0.2, 0.01, -0.3]	**5**
day	[0.2, -0.1, -1.3, 0.5, 0.1, 0.8, 0.4, 0.2, 0.01, -0.3]	**6**

Word emb:

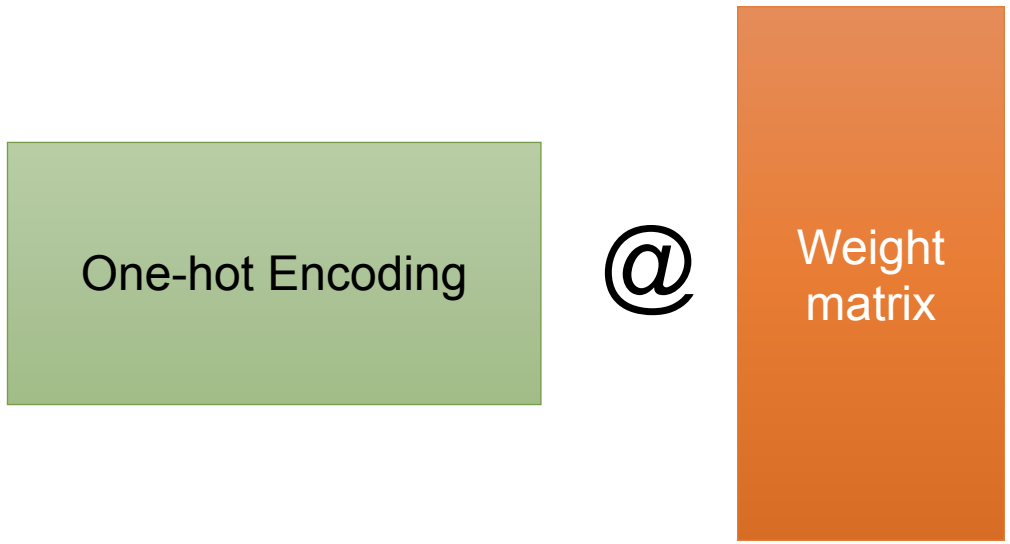
Tokenizer: **[2, 3, 4, 6]**

[“Have”, “a”, “good”, “day”]

Input Sequence: *Have a good day*

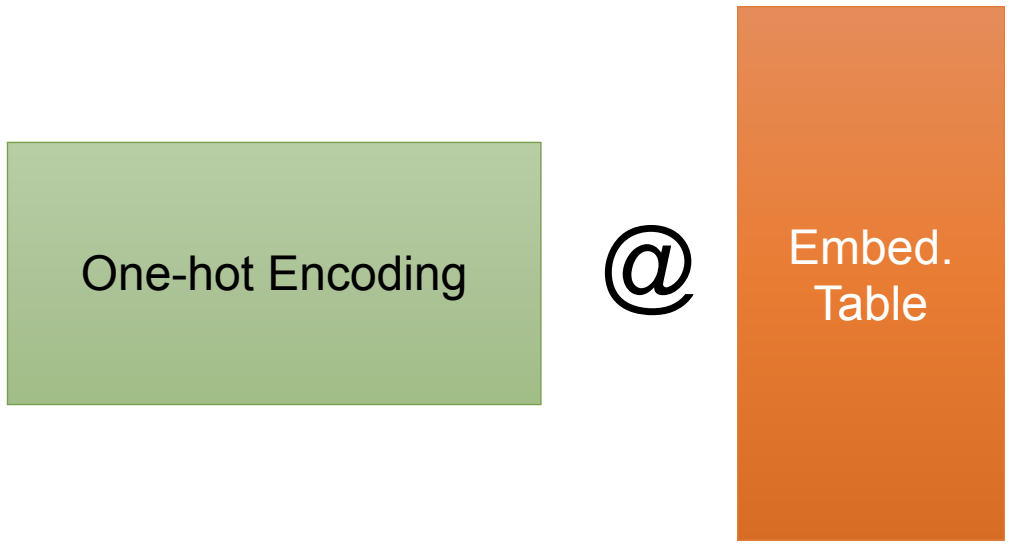
Relationship b/w Word Embeddings and One-Hot Enc.

- One hot encoding **X** of shape [sequence length, vocab_size]
- Then assume we apply a weight layer X @ W, where W is of shape [vocab_size, H]
- One-hot vector **selects a row of W**
- **Equivalent to using an embedding table**



Relationship b/w Word Embeddings and One-Hot Enc.

- One hot encoding **X** of shape [sequence length, vocab_size]
- Then we assume we apply weight layer X*W
- One-hot vector **selects a column of W**
- **Equivalent to using an embedding table**



Why word embeddings?

- Table lookup is **much faster** as it avoids BIG matrix multiplications

Embedding Table

[EOS]	[0.2, -0.1, 0.0, 0.5, 0.1, 0.0, 0.0, 0.2, 0.01, 0.0]	**0**
the	[0.2, -0.1, 0.1, -0.5, 0.1, -0.08, 0.04, 0.2, 0.01, -0.3]	**1**
Have	[0.2, -0.1, 0.45, -0.5, 0.1, 0.8, 0.4, 0.2, 0.01, -0.3]	**2**
a	[0.2, -0.1, -0.9, 0.5, 0.1, -0.3, 0.4, -0.4, 0.01, -0.3]	**3**
good	[0.2, -0.1, 0.9, 0.5, 0.1, 0.8, 0.4, 0.2, 0.01, -0.3]	**4**
great	[0.2, -0.1, 0.9, 0.5, 0.1, 1.0, 0.4, 0.2, 0.01, -0.3]	**5**
day	[0.2, -0.1, -1.3, 0.5, 0.1, 0.8, 0.4, 0.2, 0.01, -0.3]	**6**

Word emb:

Tokenizer: **[2, 3, 4, 6]**

[“Have”, “a”, “good”, “day”]

Input Sequence: *Have a good day*

Keywords

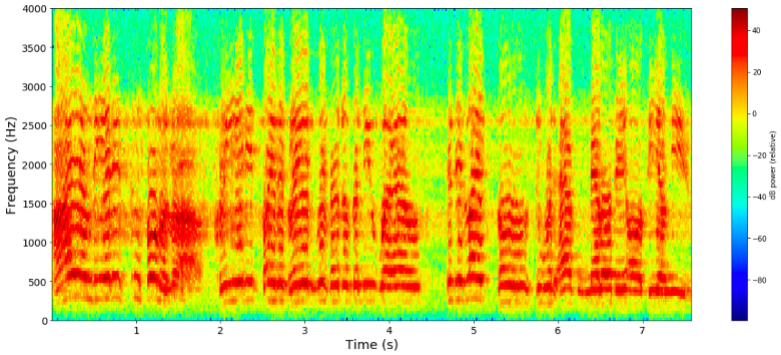
- Word Embeddings
- **Backpropagation-through-time**
- Teacher forcing
- Long-term dependencies
- Long short-term memory
- Gating

RNNs process sequential data

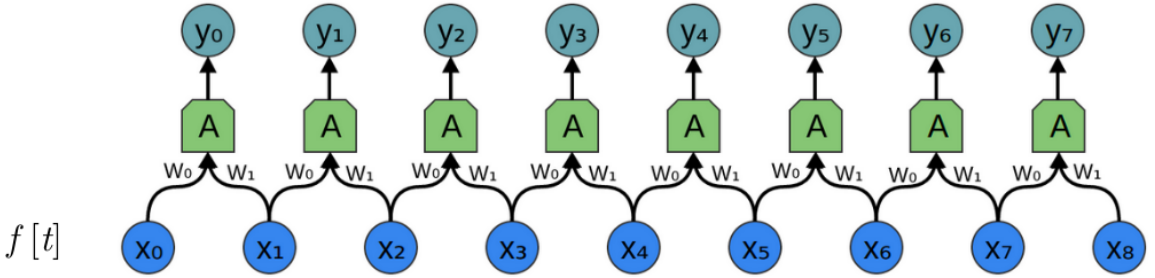
- Recurrent Neural Networks are a family of neural networks for processing sequential data
- Just as CNN is specialized for processing grid of values, e.g., image
  - RNN is specialized for processing a sequence of values $x^{(1)}, \ldots, x^{(\tau)}$
- Just as CNNs can readily scale images with large width/height and process variable size images
  - RNNs can scale to much longer sequences than would be practical for networks without sequence-based specialization
  - RNNs can also process variable-length sequences

Examples of Sequential Data and Tasks

- Sequence data: sentences, speech, stock market, signal data
- Sequence-to-sequence Tasks
  - Speech recognition
    - decompose sound waves into frequency and amplitude using Fourier transforms yielding a spectrograms
  - Named Entity Recognition
    - Input: Jim bought 300 shares of Acme Corp. in 2006
    - NER: $[\text{Jim}]_{\text{Person}}$ bought 300 shares of $[\text{Acme Corp.}]_{\text{Organization}}$ in $[\text{2006}]_{\text{Time}}$
- Sequence-to-symbol
  - Sentiment
  - Speaker recognition



Recall: 1-D Convolution



Kernel $g(t)$:
$[\ldots 0, w_1, w_0, 0 \ldots]$

Equations for outputs of this network:

$$y_0 = \sigma(W_0 x_0 + W_1 x_1 - b)$$

$$y_1 = \sigma(W_0 x_1 + W_1 x_2 - b)$$

etc. upto y_8

Note that kernel gets flipped in convolution

We can also write the equations in terms of elements of a general 8×8 weight matrix W as:

$$y_0 = \sigma(W_{0,0} x_0 + W_{0,1} x_1 + W_{0,2} x_2 \ldots)$$

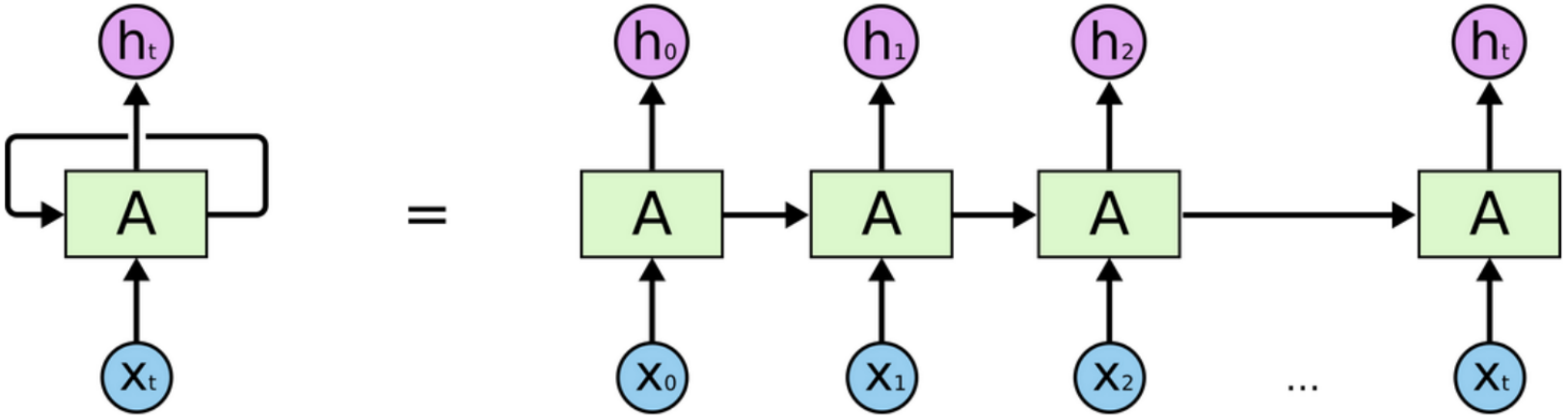
$$y_1 = \sigma(W_{1,0} x_0 + W_{1,1} x_1 + W_{1,2} x_2 \ldots)$$

where $W = \begin{bmatrix} w_0 & w_1 & 0 & 0 & \ldots \\ 0 & w_0 & w_1 & 0 & \ldots \\ 0 & 0 & w_0 & w_1 & \ldots \\ 0 & 0 & 0 & w_0 & \ldots \\ \ldots & \ldots & \ldots & \ldots & \ldots \end{bmatrix}$

➢ http://colah.github.io/posts/2014-07-Understanding-Convolutions/

Recurrent neural networks

- RNNs share parameters in a different way
  - Each member of output is a function of previous members of output
  - Each output produced using same update rule applied to previous outputs
  - This recurrent formulation results in sharing of parameters through a very deep computational graph
- An unrolled RNN



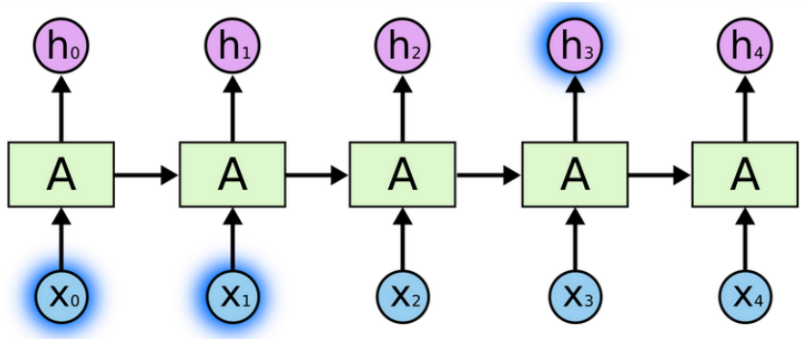
RNNs share same weights across Time Step

- To go from multi-layer networks to RNNs:
  - Need to share parameters across different parts of a model
  - Separate parameters for each value of input cannot generalize to sequence lengths not seen during training
  - Share statistical strength across different sequence lengths and across different positions in time

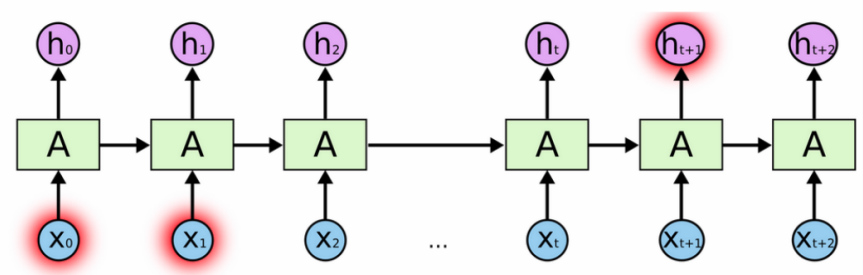
- Sharing important when information can occur at multiple positions in the sequence
  - Given “I went to Nepal in 1999 ” and “In 1999, I went to Nepal ”, an ML method to extract year, should extract 1999 whether in position 6 or 2
  - A feed-forward network that processes sentences of fixed length would have to learn all of the rules of language separately at each position
  - An RNN shares the same weights across several time steps

Problem of Long-Term Dependencies

- Easy to predict last word in “the clouds are in the **sky**”
  - When gap between relevant information and place that it’s needed is small, RNNs can learn to use the past information



- “I grew up in France...[an entire story]...
  I speak fluent **French**.”
  - Large gap between relevant information and point where it is needed



- In principle RNNs can handle it, but fail in practice
  - LSTMs offer a solution

RNN operating on a sequence

- RNNs operate on a sequence that contain vector $x^{(t)}$ with time step index t, ranging from 1 to τ
  - Sequence $x^{(1)}, \ldots, x^{(\tau)}$
  - RNNs operate on minibatches of sequences of length τ

- Some remarks about sequences
  - The steps need not refer to passage of time in the real world
  - RNNs can be applied in two-dimensions across spatial data such as image
  - Even when applied to time sequences, network may have connections going backwards in time, provided entire sequence is observed before it is provided to network

Keywords

- **Word Embeddings**
- Recurrent Neural Networks
- **Backpropagation-through-time**
- Teacher forcing
- Long-term dependencies
- Long short-term memory
- Gating

Unfolding Computational Graphs

- Recall: A Computational Graph is a way to formalize the structure of a set of computations
  - Such as mapping inputs and parameters to outputs and loss

- We can unfold a recursive or recurrent computation into a computational graph that has a repetitive structure
  - Corresponding to a chain of events

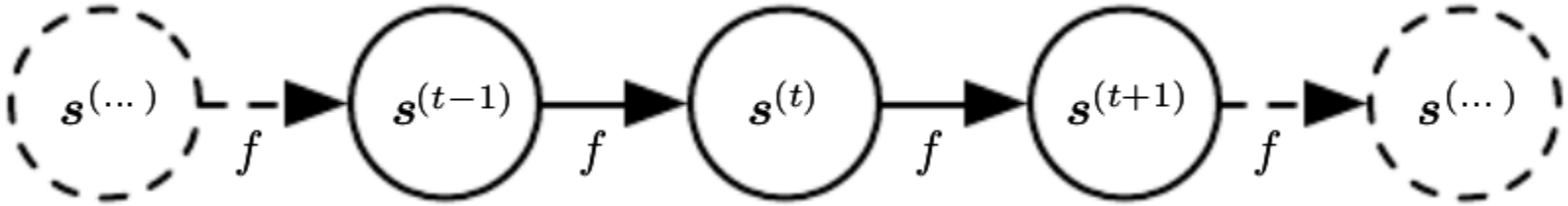
- Unfolding this graph results in sharing of parameters across a deep network structure

Unfolded dynamical system

➢ The classical dynamical system described by

$$\boldsymbol{s}^{(t)} = f\left(\boldsymbol{s}^{(t-1)}; \boldsymbol{\theta}\right)$$

is illustrated as an unfolded computational graph:



➢ Each node represents state at some time t

➢ Function f maps state at time t to the state at $t + 1$

➢ The same parameters (the same value of θ used to parameterize f) are used for all time steps

Dynamical system driven by external signal

➢ As another example, consider a dynamical system driven by external (input) signal $\boldsymbol{x}^{(t)}$

$$\boldsymbol{s}^{(t)} = f\left(\boldsymbol{s}^{(t-1)}, \boldsymbol{x}^{(t)}; \boldsymbol{\theta}\right)$$

- ➢ State now contains information about the whole past input sequence

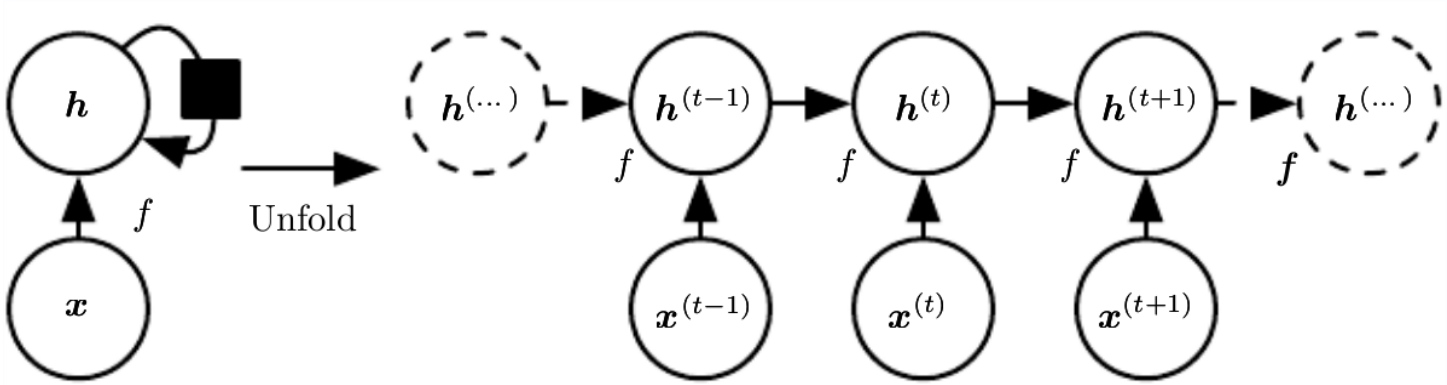
➢ Recurrent neural networks can be built in many ways

- ➢ Much as (almost) any function can be modeled by a feedforward neural network, any function involving recurrence can be considered to be modeled by a recurrent neural network

Defining values of hidden units in RNNs

- Many recurrent neural nets use same equation (as dynamical system with external input) to define values of hidden units
  - To indicate that the state is hidden rewrite using variable $\boldsymbol{h}$ for state

$$\boldsymbol{h}^{(t)} = f\left(\boldsymbol{h}^{(t-1)}, \boldsymbol{x}^{(t)}; \boldsymbol{\theta}\right)$$



- Typical RNNs have extra architectural features such as output layers that read information out of state $\boldsymbol{h}$ to make predictions

Predicting the Future from the Past

- When RNN is required to perform a task of predicting the future from the past, network typically learns to use $\boldsymbol{h}^{(t)}$ as a lossy summary of the past sequence of inputs up to time point t
- The summary is in general lossy since it maps a sequence of arbitrary length $(\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(t)})$ to a fixed length vector $\mathbf{h}^{(t)}$
- Depending on criterion, summary keeps some aspects of past sequence more precisely than other aspects
  - RNN used in statistical language modeling, typically to predict next word from past words
  - it may not be necessary to store all information up to time t but only enough information to predict rest of sentence

Unfolding process allows learning a single model

- The unfolding process introduces two major advantages:
  1. Regardless of sequence length, learned model has same input size
     - because it is specified in terms of transition from one state to another state rather than specified in terms of a variable length history of states
  2. Possible to use same function f with same parameters at every step

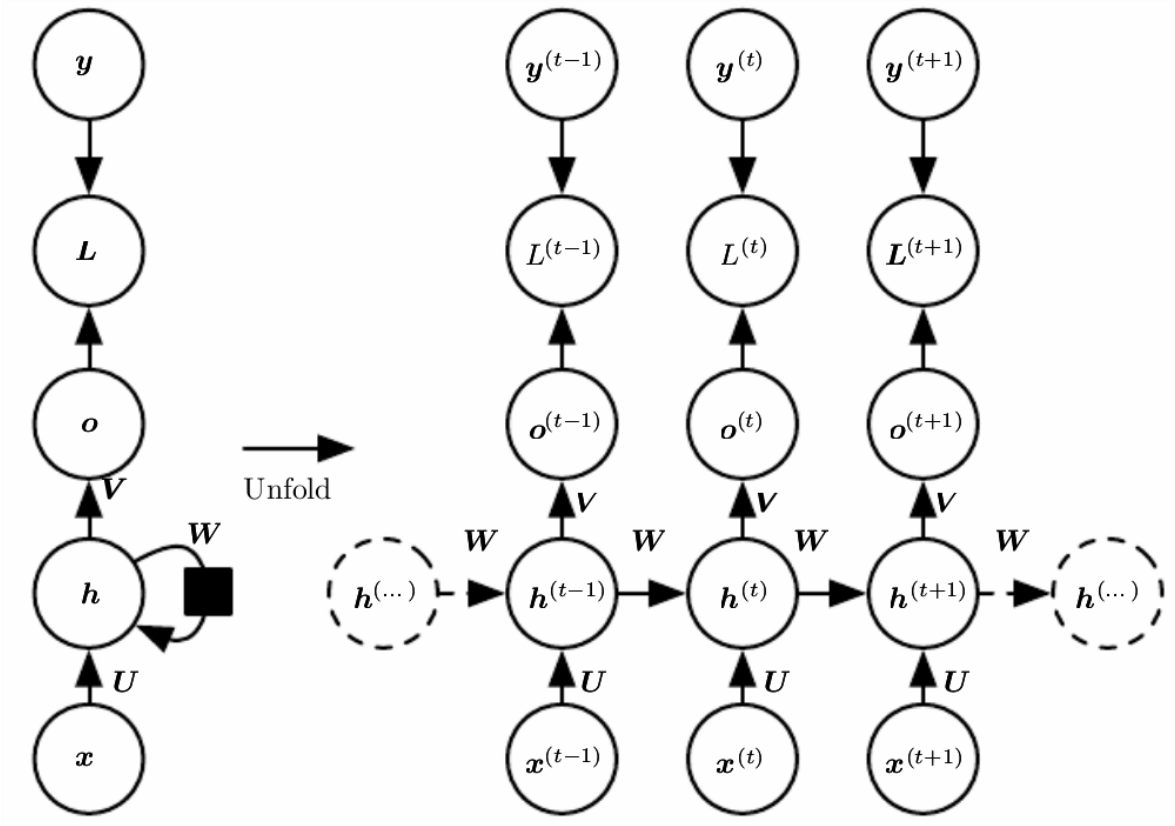
- These two factors make it possible to learn a single model f
  - that operates on all time steps and all sequence lengths
  - rather than needing separate model $g^{(t)}$ for all possible time steps
- Learning a single shared model allows:
  - Generalization to sequence lengths that did not appear in the training
  - Allows model to be estimated with far fewer training examples than would be required without parameter sharing

Three design patterns of RNNs

- Design 1: Output: each time step; Recurrence: hidden units
- Design 2: Output: each time step; Recurrence: output units
- Design 3: Output: one at the end; Recurrence: hidden units

Three design patterns of RNNs

- **Design 1: Output: each time step; Recurrence: hidden units**
- Design 2: Output: each time step; Recurrence: output units
- Design 3: Output: one at the end; Recurrence: hidden units

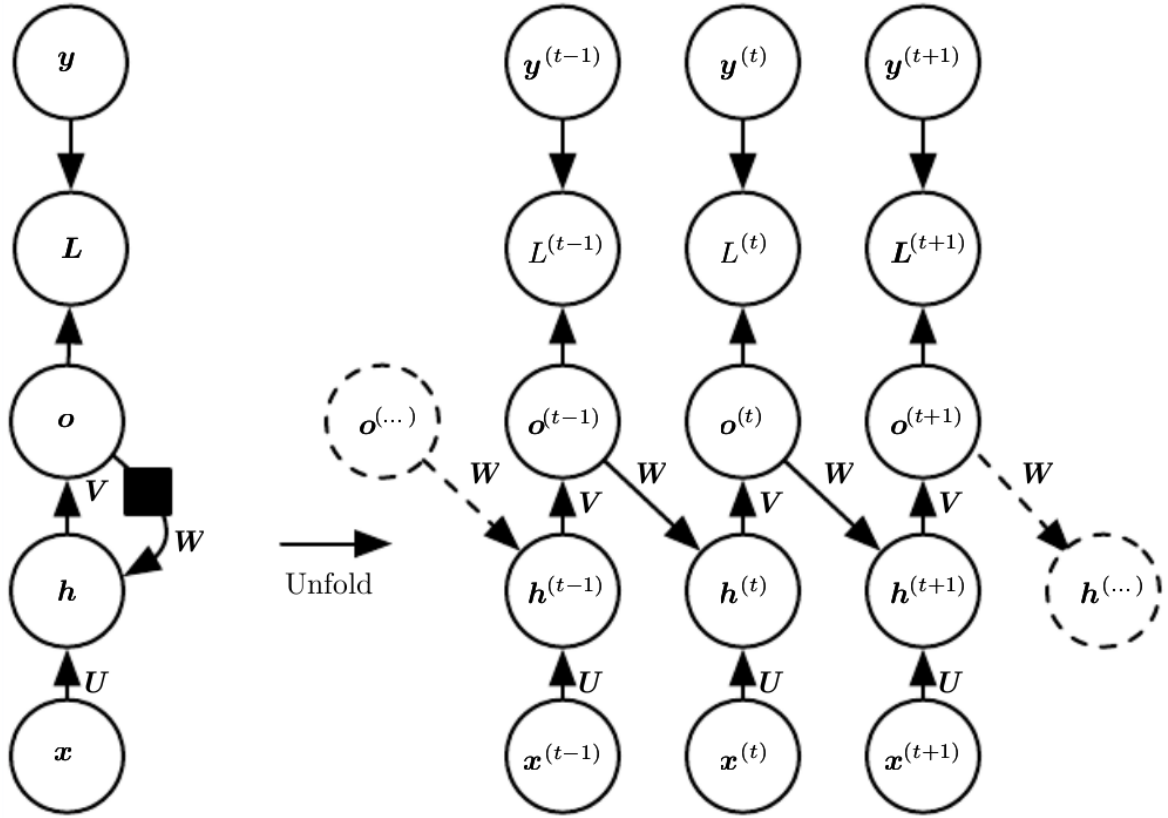


Three design patterns of RNNs

- Design 1: Output: each time step;
  Recurrence: hidden units

- **Design 2: Output: each time step;**
  Recurrence: output units

- Design 3: Output: one at the end;
  Recurrence: hidden units

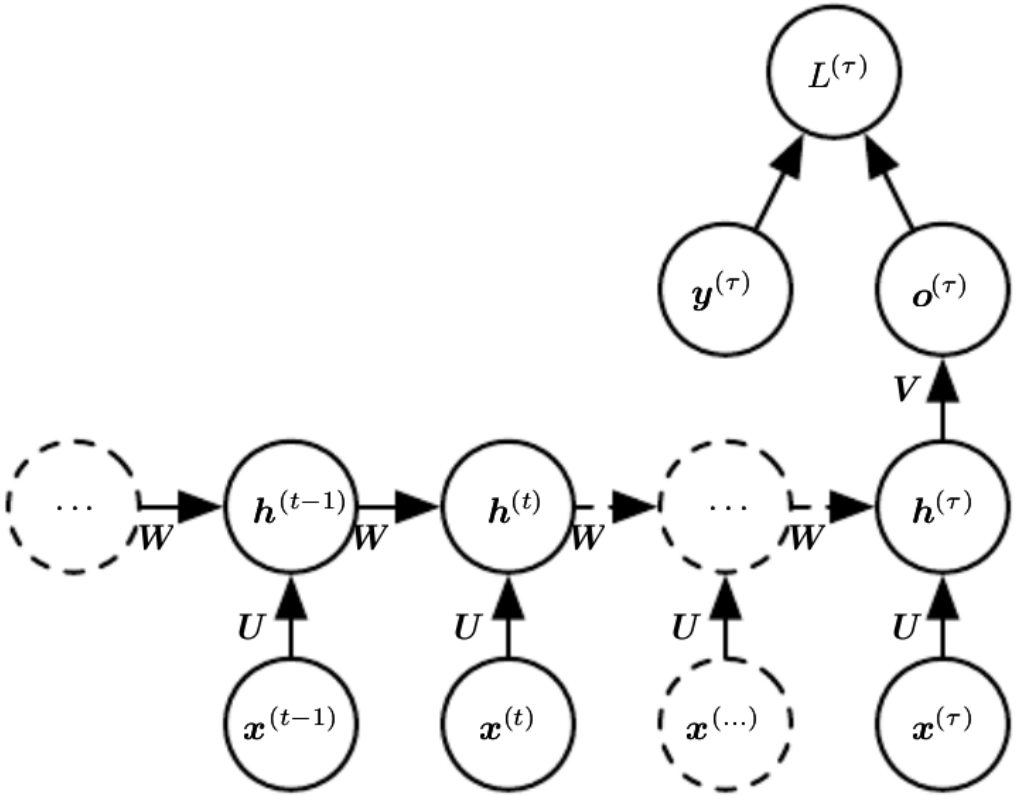


Three design patterns of RNNs

- Design 1: Output: each time step;
  Recurrence: hidden units

- Design 2: Output: each time step;
  Recurrence: output units

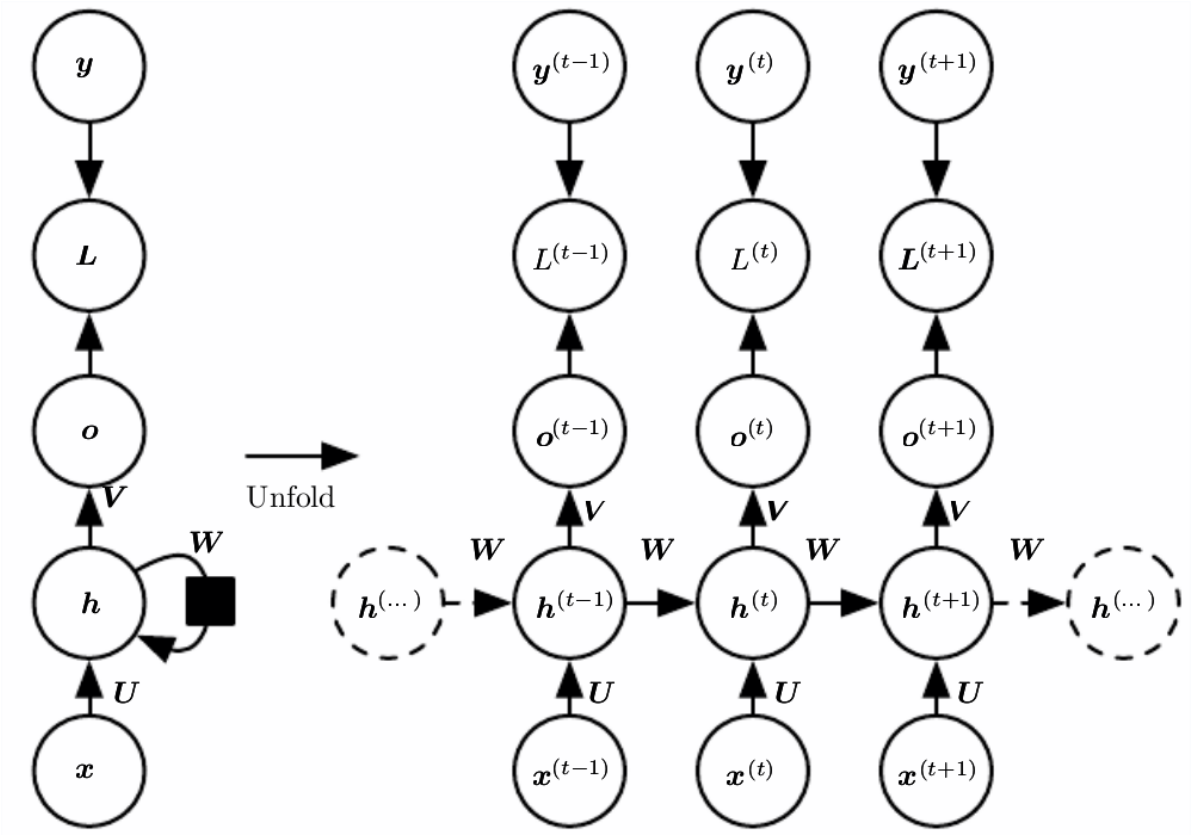
- **Design 3: Output: one at the end;**
  Recurrence: hidden units



Design 1: RNN with recurrence between hidden units

- Maps input sequence $\boldsymbol{x}$ to output $\boldsymbol{o}$ values
  - Loss L measures how far each $\boldsymbol{o}$ is from the corresponding target $\boldsymbol{y}$
  - With softmax outputs we assume $\boldsymbol{o}$ is the unnormalized log probabilities
  - Loss L internally computes $\boldsymbol{y} = \text{softmax}(\boldsymbol{o})$ and compares to target y
  - Let's assume we have tanh activation function

- Any function computable by a Turing Machine is computable by such an RNN of finite size



Design 1: Forward Calculation

➢ We need to specify initial state $\boldsymbol{h}^{(0)}$, then for each time point:

$$\boldsymbol{a}^{(t)} = \boldsymbol{b} + \boldsymbol{W}\boldsymbol{h}^{(t-1)} + \boldsymbol{U}\boldsymbol{x}^{(t)}$$

$$\boldsymbol{h}^{(t)} = \tanh\left(\boldsymbol{a}^{(t)}\right)$$

$$\boldsymbol{o}^{(t)} = \boldsymbol{c} + \boldsymbol{V}\boldsymbol{h}^{(t)}$$

$$\hat{\boldsymbol{y}}^{(t)} = \mathrm{softmax}\left(\boldsymbol{o}^{(t)}\right)$$

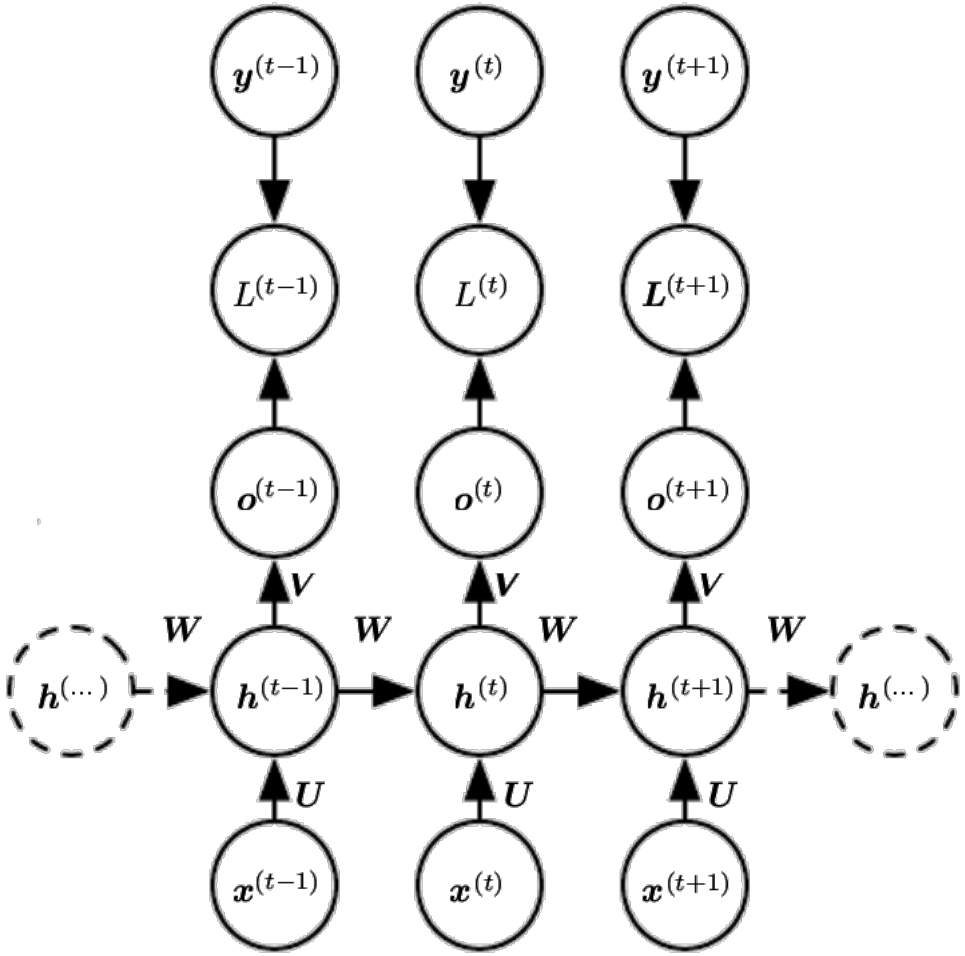
➢ The Loss function can simply defined as the sum of the loss at each timepoint:

$$L = \sum_t L^{(t)} = -\sum_t \log p_{model}\left(\boldsymbol{y}^{(t)} \mid \left\{\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(t)}\right\}\right)$$

Design 1: Gradient Calculation

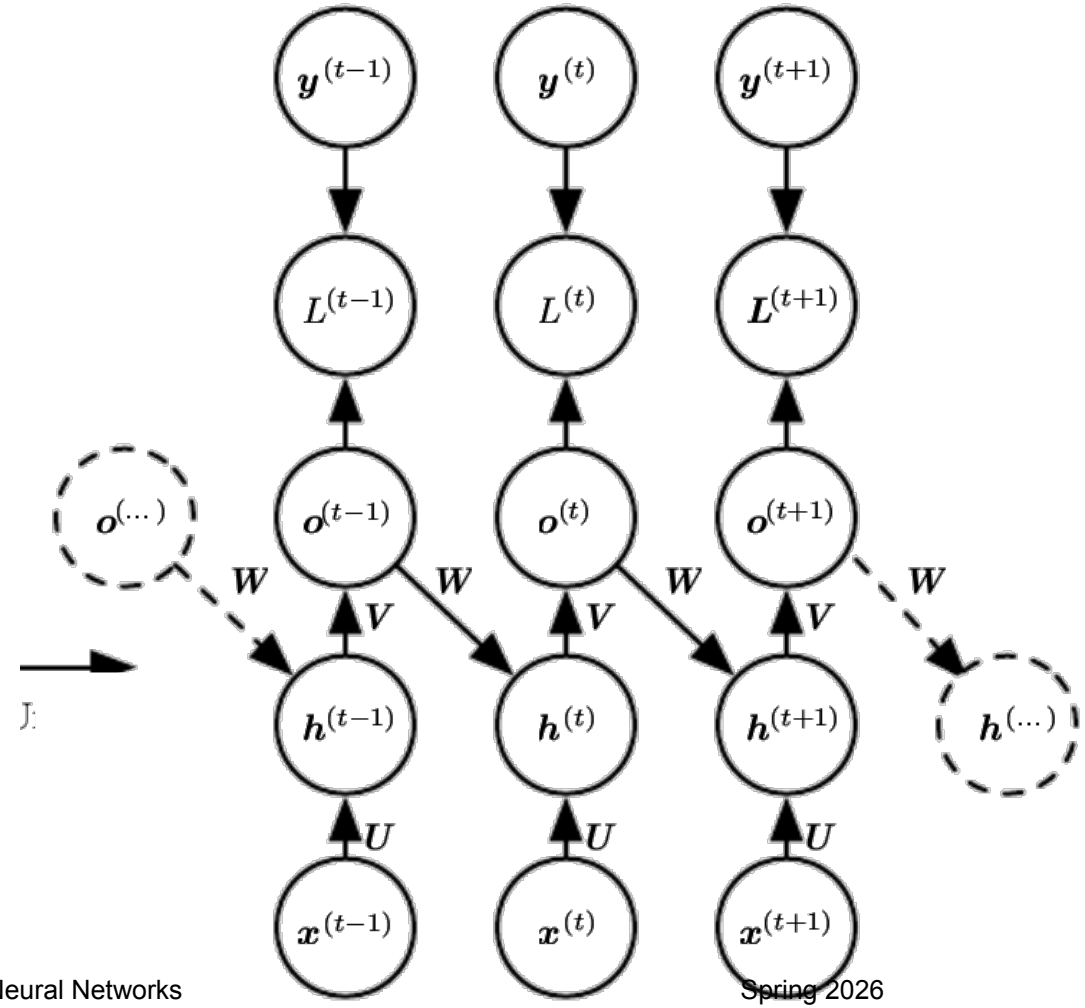
- Computing the gradient of this loss function with respect to the parameters is an expensive operation.
  - Requires forward propagation pass through the unrolled graph
  - followed by a backward pass
  - The runtime is $O(\tau)$ and cannot be reduced by parallelization (each time step may only be computed after the previous one)
  - The forward pass must be stored until they are reused during the backward pass, so the memory cost is also $O(\tau)$.

- The back-propagation algorithm applied to the unrolled graph with $O(\tau)$ cost is called **back-propagation through time** or **BPTT**



Design 2: Recurrence from Output to Hidden

- Less powerful than Design 1
  - It cannot simulate a universal Turing Machine
  - It requires that the output capture all information of past to predict future
- Advantage
  - In comparing loss function to output all time steps are decoupled
    - Each step can be trained in isolation
  - Training can be parallelized
    - Gradient for each step can be computed in isolation
    - No need to compute output for the previous step first, because training set provides ideal value of output
- Trained with **Teacher Forcing**



Keywords

- Word Embeddings
- Recurrent Neural Networks
- Backpropagation-through-time
- **Teacher forcing**
- Long-term dependencies
- Long short-term memory
- Gating

Teacher Forcing

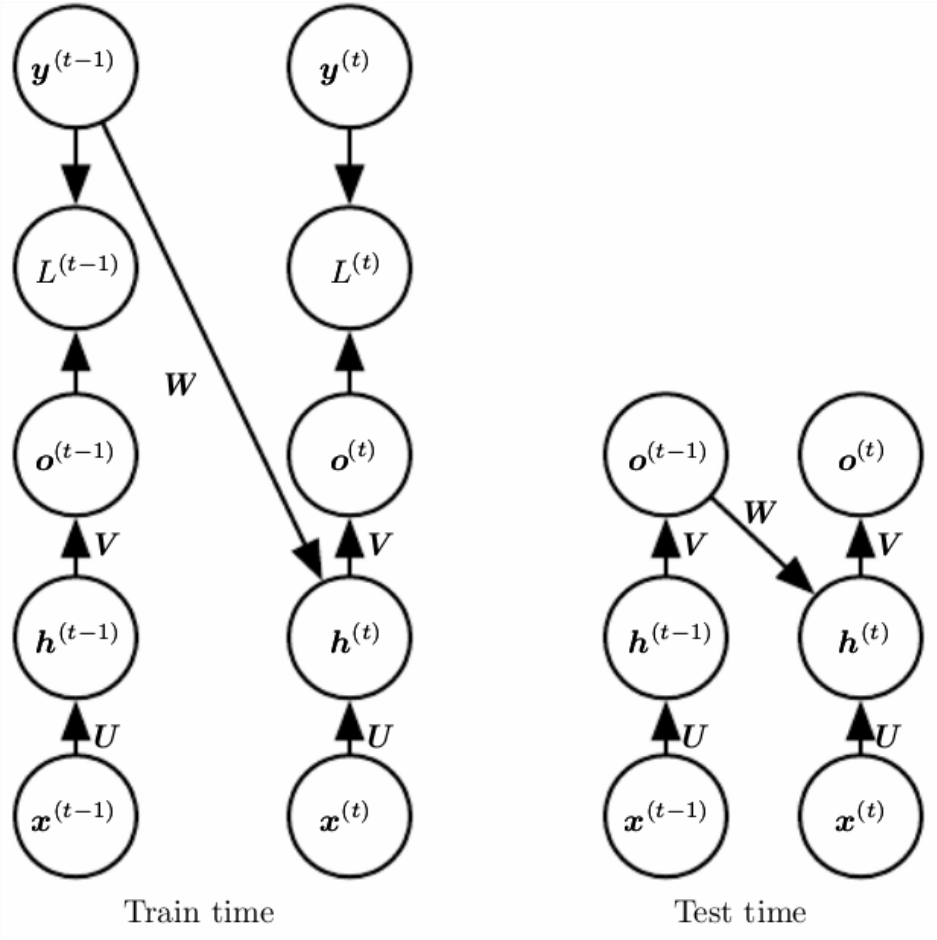
- Teacher forcing during training means:
  - Instead of using model's own previous output (possibly erroneous)
  - We use the ground truth as input for the next iteration

- This means we assume that the previous step has produced the correct answer already.

- Since the correct values are already know, we can train each time-step in parallel and independently

Illustration of Teacher Forcing

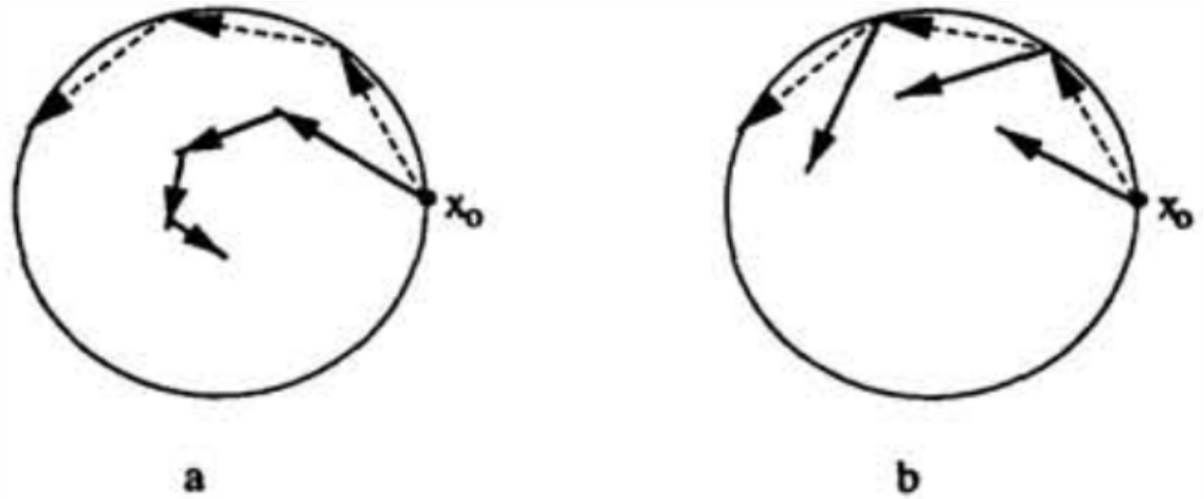
Train time:
We feed the correct output $\boldsymbol{y}^{(t)}$ (from teacher) drawn from the training set as input to $\boldsymbol{h}^{(t+1)}$



Test time:
True output is not known.
We approximate the correct output $\boldsymbol{y}^{(t)}$ with the model's output $\boldsymbol{o}^{(t)}$ and feed the output back to the model

Visualizing Teacher Forcing

- Imagine that the network is learning to follow a trajectory
- It goes astray (because the weights are wrong) but teacher forcing puts the net back on its trajectory

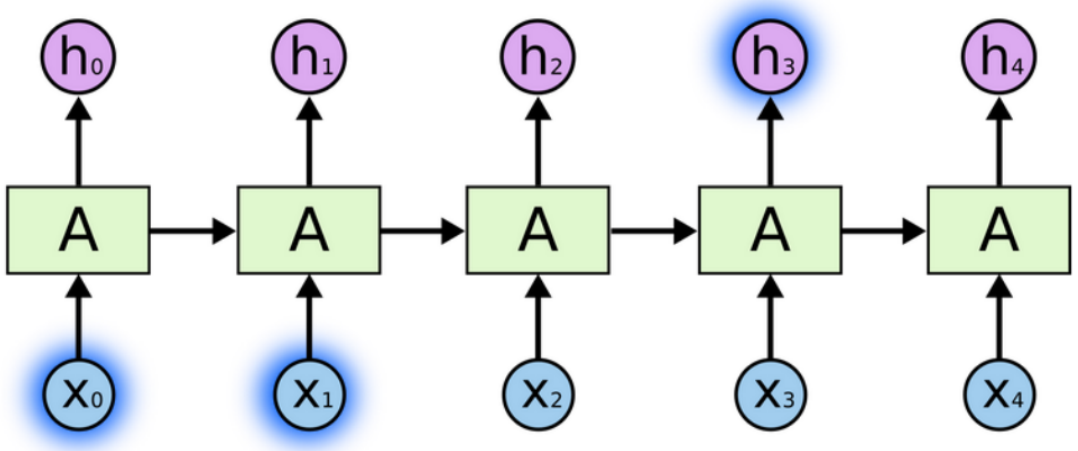


Keywords

- Word Embeddings
- Recurrent Neural Networks
- Backpropagation-through-time
- Teacher forcing
- **Long-term dependencies**
- Long short-term memory
- Gating

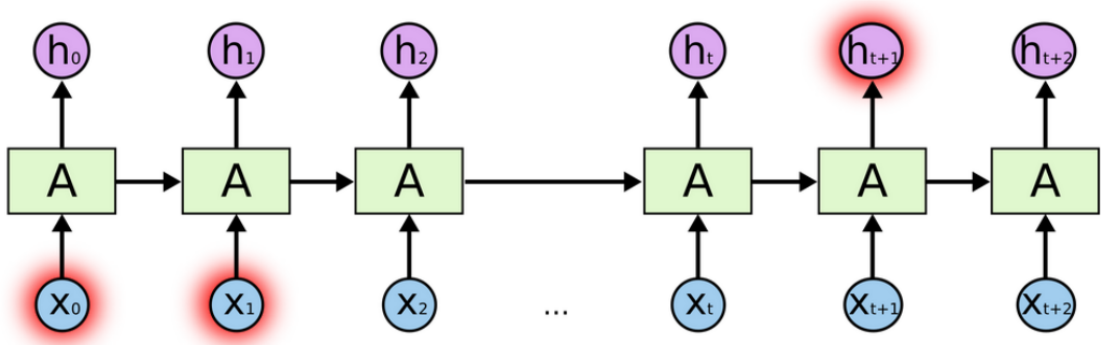
Prediction with only recent previous information

- RNNs connect previous information to the present task
  - Previous video frames may help understand present frame
- Sometimes we need only look at recent previous information to predict
  - To predict the last word of “The clouds are in the sky” we don’t need any further context. It is obvious that the word is “sky”



Long-term Connections not Possible

- here are cases where we need more context
- To predict the last word in the sentence “I grew up in France....I speak fluent French”
  - Using only recent information suggests that the last word is the name of a language. But more distant past indicates that it is French
  - It is possible that the gap between the relevant information and where it is needed is very large



Challenge of Long-Term Dependencies

- Neural network optimization face a difficulty when computational graphs become deep, e.g.,
  - Feedforward networks with many layers
  - RNNs that repeatedly apply the same operation at each time step of a long temporal sequence

- Gradients propagated over many stages tend to either vanish (most of the time) or explode (damaging optimization)

- The difficulty with long-term dependencies arise from exponentially smaller weights given to long-term interactions (back-propagating the error signal involves many multiplications)

Vanishing and Exploding Gradient Problem

- Suppose a computational graph consists of repeatedly multiplying by a matrix $\boldsymbol{W}$
- After t steps this is equivalent to multiplying by $\boldsymbol{W}^t$
- Suppose W has an eigendecomposition $\boldsymbol{W} = \boldsymbol{V}\text{diag}\left(\boldsymbol{\lambda}\right)\boldsymbol{V}^{-1}$:

$$\boldsymbol{W}^t = \left(\boldsymbol{V}\text{diag}(\boldsymbol{\lambda})\boldsymbol{V}^{-1}\right)^t = \boldsymbol{V}\text{diag}\left(\boldsymbol{\lambda}\right)^t\boldsymbol{V}^{-1}$$

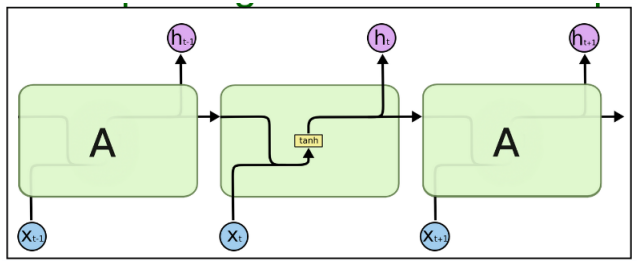
  - Any eigenvalues λ i that are not near an absolute value of 1 will either explode if they are greater than 1 in magnitude and vanish if they are less than 1 in magnitude
- Vanishing gradients make it difficult to know which direction the parameters should move to improve cost
- Exploding gradients make learning unstable

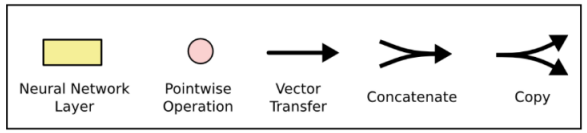
Keywords

- Word Embeddings
- Recurrent Neural Networks
- Backpropagation-through-time
- Teacher forcing
- Long-term dependencies
- **Long short-term memory**
- **Gating**

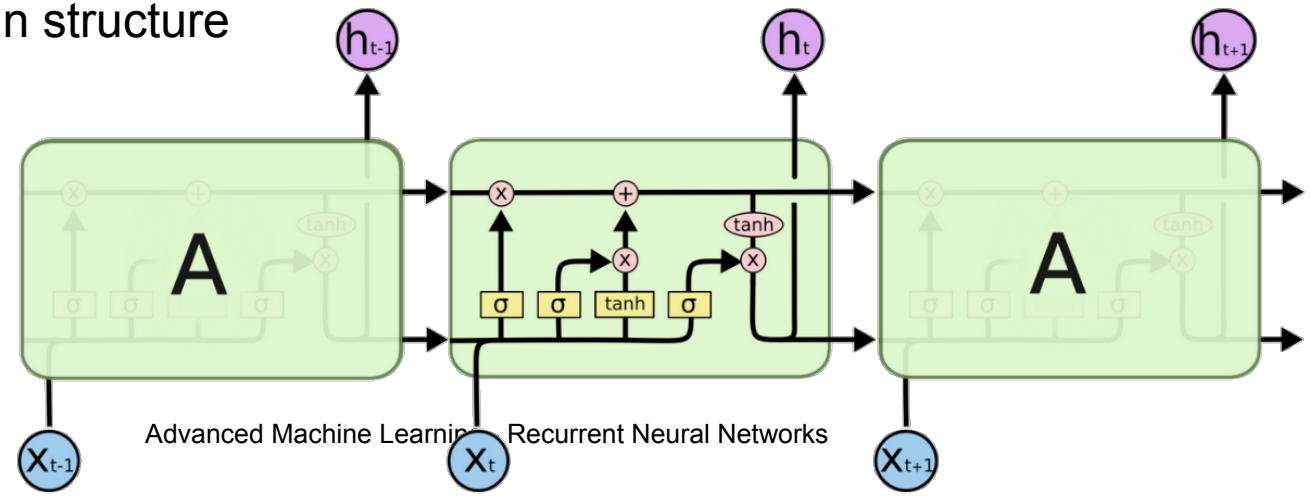
Long Short Term Memory

- Explicitly designed to avoid the long-term dependency problem
- RNNs have the form of a repeating chain structure
  - The repeating module has a simple structure such as tanh

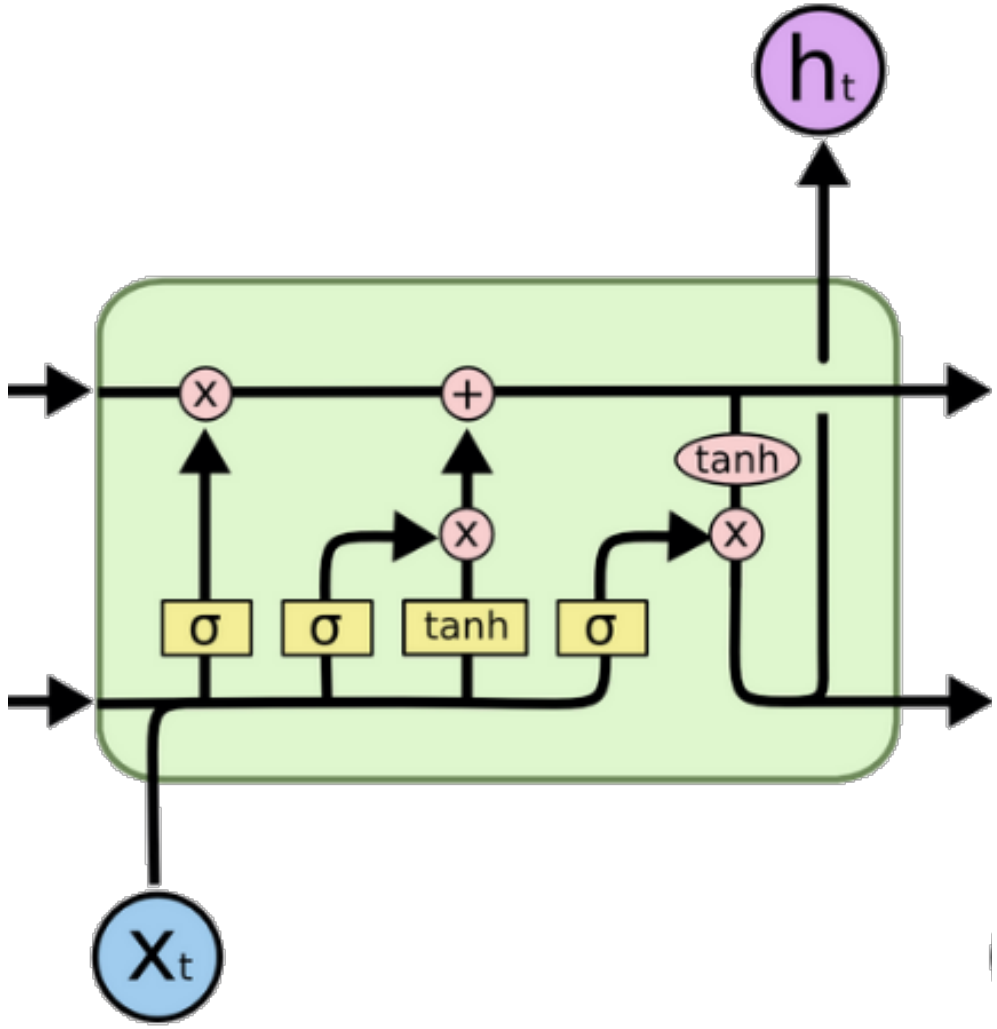




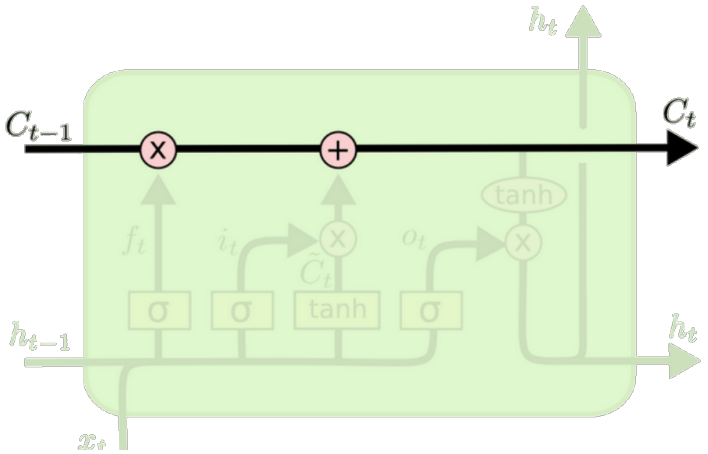
- LSTMs also have a chain structure



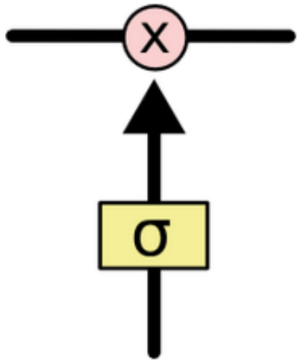
LSTM Unit



Core idea behind LSTM



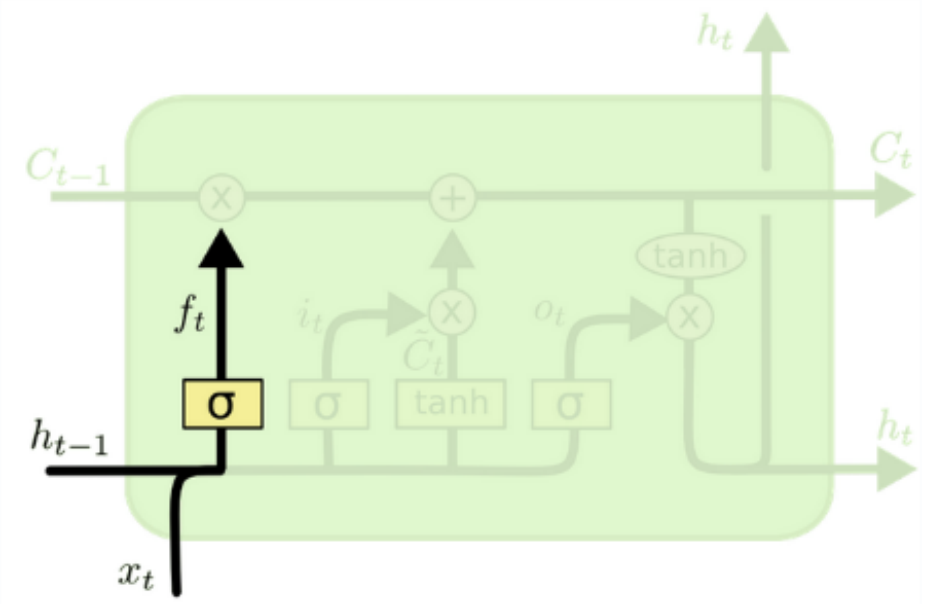
- The key to LSTM is the **cell state, C_t**, the horizontal line running through the top of the diagram
  - Like a conveyor belt
  - Runs through entire chain with minor interactions
- **LSTM does have the ability to remove/add information to cell state regulated by structures called gates**
  - Gates are an optional way to let information through
  - Consist of a sigmoid and a multiplication operation
  - Sigmoid outputs a value between 0 and 1
    - 0 means let nothing through
    - 1 means let everything through
- LSTM has three of these gates, to protect and control cell state



Step-by-step LSTM walk through

- **First step**: information to throw away from cell state
  - **Called forget gate layer**
  - Uses $h^{(t-1)}$ and $x^{(t)}$ and outputs a number between 0 and 1 for each $C^{(t-1)}$
  - $f^{(t)} = \sigma\left(\boldsymbol{W}_f \cdot \left[\boldsymbol{h}^{(t-1)}, \boldsymbol{x}^{(t)}\right] + b_f\right)$

- **Example**: In language models
  - Consider to predict the next word based on all previous ones.
  - The cell state may include the gender of the present subject so that the proper pronouns can be used
  - When we see a new subject we want to forget old subject



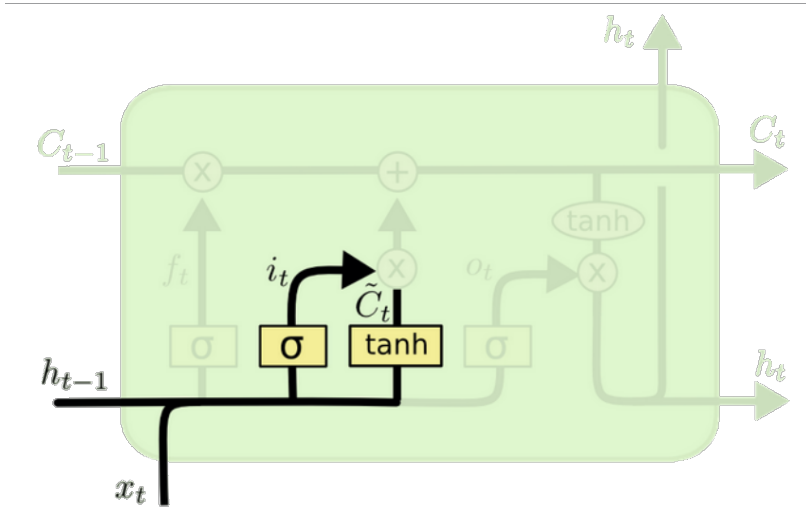
LSTM walk through: Second step

- **Second step**: what to store in the cell state
  - First a sigmoid layer called **Input gate layer**:
    - decides which values we will update
  - Next a tanh layer creates a vector of new candidate values $\tilde{C}^{(t)}$ that could be added to the state.

$$i^{(t)} = \sigma\left(\boldsymbol{W}_i \cdot \left[\boldsymbol{h}^{(t-1)}, \boldsymbol{x}^{(t)}\right] + b_i\right)$$

$$\tilde{C}^{(t)} = \tanh\left(\boldsymbol{W}_C \cdot \left[\boldsymbol{h}^{(t-1)}, \boldsymbol{x}^{(t)}\right] + b_C\right)$$

  - In the third step we will combine these two to create an update to the cell state

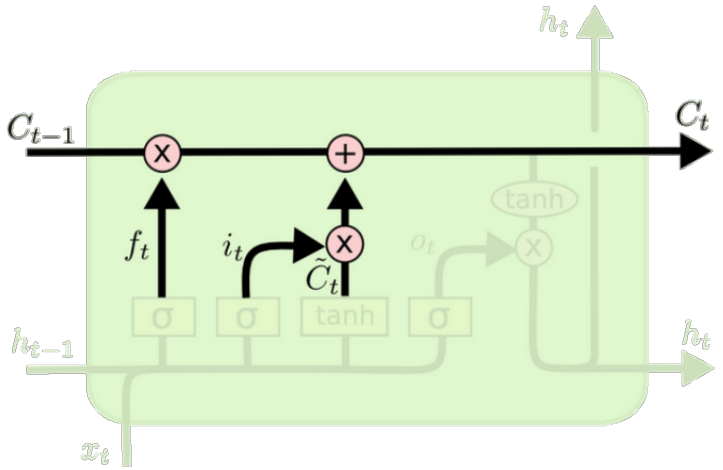


- **In language model**:
  - we'd want to add the gender of the new subject to the cell state, to replace the old One we are forgetting

LSTM walk through: Third step

- **Third step**: It's now time to update old cell state $C^{(t-1)}$ into new cell state $C^{(t)}$
  - The previous step decided what we need to do
  - We just need to do correctly modify the state
  - We multiply the old state by $f^{(t)}$, forgetting the things we decided to forget earlier.
  - Then we add $i^{(t)} \cdot \tilde{C}^{(t)}$:

$$C^{(t)} = f^{(t)} \cdot C^{(t-1)} + i^{(t)} \cdot \tilde{C}^{(t)}$$



- **In language model**:
  - this is where we'd actually drop the information about the old subject's gender and add the new information, as we decided in previous steps

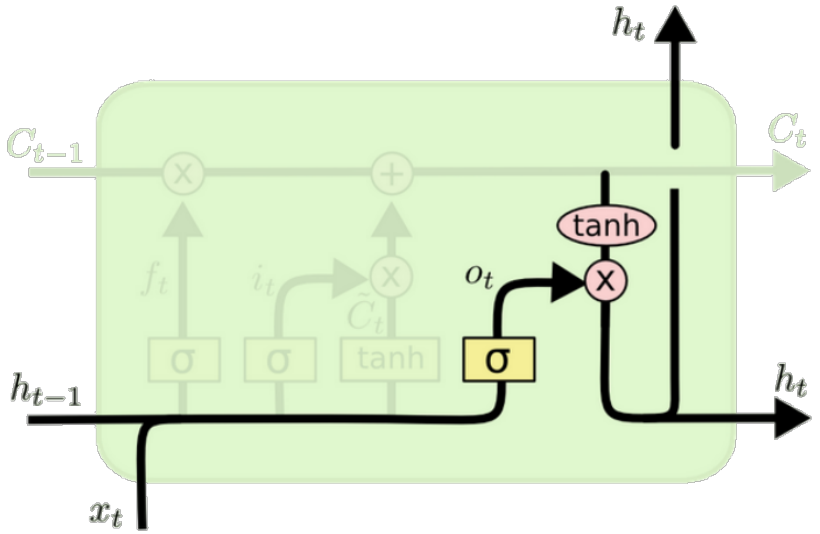
LSTM walk through: Fourth step

- **Fourth Step**: Finally we decide what we are going to output
  - This output will be a **filtered version of our cell state**
  - First we run a sigmoid layer which decides what parts of cell state we're going to output.
  - Then we put the cell state through tanh (to push values to be between -1 and 1)

$$o^{(t)} = \sigma\left(W_o \cdot \left[h^{(t-1)}, x^{(t)}\right] + b_o\right)$$

$$h^{(t)} = o_t \cdot \tanh\left(C^{(t)}\right)$$

- **In language model**:
  - Since it just saw a subject it might want to output information relevant to a verb, in case that is what is coming next, e.g., it might output whether the subject is singular or plural so that we know what form a verb should be conjugated into if that's what follows next.



LSTM

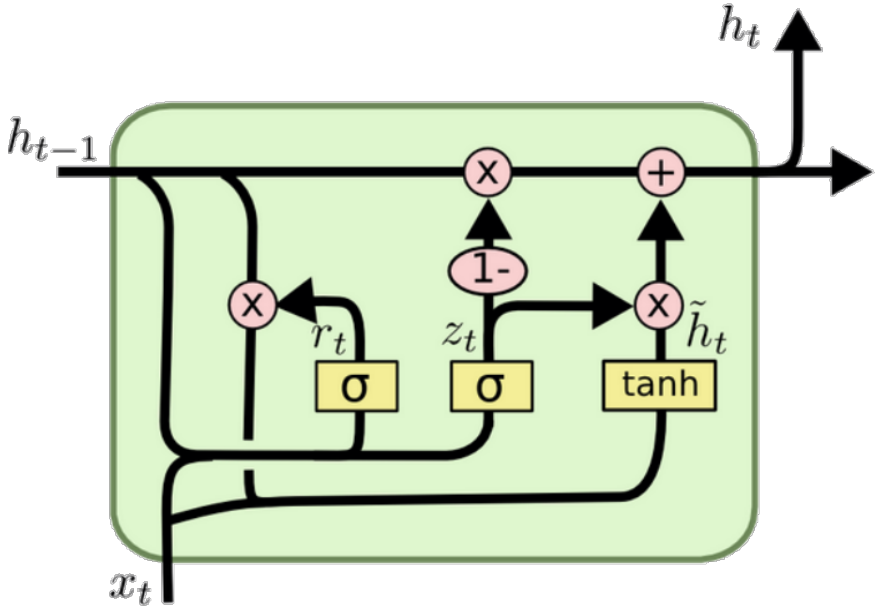
- Contribution of LSTMs:
  - clever idea of introducing self-loops to produce paths where the gradient can flow for long durations

- A crucial addition: make weight on this self-loop conditioned on the context, rather than fixed
  - By making weight of this self-loop gated (controlled by another hidden unit), time-scale can be changed dynamically
  - Even for an LSTM with fixed parameters, time scale of integration can change based on the input sequence
    - Because time constants are output by the model itself

- LSTM found extremely successful in:
  - Unconstrained handwriting recognition, Speech recognition
  - Handwriting generation, Machine Translation
  - Image Captioning, Parsing

Gated Recurrent Unit (GRU)

- A dramatic variant of LSTM
  - It combines the forget and input gates into a single update gate
  - It also merges the cell state and hidden state, and makes some other changes
  - The resulting model is simpler than LSTM models
  - Has become increasingly popular

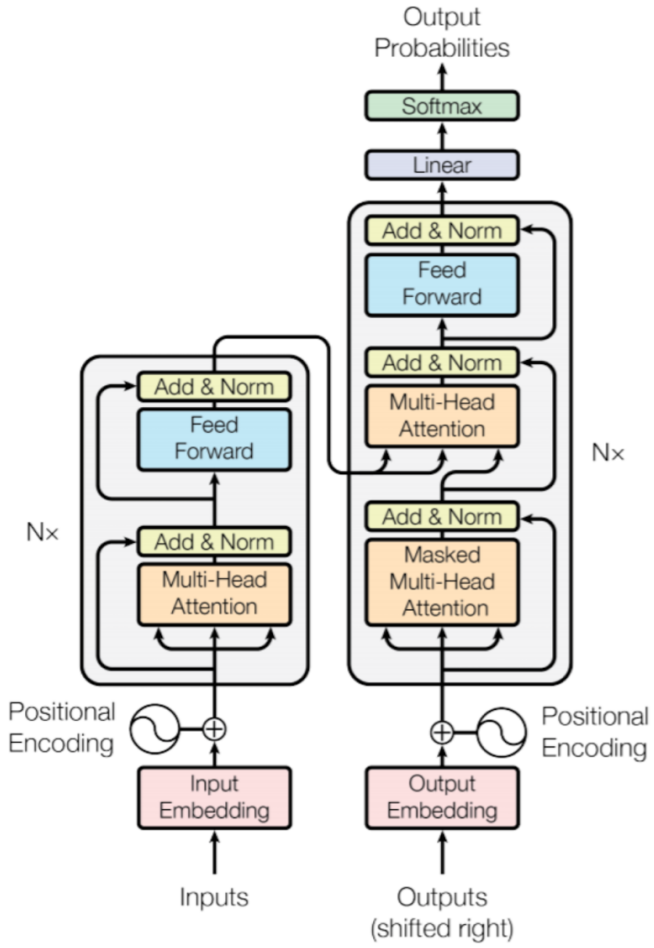


Keywords

- Word Embeddings
- Recurrent Neural Networks
- Backpropagation-through-time
- Teacher forcing
- Long-term dependencies
- Long short-term memory
- Gating

Summary and Outlook

- RNNs are powerful sequence models
  - Can process arbitrary length inputs
  - Captures patterns regardless of exact position
  - In theory, any function computable by a Turing Machine is computable by an RNN
- **Drawbacks**
  - Long-term dependencies still challenging
  - Sequential dependencies, thus poorly parallelizable
- **Outlook**
  - Attention mechanism to deal with long-term dependencies
  - Transformers models to process entire sequences in parallel



How’s my teaching?

https://www.admonymous.co/lukasgalke

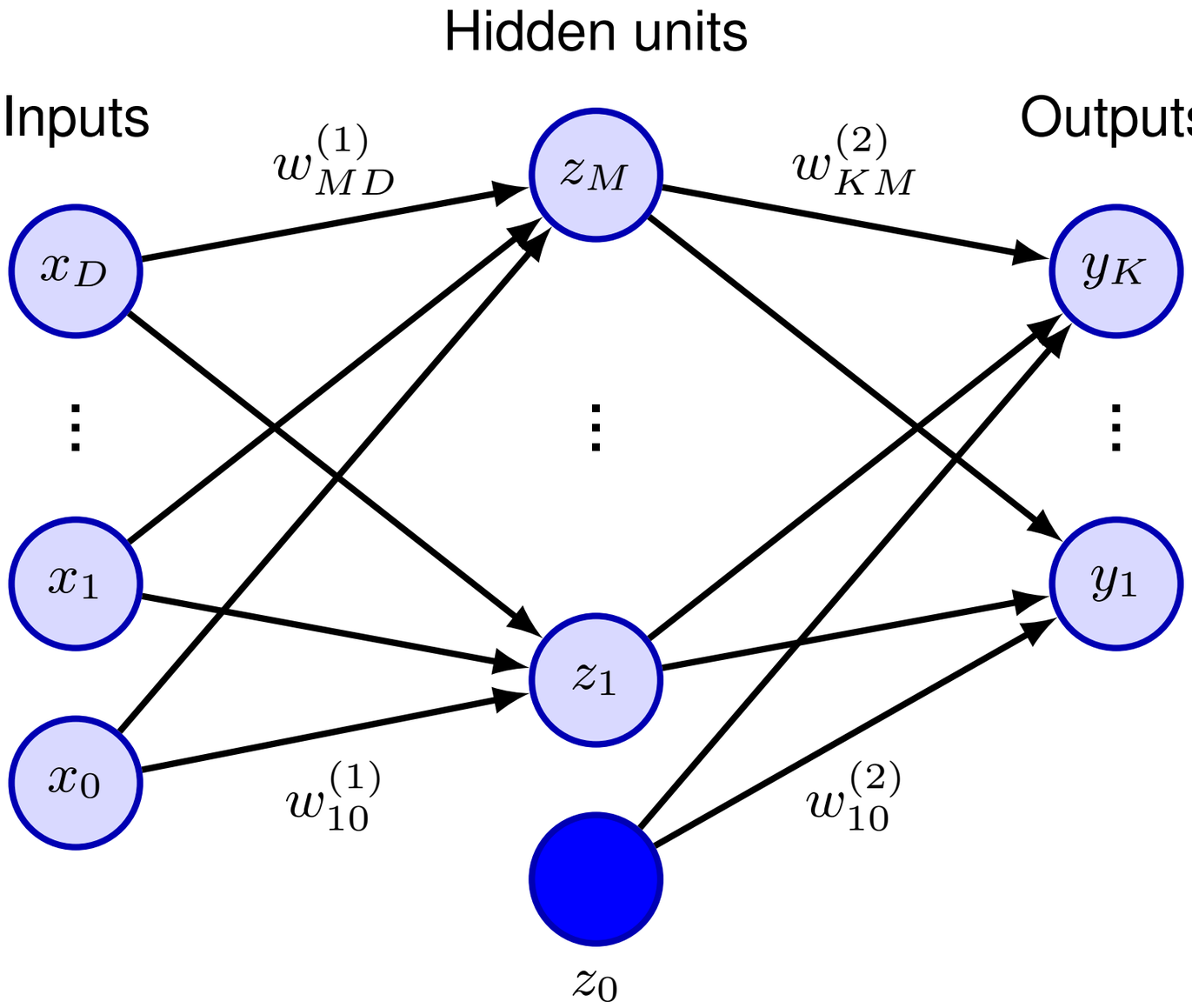
Advanced Machine Learning

Nonlinear Recap

Lukas Galke Poech

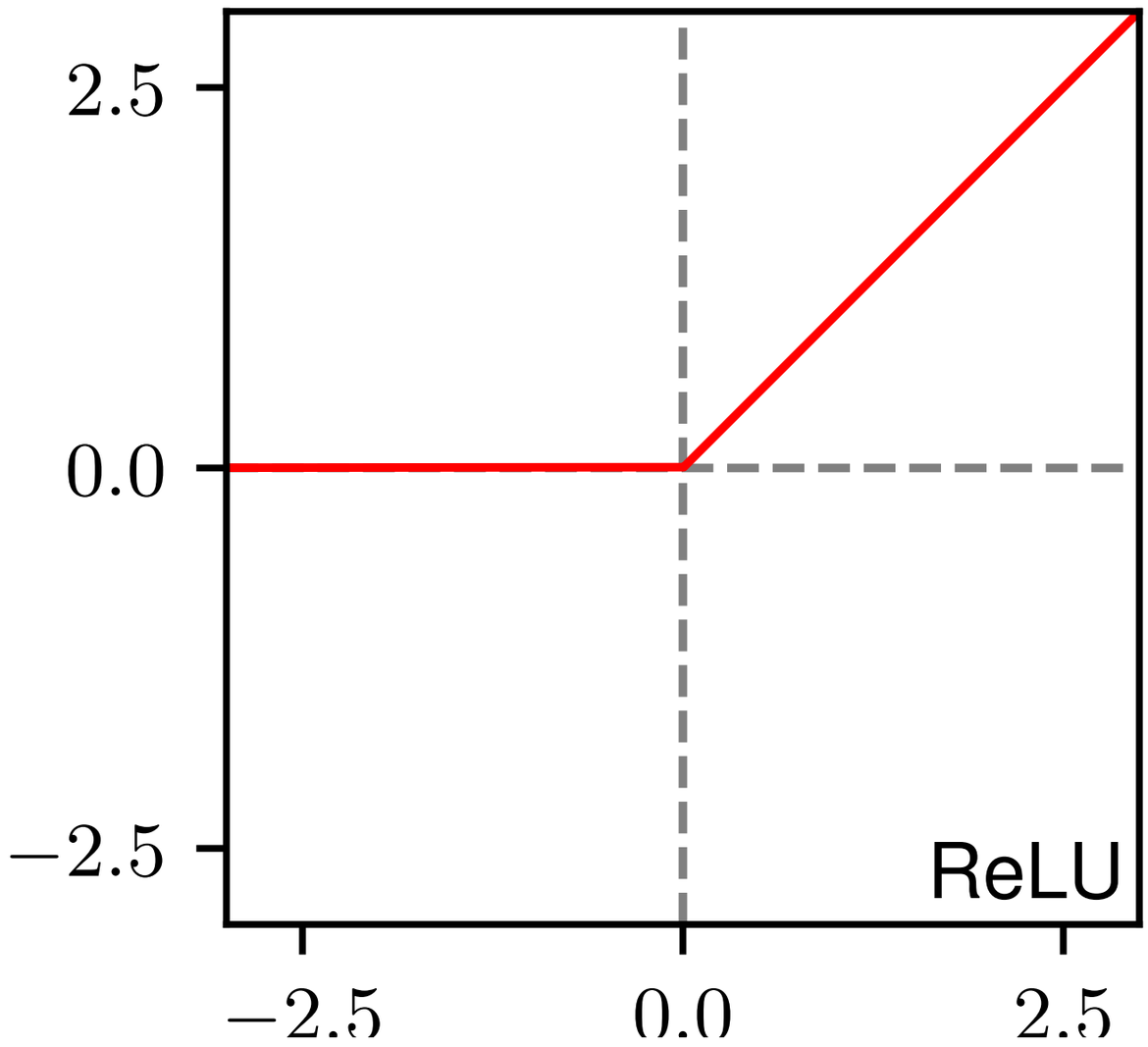
- *Probabilistic Modeling*
- **Deep Neural Networks**
- **Optimization & Regularization**
- Transformers
- Graph Neural Networks
- Autoencoders
- *Basics of Interpretability*

Feedforward Networks



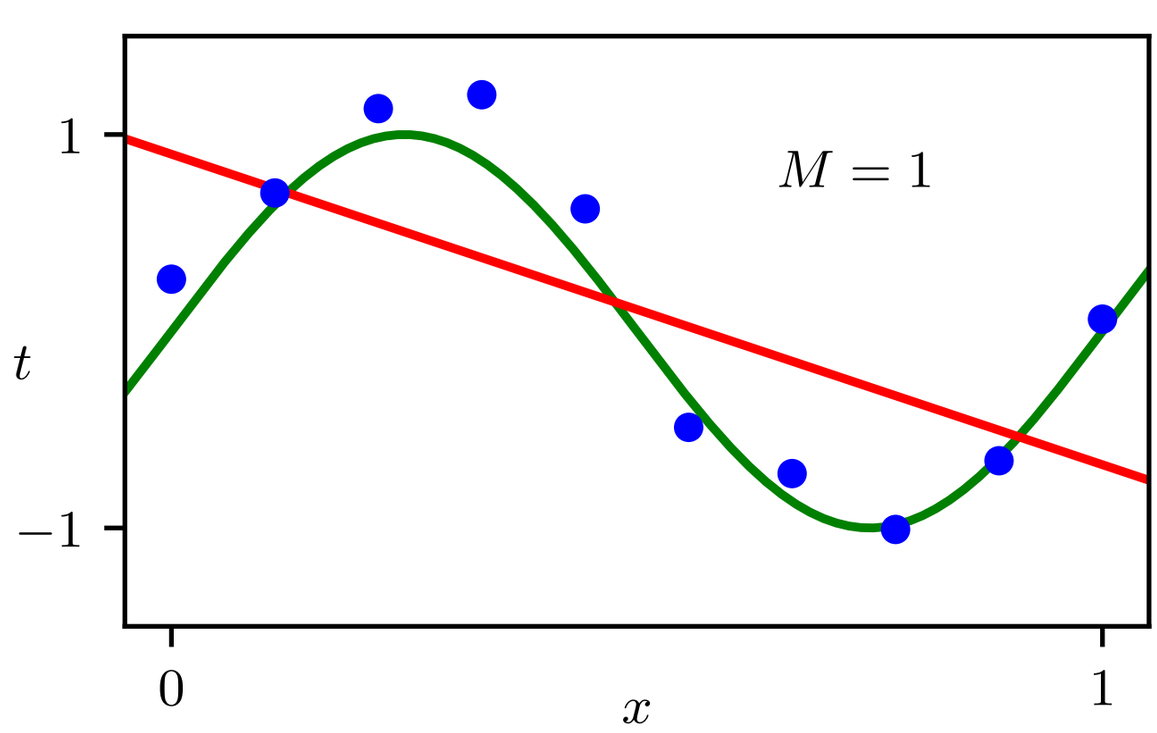
Activation Functions

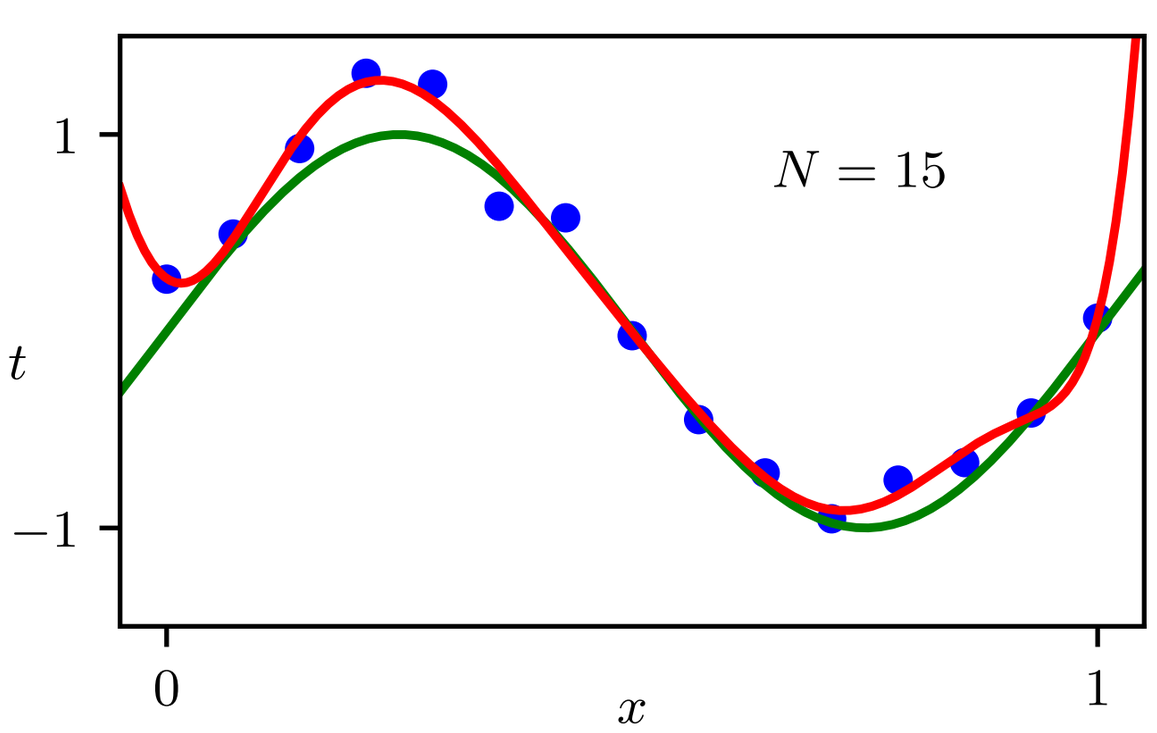
- Problems of sigmoidal activation functions
- Why piecewise linear functions, like ReLU are preferable
- Dead ReLU



Universal Approximation Theorem

A deep neural network with at least one hidden layer can approximate any compact function to an arbitrary degree of accuracy, depending on the width of the hidden layer(s).

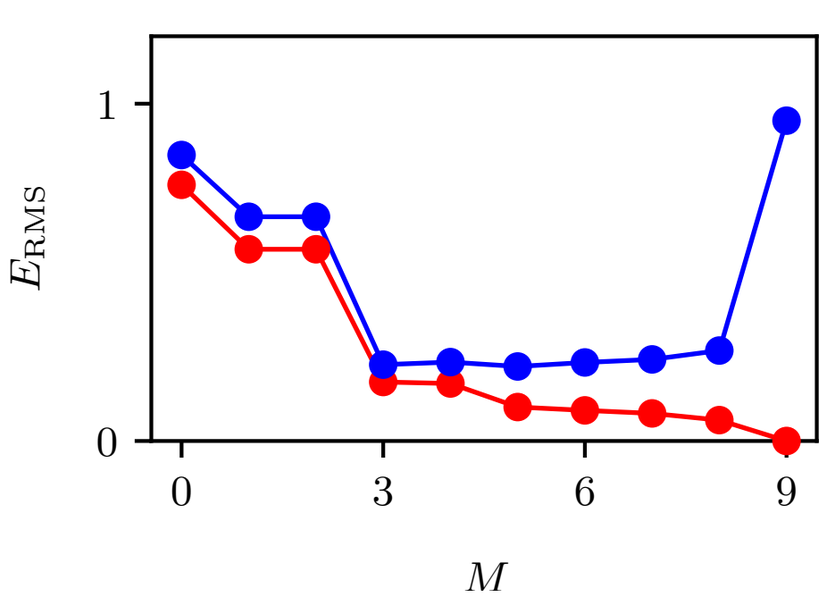


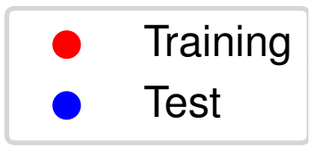


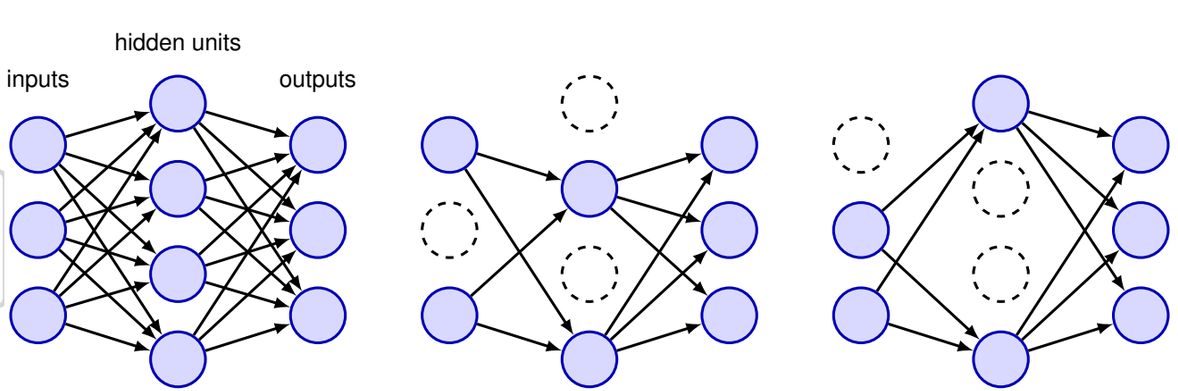
Regularization

Decrease validation error (often) at the expense of training error

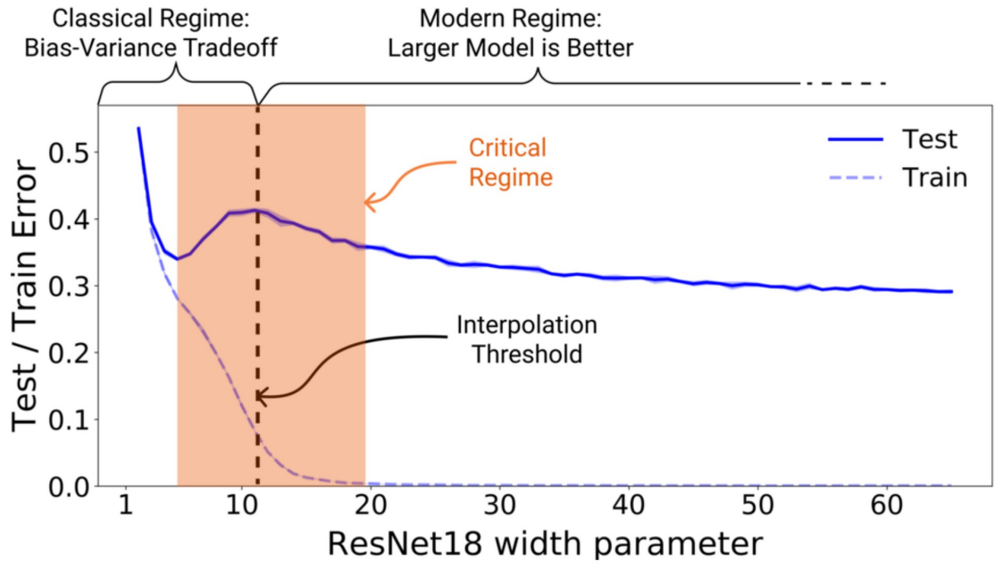
- Data augmentation
- Parameter penalties (L1, L2, weight decay)
- Dropout

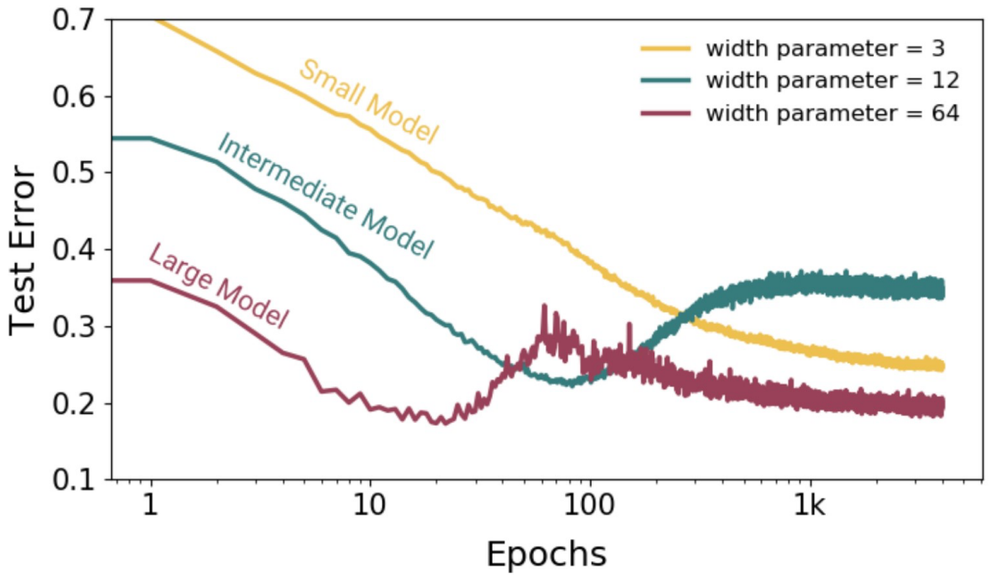






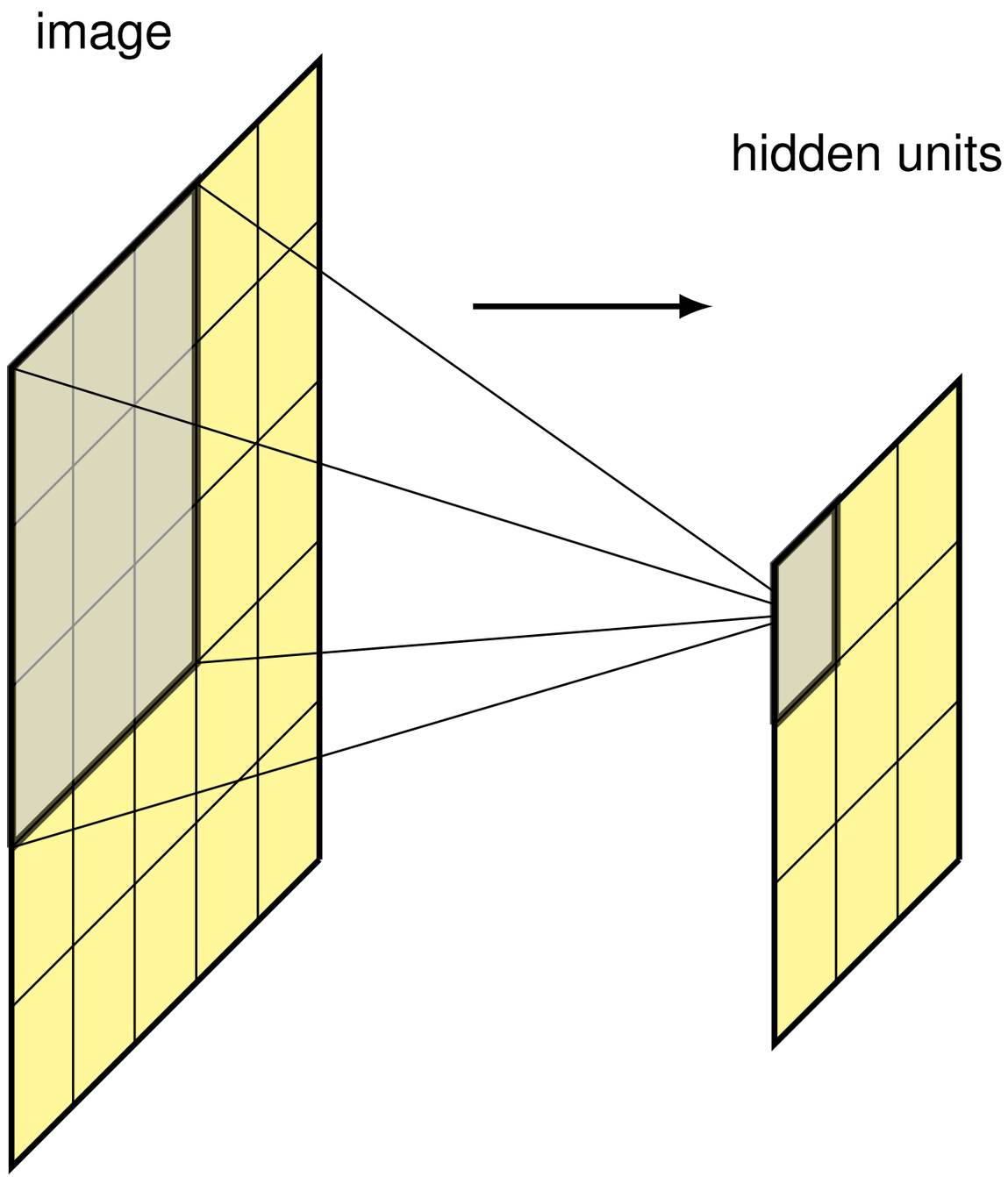
Deep Double Descent





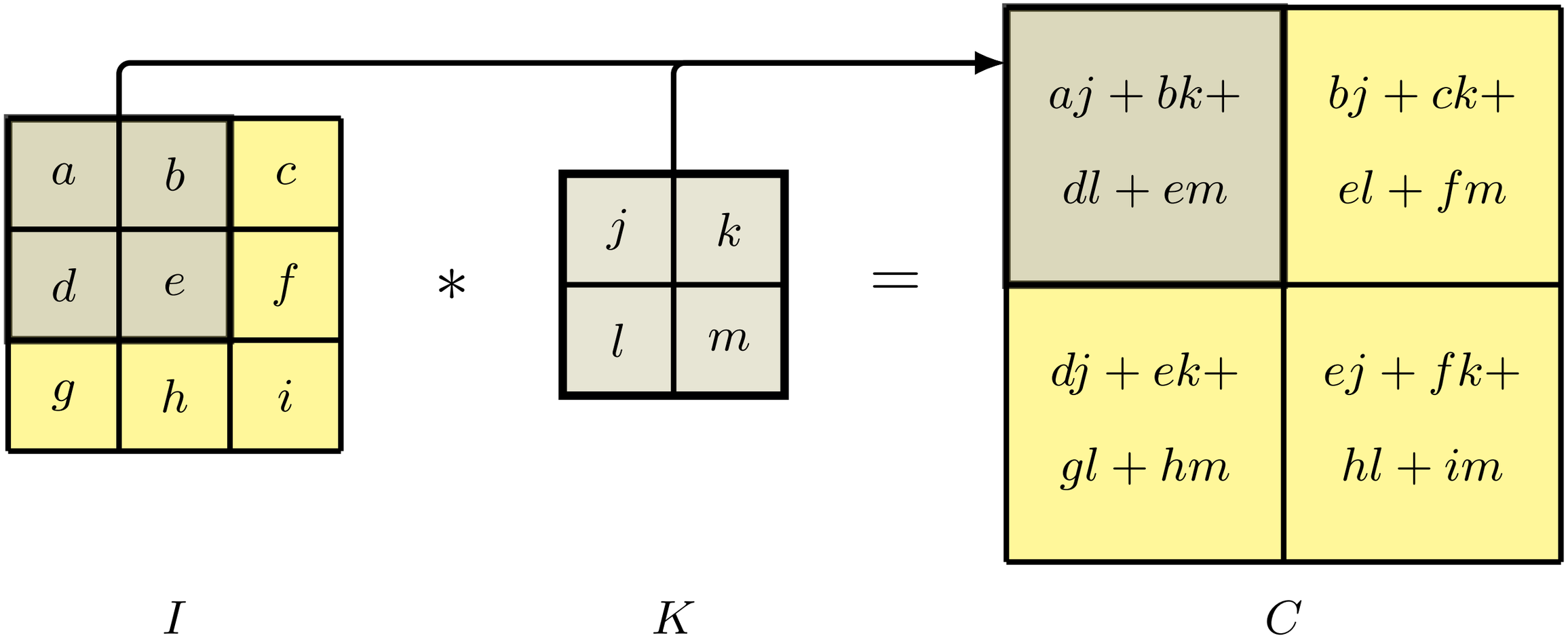
Convolutional Neural Networks

- Convolution
- Pooling
- Normalization
- Parameter sharing
- Residual connections

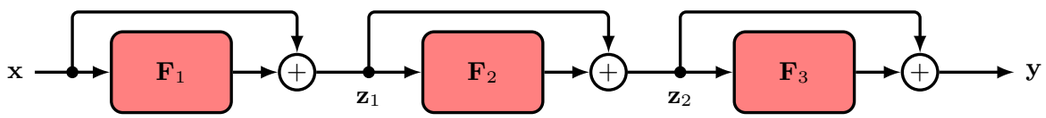


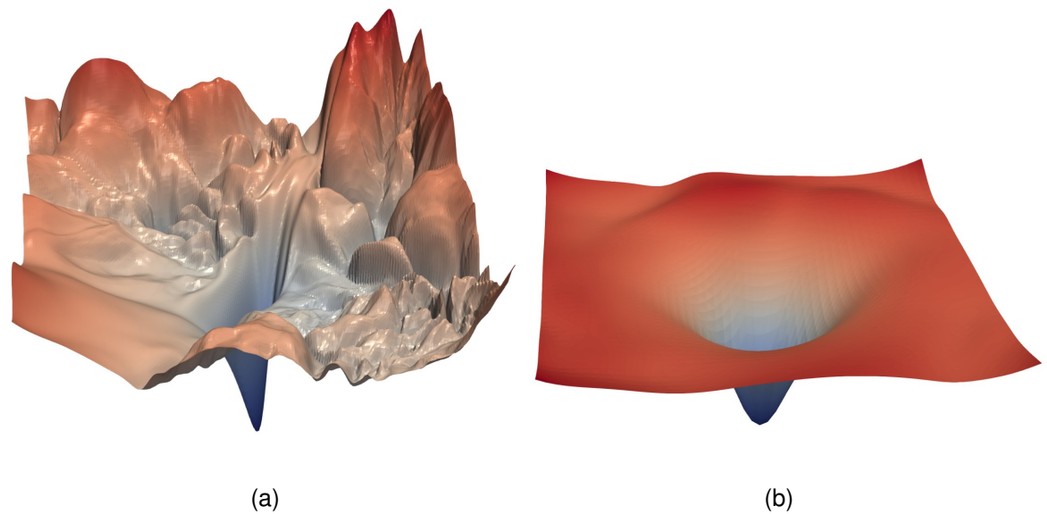
Parameter Sharing

The same parameters are used at different locations in the model.



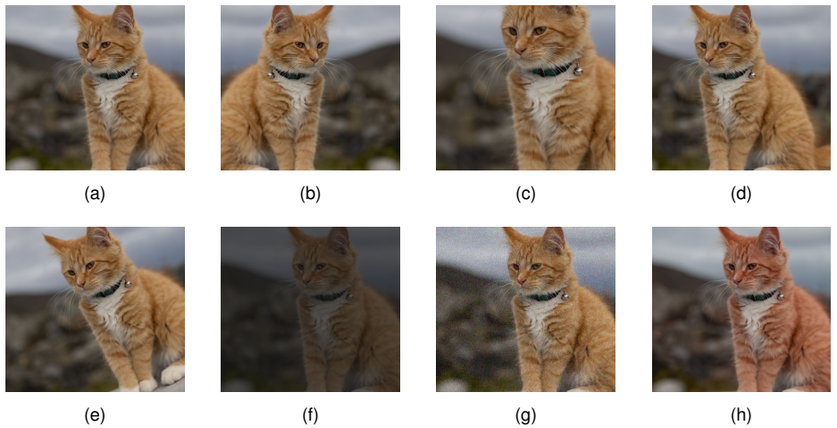
Residual connections

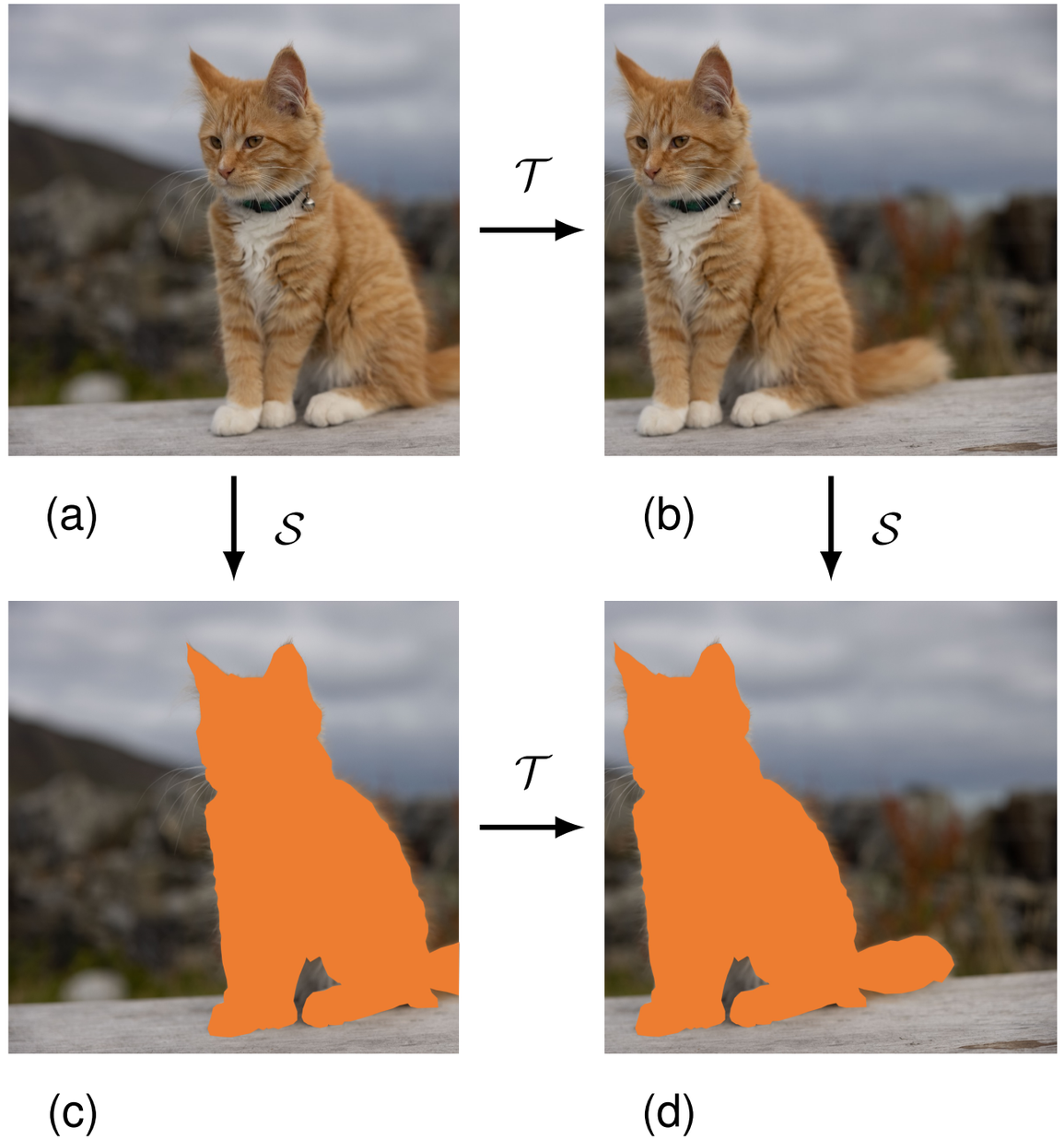




(a) (b)

Invariance (Left) and Equivariance (Right)





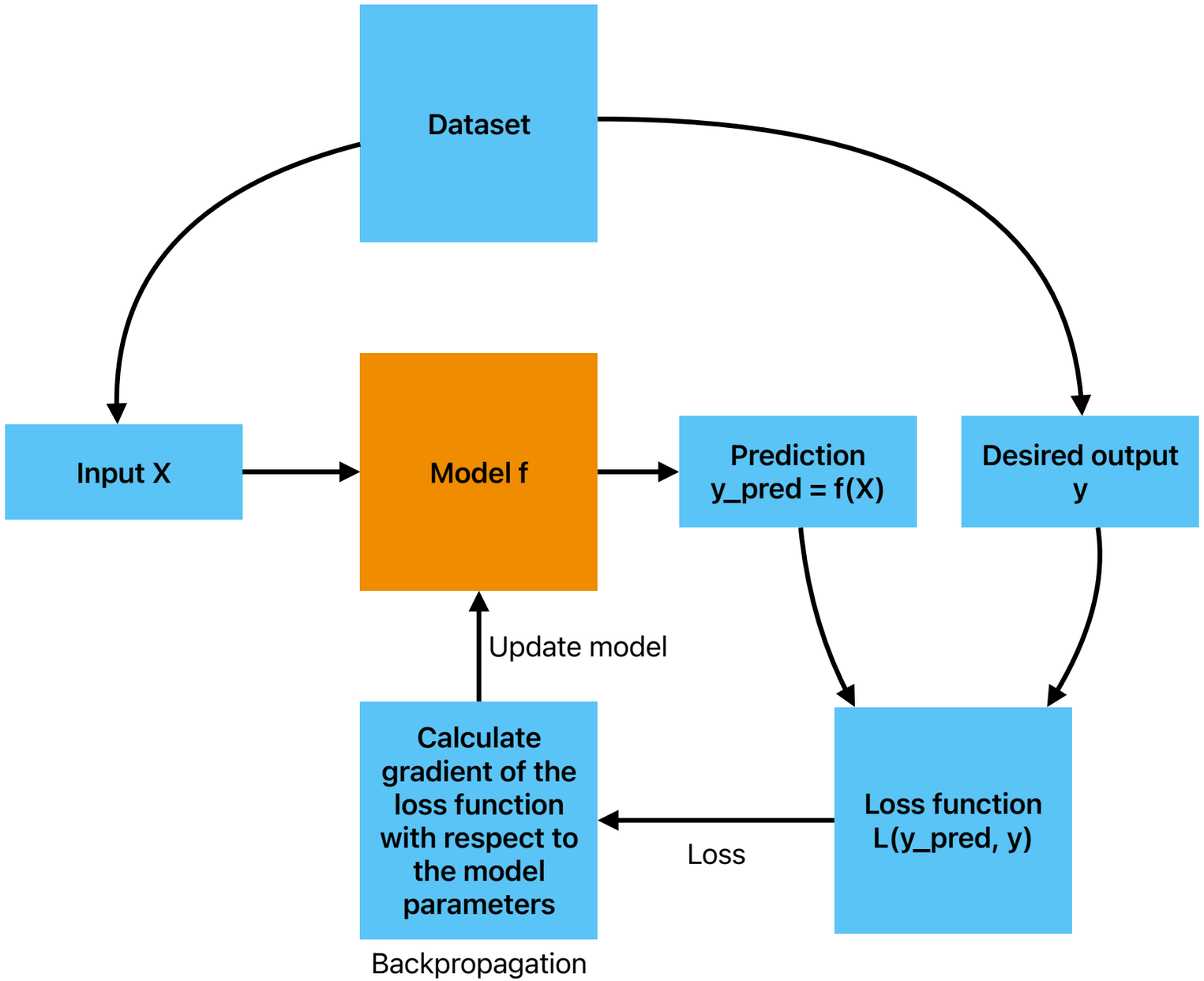
Backpropagation and optimizers

Optimizers and their challenges

Momentum

Adam and AdamW

Backprop


Dataset
Input X
Model f
Prediction
y_pred = f(X)
Desired output
y
Update model
Calculate
gradient of the
loss function
with respect to
the model
parameters
Loss
Loss function
L(y_pred, y)
Backpropagation

Loss Functions

Maximum Likelihood and Cross-entropy

Maximum likelihood estimation (MLE) aims to find model parameters that maximize the likelihood of observed data.

Likelihood function: $L(\theta) = \prod_i q(x_i; \theta)$

Log-likelihood: $\log L(\theta) = \sum_i \log q(x_i; \theta)$

Negative log likelihood: $-\sum_i \log q(x_i; \theta)$

This is exactly the **cross-entropy** between the true distribution $p(x)$ and the predicted distribution $q(x)$

$$H(P, Q) = -\mathbb{E}_{x \sim p}[\log q(x)]$$

Connection between Cross-entropy and KL Divergence

$$H(p, q) = H(p) + D_{\mathrm{KL}}(p||q)$$

minimizing for cross-entropy is equivalent to minimizing for KL divergence, as $H(p)$ is constant

Cross-Entropy Loss

Softmax $y_i = \frac{\exp(y_i)}{\sum_j \exp(y_j)}$

Cross-entropy loss: Softmax + neg. log likelihood

$$L(\mathbf{y}, t) = -\log \frac{\exp(y_t)}{\sum_j \exp(y_j)}$$

$$= -y_t + \log \sum_j \exp(y_j)$$

Key points

If we want to model $p_{\text{data}}(y|x)$, principle of maximum likelihood suggests we use $-\log p_{\text{model}}(y|x;\theta)$ as cost function

minimizing Cross-entropy / KL divergence / negative log likelihood

Next Up: Architectures

Department of Mathematics
and Computer Science

sdu.dk

#sdudk

Transformers

Advanced Machine Learning, Spring 2026

Lukas Galke Poech, some material based on slides by Lucas Beyer

Recap: Word Embeddings

- Segment sequence into tokens
- For each token, look up its index in the vocabulary
- Fetch the row corresponding to the index from the embedding table
- And proceed with the vector found in the embedding table

Embedding Table

[EOS]	[0.2, -0.1, 0.0, 0.5, 0.1, 0.0, 0.0, 0.2, 0.01, 0.0]	**0**
the	[0.2, -0.1, 0.1, -0.5, 0.1, -0.08, 0.04, 0.2, 0.01, -0.3]	**1**
Have	[0.2, -0.1, 0.45, -0.5, 0.1, 0.8, 0.4, 0.2, 0.01, -0.3]	**2**
a	[0.2, -0.1, -0.9, 0.5, 0.1, -0.3, 0.4, -0.4, 0.01, -0.3]	**3**
good	[0.2, -0.1, 0.9, 0.5, 0.1, 0.8, 0.4, 0.2, 0.01, -0.3]	**4**
great	[0.2, -0.1, 0.9, 0.5, 0.1, 1.0, 0.4, 0.2, 0.01, -0.3]	**5**
day	[0.2, -0.1, -1.3, 0.5, 0.1, 0.8, 0.4, 0.2, 0.01, -0.3]	**6**

Word emb:

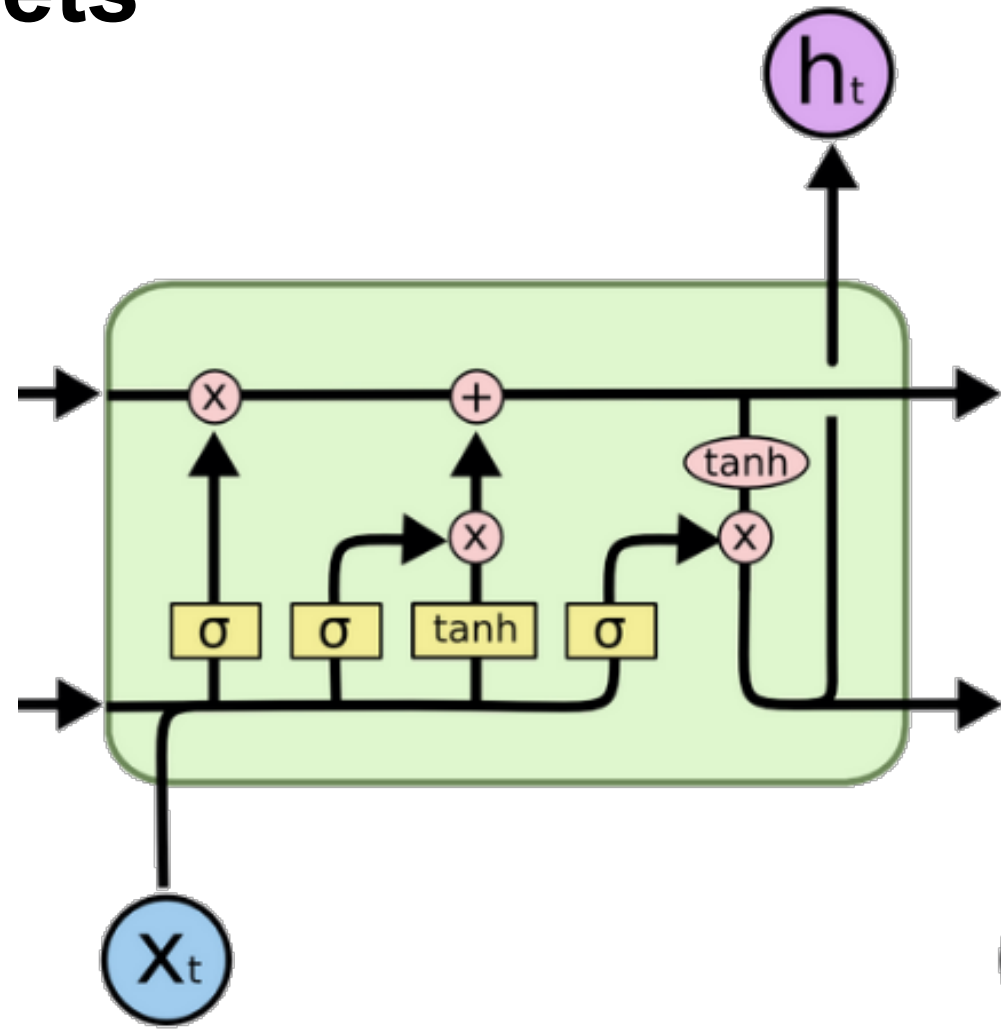
Tokenizer: **[2, 3, 4, 6]**

[“Have”, “a”, “good”, “day”]

Input Sequence: *Have a good day*

Recap: Recurrent Neural Nets

- RNNs are powerful sequence models
  - Can process arbitrary length inputs
  - Parameter sharing across positions
- **Drawbacks/Challenges:**
  - Long-range dependencies still challenging
  - Poorly parallelizable, processing of each token depends on the previous tokens
- **Transformers to the rescue?**



Keywords

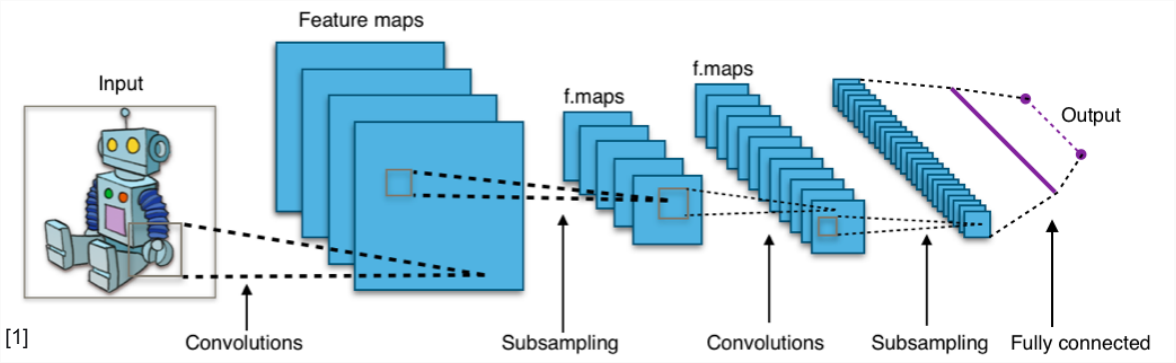
- Attention
- Transformer architecture
- Language models
- Transformers across modalities

The classic landscape:

One architecture per "community"

Computer Vision

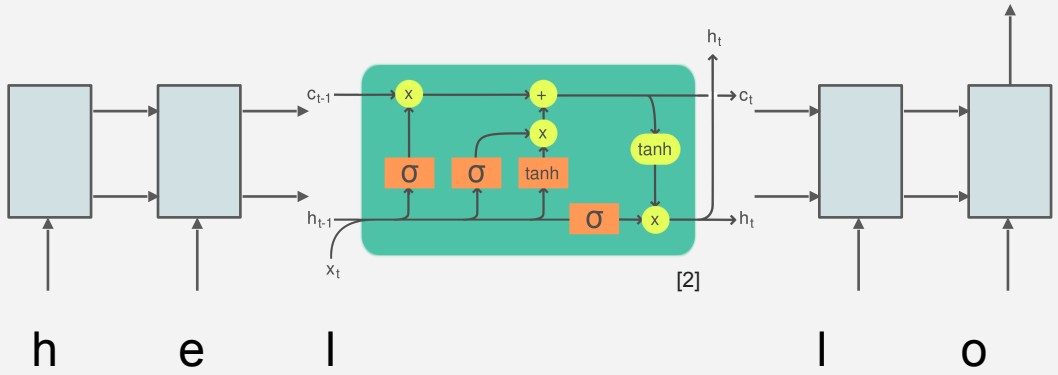
Convolutional NNs (+ResNets)



[1]

Natural Lang. Proc.

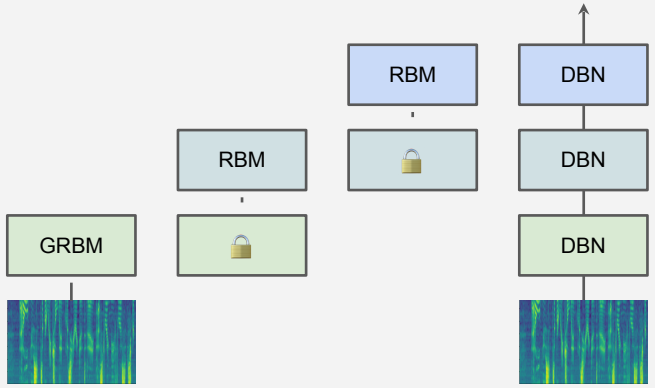
Recurrent NNs (+LSTMs)



h e l l o

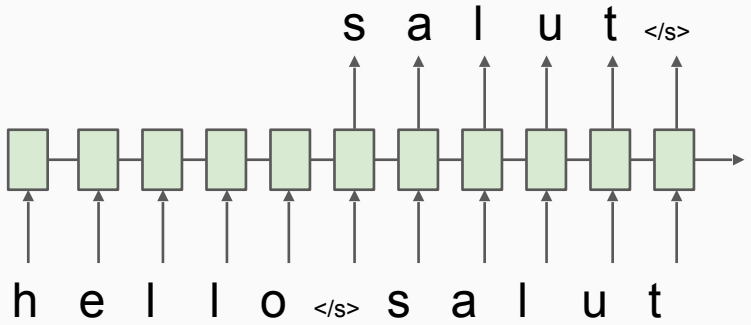
Speech

Deep Belief Nets (+non-DL)



Translation

Seq2Seq



RL

BC/GAIL

Algorithm 1 Generative adversarial imitation learning

1: **Input:** Expert trajectories $\tau_E \sim \pi_E$, initial policy and discriminator parameters θ_0, w_0
2: **for** $i = 0, 1, 2, \ldots$ **do**
3: Sample trajectories $\tau_i \sim \pi_{\theta_i}$
4: Update the discriminator parameters from w_i to w_{i+1} with the gradient

$$\hat{\mathbb{E}}_{\tau_i}[\nabla_w \log(D_w(s,a))] + \hat{\mathbb{E}}_{\tau_E}[\nabla_w \log(1 - D_w(s,a))] \quad (17)$$

5: Take a policy step from θ_i to θ_{i+1}, using the TRPO rule with cost function $\log(D_{w_{i+1}}(s,a))$. Specifically, take a KL-constrained natural gradient step with

$$\hat{\mathbb{E}}_{\tau_i}[\nabla_\theta \log \pi_\theta(a|s) Q(s,a)] - \lambda \nabla_\theta H(\pi_\theta), \quad (18)$$
$$\text{where } Q(\bar{s},\bar{a}) = \hat{\mathbb{E}}_{\tau_i}[\log(D_{w_{i+1}}(s,a)) \mid s_0 = \bar{s}, a_0 = \bar{a}]$$

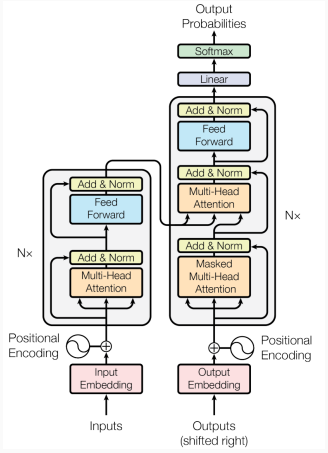
6: **end for**

[1] CNN image CC-BY-SA by Aphex34 for Wikipedia https://commons.wikimedia.org/wiki/File:Typical_cnn.png
[2] RNN image CC-BY-SA by GChe for Wikipedia https://commons.wikimedia.org/wiki/File:The_LSTM_Cell.svg

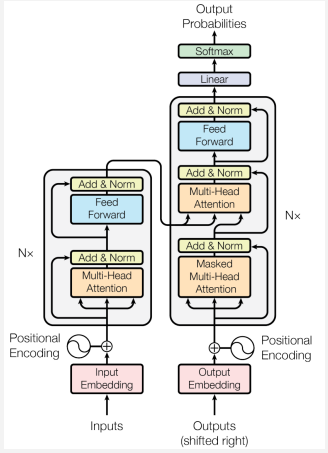
The Transformer's takeover:

One community at a time

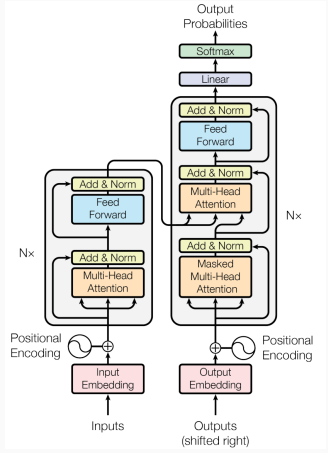
Computer Vision



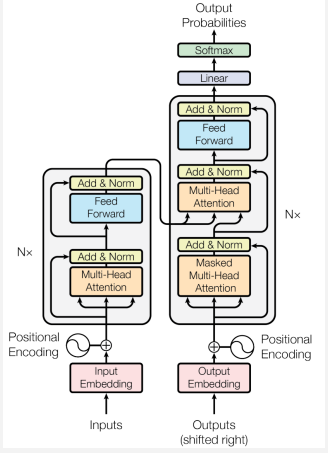
Natural Lang. Proc.



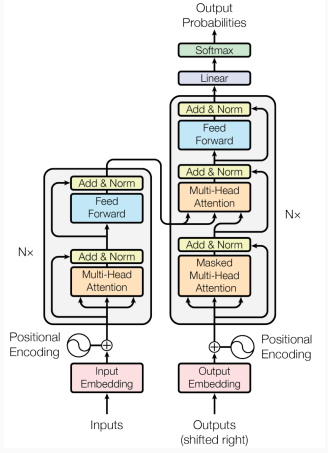
Reinf. Learning



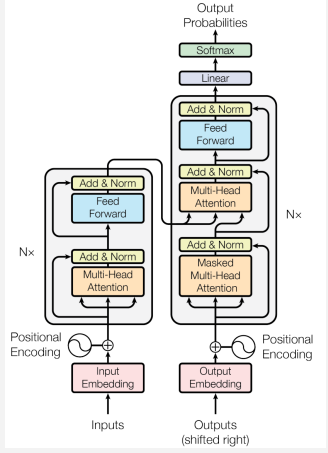
Speech



Translation



Graphs/Science



Transformer image source: "Attention Is All You Need" paper

Keywords

- **Attention**
- Transformer architecture
- Language models
- Transformers across modalities

The origins:

Translation, learned alignment

Neural Machine Translation by Jointly Learning to Align and Translate

2014, Dzmitry Bahdanau, KyungHyun Cho, Yoshua Bengio

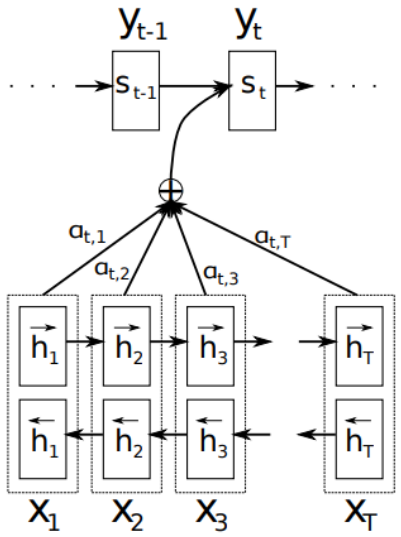
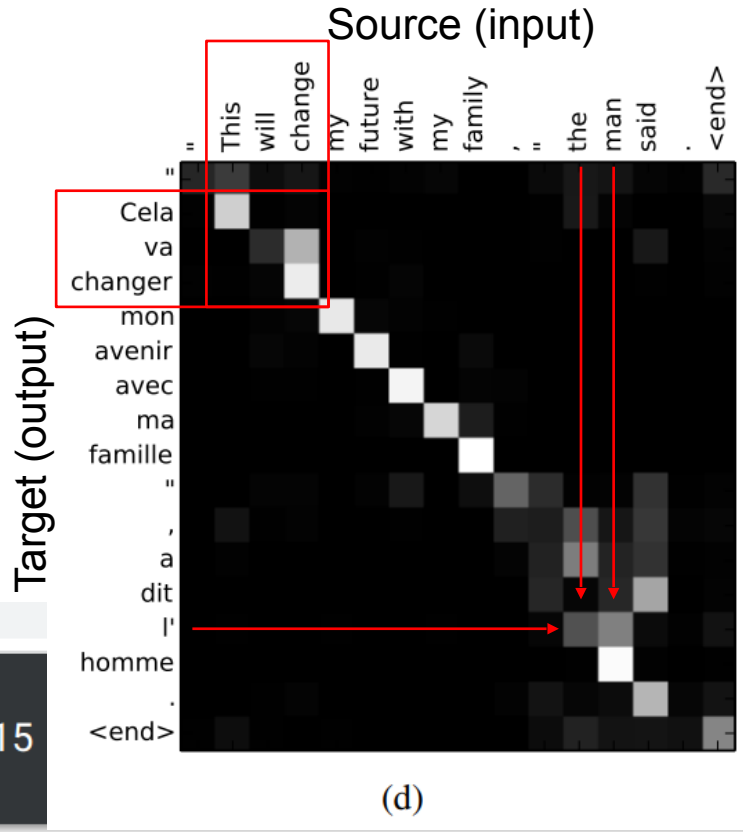


Figure 1: The graphical illustration of the proposed model trying to generate the t-th target word y_t given a source sentence $(x_1, x_2, \ldots, x_T)$.



4 / 15

attention 1/3

The probability α_{ij}, or its associated energy e_{ij}, reflects the importance of the annotation h_j with respect to the previous hidden state s_{i-1} in deciding the next state s_i and generating y_i. Intuitively, this implements a mechanism of attention in the decoder. The decoder decides parts of the source sentence to pay attention to. By letting the decoder have an attention mechanism, we relieve the encoder from the burden of having to encode all information in the source sentence into a fixed-length vector. With this new approach the information can be spread throughout the sequence of annotations, which can be selectively retrieved by the decoder accordingly.

Attention Is All You Need

2017, Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, Illia Polosukhin

Attention is a function similar to a "soft" kv dictionary lookup:

1. Attention weights $a_{1:N}$ are query-key similarities:

   $\hat{a}_i = q \cdot k_i$

   Normalized via softmax: $a_i = e^{\hat{a}i} / \sum_j e^{\hat{a}j}$

2. Output z is attention-weighted average of values $v_{1:N}$:

   $z = \sum_i \hat{a}_i v_i = \hat{a} \cdot v$

3. Usually, k and v are derived from the same input x:

   $k = W_k \cdot x$

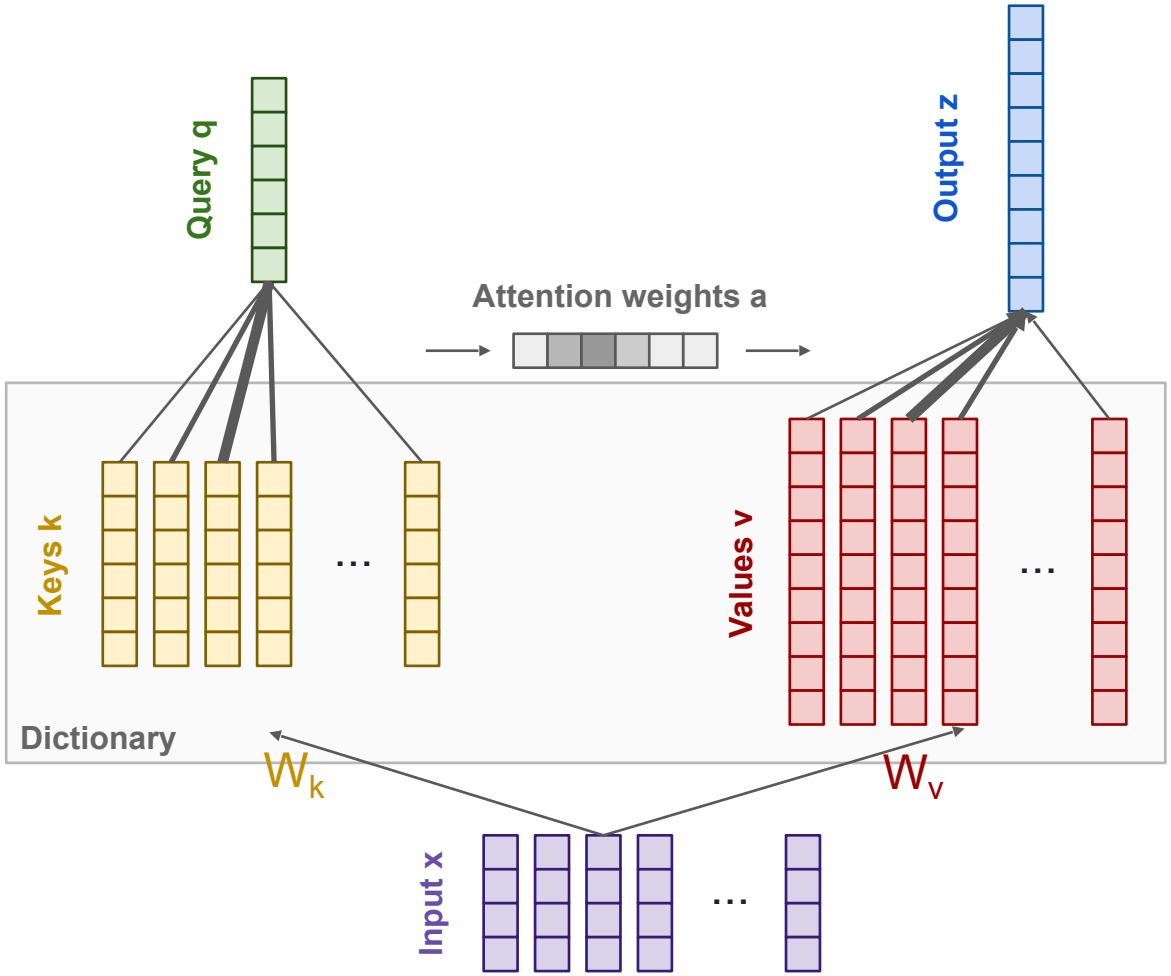
   $v = W_v \cdot x$

The query q can come from a separate input y:

$q = W_q \cdot y$

Or from the same input x! Then we call it "self attention":

$q = W_q \cdot x$



Historical side-note: "non-local NNs" in computer vision and "relational NNs" in RL appeared almost at the same time and contain the same core idea!

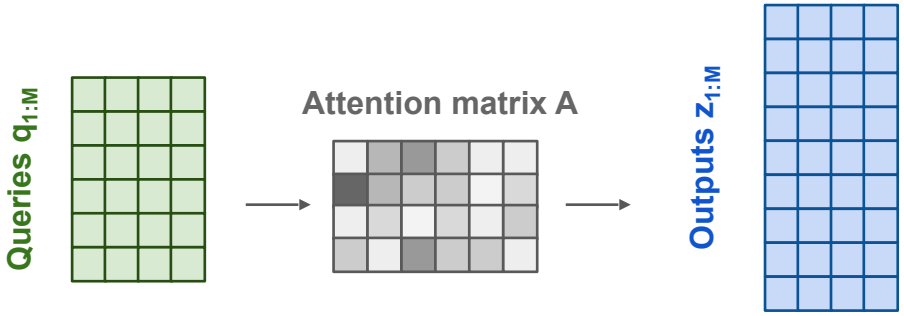
Attention Is All You Need

2017, Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, Illia Polosukhin

But that's not actually it! There are a few more details:

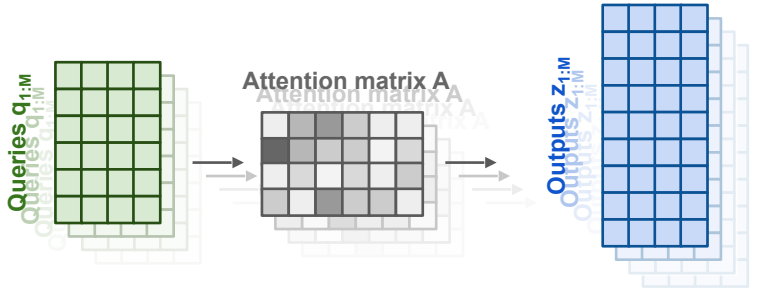
1. We usually use **many queries** $q_{1:M}$, not just one. Stacking them leads to the Attention matrix $A_{1:N,1:M}$ and subsequently to many outputs:

$$z_{1:M} = \text{Attn}(q_{1:M},x) = [\text{Attn}(q_1,x) \mid \text{Attn}(q_2,x) \mid ... \mid \text{Attn}(q_M,x)]$$



2. We usually use "**multi-head**" attention. This means the operation is repeated K times and the results are concatenated along the feature dimension. Ws differ.

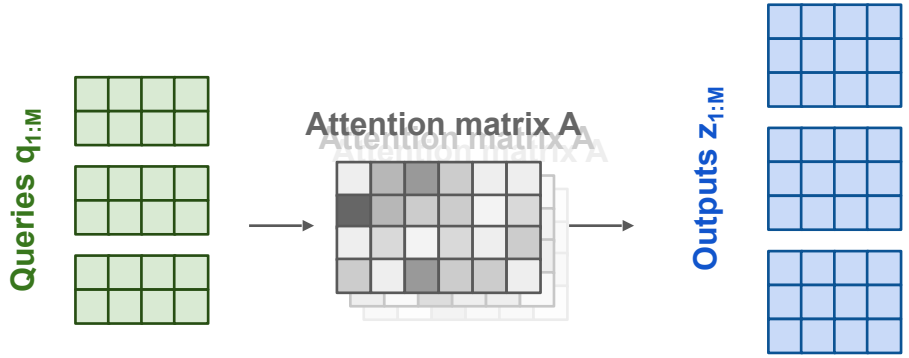
$$z_i = \begin{pmatrix} \text{Attn}_1(q_i,x) \\ \text{Attn}_2(q_i,x) \\ .... \\ \text{Attn}_K(q_i,x) \end{pmatrix}$$



3. The most commonly seen formulation:

$$z = \text{softmax}(QK'/\sqrt{d_{key}})V$$

Note that the complexity is $O(N^2)$

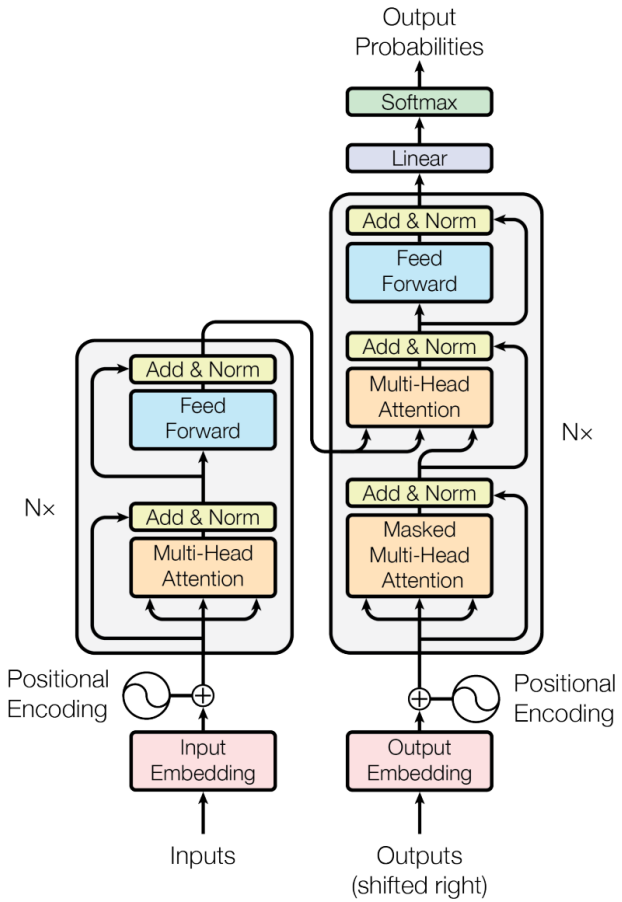


Keywords

- Attention
- **Transformer architecture**
- Language models
- Transformers across modalities

Attention Is All You Need - The Transformer architecture

2017, Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, Illia Polosukhin

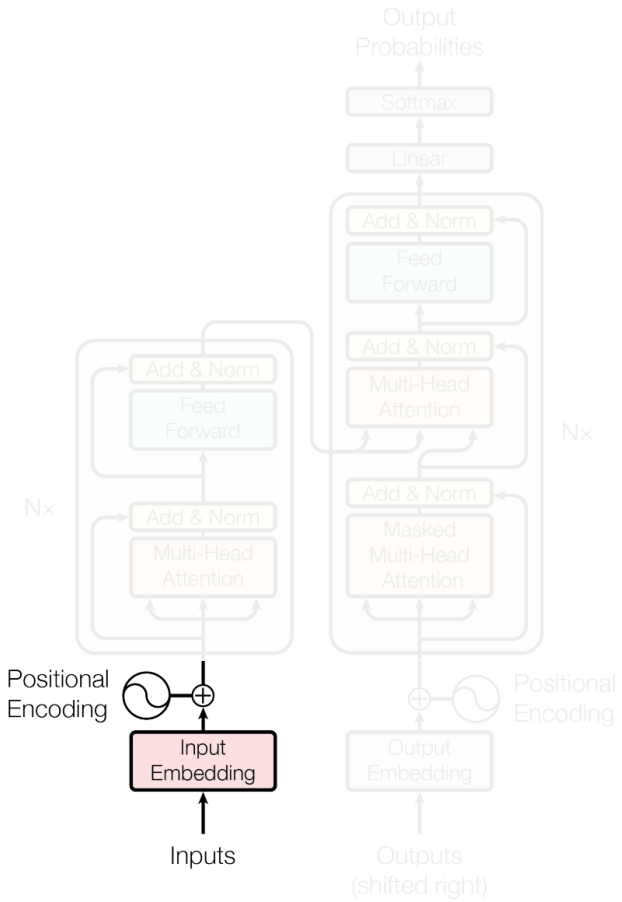


Thanks to Basil Mustafa for slide inspiration

Transformer image source: "Attention Is All You Need" paper

Attention Is All You Need - The Transformer architecture

2017, Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, Illia Polosukhin



Input (Tokenization and) Embedding

Input text is first split into pieces. Can be characters, word, "tokens":

```
"The detective investigated" -> [The_] [detective_] [invest] [igat] [ed_]
```

Tokens are indices into the "vocabulary":

```
[The_] [detective_] [invest] [igat] [ed_] -> [3 721 68 1337 42]
```

Each vocab entry corresponds to a learned d_{model}-dimensional vector.

```
[3 721 68 1337 42] -> [ [0.123, -5.234, ...], [...], [...], [...], [...] ]
```

Positional Encoding

Remember attention is permutation invariant, but language is not!

(“The mouse ate the cat” vs “The cat ate the mouse”)

Need to encode position of each word; just add something.

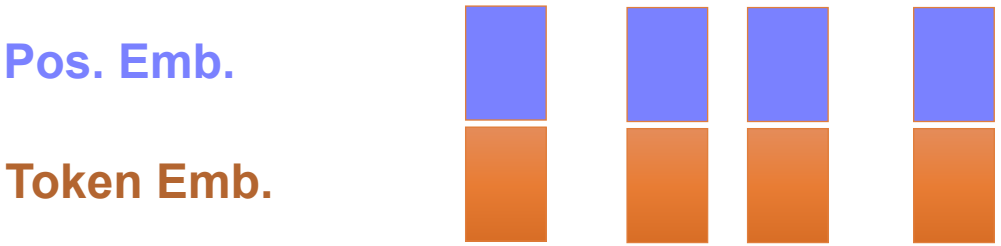
Think `[The_] + 10  [detective_] + 20  [invest] + 30` ... but smarter.

Thanks to Basil Mustafa for slide inspiration

Transformer image source: "Attention Is All You Need" paper

Position embeddings

➢Add another embedding table, indexed by positions from 0 (first position) to some maximum



Positions **[0, 1, 2, 3]**

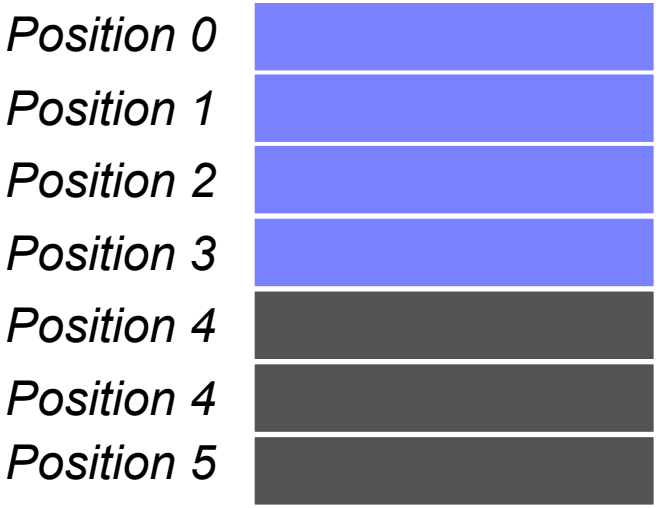
Token indices **[2, 3, 4, 6]**

[“Have”, “a”, “good”, “day”]

Input Sequence: *Have a good day*

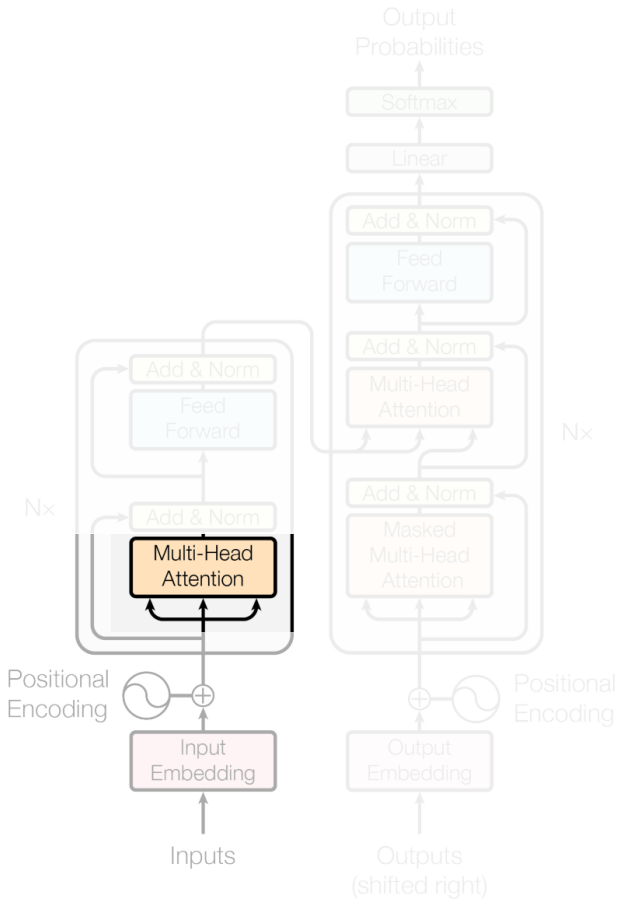
Word Embeddings

Position Embeddings



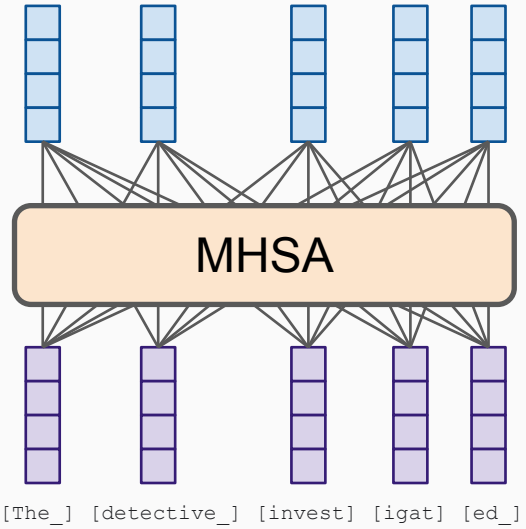
Attention Is All You Need - The Transformer architecture

2017, Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, Illia Polosukhin



Multi-headed Self-Attention

Meaning the input sequence is used to create queries, keys, and values!
Each token can "look around" the whole input, and decide how to update its representation based on what it sees.

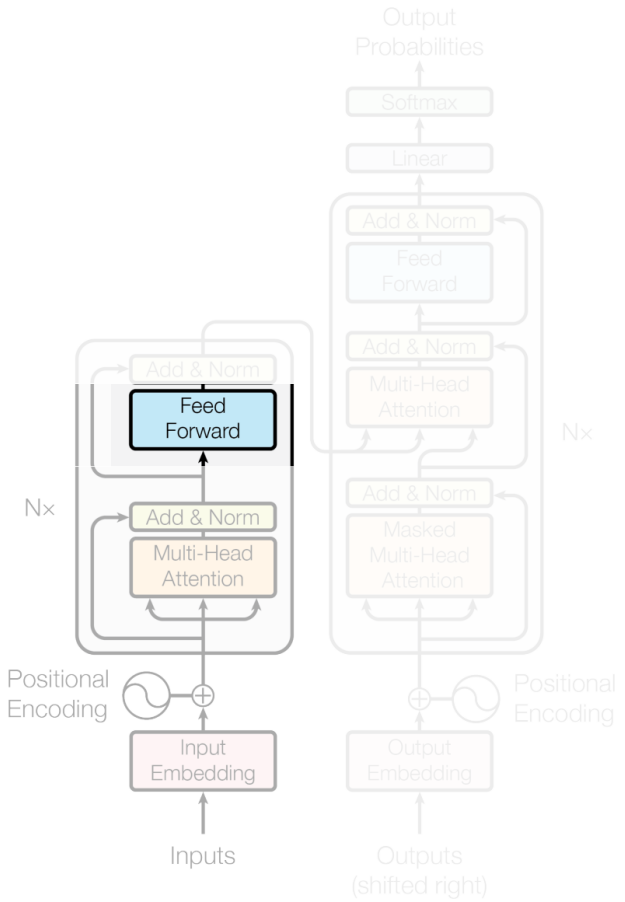


Thanks to Basil Mustafa for slide inspiration

Transformer image source: "Attention Is All You Need" paper

Attention Is All You Need - The Transformer architecture

2017, Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, Illia Polosukhin



Point-wise MLP

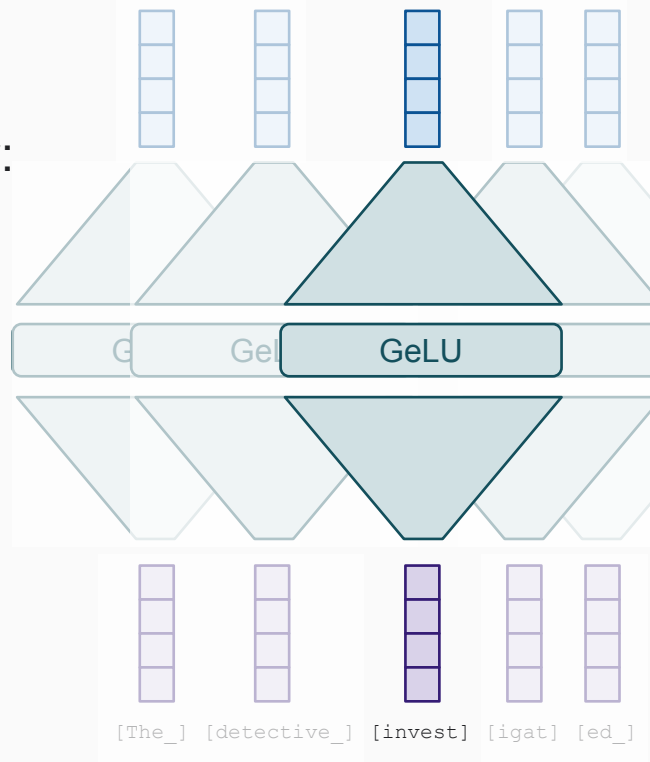
A simple MLP applied to each token individually:

$$z_i = W_2\,\mathrm{GeLU}(W_1 x + b_1) + b_2$$

Think of it as each token pondering for itself about what it has observed previously. There's some weak evidence this is where "world knowledge" is stored, too.

It contains the bulk of the parameters. When people make giant models and sparse/moe, this is what becomes giant.

Some people like to call it 1x1 convolution.

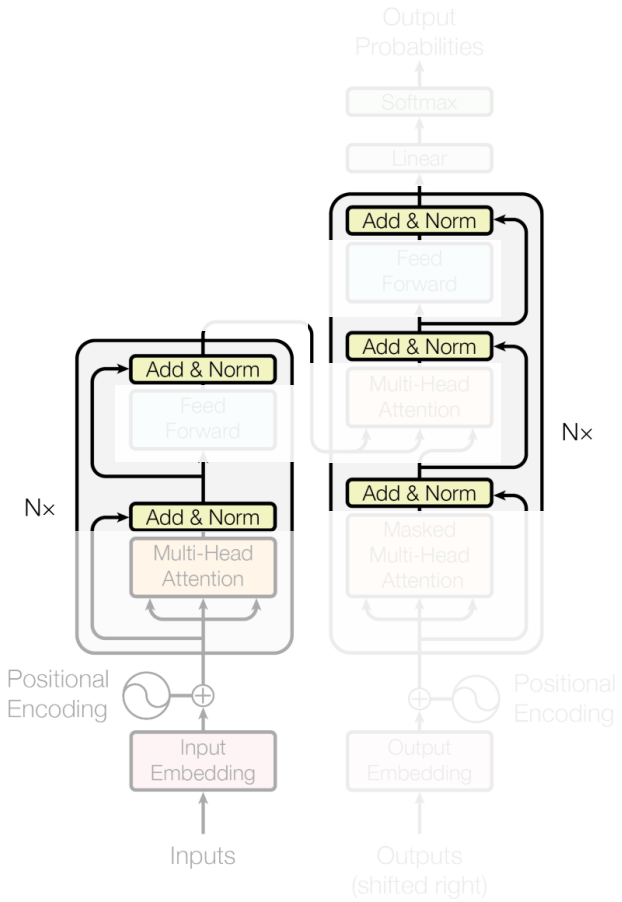


Thanks to Basil Mustafa for slide inspiration

Transformer image source: "Attention Is All You Need" paper

Attention Is All You Need - The Transformer architecture

2017, Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, Illia Polosukhin



Residual connections

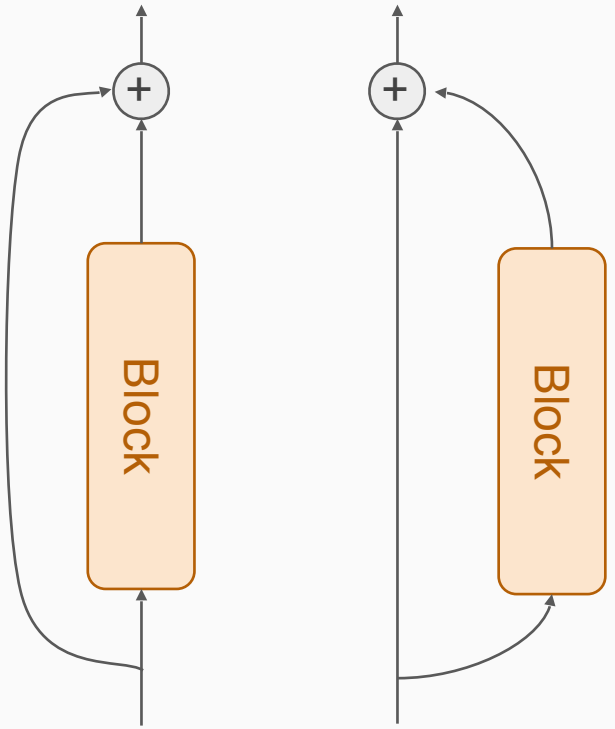
"Skip connection" == "Residual block"

Each module's output has the exact same shape as its input.

Following ResNets, the module computes a "residual" instead of a new value:

$$z_i = \text{Module}(x_i) + x_i$$

This was shown to dramatically improve trainability.



LayerNorm

Normalization also dramatically improves trainability.

There's **post-norm** (original)

$$z_i = \text{LN}(\text{Module}(x_i) + x_i)$$

and **pre-norm** (modern)

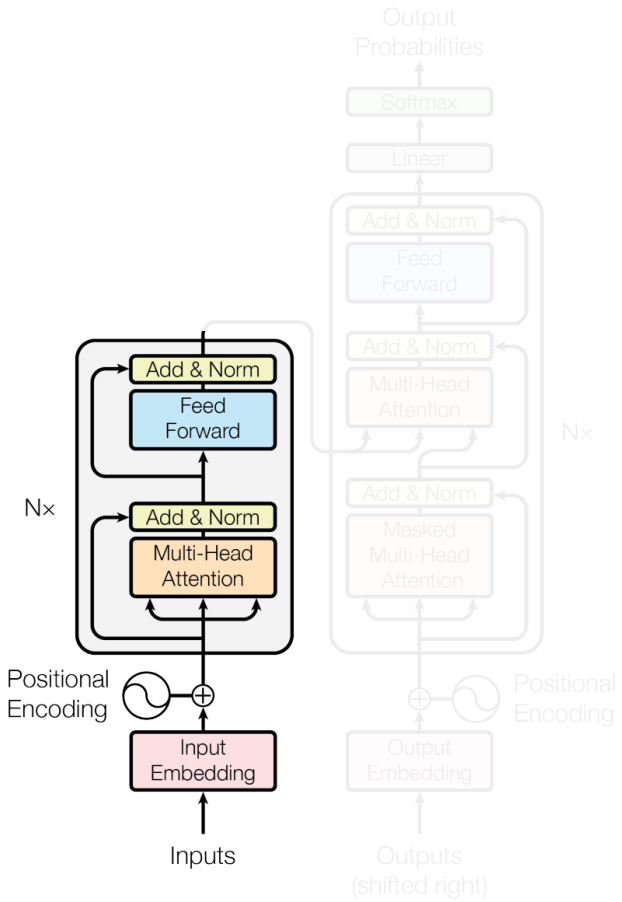
$$z_i = \text{Module}(\text{LN}(x_i)) + x_i$$

Thanks to Basil Mustafa for slide inspiration

Transformer image source: "Attention Is All You Need" paper

Attention Is All You Need - The Transformer architecture

2017, Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, Illia Polosukhin



Encoding / Encoder

Since input and output shapes are identical, we can stack N such blocks.

Typically, N=6 ("base"), N=12 ("large") or more.

Encoder output is a "heavily processed" (think: "high level, contextualized") version of the input tokens, i.e. a sequence.

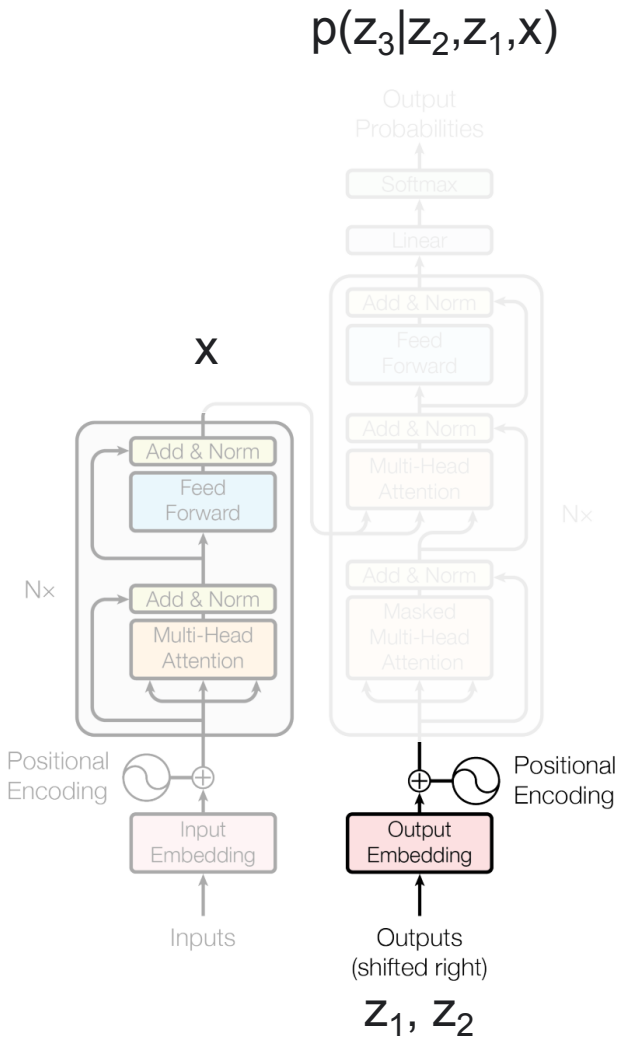
This has nothing to do with the requested output yet (think: translation). That comes with the decoder.

Thanks to Basil Mustafa for slide inspiration

Transformer image source: "Attention Is All You Need" paper

Attention Is All You Need - The Transformer architecture

2017, Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, Illia Polosukhin



Thanks to Basil Mustafa for slide inspiration

Transformer image source: "Attention Is All You Need" paper

Decoding / the Decoder (alternatively Generating / the Generator)

What we want to model: $p(z|x)$

for example, in translation: $p(z \mid$ "the detective investigated"$) \forall z$

Seems impossible at first, but we can *exactly* decompose into tokens:

$$p(z|x) = p(z_1|x)\, p(z_2|z_1,x)\, p(z_3|z_2,z_1,x)...$$

Meaning, we can generate the answer one token at a time.
Each p is a full pass through the model.

For generating $p(z_3|z_2,z_1,x)$:

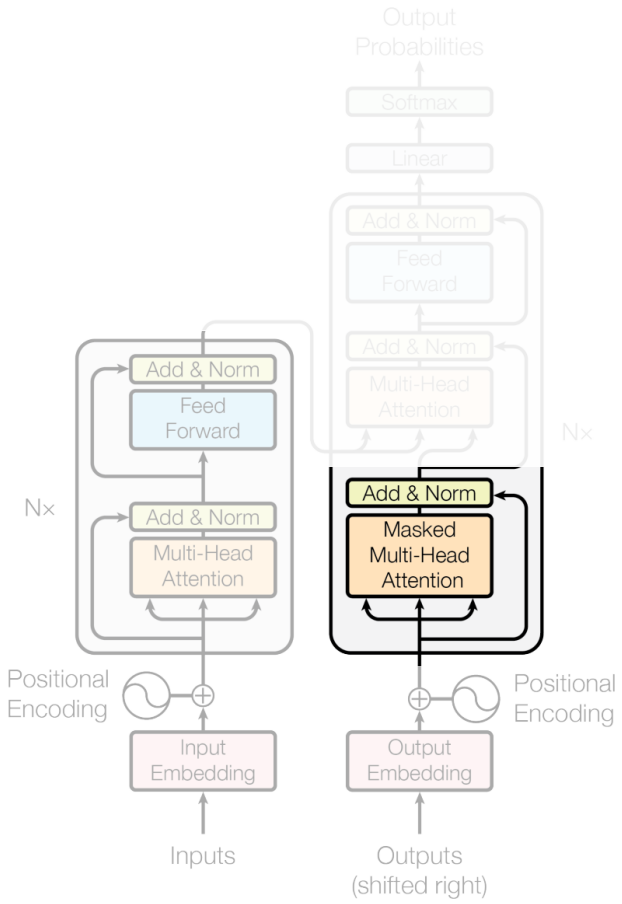
x comes from the encoder,

z_1, z_2 is what we have predicted so far, goes into the decoder.

Once we have p(z|x) we still need to actually sample a sentence such as "le détective a enquêté". Many strategies: greedy, beam-search, ...

Attention Is All You Need - The Transformer architecture

2017, Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, Illia Polosukhin



At training time: Masked self-attention

This is regular self-attention as in the encoder, to process what's been decoded so far, eg z_2,z_1 in $p(z_3|z_2,z_1,x)$, but with a trick...

If we had to train on one single $p(z_3|z_2,z_1,x)$ at a time: SLOW!

Instead, train on all $p(z_i|z_{1:i},x)$ simultaneously.

How? In the attention weights for z_i, set all entries i:N to 0.

This way, each token only sees the already generated ones.

At generation time

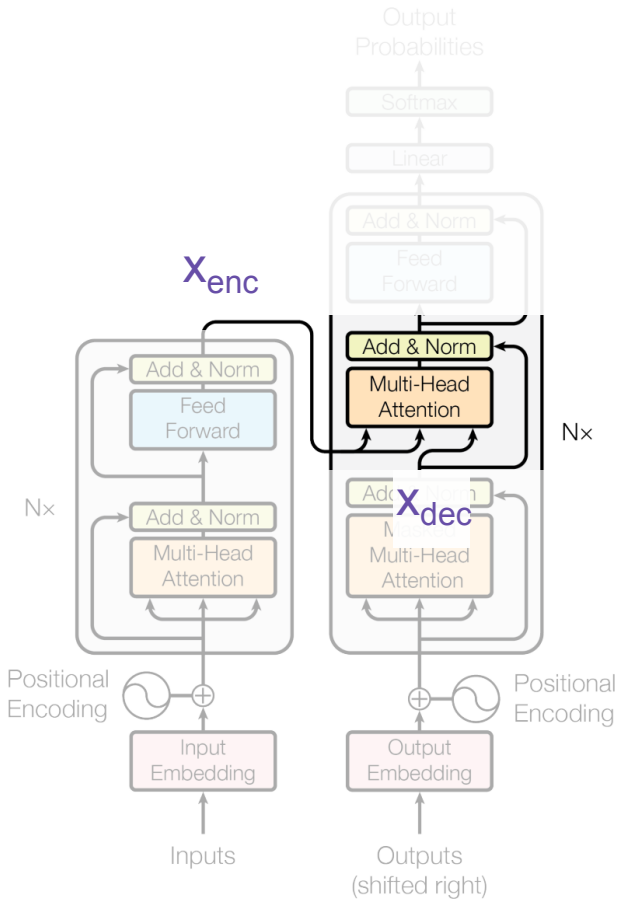
There is no such trick. We need to generate one z_i at a time. This is why autoregressive decoding is extremely slow.

Thanks to Basil Mustafa for slide inspiration

Transformer image source: "Attention Is All You Need" paper

Attention Is All You Need - The Transformer architecture

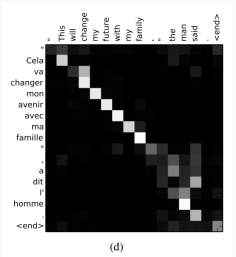
2017, Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, Illia Polosukhin



"Cross" attention

Each decoded token can "look at" the encoder's output:

$$\text{Attn}(q=W_q x_{dec},\ k=W_k x_{enc},\ v=W_v x_{enc})$$



This is the same as in the 2014 paper.
This is where |x in $p(z_3|z_2,z_1,x)$ comes from.

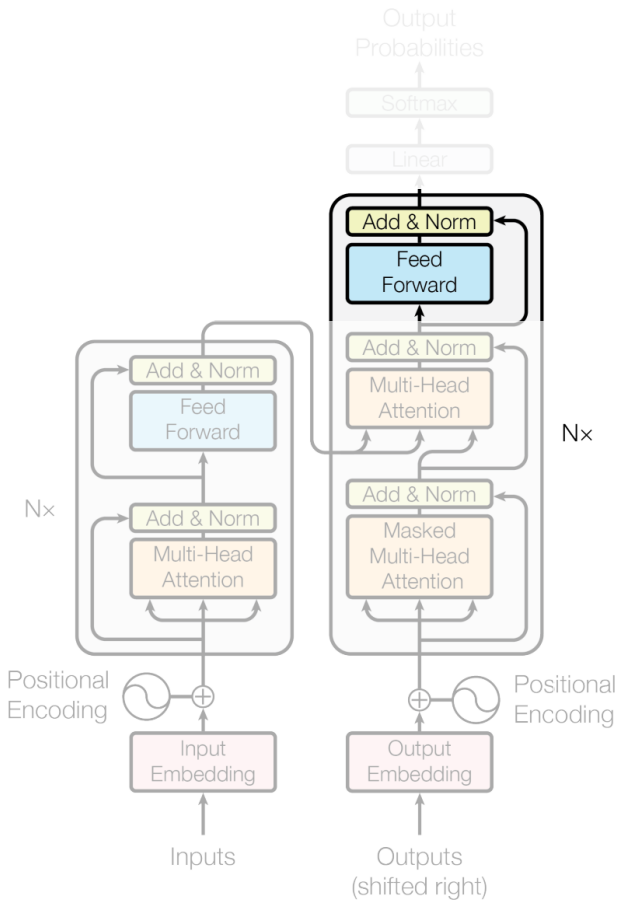
Because self-attention is so widely used, people have started just calling it "attention".
Hence, we now often need to explicitly call this "cross attention".

Thanks to Basil Mustafa for slide inspiration

Transformer image source: "Attention Is All You Need" paper ; heatmap image source: "Neural Machine Translation by Jointly Learning to Align and Translate" paper

Attention Is All You Need - The Transformer architecture

2017, Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, Illia Polosukhin



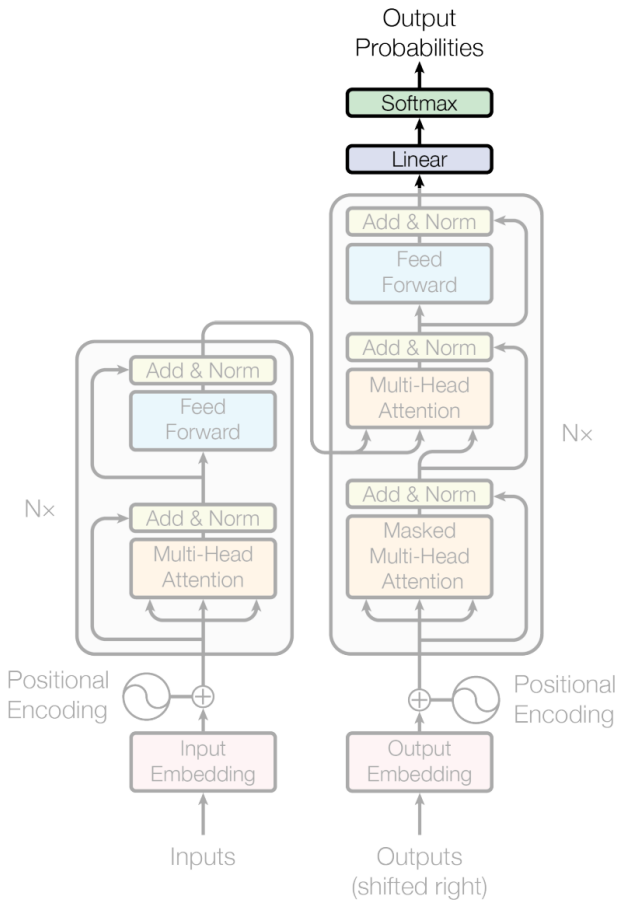
Feedforward and stack layers.

Thanks to Basil Mustafa for slide inspiration

Transformer image source: "Attention Is All You Need" paper

Attention Is All You Need - The Transformer architecture

2017, Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, Illia Polosukhin



Output layer

Assume we have already generated K tokens, generate the next one.

The decoder was used to gather all information necessary to predict a probability distribution for the next token (K), over the whole vocab.

Simple:

- linear projection of token K
- SoftMax normalization

Thanks to Basil Mustafa for slide inspiration

Transformer image source: "Attention Is All You Need" paper

Attention Is All You Need - Summary and results

2017, Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, Illia Polosukhin

Table 2: The Transformer achieves better BLEU scores than previous state-of-the-art models on the English-to-German and English-to-French newstest2014 tests at a fraction of the training cost.

Model	BLEU		Training Cost (FLOPs)	
	EN-DE	EN-FR	EN-DE	EN-FR
ByteNet [18]	23.75			
Deep-Att + PosUnk [39]		39.2		$1.0 \cdot 10^{20}$
GNMT + RL [38]	24.6	39.92	$2.3 \cdot 10^{19}$	$1.4 \cdot 10^{20}$
ConvS2S [9]	25.16	40.46	$9.6 \cdot 10^{18}$	$1.5 \cdot 10^{20}$
MoE [32]	26.03	40.56	$2.0 \cdot 10^{19}$	$1.2 \cdot 10^{20}$
Deep-Att + PosUnk Ensemble [39]		40.4		$8.0 \cdot 10^{20}$
GNMT + RL Ensemble [38]	26.30	41.16	$1.8 \cdot 10^{20}$	$1.1 \cdot 10^{21}$
ConvS2S Ensemble [9]	26.36	**41.29**	$7.7 \cdot 10^{19}$	$1.2 \cdot 10^{21}$
Transformer (base model)	27.3	38.1	**$3.3 \cdot 10^{18}$**	
Transformer (big)	**28.4**	**41.8**	$2.3 \cdot 10^{19}$	

Animation source: https://ai.googleblog.com/2017/08/transformer-novel-neural-network.html
Table source: "Attention Is All You Need" paper

Keywords

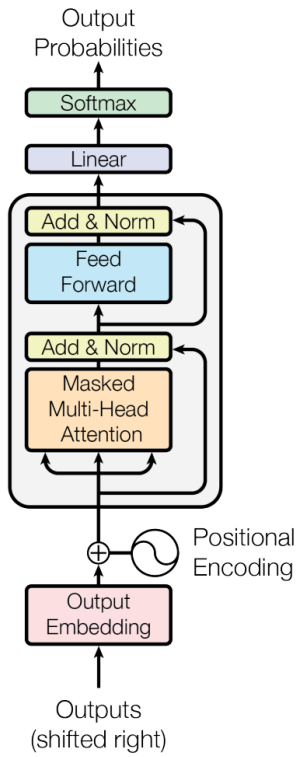
- Attention
- Transformer architecture
- **Language models**
- Transformers across modalities

The first (1.5th) big takeover:

Language Modeling / NLP

Decoder-only GPT

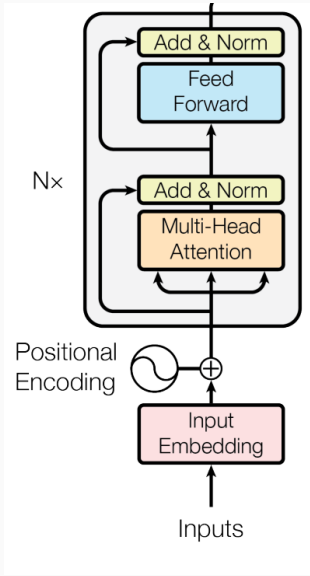
[sat_]



[START] [The_] [cat_]

Encoder-only BERT

[*] [*] **[sat_]** [*] **[the_]** [*]



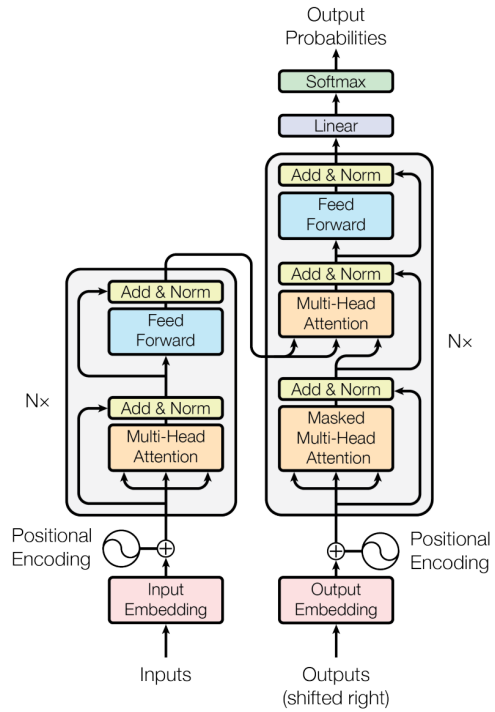
[The_] [cat_] **[MASK]** [on_] **[MASK]** [mat_]

Enc-Dec T5

Das ist gut.

A storm in Attala caused 6 victims.

This is not toxic.



Translate EN-DE: This is good.

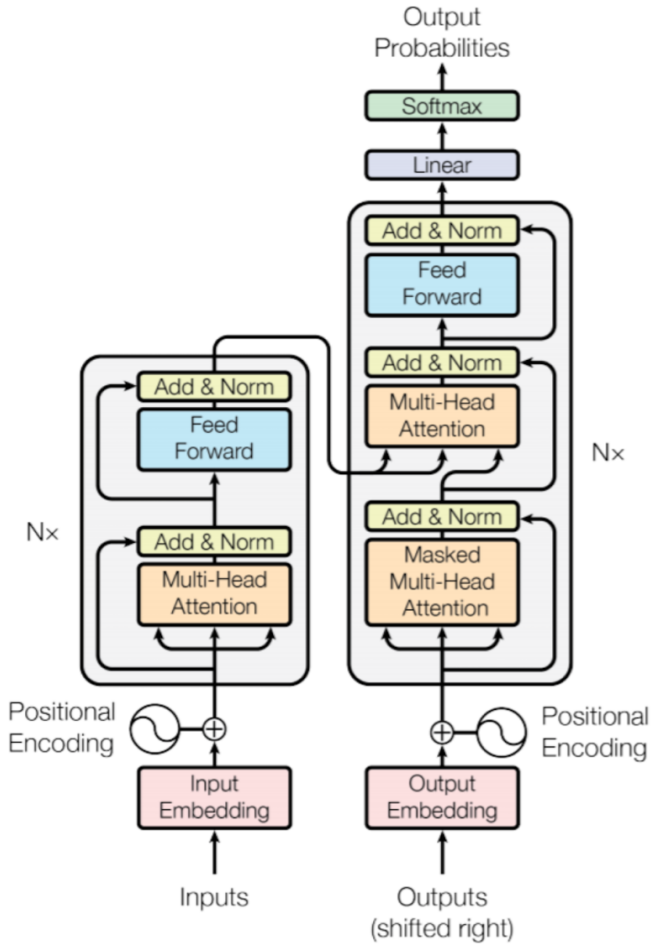
Summarize: state authorities dispatched…

Is this toxic: You look beautiful today!

Transformer image source: "Attention Is All You Need" paper

Main advantages over RNNs

- **RNN Drawbacks**
  - Long-term dependencies still challenging
  - Sequential dependencies, thus poorly parallelizable
- **Transformers**
  - **Attention mechanism deals with long-term dependencies**
  - **Transformers can process entire sequences in parallel**



Keywords

- Attention
- Transformer architecture
- Language models
- **Transformers across modalities**

The second big takeover:

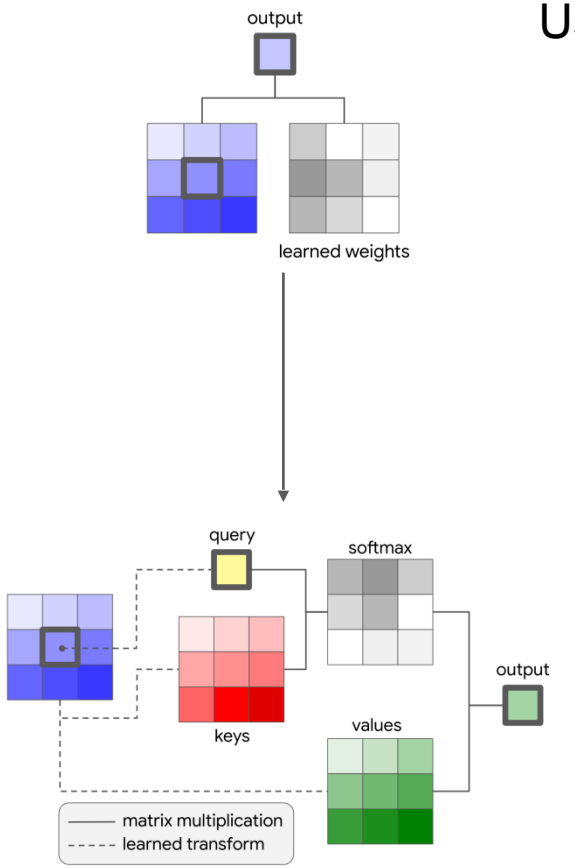
Computer Vision

Many prior works attempted to introduce self-attention at the pixel level.

For 224px², that's 50k sequence length, too much!

Previous approaches

1. On pixels, but locally or factorized



Usually replaces 3x3 conv in ResNet:

stage	output	ResNet-50	**LR-Net-50 (7×7, m=8)**
res1	112×112	7×7 conv, 64, stride 2	**1×1, 64** **7×7 LR, 64, stride 2**
res2	56×56	3×3 max pool, stride 2	3×3 max pool, stride 2
		[1×1, 64 3×3 conv, 64 1×1, 256] ×3	[1×1, 100 **7×7 LR, 100** 1×1, 256] ×3
res3	28×28	[1×1, 128 3×3 conv, 128 1×1, 512] ×4	[1×1, 200 **7×7 LR, 200** 1×1, 512] ×4
res4	14×14	[1×1, 256 3×3 conv, 256 1×1, 1024] ×6	[1×1, 400 **7×7 LR, 400** 1×1, 1024] ×6
res5	7×7	[1×1, 512 3×3 conv, 512 1×1, 2048] ×3	[1×1, 800 **7×7 LR, 800** 1×1, 2048] ×3
	1×1	global average pool 1000-d fc, softmax	global average pool 1000-d fc, softmax
# params		**25.5**×10^6	**23.3**×10^6
FLOPs		**4.3**×10^9	**4.3**×10^9

Results:

Are usually "meh", nothing to call home about

Do not justify increased complexity

Do not justify slowdown over convolutions

Examples:

Non-local NN (Wang et.al. 2017)

SASANet (Stand-Alone Self-Attention in Vision Models)

HaloNet (Scaling Local Self-Attn for Parameter Efficient...)

LR-Net (Local Relation Networks for Image Recognition)

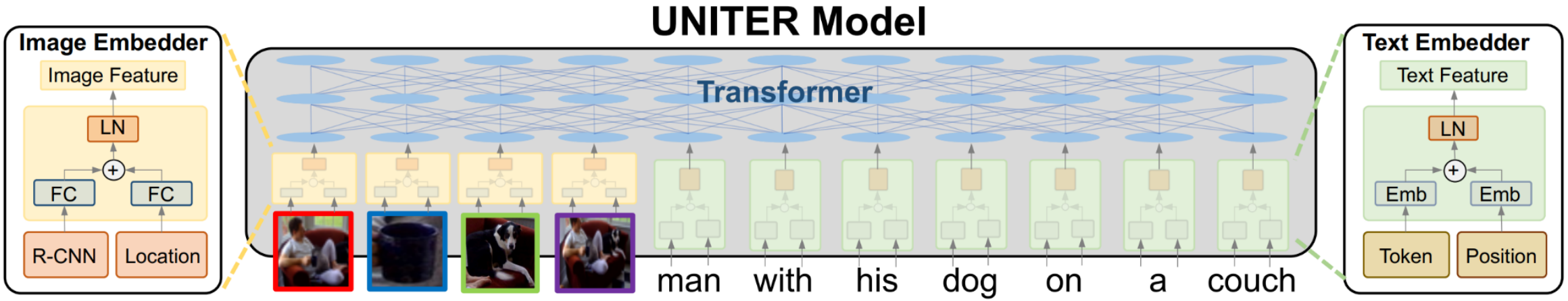
SANet (Exploring Self-attention for Image Recognition)

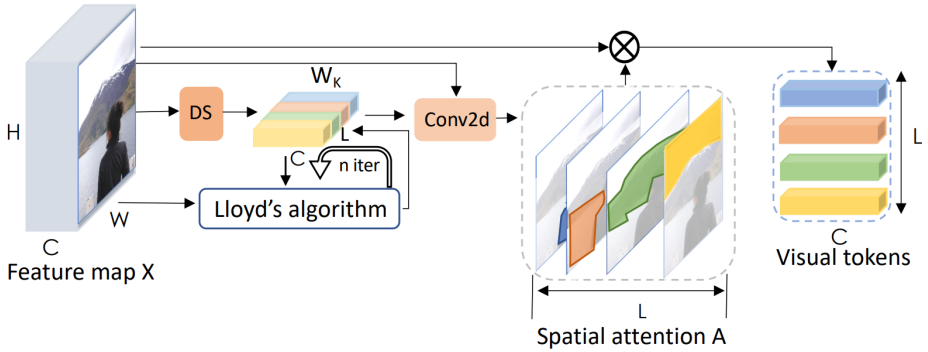
...

Image credit: Stand-Alone Self-Attention in Vision Models by Ramachandran et.al.
Image credit: Local Relation Networks for Image Recognition by Hu et.al.

Previous approaches

2. Globally, after/inside a full-blown CNN, or even detector/segmenter!





Cons:
result is highly complex, often multi-stage trained architecture.
not from pixels, i.e. transformer can't "learn to fix" the (often frozen!) CNN's mistakes.

Examples:
DETR (Carion, Massa et.al. 2020)
UNITER (Chen, Li, Yu et.al. 2019)
VisualBERT (Li et.al. 20190)
Visual Transformers (Wu et.al. 2020)
ViLBERT (Lu et.al. 2019)
etc...

Image credit: UNITER: UNiversal Image-TExt Representation Learning by Chen et.al.
Image credit: Visual Transformers: Token-based Image Representation and Processing for Computer Vision by Wu et.al.

An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale

2020, A Dosovitskiy, L Beyer, A Kolesnikov, D Weissenborn, X Zhai, T Unterthiner, M Dehghani, M Minderer, G Heigold, S Gelly, J Uszkoreit, N Houlsby

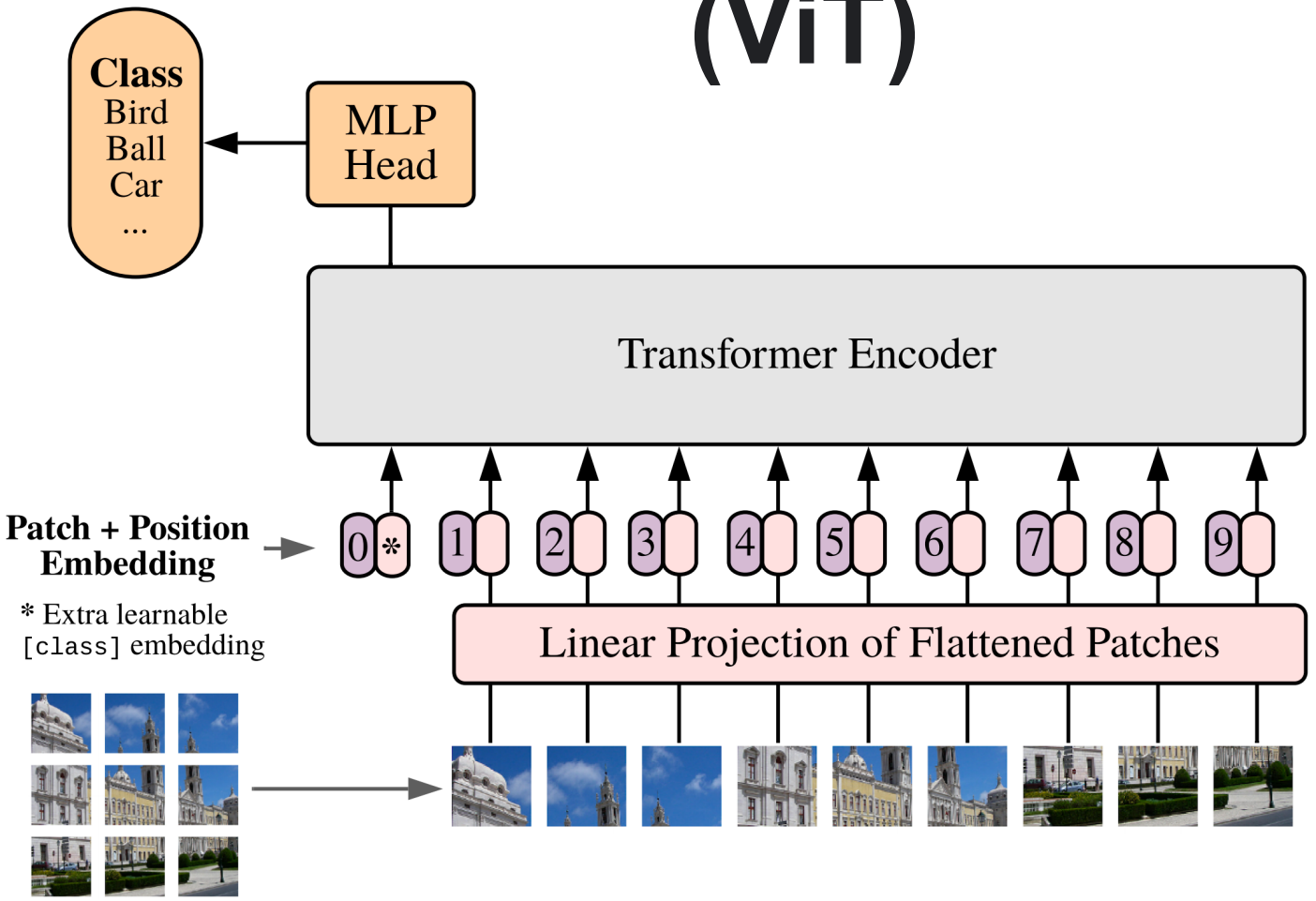
Many prior works attempted to introduce self-attention at the pixel level.

For 224px², that's 50k sequence length, too much!

Thus, most works restrict attention to local pixel neighborhoods, or as high-level mechanism on top of detections.

The **key breakthrough** in using the full Transformer architecture, standalone, was to **"tokenize" the image** by **cutting it into patches** of 16px², and treating each patch as a token, e.g. embedding it into input space.

Vision Transformer (ViT)



The third big takeover:

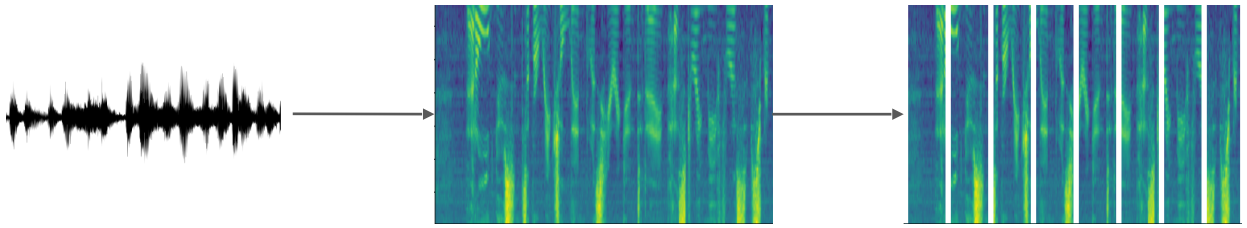
Speech

Conformer: Convolution-augmented Transformer for Speech Recognition

2020, A Gulati, J Qin, C-C Chiu, N Parmar, Y Zhang, J Yu, W Han, S Wang, Z Zhang, Y Wu, R Pang

[igat]

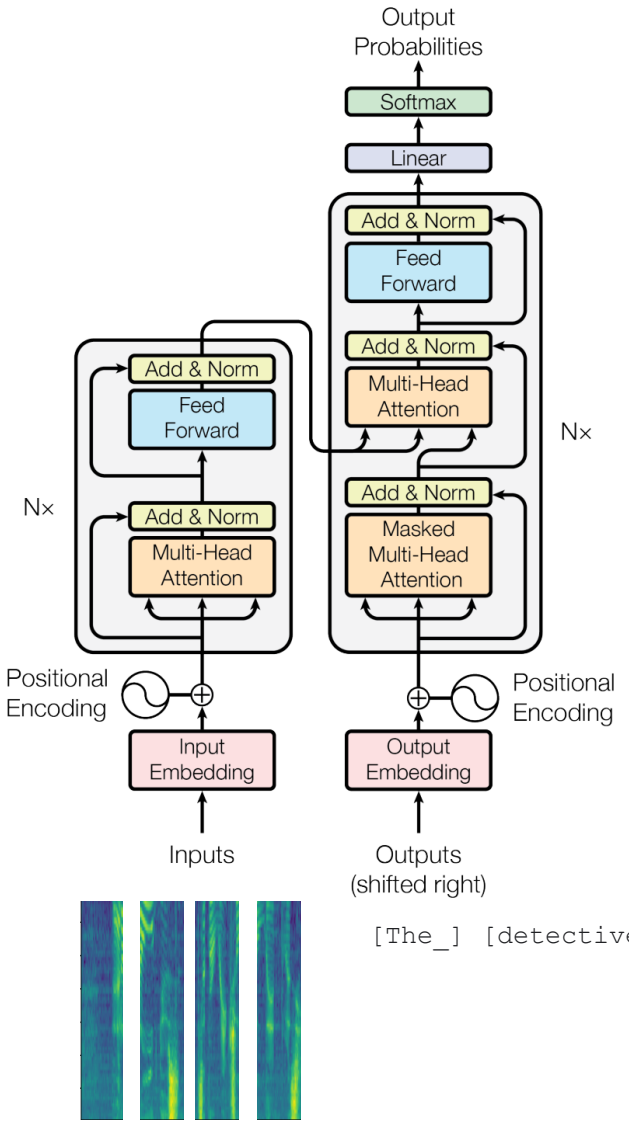
Largely the same story as in computer vision.
But with spectrograms instead of images.



Conformer adds a third type of block using convolutions, and slightly reorder blocks, but overall very transformer-like.

Exists as encoder-decoder variant, or as encoder-only variant with CTC loss.

UPDATE Sept 2022: Plain transformer model in “Whisper” by
A Radford, J W Kim, T Xu, G Brockman, C McLeavey, I Sutskever



Transformer image source: "Attention Is All You Need" paper

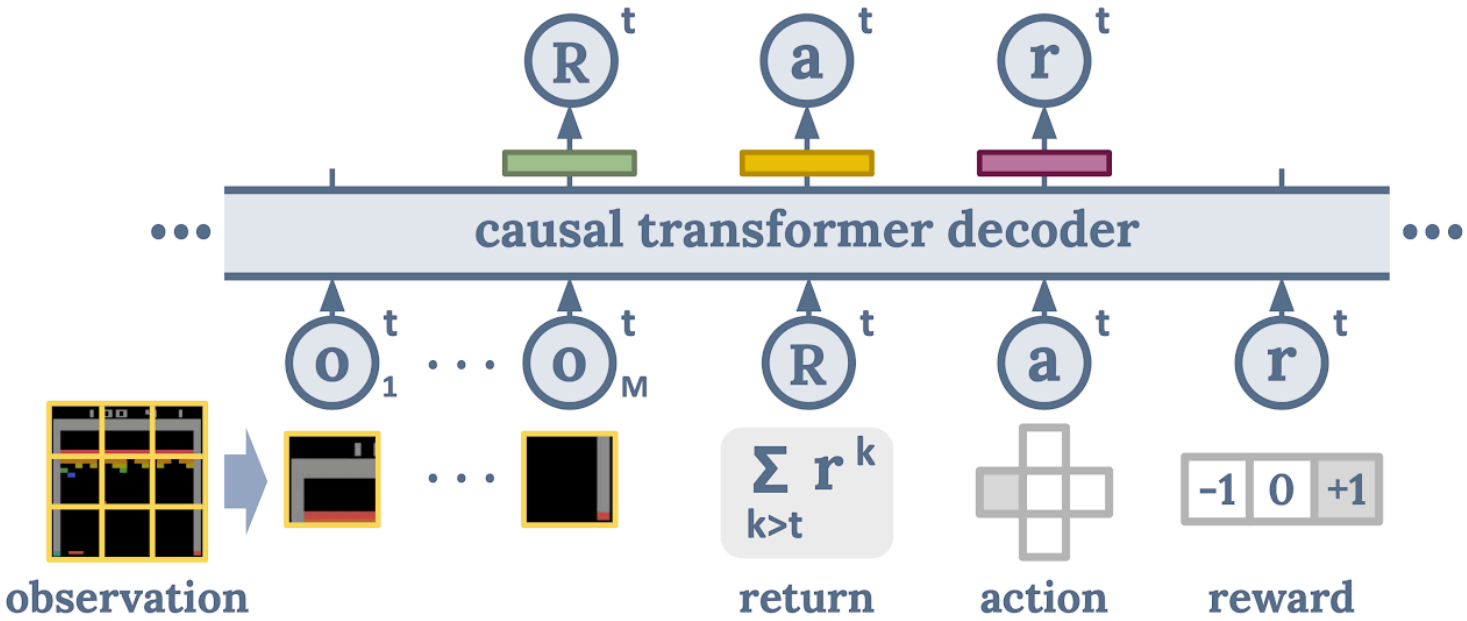
The fourth big takeover:

Reinforcement Learning

Decision Transformer: Reinforcement Learning via Sequence Modeling

2021, L Chen, K Lu, A Rajeswaran, K Lee, A Grover, M Laskin, P Abbeel, A Srinivas, I Mordatch

Cast the (supervised/offline) RL problem into a sequence ("language") modeling task:



Can generate/decode sequences of actions with desired return (eg skill)

The trick is prompting: `"The following is a trajectory of an expert player: [obs] ..."`

Slide credit: Igor Mordatch

The Transformer's
Unification of communities

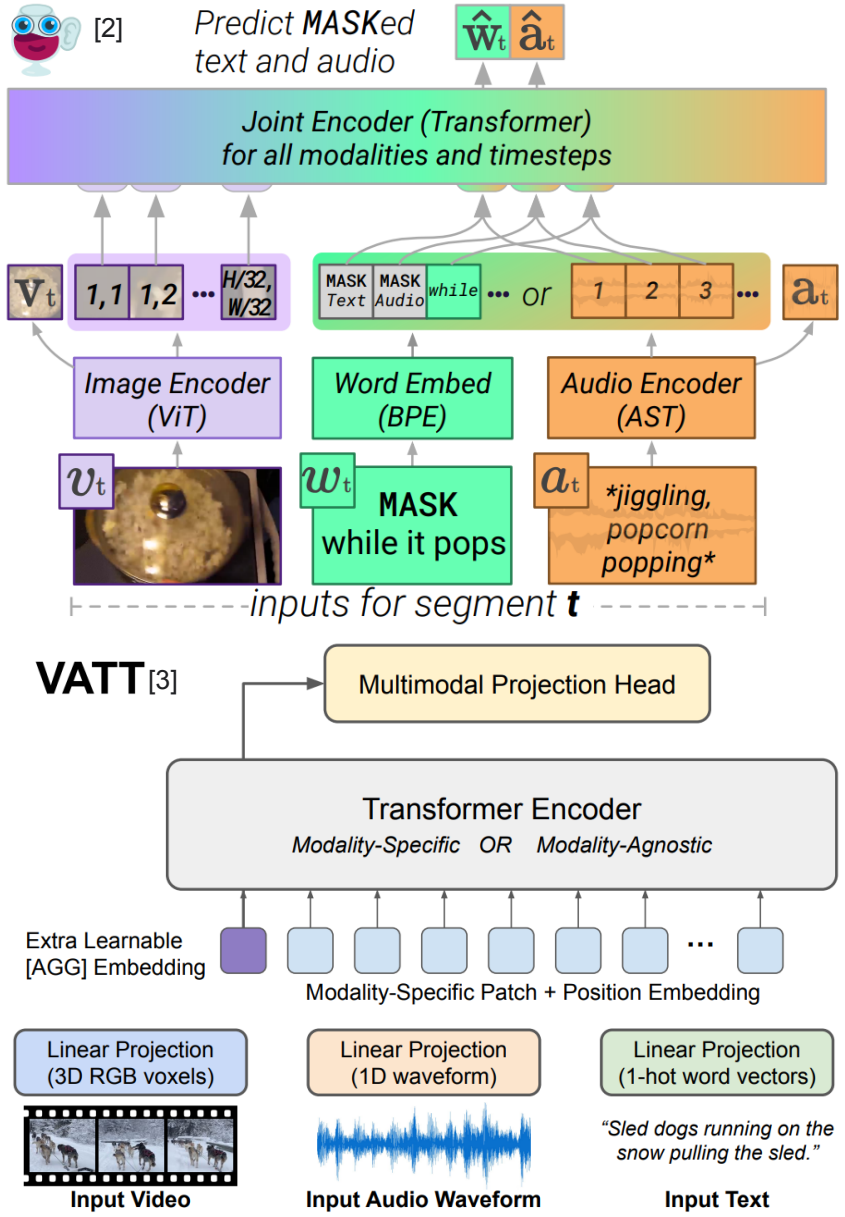
Anything you can tokenize, you can feed to Transformer

ca 2021 and onwards

Tokenize different modalities each in their own way (some kind of "patching"), and send them all jointly into a Transformer...

Seems to just work...
Currently an explosion of works doing this!

[1]



Images from:
[1] LIMoE by B Mustafa, C Riquelme, J Puigcerver, R Jenatton, N Houlsby
[2] MERLOT Reserve by R Zellers, J Lu, X Lu, Y Yu, Y Zhao, M Salehi, A Kusupati, J Hessel, A Farhadi, Y Choi
[3] VATT by H Akbari, L Yuan, R Qian, W-H Chuang, S-F Chang, Y Cui, B Gong

A note on
Efficient Transformers

A note on Efficient Transformers

The self-attention operation complexity is $O(N^2)$ for sequence length N.

We'd like to use large N:

- Whole articles or books
- Full video movies
- High resolution images

Many O(N) approximations to the full self-attention have been proposed in the past two years.

Unfortunately, none provides a clear improvement. They always trade-off between speed and quality.

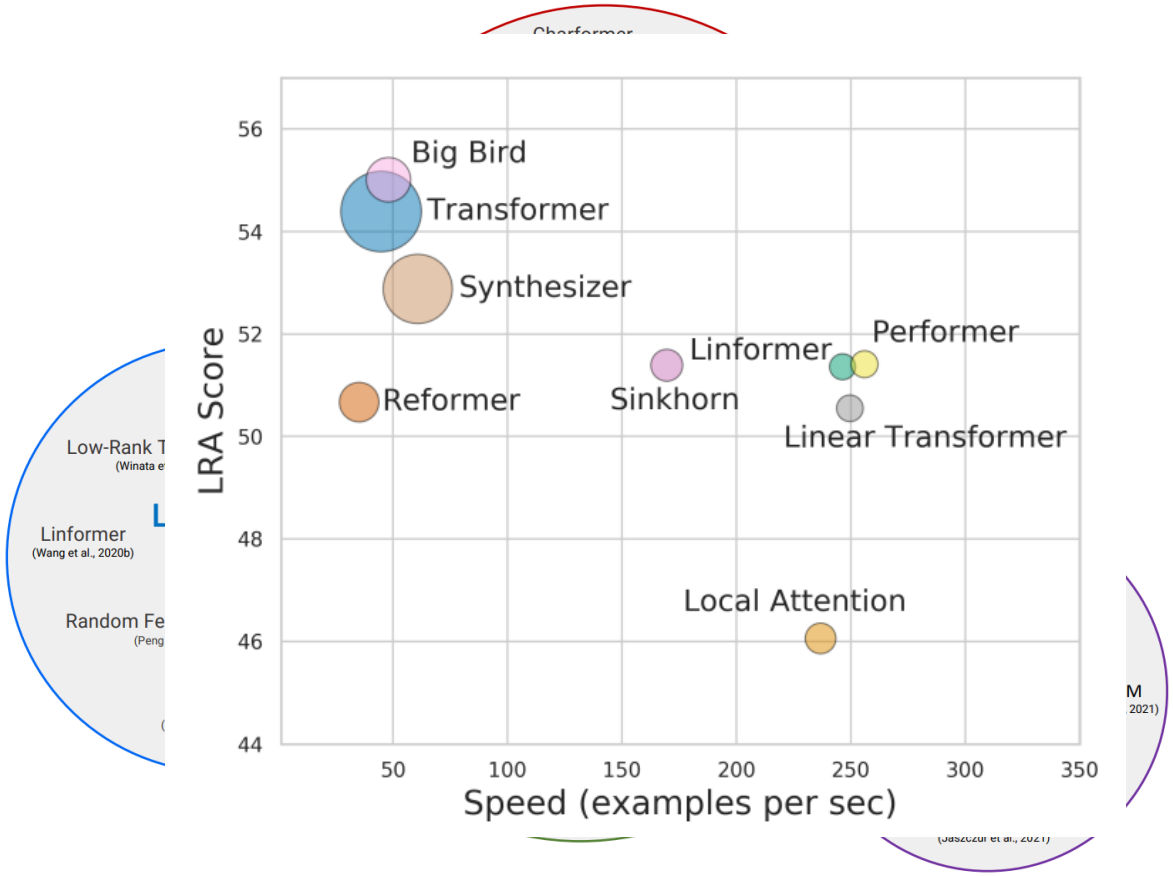


Figure 2: Taxonomy of Efficient Transformer Architectures.

Based on "Efficient Transformers: A Survey" by Yi Tay, Mostafa Dehghani, Dara Bahri, Donald Metzler
and "Long Range Arena: A Benchmark for Efficient Transformers" by Y Tay, M Dehghani, S Abnar, Y Shen, D Bahri, P Pham, J Rao, L Yang, S Ruder, D Metzler

Summary

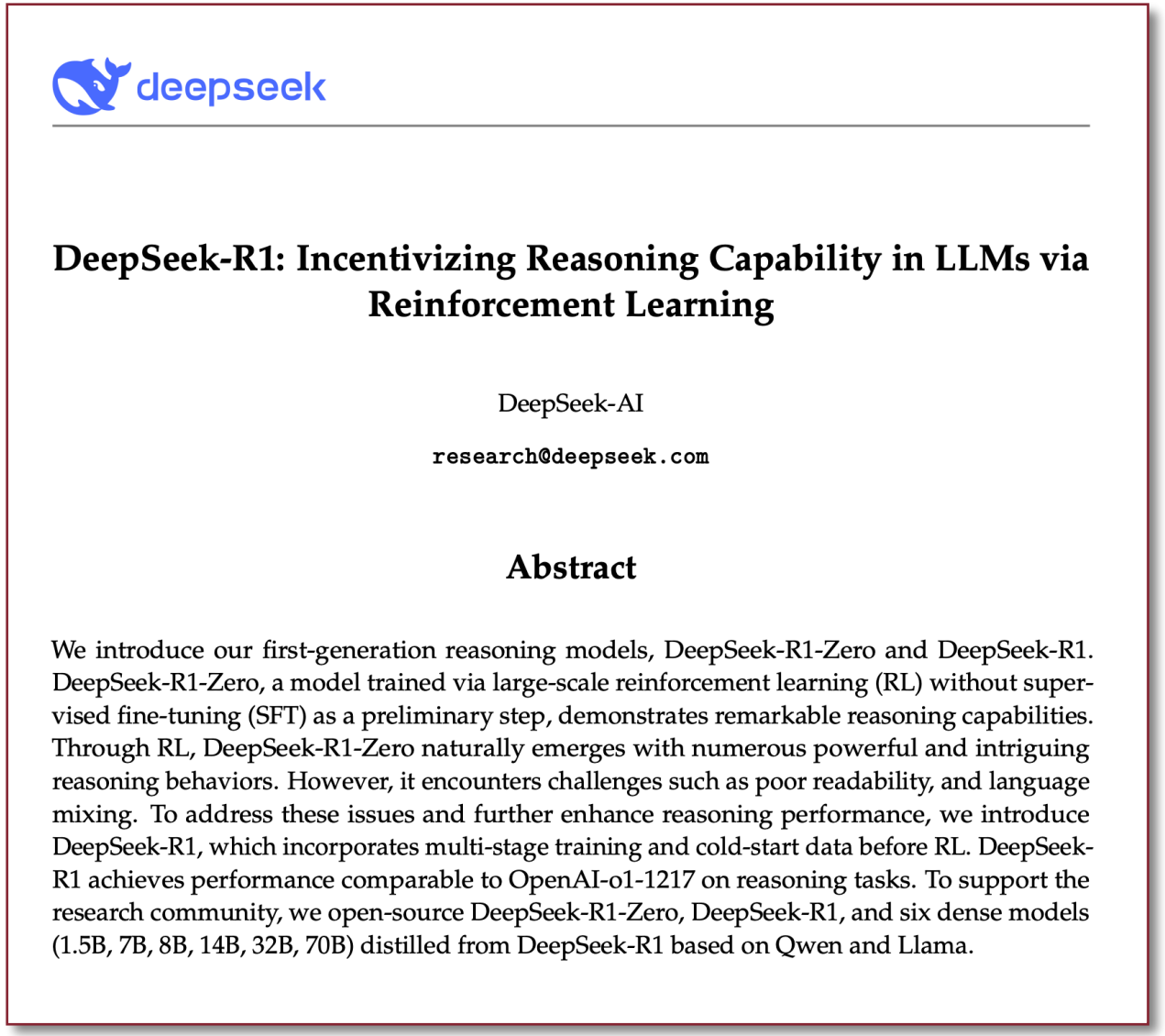
- **Word embeddings** are a shortcut to one-hot encoding + weight layer
- **Attention mechanism** selectively focuses on different parts of the input.
- **Transformers** are organized in blocks, with each block being composed of (A) **a self-attention layer** and (B) **a feedforward layer** + interleaved **residual connections** + **normalization layers**
- **Transformers** are very powerful and spread across communities, as they **can handle long-range dependencies** and are **very well parallelizable**.
- *Some considerations:*
  - The position of each token must be encoded explicitly.
  - Attention scores require $O(N^2)$ time and space, for seq. length N.
  - **One more thing: Transformers are data hungry**

Transformers • Keywords

- One-hot encoding
- Word embeddings
- Attention
- Transformers
- Transformers across modalities

Outlook

- Large Language Models
- A closer look at a recent Transformer-based language model



deepseek

DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning

DeepSeek-AI

research@deepseek.com

Abstract

We introduce our first-generation reasoning models, DeepSeek-R1-Zero and DeepSeek-R1. DeepSeek-R1-Zero, a model trained via large-scale reinforcement learning (RL) without supervised fine-tuning (SFT) as a preliminary step, demonstrates remarkable reasoning capabilities. Through RL, DeepSeek-R1-Zero naturally emerges with numerous powerful and intriguing reasoning behaviors. However, it encounters challenges such as poor readability, and language mixing. To address these issues and further enhance reasoning performance, we introduce DeepSeek-R1, which incorporates multi-stage training and cold-start data before RL. DeepSeek-R1 achieves performance comparable to OpenAI-o1-1217 on reasoning tasks. To support the research community, we open-source DeepSeek-R1-Zero, DeepSeek-R1, and six dense models (1.5B, 7B, 8B, 14B, 32B, 70B) distilled from DeepSeek-R1 based on Qwen and Llama.

How’s my teaching?

https://www.admonymous.co/lukasgalke

Large Language Models

Lukas Galke Poech

Plan for today

1. Recap: Transformers
2. Large Language Models
3. DeepSeek

Recap: Transformers

→ Blackboard.

Language Modeling

Predict the next word.

Estimate $p\left(x_t \middle| x_{t-1}, x_{t-2}, \ldots, x_0\right)$

By adding a linear *language modeling head* on top of the transformer. The language modeling head is just a linear layer with $N_{\text{vocabulary}}$ outputs, followed by softmax & cross-entropy loss

Large Language Models

Three main training stages

1. Pre-training
2. Supervised Finetuning
3. Reinforcement learning

Pretraining

Language modeling objective

Large amounts of low-to-medium-quality data

Supervised Finetuning (SFT)

Language modeling objective

Small amounts of high-quality data

Reinforcement Learning from Human Feedback (RLHF)

1. Language model generate utterances
2. Humans judge the responses
3. Reward model is trained to "simulate" human judges
4. Language model is updated to increase reward

LLM Paperstorm

LLM Paperstorm

Goal: Investigate the **model architectures** of recent LLMs.

Paper pool:

- Gemma 4
- DeepSeek V3.2
- Qwen3

What to do?

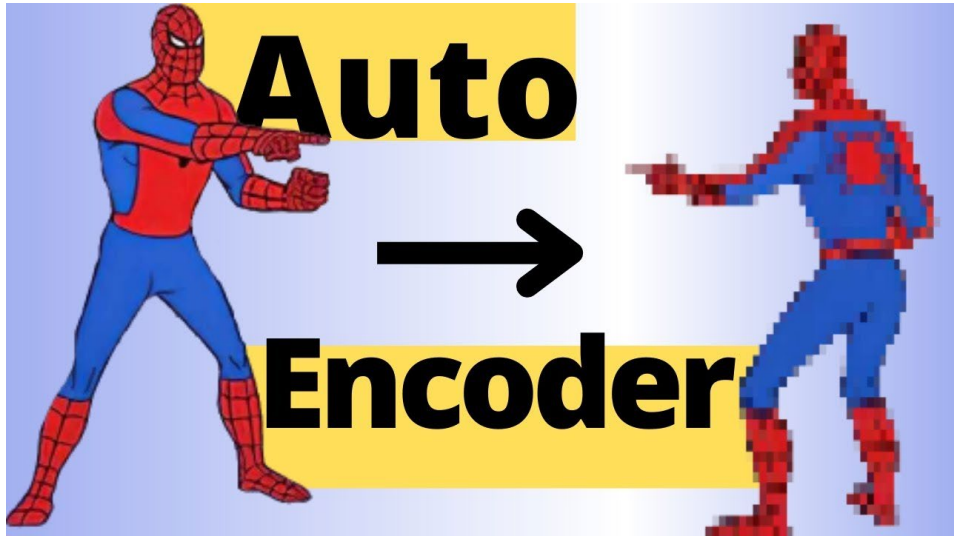
- Groups of 2-3 people.
- 1 paper per group.
- 20 Minutes, then we bring together results.
- **Hint**: You may need to look into previous papers!
- **Tools**: You may use AI assistance as you desire.

Department of Mathematics
and Computer Science

~~DSK809 Deep Learning:~~

Autoencoders

Anton D. Lautrup
lautrup@imada.sdu.dk



Overview

01

Autoencoders

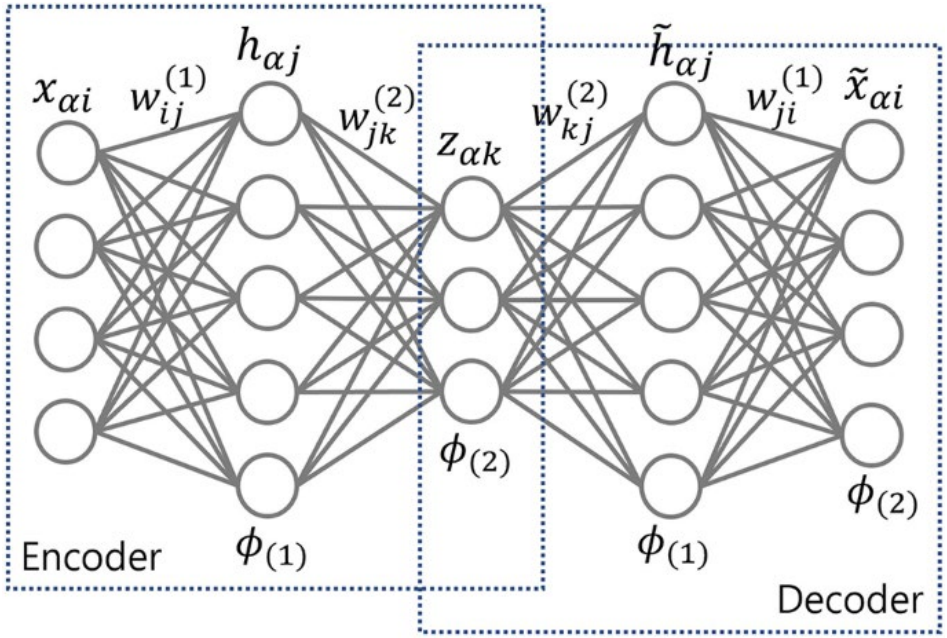
02

Latent space

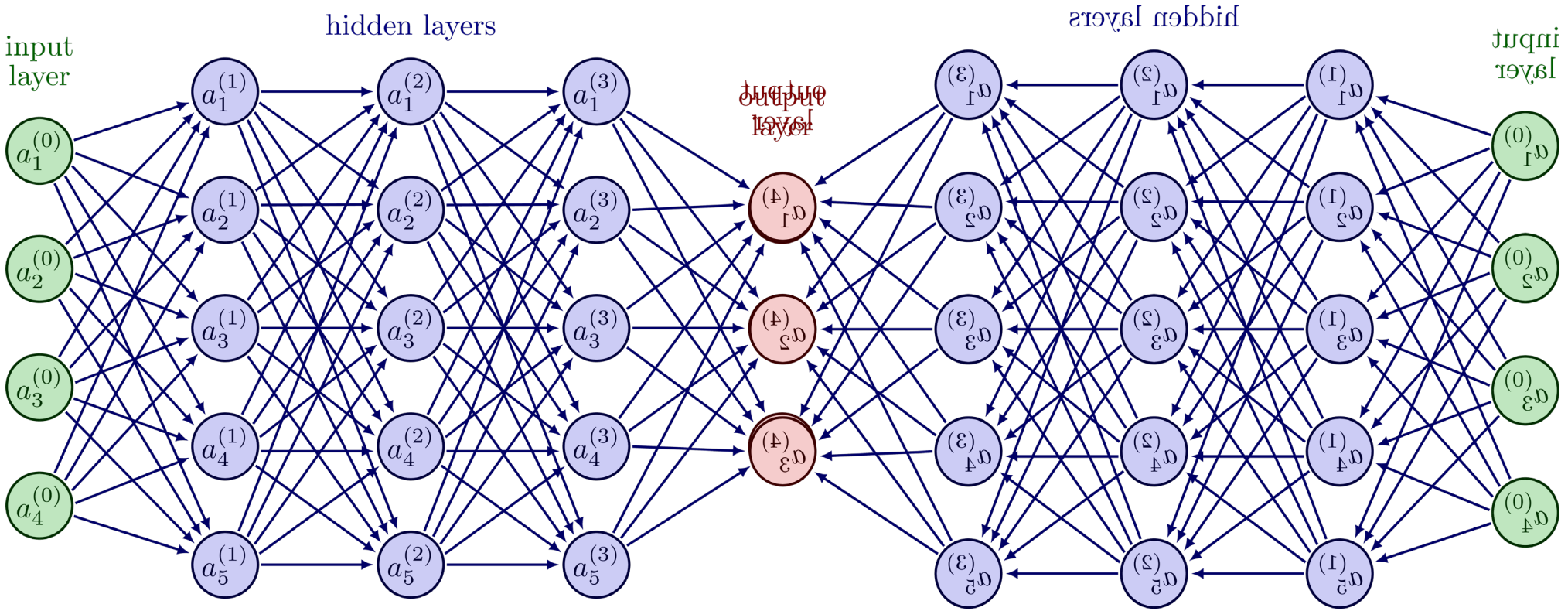
03

Variational autoencoders

Autoencoders



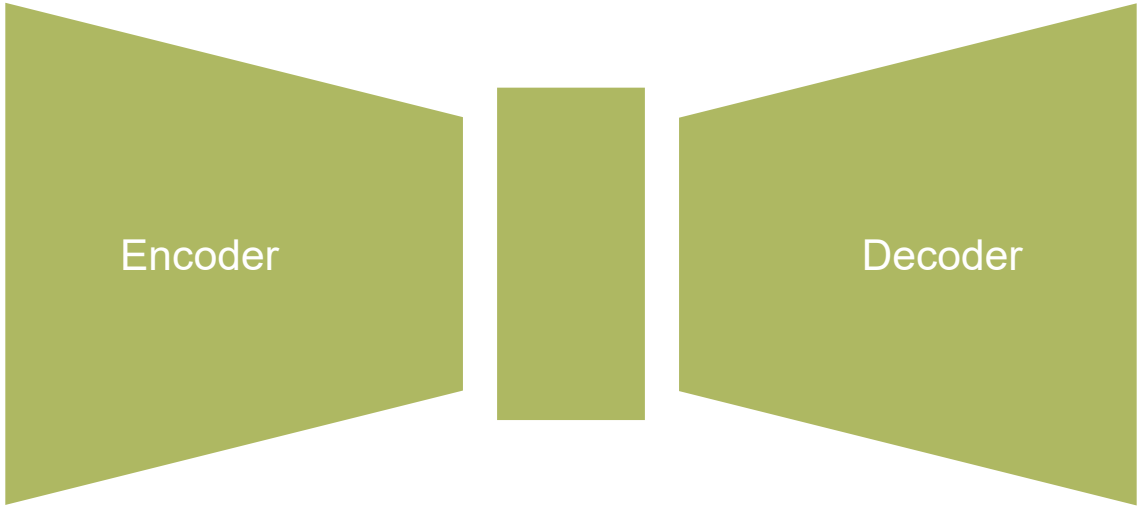
What are autoencoders really?



What is the fuss about?

ID	Height	Age	BP	Colour	Pet
1	180	43	120	red	no
2	210	55	141	red	no
3	182	41	118	blue	yes

The red fox jumps …

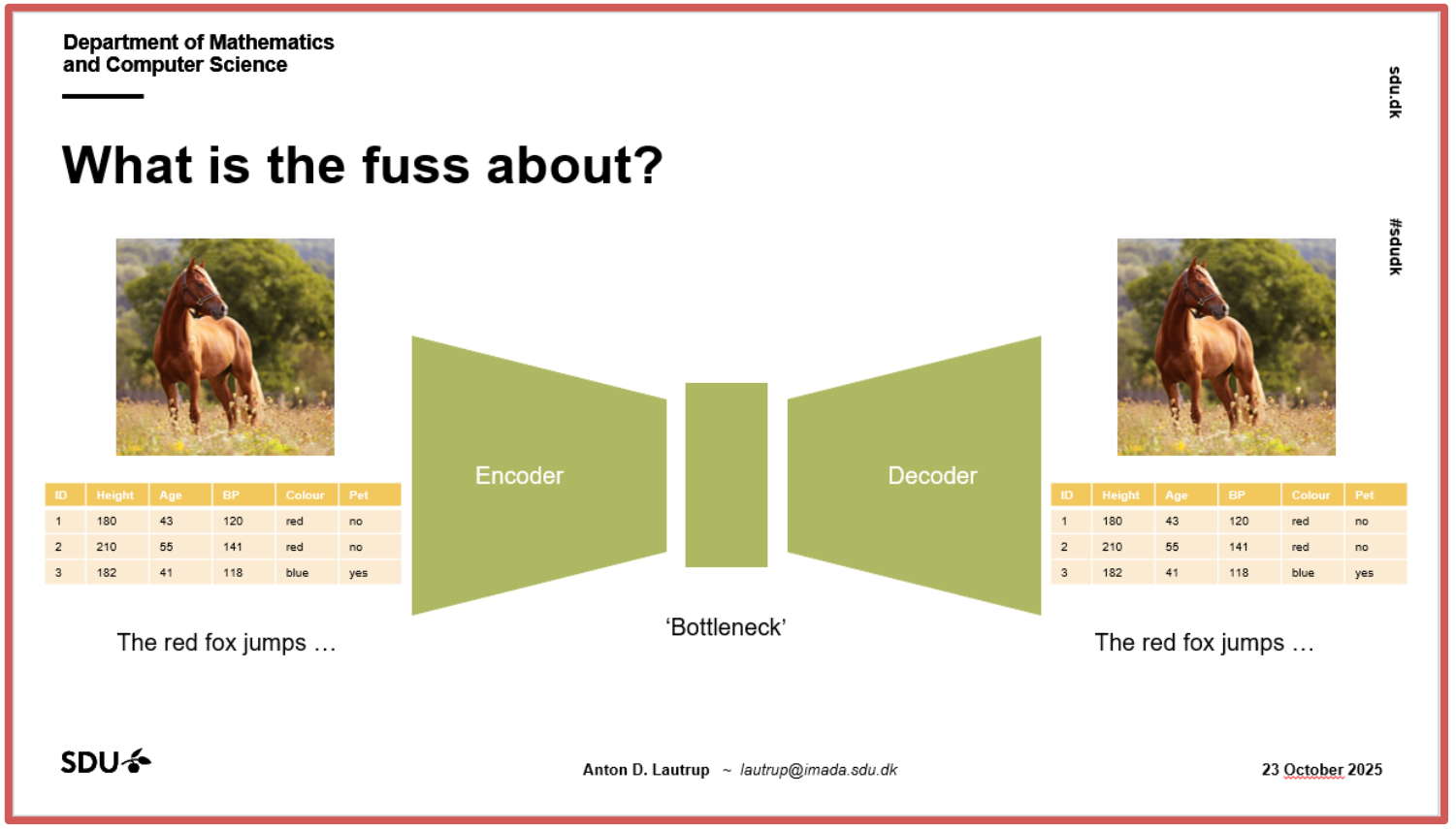


‘Bottleneck’

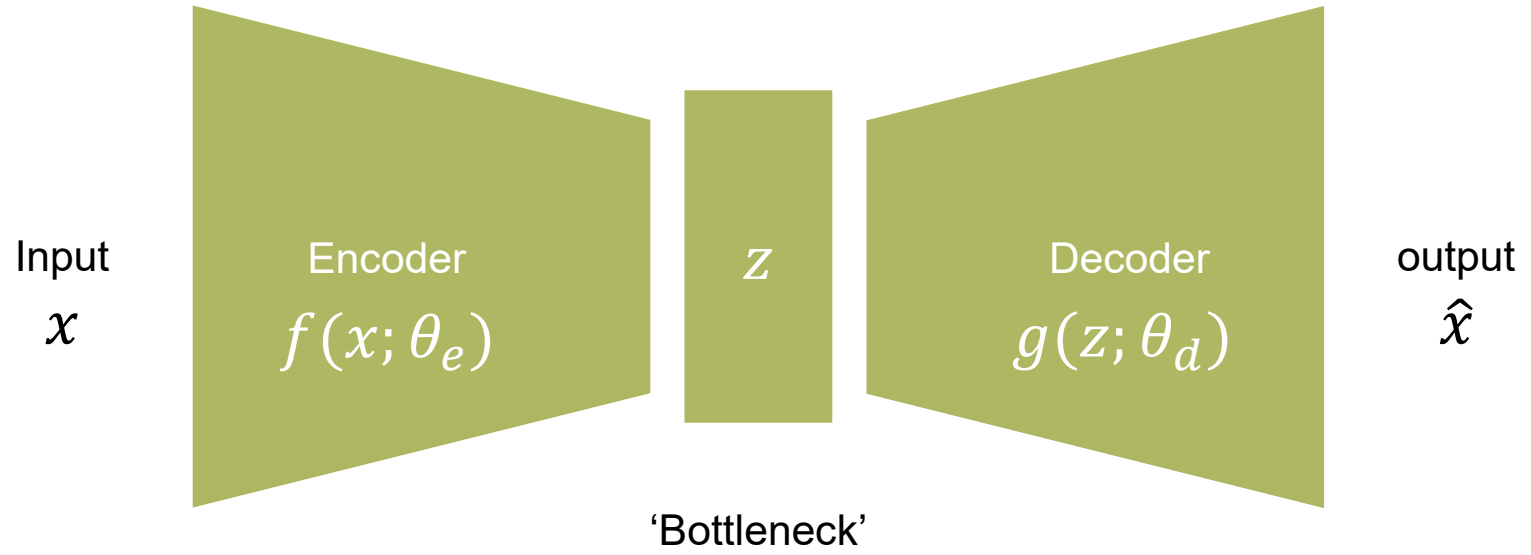
ID	Height	Age	BP	Colour	Pet
1	180	43	120	red	no
2	210	55	141	red	no
3	182	41	118	blue	yes

The red fox jumps …

What is the use?

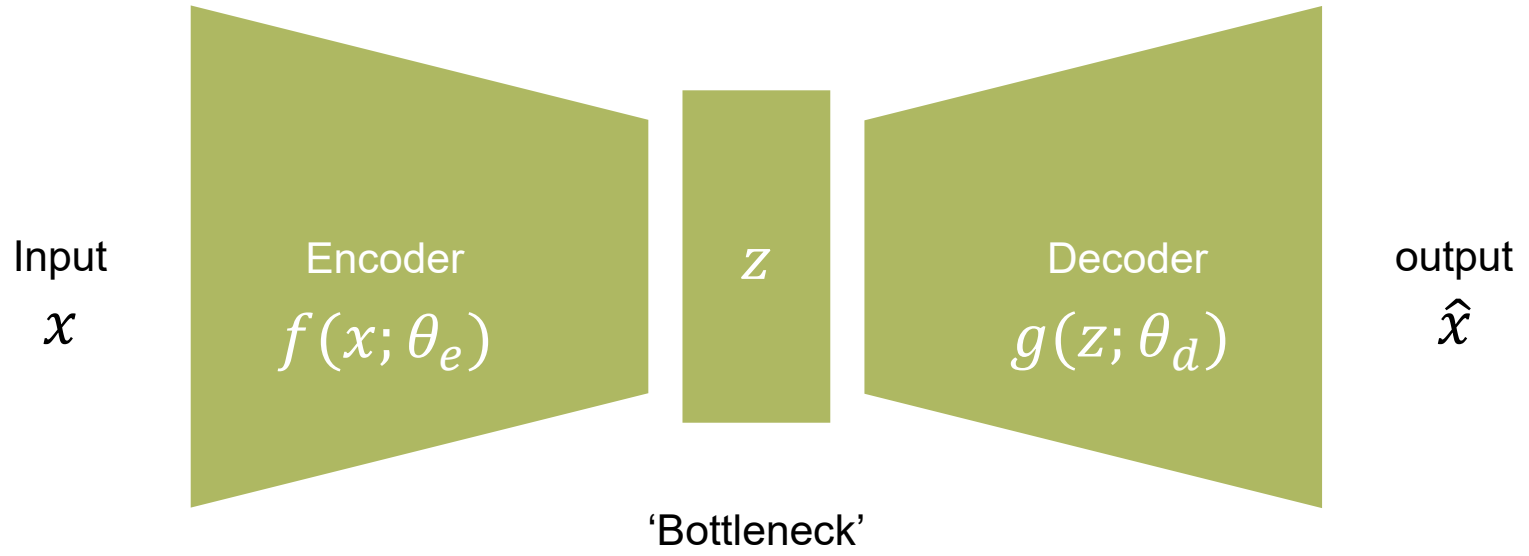


Objective function



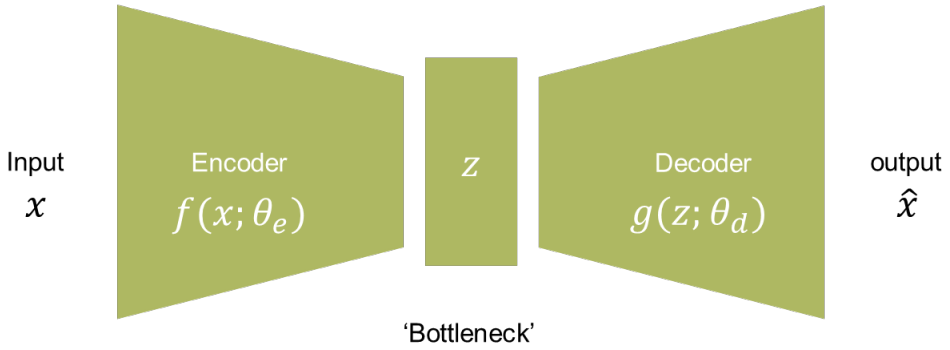
Find optimal parameters $\theta = \{\theta_e, \theta_d\}$ such that input and reconstruction are equal

Objective function



$$\underset{\theta_e,\theta_d}{\operatorname{argmin}} \mathcal{L}(x_i, g(f(x_i;\theta_e);\theta_d))$$

Reconstruction Loss

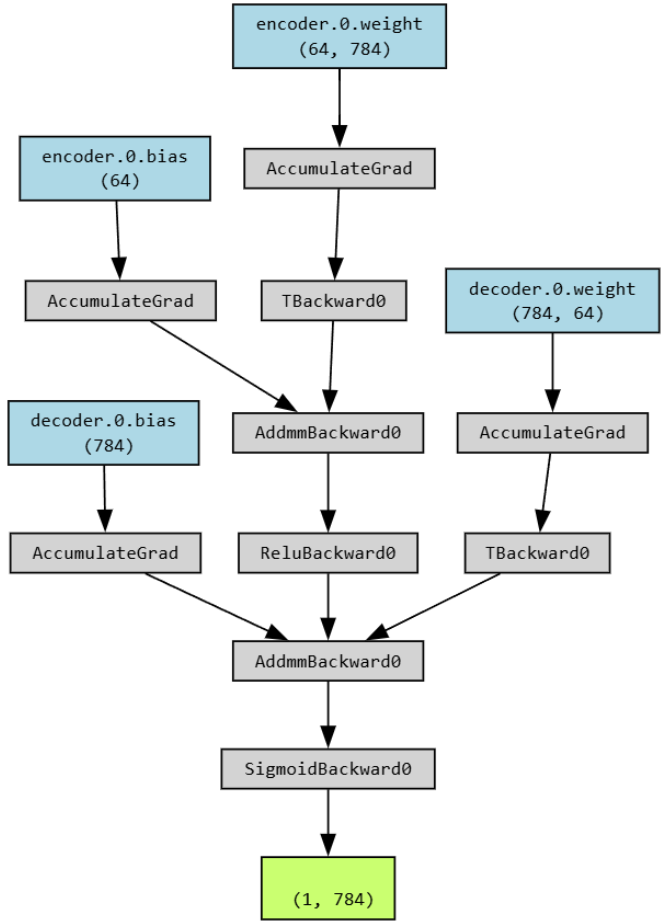


$$\min_{\theta_e,\theta_d} \mathcal{L}(x_i, g(f(x_i;\theta_e);\theta_d)$$

	$\mathcal{L}(x,\hat{x})$	
Mean squared error	$\frac{1}{N}\sum_i (x_i - \hat{x}_i)^2$	Standard choice: continuous data
Binary cross entropy	$-\frac{1}{N}\sum_i [x_i \log \hat{x}_i - (1-x_i)\log(1-\hat{x}_i)]$	Normalised [0,1] data + sigmoid
Mean absolute error	$\frac{1}{N}\sum_i \lvert x_i - \hat{x}_i \rvert$	Unusual, less sensitive to outliers

Building autoencoder in PyTorch

```python
class Autoencoder(torch.nn.Module):
    def __init__(self, input_size, latent_size):
        super(Autoencoder, self).__init__()
        self.encoder = nn.Sequential(
            nn.Linear(input_size, latent_size),
            nn.ReLU(True)
        )
        self.decoder = nn.Sequential(
            nn.Linear(latent_size, input_size),
            nn.Sigmoid()
        )

    def forward(self, x):
        x = self.encoder(x)
        x = self.decoder(x)
        return x
```



Minimal training loop

```python
### Example use
device = 'cuda' if torch.cuda.is_available() else 'cpu'

model = Autoencoder(28*28, 64).to(device)

criterion = nn.MSELoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)

dataloader = torch.utils.data.DataLoader(training_data, batch_size=64, shuffle=True)
for epoch in range(5):
    for data, _ in dataloader:
        data = data.view(data.size(0), -1).to(device)  # flatten
        optimizer.zero_grad()
        output = model(data)
        loss = criterion(output, data)
        loss.backward()
        optimizer.step()

    print(f"Epoch {epoch+1}, Loss: {loss.item()}")
```

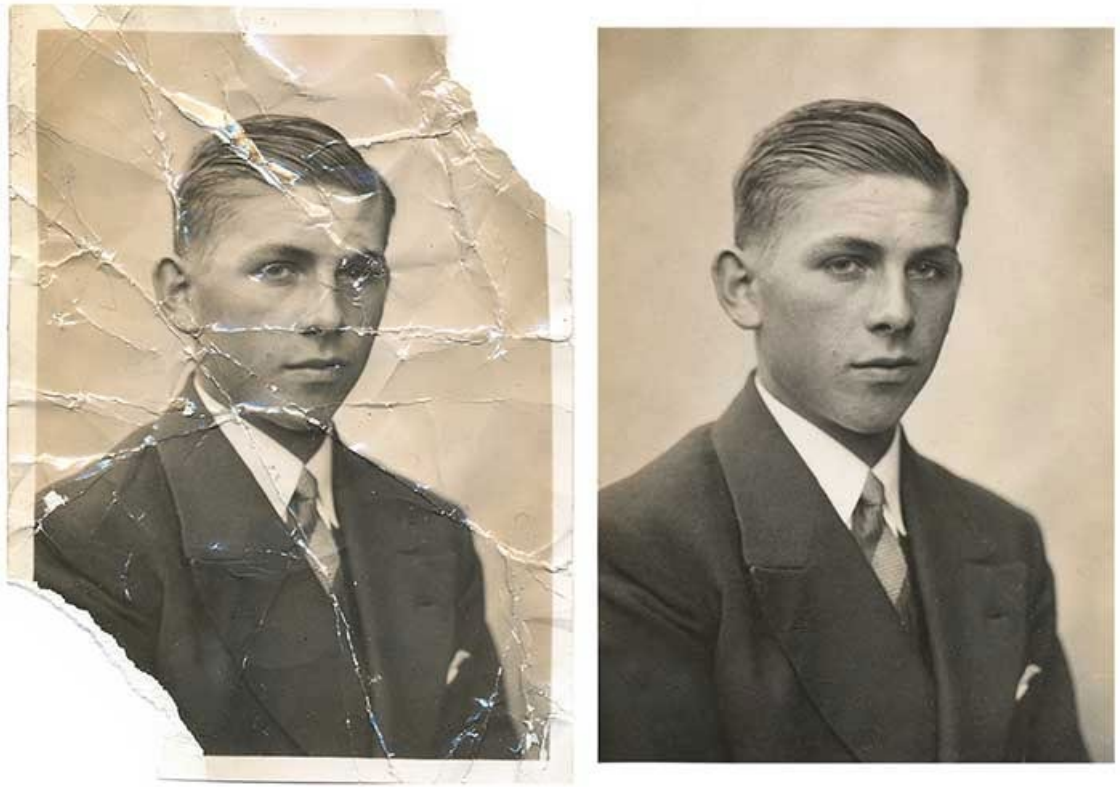
Building autoencoder in PyTorch

```
Autoencoder(
  (encoder): Encoder(
    (encoder): Sequential(
      (0): Linear(in_features=784, out_features=64, bias=True)
      (1): ReLU(inplace=True)
    )
  )
  (decoder): Decoder(
    (decoder): Sequential(
      (0): Linear(in_features=64, out_features=784, bias=True)
      (1): Sigmoid()
    )
  )
)
```

```python
class Decoder(torch.nn.Module):
    def __init__(self, input_size, latent_size):
        super(Decoder, self).__init__()
        self.decoder = nn.Sequential(
            nn.Linear(latent_size, input_size),
            nn.Sigmoid()
        )

    def forward(self, x):
        x = self.decoder(x)
        return x

class Encoder(torch.nn.Module):
    def __init__(self, input_size, latent_size):
        super(Encoder, self).__init__()
        self.encoder = nn.Sequential(
            nn.Linear(input_size, latent_size),
            nn.ReLU(True)
        )

    def forward(self, x):
        x = self.encoder(x)
        return x

class Autoencoder(torch.nn.Module):
    def __init__(self, input_size, latent_size):
        super(Autoencoder, self).__init__()
        self.encoder = Encoder(input_size, latent_size)
        self.decoder = Decoder(input_size, latent_size)

    def forward(self, x):
        x = self.encoder(x)
        x = self.decoder(x)
        return x
```

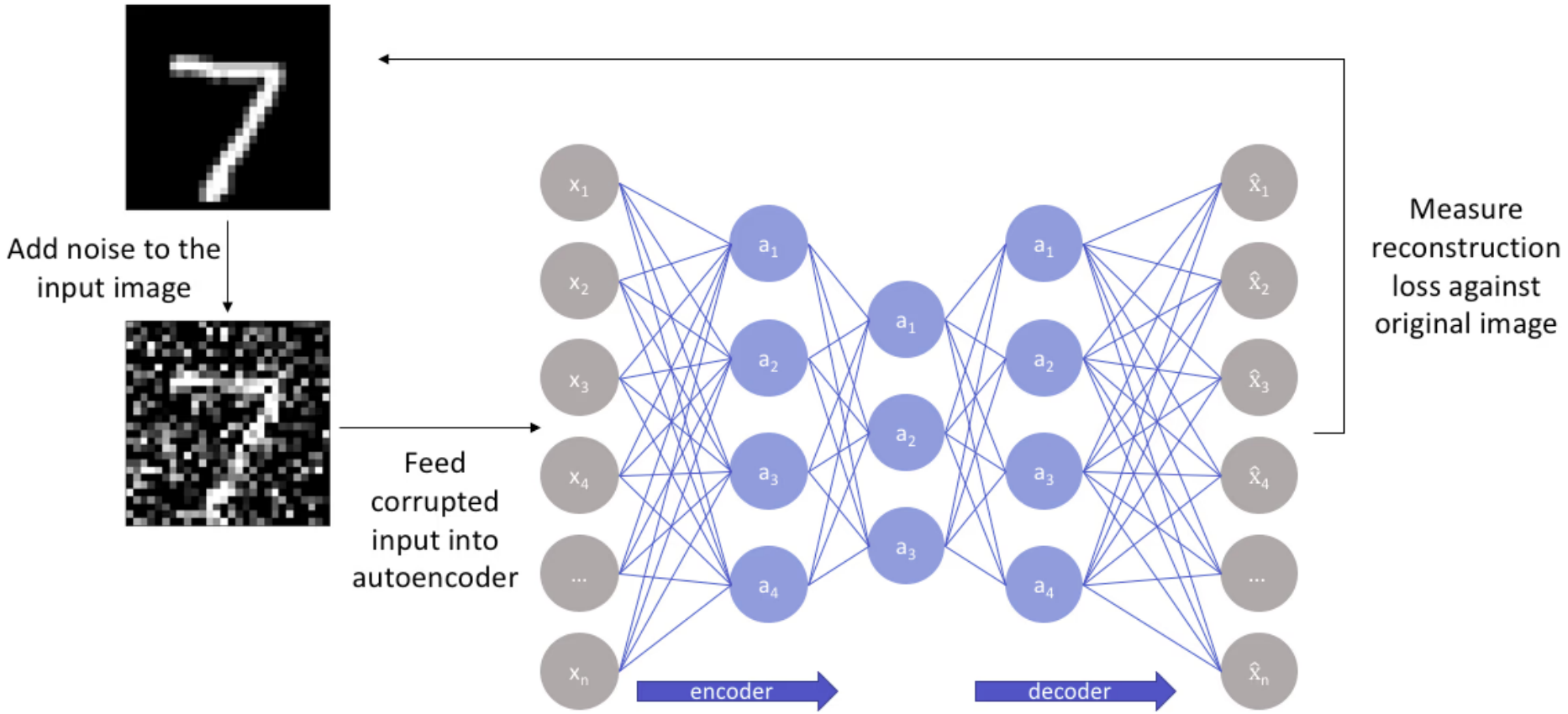
Case example:

Image Restoration



Anton D. Lautrup ~ *lautrup@imada.sdu.dk*

Denoising Autoencoder

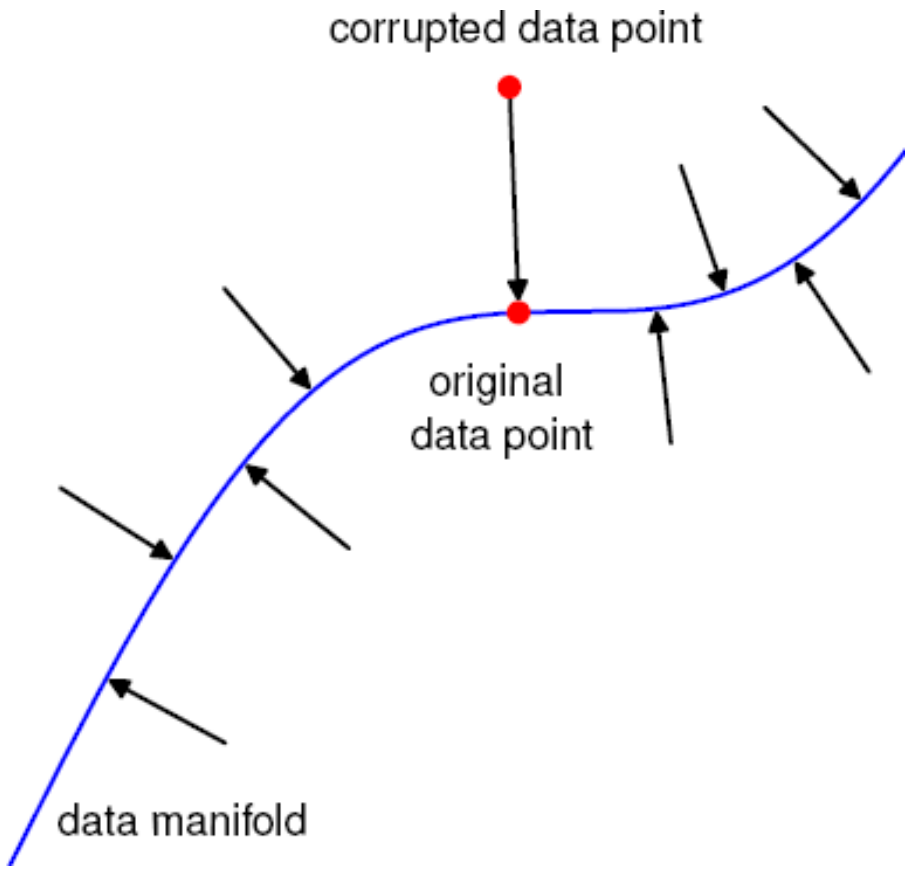


Denoising Autoencoder

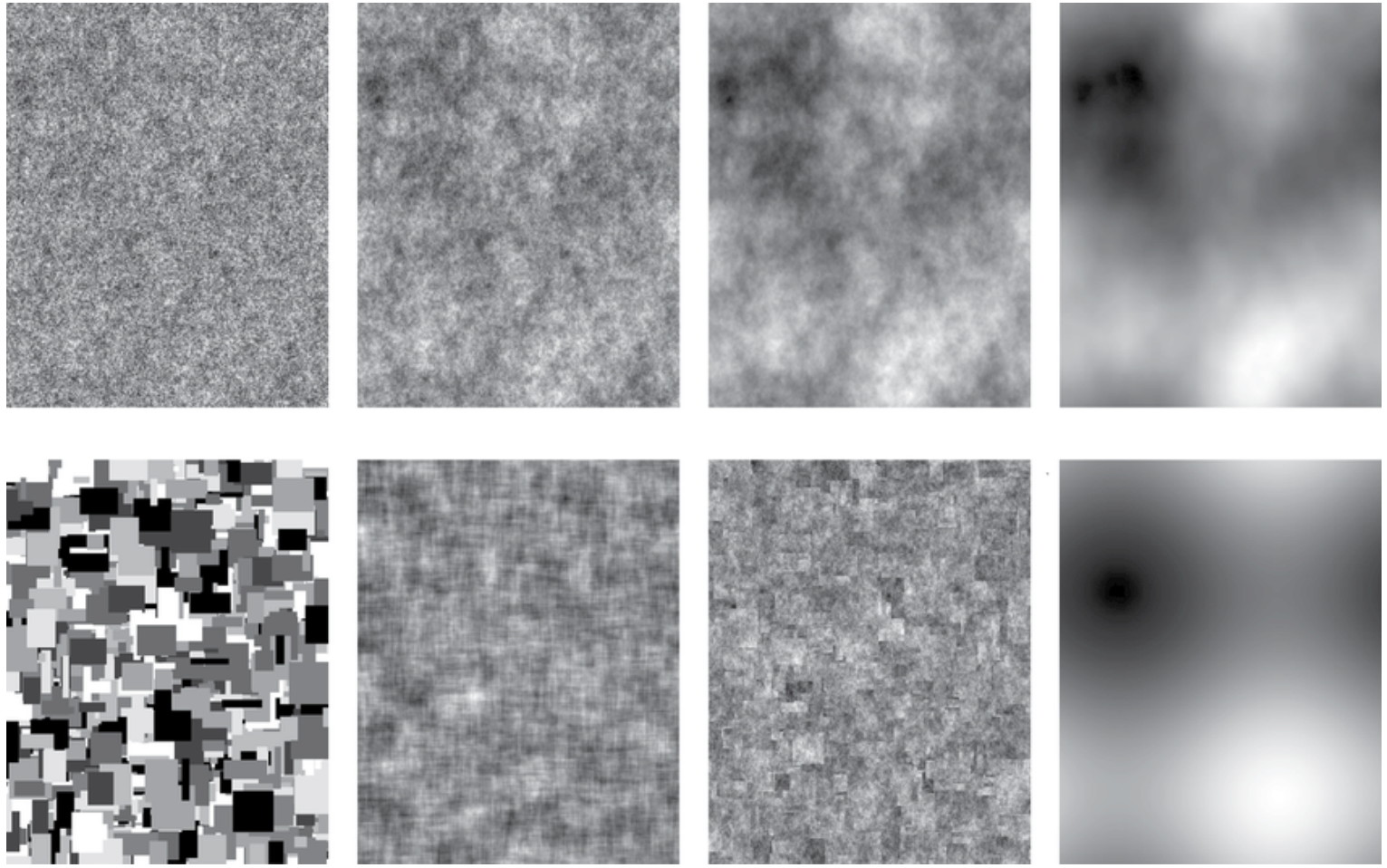
Minimise:

$$\mathcal{L}_{rec} = \frac{1}{2} \left| g\left(f(\tilde{x})\right) - x \right|^2$$

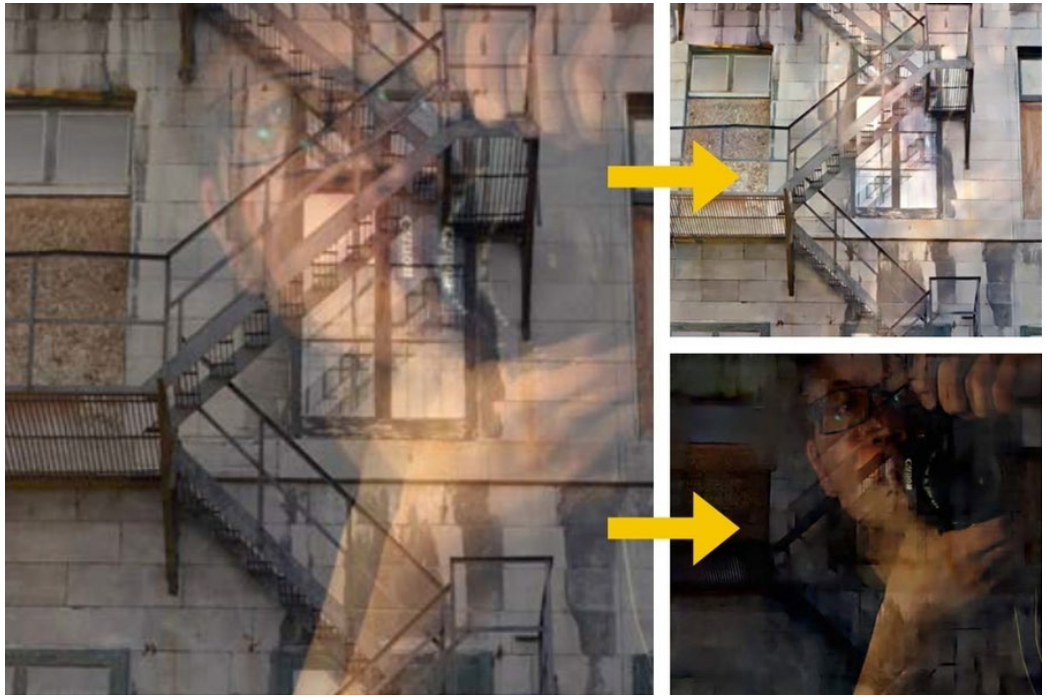
Where $\tilde{x}$ is a corrupted version of x.



Filters

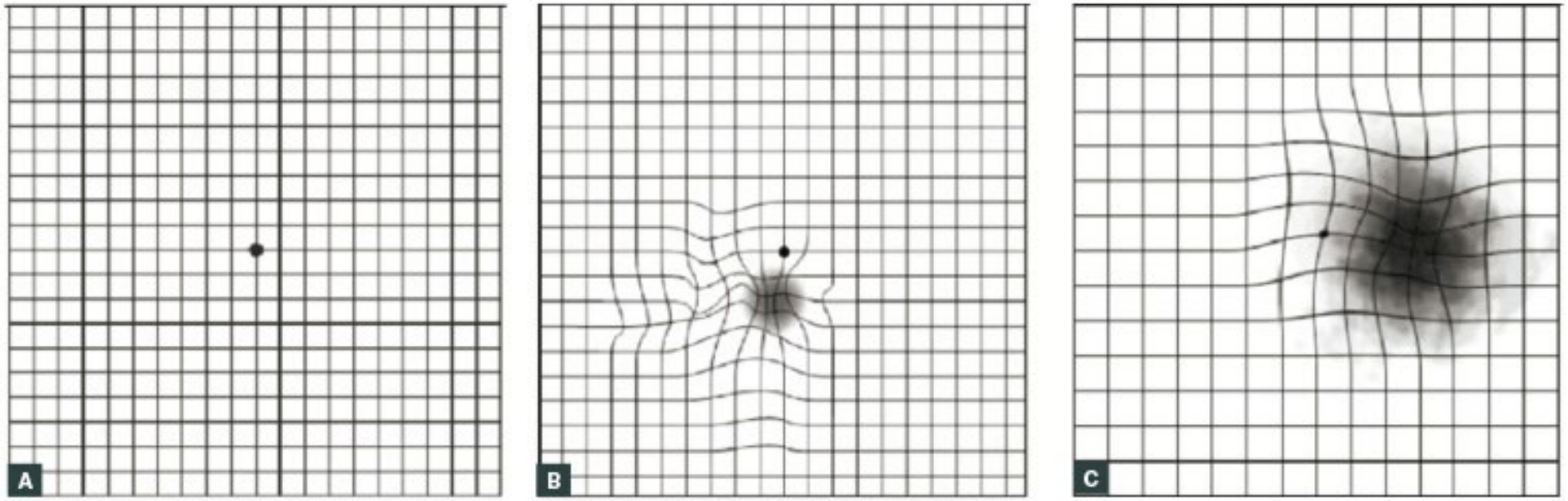


Artefacts



Brion Stoutarm
Legendary Creature — Giant Warrior
Lifelink
, , Sacrifice a creature other than Brion Stoutarm: Brion Stoutarm deals damage equal to the sacrificed creature's power to target player.
4/4

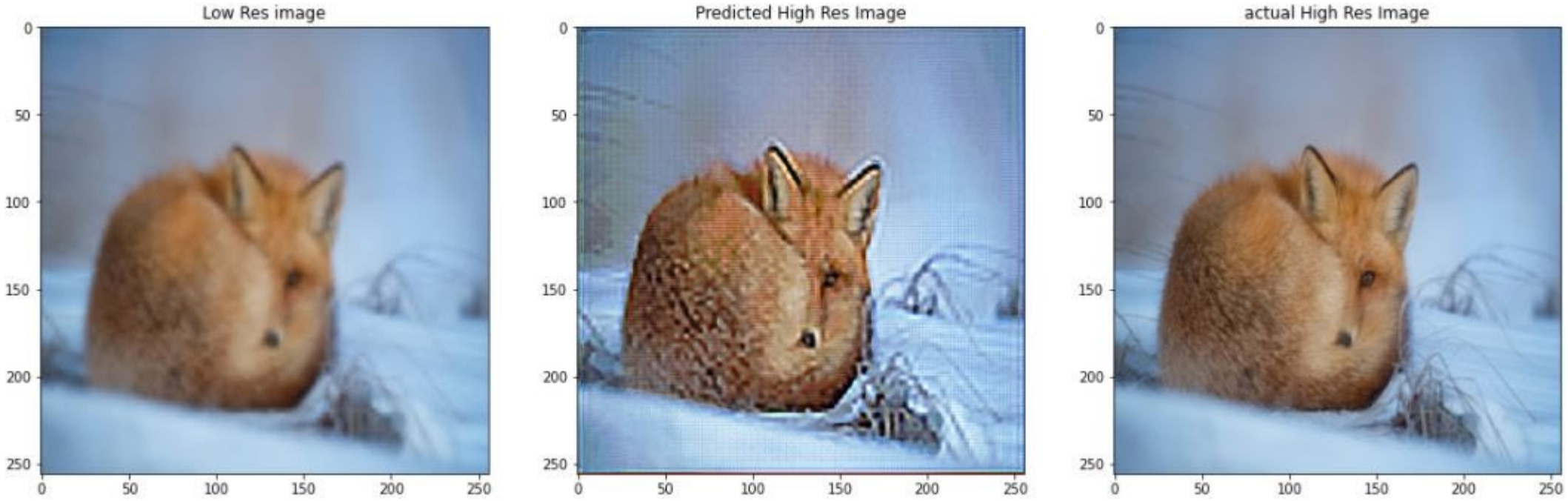
Distortion



Masking

C. M. Bishop, H. Bishop, *Deep Learning*, https://doi.org/10.1007/978-3-031-45468-4_19

Compression



DLAA
TAA
FSR
DLSS

NVIDIA

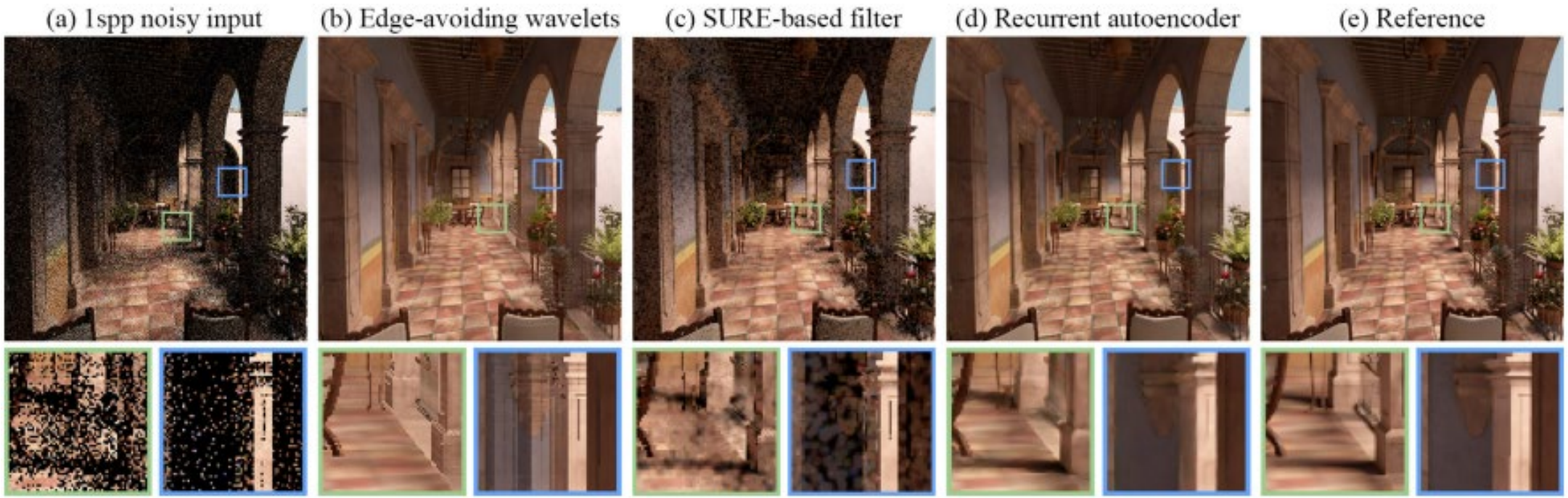
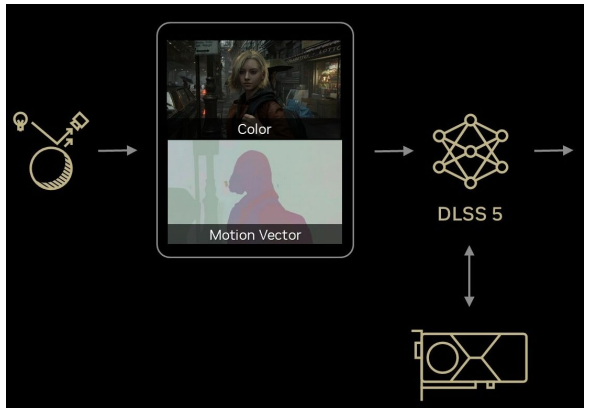


Fig. 1. Left to right: (a) noisy image generated using path-traced global illumination with one indirect inter-reflection and 1 sample/pixel; (b) edge-avoiding wavelet filter [Dammertz et al. 2010] (10.3ms at 720p, SSIM: 0.7737); (c) SURE-based filter [Li et al. 2012] (74.2ms, SSIM: 0.5960); (d) our recurrent denoising autoencoder (54.9ms, SSIM: 0.8438); (e) reference path-traced image with 4096 samples/pixel.

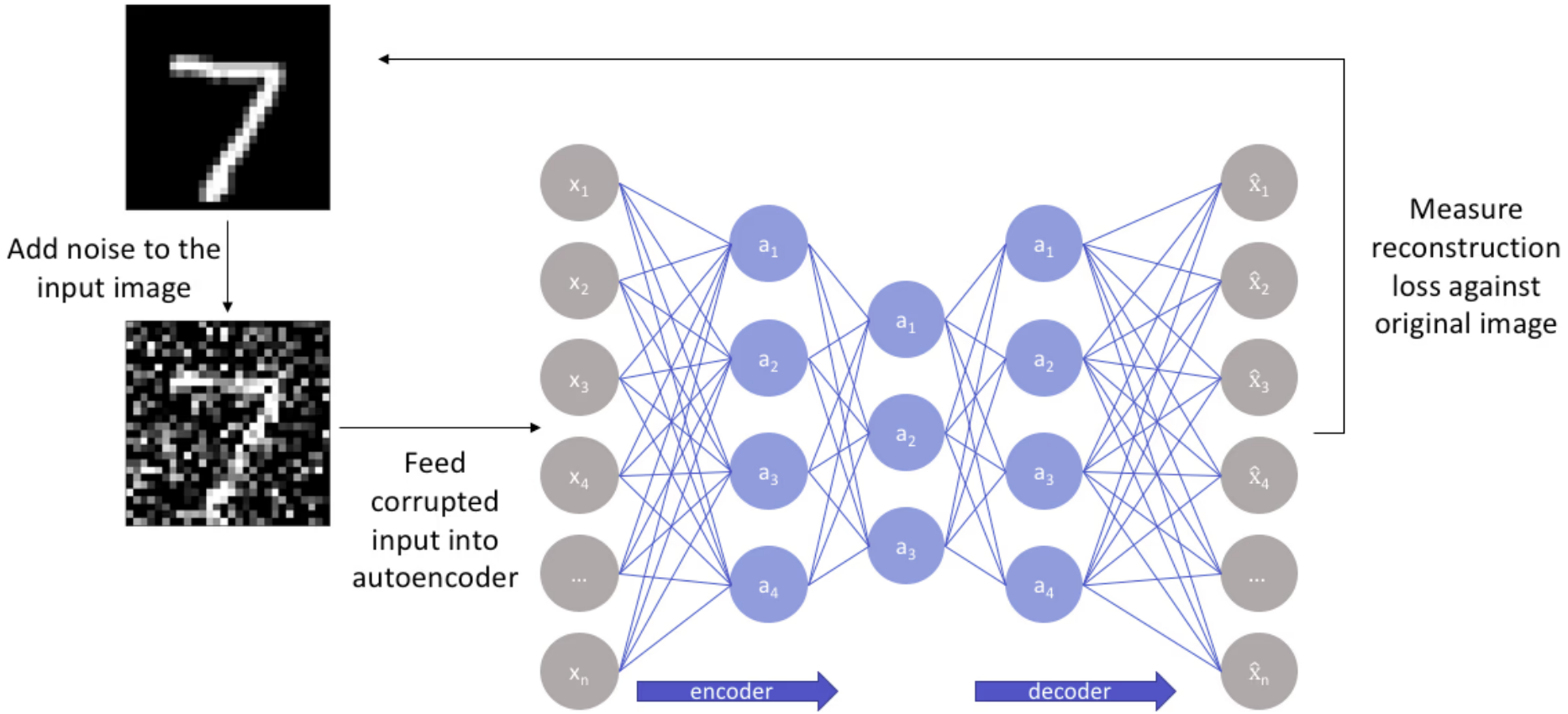
DLSS 5


Color
Motion Vector
DLSS 5

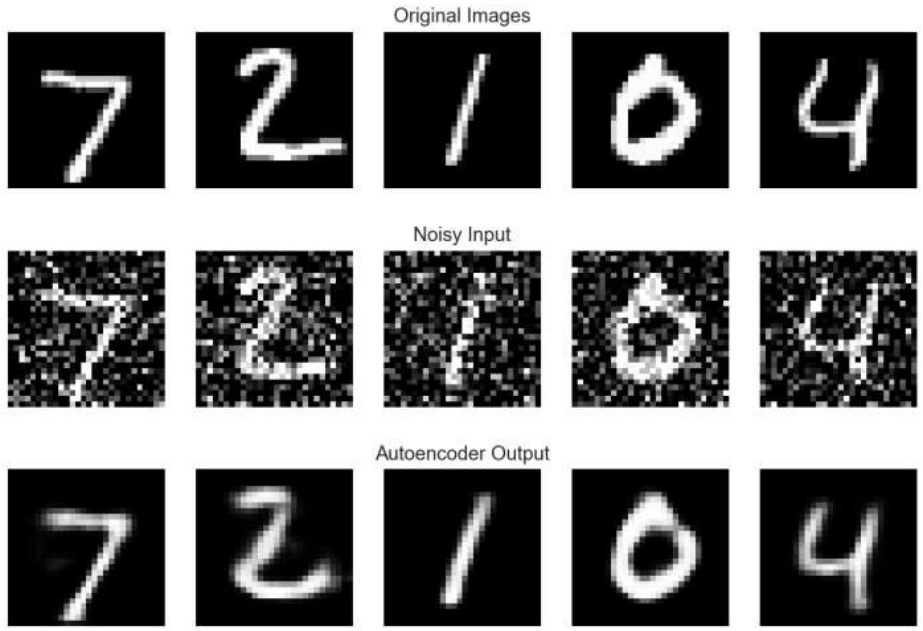
Monday - Friday
RESIDENT EVIL
requiem
CIGARETTES
LOTTO
DLSS 5 Off

Monday - Friday
RESIDENT EVIL
requiem
CIGARETTES
DLSS 5 On

Denoising Autoencoder



Generalisation?

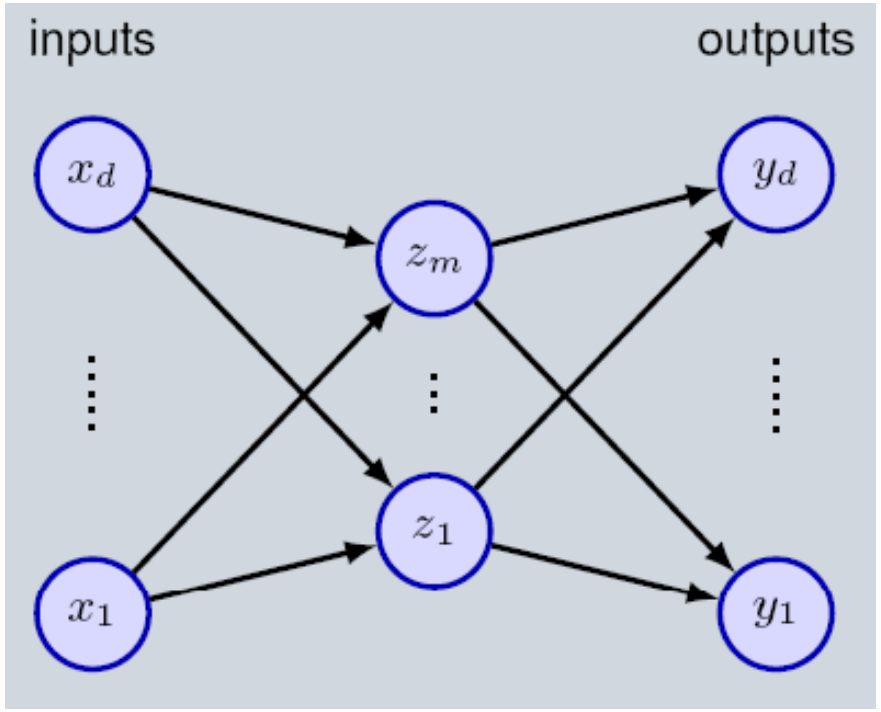


Department of Mathematics
and Computer Science

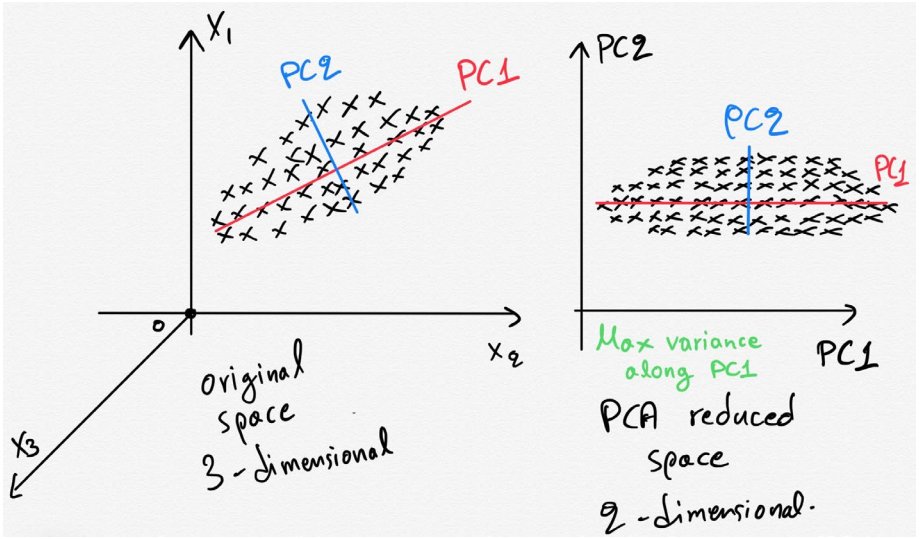
Latent space

Anton D. Lautrup ~ *lautrup@imada.sdu.dk*

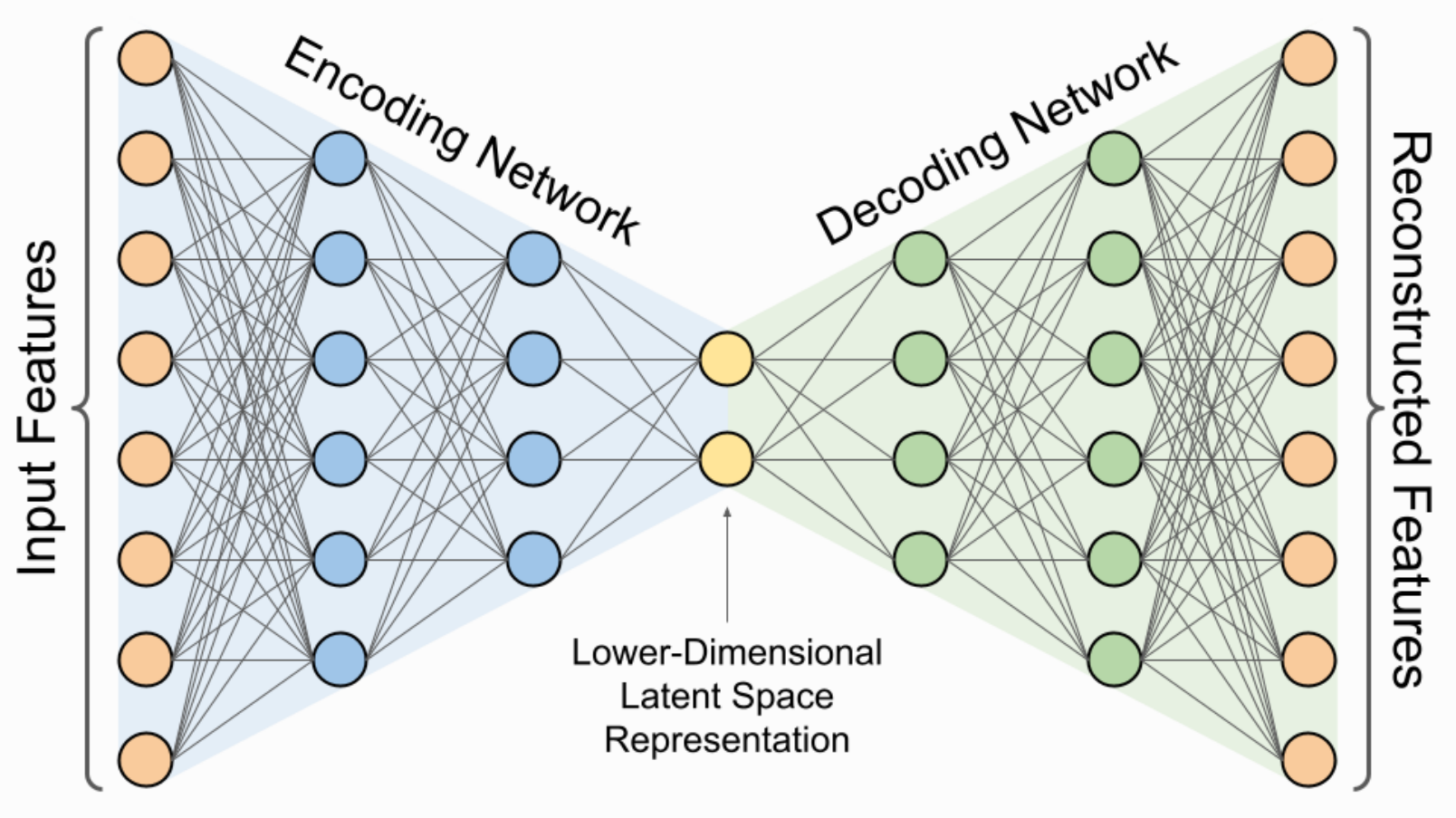
Edge case with no non-linearities



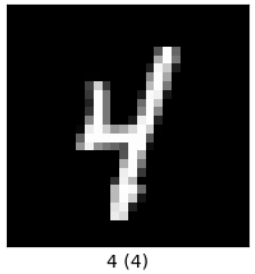
- Both encoder f and decoder g are just linear projections

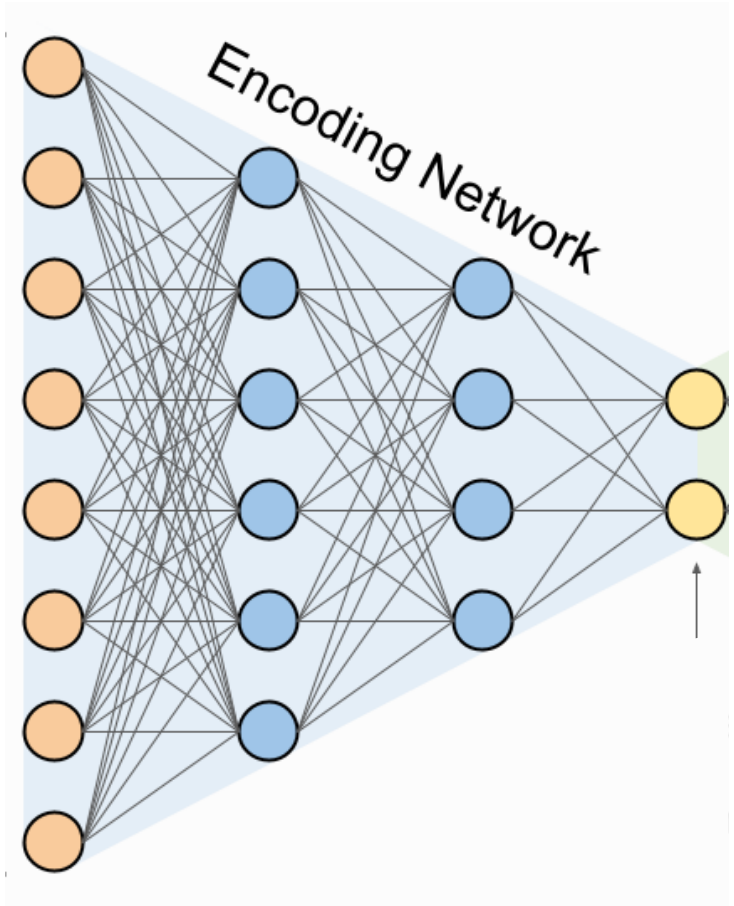


Automatic feature extraction



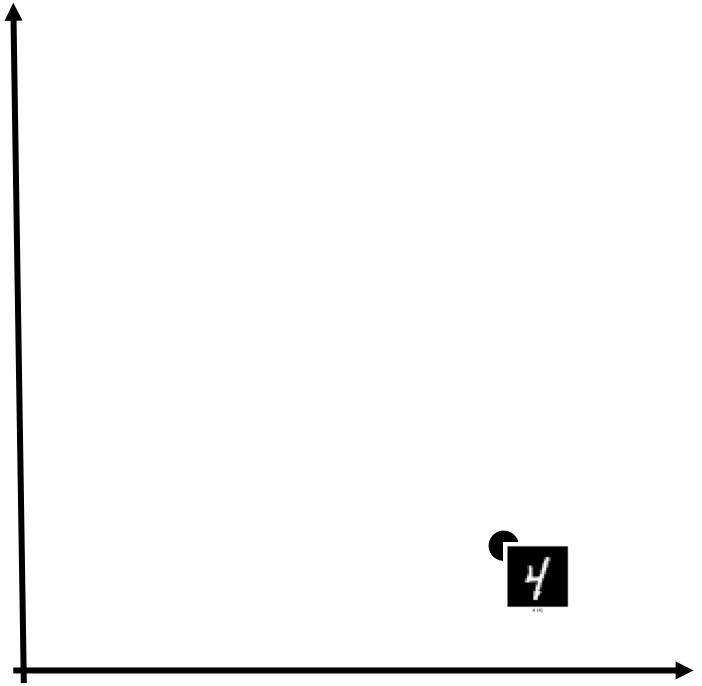
Low dimensional representation



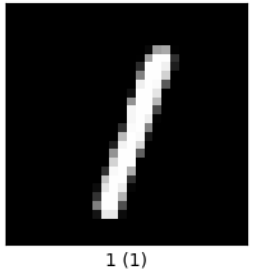


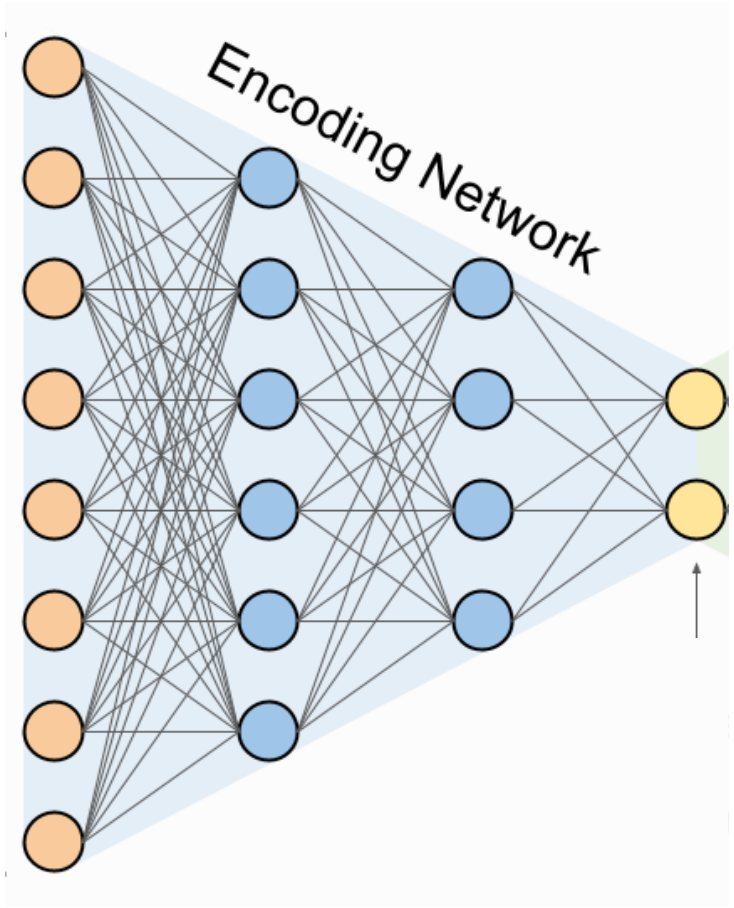
0.7

0.2



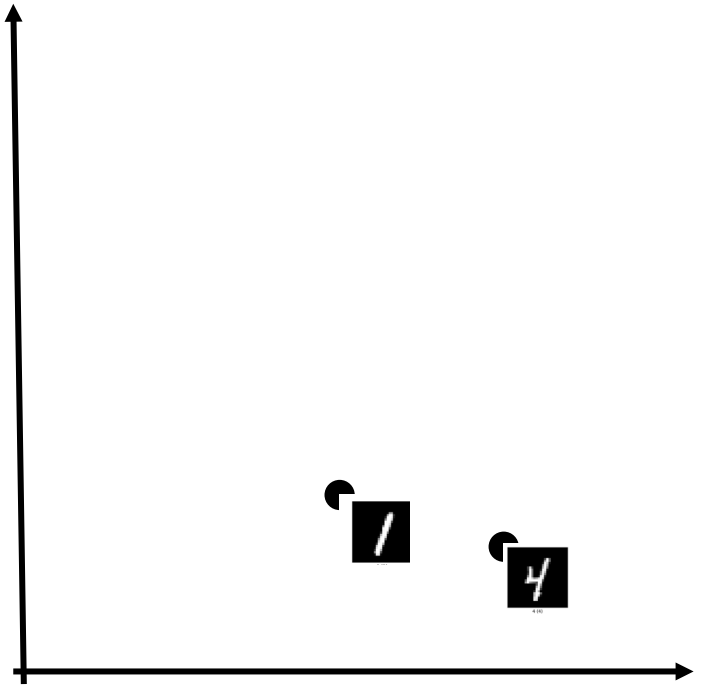
Low dimensional representation



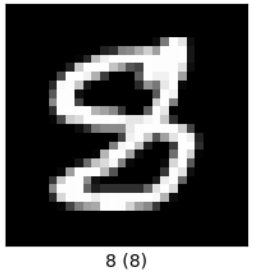


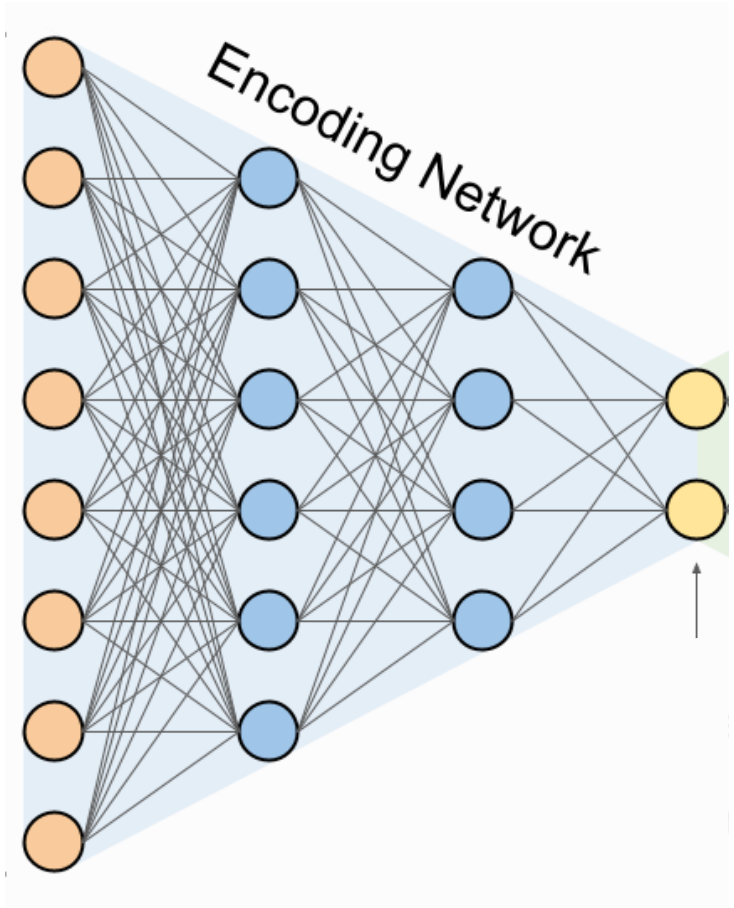
0.5

0.3



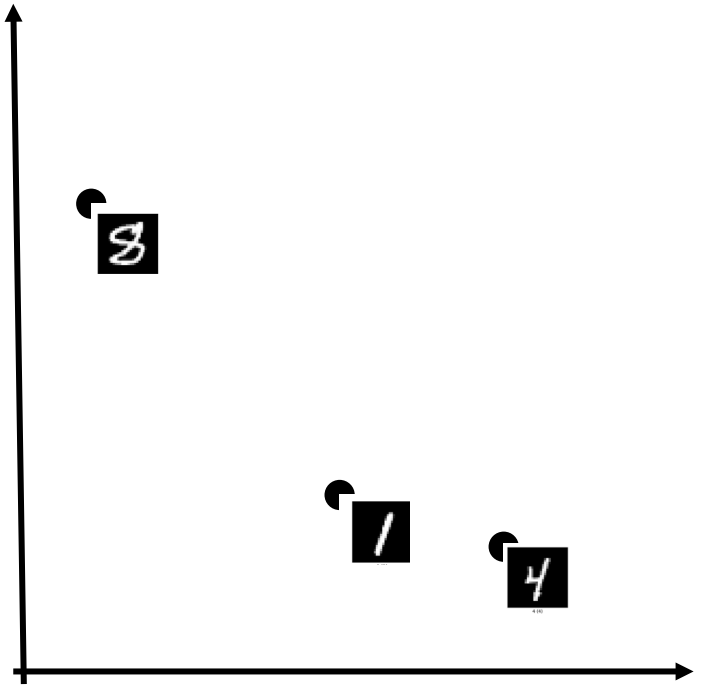
Low dimensional representation



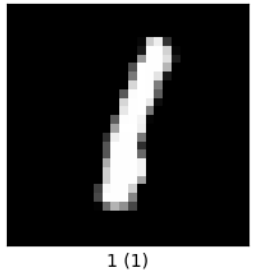


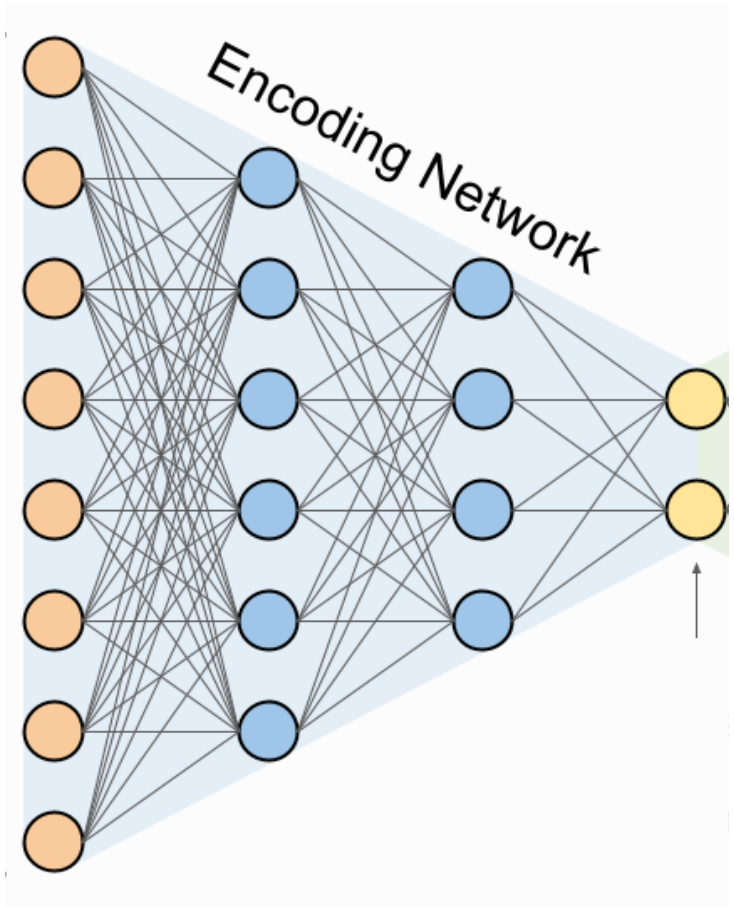
0.1

0.7



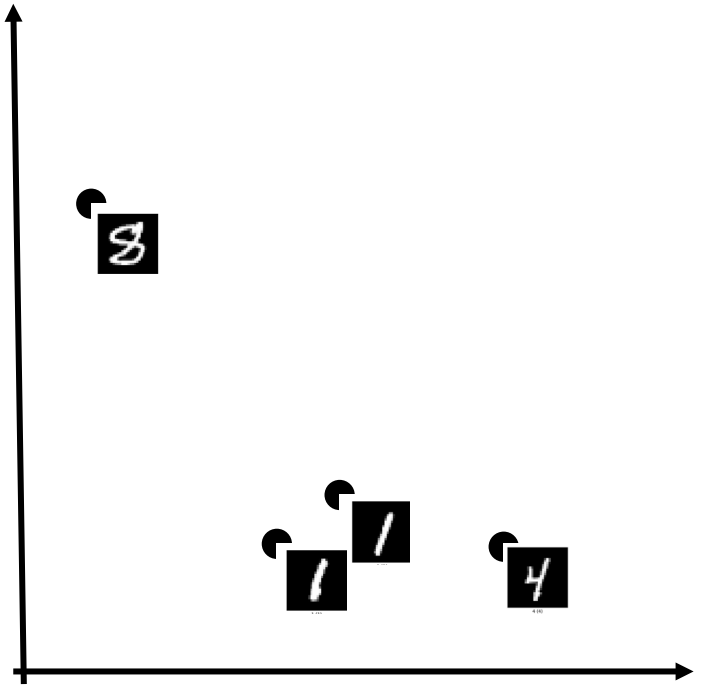
Low dimensional representation



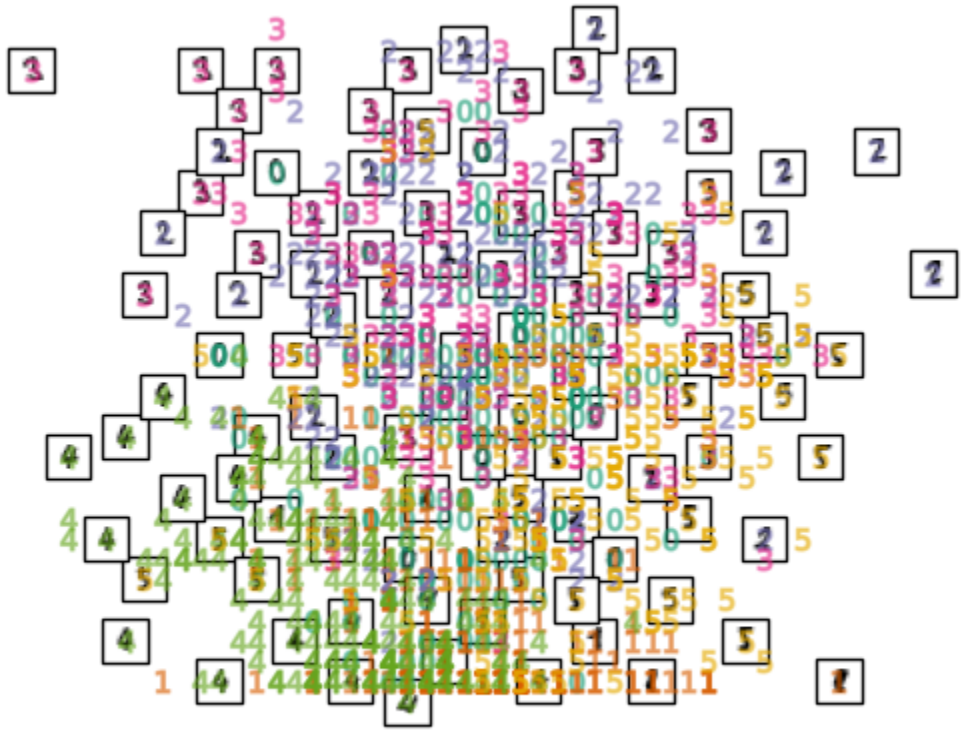


0.4

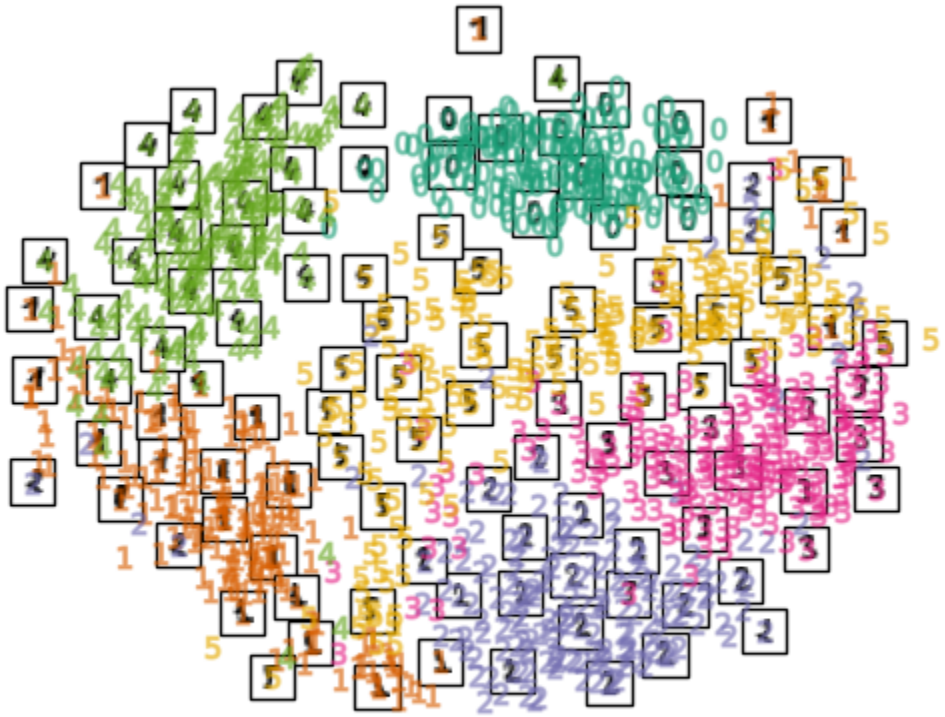
0.2



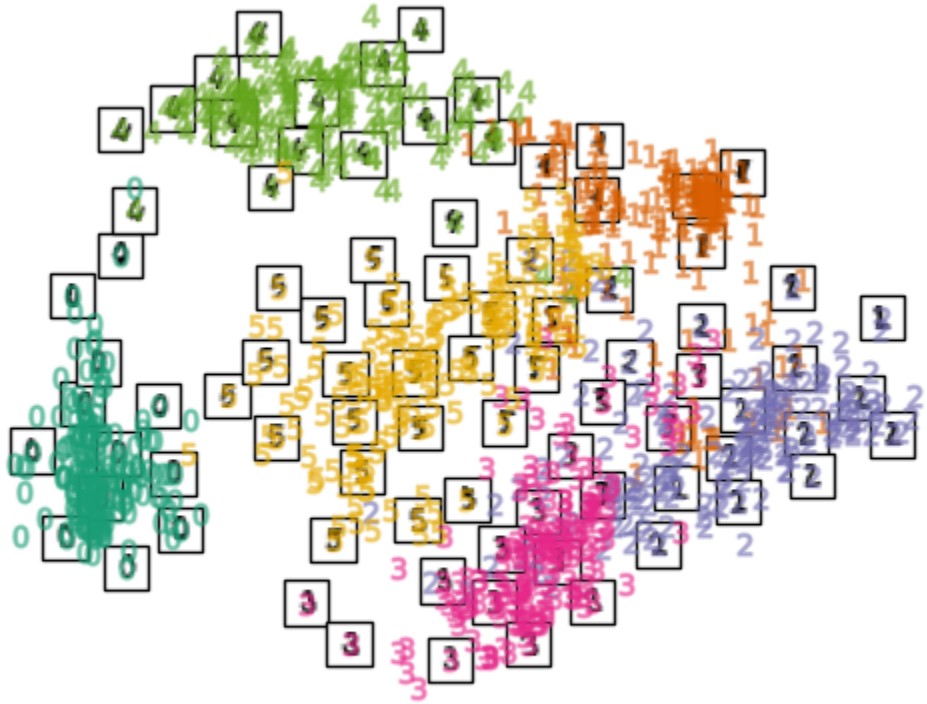
Variation



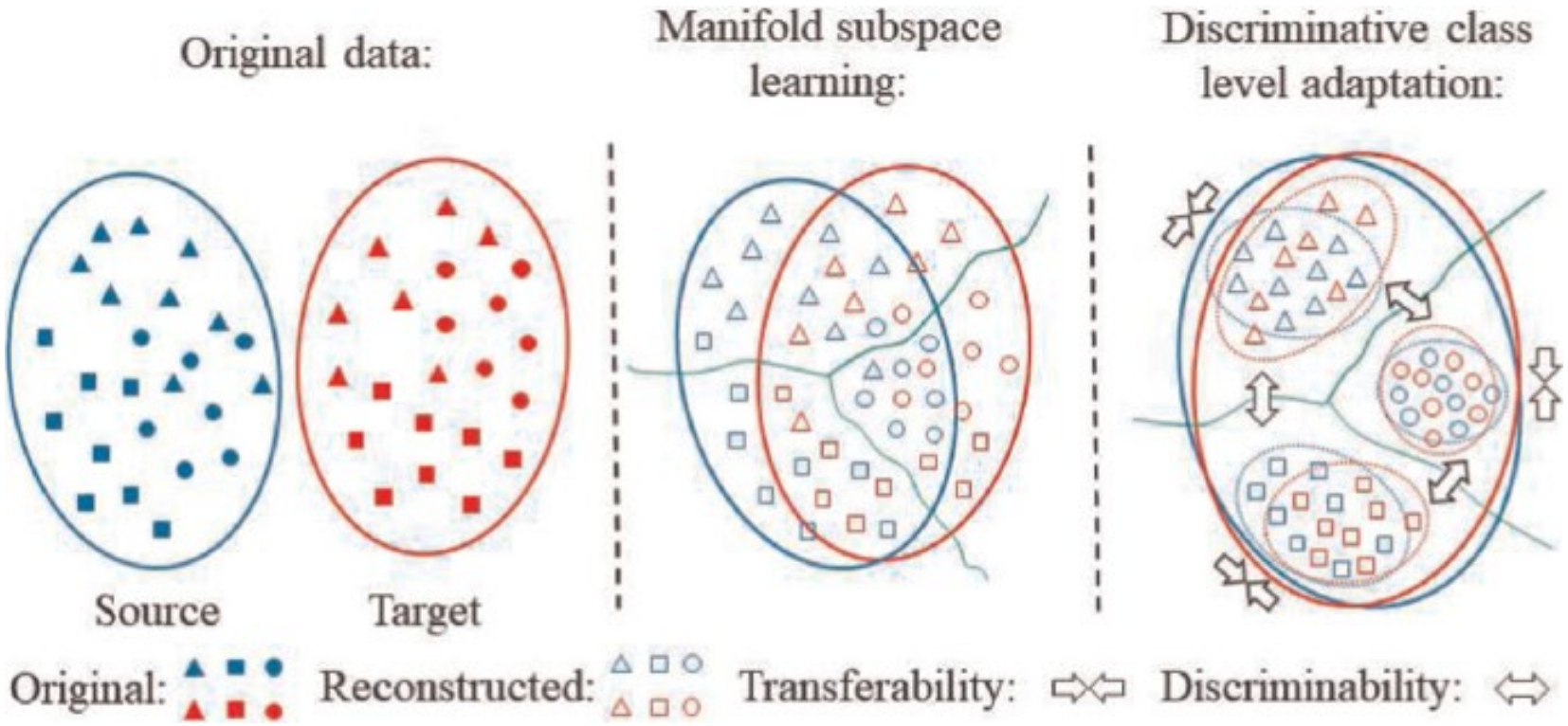
Variation



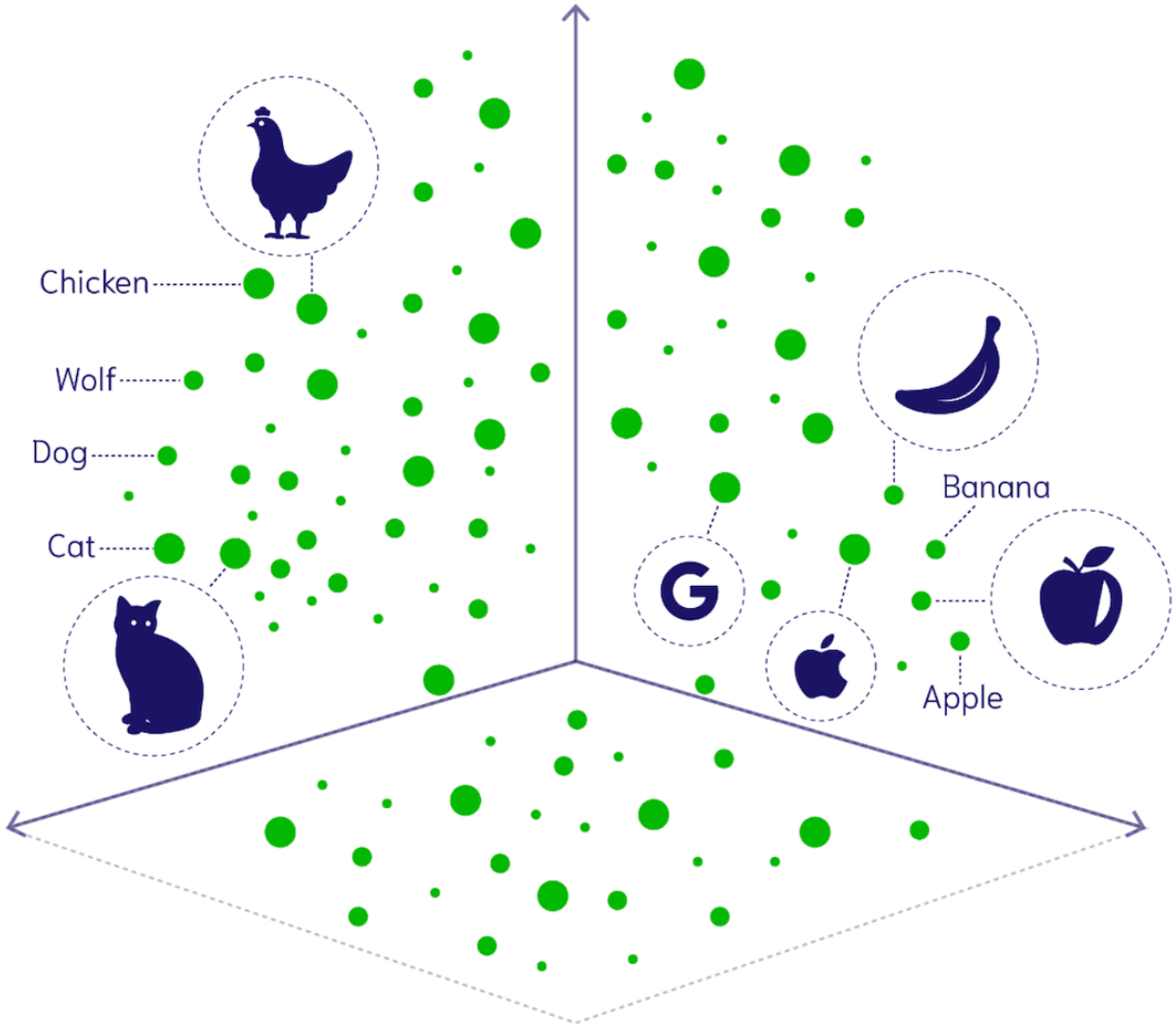
Variation



Manifold alignment

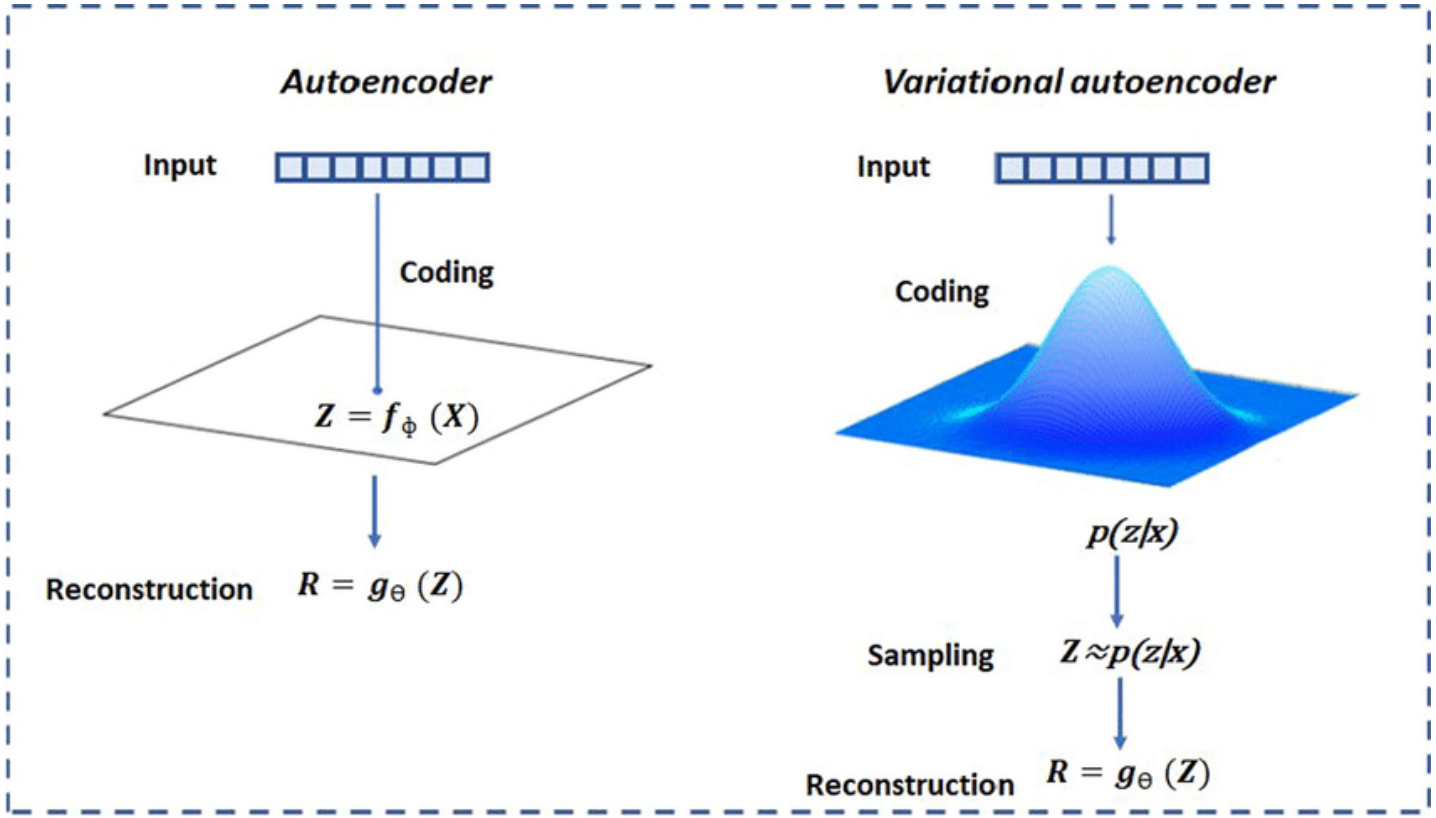


Multimodal spaces

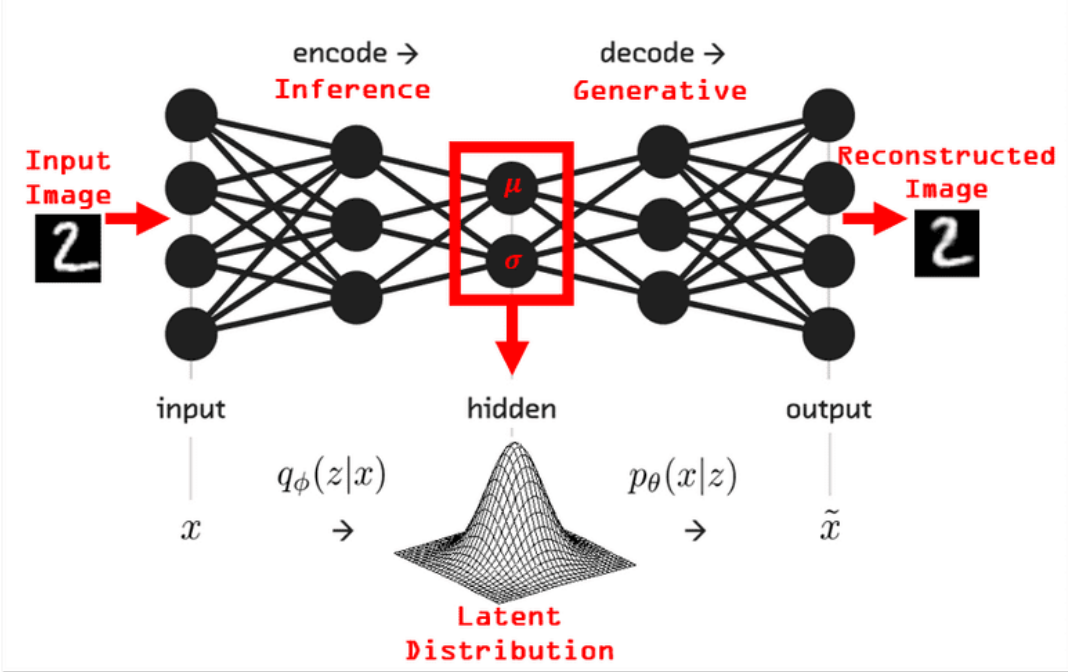


Shape of the latent space?

Central symmetry

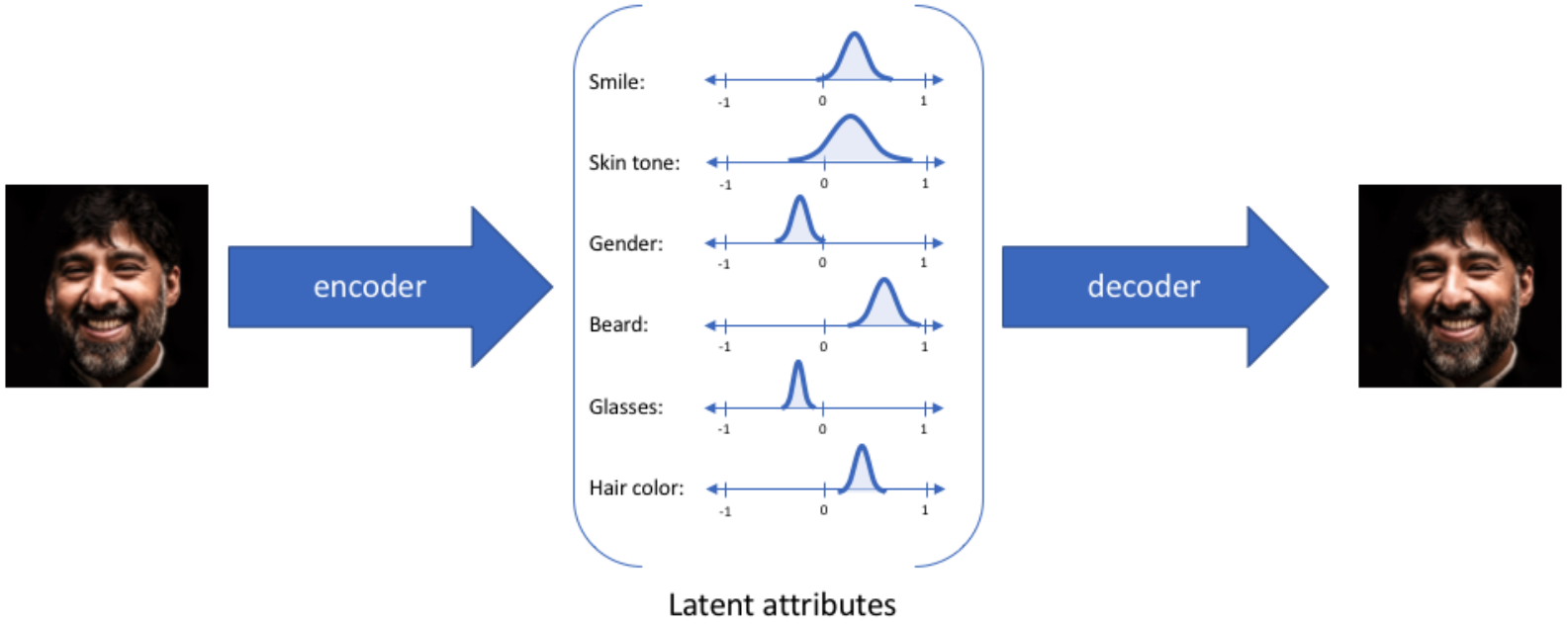


Variational Autoencoders (VAE)



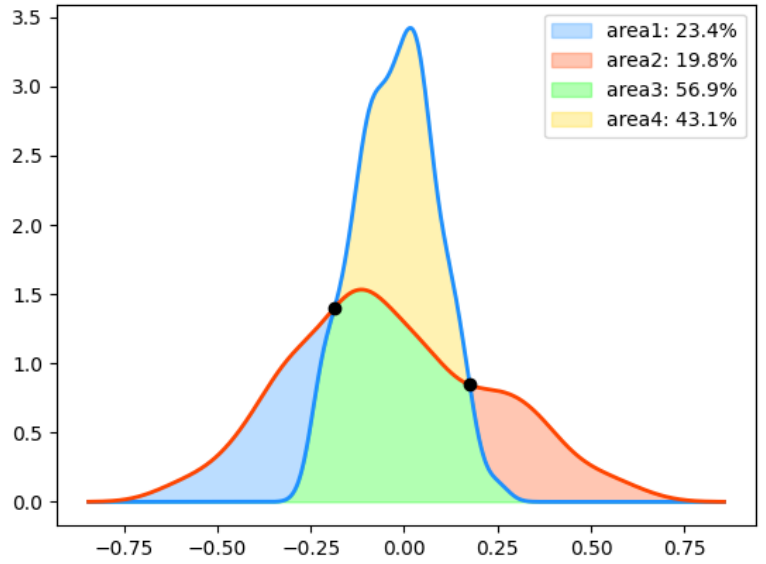
Anton D. Lautrup ~ *lautrup@imada.sdu.dk*

Impose meaning on the latent variables



How to shape the latent space?

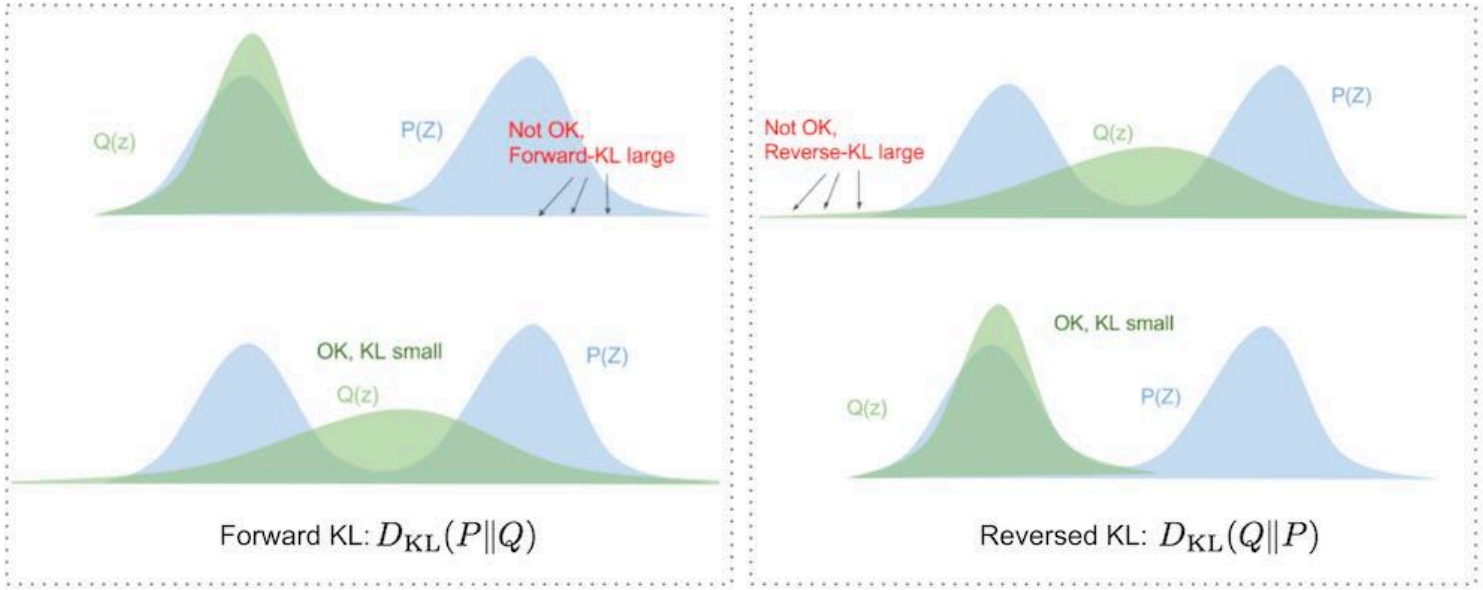
$$\mathcal{L}(x, \hat{x}) \rightarrow \mathcal{L}(x, \hat{x}) + Measure\ of\ distributional\ overlap$$



Anton D. Lautrup ~ *lautrup@imada.sdu.dk*

Kullback-Leibler divergence

$$D_{KL}(P||Q) = \sum_{x \in \mathcal{X}} P(x) \log \frac{P(x)}{Q(x)}$$



VAE Loss

$$KL(p,q) = \int [\, p(x)\log(p(x)) - p(x)\log(q(x))]\; dx$$
$$= -\frac{1}{2}[1+\log(2\pi\sigma_1^2)] - \left[-\frac{1}{2}\log(2\pi\sigma_2^2) - \frac{\sigma_1^2 + (\mu_1-\mu_2)^2}{2\sigma_2^2}\right]$$
$$= \log\frac{\sigma_2}{\sigma_1} + \frac{\sigma_1^2 + (\mu_1-\mu_2)^2}{2\sigma_2^2} - \frac{1}{2}$$

$$\mathcal{L}(x,\hat{x}) = MSE + KLD$$

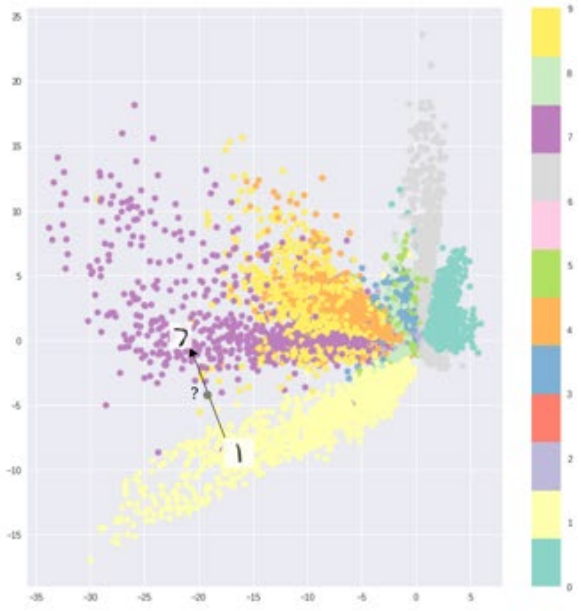
$$KLD = \frac{1}{2}\sum_i [1 + \log\sigma_i^2 - \mu_i^2 - \sigma_i^2]$$

Mean and standard derivation of our training sample embedding

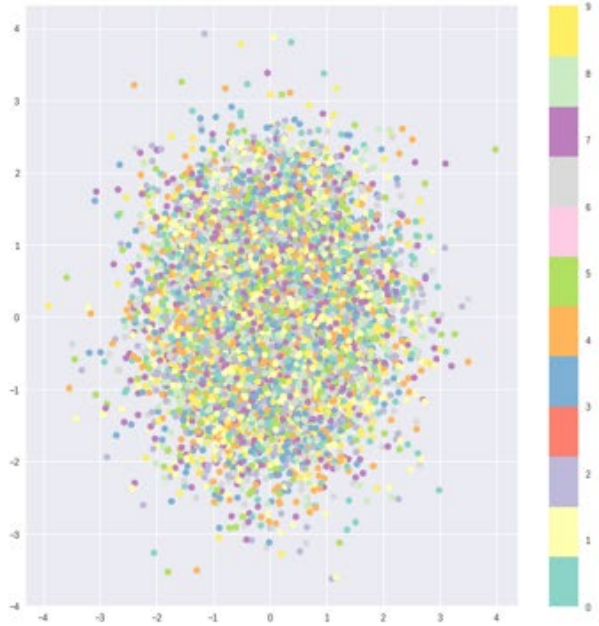
(Target distribution is taken as implicit; mean = 0 and std = 1)

Effect:

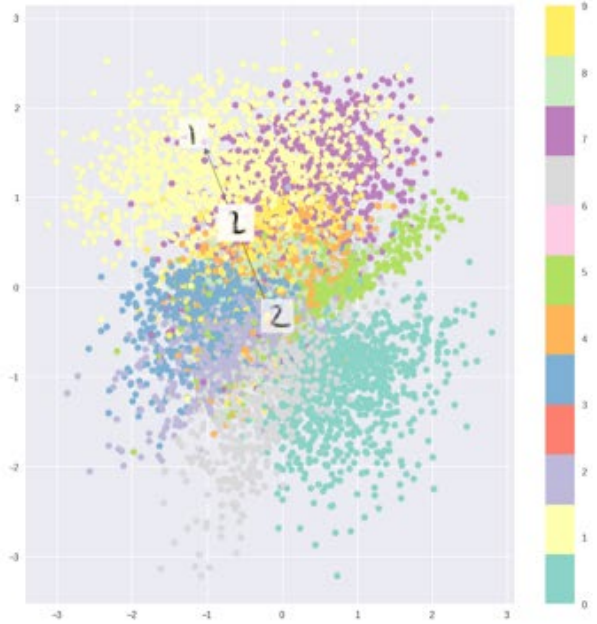
Only reconstruction loss



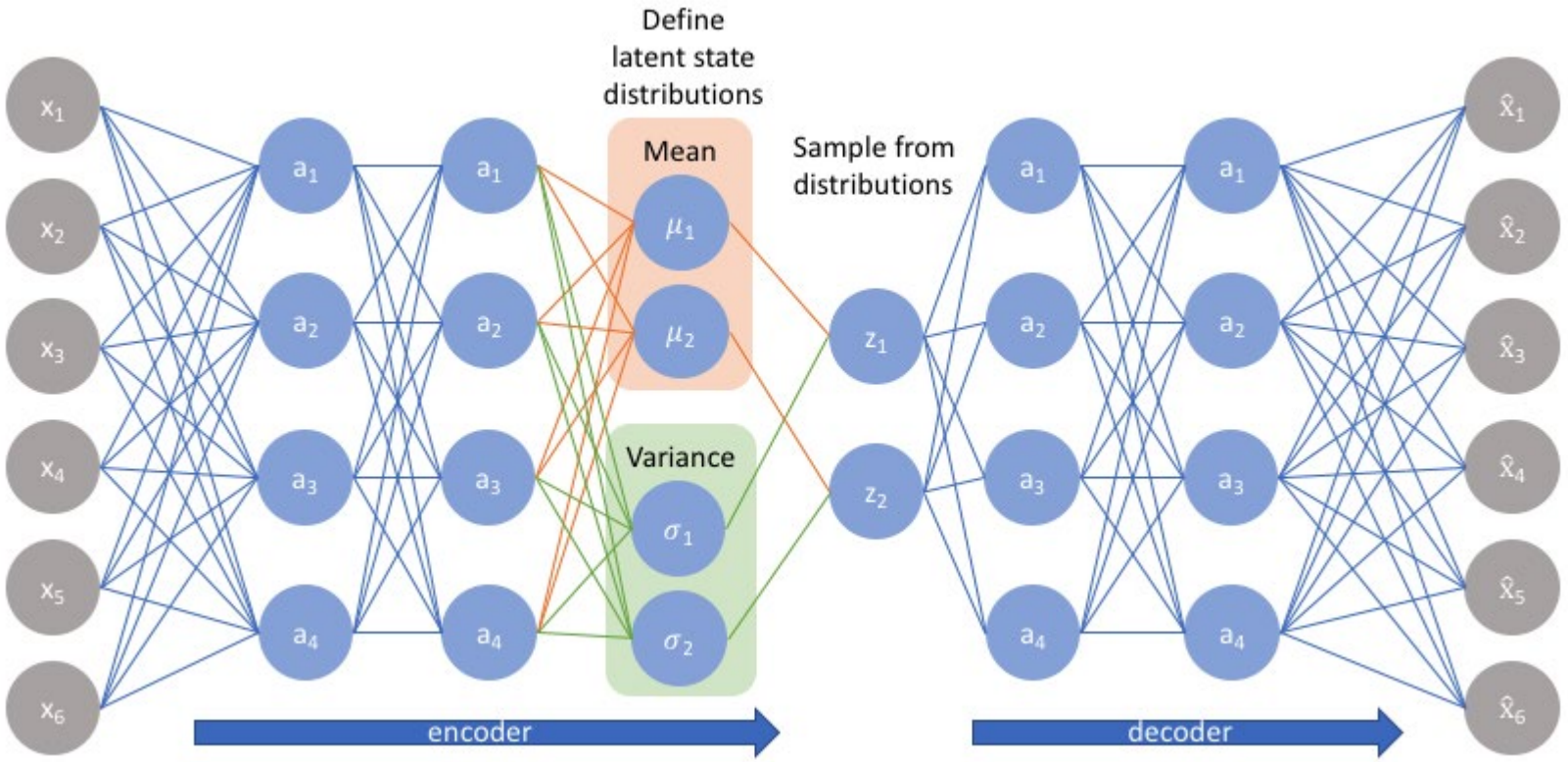
Only KL divergence



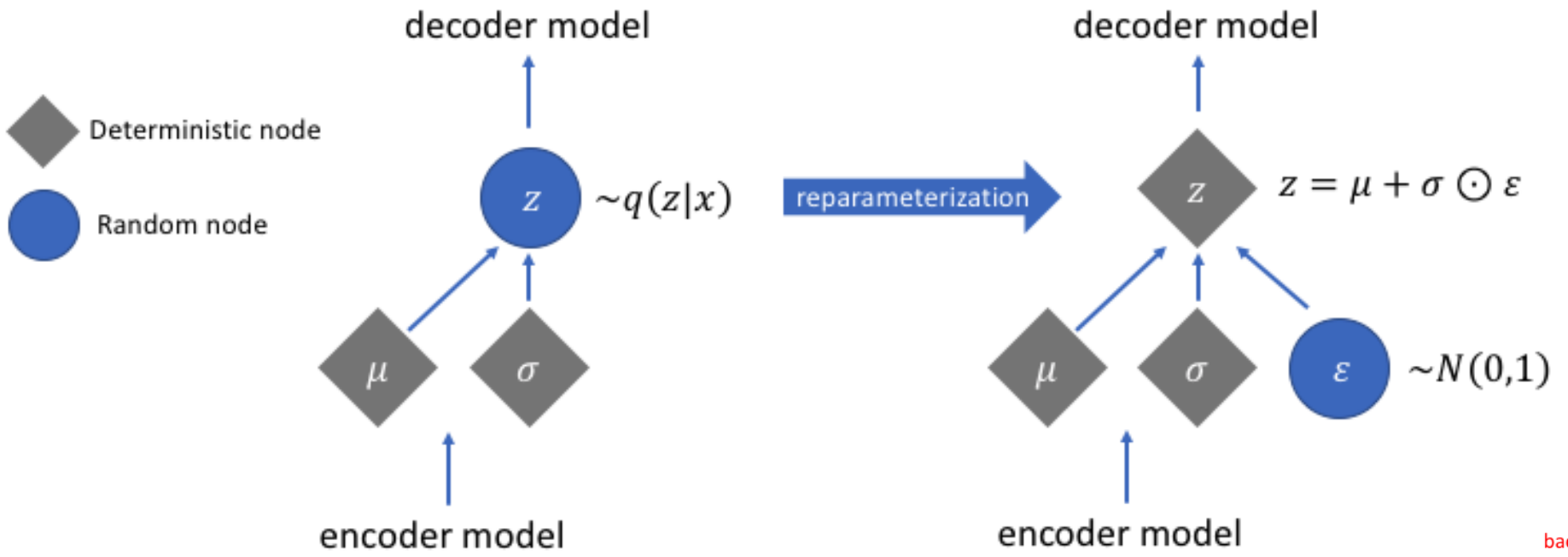
Combination

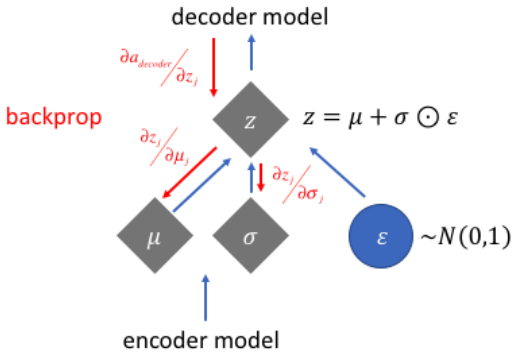


Architecture adjustments



Re-parametrisation





https://www.jeremyjordan.me/variational-autoencoders/

In code

```python
class VAE(torch.nn.Module):
    def __init__(self, input_size, latent_size):
        super(VAE, self).__init__()
        self.encoder = nn.Sequential(
            nn.Linear(input_size, latent_size),
            nn.LeakyReLU(True)
        )

        self.mean_layer = nn.Linear(latent_size, 2)
        self.logvar_layer = nn.Linear(latent_size, 2)

        self.decoder = nn.Sequential(
            nn.Linear(2, latent_size),
            nn.LeakyReLU(True),
            nn.Linear(latent_size, input_size)
        )

    def encode(self, x):
        x = self.encoder(x)
        mean = self.mean_layer(x)
        logvar = self.logvar_layer(x)
        return mean, logvar

    def reparameterization(self, mean, var):
        epsilon = torch.randn_like(var).to(device)
        z = mean + var * epsilon
        return z

    def decode(self, x):
        return self.decoder(x)

    def forward(self, x):
        mean, logvar = self.encode(x)
        z = self.reparameterization(mean, logvar)
        x_hat = self.decode(z)
        return x_hat, mean, logvar
```

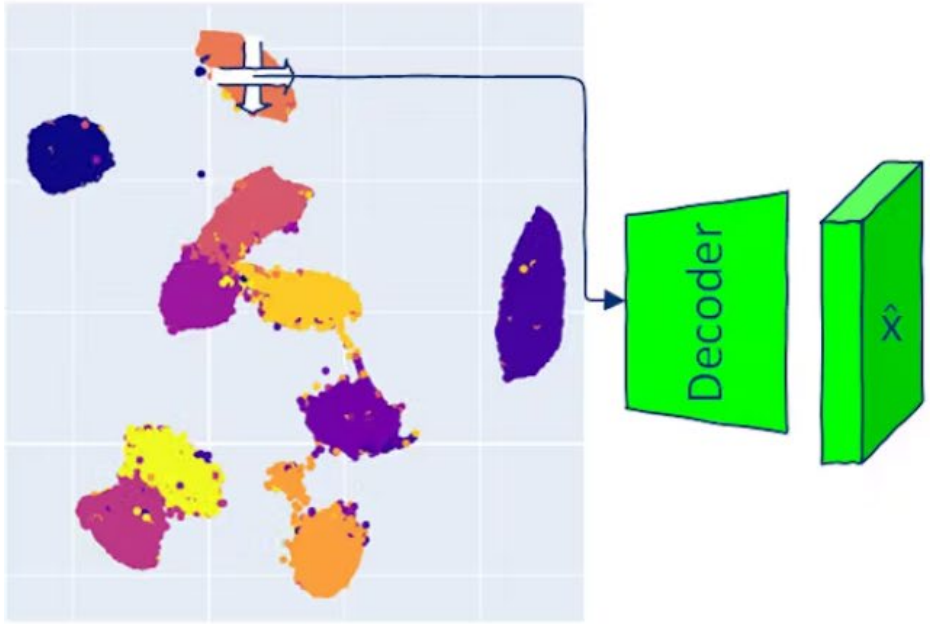
Compact variant

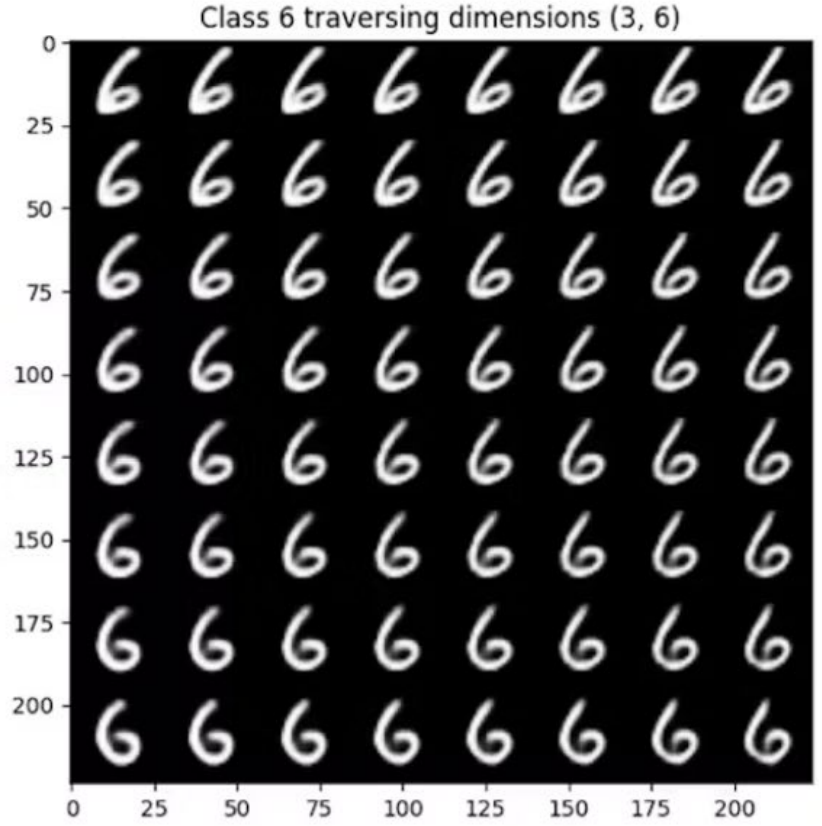
```python
class VAE(torch.nn.Module):
    def __init__(self, input_size, latent_size):
        super(VAE, self).__init__()
        self.encoder = nn.Sequential(
            nn.Linear(input_size, 128),
            nn.ReLU(True),
            nn.Linear(128, latent_size * 2)  # mean and logvar
        )
        self.decoder = nn.Sequential(
            nn.Linear(latent_size, 128),
            nn.ReLU(True),
            nn.Linear(128, input_size),
            nn.Sigmoid()
        )

    def encode(self, x):
        x = self.encoder(x)
        mean = x[:, :latent_size]
        logvar = x[:, latent_size:]
        return mean, logvar

    def reparameterization(self, mean, var):
        epsilon = torch.randn_like(var).to(mean.device)
        z = mean + var * epsilon
        return z

    def decode(self, x):
        return self.decoder(x)

    def forward(self, x):
        mean, logvar = self.encode(x)
        z = self.reparameterization(mean, logvar)
        x_hat = self.decode(z)
        return x_hat, mean, logvar
```

If you have labels, use them!

⇒ Conditioning

Sample from gaussians

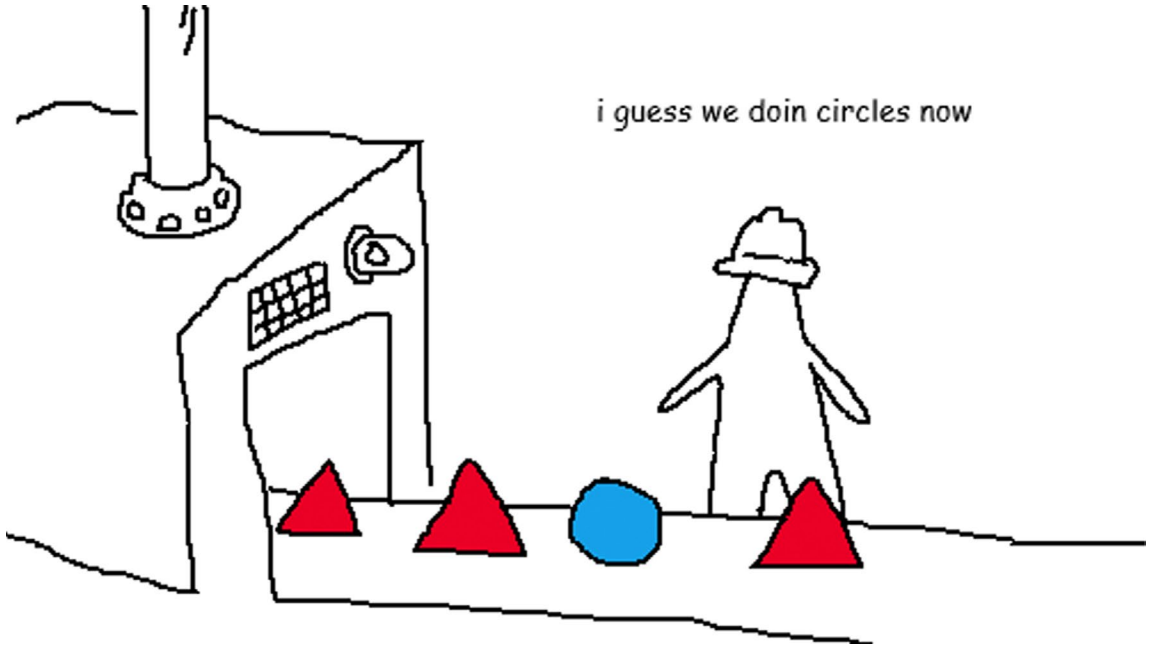




https://hackernoon.com/how-to-sample-from-latent-space-with-variational-autoencoder

Case example:

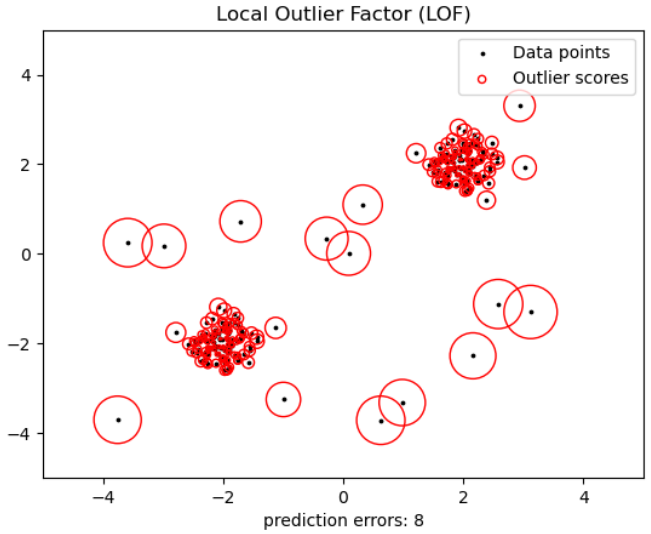
Anomaly Detection



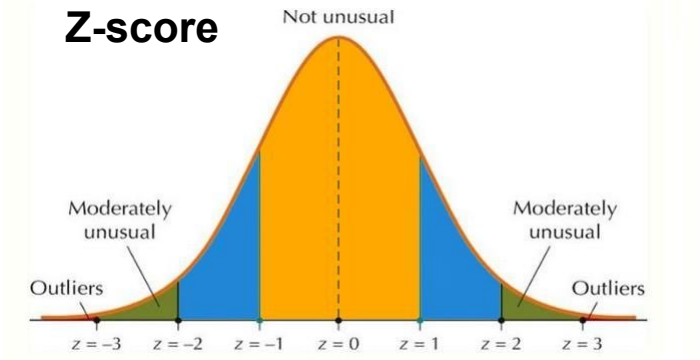
Anton D. Lautrup ~ *lautrup@imada.sdu.dk*

Anomaly detection

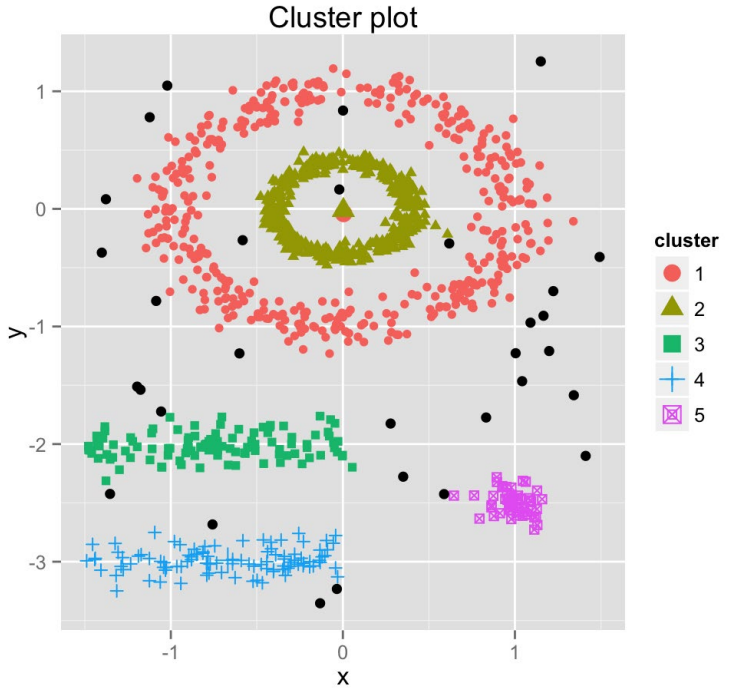
Density based



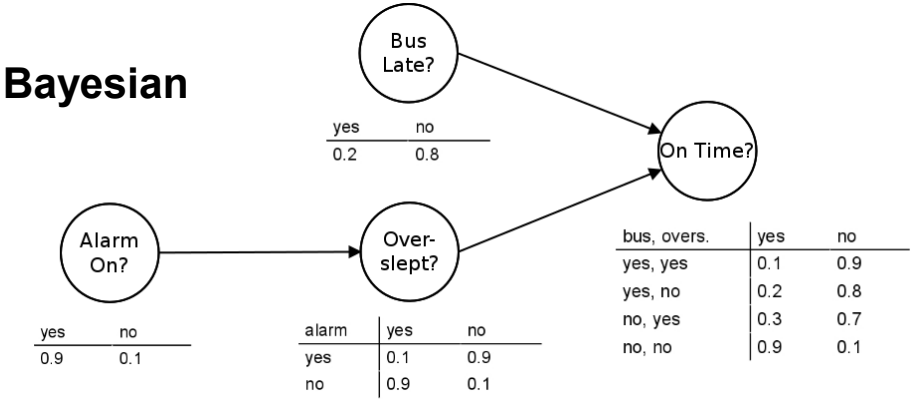
Z-score



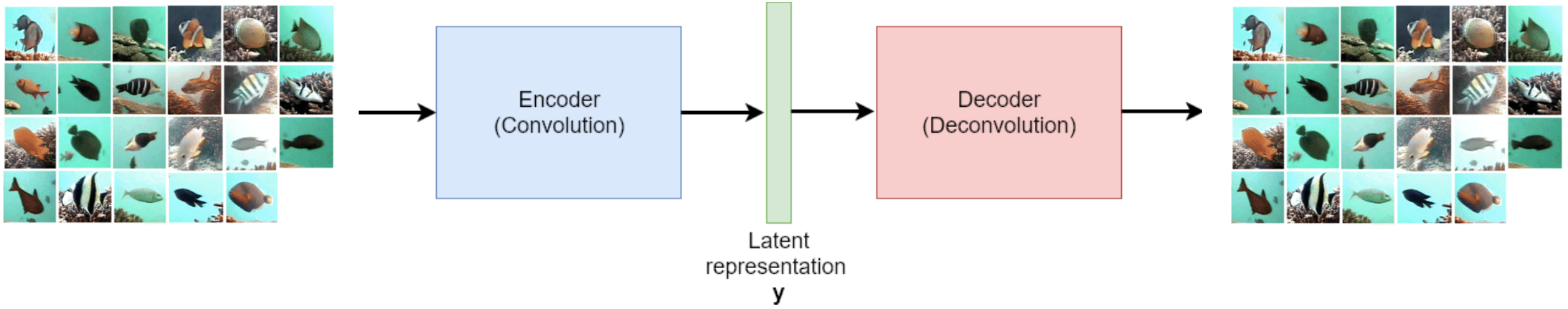
Cluster based

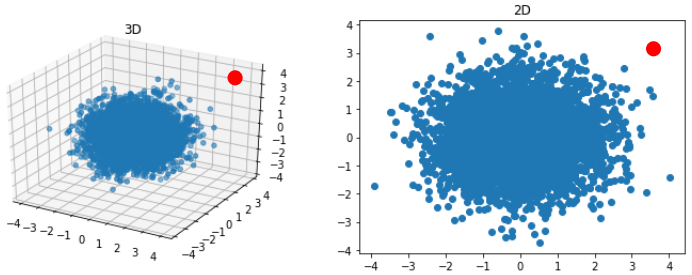


Bayesian



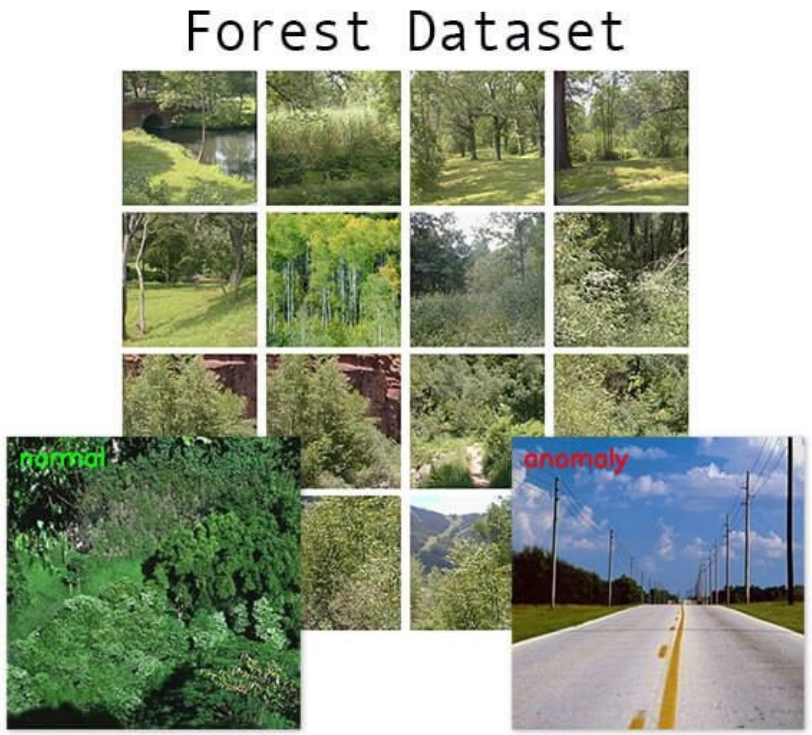
Reconstruction based anomalies

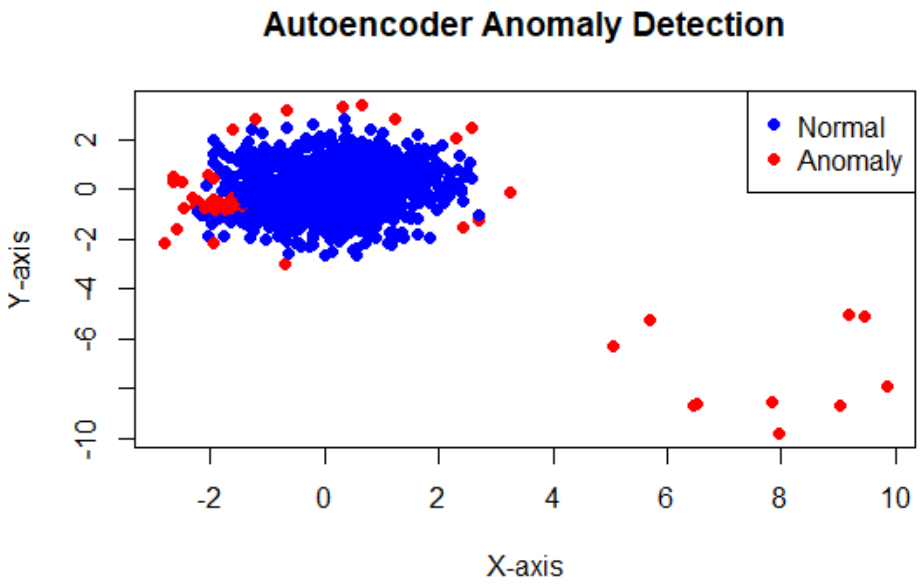




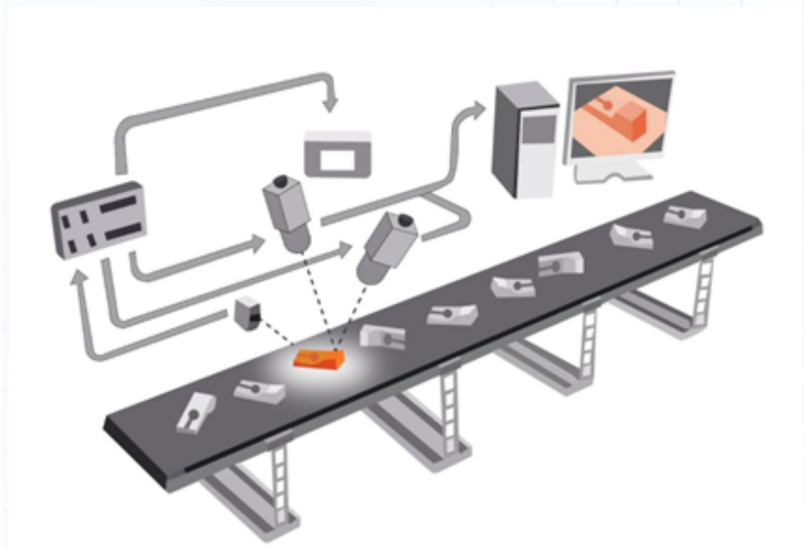
E(rec_loss) << anomalous rec_loss

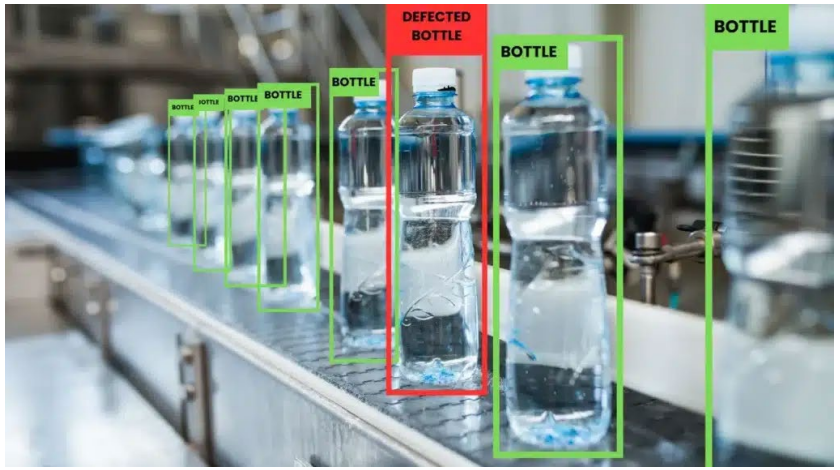
Unsupervised anomaly detection

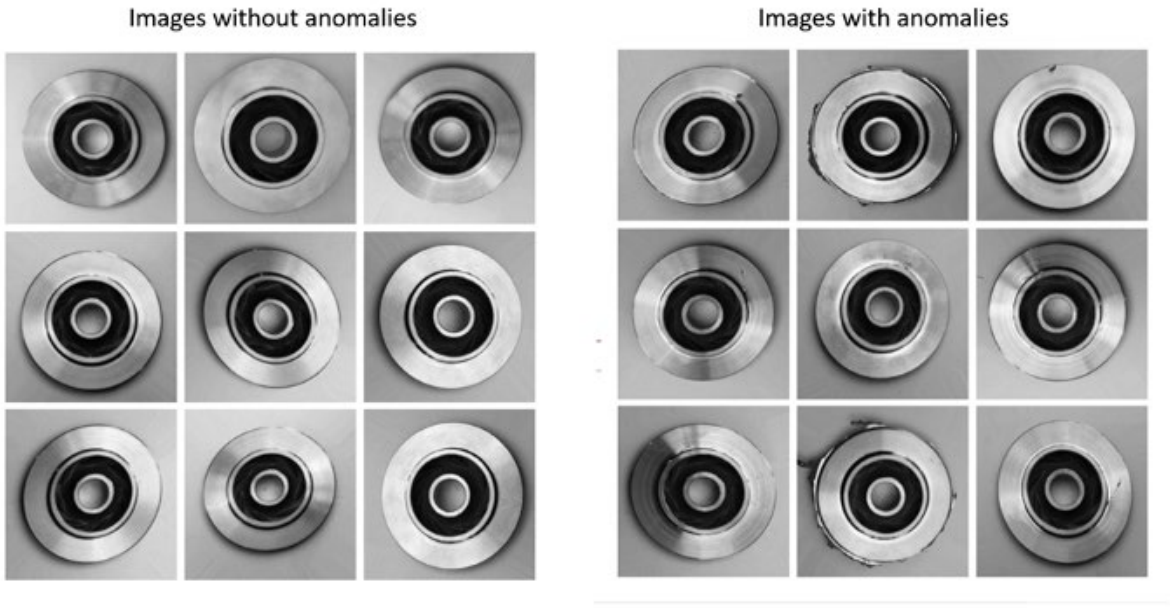




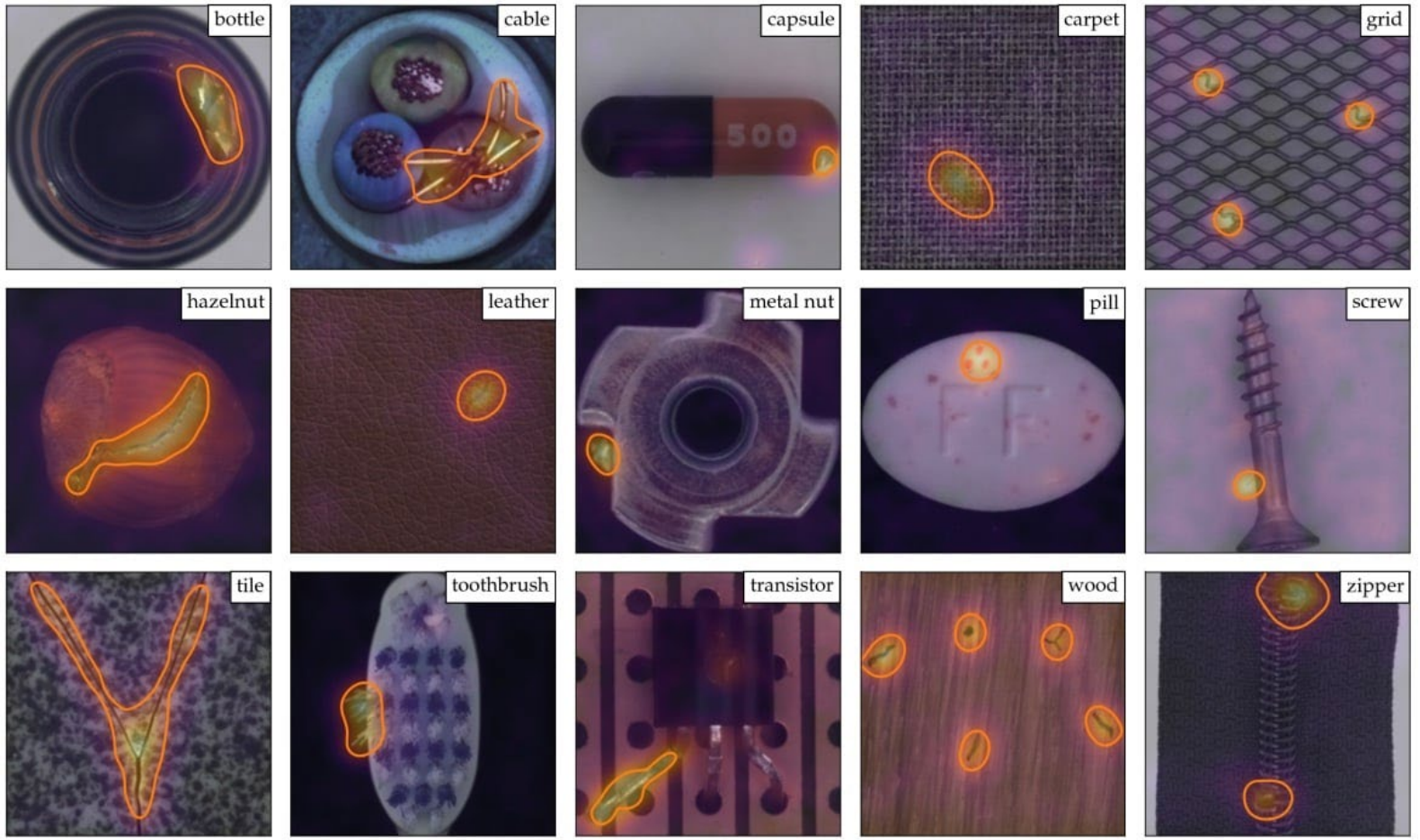
Practical case



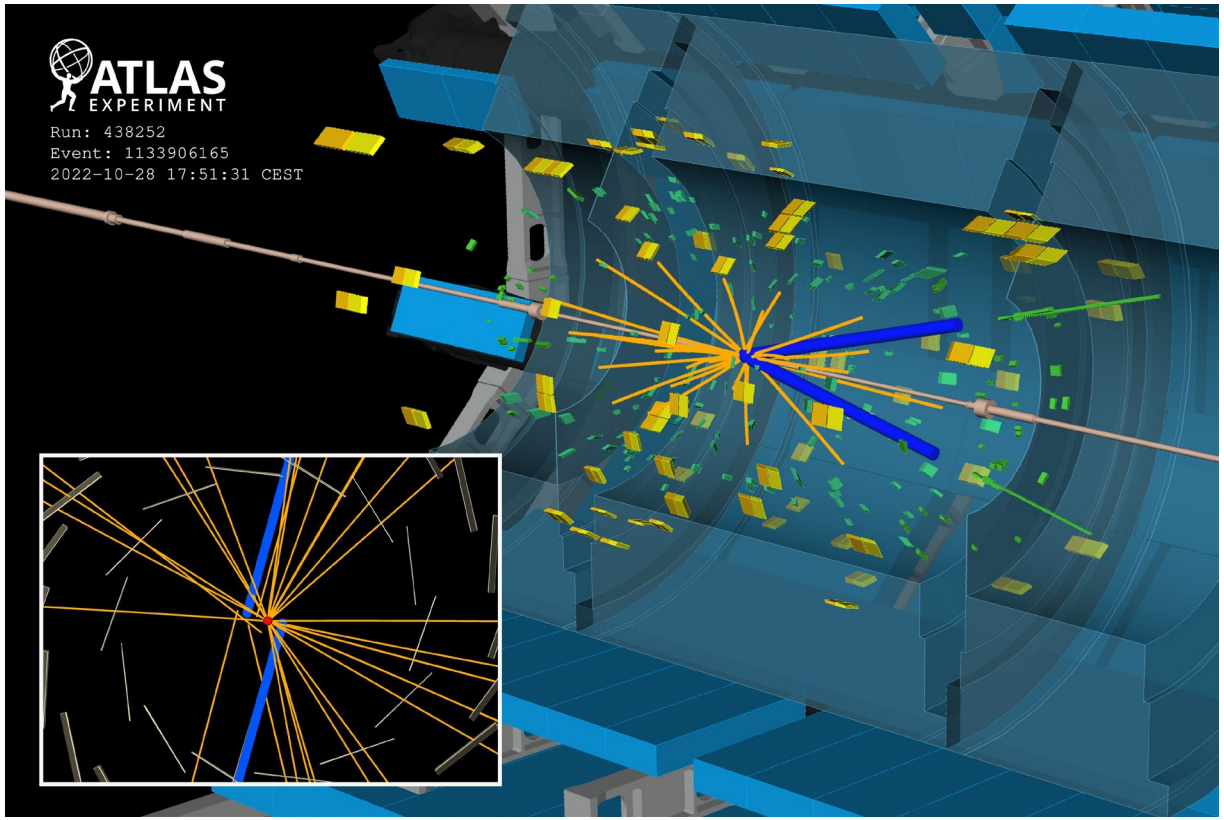


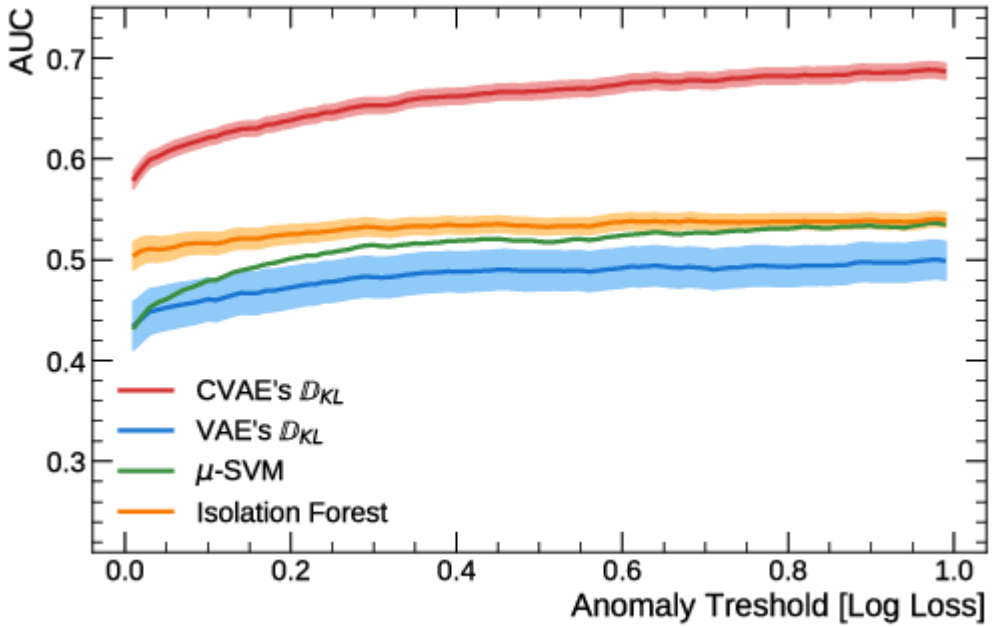


Anomaly + explanations



Practical case





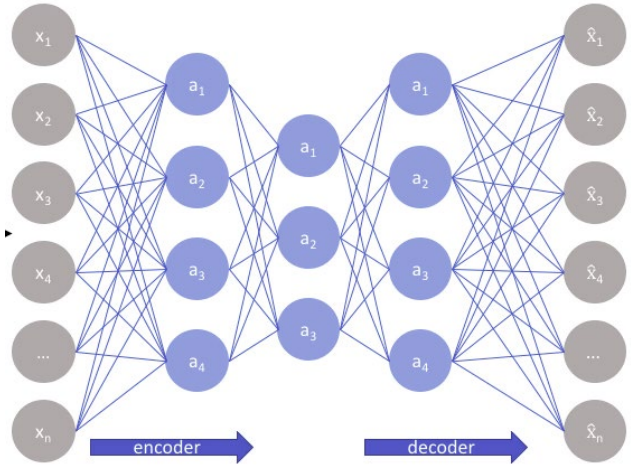
Case example:

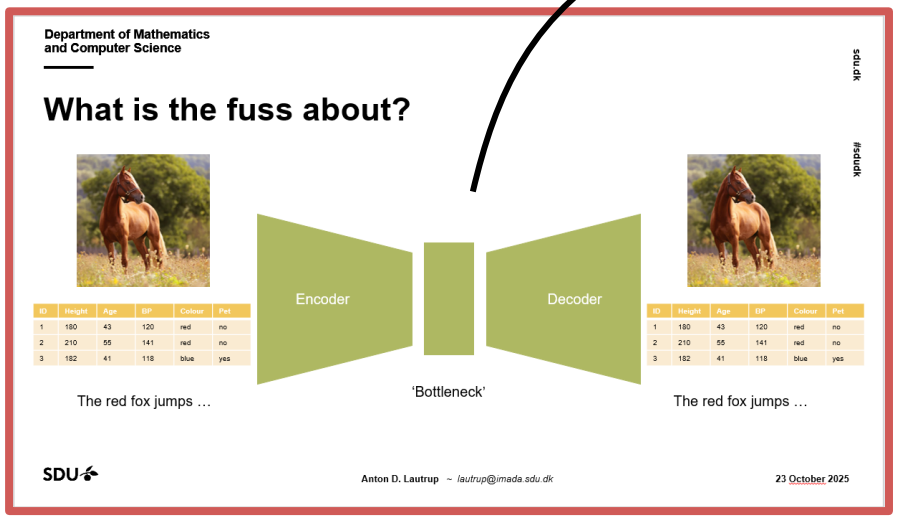
Interpretability with Sparse Autoencoders

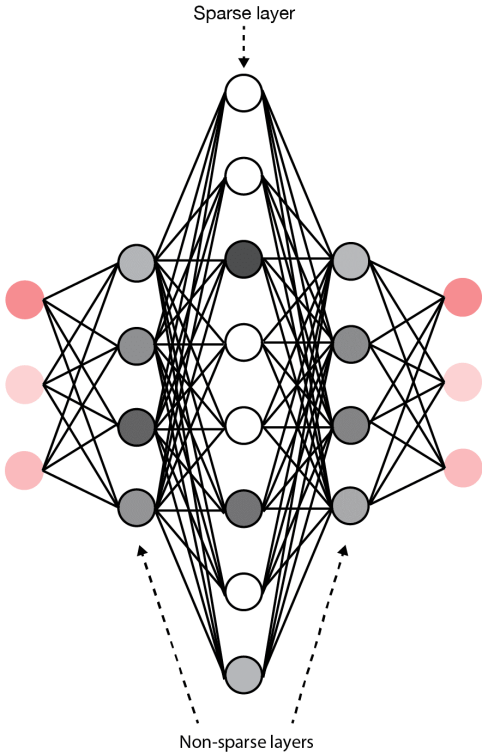
SDU

Anton D. Lautrup ~ *lautrup@imada.sdu.dk*

Compression vs sparsity







Sparsity constraint

→ Build an autoencoder that can disentangle rich semantic model states into human interpretable features (higher-dimensional but sparse).

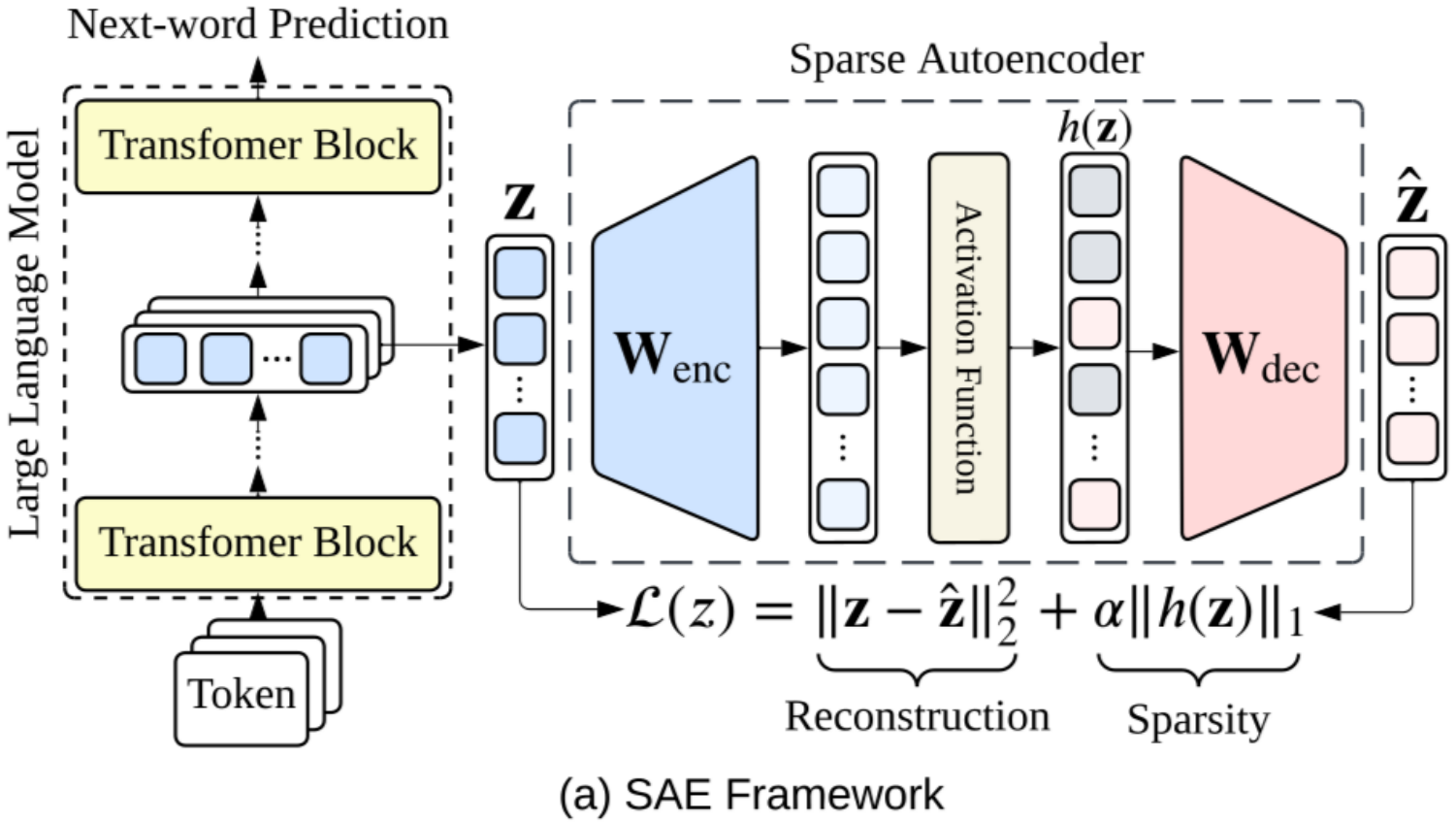
"L1 regularisation" on the embedding

$$\mathcal{L}(x, z) = \frac{1}{2}|g(f(x)) - x|^2 + \alpha|z|$$

(regular rec. loss)

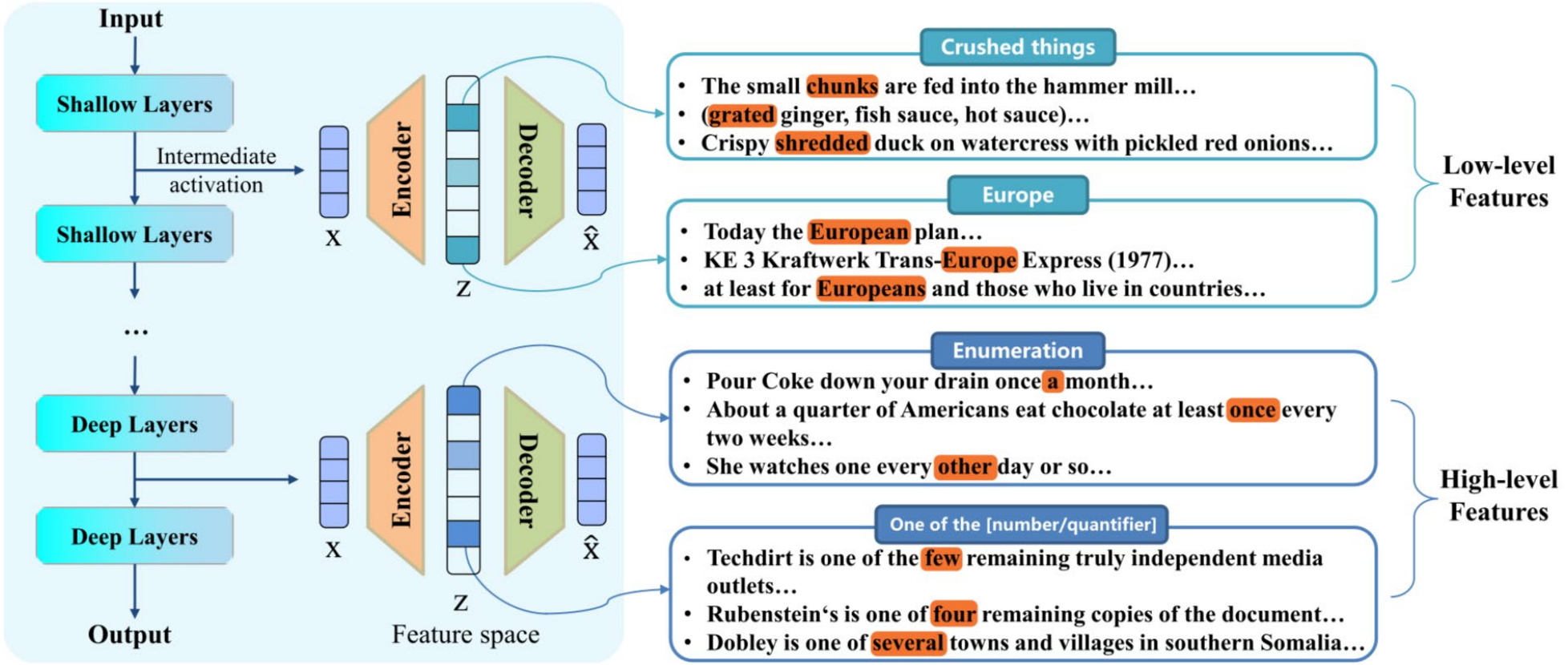
(strength parameter)

LLM Explanations



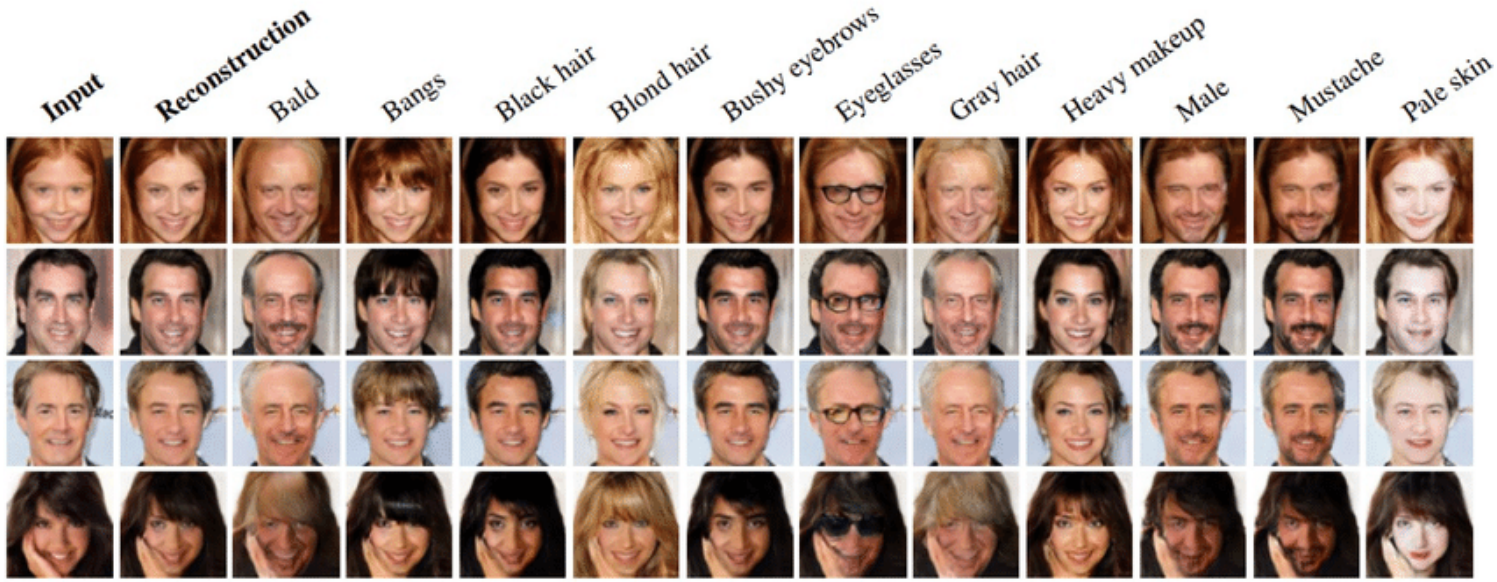
(a) SAE Framework

LLM Explanations



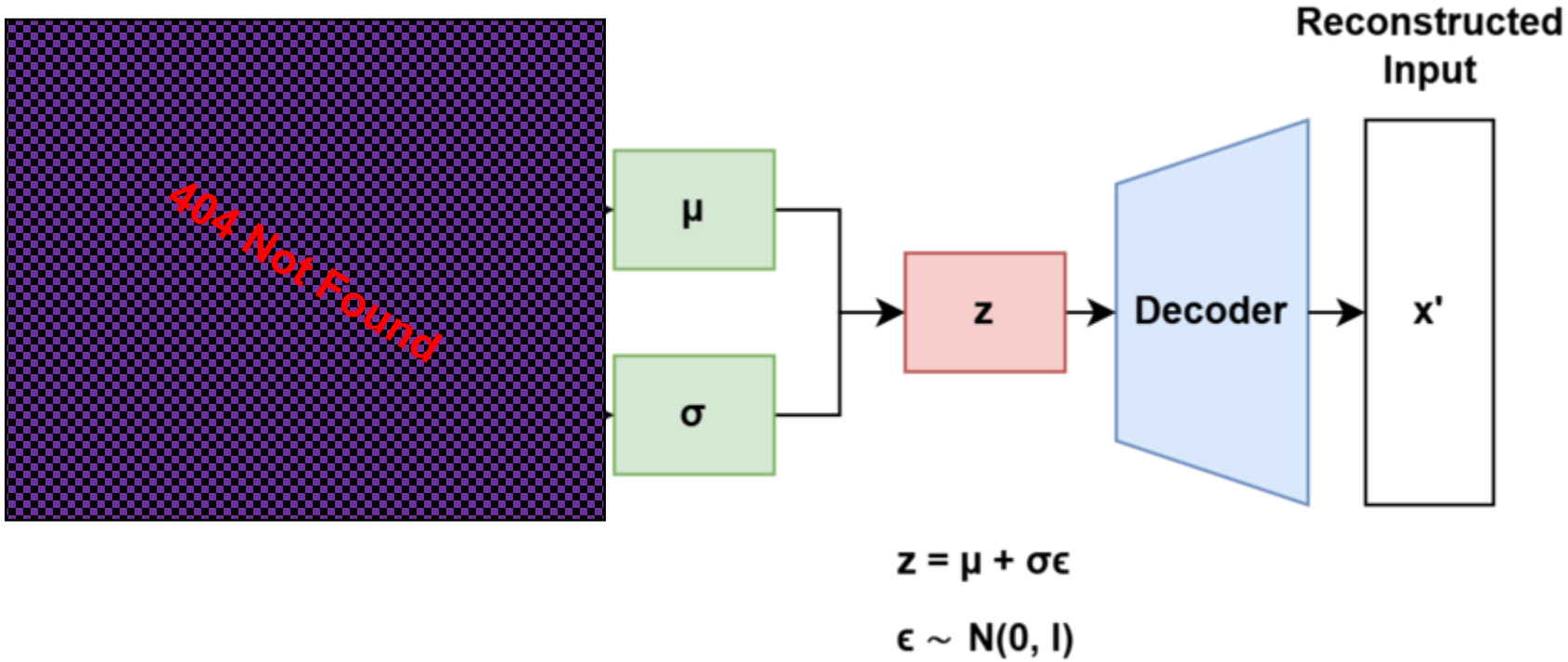
Case example:

Generative Model

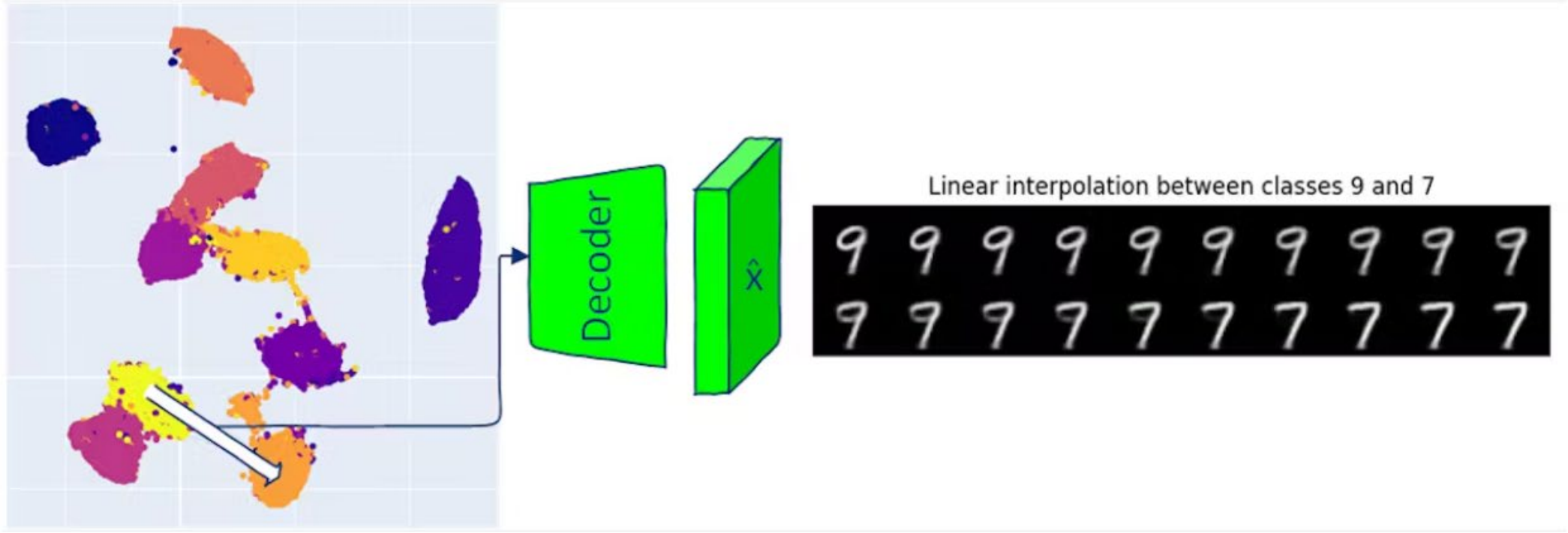


Anton D. Lautrup ~ *lautrup@imada.sdu.dk*

Generative VAE



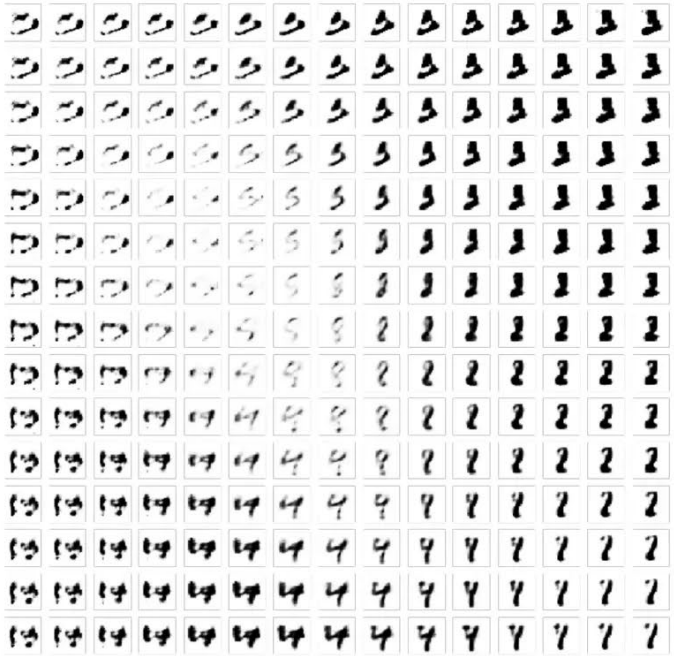
Not a very far-fetched next step

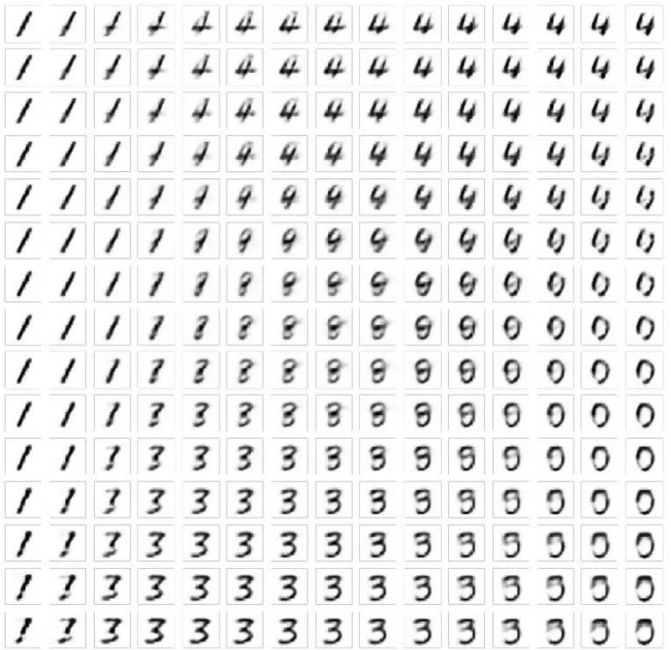


Steven Flores, https://www.slideshare.net/slideshow/variational-autoencoders-for-image-generation/142217349

AE vs VAE digit generation

left = AE generated digits, right = VAE generation digits (10-D latent space).





Blessing/Curse of dimensionality

EXAMPLE 1 Consider how many *almost orthogonal* unit vectors we can have in space, such that all pairwise angles lie between 88 degrees and 92 degrees.

In $\Re^2$ the answer is 2. In $\Re^3$ it is 3. (Prove these to yourself.)

In $\Re^d$ the answer is $\exp(cd)$ where $c > 0$ is some constant.

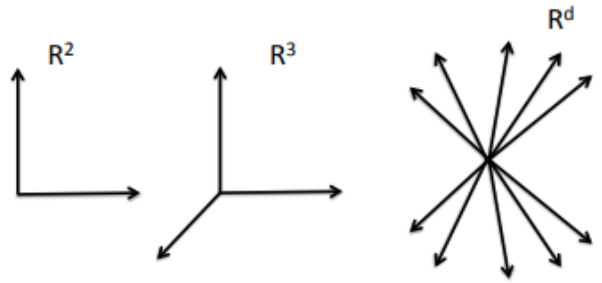
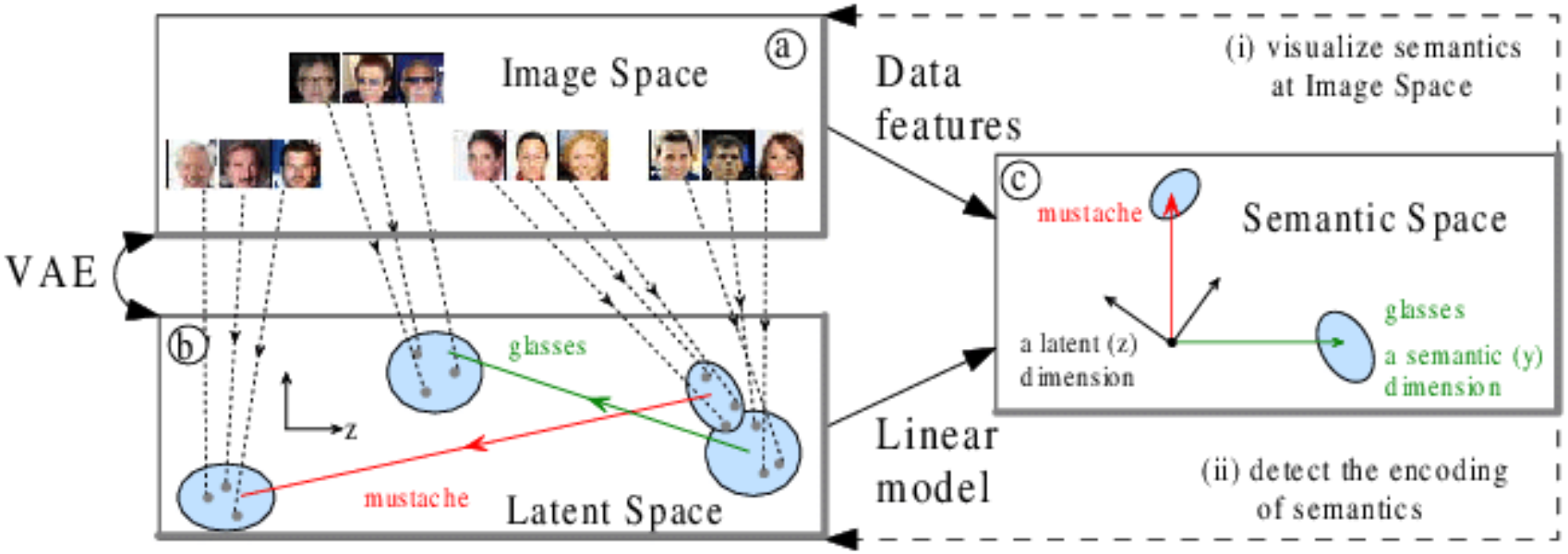


Figure 1: Number of almost-orthogonal vectors in $\Re^2, \Re^3, \Re^d$

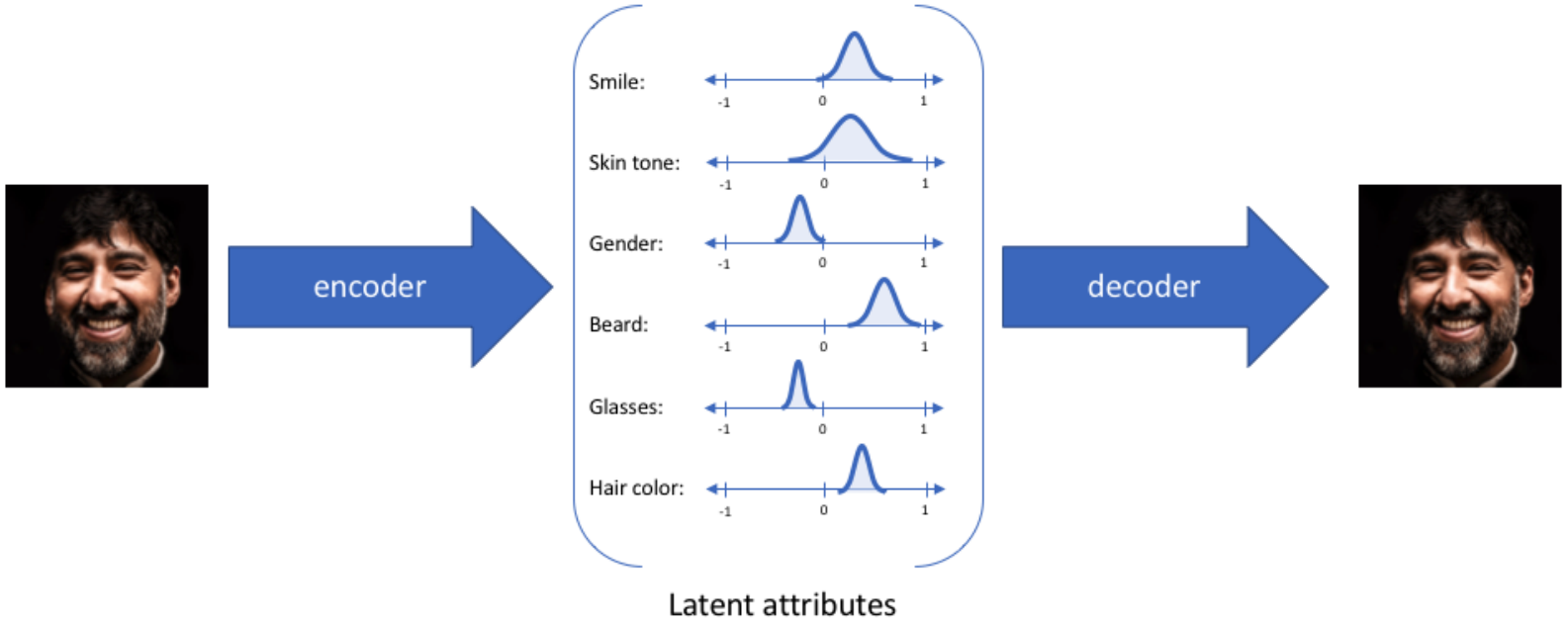
Semantic vs. specific dimensions



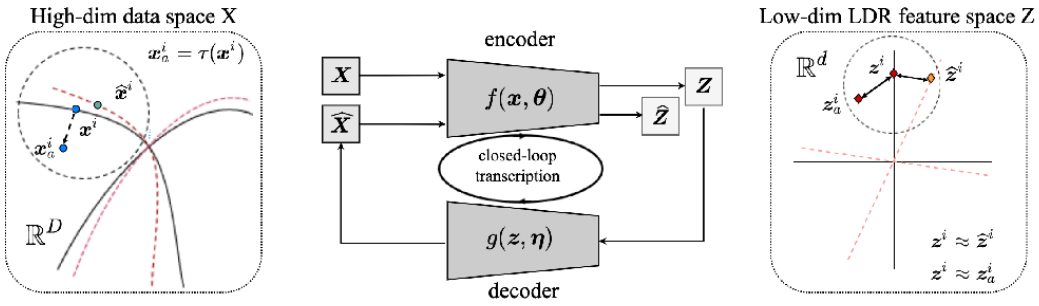
$$\neq \begin{pmatrix} z_1 \\ z_2 \\ z_3 \\ z_4 \\ z_5 \\ \vdots \\ z_d \end{pmatrix}$$

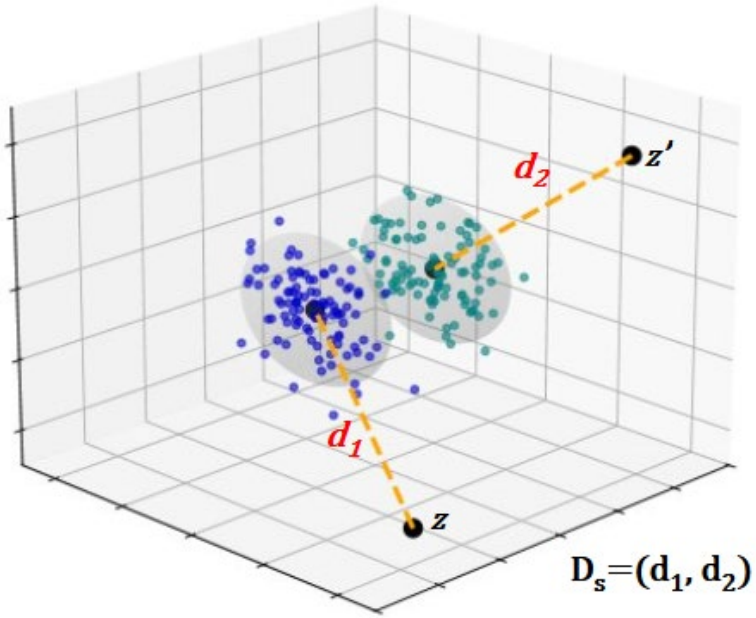
LatentVis: Investigating and Comparing Variational Auto-Encoders via Their Latent Space. https://ceur-ws.org/Vol-2699/paper04.pdf

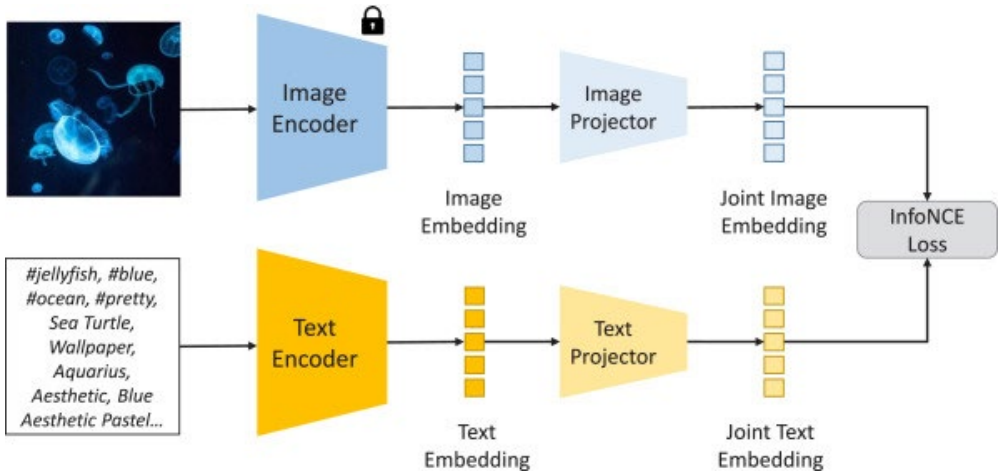
Impose meaning on the latent variables



Everything is embeddings

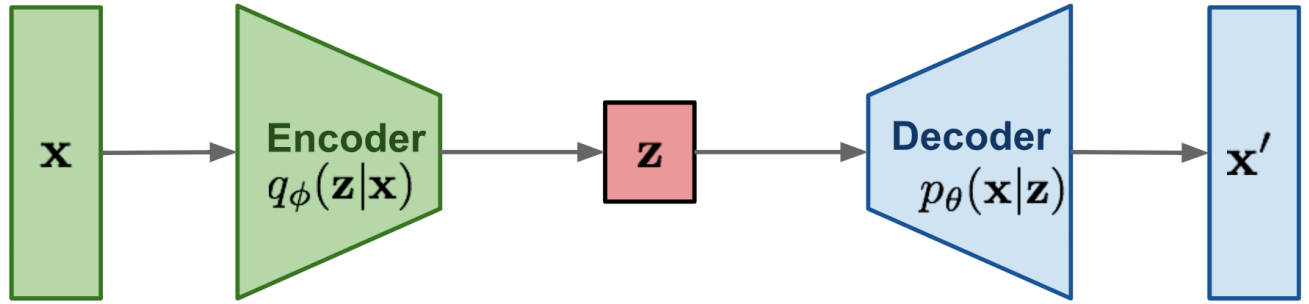




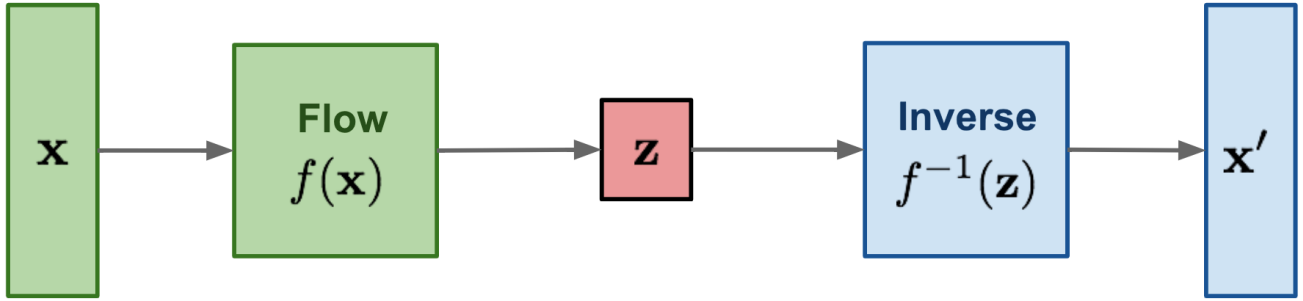


Flow Models:
Just to let you know it exists

VAE: maximize ELBO.



Flow-based generative models: minimize the negative log-likelihood



https://arxiv.org/pdf/1908.09257

Questions?

Generative Models

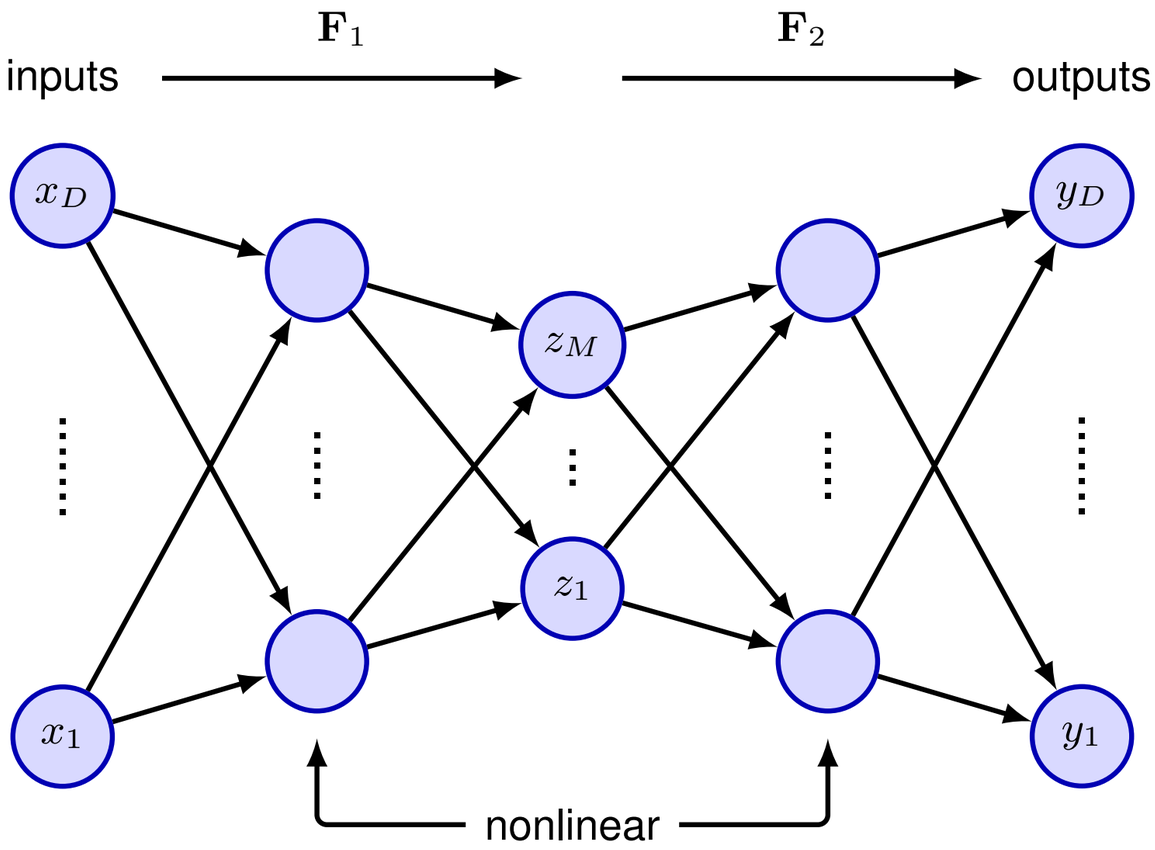
AI506: Advanced Machine Learning

Spring 2026

Lukas Galke

Brief recap: Autoencoders come in many variants

- Linear/nonlinear AEs
- Shallow/deep AEs
- Under-/overcomplete AEs
- Regularized AEs
  - Denoising AEs
  - Sparse AEs
  - Variational AEs
  - …



What are generative models

The goal is to learn the underlying data-generating distribution $p(x)$, such that you can sample from it.

Energy-based Models
and why Generative Modeling is hard!

Idea: Let's just learn an arbitrary model to predict probabilities $p(x)$.

Energy based models: $p(x) \approx \frac{e^{-\beta f_\theta(x)}}{Z(\theta)}$

where Z is the partition function, used for normalizing the output of the energy-based model to form a probability distribution.

You can express $Z = \int_{x \in \mathcal{X}} e^{-\beta f_\theta(x)}\, dx$

But integrating over the whole space $\mathcal{X}$ is usually intractable.

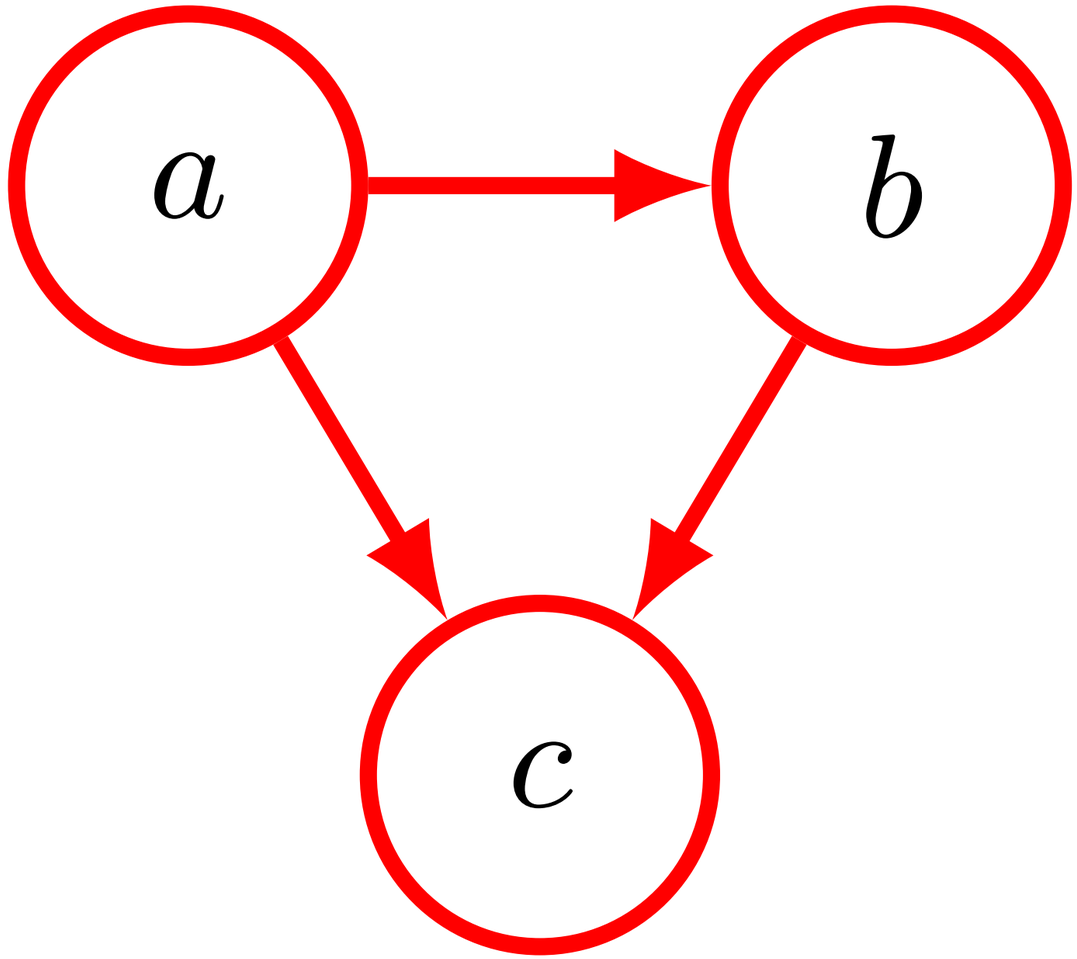
Keywords

- Energy-based models
- Probabilistic graphical models
- Bayesian Networks vs. Markov Networks
- Neural networks as generative models
- Generative adversarial networks

Probabilistic Graphical Models

Directed Graphs

- A set of nodes
- A set of edges, each connecting two nodes.
  In probabilistic graphical models:
- Each node is a random variable
- Each edge is a probabilistic relationship between the random variables



Factorization

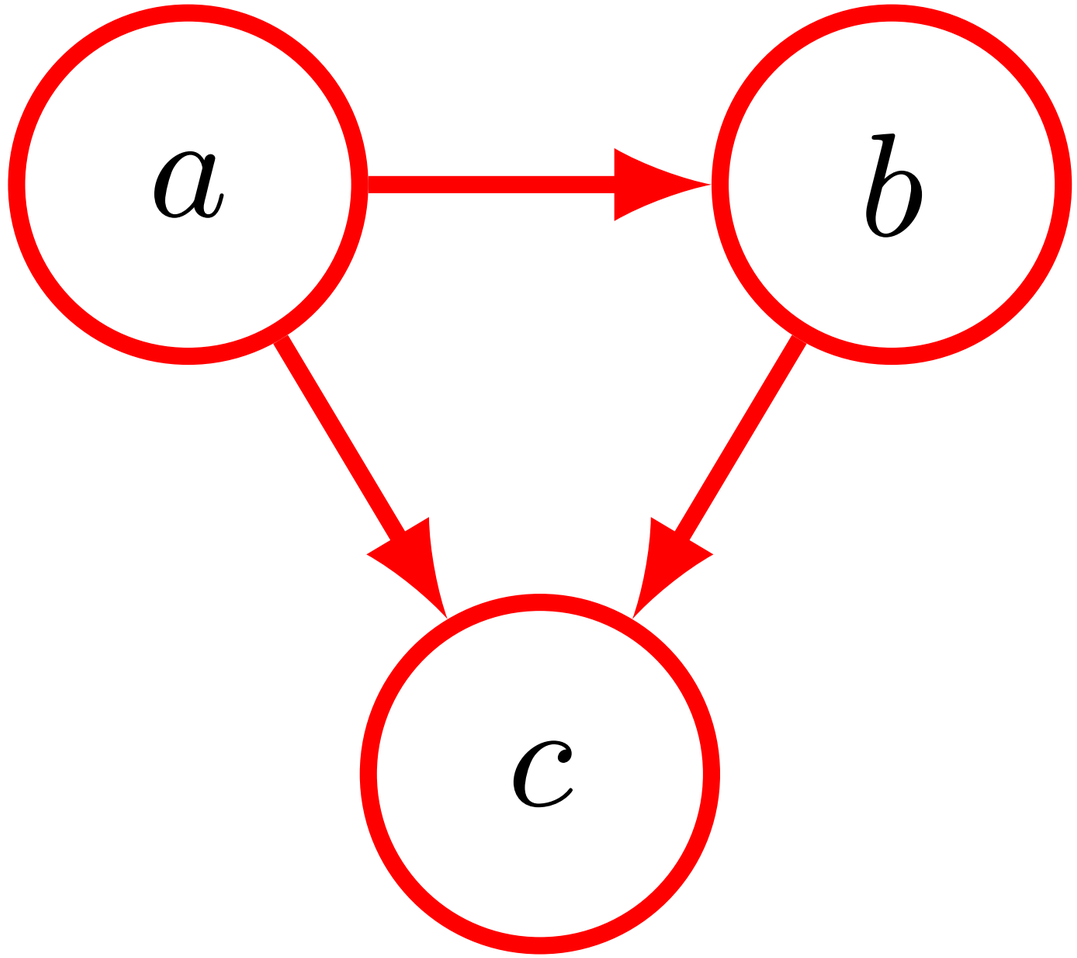
Any joint probability distribution can be factorized.

$$p(a, b, c) = p(c|a, b)p(a, b)$$

and further:

$$p(a, b, c) = p(c|a, b)p(b|a)p(a)$$

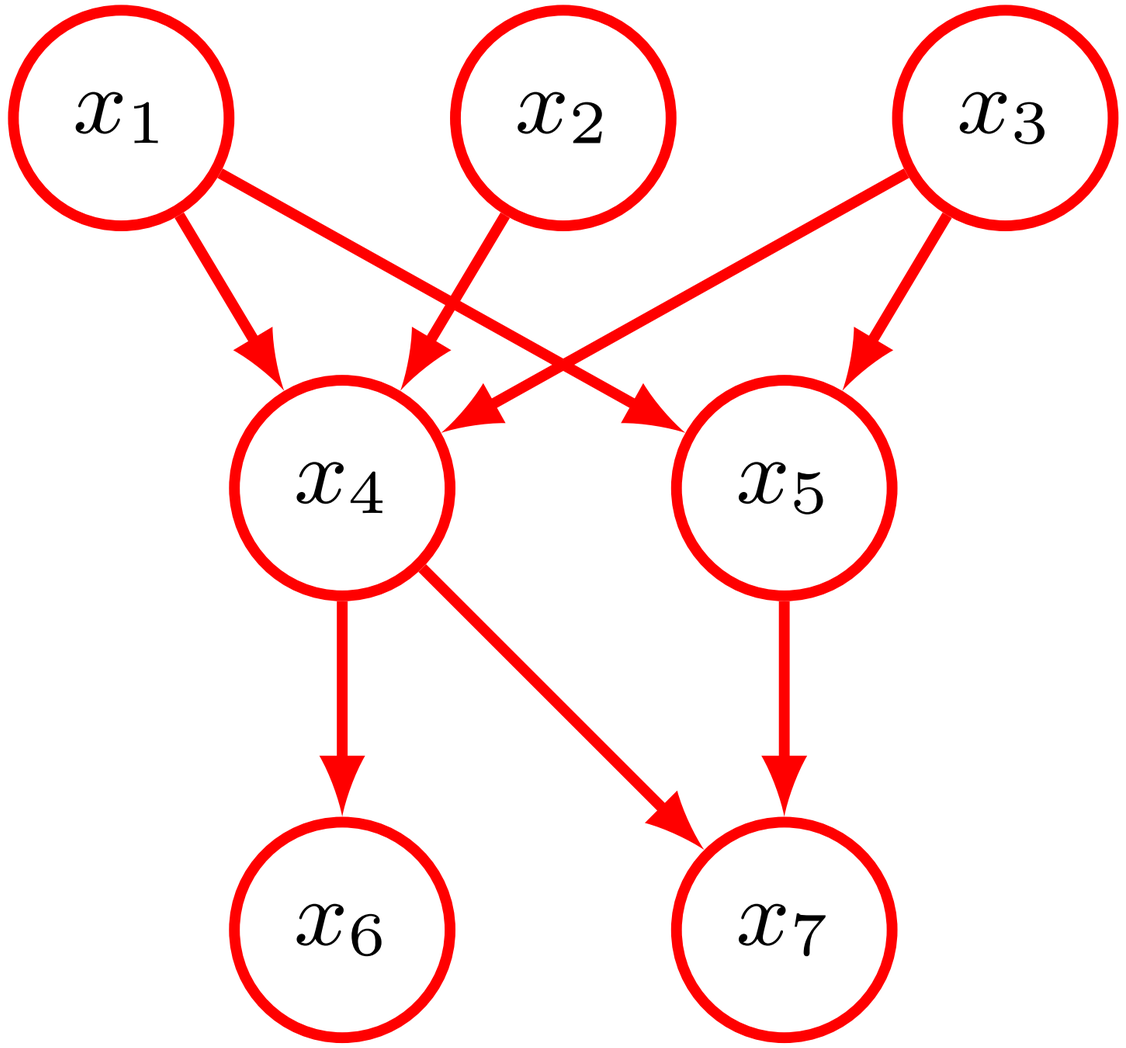
Note that an arrow $x \longrightarrow y$ reads is interpreted as the conditional probability $p(y|x)$



A more complex example

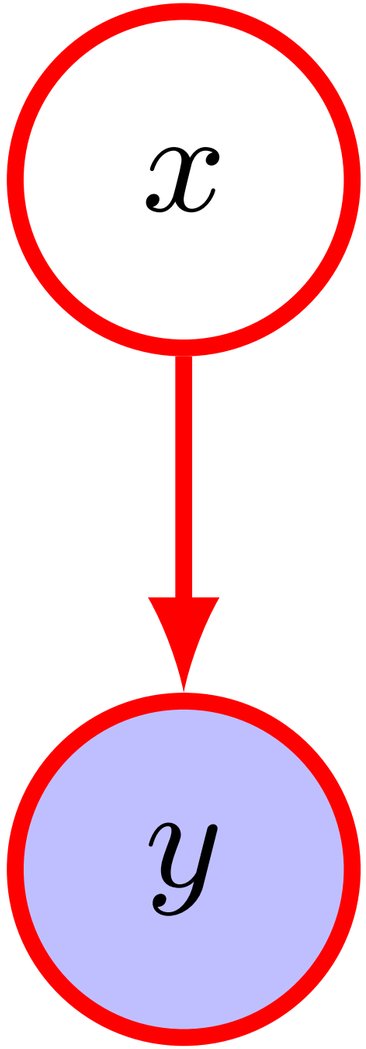
$$p(x_1, x_2, x_3) =$$
$$p(x_1)p(x_2)p(x_3) \cdot$$
$$p(x_4|x_1, x_2, x_3)p(x_5|x_1, x_3) \cdot$$
$$p(x_6|x_4) \cdot p(x_7|x_4, x_5)$$

Nodes can be discrete (e.g., Bernoulli) or continuous (e.g., Gaussian)



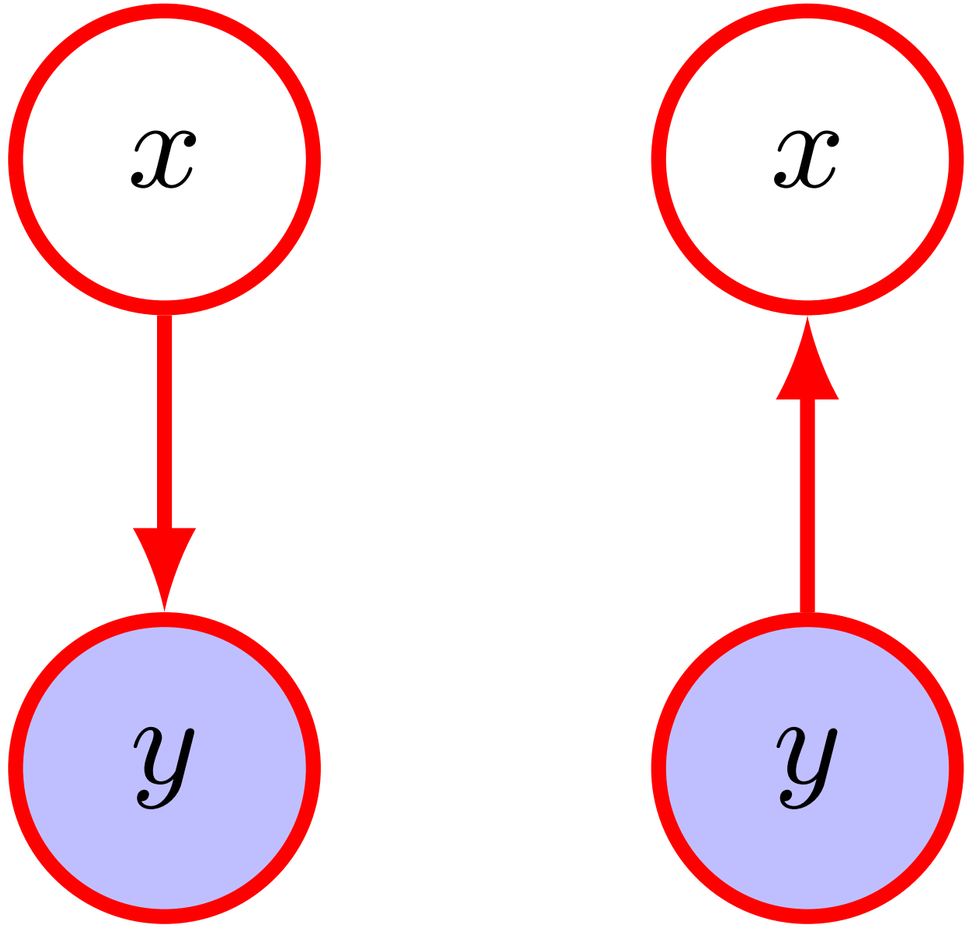
Observed and latent variables

- Nodes can correspond to **observed** = what we see in the data
- …or **latent/hidden** variables = underlying factors of variation



Inference

- Suppose we observe a value of y
- Prior distribution $p(x)$
- $p(x|y) = \frac{p(y|x)p(x)}{p(y)}$ (Bayes)
- We can update our $p(x)$ accordingly to what was observed
- Similar calculations can be done for more complex graphs.



Expectation-Maximization algorithm

1. Initialize parameters with an initial guess
2. Expectation step: Calculate the probability of each data point belonging to each possible hidden variable based on current parameters.
3. Update the parameters to maximize the log-likelihood of the data, using probabilities calculated in the E-step.
4. Iterate: Repeat 2. And 3. until parameters stabilize.

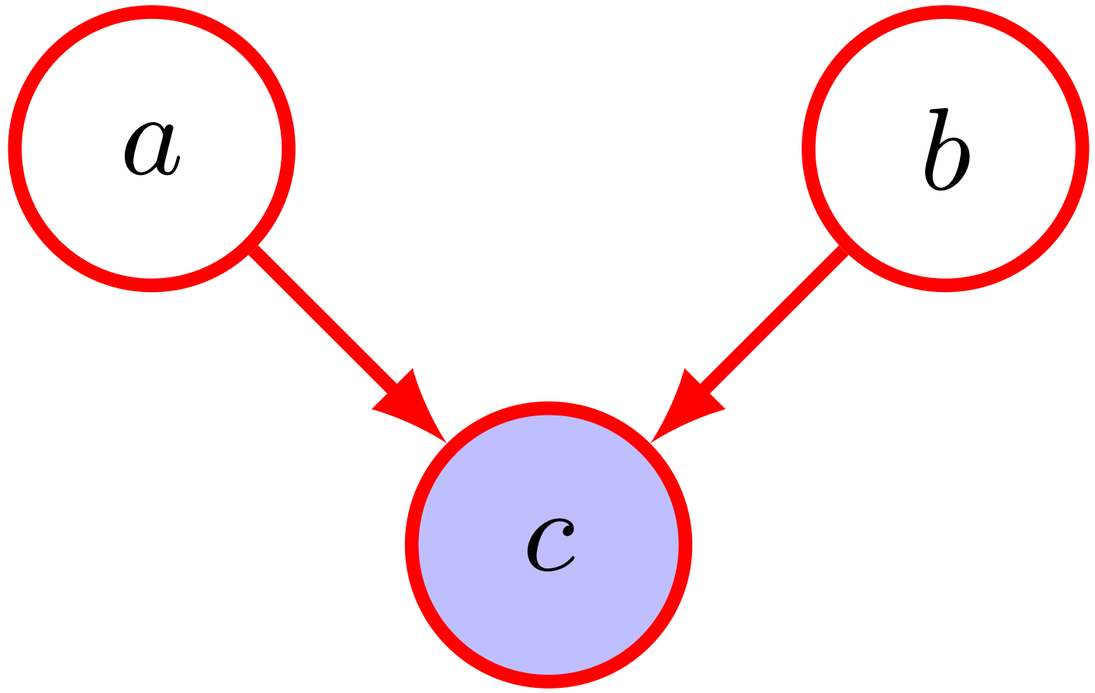
Key Problem: Explaining away effect

Two independent causes become conditionally dependent given their common effect.

Example: Earthquake (a) and Burglary (b) may both cause an alarm (c).

An observed alarm (c) will increase our belief of both earthquake (a) and burglary (b).

But if we then knew it was an actually an earthquake (a), we can decrease our belief in a burglary (b).

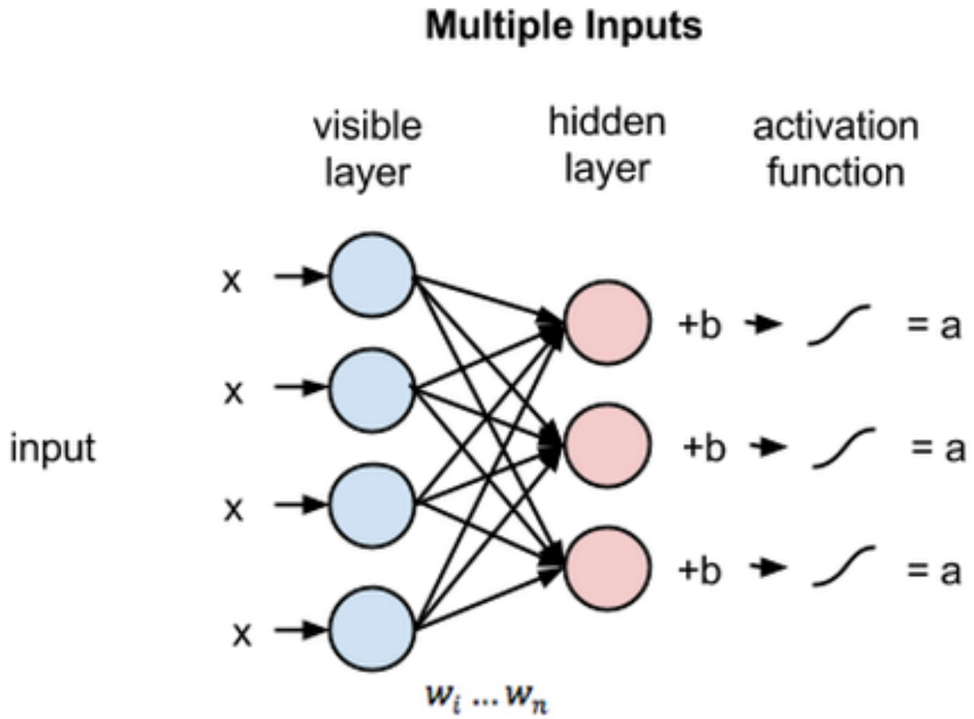


Major families of probabilistic graphical models

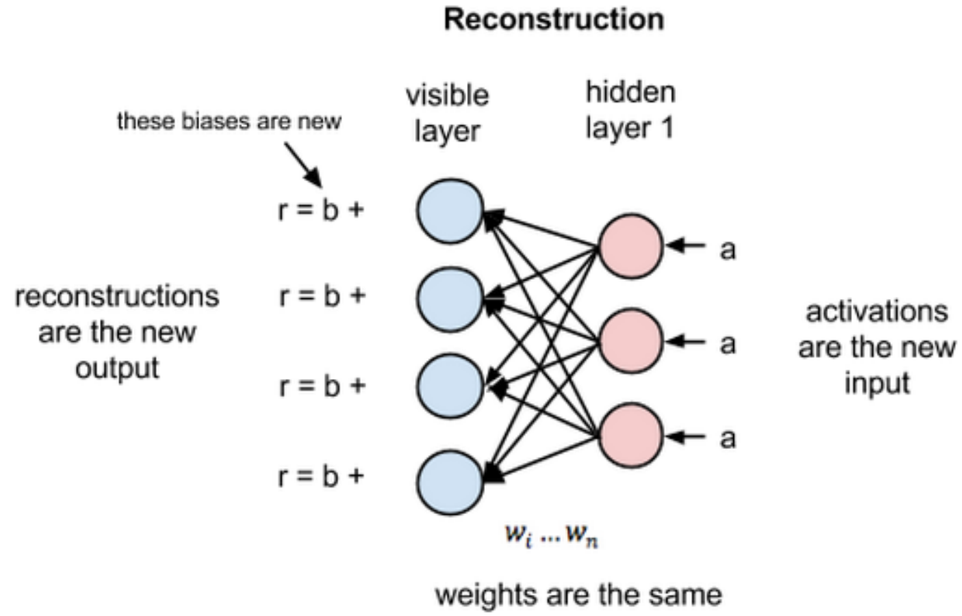
1. **Bayesian networks**: Directed acyclic graph
2. **Markov networks**: Undirected graph
   - Special case: **Restricted Boltzmann Machines** (1 layer of observed, and 1 layer of hidden units).

Restricted Boltzmann Machines are similar to Autoencoders

Step 1: Inferring latent variables from observed ones



Step 2: Reconstructing observed variables from latent ones



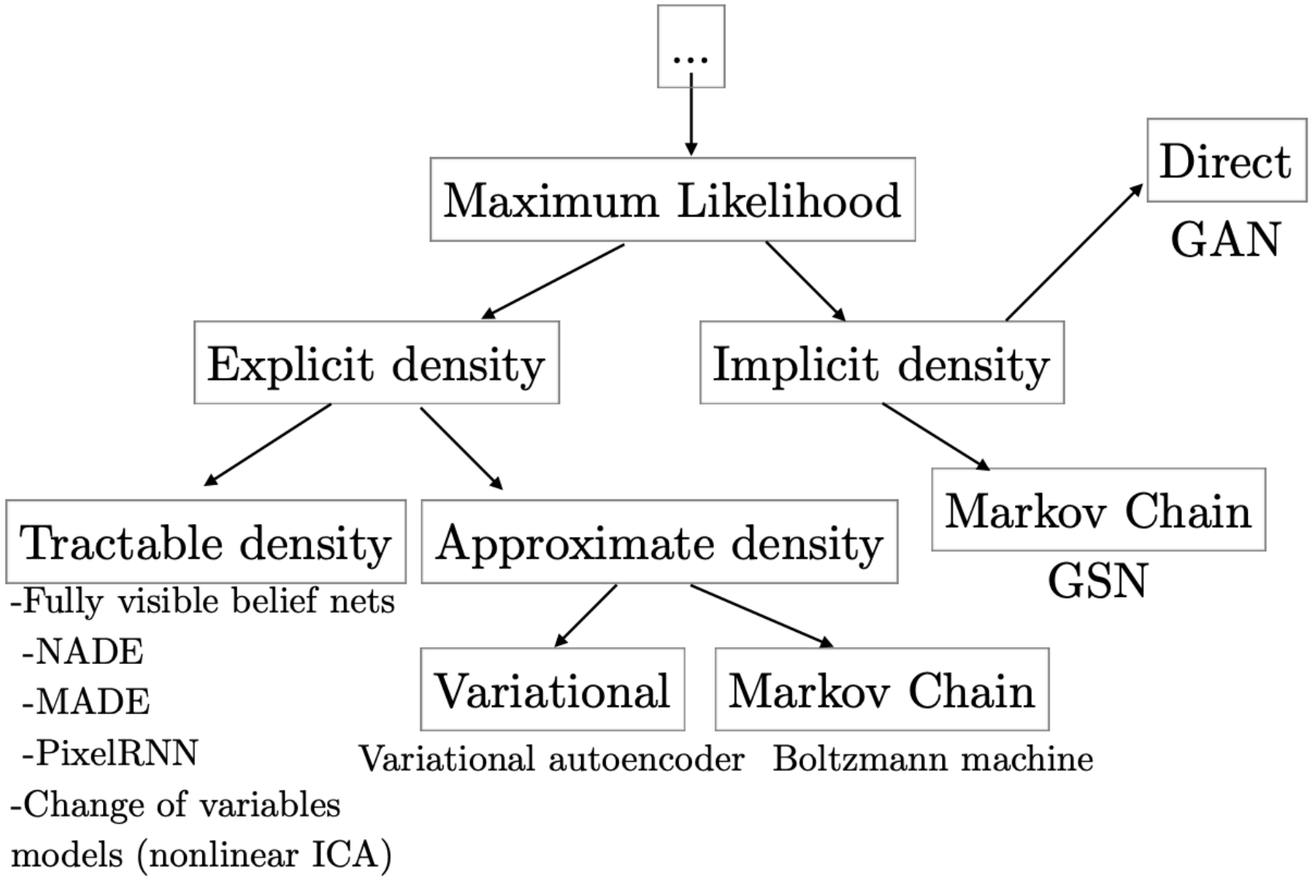
Why are probabilistic graphical models generative?

You can sample from them, as you model the full joint probability distribution.

Summary: Probabilistic Graphical Models

- Probabilistic graphical models are different, but still somewhat related to neural nets (which are deterministic)
- Some concepts are carried over to neural nets (e.g., factorization, having latent variables, the term "hidden units")
- Deep learning has a common history, coming from 90% neural nets, but also 10% probabilistic graphical models
- In deep neural networks models you allow maximum flexibility for your learnable parameters, and do not need to design the graph by hand.

A Birds-Eye perspective



Neural Networks as Generative Models

We want to just give the model data and have it figure everything out by itself!

We know already two types of generative models based on deep neural networks. **Question**: which ones?

- Variational Autoencoders
- Decoder-only Transformers

We know already two types of generative models based on deep neural networks:

- Decoder-only Transformers
- Variational Autoencoders

Decoder-only Transformers are Generative Sequence Models

Goal is given a dataset $\{x_i\}_{I<N}$, learn $p(x)$.

Idea of sequence models:

$$p(x) = \prod_{I=0}^{N} p(x_i|x_{<i})$$

Unrolled: $p(x) = p(x_0) \cdot p(x_1|x_0) \cdot p(x_2|x_0, x_1) \ldots$

Variational Autoencoder Revisited

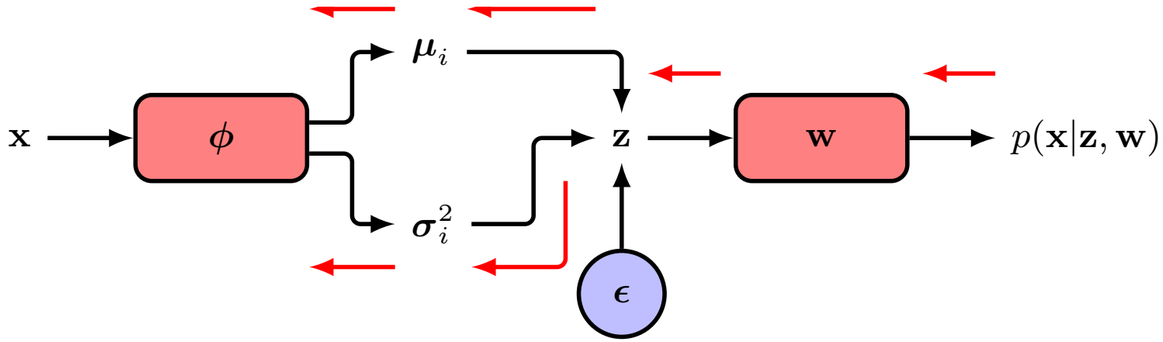
Reparametrization trick

$$z = \mu + \sigma * \epsilon$$

where μ and σ are predicted, but $\epsilon \sim \mathcal{N}(0, 1)$

Question 1: Why do you need the reparametrization trick?

Question 2: How can you sample from the model?



Maximize Variational Lower-Bound

Consider a VAE with a probabilistic encoder $q_\phi(z|x)$ and decoder $p_\theta(x|z)$.

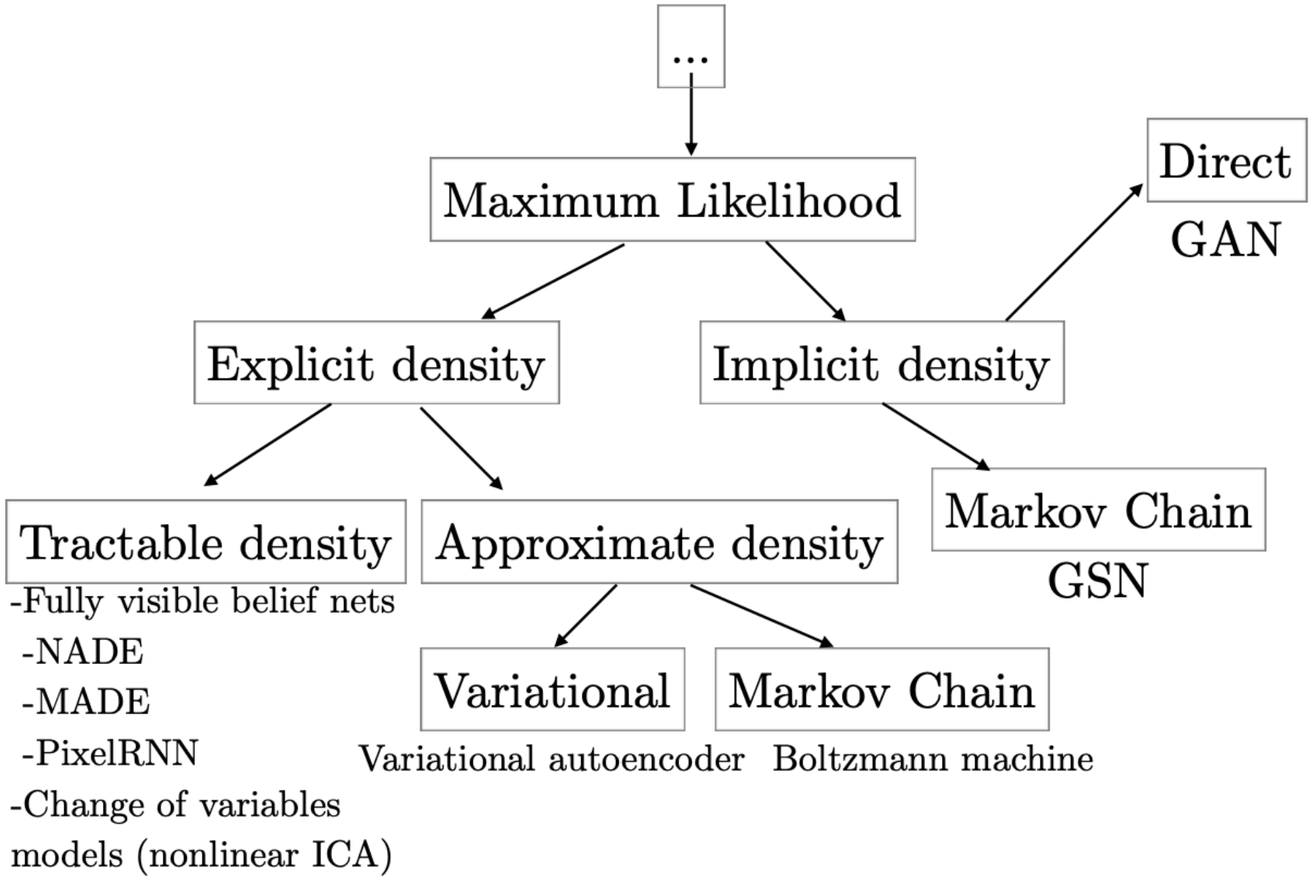
Training objective:

$$\log p_\theta(x) \geq \mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x_i|z)] - D_{\mathrm{KL}}(q_\phi(z|x)||p(z))$$

Intuition: Intuition: reconstruct x well (first term) while keeping the approximate posterior close to a simple prior (second term).

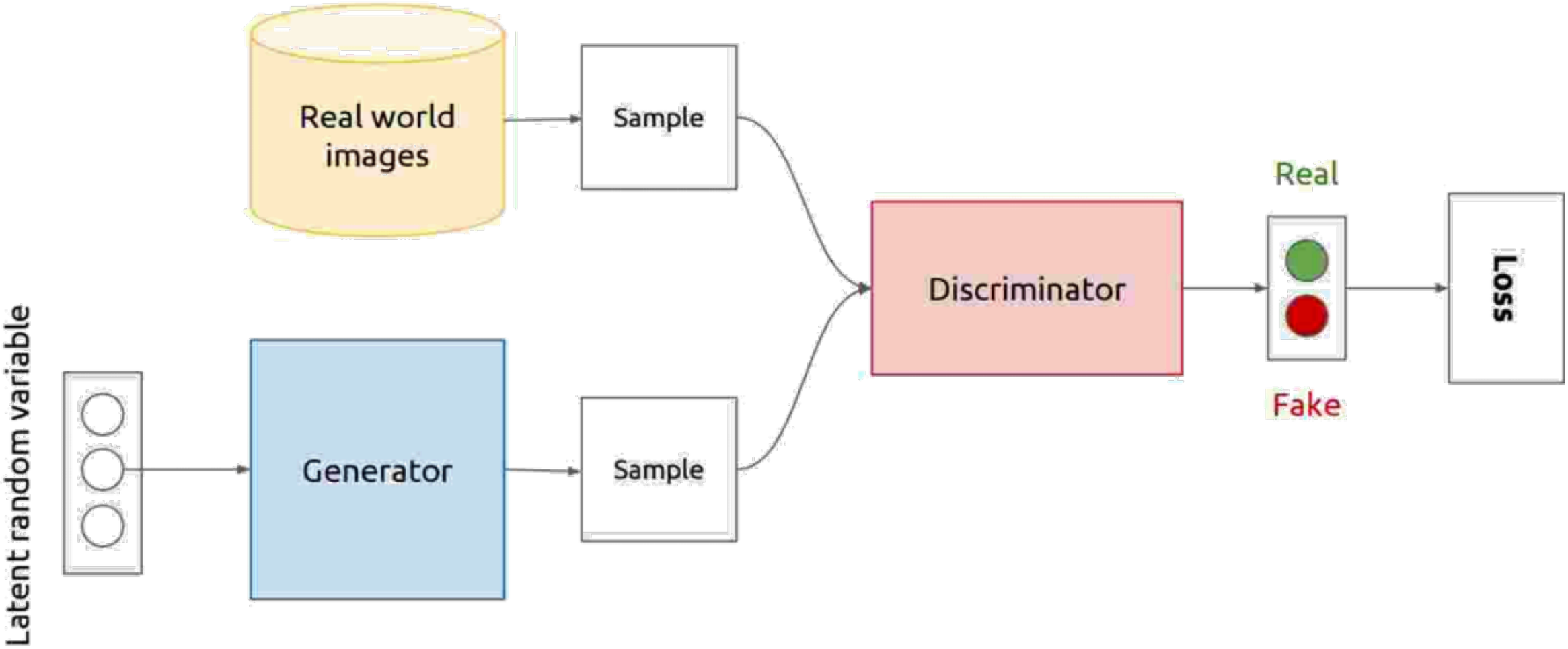
Question: Why is the variational autoencoder a generative model, but other autoencoder variants, like the denoising autoencoder, are not?

Back to the Birds-Eye perspective



Generative Adversarial Networks

Key idea of Generative Adversarial Networks (GANs)



The GAN Training Algorithm

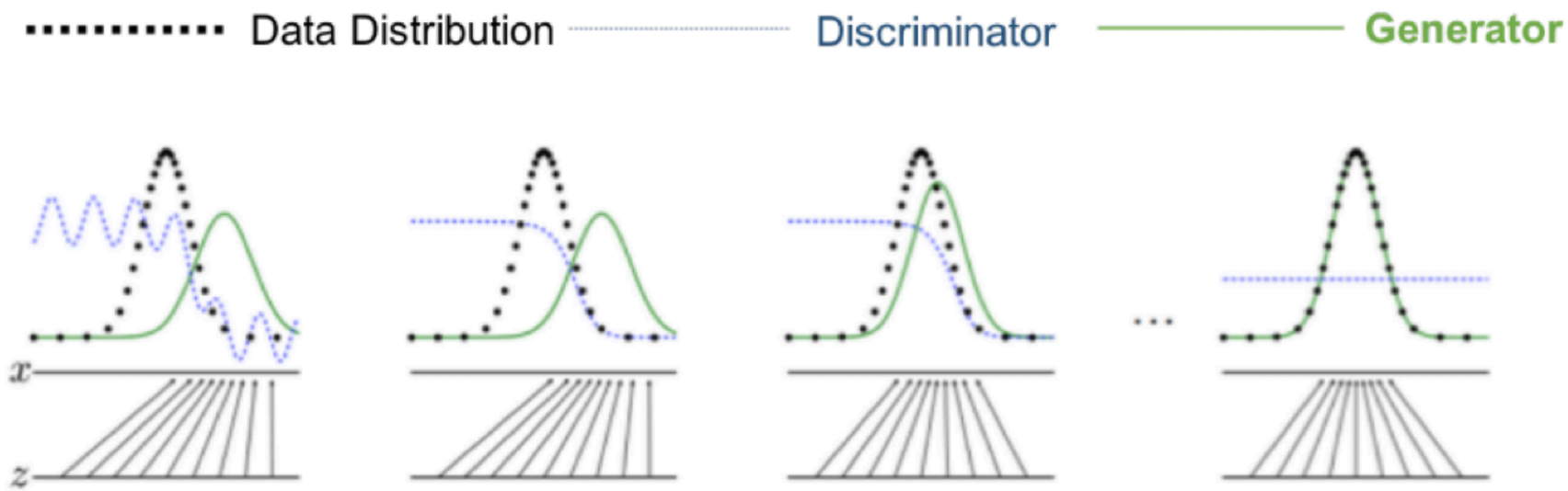
1. Sample real data points x
2. Sample noise z
3. Apply generator G_θ to transform noise $G_\theta(z)$
4. Apply discriminator D_ϕ to distinguish between real x and fake $G_\theta(z)$.
5. Optimize a min max game.

$$\min_\theta \max_\phi (G_\theta, D_\phi) = \mathbb{E}_{x \sim p_{\text{data}}}[\log D_\phi(X)] + \mathbb{E}_{z \sim p(z)}[\log(1 - D_\phi(G_\theta(z))]$$

Which is generated by a GAN?

Caveats: Min-max optimization is a little unstable

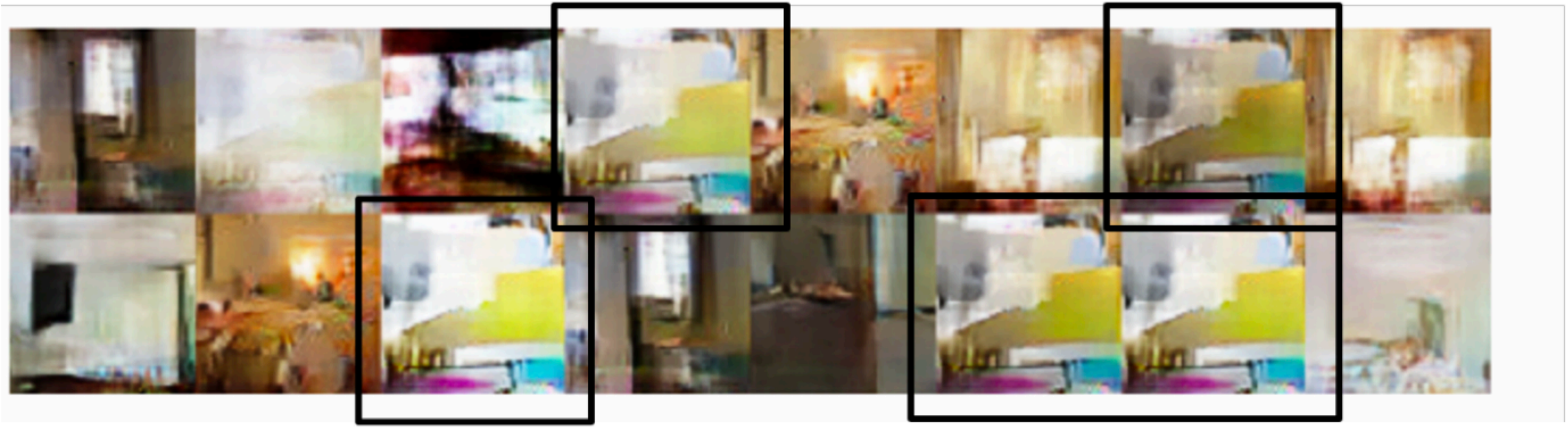
For example: If discriminator becomes too good early on, there is no way the generator can catch up.



Goodfellow et al., 2014

Caveat 2: Sometimes there is mode collapse

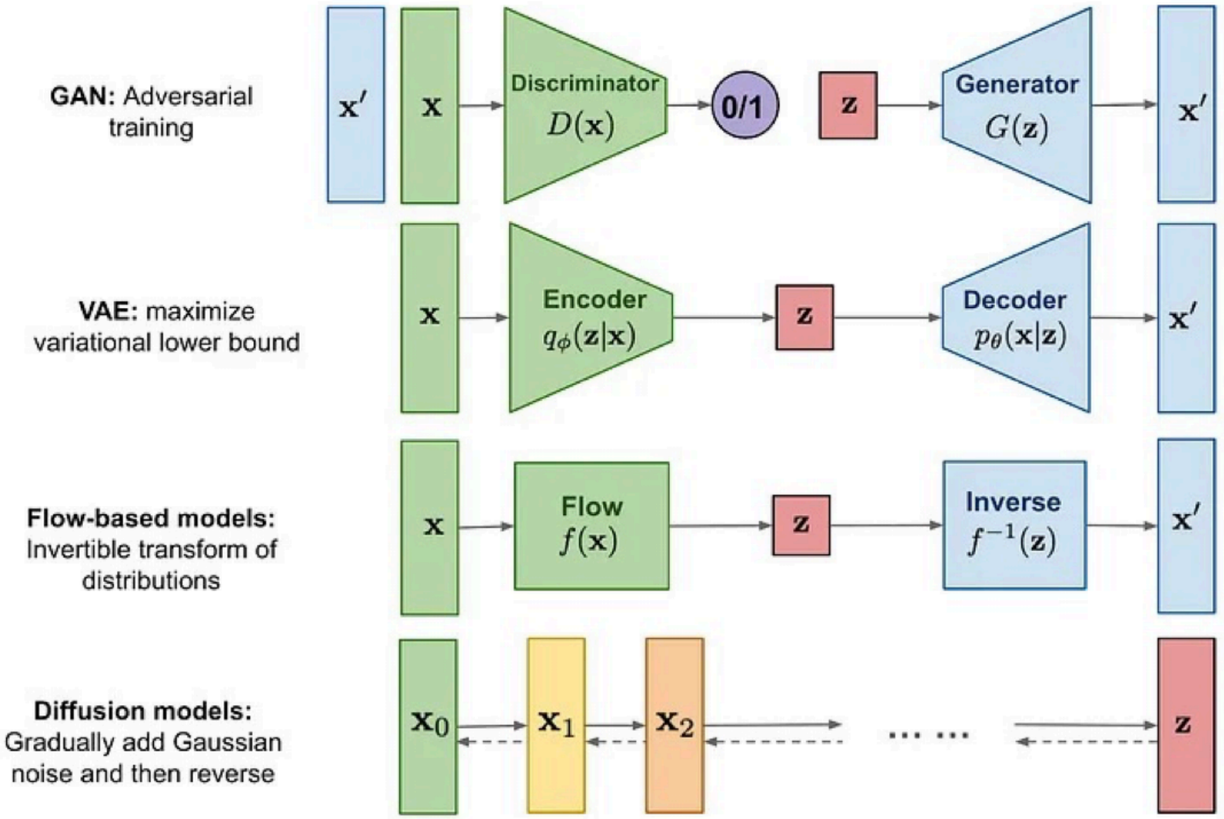
When the generator finds a strategy where it can fool the discriminator well, it may focus too much on this type of images.



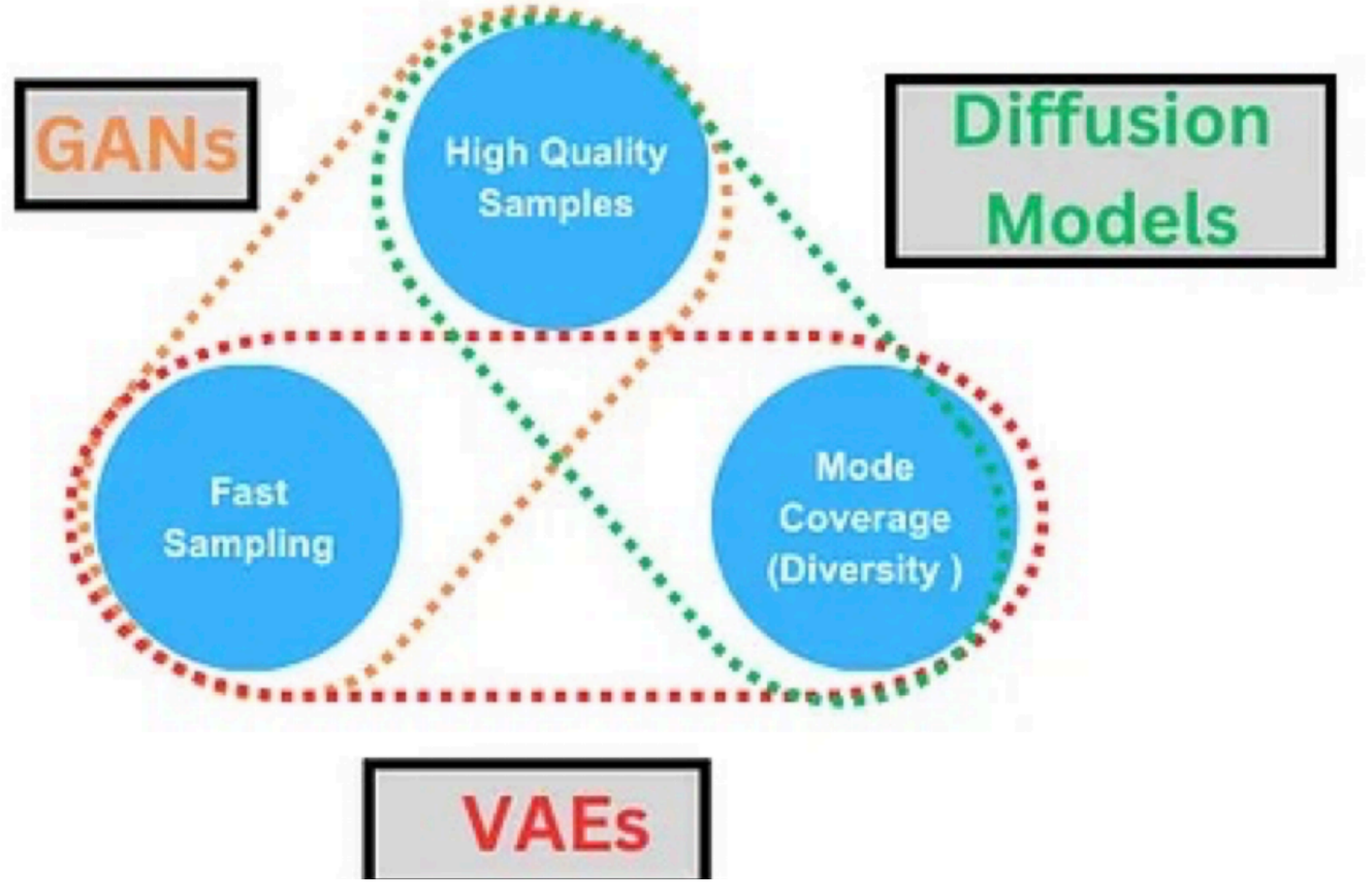
Arjovsky et al., 2017

Current Image generators are based on Diffusion models

More about that in the Computer Vision course SOON Fall

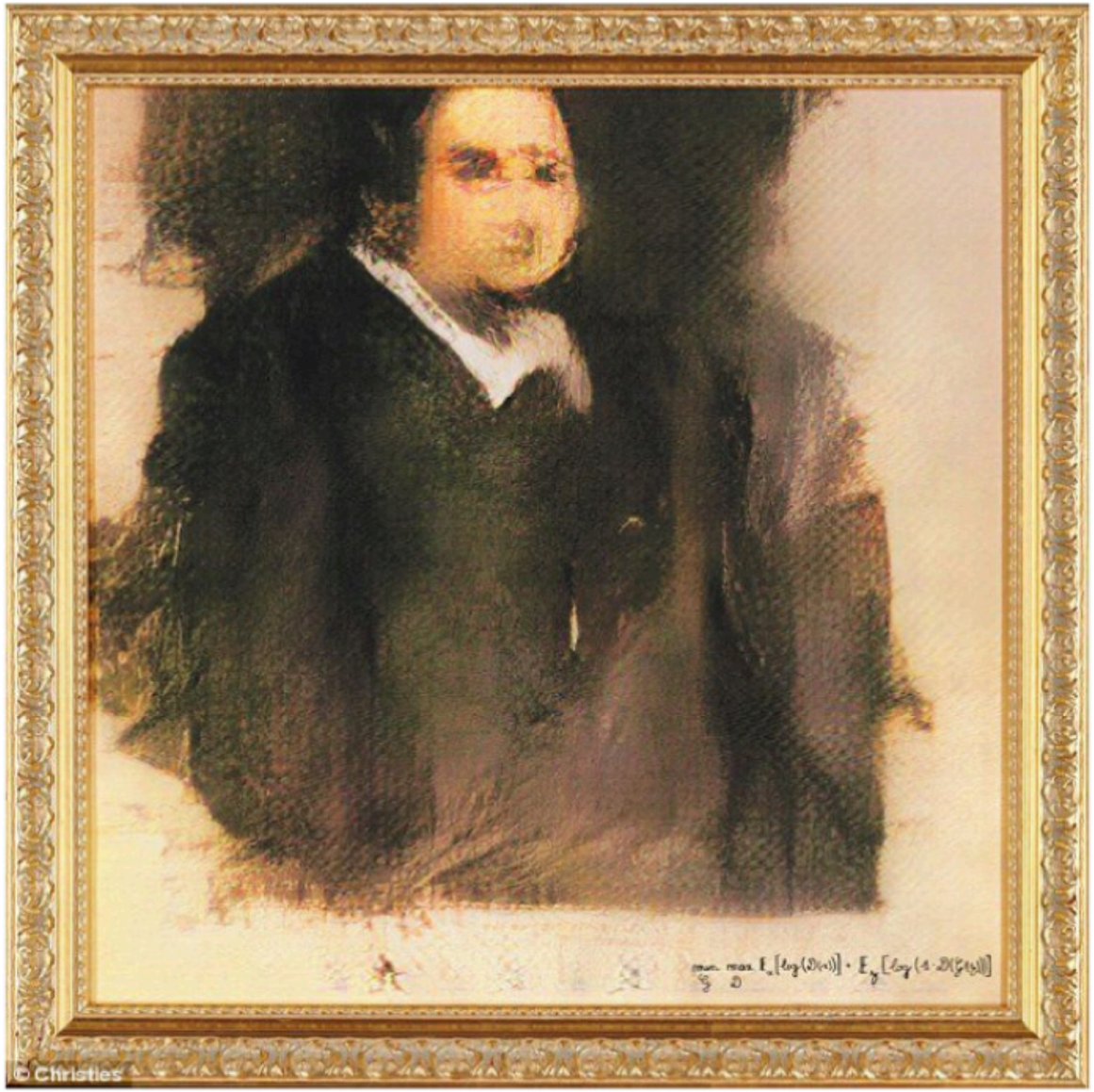


You cannot have it all ;)



Keywords

- Energy-based models
- Probabilistic graphical models
- Bayesian Networks vs. Markov Networks
- Neural networks as generative models
- Generative adversarial networks


© Christies

How's my teaching?

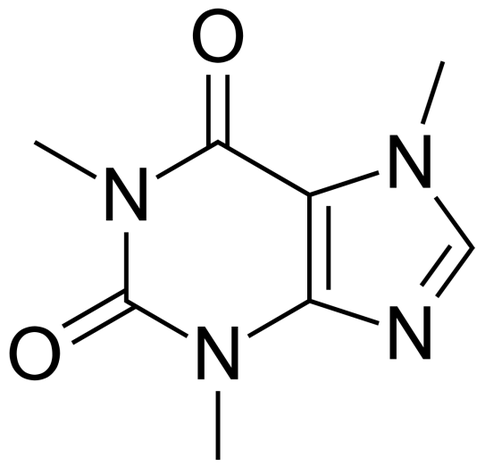
Graph Neural Networks

Lukas Galke Poech

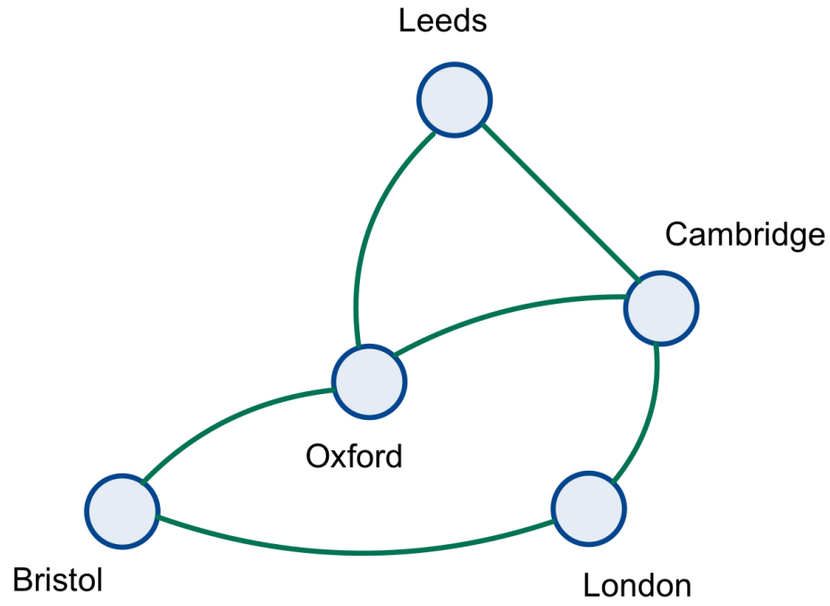
Keywords

- Neural message passing
- Graph neural networks
- Graph convolution
- Jumping knowledge
- Graph attention

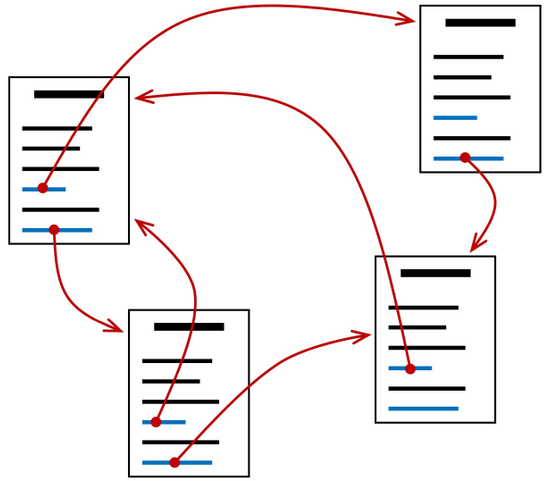
Graphs in the wild



(a)



(b)

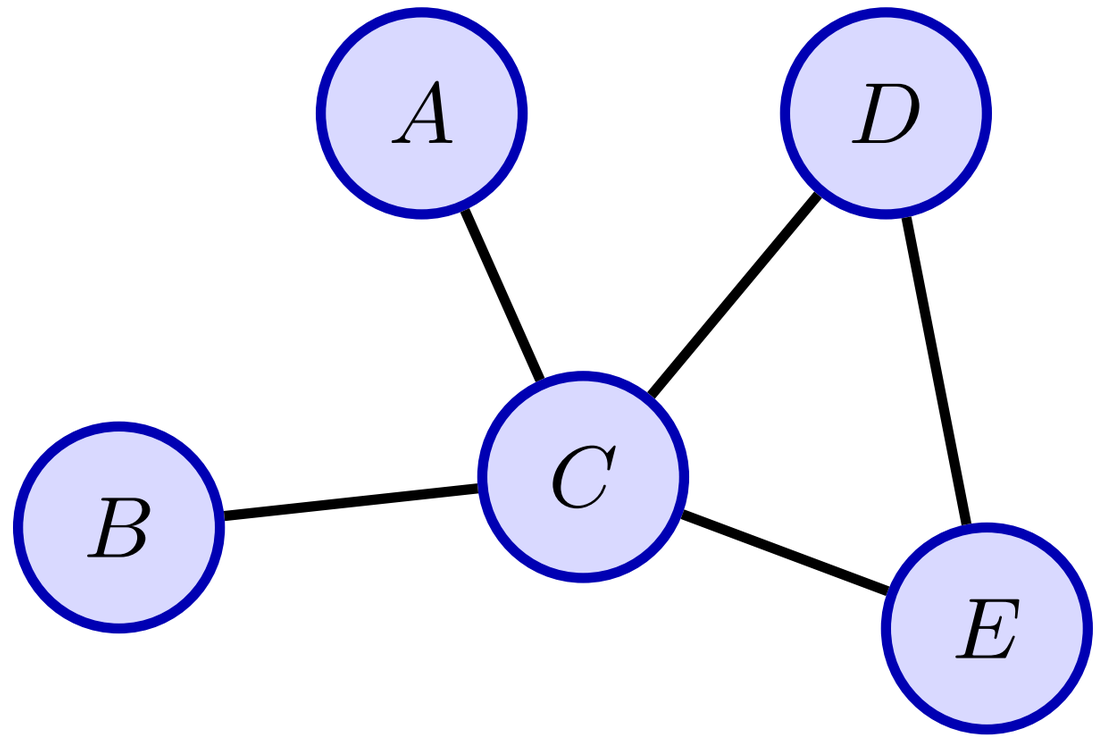


(c)

a) Molecules, b) Traffic networks, c) Citation networks, ...

Graphs

- Sets of nodes V
- and edges $E \subseteq V \times V$
- Node features: $V \to \mathbb{R}^d$



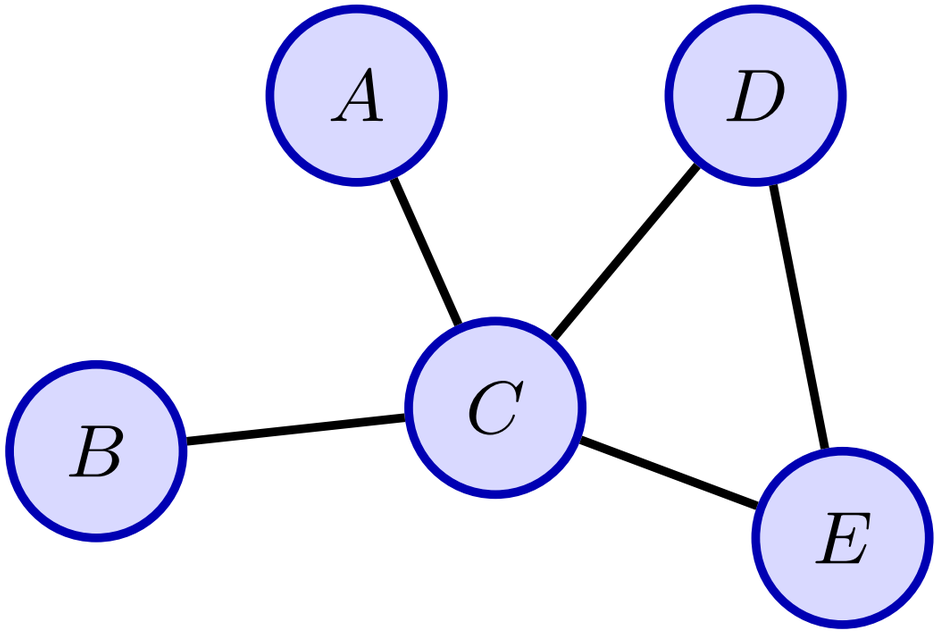
Common tasks on graphs

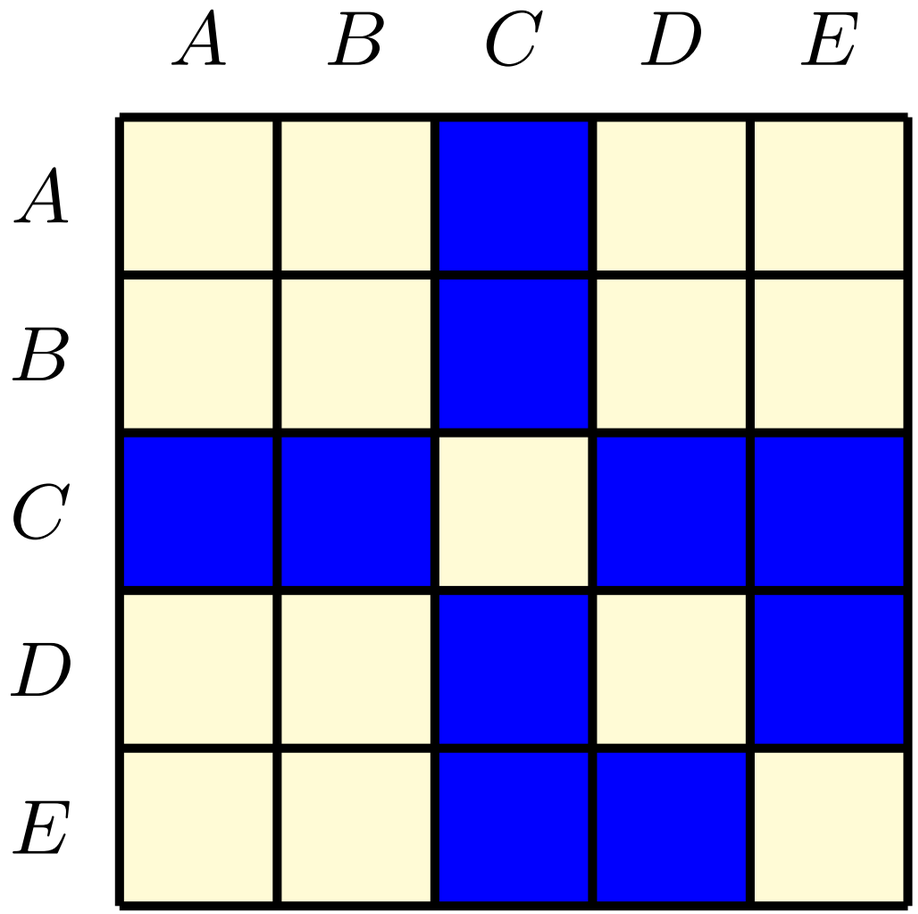
Node classification: Predict the class of each node.

Graph classification: Predict the class of an entire graph.

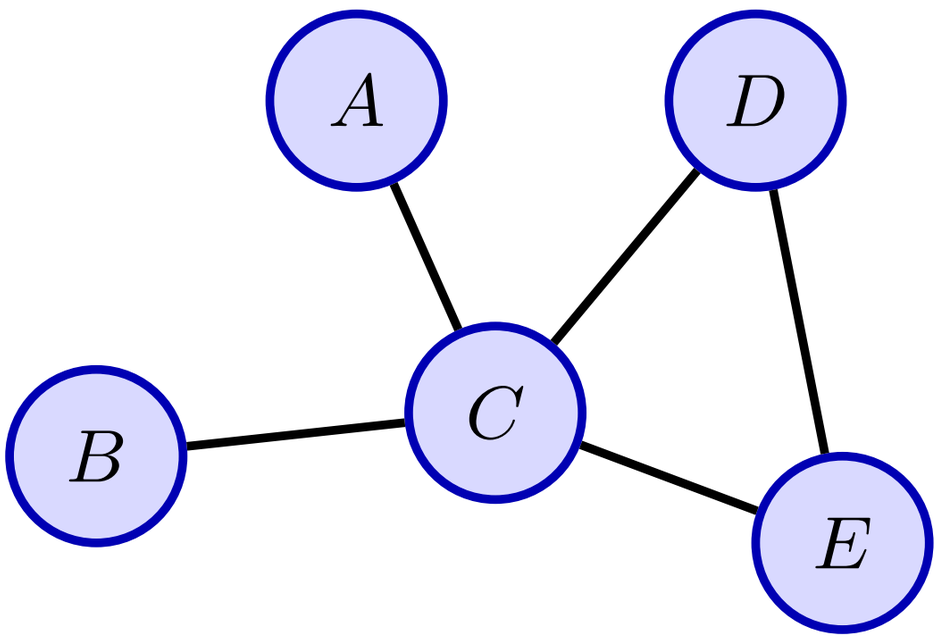
Link prediction: Predict the (future) existence of an edge.

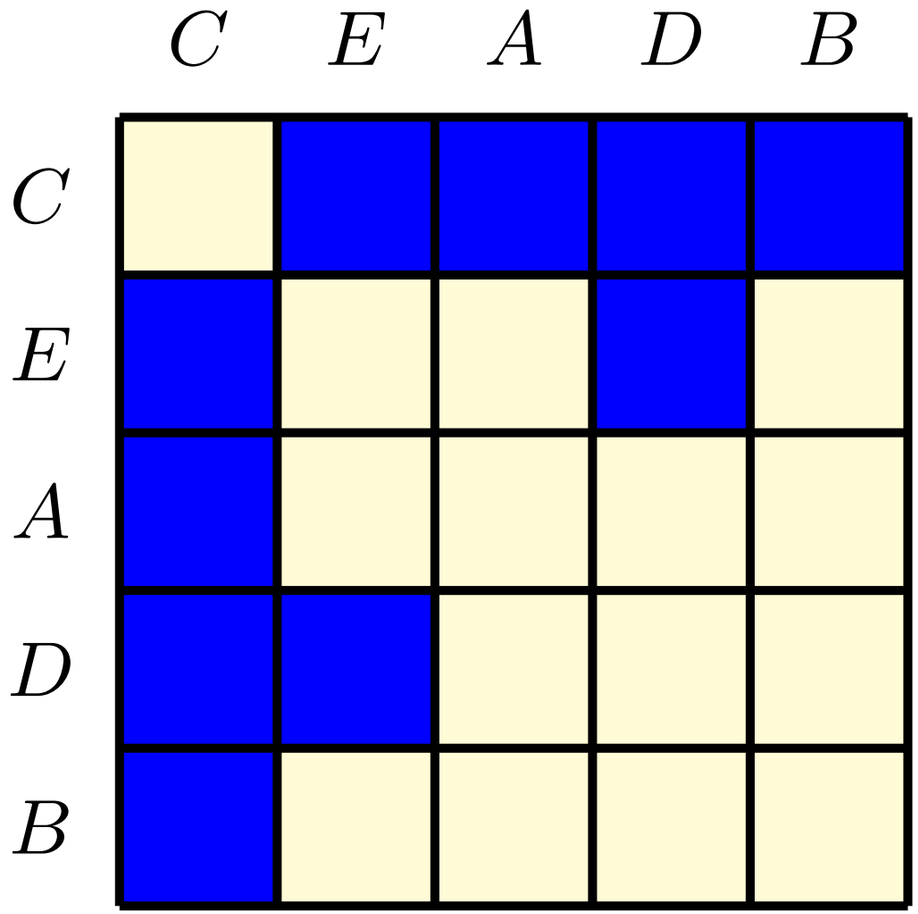
The adjacency matrix





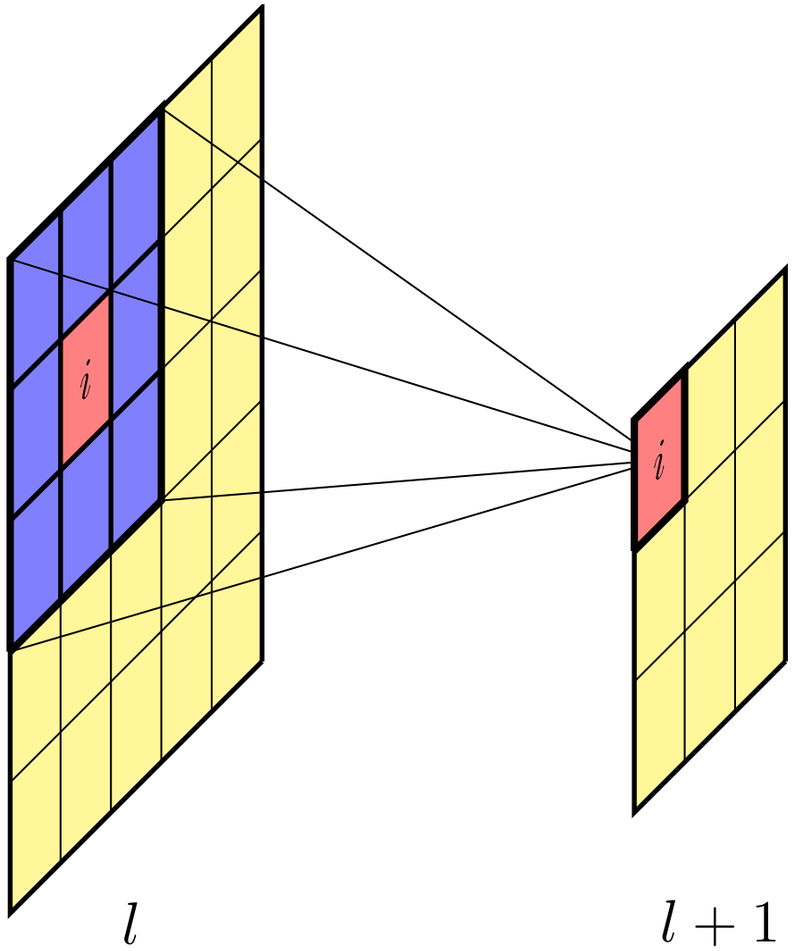
Permutation invariance

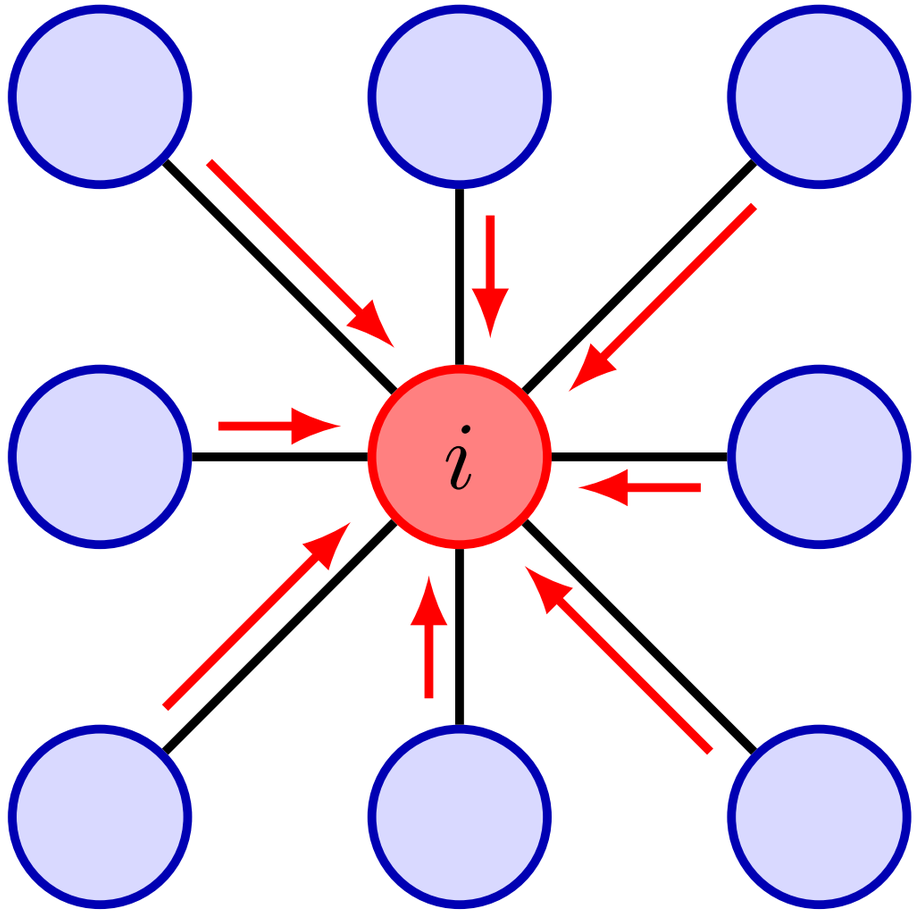




How to process graph-structured data?

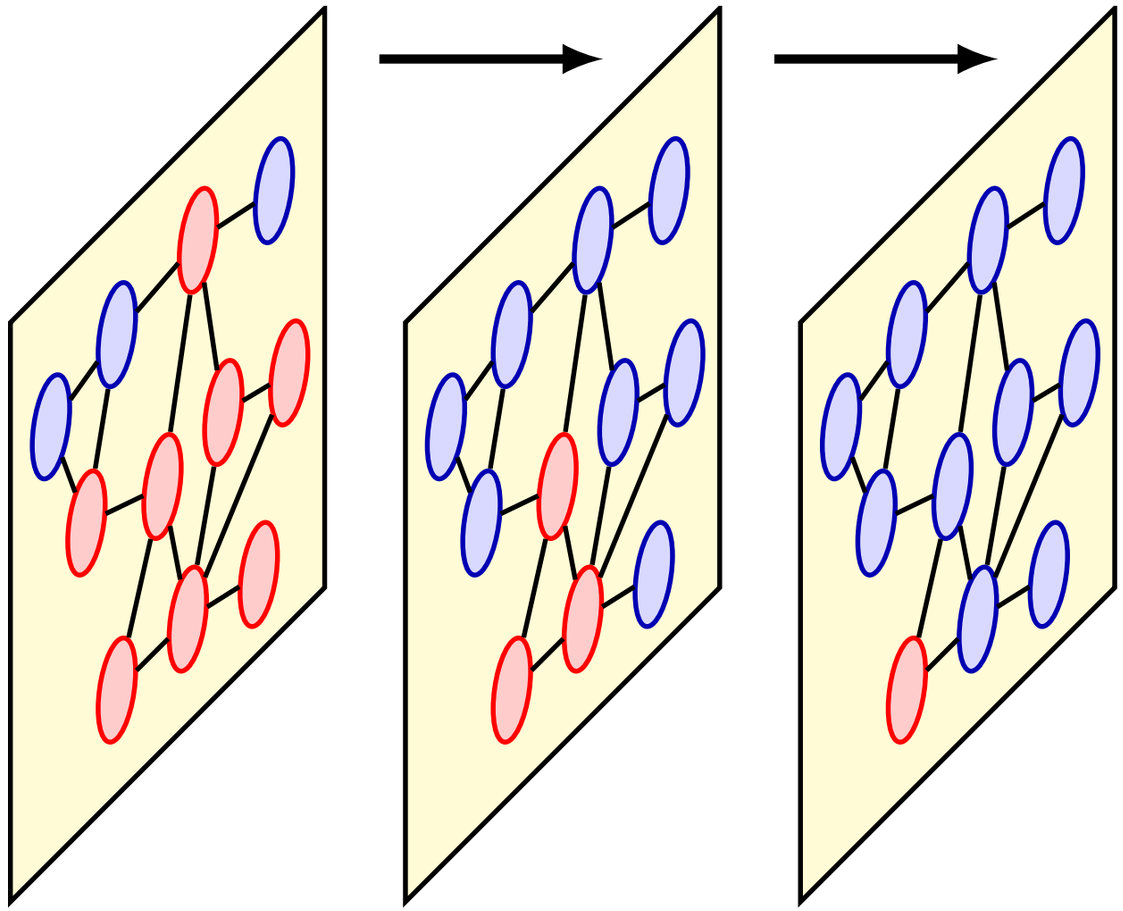
Convolution for Graphs?





Neighborhood Aggregation

... through repeated convolution along the graph structure



Neural message passing

A generalized formulation based on two operations:

AGGREGATE: A method to aggregate messages from all neighbors.

UPDATE: Update each node representation based on aggregated messages.

Neural message passing (cont.)

Set $\mathbf{h_u}^{(0)}$ to the input features of node u.

$$\mathbf{h_u}^{(k+1)} = \mathrm{UPDATE}\left(\mathbf{h_u}^{(k)}, \mathbf{m}_{\mathcal{N}(u)}^{(k)}\right)$$

where $\mathcal{N}(u)$ gives us a set of all neighbor nodes of node u and $\mathbf{m}_{\mathcal{N}(u)}^{(k)}$ are the aggregated messages:

$$\mathbf{m}_{\mathcal{N}(u)}^{(k)} = \mathrm{AGGREGATE}^{(k)}\left(\left\{\mathbf{h_v}^{(k)}, \forall v \in \mathcal{N}(u)\right\}\right)$$

Neural message passing (cont.)

After running K iterations of message passing, we can use the output of the final layer to define the embeddings for each node, i.e.,

$$\mathbf{z_u} = \mathbf{h_u}^{(K)}, \forall u \in V.$$

The basic graph neural network

$$\mathbf{h}_u^{(k)} = \sigma\left(\mathbf{W}_{\text{self}}^{(k)}\mathbf{h}_u^{(k-1)} + \mathbf{W}_{\text{neigh}}^{(k)}\sum_{v\in\mathcal{N}(u)}\mathbf{h}_v^{(k-1)} + \mathbf{b}^{(k)}\right)$$

The basic graph neural network

$$\text{UPDATE}\left(\mathbf{h_u}^{(k)}, \mathbf{m}_{\mathcal{N}(u)}^{(k)}\right) = \mathbf{W}_{\text{self}}^{(k)}\mathbf{h}_u^{(k-1)} + \mathbf{W}_{\text{neigh}}^{(k)}\mathbf{m}_{\mathcal{N}(u)}^{(k)} + \mathbf{b}^{(k)}$$

$$\mathbf{m}_{\mathcal{N}(u)}^{(k)} = \text{AGGREGATE}^{(k)}\left(\left\{\mathbf{h_v}^{(k)}, \forall v \in \mathcal{N}(u)\right\}\right) = \sum_{v \in \mathcal{N}(u)} \mathbf{h}_v^{(k-1)}$$

Neighborhood normalization

Graph *convolutional* networks use a symmetric normalization:

$$\mathbf{m}_{\mathcal{N}(u)} = \sum_{v \in \mathcal{N}(u)} \frac{\mathbf{h_v}}{\sqrt{|\mathcal{N}(u)| \cdot |\mathcal{N}(v)|}}$$

Alternatively, use the number of in-going edges for normalization:

$$\mathbf{m}_{\mathcal{N}(u)} = \frac{\sum_{v \in \mathcal{N}(u)} \mathbf{h_v}}{|\mathcal{N}(u)|}$$

Graph convolution (Kipf & Welling, 2017)

A specific, very popular form of graph neural network. Here in matrix notation:

- Nodes features $\mathbf{X} = \mathbf{H}^{(0)}$
- Adjacency matrix $\mathbf{A}$

$$\mathbf{H}^{(\mathbf{k}+\mathbf{1})} = \sigma\left(\hat{\mathbf{A}}\mathbf{H}^{(\mathbf{k})}\mathbf{W}^{(\mathbf{k})} + b\right)$$

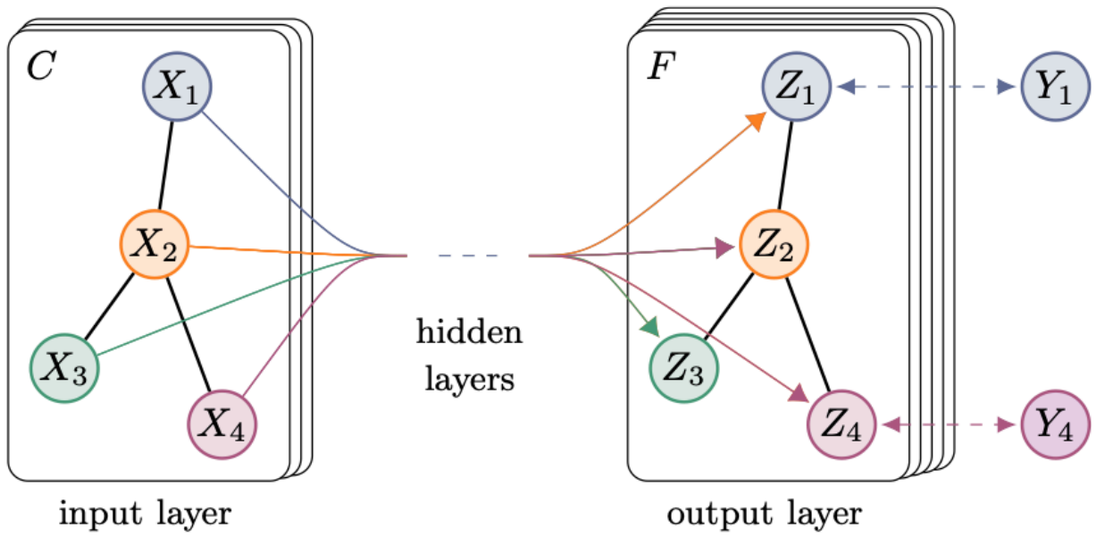
where $\hat{\mathbf{A}}$ is a normalized version of the adjacency matrix, with *self-loops* added.

Task: Node Classification

Task: Classify each node, given node features $\mathbf{X}$ and adjacency matrix $\mathbf{A}$.

Key assumption: Nodes that are connected, are more likely to belong to the same class.

$$\mathbf{Z} = f(\mathbf{X}, \mathbf{A}) = \operatorname{softmax}\left(\hat{\mathbf{A}}\,\operatorname{ReLU}\left(\hat{\mathbf{A}}\mathbf{X}\mathbf{W}^{(0)}\right)\mathbf{W}^{(1)}\right)$$



Task: Graph Classification

For graph-level tasks, we need to aggregate the entire graph.

Key strategy: **Pooling** after GNN layers, often called **READOUT** operation.

$$\mathrm{READOUT}(\mathbf{h}^{(k)}) = \sum_{u \in V} \mathbf{h_u}^{(k)}$$

where $h^{(k)}$ are the node representations after applying k GNN layers.

Then, apply a final linear layer or MLP on the **READOUT** result.

$$z = \mathrm{MLP}\left(\mathrm{READOUT}(\mathbf{h}^{(k)})\right)$$

Keywords

- **Neural message passing**
- **Graph neural networks**
- **Graph convolution**
- Jumping knowledge
- Graph attention

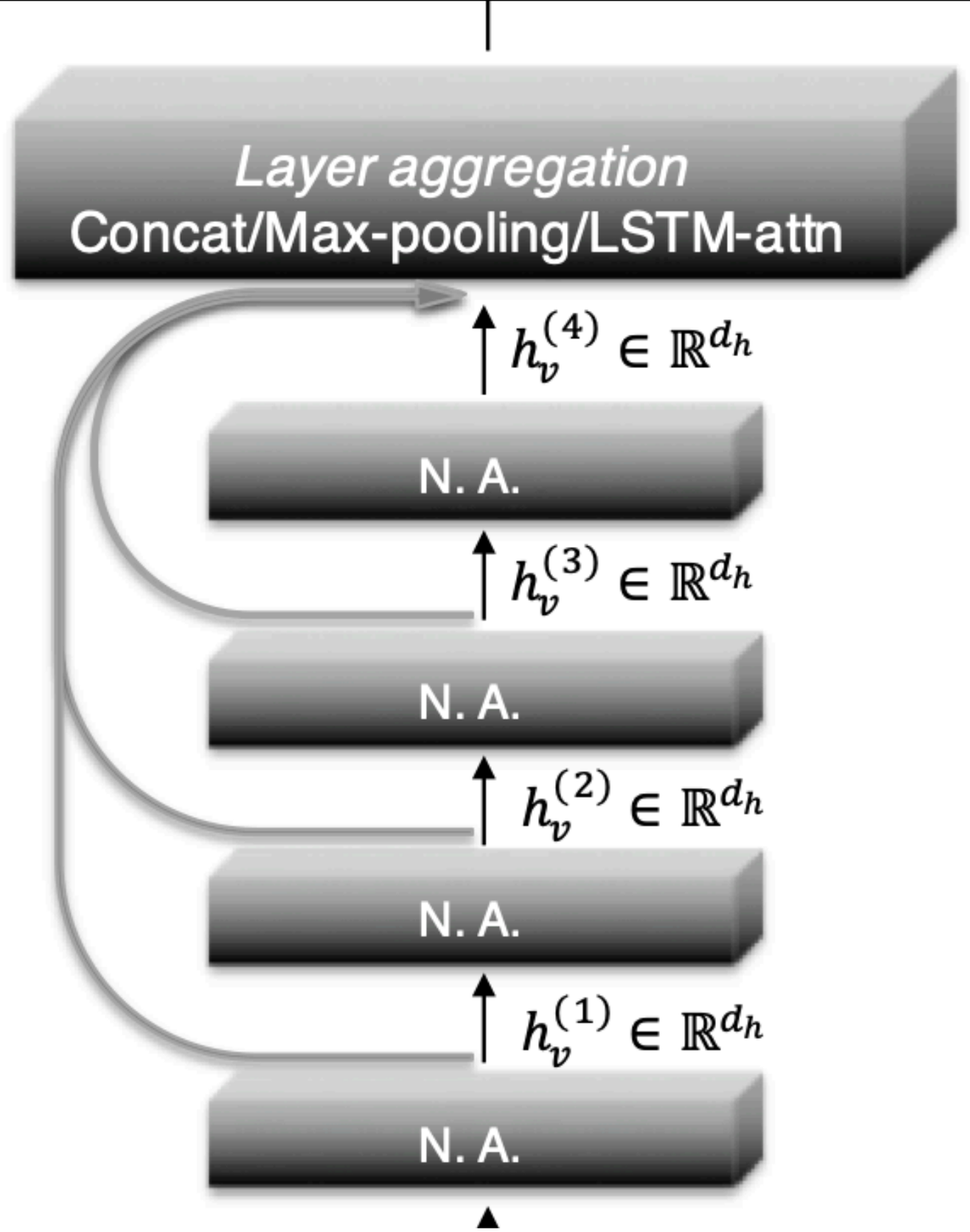
Jumping knowledge

Oversmoothing is a common problem when stacking too many layers, due to many consecutive ~averaging operations.

What helps? Insert a connection from each layer's output to the final layer, called **jumping knowledge**:

$$\mathbf{z} = \mathrm{MLP}\left(\mathbf{h}^{(1)} \oplus \mathbf{h}^{(2)} \oplus \cdots \oplus \mathbf{h}^{(k)}\right)$$

where $\cdot \oplus \cdot$ is concatenation.



Graph attention (GAT)

In basic graph neural networks, all edges are considered to have equal weight.

But some edges may be more important than others.

Graph attention: learn to predict the weights of the edges with an attention mechanism:

$$\mathbf{m}_{\mathcal{N}(u)} = \sum_{v \in \mathcal{N}(u)} \alpha_{u,v} \mathbf{h_u}$$

where $\alpha_{u,v} = \mathbf{softmax}_{\mathcal{N}(u)} \left(\mathbf{a}_T \left[\mathbf{W h_u} \oplus \mathbf{W h_v} \right] \right)$ with trainable vector $\mathbf{a}$.

Summary

- **Graph neural networks** enable us to make use of neural networks for tasks on graph-structured data.
- **Neural message passing** is the general framework to specify graph neural networks through **UPDATE** and **AGGREGATE** operations.
- **Graph convolution** performs convolution along graph structure
- **Jumping knowledge**, i.e., shortcut connections to the final layer, helps to avoid oversmoothing (similar to residual connections).
- **Graph attention** predicts edge weights from participating nodes.
- Graph-level tasks require a global **pooling** mechanism.

Keywords

- Neural message passing
- Graph neural networks
- Graph convolution
- Jumping knowledge
- Graph attention

Further reading:

Graph Representation Learning book by Will Hamilton

How's my teaching

admonymous.co/lukasgalke

Interpretability

AI506: Advanced Machine Learning

Spring 2026

Lukas Galke Poech

Keywords

- Interpretability and explanations
- Human vs. functional groundedness
- Intrinsic vs. post-hoc interpretability
- Causal interpretability
- Mechanistic interpretability

Why is interpretability important?

Ethics: Aligned with common moral values?

AI Safety: Does the model perform within expectations?

Accountability: When does the model fail in production? (cf. EU AI Act)

Scientific Understanding: Generate hypothesis and knowledge

What is interpretability?

- *Interpretability* is the “ability to explain or to present in understandable terms to a human” (Doshi-Velez & Kim, 2017)
- Interpretability is the degree to which a human can understand the cause of a decision. (Miller, 2019)
- Effective explanations must be selective [ideally only 1-2 causes] (Miller, 2019)

What makes a good explanation?

- **Application-grounded**: Explanations are evaluated in the environment where the model is deployed (ideal but usually not feasible).
- **Human-grounded**: Is the explanation useful to humans?
- **Functionally-grounded**: Does the explanation actually have something to do with what happens in the model?

Advanced machine learning finds stuff and decides things

- But **how** does it finds things and **why** does it make certain decisions?
- Deep nets are extremely powerful, but very poorly interpretable.
- What can we do to make them more interpretable?

Intrinsic vs. post-hoc interpretability

Intrinsic = interpretability integrated in the model

Post-hoc = interpret the model after it has been trained

Feature attribution

The goal is to find explanations that tell us what input features matter for the model's output.

Let's look at a few methods!

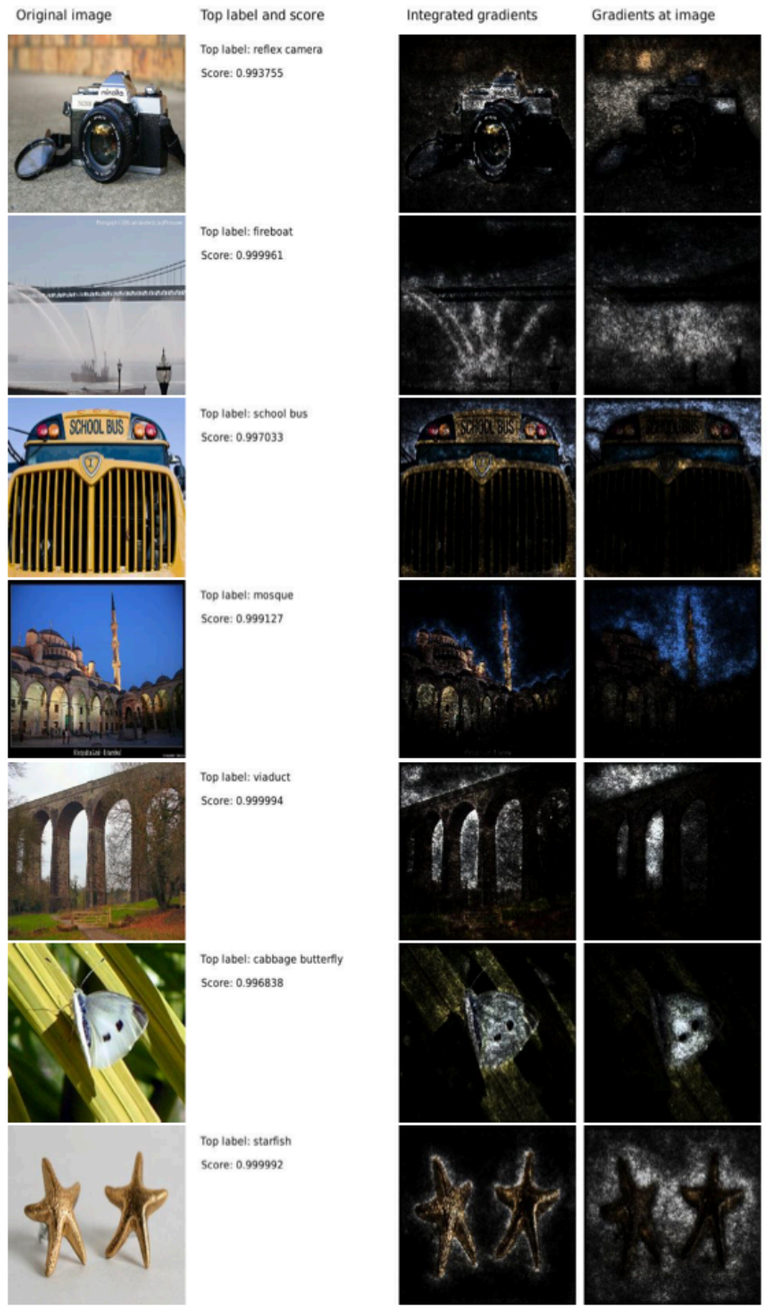
Common post-hoc methods

LIME: Train a linear regression (interpretable!) to predict the output of the studied model.

SHAP: Test all possible combinations of inputs, and observe when the model output changes.

Axiomatic post hoc methods

Integrated gradients:
Calculate the derivative with respect to the input
(=quantify the effect a small change in input would have on the output)



Model-specific approaches

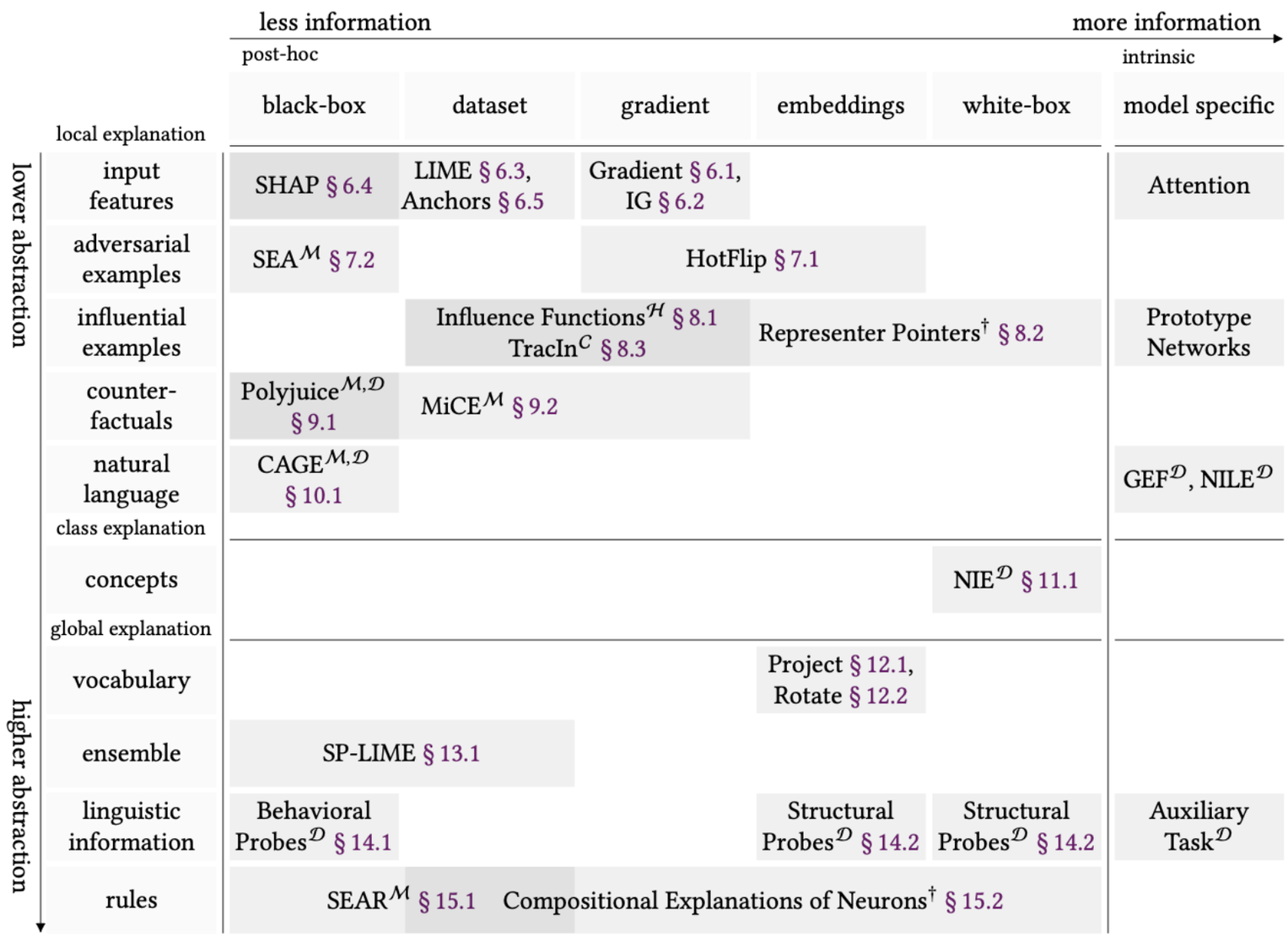
Just look at the attention values and take those as feature attribution.

2019

Sarah Wiegreffe, Yuval Pinter:
Attention is not not Explanation. EMNLP/IJCNLP (1) 2019: 11-20

Sarthak Jain, Byron C. Wallace:
Attention is not Explanation. NAACL-HLT (1) 2019: 3543-3556

There are many more



Keywords

- **Interpretability and Explanations**
- **Human vs. functional groundedness**
- **Intrinsic vs. post-hoc interpretability**
- Probes
- Causal Interpretability
- Mechanistic Interpretability

Linear Probes

A (linear) probe is a (linear) classifier that takes the activations of another models as input and predicts a property of interest.

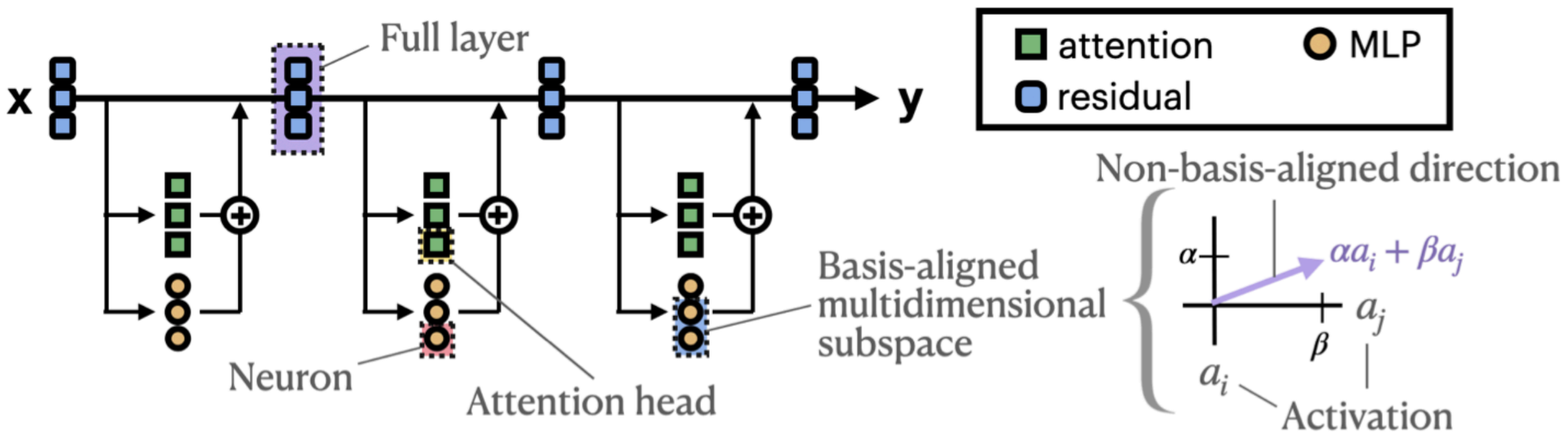
This tells us whether the property of interest is linearly separable in the representation.

In practice: Keep some held-out data, to evaluate how well the probe generalizes.

Causal Interpretability

- Based on Causal Mediation Analysis (Pearl): Does X cause Y through some intermediate variable Z?
  Think of the z as an intermediate hidden layer representation of a neural net

But what is the right mediator?



Open question!

Mechanistic Interpretability

Reverse engineering neural networks

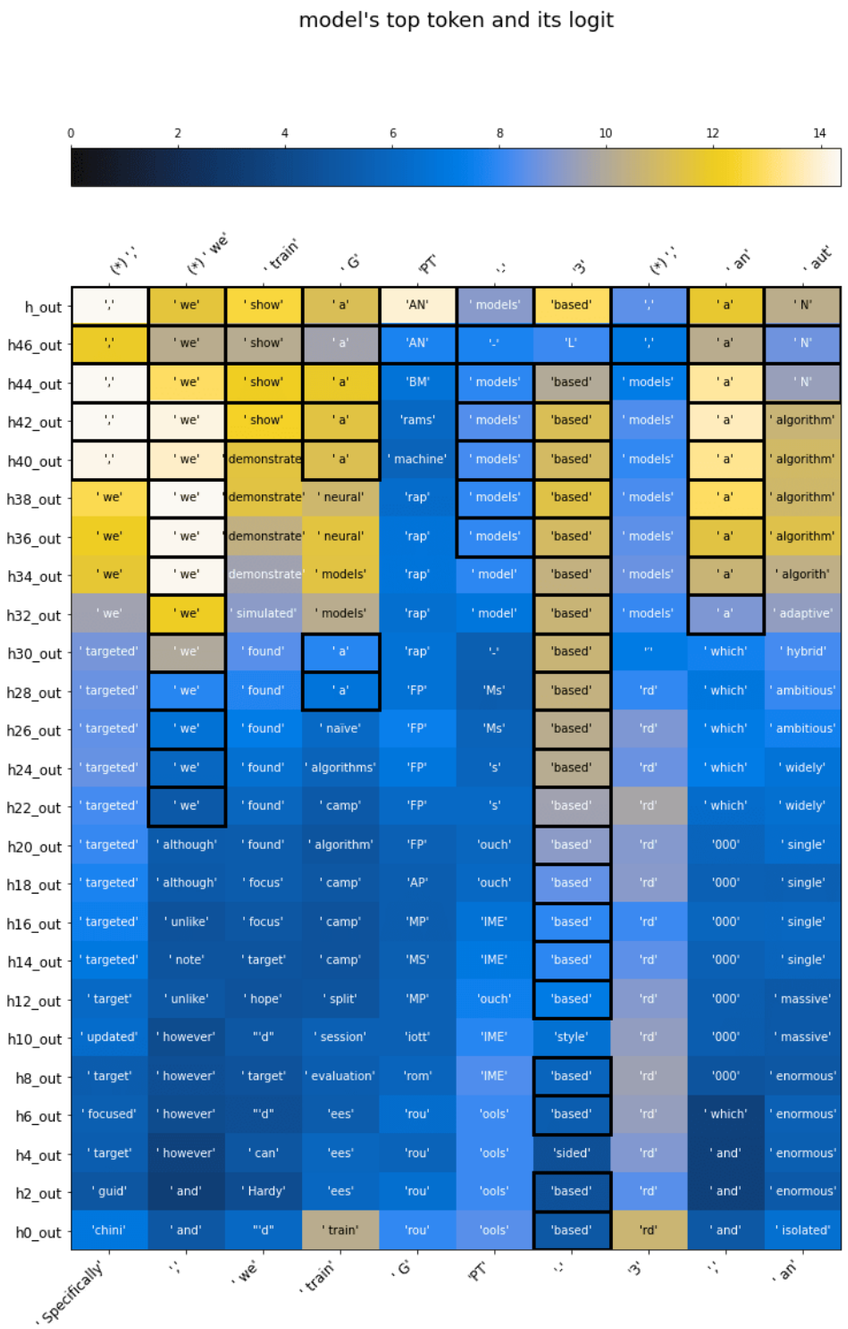
Identifying features and circuits

Causal analysis ("mechanisms")

Mech Interp goes beyond input features and seeks to figure out what mechanisms in the model leads to the output.

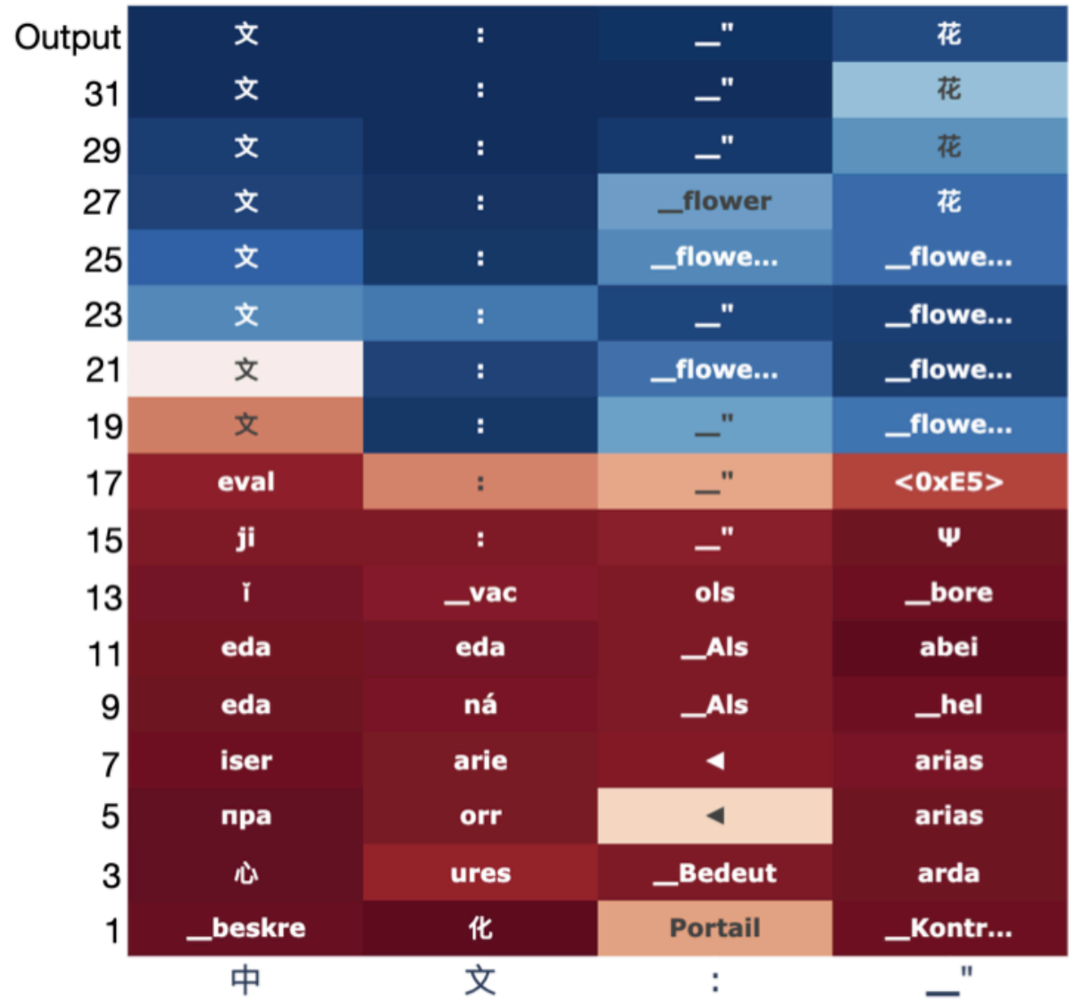
The logit lens

Prematurely apply the final language modeling head to intermediate representations of a transformer



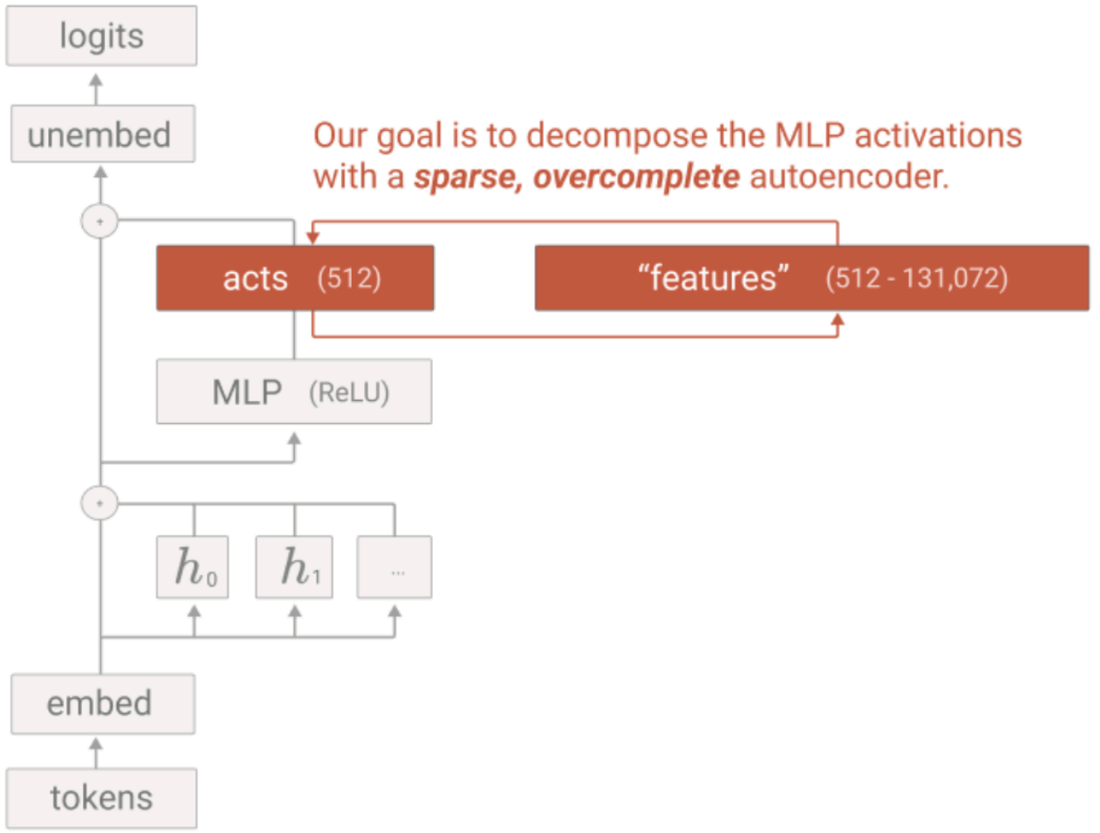
Do Llamas work in English?

Multilingual language models "think" in a concept space that happens to be closest to English (dominant training language)

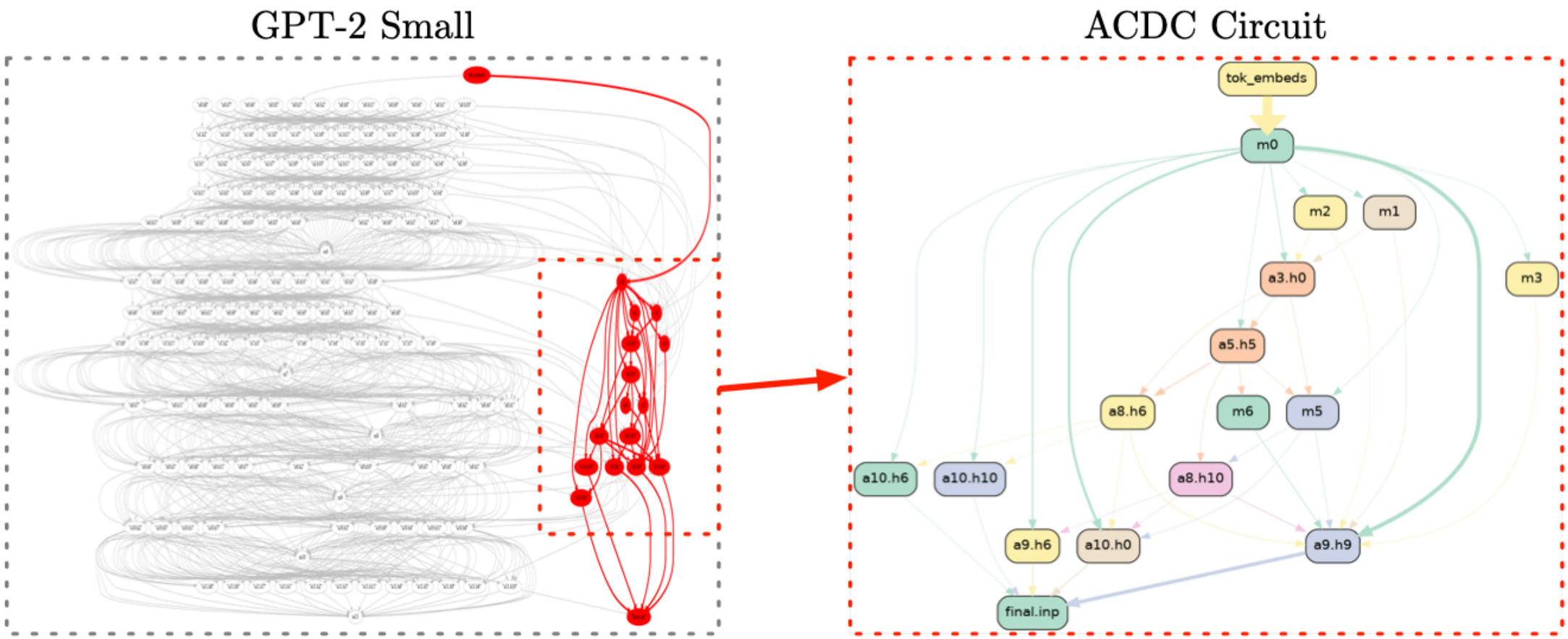


Sparse Autoencoders

Train a sparse autoencoder on the activations of another neural network to learn meaningful features.

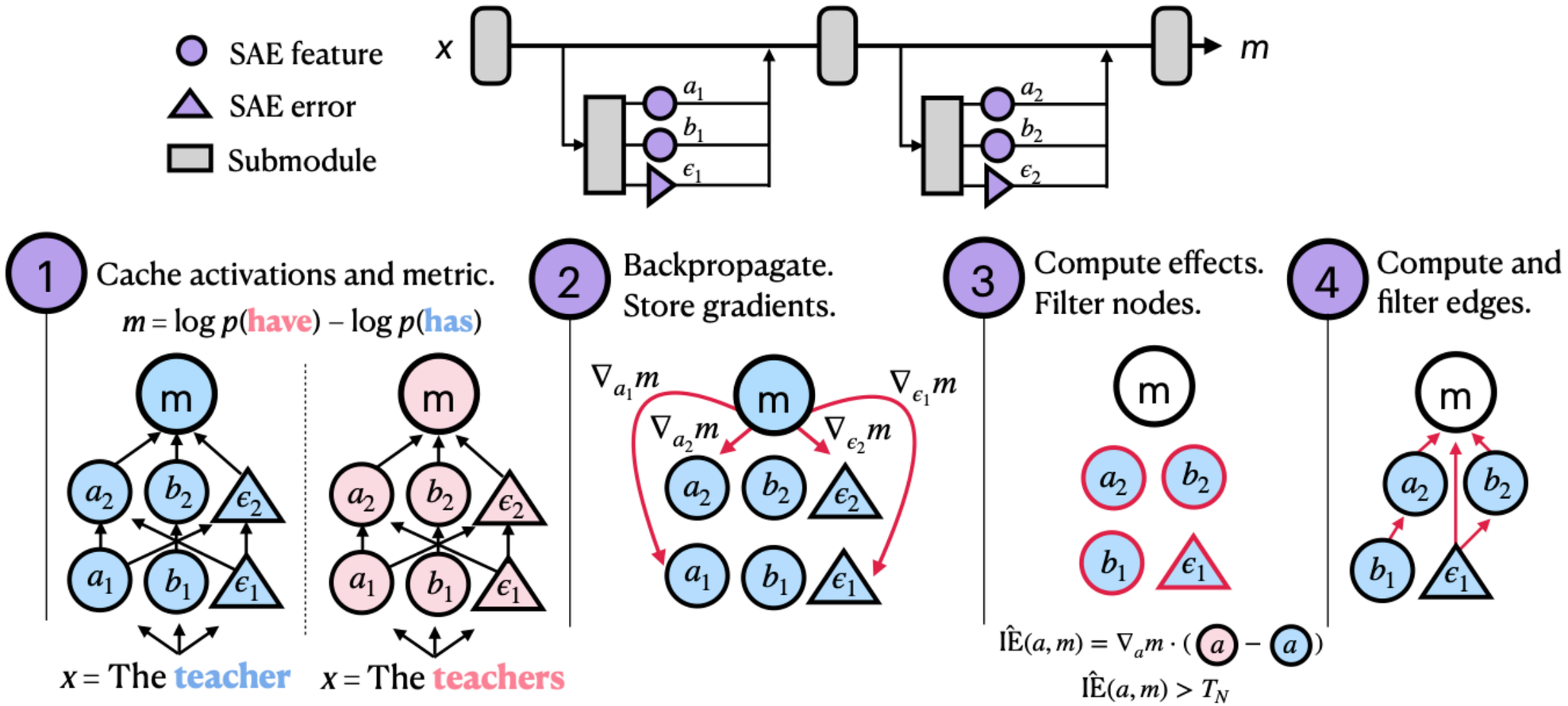


Automated Circuit Discovery



Sparse feature circuits

SAEs + Circuit Discovery Pipeline



Keywords

- **Interpretability and Explanations**
- **Human vs. functional groundedness**
- **Intrinsic vs. post-hoc interpretability**
- **Causal Interpretability**
- **Mechanistic Interpretability**

How's my teaching?

admonymous.co/lukasgalke